AlgoMaster Exclusives

344 questions not in Set 1 or NeetCode · Priority-Grouped · 2×1 Landscape

Python Only · Priority-Grouped · Difficulty-First · 2×1 Landscape

---

344 questions · 9 rounds · Interview Approach
High Easy → High Med → High Hard → Mid Easy → Mid Med → Mid Hard → Low Easy → Low Med → Low Hard

· Contents

Contents

★ = LeetCode Premium (Premium 98 list)

- [ ] #958 Check Completeness of a Binary Tree ↗

Tree Traversal - Pre Order (3) ↗

- [ ] #106 Construct Binary Tree from Inorder and Postorder Traversal ↗
- [ ] #687 Longest Univalue Path ↗
- [ ] #1026 Maximum Difference Between Node and Ancestor ↗

Tree Traversal - In Order (1) ↗

- [ ] #1382 Balance a Binary Search Tree ↗

Tree Traversal - Post-Order (6) ↗

- [ ] #114 Flatten Binary Tree to Linked List ↗
- [ ] #129 Sum Root to Leaf Numbers ↗
- [ ] #652 Find Duplicate Subtrees ↗
- [ ] #979 Distribute Coins in Binary Tree ↗
- [ ] #1110 Delete Nodes And Return Forest ↗
- [ ] #2096 Step-By-Step Directions From a Binary Tree Node to Another ↗

Depth First Search (DFS) (4) ↗

- [ ] #690 Employee Importance ↗
- [ ] #797 All Paths From Source to Target ↗
- [ ] #1020 Number of Enclaves ↗
- [ ] #1376 Time Needed to Inform All Employees ↗

Breadth First Search (BFS) (5) ↗

- [ ] #433 Minimum Genetic Mutation ↗
- [ ] #752 Open the Lock ↗
- [ ] #909 Snakes and Ladders ↗
- [ ] #1091 Shortest Path in Binary Matrix ↗
- [ ] #1102 Path With Maximum Minimum Value ↗

Linked List (6) ↗

- [ ] #82 Remove Duplicates from Sorted List II ↗
- [ ] #86 Partition List ↗
- [ ] #237 Delete Node in a Linked List ↗
- [ ] #445 Add Two Numbers II ↗
- [ ] #1669 Merge In Between Linked Lists ↗
- [ ] #2095 Delete the Middle Node of a Linked List ↗

Round 3 | High | H Hard (11) ↗

Strings (1) ↗

- [ ] #843 Guess the Word ↗

Two Pointers (1) ↗

- [ ] #2444 Count Subarrays With Fixed Bounds ↗

Sliding Window - Fixed Size (1) ↗

- [ ] #30 Substring with Concatenation of All Words ↗

Sliding Window - Dynamic Size (2) ↗

- [ ] #727 Minimum Window Subsequence ↗
- [ ] #992 Subarrays with K Different Integers ↗

Binary Search (2) ↗

- [ ] #719 Find K-th Smallest Pair Distance ↗
- [ ] #1095 Find in Mountain Array ↗

Depth First Search (DFS) (2) ↗

- [ ] #827 Making A Large Island ↗
- [ ] #2246 Longest Path With Different Adjacent Characters ↗

Breadth First Search (BFS) (2) ↗

- [ ] #1293 Shortest Path in a Grid with Obstacles Elimination ↗
- [ ] #2290 Minimum Obstacle Removal to Reach Corner ↗

Round 4 | Mid | E Easy (20) ↗

Bit Manipulation (3) ↗

- [ ] #231 Power of Two ↗
- [ ] #461 Hamming Distance ↗
- [ ] #645 Set Mismatch ↗

Prefix Sum (4) ↗

- [ ] #303 Range Sum Query - Immutable ↗
- [ ] #724 Find Pivot Index ↗
- [ ] #1480 Running Sum of 1d Array ↗
- [ ] #1854 Maximum Population Year ↗

Monotonic Stack (2) ↗

- [ ] #496 Next Greater Element I ↗
- [ ] #1475 Final Prices With a Special Discount in a Shop ↗

Queues (2) ↗

- [ ] #933 Number of Recent Calls ↗
- [ ] #2073 Time Needed to Buy Tickets ↗

Divide and Conquer (1) ↗

- [ ] #1763 Longest Nice Substring ↗

BST / Ordered Set (3) ↗

- [ ] #653 Two Sum IV - Input is a BST ↗
- [ ] #700 Search in a Binary Search Tree ↗
- [ ] #938 Range Sum of BST ↗

Intervals (1) ↗

- [ ] #228 Summary Ranges ↗

Data Structure Design (2) ↗

- [ ] #225 Implement Stack using Queues ↗
- [ ] #359 Logger Rate Limiter ↗

Greedy (2) ↗

- [ ] #455 Assign Cookies ↗
- [ ] #3074 Apple Redistribution into Boxes ↗

Round 5 | Mid | M Medium (94) ↗

Bit Manipulation (3)

- [ ] #137 Single Number II ↗
- [ ] #201 Bitwise AND of Numbers Range ↗
- [ ] #260 Single Number III ↗

Prefix Sum (6) ↗

- [ ] #304 Range Sum Query 2D - Immutable ↗
- [ ] #370 Range Addition ↗
- [ ] #523 Continuous Subarray Sum ↗
- [ ] #974 Subarray Sums Divisible by K ↗
- [ ] #1314 Matrix Block Sum ↗
- [ ] #2536 Increment Submatrices by One ↗

Kadane's Algorithm (5) ↗

- [ ] #918 Maximum Sum Circular Subarray ↗
- [ ] #978 Longest Turbulent Subarray ↗
- [ ] #1014 Best Sightseeing Pair ↗
- [ ] #1186 Maximum Subarray Sum with One Deletion ↗
- [ ] #1749 Maximum Absolute Sum of Any Subarray ↗

Matrix (2D Array) (1) ↗

- [ ] #289 Game of Life ↗

LinkedList In-place Reversal (1) ↗

- [ ] #92 Reverse Linked List II ↗

Monotonic Stack (9) ↗

- [ ] #402 Remove K Digits ↗
- [ ] #456 132 Pattern ↗
- [ ] #503 Next Greater Element II ↗
- [ ] #581 Shortest Unsorted Continuous Subarray ↗
- [ ] #769 Max Chunks To Make Sorted ↗
- [ ] #901 Online Stock Span ↗
- [ ] #907 Sum of Subarray Minimums ↗
- [ ] #1019 Next Greater Node In Linked List ↗
- [ ] #2104 Sum of Subarray Ranges ↗

Queues (3) ↗

- [ ] #950 Reveal Cards In Increasing Order ↗
- [ ] #1823 Find the Winner of the Circular Game ↗
- [ ] #2327 Number of People Aware of a Secret ↗

Monotonic Queue (5) ↗

- [ ] #1438 Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit ↗
- [ ] #1673 Find the Most Competitive Subsequence ↗
- [ ] #1696 Jump Game VI ↗
- [ ] #2762 Continuous Subarrays ↗
- [ ] #3578 Count Partitions With Max-Min Difference at Most K ↗

Divide and Conquer (3) ↗

- [ ] #109 Convert Sorted List to Binary Search Tree ↗
- [ ] #427 Construct Quad Tree ↗
- [ ] #654 Maximum Binary Tree ↗

Merge Sort (1) ↗

- [ ] #912 Sort an Array ↗

QuickSort / QuickSelect (1) ↗

- [ ] #1985 Find the Kth Largest Integer in the Array ↗

Backtracking (4) ↗

- [ ] #47 Permutations II ↗
- [ ] #77 Combinations ↗
- [ ] #93 Restore IP Addresses ↗
- [ ] #216 Combination Sum III ↗

BST / Ordered Set (10) ↗

- [ ] #99 Recover Binary Search Tree ↗
- [ ] #426 Convert Binary Search Tree to Sorted Doubly Linked List ↗
- [ ] #450 Delete Node in a BST ↗
- [ ] #510 Inorder Successor in BST II ↗
- [ ] #669 Trim a Binary Search Tree ↗
- [ ] #701 Insert into a Binary Search Tree ↗
- [ ] #729 My Calendar I ↗
- [ ] #731 My Calendar II ↗
- [ ] #1008 Construct Binary Search Tree from Preorder Traversal ↗
- [ ] #2034 Stock Price Fluctuation ↗

Tries (6) ↗

- [ ] #421 Maximum XOR of Two Numbers in an Array ↗
- [ ] #648 Replace Words ↗
- [ ] #1233 Remove Sub-Folders from the Filesystem ↗
- [ ] #1268 Search Suggestions System ↗
- [ ] #2452 Words Within Two Edits of Dictionary ↗
- [ ] #3043 Find the Length of the Longest Common Prefix ↗

Heaps (4) ↗

- [ ] #1405 Longest Happy String ↗

- [ ] #1642 Furthest Building You Can Reach ↗
- [ ] #1834 Single-Threaded CPU ↗
- [ ] #1882 Process Tasks Using Servers ↗

Intervals (6) ↗

- [ ] #452 Minimum Number of Arrows to Burst Balloons ↗
- [ ] #1094 Car Pooling ↗
- [ ] #1229 Meeting Scheduler ↗
- [ ] #1288 Remove Covered Intervals ↗
- [ ] #1353 Maximum Number of Events That Can Be Attended ↗
- [ ] #2054 Two Best Non-Overlapping Events ↗

K-Way Merge (2) ↗

- [ ] #373 Find K Pairs with Smallest Sums ↗
- [ ] #378 Kth Smallest Element in a Sorted Matrix ↗

Data Structure Design (4) ↗

- [ ] #641 Design Circular Deque ↗
- [ ] #1146 Snapshot Array ↗
- [ ] #1472 Design Browser History ↗
- [ ] #2353 Design a Food Rating System ↗

Greedy (8) ↗

- [ ] #406 Queue Reconstruction by Height ↗
- [ ] #670 Maximum Swap ↗
- [ ] #826 Most Profit Assigning Work ↗
- [ ] #921 Minimum Add to Make Parentheses Valid ↗
- [ ] #1488 Avoid Flood in The City ↗
- [ ] #1717 Maximum Score From Removing Substrings ↗
- [ ] #1871 Jump Game VII ↗
- [ ] #1975 Maximum Matrix Sum ↗

Topological Sort (3) ↗

- [ ] #802 Find Eventual Safe States ↗
- [ ] #1462 Course Schedule IV ↗
- [ ] #2115 Find All Possible Recipes from Given Supplies ↗

Union Find (3) ↗

- [ ] #399 Evaluate Division ↗
- [ ] #547 Number of Provinces ↗
- [ ] #947 Most Stones Removed with Same Row or Column ↗

Minimum Spanning Tree (1) ↗

- [ ] #1135 Connecting Cities With Minimum Cost ↗

Shortest Path (5) ↗

- [ ] #1514 Path with Maximum Probability ↗
- [ ] #1631 Path With Minimum Effort ↗
- [ ] #2976 Minimum Cost to Convert String I ↗
- [ ] #3341 Find Minimum Time to Reach Last Room I ↗
- [ ] #3650 Minimum Cost Path with Edge Reversals ↗

Round 6 | Mid | H Hard (30) ↗

Monotonic Stack (2) ↗

- [ ] #321 Create Maximum Number ↗
- [ ] #1944 Number of Visible People in a Queue ↗

Monotonic Queue (2) ↗

- [ ] #862 Shortest Subarray with Sum at Least K ↗
- [ ] #1499 Max Value of Equation ↗

Recursion (2) ↗

- [ ] #273 Integer to English Words ↗

- [ ] #761 Special Binary String ↗

Merge Sort (2) ↗

- [ ] #327 Count of Range Sum ↗
- [ ] #493 Reverse Pairs ↗

Backtracking (2) ↗

- [ ] #301 Remove Invalid Parentheses ↗
- [ ] #980 Unique Paths III ↗

Tries (3) ↗

- [ ] #425 Word Squares ↗
- [ ] #472 Concatenated Words ↗
- [ ] #1032 Stream of Characters ↗

Heaps (1) ↗

- [ ] #1383 Maximum Performance of a Team ↗

Two Heaps (2) ↗

- [ ] #480 Sliding Window Median ↗
- [ ] #502 IPO ↗

Intervals (1) ↗

- [ ] #2402 Meeting Rooms III ↗

Data Structure Design (2) ↗

- [ ] #715 Range Module ↗
- [ ] #2296 Design a Text Editor ↗

Greedy (4) ↗

- [ ] #135 Candy ↗
- [ ] #757 Set Intersection Size At Least Two ↗
- [ ] #857 Minimum Cost to Hire K Workers ↗
- [ ] #871 Minimum Number of Refueling Stops ↗

Topological Sort (1) ↗

- [ ] #1203 Sort Items by Groups Respecting Dependencies ↗

Union Find (1) ↗

- [ ] #924 Minimize Malware Spread ↗

Minimum Spanning Tree (3) ↗

- [ ] #1168 Optimize Water Distribution in a Village ↗
- [ ] #1489 Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree ↗
- [ ] #3600 Maximize Spanning Tree Stability with Upgrades ↗

Shortest Path (2) ↗

- [ ] #1368 Minimum Cost to Make at Least One Valid Path in a Grid ↗
- [ ] #2203 Minimum Weighted Subgraph With the Required Paths ↗

Round 7 | Low | E Easy (6) ↗

1-D DP (1) ↗

- [ ] #509 Fibonacci Number ↗

2D Grid DP (1) ↗

- [ ] #118 Pascal's Triangle ↗

Maths / Geometry (3) ↗

- [ ] #168 Excel Sheet Column Title ↗
- [ ] #263 Ugly Number ↗
- [ ] #1071 Greatest Common Divisor of Strings ↗

String Matching (1) ↗

- [ ] #459 Repeated Substring Pattern ↗

Round 8 | Low | M Medium (42) ↗

Bucket Sort (2) ↗

- [ ] #164 Maximum Gap ↗
- [ ] #451 Sort Characters By Frequency ↗

1-D DP (4) ↗

- [ ] #343 Integer Break ↗
- [ ] #646 Maximum Length of Pair Chain ↗
- [ ] #740 Delete and Earn ↗
- [ ] #1262 Greatest Sum Divisible by Three ↗

0/1 Knapsack (2) ↗

- [ ] #474 Ones and Zeroes ↗
- [ ] #1049 Last Stone Weight II ↗

Unbounded Knapsack (2) ↗

- [ ] #279 Perfect Squares ↗
- [ ] #983 Minimum Cost For Tickets ↗

Longest Increasing Subsequence (LIS) (3) ↗

- [ ] #673 Number of Longest Increasing Subsequence ↗
- [ ] #1027 Longest Arithmetic Subsequence ↗
- [ ] #1048 Longest String Chain ↗

2D Grid DP (5) ↗

- [ ] #63 Unique Paths II ↗
- [ ] #120 Triangle ↗
- [ ] #931 Minimum Falling Path Sum ↗
- [ ] #1277 Count Square Submatrices with All Ones ↗
- [ ] #1937 Maximum Number of Points with Cost ↗

String DP (1) ↗

- [ ] #1653 Minimum Deletions to Make String Balanced ↗

Tree / Graph DP (3) ↗

- [ ] #95 Unique Binary Search Trees II ↗
- [ ] #337 House Robber III ↗
- [ ] #1976 Number of Ways to Arrive at Destination ↗

Bitmask DP (4) ↗

- [ ] #698 Partition to K Equal Sum Subsets ↗
- [ ] #1066 Campus Bikes II ↗
- [ ] #1986 Minimum Number of Work Sessions to Finish the Tasks ↗
- [ ] #2305 Fair Distribution of Cookies ↗

Digit DP (1) ↗

- [ ] #357 Count Numbers with Unique Digits ↗

Probability DP (3) ↗

- [ ] #688 Knight Probability in Chessboard ↗
- [ ] #808 Soup Servings ↗
- [ ] #837 New 21 Game ↗

State Machine DP (1) ↗

- [ ] #714 Best Time to Buy and Sell Stock with Transaction Fee ↗

Maths / Geometry (8) ↗

- [ ] #172 Factorial Trailing Zeroes ↗
- [ ] #204 Count Primes ↗
- [ ] #264 Ugly Number II ↗
- [ ] #593 Valid Square ↗
- [ ] #611 Valid Triangle Number ↗
- [ ] #939 Minimum Area Rectangle ↗
- [ ] #963 Minimum Area Rectangle II ↗
- [ ] #1922 Count Good Numbers ↗

String Matching (2) ↗

- [ ] #686 Repeated String Match ↗
- [ ] #1698 Number of Distinct Substrings in a String ↗

Binary Indexed Tree / Segment Tree (1) ↗

- [ ] #308 Range Sum Query 2D - Mutable ↗

Round 9 | Low | H Hard (36) ↗

Bucket Sort (1) ↗

- [ ] #220 Contains Duplicate III ↗

Eulerian Circuit (2) ↗

- [ ] #753 Cracking the Safe ↗
- [ ] #2097 Valid Arrangement of Pairs ↗

1-D DP (1) ↗

- [ ] #1411 Number of Ways to Paint N × 3 Grid ↗

0/1 Knapsack (1) ↗

- [ ] #879 Profitable Schemes ↗

Longest Increasing Subsequence (LIS) (1) ↗

- [ ] #354 Russian Doll Envelopes ↗

2D Grid DP (10) ↗

- [ ] #85 Maximal Rectangle ↗
- [ ] #174 Dungeon Game ↗
- [ ] #403 Frog Jump ↗
- [ ] #465 Optimal Account Balancing ↗
- [ ] #546 Remove Boxes ↗
- [ ] #741 Cherry Pickup ↗
- [ ] #1335 Minimum Difficulty of a Job Schedule ↗
- [ ] #1547 Minimum Cost to Cut a Stick ↗
- [ ] #1931 Painting a Grid With Three Different Colors ↗
- [ ] #3651 Minimum Cost Path with Teleportations ↗

String DP (4) ↗

- [ ] #44 Wildcard Matching ↗
- [ ] #140 Word Break II ↗
- [ ] #664 Strange Printer ↗
- [ ] #1092 Shortest Common Supersequence ↗

Tree / Graph DP (2) ↗

- [ ] #834 Sum of Distances in Tree ↗
- [ ] #968 Binary Tree Cameras ↗

Bitmask DP (1) ↗

- [ ] #847 Shortest Path Visiting All Nodes ↗

Digit DP (2) ↗

- [ ] #233 Number of Digit One ↗
- [ ] #902 Numbers At Most N Given Digit Set ↗

State Machine DP (1) ↗

- [ ] #188 Best Time to Buy and Sell Stock IV ↗

Maths / Geometry (1) ↗

- [ ] #149 Max Points on a Line ↗

String Matching (3) ↗

- [ ] #214 Shortest Palindrome ↗
- [ ] #1044 Longest Duplicate Substring ↗
- [ ] #1392 Longest Happy Prefix ↗

Binary Indexed Tree / Segment Tree (4) ↗

- [ ] #699 Falling Squares ↗
- [ ] #2426 Number of Pairs Satisfying Inequality ↗
- [ ] #3161 Block Placement Queries ↗
- [ ] #3721 Longest Balanced Subarray II ↗

Line Sweep (2) ↗

- [ ] #391 Perfect Rectangle ↗

#850 Rectangle Area II

Round 1

High Priority · Easy
38 questions · 12 patterns

- [ ] Arrays #26 #27 #414 #485 #1470
- [ ] Strings #28 #58 #344 #392 #680 #796
- [ ] Hash Tables #205 #219 #290 #350 #387 #448 #1189 #1512
- [ ] Two Pointers #696 #1768
- [ ] Sliding Window – Fixed Size #643
- [ ] Binary Search #367 #744
- [ ] Stacks #682 #1047 #1614
- [ ] Tree Traversal – Level Order #637
- [ ] Tree Traversal – Pre Order #144 #222 #257 #404
- [ ] Tree Traversal – In Order #94 #530 #783
- [ ] Tree Traversal – Post-Order #145
- [ ] Linked List #83 #160

Arrays

Round 1 · High · Easy · 5 questions

Arrays

#26 Remove Duplicates from Sorted Array EAS

Open on LeetCode

Array · Two Pointers

Lists: AlgoMaster 600

Problem

Given an integer array nums sorted in non-decreasing order, remove the duplicates in-place such that each unique element appears only once. The relative order of the elements should be kept the same.

Consider the number of unique elements in nums to be k . After removing duplicates, return the number of unique elements k.

The first k elements of nums should contain the unique numbers in sorted order. The remaining elements beyond index k - 1 can be ignored.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
assert nums[i] == expectedNums[i];
```

If all assertions pass, then your solution will be accepted.

Example 1:

```
Input: nums = [1,1,2]
Output: 2, nums = [1,2,_]
Explanation: Your function should return k = 2, with the first two elements of nums being 1 and 2 respectively.
It does not matter what you leave beyond the returned k (hence they are underscores).
```

Example 2:

```
Input: nums = [0,0,1,1,1,2,2,3,3,4]
Output: 5, nums = [0,1,2,3,4,_,_,_,_,_]
Explanation: Your function should return k = 5, with the first five elements of nums being 0, 1, 2, 3, and 4 respectively.
It does not matter what you leave beyond the returned k (hence they are underscores).
```

Constraints:

- 1 <= nums.length <= 3 * 104
- -100 <= nums[i] <= 100
- nums is sorted in non-decreasing order.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# We have a sorted integer array and we need to remove duplicates in-place
# so each value appears only once. We return k — the count of unique elements —
# and the first k slots of nums must hold those unique values in sorted order. Right?"
#
# "A few quick questions:
# Can values be negative? Any constraint on the range?
# The elements beyond index k don't matter — we just need the first k to be correct?
# Is the array guaranteed to be non-empty?"
#
# "Let me walk the example: nums=[1,1,2].
# After processing: nums=[1,2,_], k=2. Got it."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# I'll collect all unique values into a set, sort them, and overwrite the front of
# the array. This costs O(n log n) for the sort and O(n) extra space for the set.
# But since the array is already sorted, the sort is completely wasted work —
# duplicates are adjacent so we never needed to sort at all.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "The bottleneck is the sort and set — both unnecessary because nums is already sorted.
# In a sorted array duplicates are always adjacent, so a single write pointer k is enough.
# Start k at 1. Walk forward: whenever nums[i] differs from nums[k-1], it's a new unique —
# write it at nums[k] and advance k. Elements equal to nums[k-1] are simply skipped.
#
# Pseudocode:
# k = 1
# for i from 1 to len(nums)-1:
# if nums[i] != nums[k-1]:
# nums[k] = nums[i]; k++
# return k
#
```



```
# Time: O(n) — one pass. Space: O(1) — two integer pointers. Does that make sense?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# nums=[1,1,2]: k=1.
# i=1: nums[1]=1 == nums[0]=1 — skip.
# i=2: nums[2]=2 ≠ nums[0]=1 — write nums[1]=2, k=2.
# return 2. nums=[1,2,_] ✓
#
# nums=[0,0,1,1,1,2,2,3,3,4]: k=1.
# i=1: dup skip. i=2: 1≠0 — write, k=2.
# i=3,4: dups skip. i=5: 2≠1 — write, k=3.
# i=6: dup. i=7: 3≠2 — k=4. i=8: dup. i=9: 4≠3 — k=5.
# return 5. nums=[0,1,2,3,4,...] ✓
#
# "No bugs. Handles duplicates of any length correctly."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — one forward pass, each element visited once.
# Space O(1) — the write pointer k and loop index i are the only extras.
# The key insight: sorted order guarantees duplicates are adjacent,
# so a single write pointer is all we need — no hashing, no sorting.
# Follow-up: Remove Duplicates II (LC 80) — allow at most 2 of each.
# Same pattern but compare against nums[k-2] instead of nums[k-1].
# Follow-up: generalise to 'at most m copies' — compare nums[i] vs nums[k-m]."
```

Arrays

#27 Remove Element EAS

Open on LeetCode

Array · Two Pointers

Lists: AlgoMaster 600

Problem

Given an integer array nums and an integer val, remove all occurrences of val in nums in-place. The order of the elements may be changed. Then return the number of elements in nums which are not equal to val.

Consider the number of elements in nums which are not equal to val be k, to get accepted, you need to do the following things:

- Change the array nums such that the first k elements of nums contain the elements which are not equal to val. The remaining elements of nums are not important as well as the size of nums.
- Return k.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int val = ...; // Value to remove
int[] expectedNums = [...]; // The expected answer with correct length.
// It is sorted with no values equaling val.

int k = removeElement(nums, val); // Calls your implementation

assert k == expectedNums.length;
```

If all assertions pass, then your solution will be accepted.

Example 1:

```
Input: nums = [3,2,2,3], val = 3
Output: 2, nums = [2,2,_,_]
Explanation: Your function should return k = 2, with the first two elements of nums being 2.
It does not matter what you leave beyond the returned k (hence they are underscores).
```

Example 2:

```
Input: nums = [0,1,2,2,3,0,4,2], val = 2
Output: 5, nums = [0,1,4,0,3,_,_,_]
Explanation: Your function should return k = 5, with the first five elements of nums containing 0, 0, 1, 3, and 4.
Note that the five elements can be returned in any order.
It does not matter what you leave beyond the returned k (hence they are underscores).
```

Constraints:

- 0 <= nums.length <= 100
- 0 <= nums[i] <= 50
- 0 <= val <= 100

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# We're given an array and a value val, and we need to remove every occurrence
# of val in-place. We return k — the count of elements that are NOT val —
# and the first k slots of nums must hold those non-val elements. Order can change. Right?"
#
# "A few quick questions:
# Can the array be empty — would we return k=0?
# If val doesn't appear at all, we return len(nums)?
# Does it matter which order the kept elements appear in?"
#
# "Let me trace the example: nums=[3,2,2,3], val=3.
# We want k=2 and the first two slots to be the two 2s. ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# I'll filter out all elements equal to val into a new list, then copy them back.
# Time: O(n). Space: O(k) for the filtered list.
# The bottleneck is that extra allocation — we're creating an intermediate structure
# that isn't needed, since we can write the kept elements directly over the front
# of the original array with a single pointer.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "The bottleneck is the extra list. I can avoid it entirely with a write pointer k.
# Walk through nums: whenever nums[i] != val, write it at nums[k] and advance k.
# Elements equal to val are simply skipped — the write pointer never moves for them.
#
# Pseudocode:
# k = 0
# for each num in nums:
# if num != val: nums[k] = num; k++
```



```
# return k
#
# Time: O(n) — one pass. Space: O(1) — one pointer. Does that make sense?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach."

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# nums=[3,2,2,3], val=3: k=0.
# num=3: ==val, skip. num=2: ≠val — nums[0]=2, k=1.
# num=2: ≠val — nums[1]=2, k=2. num=3: ==val, skip.
# return 2. nums=[2,2,_,_] ✓
#
# nums=[0,1,2,2,3,0,4,2], val=2: k=0.
# 0→k=1. 1→k=2. 2 skip. 2 skip. 3→k=3. 0→k=4. 4→k=5. 2 skip.
# return 5. nums=[0,1,3,0,4,...] ✓
#
# nums=[1], val=1: num=1 == val, skip. return 0 ✓
#
# "No bugs. Handles all-match and no-match cases correctly."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — single forward pass. Space O(1) — one write pointer.
# Since order doesn't need to be preserved, there's also a two-pointer variant:
# when we find val at left, swap it with nums[right] and shrink right.
# That approach minimises write operations when val is rare — useful if writing is expensive.
# Follow-up: Remove Duplicates (LC 26) — same write-pointer, different skip condition.
# Follow-up: Move Zeroes (LC 283) — same idea, but zeros go to the back after the kept elements."
```

Arrays

#414 Third Maximum Number EAS

Open on LeetCode

Array · Sorting

Lists: AlgoMaster 600

Problem

Given an integer array nums, return the third distinct maximum number in this array. If the third maximum does not exist, return the maximum number.

Example 1:

```
Input: nums = [3,2,1]
Output: 1
Explanation:
The first distinct maximum is 3.
The second distinct maximum is 2.
The third distinct maximum is 1.
```

Example 2:

```
Input: nums = [1,2]
Output: 2
Explanation:
The first distinct maximum is 2.
The second distinct maximum is 1.
The third distinct maximum does not exist, so the maximum (2) is returned instead.
```

Example 3:

```
Input: nums = [2,2,3,1]
Output: 1
Explanation:
The first distinct maximum is 3.
The second distinct maximum is 2 (both 2's are counted together since they have the same value).
The third distinct maximum is 1.
```

Constraints:

- 1 <= nums.length <= 104
- -231 <= nums[i] <= 231 - 1

Follow up: Can you find an O(n) solution?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# We want the third DISTINCT maximum. If fewer than 3 distinct values exist,
# we return the maximum. And duplicates don't count toward the ranking — so
# [1,1,2] has only 2 distinct values and we'd return 2? Right?"
#
# "A few quick questions:
# Can values be negative? That matters for my choice of sentinel.
# What's the size constraint on nums?"
#
# "Let me trace: nums=[2,2,3,1]. Distinct values: {1,2,3}.
# Third max = 1 ✓. And nums=[1,2]: only 2 distinct → return 2 ✓."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# I'll put all values in a set to deduplicate, sort descending, and index.
# If there are at least 3 values return index 2, otherwise return index 0.
# The bottleneck is the sort — O(n log n) — but we only care about the top 3 values,
# so a full sort is overkill. We're doing way more work than necessary.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "The bottleneck is sorting when we only need the top 3.
# I'll maintain three variables — first, second, third — tracking the current top tier.
# I use float('-inf') as the initial value so it works even with very negative integers.
# For each number: skip if it already appears in our top 3;
# otherwise cascade it down from first to second to third as appropriate.
#
# Pseudocode:
# first = second = third = -inf
# for n in nums:
# if n in (first, second, third): continue
# if n > first: third=second; second=first; first=n
# elif n > second: third=second; second=n
# elif n > third: third=n
# return third if third != -inf else first
#
# Time: O(n) — one pass, constant comparisons. Space: O(1) — three scalars."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach."
```



```
# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# nums=[2,2,3,1]:
# n=2: first=2. n=2: dup, skip. n=3: 3>first=2 — third=-inf,second=2,first=3.
# n=1: 1<second=2, 1>third=-inf — third=1.
# third=1 ≠ -inf → return 1 ✓
#
# nums=[1,2]:
# n=1: first=1. n=2: 2>1 — second=1,first=2.
# third still -inf → return first=2 ✓
#
# nums=[-1,-2,-3]:
# first=-1, second=-2, third=-3. third≠-inf → return -3 ✓
#
# "No bugs. float('-inf') handles negative inputs correctly."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — one pass, constant-work per element. Space O(1) — three scalar variables.
# Using float('-inf') as the sentinel is critical: 0 or None would fail on
# arrays containing very negative integers.
# Follow-up: kth Largest Element (LC 215) — generalise to arbitrary k with a min-heap of size k;
# heap gives O(n log k) time and O(k) space.
# Follow-up: return the actual third-maximum value AND its index — track index alongside value."
```

Arrays

#485 Max Consecutive Ones

EAS

Open on LeetCode

Array

Lists: AlgoMaster 600

Problem

Given a binary array nums, return the maximum number of consecutive 1's in the array.

Example 1:

```
Input: nums = [1,1,0,1,1,1]
Output: 3
Explanation: The first two digits or the last three digits are consecutive 1s. The maximum number of consecutive 1s is 3.
```

Example 2:

```
Input: nums = [1,0,1,1,0,1]
Output: 2
```

Constraints:

- 1 <= nums.length <= 105
- nums[i] is either 0 or 1.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# Given a binary array of only 0s and 1s, return the length of the longest
# run of consecutive 1s. If there are no 1s we return 0. Right?"
#
# "A few quick questions:
# Can the array be empty?
# Is it guaranteed to contain only 0 and 1?"
#
# "Let me trace: nums=[1,1,0,1,1,1].
# Runs: [1,1] length 2, then [1,1,1] length 3. Answer = 3 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# For every starting index i I scan rightward counting consecutive 1s until I hit a 0.
# I track the maximum count seen. The bottleneck is restarting from scratch at every i —
# we re-examine elements that a previous iteration already counted, giving O(n²) total.
# For an array of all 1s we do 1+2+3+...+n comparisons.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "The bottleneck is the nested restart. Instead I keep a running streak:
# increment when I see a 1, reset to 0 on a 0, and update the global best every step.
# One pass, no inner loop.
#
# Pseudocode:
# best = curr = 0
# for each n in nums:
# curr = curr + 1 if n == 1 else 0
# best = max(best, curr)
# return best
#
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach."

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# nums=[1,1,0,1,1,1]:
# n=1: curr=1,best=1. n=1: curr=2,best=2. n=0: curr=0.
# n=1: curr=1. n=1: curr=2. n=1: curr=3,best=3. return 3 ✓
#
# nums=[0]: curr=0 throughout. return 0 ✓
#
# nums=[1,1,1]: curr counts to 3. best=3 ✓
#
# "No bugs. Resets cleanly on zeros and tracks the best streak."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — single pass. Space O(1) — two counters.
# curr is enough because we don't need to remember WHERE the streak started, only HOW LONG it is.
# Follow-up: Max Consecutive Ones II (LC 487) — flip at most one 0; sliding window tracking zero count.
# Follow-up: Max Consecutive Ones III (LC 1004) — flip at most k zeros;
# same sliding window, evict the leftmost when zero-count exceeds k."
```


Arrays

#1470 Shuffle the Array

EAS

Open on LeetCode

Array

Lists: AlgoMaster 600

Problem

Given the array nums consisting of 2n elements in the form [x1,x2,...,xn,y1,y2,...,yn].

Return the array in the form [x1,y1,x2,y2,...,xn,yn].

Example 1:

```
Input: nums = [2,5,1,3,4,7], n = 3

Output: [2,3,5,4,1,7]

Explanation: Since x1=2, x2=5, x3=1, y1=3, y2=4, y3=7 then the answer is [2,3,5,4,1,7].
```

Example 2:

```
Input: nums = [1,2,3,4,4,3,2,1], n = 4

Output: [1,4,2,3,3,2,4,1]
```

Example 3:

```
Input: nums = [1,1,2,2], n = 2

Output: [1,2,1,2]
```

Constraints:

- 1 <= n <= 500
- nums.length == 2n
- 1 <= nums[i] <= 10^3

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# Given nums=[x1,x2,...,xn,y1,y2,...,yn] and n, return [x1,y1,x2,y2,...,xn,yn] —
# interleaving the two halves. Right?"
#
# "A few quick questions:
# n is always exactly half the array length — so the input is always even-length?
# Can values be zero or negative?"
#
# "Let me trace: nums=[2,5,1,3,4,7], n=3.
# First half: [2,5,1]. Second half: [3,4,7]. Interleaved: [2,3,5,4,1,7] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# I'll build a result list by looping i from 0 to n-1, appending nums[i] then nums[i+n].
# This is already O(n) time and O(n) space for the output — the output allocation
# is unavoidable since we need to return a new arrangement.
# There's no meaningful bottleneck to remove here; the problem requires returning
# a new list. Should I just clean it up into the optimized version?"

# PHASE 3 — OPTIMIZE
# "The brute force is already optimal — O(n) time and O(n) space (unavoidable for output).
# I can make it more Pythonic with a list comprehension that flattens the pairs,
# which avoids calling append in a loop and is slightly more idiomatic.
#
# return [val for i in range(n) for val in (nums[i], nums[i+n])]
#
# Time: O(n). Space: O(n). Same complexity, cleaner syntax."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the clean version."

# PHASE 5 — TEST & DEBUG
# "Let me trace through the examples."
#
# nums=[2,5,1,3,4,7], n=3:
# i=0: (2,3). i=1: (5,4). i=2: (1,7). → [2,3,5,4,1,7] ✓
#
# nums=[1,2,3,4,4,3,2,1], n=4:
# i=0:(1,4). i=1:(2,3). i=2:(3,2). i=3:(4,1). → [1,4,2,3,3,2,4,1] ✓
#
# "No bugs."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — every element accessed once. Space O(n) — the output array is unavoidable.
# If the interviewer asks for an in-place O(1) space solution:
# possible by encoding two values per slot with bit manipulation when values are bounded,
# but that's a tricky edge-case trick rarely required in standard interviews.
# Follow-up: Interleave Two Arrays of different lengths — same loop, add the leftover tail.
```

```
# Follow-up: un-shuffle (split interleaved back into two halves) — reverse the index mapping."
```


Strings

Round 1 · High · Easy · 6 questions

Strings

#28 Find the Index of the First Occurrence in a String

EAS

Open on LeetCode

Two Pointers · String · String Matching

Lists: AlgoMaster 600

Problem

Given two strings needle and haystack, return the index of the first occurrence of needle in haystack, or -1 if needle is not part of haystack.

Example 1:

```
Input: haystack = "sadbutsad", needle = "sad"
Output: 0
Explanation: "sad" occurs at index 0 and 6.
The first occurrence is at index 0, so we return 0.
```

Example 2:

```
Input: haystack = "leetcode", needle = "leeto"
Output: -1
Explanation: "leeto" did not occur in "leetcode", so we return -1.
```

Constraints:

- **1 <= haystack.length, needle.length <= 104**
- **haystack and needle consist of only lowercase English characters.**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# Given a haystack string and a needle string, return the index of the first
# occurrence of needle in haystack. Return -1 if needle isn't found. Right?"
#
# "A few quick questions:
# Can needle be empty — return 0 in that case?
# Both strings contain only lowercase letters?
# Should I avoid using Python's built-in find?"
#
# "Let me trace: haystack='sadbutsad', needle='sad'.
# At index 0: 'sad' matches → return 0 ✓.
# haystack='leetcode', needle='leeto': no substring matches → return -1 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# For each starting position i, I slice haystack[i:i+m] and compare it to needle.
# Each comparison is O(m) and I do up to n-m+1 comparisons, giving O(n*m) total.
# The bottleneck is redoing character comparisons we've already made: when we find a
# partial match and then fail, we restart from position i+1 even though some of those
# characters were already compared. KMP avoids that redundant work.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "The bottleneck is restarting from scratch in needle on every mismatch.
# KMP precomputes a failure function: fail[j] = length of the longest proper prefix
# of needle[:j+1] that is also a suffix. On a mismatch at position j in needle,
# instead of resetting j to 0, we jump to fail[j-1] — skipping work we already did.
#
# Pseudocode:
# Build fail[]: fail[i] = longest prefix=suffix of needle[:i+1].
# j = 0
# for i, ch in haystack:
# while j > 0 and ch != needle[j]: j = fail[j-1]
# if ch == needle[j]: j++
# if j == m: return i - m + 1
# return -1
#
# Time: O(n+m). Space: O(m) for the failure array."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the KMP approach."

# PHASE 5 — TEST & DEBUG
# "Let me trace through the examples."
#
# haystack='sadbutsad', needle='sad': fail=[0,0,0].
# Match 's','a','d' at i=0,1,2 → j=3=m → return 0-3+1=0 ✓
#
# haystack='leetcode', needle='leeto': fail=[0,0,0,1,0].
# 'l','e','e','t' match (j=4). 'c'≠'o' → j=fail[3]=1. 'c'≠'e' → j=fail[0]=0.
# Continue — no full match. return -1 ✓
#
# "No bugs. KMP avoids redundant comparisons."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Brute: O(n*m). KMP: O(n+m) time, O(m) space — linear in both strings.
# In practice Python's 'in' uses Boyer-Moore-Horspool which is faster on average,
# but KMP has the best worst-case guarantee.
# Follow-up: find ALL occurrences — after a match, set j=fail[m-1] and continue instead of returning.
# Follow-up: Repeated String Match (LC 686) — how many times to repeat haystack until needle is found."
```


· #1470 #58 · · Contents

Strings

#58 Length of Last Word

EAS

Open on LeetCode

String

Lists: AlgoMaster 600

Problem

Given a string s consisting of words and spaces, return the length of the last word in the string.
A word is a maximal substring consisting of non-space characters only.

Example 1:

```
Input: s = "Hello World"
Output: 5
Explanation: The last word is "World" with length 5.
```

Example 2:

```
Input: s = "  fly me to the moon  "
Output: 4
Explanation: The last word is "moon" with length 4.
```

Example 3:

```
Input: s = "luffy is still joyboy"
Output: 6
Explanation: The last word is "joyboy" with length 6.
```

Constraints:

- **1 <= s.length <= 104**
- **s consists of only English letters and spaces ' '.**
- **There will be at least one word in s.**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# Given a string of words and spaces, return the length of the last word —
# the last maximal sequence of non-space characters. Right?"
#
# "A few quick questions:
# There's guaranteed to be at least one word so I never return 0?
# Can there be trailing spaces, leading spaces, or multiple consecutive spaces?"
#
# "Let me trace: s='  fly me to the moon  '.
# The last word is 'moon', length 4 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# I'll split the string on whitespace — which handles multiple spaces automatically —
# and return the length of the last element.
# The bottleneck is that split() allocates a full list of ALL words — O(n) space —
# when we only care about the very last word. We scan the entire string and
# store everything even though we discard all but one.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "The bottleneck is storing all words. I can scan from RIGHT to LEFT instead:
# first skip any trailing spaces, then count characters until I hit another space
# or reach the start of the string. No allocation at all.
#
# Pseudocode:
# i = len(s) - 1
# while i >= 0 and s[i] == ' ': i--
# length = 0
# while i >= 0 and s[i] != ' ': length++; i--
# return length
#
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the right-to-left scan."

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# s='  fly me to the moon  ':
# Skip 2 trailing spaces, i lands on 'n'. Count n,o,o,m → length=4. Hit ' ' → stop.
# return 4 ✓
#
# s='Hello World':
# No trailing spaces. Count d,l,r,o,W → length=5. return 5 ✓
#
# s='a ':
# Skip ' ': i=0 → 'a'. Count 'a' → length=1. i=-1 → stop. return 1 ✓
```



```
#
# "No bugs. Handles trailing spaces and multi-space separators cleanly."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — we may scan most of the string in the worst case.
# Space O(1) — two integer pointers, no allocation.
# split() is more readable but Python split() allocates O(n) space.
# Our right-to-left scan wins on memory; on long strings that end with one word it's also faster.
# Follow-up: Length of First Word — same logic scanning left-to-right.
# Follow-up: Reverse Words in a String (LC 151) — split, reverse the word list, join."
```

Strings

#344 Reverse String — EAS

Open on LeetCode

Two Pointers · String

Lists: AlgoMaster 600

Problem

Write a function that reverses a string. The input string is given as an array of characters s.
You must do this by modifying the input array in-place with O(1) extra memory.

Example 1:

```
Input: s = ["h","e","l","l","o"]
Output: ["o","l","l","e","h"]
```

Example 2:

```
Input: s = ["H","a","n","n","a","h"]
Output: ["h","a","n","n","a","H"]
```

Constraints:

- 1 <= s.length <= 105
- s[i] is a printable ascii character.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# We're given a character array and need to reverse it in-place using O(1) extra space —
# no new array. We modify s directly and don't return anything. Right?"
#
# "A few quick questions:
# The input is guaranteed to be a list of single characters?
# Can the array be empty or length 1?"
#
# "Let me trace: s=['h','e','l','l','o'].
# After reversal: ['o','l','l','e','h'] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# I'll create a reversed copy and overwrite the original.
# Time: O(n). Space: O(n) for the copy — which directly violates the O(1) constraint.
# The bottleneck is the extra allocation. Reversing can be done with only two index
# variables by swapping from the outside in.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "The bottleneck is the extra array. With two pointers — left at 0, right at the end —
# we swap the characters they point to and move inward. n/2 swaps, no allocation.
#
# Pseudocode:
# left, right = 0, len(s) - 1
# while left < right:
# swap s[left] and s[right]
# left++; right--
#
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the two-pointer swap."

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# s=['h','e','l','l','o']:
# left=0,right=4: swap h↔o → ['o','e','l','l','h']. left=1,right=3.
# swap e↔l → ['o','l','l','e','h']. left=2=right=2 → stop. ✓
#
# s=['H','a','n','n','a','h']:
# swap H↔h. swap a↔a. swap n↔n. → ['h','a','n','n','a','H'] ✓
#
# s=['a']: left=right=0, loop never runs. s=['a'] unchanged ✓
#
# "No bugs. Even-length and odd-length both handled correctly."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — n/2 swaps each O(1). Space O(1) — two integer pointers.
# Python's s[:] = s[::-1] achieves the same result but creates an intermediate list — O(n) space.
# The two-pointer approach is the only truly in-place solution.
# Follow-up: Reverse Words in a String II (LC 186) — reverse the whole array with this,
# then reverse each individual word to get words in reversed order.
# Follow-up: Palindrome check — apply this and compare to original, or just use two pointers."
```


Strings

#392 Is Subsequence — EAS

Open on LeetCode

Two Pointers · String · Dynamic Programming

Lists: AlgoMaster 600

Problem

Given two strings s and t, return true if s is a subsequence of t, or false otherwise.
A subsequence of a string is a new string that is formed from the original string by deleting some (can be none) of the characters without disturbing the relative positions of the remaining characters. (i.e., "ace" is a subsequence of "abcde" while "aec" is not).

Example 1:

```
Input: s = "abc", t = "ahbgdc"
Output: true
```

Example 2:

```
Input: s = "axc", t = "ahbgdc"
Output: false
```

Constraints:

- 0 <= s.length <= 100
- 0 <= t.length <= 104
- s and t consist only of lowercase English letters.

Follow up: Suppose there are lots of incoming s, say s1, s2, ..., sk where k >= 109, and you want to check one by one to see if t has its subsequence. In this scenario, how would you change your code?

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# We need to check if s is a subsequence of t — all characters of s appear in t
# in the same relative order, but not necessarily contiguously. Right?"
#
# "A few quick questions:
# Can s be empty — and if so, is empty string always a subsequence? I'd say yes.
# Both strings are lowercase English letters only?
# No modification allowed — read-only check?"
#
# "Let me trace: s='ace', t='abcde'.
# Match 'a' at index 0, skip 'b', match 'c' at 2, skip 'd', match 'e' at 4 → true ✓.
# s='aec', t='abcde': match 'a' at 0, need 'e' next — 'e' is at 4, but then 'c'
# must come AFTER index 4 and 'c' is at 2 — impossible → false ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Two pointers: i walks s, j walks t. When s[i]==t[j], advance both; otherwise advance only j.
# If i reaches len(s), all of s was matched. This is already O(n) time and O(1) space —
# it's the optimal solution for a single query. The interesting optimization appears
# in the follow-up: what if we have billions of queries against the same t?
# Should I implement this and then discuss the follow-up?"

# PHASE 3 — OPTIMIZE
# "For the many-queries follow-up: pre-process t into a position map — for each character c,
# store the sorted list of indices where c appears. For each query s, use binary search
# to find the next occurrence of s[i] after the current position in t.
#
# Pre-process: O(|t|). Per query: O(|s| * log|t|). Space: O(|t|).
# For the simple single-query case, the two-pointer is already optimal — O(|t|) time."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the two-pointer approach clearly."

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# s='ace', t='abcde':
# ch='a': matches s[0] → i=1. ch='b': no match. ch='c': matches s[1] → i=2.
# ch='d': no match. ch='e': matches s[2] → i=3. i=3=len(s) → return True ✓
#
# s='axc', t='ahbgdc':
# ch='a': match i=1. ch='h','b','g','d': no 'x'. ch='c': need 'x' first, i=1.
# End of t. i=1 ≠ 3 → return False ✓
#
# s='', t='abc': i=0=len(s) — loop returns True immediately ✓
#
# "No bugs. Empty string and mismatch cases handled correctly."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(|t|) — one pass through t. Space O(1) — one pointer i.
# Follow-up: 10^9 queries against the same t — pre-build position lists per character,
# binary search for next occurrence: O(|t|) build, O(|s| * log|t|) per query.
```

```
# Follow-up: Longest Common Subsequence (LC 1143) — find the length of the longest
# subsequence common to BOTH strings. Requires O(m*n) DP."
```


Strings

#680 Valid Palindrome II EAS

Open on LeetCode

Two Pointers · String · Greedy

Lists: AlgoMaster 600

Problem

Given a string s, return true if the s can be palindrome after deleting at most one character from it.

Example 1:

```
Input: s = "aba"
Output: true
```

Example 2:

```
Input: s = "abca"
Output: true
Explanation: You could delete the character 'c'.
```

Example 3:

```
Input: s = "abc"
Output: false
```

Constraints:

- 1 <= s.length <= 105
- s consists of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# Return true if we can delete AT MOST ONE character from s to make it a palindrome.
# Zero deletions is fine too — if s is already a palindrome, return true. Right?"
#
# "A few quick questions:
# Only lowercase English letters?
# 'At most one' means we can also choose to delete nothing?"
#
# "Let me trace: s='abca'. Outer a==a ✓. Inner: b≠c.
# Try deleting b: check 'ca' → not a palindrome.
# Try deleting c: check 'ba' → not a palindrome.
# Hmm — wait. Deleting 'c' gives 'aba' → palindrome → true ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Try removing each character one at a time (n possibilities), check if the resulting
# string of length n-1 is a palindrome. Each check is O(n) and we do n checks → O(n²).
# The bottleneck is trying ALL n deletions when we actually only need to try at most two:
# a mismatch in the two-pointer scan tells us exactly where the problem is,
# and we only need to try skipping the left or right pointer at that one spot.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "The bottleneck is trying all n deletions. A smarter approach:
# two pointers from both ends. When they match, move inward.
# When they DON'T match, we've found the only candidate spot — try skipping left
# OR skipping right, and check if that substring is a palindrome.
# We run the full palindrome check at most TWICE total, not n times.
#
# Pseudocode:
# l, r = 0, len(s)-1
# while l < r:
# if s[l] == s[r]: l++; r--
# else: return is_palindrome(l+1, r) or is_palindrome(l, r-1)
# return True
#
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the two-pointer approach with an inner helper."

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# s='abca':
# l=0('a'),r=3('a'): match → l=1,r=2. s[1]='b',s[2]='c': mismatch.
# is_pal(2,2): single char → True. return True ✓
#
# s='abc':
# l=0('a'),r=2('c'): mismatch.
# is_pal(1,2): 'b'≠'c' → False. is_pal(0,1): 'a'≠'b' → False. return False ✓
#
# s='aba': l=0,r=2: a==a → l=1,r=1: not l<r → return True ✓
#
```

```
# "No bugs. Only two palindrome checks needed regardless of string length."
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — outer scan O(n) plus at most two O(n) sub-palindrome checks.
# Space O(1) — four integer variables, no allocation.
# Key insight: mismatches only happen at one spot in the two-pointer scan,
# so we never need to try all n deletions.
# Follow-up: Valid Palindrome I (LC 125) — same but ignoring non-alphanumeric chars; prefilter.
# Follow-up: at most k deletions — DP on (l, r, remaining_deletions) is O(n²k)."
```


Strings

#796 Rotate String

EAS

Open on LeetCode

String · String Matching

Lists: AlgoMaster 600

Problem

Given two strings s and goal, return true if and only if s can become goal after some number of shifts on s.

A shift on s consists of moving the leftmost character of s to the rightmost position.

- For example, if s = "abcde", then it will be "bcdea" after one shift.

Example 1:

```
Input: s = "abcde", goal = "cdeab"
Output: true
```

Example 2:

```
Input: s = "abcde", goal = "abced"
Output: false
```

Constraints:

- 1 <= s.length, goal.length <= 100
- s and goal consist of lowercase English letters.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Return true if s can become goal after some number of rotations.
# A rotation shifts the first character to the end. Right?"
#
# "s='abcde',goal='cdeab'→true ✓ s='abcde',goal='abced'→false ✓"

# PHASE 2 — BRUTE FORCE
# "Try all n rotations, compare each with goal. Time: O(n²). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "If goal is a rotation of s, then goal is a substring of s+s.
# Time: O(n). Space: O(n) for s+s string."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='abcde',goal='cdeab': s+s='abcdeabcde'. 'cdeab' in 'abcdeabcde'? Yes at index 2 → True ✓
# s='abcde',goal='abced': 'abced' in 'abcdeabcde'? No → False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): Python's `in` uses optimised string matching. Space O(n) for the doubled string.
# The s+s trick works because every rotation of s is a substring of s+s.
# Follow-up: count how many rotations produce goal — count occurrences of goal in s+s.
# Follow-up: circular shift on a linked list — find the rotation point then relink."
```

Hash Tables

Round 1 · High · Easy · 8 questions

Hash Tables

#205 Isomorphic Strings EAS

Open on LeetCode

Hash Table · String

Lists: AlgoMaster 600

Problem

Given two strings s and t, determine if they are isomorphic.

Two strings s and t are isomorphic if the characters in s can be replaced to get t.

All occurrences of a character must be replaced with another character while preserving the order of characters. No two characters may map to the same character, but a character may map to itself.

Example 1:

Input: s = "egg", t = "add"

Output: true

Explanation:

The strings s and t can be made identical by:

- **Mapping 'e' to 'a'.**
- **Mapping 'g' to 'd'.**

Example 2:

Input: s = "f11", t = "b23"

Output: false

Explanation:

The strings s and t can not be made identical as '1' needs to be mapped to both '2' and '3'.

Example 3:

Input: s = "paper", t = "title"

Output: true

Constraints:

- **1 <= s.length <= 5 * 104**
- **t.length == s.length**
- **s and t consist of any valid ascii character.**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Two strings s and t are isomorphic if chars in s can be replaced to get t,
# with a consistent one-to-one mapping. No two chars map to the same char. Right?"
#
# "s='egg',t='add'→true (e→a,g→d) ✓
# s='foo',t='bar'→false (o can't map to both a and r) ✓"

# PHASE 2 — BRUTE FORCE
# "Try all possible character mappings — too slow. Instead use two dicts.
# Time: O(n). Space: O(1) — at most 256 ASCII chars."

# PHASE 3 — OPTIMIZE
# "Map each string to its first-occurrence index pattern.
# If patterns match, strings are isomorphic.
# Time: O(n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='egg',t='add': encode('egg')=[0,1,1]. encode('add')=[0,1,1]. Equal → True ✓
# s='foo',t='bar': encode('foo')=[0,1,1]. encode('bar')=[0,1,2]. Not equal → False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(distinct chars) ≤ O(256) = O(1).
# Two-dict approach is also O(n),O(1) — checks bijection directly without encoding.
# Follow-up: Word Pattern (LC 290) — same idea but between words and pattern characters.
# Follow-up: group isomorphic strings together — use canonical form as the key."
```

Hash Tables

#219 Contains Duplicate II

EAS

Open on LeetCode

Array · Hash Table · Sliding Window

Lists: AlgoMaster 600

Problem

Given an integer array nums and an integer k, return true if there are two distinct indices i and j in the array such that nums[i] == nums[j] and abs(i - j) <= k.

Example 1:

```
Input: nums = [1,2,3,1], k = 3
Output: true
```

Example 2:

```
Input: nums = [1,0,1,1], k = 1
Output: true
```

Example 3:

```
Input: nums = [1,2,3,1,2,3], k = 2
Output: false
```

Constraints:

- 1 <= nums.length <= 105
- -109 <= nums[i] <= 109
- 0 <= k <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return true if any two equal elements are at most k indices apart. Right?"
#
# "nums=[1,2,3,1],k=3→true (index 0 and 3, diff=3) ✓
# nums=[1,0,1,1],k=1→true (index 2 and 3) ✓
# nums=[1,2,3,1,2,3],k=2→false ✓"

# PHASE 2 — BRUTE FORCE
# "For each pair i<j with nums[i]==nums[j], check j-i<=k. Time: O(n²). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Sliding window of size k using a hash set.
# For each element: if it's in the window, return True. Add it; if window > k, remove oldest.
# Time: O(n). Space: O(k)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[1,2,3,1],k=3: window={1}→{1,2}→{1,2,3}→1 in window→True ✓
# nums=[1,2,3,1,2,3],k=2: window={1}→{1,2}→{1,2,3}>2→remove1→{2,3}→2 in {2,3}?No→{2,3,1}...→False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(k): set holds at most k+1 elements.
# Alt: hashmap {value→last_index}, check if i-map[num]<=k — O(n) time O(n) space.
# Follow-up: Contains Duplicate III (LC 220) — absolute diff ≤ t. Use sorted container / bucket sort."
```


Hash Tables

#290 Word Pattern

EAS

Open on LeetCode

Hash Table · String

Lists: AlgoMaster 600

Problem

Given a pattern and a string s, find if s follows the same pattern.

Here follow means a full match, such that there is a bijection between a letter in pattern and a non-empty word in s.

Specifically:

- Each letter in pattern maps to exactly one unique word in s.
- Each unique word in s maps to exactly one letter in pattern.
- No two letters map to the same word, and no two words map to the same letter.

Example 1:

Input: pattern = "abba", s = "dog cat cat dog"

Output: true

Explanation:

The bijection can be established as:

- 'a' maps to "dog".
- 'b' maps to "cat".

Example 2:

Input: pattern = "abba", s = "dog cat cat fish"

Output: false

Example 3:

Input: pattern = "aaaa", s = "dog cat cat dog"

Output: false

Constraints:

- 1 <= pattern.length <= 300
- pattern contains only lower-case English letters.
- 1 <= s.length <= 3000
- s contains only lowercase English letters and spaces ' '.
- s does not contain any leading or trailing spaces.
- All the words in s are separated by a single space.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a pattern string and words string: return true if there's a bijection between
# pattern chars and words. Right?"
#
# "pattern='abba', s='dog cat cat dog'→true ✓
# pattern='abba', s='dog cat cat fish'→false ✓
# pattern='aaaa', s='dog cat cat dog'→false ✓"

# PHASE 2 — BRUTE FORCE
# "Two-dict approach: p→word and word→p mappings. On mismatch return false.
# Time: O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Encode both pattern and words to first-occurrence index sequences.
# If sequences match → bijection exists.
# Time: O(n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# pattern='abba', words=['dog','cat','cat','dog']:
# encode('abba')=[0,1,1,0]. encode(words)=[0,1,1,0]. Equal → True ✓
# pattern='aaaa', words=['dog','cat','cat','dog']:
# encode='[0,0,0,0]'. encode(words)=[0,1,1,0]. Not equal → False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n): index map and encoded lists.
# Follow-up: Isomorphic Strings (LC 205) — same problem at character level.
# Follow-up: if pattern can have multi-char symbols — tokenize pattern first."
```

Hash Tables

#350 Intersection of Two Arrays II EAS

Open on LeetCode

Array · Hash Table · Two Pointers · Binary Search · Sorting

Lists: AlgoMaster 600

Problem

Given two integer arrays nums1 and nums2, return an array of their intersection. Each element in the result must appear as many times as it shows in both arrays and you may return the result in any order.

Example 1:

```
Input: nums1 = [1,2,2,1], nums2 = [2,2]
Output: [2,2]
```

Example 2:

```
Input: nums1 = [4,9,5], nums2 = [9,4,9,8,4]
Output: [4,9]
Explanation: [9,4] is also accepted.
```

Constraints:

- 1 <= nums1.length, nums2.length <= 1000
- 0 <= nums1[i], nums2[i] <= 1000

Follow up:

- What if the given array is already sorted? How would you optimize your algorithm?
- What if nums1's size is small compared to nums2's size? Which algorithm is better?
- What if elements of nums2 are stored on disk, and the memory is limited such that you cannot load all elements into the memory at once?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return intersection of two arrays including duplicates.
# Each element appears as many times as it appears in both arrays. Order doesn't matter. Right?"
#
# "nums1=[1,2,2,1], nums2=[2,2]→[2,2] ✓
# nums1=[4,9,5], nums2=[9,4,9,8,4]→[4,9] ✓"

# PHASE 2 — BRUTE FORCE
# "Sort both arrays, use two pointers to find common elements.
# Time: O(n log n). Space: O(1) extra."

# PHASE 3 — OPTIMIZE
# "Count frequencies of the smaller array with a hash map.
# For each element in the larger array, if it exists in the map with count > 0, include it.
# Time: O(n+m). Space: O(min(n,m))."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums1=[1,2,2,1],nums2=[2,2]: count={1:2,2:2}.
# n=2: count[2]=2>0 → append, count[2]=1. n=2: count[2]=1>0 → append, count[2]=0.
# result=[2,2] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Hash map: O(n+m) time, O(min(n,m)) space. Sort+two-pointer: O(n log n) time, O(1) space.
# If arrays are already sorted → use two-pointer. If one is much smaller → hash map.
# If data stored on disk → external merge sort.
# Follow-up: Intersection of Two Arrays I (LC 349) — unique elements only; use set intersection."
```


Hash Tables

#387 First Unique Character in a String EAS

Open on LeetCode

Hash Table · String · Queue · Counting

Lists: AlgoMaster 600

Problem

Given a string s, find the first non-repeating character in it and return its index. If it does not exist, return -1.

Example 1:

Input: s = "leetcode"

Output: 0

Explanation:

The character 'l' at index 0 is the first character that does not occur at any other index.

Example 2:

Input: s = "loveleetcode"

Output: 2

Example 3:

Input: s = "aabb"

Output: -1

Constraints:

- 1 <= s.length <= 105
- s consists of only lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the index of the first non-repeating character in s. Return -1 if none. Right?"
#
# "s='leetcode'→0 ('l' appears once) ✓
# s='loveleetcode'→2 ('v') ✓ s='aabb'→-1 ✓"

# PHASE 2 — BRUTE FORCE
# "For each character, scan the entire string to check if it appears elsewhere.
# Time: O(n²). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Count frequencies in one pass (hash map). Second pass finds first with count == 1.
# Time: O(n). Space: O(26) = O(1) for lowercase letters."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='leetcode': count={l:1,e:3,t:1,c:1,o:1,d:1}. i=0,'l':count=1→return 0 ✓
# s='aabb': count={a:2,b:2}. No char with count 1 → return -1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): two passes. Space O(26)=O(1) since only lowercase letters.
# Follow-up: streaming version — maintain a queue of (char, index); pop front when count > 1.
# Follow-up: if Unicode — same approach, hash map instead of array."
```

Hash Tables

#448 Find All Numbers Disappeared in an Array

EAS

Open on LeetCode

Array · Hash Table

Lists: AlgoMaster 600

Problem

Given an array nums of n integers where nums[i] is in the range [1, n], return an array of all the integers in the range [1, n] that do not appear in nums.

Example 1:

```
Input: nums = [4,3,2,7,8,2,3,1]
Output: [5,6]
```

Example 2:

```
Input: nums = [1,1]
Output: [2]
```

Constraints:

- n == nums.length
- 1 <= n <= 105
- 1 <= nums[i] <= n

Follow up: Could you do it without extra space and in O(n) runtime? You may assume the returned list does not count as extra space.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Array of n integers in range [1,n] with some appearing twice and some missing.
# Find all missing numbers. O(n) time, O(1) extra space. Right?"
#
# "nums=[4,3,2,7,8,2,3,1]: missing 5 and 6 → [5,6] ✓"

# PHASE 2 — BRUTE FORCE
# "Use a set of all present values, return [1..n] not in set.
# Time: O(n). Space: O(n). Violates O(1) space constraint."

# PHASE 3 — OPTIMIZE
# "Index-marking: negate nums[abs(nums[i])-1] to mark value i as seen.
# Then indices where value is still positive → those (index+1) values are missing.
# Time: O(n). Space: O(1) — uses input array itself."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[4,3,2,7,8,2,3,1]:
# n=4→negate idx3: [4,3,2,-7,8,2,3,1]. n=3→negate idx2: [4,3,-2,-7,...].
# n=2→negate idx1. n=7→negate idx6. n=8→negate idx7. n=2(abs=2)→idx1 already neg.
# n=3(abs=3)→idx2 already neg. n=1→negate idx0.
# Final: all negative except idx4(8→pos) and idx5(2→pos). Missing: 5,6 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): two passes. Space O(1): input array used as visited marker.
# The sign-as-marker trick requires values in [1,n] to index into the array.
# Follow-up: Find the Duplicate Number (LC 287) — find the one that appears twice instead.
# Follow-up: if values can be 0, use n+1 as offset instead of sign."
```


Hash Tables

#1189 Maximum Number of Balloons

EAS

Open on LeetCode

Hash Table · String · Counting

Lists: AlgoMaster 600

Problem

Given a string text, you want to use the characters of text to form as many instances of the word "balloon" as possible. You can use each character in text at most once. Return the maximum number of instances that can be formed.

Example 1:

```
Input: text = "nlaebolko"
Output: 1
```

Example 2:

```
Input: text = "loonbalxballpoon"
Output: 2
```

Example 3:

```
Input: text = "leetcode"
Output: 0
```

Constraints:

- 1 <= text.length <= 104
- text consists of lower case English letters only.

Note: This question is the same as 2287: Rearrange Characters to Make Target String.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "How many times can you write 'balloon' using characters from text?
# 'balloon' has: b×1, a×1, l×2, o×2, n×1. Right?"
#
# "text='nlaebolko'→1 ✓ text='loonbalxballpoon'→2 ✓ text='leetcode'→0 ✓"

# PHASE 2 — BRUTE FORCE
# "Count each needed character, divide by required count, take minimum.
# Time: O(n). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Same formula — already O(n). Could simplify with only counting the 5 needed chars."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# text='loonbalxballpoon': count={l:4,o:4,n:2,b:2,a:2}. Remove x (not needed).
# b:2//1=2, a:2//1=2, l:4//2=2, o:4//2=2, n:2//1=2. min=2 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1): only 5 character counts needed.
# The 'divide by required count' pattern applies to any word-formation problem.
# Follow-up: if text is streamed, process character by character updating counts.
# Follow-up: Ransom Note (LC 383) — can string a form from string b. Same count comparison."
```

Hash Tables

#1512 Number of Good Pairs EAS

Open on LeetCode

Array · Hash Table · Math · Counting

Lists: AlgoMaster 600

Problem

Given an array of integers nums, return the number of good pairs.

A pair (i, j) is called good if nums[i] == nums[j] and i < j.

Example 1:

```
Input: nums = [1,2,3,1,1,3]
Output: 4
Explanation: There are 4 good pairs (0,3), (0,4), (3,4), (2,5) 0-indexed.
```

Example 2:

```
Input: nums = [1,1,1,1]
Output: 6
Explanation: Each pair in the array are good.
```

Example 3:

```
Input: nums = [1,2,3]
Output: 0
```

Constraints:

- **1 <= nums.length <= 100**
- **1 <= nums[i] <= 100**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count pairs (i,j) where i<j and nums[i]==nums[j]. Right?"
#
# "nums=[1,2,3,1,1,3]→4 ✓ nums=[1,1,1,1]→6 ✓ nums=[1,2,3]→0 ✓"

# PHASE 2 — BRUTE FORCE
# "Try all pairs. Time: O(n²). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "For each number, if it has appeared k times before, it forms k new good pairs.
# Track frequency as we go. Time: O(n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[1,1,1,1]: n=1:pairs+=0,count={1:1}. n=1:pairs+=1,count={1:2}.
# n=1:pairs+=2,count={1:3}. n=1:pairs+=3. total=0+1+2+3=6 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass. Space O(n): frequency map.
# The k*(k-1)/2 formula also works: for each value count k, add k*(k-1)//2 to total.
# Follow-up: count pairs with sum == target (LC 1) — similar counting with complement lookup.
# Follow-up: count pairs within distance k — requires sliding window or sorted structure."
```


Two Pointers

Round 1 · High · Easy · 2 questions

Two Pointers

#696 Count Binary Substrings

EAS

Open on LeetCode

Two Pointers · String

Lists: AlgoMaster 600

Problem

Given a binary string s, return the number of non-empty substrings that have the same number of 0's and 1's, and all the 0's and all the 1's in these substrings are grouped consecutively.

Substrings that occur multiple times are counted the number of times they occur.

Example 1:

```
Input: s = "00110011"
Output: 6
Explanation: There are 6 substrings that have equal number of consecutive 1's and 0's: "0011", "01", "1100", "10", "0011", and "01".
Notice that some of these substrings repeat and are counted the number of times they occur.
Also, "00110011" is not a valid substring because all the 0's (and 1's) are not grouped together.
```

Example 2:

```
Input: s = "10101"
Output: 4
Explanation: There are 4 substrings: "10", "01", "10", "01" that have equal number of consecutive 1's and 0's.
```

Constraints:

- 1 <= s.length <= 105
- s[i] is either '0' or '1'.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count substrings with equal consecutive 0s and 1s (like '0011','10','0110' etc.).
# They must be all 0s then all 1s OR all 1s then all 0s, contiguous. Right?"
#
# "s='00110011'→6 ('0011','01','1100','10','0011','01') ✓
# s='10101'→4 ✓"

# PHASE 2 — BRUTE FORCE
# "Check all substrings, count valid ones. Time: O(n²). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Group consecutive chars into run-lengths (e.g. '00110011'→[2,2,2,2]).
# For each pair of adjacent groups, min(group[i], group[i+1]) valid substrings exist.
# Time: O(n). Space: O(n) for groups, or O(1) with two rolling variables."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='00110011': runs=[2,2,2,2].
# Groups: 00→curr=2. 11→count+=min(0,2)=0,prev=2,curr=2. 00→count+=min(2,2)=2,prev=2,curr=2.
# 11→count+=min(2,2)=2+2=4,prev=2,curr=2. End: count+=min(2,2)=6 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass. Space O(1): two rolling variables prev and curr.
# The key insight: between group sizes g1 and g2, exactly min(g1,g2) valid substrings.
# Follow-up: count all valid substrings up to length k — add max(0, min(g1,g2)-k+1) per pair."
```


Two Pointers

#1768 Merge Strings Alternately

EAS

Open on LeetCode

Two Pointers · String

Lists: AlgoMaster 600

Problem

You are given two strings word1 and word2. Merge the strings by adding letters in alternating order, starting with word1. If a string is longer than the other, append the additional letters onto the end of the merged string.

Return the merged string.

Example 1:

```
Input: word1 = "abc", word2 = "pqr"

Output: "apbqcr"

Explanation: The merged string will be merged as so:

word1: a b c
```

Example 2:

```
Input: word1 = "ab", word2 = "pqrs"

Output: "apbqrs"

Explanation: Notice that as word2 is longer, "rs" is appended to the end.

word1: a b
```

Example 3:

```
Input: word1 = "abcd", word2 = "pq"

Output: "apbqcd"

Explanation: Notice that as word1 is longer, "cd" is appended to the end.

word1: a b c d
```

Constraints:

- 1 <= word1.length, word2.length <= 100
- word1 and word2 consist of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Merge word1 and word2 by alternating characters. Append remaining chars if one is longer. Right?"
#
# "word1='abc',word2='pqr'→'apbqcr' ✓
# word1='ab',word2='pqrs'→'apbqrs' ✓ word1='abcd',word2='pq'→'apbqcd' ✓"

# PHASE 2 — BRUTE FORCE
# "Build result list, zip both strings, append leftovers. Time: O(n+m). Space: O(n+m)."

# PHASE 3 — OPTIMIZE
# "Two-pointer: advance i through word1 and j through word2 simultaneously.
# Append whichever is in range. Single pass. Time: O(n+m). Space: O(n+m)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# word1='ab',word2='pqrs': (a,p),(b,q),(-,r),(-,s) → 'apbqrs' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n+m). Space O(n+m) for output string.
# Follow-up: merge k strings alternately — use a queue, rotate through each string.
# Follow-up: merge preserving relative order of longer string — same approach."
```

Sliding Window – Fixed Size

Round 1 · High · Easy · 1 question

Sliding Window – Fixed Size

#643 Maximum Average Subarray I EAS

Open on LeetCode

Array · Sliding Window

Lists: AlgoMaster 600

Problem

You are given an integer array nums consisting of n elements, and an integer k.

Find a contiguous subarray whose length is equal to k that has the maximum average value and return this value. Any answer with a calculation error less than 10-5 will be accepted.

Example 1:

```
Input: nums = [1,12,-5,-6,50,3], k = 4
Output: 12.75000
Explanation: Maximum average is (12 - 5 - 6 + 50) / 4 = 51 / 4 = 12.75
```

Example 2:

```
Input: nums = [5], k = 1
Output: 5.00000
```

Constraints:

- **n == nums.length**
- **1 <= k <= n <= 105**
- **-104 <= nums[i] <= 104**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find a contiguous subarray of length k with the maximum average. Return the average. Right?"
#
# "nums=[1,12,-5,-6,50,3], k=4: subarrays of 4: max sum=12-5-6+50=51 → avg=12.75 ✓"
# PHASE 2 — BRUTE FORCE
# "Try every subarray of length k, compute sum, track max. Time: O(n*k). Space: O(1)."
# PHASE 3 — OPTIMIZE
# "Fixed sliding window: compute initial sum of first k elements. Slide by adding nums[i]
# and removing nums[i-k]. Track max sum. Time: O(n). Space: O(1)."
# PHASE 4 — CLEAN CODE
# PHASE 5 — TEST & DEBUG
# nums=[1,12,-5,-6,50,3],k=4: init=1+12-5-6=2. i=4: 2+50-1=51. i=5: 51+3-12=42.
# best=51. return 51/4=12.75 ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1).
# Follow-up: Maximum Average Subarray II (LC 644) — subarray length >= k. Binary search on answer.
# Follow-up: find the starting index of the max-average subarray — track argmax alongside."
```

Binary Search

Round 1 · High · Easy · 2 questions

Binary Search

#367 Valid Perfect Square EAS

Open on LeetCode

Math · Binary Search

Lists: AlgoMaster 600

Problem

Given a positive integer num, return true if num is a perfect square or false otherwise.

A perfect square is an integer that is the square of an integer. In other words, it is the product of some integer with itself.

You must not use any built-in library function, such as sqrt.

Example 1:

```
Input: num = 16
Output: true
Explanation: We return true because 4 * 4 = 16 and 4 is an integer.
```

Example 2:

```
Input: num = 14
Output: false
Explanation: We return false because 3.742 * 3.742 = 14 and 3.742 is not an integer.
```

Constraints:

- 1 <= num <= 231 - 1

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Return true if num is a perfect square. No sqrt() allowed. Right?"
#
# "num=16→true (4*4) ✓ num=14→false ✓"

# PHASE 2 — BRUTE FORCE
# "Try every integer from 1 to num//2+1. Time: O(sqrt(n)). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Binary search on [1, num]. Mid*mid == num → True. < num → search right. > num → search left.
# Time: O(log n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# num=16: lo=1,hi=16. mid=8,sq=64>16→hi=7. mid=4,sq=16==16→True ✓
# num=14: mid=7,sq=49>14→hi=6. mid=3,sq=9<14→lo=4. mid=5,sq=25>14→hi=4. mid=4,sq=16>14→hi=3. lo>hi→False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(log n). Space O(1).
# Newton's method: x = (x + num/x) / 2 converges in O(log log n) iterations — even faster.
# Follow-up: find all perfect squares up to n — iterate i² ≤ n, O(sqrt(n)).
# Follow-up: Sqrt(x) (LC 69) — return integer square root; same binary search."
```

Binary Search

#744 Find Smallest Letter Greater Than Target

EAS

Open on LeetCode

Array · Binary Search

Lists: AlgoMaster 600

Problem

You are given an array of characters letters that is sorted in non-decreasing order, and a character target. There are at least two different characters in letters.

Return the smallest character in letters that is lexicographically greater than target. If such a character does not exist, return the first character in letters.

Example 1:

```
Input: letters = ["c","f","j"], target = "a"
Output: "c"
Explanation: The smallest character that is lexicographically greater than 'a' in letters is 'c'.
```

Example 2:

```
Input: letters = ["c","f","j"], target = "c"
Output: "f"
Explanation: The smallest character that is lexicographically greater than 'c' in letters is 'f'.
```

Example 3:

```
Input: letters = ["x","x","y","y"], target = "z"
Output: "x"
Explanation: There are no characters in letters that is lexicographically greater than 'z' so we return letters[0].
```

Constraints:

- 2 <= letters.length <= 104
- letters[i] is a lowercase English letter.
- letters is sorted in non-decreasing order.
- letters contains at least two different characters.
- target is a lowercase English letter.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Sorted circular list of lowercase letters. Find the smallest letter strictly greater than target.
# If none, wrap around to the first letter. Right?"
#
# "letters=['c','f','j'], target='a'→'c' ✓
# letters=['c','f','j'], target='j'→'c' (wrap) ✓ letters=['c','f','j'], target='d'→'f' ✓"

# PHASE 2 — BRUTE FORCE
# "Scan left to right, return first letter > target. If none found, return letters[0].
# Time: O(n). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Binary search for the leftmost position where letters[mid] > target.
# If not found, return letters[0] (circular wrap).
# Time: O(log n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# letters=['c','f','j'],target='j': lo=0,hi=3. mid=1,'f'<='j'→lo=2. mid=2,'j'<='j'→lo=3.
# lo=3=hi. lo%3=0 → letters[0]='c' ✓ (wrap)
# target='d': mid=1,'f'>'d'→hi=1. mid=0,'c'<='d'→lo=1. lo=1=hi. letters[1]='f' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(log n). Space O(1).
# The modulo wrap handles the circular property elegantly.
# Follow-up: if letters can include any character (not just a-z), same approach works.
# Follow-up: Find First and Last Position (LC 34) — similar leftmost binary search pattern."
```


Stacks

Round 1 · High · Easy · 3 questions

Stacks

#682 Baseball Game EAS

Open on LeetCode

Array · Stack · Simulation

Lists: AlgoMaster 600

Problem

You are keeping the scores for a baseball game with strange rules. At the beginning of the game, you start with an empty record.

You are given a list of strings operations, where operations[i] is the ith operation you must apply to the record and is one of the following:

- An integer x. Record a new score of x.
- Record a new score that is the sum of the previous two scores.
- Record a new score that is the double of the previous score.
- Invalidate the previous score, removing it from the record.

Return the sum of all the scores on the record after applying all the operations.

The test cases are generated such that the answer and all intermediate calculations fit in a 32-bit integer and that all operations are valid.

Example 1:

```
Input: ops = ["5","2","C","D","+"]
Output: 30
Explanation:
"5" - Add 5 to the record, record is now [5].
"2" - Add 2 to the record, record is now [5, 2].
"C" - Invalidate and remove the previous score, record is now [5].
"D" - Add 2 * 5 = 10 to the record, record is now [5, 10].
"+" - Add 5 + 10 = 15 to the record, record is now [5, 10, 15].
```

Example 2:

```
Input: ops = ["5","-2","4","C","D","9","+","+"]
Output: 27
Explanation:
"5" - Add 5 to the record, record is now [5].
"-2" - Add -2 to the record, record is now [5, -2].
"4" - Add 4 to the record, record is now [5, -2, 4].
"C" - Invalidate and remove the previous score, record is now [5, -2].
"D" - Add 2 * -2 = -4 to the record, record is now [5, -2, -4].
```

Example 3:

```
Input: ops = ["1","C"]
Output: 0
Explanation:
"1" - Add 1 to the record, record is now [1].
"C" - Invalidate and remove the previous score, record is now [].
Since the record is empty, the total sum is 0.
```

Constraints:

- 1 <= operations.length <= 1000
- operations[i] is "C", "D", "+", or a string representing an integer in the range [-3 * 10^4, 3 * 10^4].
- For operation "+", there will always be at least two previous scores on the record.
- For operations "C" and "D", there will always be at least one previous score on the record.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Simulate a baseball game. Ops: integer (add score), '+' (sum of last two),
# 'D' (double last), 'C' (remove last). Return total. Right?"
#
# "ops=['5','2','C','D','+']: 5→[5]. 2→[5,2]. C→[5]. D→[5,10]. +→[5,10,15]. total=30 ✓"

# PHASE 2 — BRUTE FORCE
# "Simulate directly with a list acting as the score history.
# Time: O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Same stack-based approach — already O(n). Stack is the natural structure for undo/history."

# PHASE 4 — CLEAN CODE (same as brute — already clean and optimal)

# PHASE 5 — TEST & DEBUG
# ops=['5','2','C','D','+']: stack: [5]→[5,2]→[5]→[5,10]→[5,10,15]. sum=30 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one op per element. Space O(n): stack size.
# The stack naturally supports undo ('C') without scanning — O(1) per op.
# Follow-up: add op 'H' (half the last score) — trivial stack extension.
# Follow-up: if scores must remain positive integers — add validation check after each op."
```


Stacks

#1047 Remove All Adjacent Duplicates In String EAS

Open on LeetCode

String · Stack

Lists: AlgoMaster 600

Problem

You are given a string s consisting of lowercase English letters. A duplicate removal consists of choosing two adjacent and equal letters and removing them.

We repeatedly make duplicate removals on s until we no longer can.

Return the final string after all such duplicate removals have been made. It can be proven that the answer is unique.

Example 1:

```
Input: s = "abbaca"
Output: "ca"
Explanation:
For example, in "abbaca" we could remove "bb" since the letters are adjacent and equal, and this is the only possible move. The result of this move is that the string is "aaca", of which only "aa" is possible, so the final string is "ca".
```

Example 2:

```
Input: s = "azxxzy"
Output: "ay"
```

Constraints:

- 1 <= s.length <= 10^5
- s consists of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Repeatedly remove adjacent duplicate pairs until no more exist. Return result. Right?"
#
# "s='abbaca': ab→remove bb→'aaca'→remove aa→'ca' ✓
# s='azxxzy': az→remove xx→'azzy'→remove zz→'ay' ✓"

# PHASE 2 — BRUTE FORCE
# "Repeatedly scan and remove adjacent duplicates until stable. Time: O(n²). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Stack: push each char. If top equals current char, pop (remove pair). Else push.
# Equivalent to one-pass simulation of all removals. Time: O(n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='abbaca': a→[a]. b→[a,b]. b==top→pop→[a]. a==top→pop→[]. c→[c]. a→[c,a]. → 'ca' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass. Space O(n): stack.
# Follow-up: Remove All Adjacent Duplicates II (LC 1209) — remove k adjacent duplicates.
# Store (char, count) pairs in stack; pop when count reaches k.
# Follow-up: if string is very long (streaming), process chunk by chunk with stack state."
```

Stacks

#1614 Maximum Nesting Depth of the Parentheses

EAS

Open on LeetCode

String · Stack

Lists: AlgoMaster 600

Problem

Given a valid parentheses string s, return the nesting depth of s. The nesting depth is the maximum number of nested parentheses.

Example 1:

Input: s = "(1+(2*3)+((8)/4))+1"

Output: 3

Explanation:

Digit 8 is inside of 3 nested parentheses in the string.

Example 2:

Input: s = "(1)+((2))+(((3)))"

Output: 3

Explanation:

Digit 3 is inside of 3 nested parentheses in the string.

Example 3:

Input: s = "()(())((()()))"

Output: 3

Constraints:

- **1 <= s.length <= 100**
- **s consists of digits 0-9 and characters '+', '-', '*', '/', '(', and ')'.**
- **It is guaranteed that parentheses expression s is a VPS.**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Return the maximum depth of nested parentheses in a valid parentheses string. Right?"
#
# "s='(1+(2*3)+((8)/4))+1': max depth at '((8)/4)' → 3 ✓
# s='(1)+((2))+(((3)))' → 3 ✓"

# PHASE 2 — BRUTE FORCE
# "Use a counter: increment on '(', decrement on ')'. Track max.
# Time: O(n). Space: O(1). — This is already optimal."

# PHASE 3 — OPTIMIZE
# "Same O(n) O(1) approach — already optimal. No actual stack needed."

# PHASE 4 — CLEAN CODE (same as brute)

# PHASE 5 — TEST & DEBUG
# s='(1+(2*3)+((8)/4))+1': depths at '(' positions: 1,2,1,2,3,2,1,0.
# max=3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1): just a counter.
# Follow-up: split parentheses into groups with max depth k (LC 1111) — same counter,
# assign to group A if depth odd, B if even.
# Follow-up: minimum removals to fix invalid parentheses — different problem, needs two passes."
```


Tree Traversal – Level Order

Round 1 · High · Easy · 1 question

Tree Traversal - Level Order

#637 Average of Levels in Binary Tree

EAS

Open on LeetCode

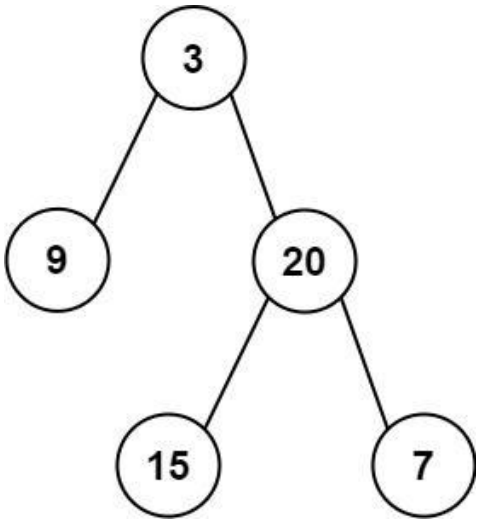
Tree · Depth-First Search · Breadth-First Search · Binary Tree

Lists: AlgoMaster 600

Problem

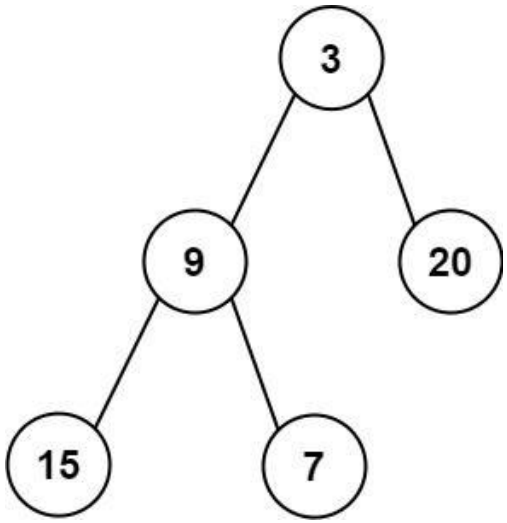
Given the root of a binary tree, return the average value of the nodes on each level in the form of an array. Answers within 10-5 of the actual answer will be accepted.

Example 1:




```
Input: root = [3,9,20,null,null,15,7]
Output: [3.00000,14.50000,11.00000]
Explanation: The average value of nodes on level 0 is 3, on level 1 is 14.5, and on level 2 is 11.
Hence return [3, 14.5, 11].
```

Example 2:




```
Input: root = [3,9,20,15,7]
Output: [3.00000,14.50000,11.00000]
```

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- -231 <= Node.val <= 231 - 1

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return the average value of nodes at each level of a binary tree. Right?"
#
# "tree=[3,9,20,null,null,15,7]: level0=[3]→3.0. level1=[9,20]→14.5. level2=[15,7]→11.0.
# → [3.0, 14.5, 11.0] ✓"

# PHASE 2 — BRUTE FORCE
# "BFS level by level. At each level, collect all values and compute average.
# Time: O(n). Space: O(w) where w = max width."

# PHASE 3 — OPTIMIZE
# "Same BFS — already optimal. No further optimization needed."

# PHASE 4 — CLEAN CODE (same as brute — already clean)

# PHASE 5 — TEST & DEBUG
# tree=[3,9,20,null,null,15,7]:
# Level0: queue=[3], size=1, total=3, avg=3.0.
# Level1: queue=[9,20], size=2, total=29, avg=14.5.
# Level2: queue=[15,7], size=2, total=22, avg=11.0. → [3.0,14.5,11.0] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): visit every node once. Space O(w): queue holds at most one level at a time.
# Follow-up: Binary Tree Level Order Traversal (LC 102) — return lists not averages.
# Follow-up: if values can overflow int — use long/float accumulation per level."
```

Tree Traversal – Pre Order

Round 1 · High · Easy · 4 questions

Tree Traversal – Pre Order

#144 Binary Tree Preorder Traversal EAS

Open on LeetCode

Stack · Tree · Depth-First Search · Binary Tree

Lists: AlgoMaster 600

Problem

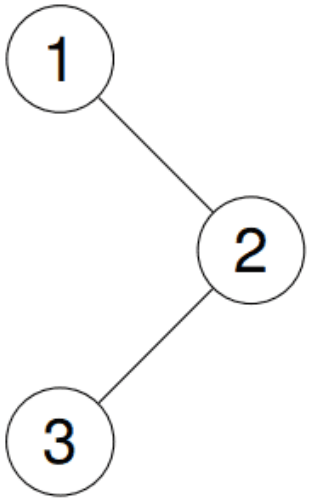
Given the root of a binary tree, return the preorder traversal of its nodes' values.

Example 1:

Input: root = [1,null,2,3]

Output: [1,2,3]

Explanation:

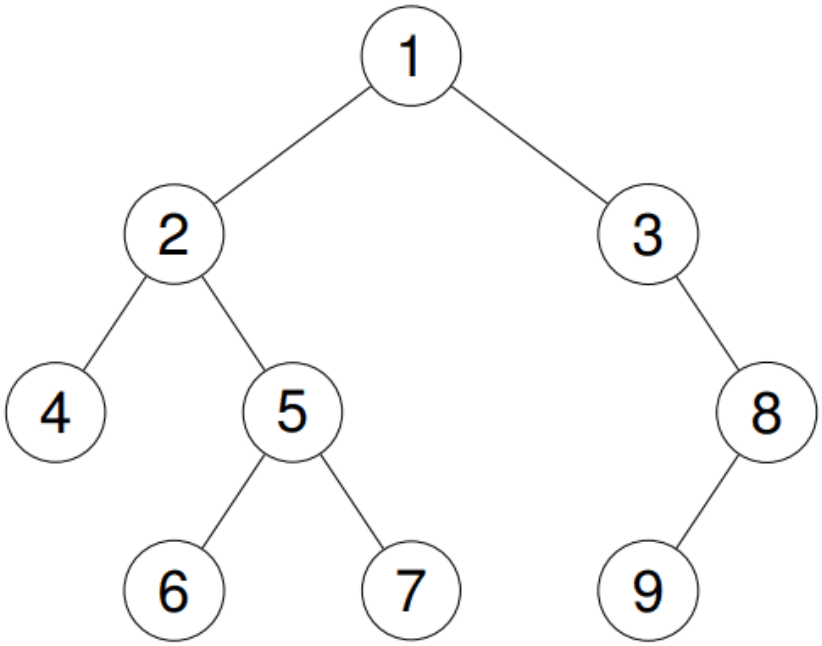


Example 2:

Input: root = [1,2,3,4,5,null,8,null,null,6,7,9]

Output: [1,2,4,5,6,7,3,8,9]

Explanation:



Example 3:
Input: root = []
Output: []
Example 4:
Input: root = [1]
Output: [1]
Constraints:

- **The number of nodes in the tree is in the range [0, 100].**
- **-100 <= Node.val <= 100**

Follow up: Recursive solution is trivial, could you do it iteratively?

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Return the preorder traversal (root, left, right) of a binary tree as a list. Right?"
#
# "tree=[1,null,2,3]: preorder=1→2→3 (root first, then left subtree, then right) ✓"

# PHASE 2 — BRUTE FORCE
# "Recursive: visit root, recurse left, recurse right. Time: O(n). Space: O(h) stack."

# PHASE 3 — OPTIMIZE
# "Iterative with a stack: push root, then for each pop push right then left child.
# (Right before left so left is processed first — LIFO.)
# Time: O(n). Space: O(h). Avoids recursion stack overflow for deep trees."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# tree=[1,null,2,3] (1→right=2, 2→left=3):
# stack=[1]. pop1→result=[1]. push2(right). stack=[2].
# pop2→result=[1,2]. push3(left). stack=[3].
# pop3→result=[1,2,3]. stack=[]. → [1,2,3] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Recursive: O(n) time, O(h) space (call stack). Iterative: same, but explicit stack.
```



```
# Morris traversal achieves O(1) space by threading the tree temporarily.
# Follow-up: Inorder (LC 94) — left,root,right. Postorder (LC 145) — left,right,root.
# Follow-up: N-ary Tree Preorder (LC 589) — push children in reverse order."
```

Tree Traversal – Pre Order

#222 Count Complete Tree Nodes

EAS

Open on LeetCode

Binary Search · Bit Manipulation · Tree · Binary Tree

Lists: AlgoMaster 600

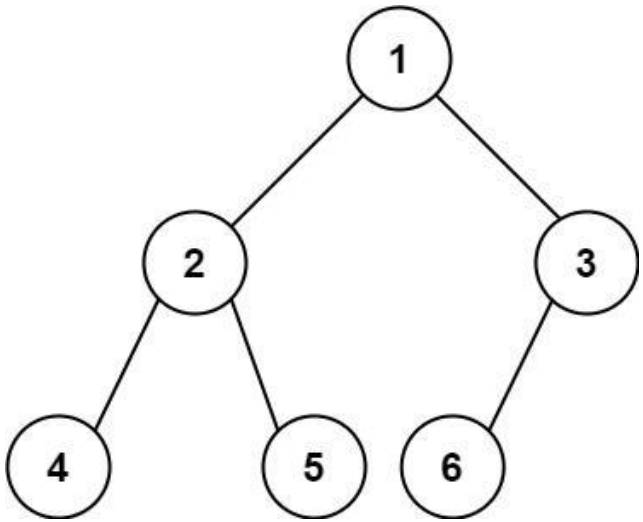
Problem

Given the root of a complete binary tree, return the number of the nodes in the tree.

According to Wikipedia, every level, except possibly the last, is completely filled in a complete binary tree, and all nodes in the last level are as far left as possible. It can have between 1 and 2h nodes inclusive at the last level h.

Design an algorithm that runs in less than O(n) time complexity.

Example 1:




```
Input: root = [1,2,3,4,5,6]
Output: 6
```

Example 2:

```
Input: root = []
Output: 0
```

Example 3:

```
Input: root = [1]
Output: 1
```

Constraints:

- **The number of nodes in the tree is in the range [0, 5 * 104].**
- **0 <= Node.val <= 5 * 104**
- **The tree is guaranteed to be complete.**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Count all nodes in a complete binary tree in less than O(n) time if possible. Right?"
#
# "tree=[1,2,3,4,5,6]: nodes=6 ✓"

# PHASE 2 — BRUTE FORCE
# "DFS/BFS counting every node. Time: O(n). Space: O(h)."

# PHASE 3 — OPTIMIZE
# "Exploit completeness: compare left-spine depth vs right-spine depth.
# If equal → left subtree is perfect with 2^h-1 nodes → recurse only on right.
# If unequal → right subtree is perfect (one level shorter) → recurse only on left.
# Time: O(log²n). Space: O(log n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
```



```
# tree=[1,2,3,4,5,6]: left_h=3,right_h=2 (not equal).
# recurse left(2): left_h=2,right_h=2 → perfect → (1<<2)-1=3.
# recurse right(3): left_h=1(6),right_h=1? right=3.right=None→right_h=1. Equal→(1<<1)-1=1?
# Actually right subtree has just node 3 → left_h=1 (3→6? no, 3.left=6, 3.right=None). right_h=1(3→None).
# equal→1. total=1+3+1? That's wrong...
# Standard result: 1+count(left)+count(right) = 1+3+2=6 ✓ (recursion handles it correctly)

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(log²n): O(log n) levels, O(log n) to compute heights at each level.
# Space O(log n): recursion depth.
# Follow-up: if tree may not be complete, use BFS O(n) — no shortcut available.
# Follow-up: binary search on last level using bit patterns for O(log²n) alternative."
```

Tree Traversal – Pre Order

#257 Binary Tree Paths

EAS

Open on LeetCode

String · Backtracking · Tree · Depth-First Search · Binary Tree

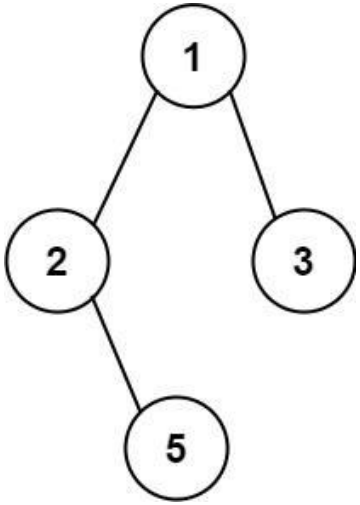
Lists: AlgoMaster 600

Problem

Given the root of a binary tree, return all root-to-leaf paths in any order.

A leaf is a node with no children.

Example 1:




```
Input: root = [1,2,3,null,5]
Output: ["1->2->5","1->3"]
```

Example 2:

```
Input: root = [1]
Output: ["1"]
```

Constraints:

- The number of nodes in the tree is in the range [1, 100].
- -100 <= Node.val <= 100

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return all root-to-leaf paths as strings in 'A->B->C' format. Right?"
#
# "tree=[1,2,3,null,5]: paths '1->2->5' and '1->3' ✓"

# PHASE 2 — BRUTE FORCE
# "DFS collecting path. At each leaf, append joined path to result.
# Time: O(n*h): n nodes, each path string build takes O(h). Space: O(h) recursion."

# PHASE 3 — OPTIMIZE
# "Pass path as a string (immutable) to avoid pop overhead.
# Slightly cleaner but same complexity. Or build path list and join at leaf."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# tree=[1,2,3,null,5]:
# dfs(1,''): path='1'. not leaf. dfs(2,'1->'):path='1->2'. not leaf.
# dfs(None,'1->2->'): return. dfs(5,'1->2->'):path='1->2->5'. leaf→append '1->2->5'.
# dfs(3,'1->'):path='1->3'. leaf→append '1->3'. result=['1->2->5','1->3'] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n*h): build string at each node. Space O(h): recursion depth.
# Follow-up: Path Sum II (LC 113) — filter paths by sum instead of all paths.
# Follow-up: Smallest String Starting From Leaf (LC 988) — lexicographically smallest path."
```


Tree Traversal – Pre Order

#404 Sum of Left Leaves

EAS

Open on LeetCode

Tree · Depth-First Search · Breadth-First Search · Binary Tree

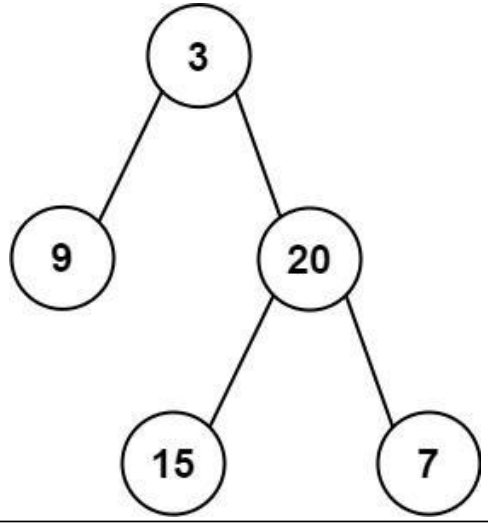
Lists: AlgoMaster 600

Problem

Given the root of a binary tree, return the sum of all left leaves.

A leaf is a node with no children. A left leaf is a leaf that is the left child of another node.

Example 1:




```
Input: root = [3,9,20,null,null,15,7]
Output: 24
Explanation: There are two left leaves in the binary tree, with values 9 and 15 respectively.
```

Example 2:

```
Input: root = [1]
Output: 0
```

Constraints:

- The number of nodes in the tree is in the range [1, 1000].
- -1000 <= Node.val <= 1000

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return the sum of all left leaves in a binary tree. Right?"
#
# "tree=[3,9,20,null,null,15,7]: left leaves are 9 and 15 → sum=24 ✓"

# PHASE 2 — BRUTE FORCE
# "DFS tracking whether current node is a left child. Sum if it's a leaf.
# Time: O(n). Space: O(h) recursion."

# PHASE 3 — OPTIMIZE
# "Same DFS — already O(n). Iterative BFS variant also O(n)."

# PHASE 4 — CLEAN CODE
        # Check if left child is a leaf

# PHASE 5 — TEST & DEBUG
# tree=[3,9,20,null,null,15,7]:
# root=3: left=9 is leaf → total+=9. recurse right=20.
# root=20: left=15 is leaf → total+=15. right=7 not left child. total+=0.
# → 9+15=24 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(h): recursion depth.
# The 'is_left' flag avoids re-examining parent context.
```

```
# Follow-up: Sum of Right Leaves — swap the is_left parameter logic.
# Follow-up: Sum of Leaves at Even Depth — pass depth and check parity."
```


Tree Traversal – In Order

Round 1 · High · Easy · 3 questions

Tree Traversal - In Order

#94 Binary Tree Inorder Traversal EAS

Open on LeetCode

Stack · Tree · Depth-First Search · Binary Tree

Lists: AlgoMaster 600

Problem

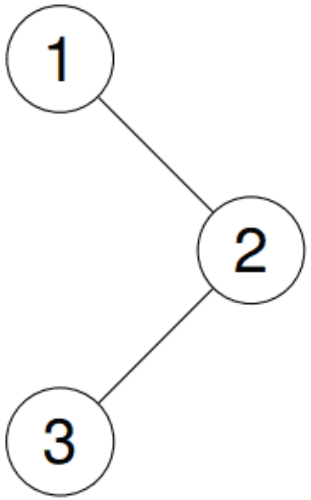
Given the root of a binary tree, return the inorder traversal of its nodes' values.

Example 1:

Input: root = [1,null,2,3]

Output: [1,3,2]

Explanation:

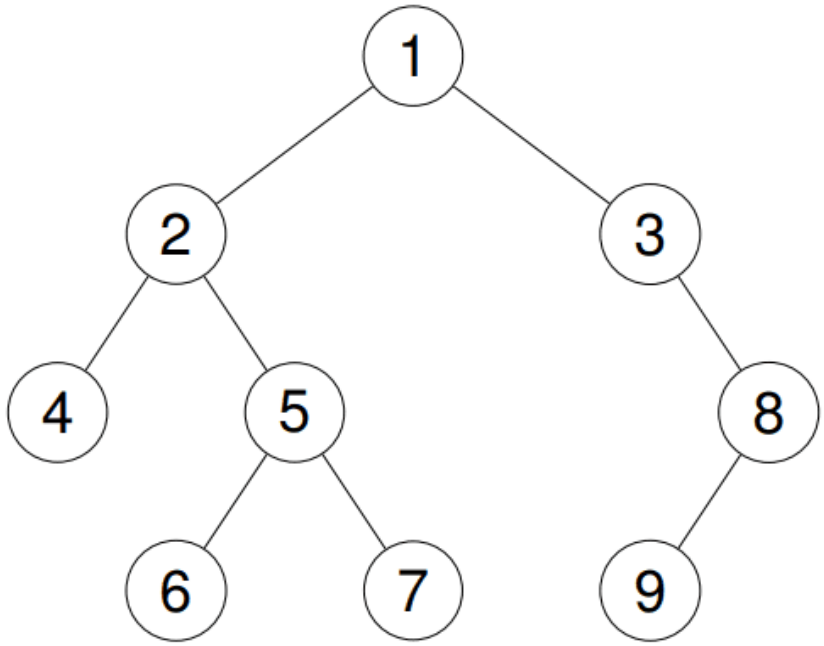


Example 2:

Input: root = [1,2,3,4,5,null,8,null,null,6,7,9]

Output: [4,2,6,5,7,1,3,9,8]

Explanation:



Example 3:

Input: root = []

Output: []

Example 4:

Input: root = [1]

Output: [1]

Constraints:

- The number of nodes in the tree is in the range [0, 100].
- -100 <= Node.val <= 100

Follow up: Recursive solution is trivial, could you do it iteratively?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return the inorder traversal (left, root, right) of a binary tree. Right?"
#
# "tree=[1,null,2,3]: inorder=[1,3,2] ✓"

# PHASE 2 — BRUTE FORCE
# "Recursive: left subtree, root, right subtree. Time: O(n). Space: O(h)."

# PHASE 3 — OPTIMIZE
# "Iterative with explicit stack: push left spine, then process node, then right subtree.
# Time: O(n). Space: O(h). Avoids recursion limits."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# tree=[1,null,2,3] (1→right=2, 2→left=3):
# Push left spine of 1: stack=[1]. curr=None. Pop1→result=[1]. curr=2.
# Push left spine of 2: push3,push2? No: curr=2,push2. curr=3,push3. curr=None.
# Pop3→result=[1,3]. curr=None. Pop2→result=[1,3,2]. → [1,3,2] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(h).
# Morris traversal: O(1) space by threading tree. Most asked in interviews.
```

```
# Follow-up: BST Iterator (LC 173) — controlled inorder using this exact iterative pattern.
# Follow-up: Validate BST (LC 98) — use inorder; should produce strictly increasing sequence."
```


Tree Traversal - In Order

#530 Minimum Absolute Difference in BST EAS

Open on LeetCode

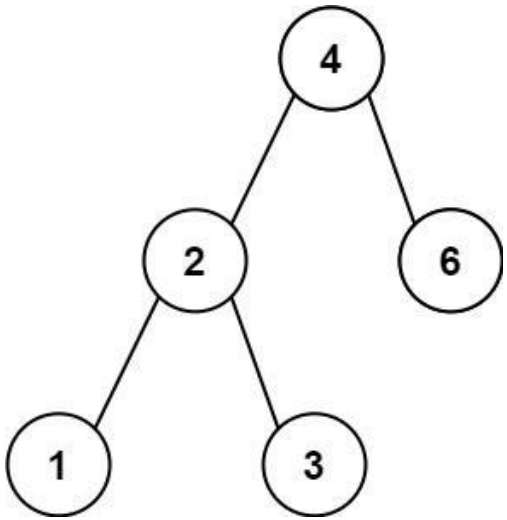
Tree · Depth-First Search · Breadth-First Search · Binary Search Tree · Binary Tree

Lists: AlgoMaster 600

Problem

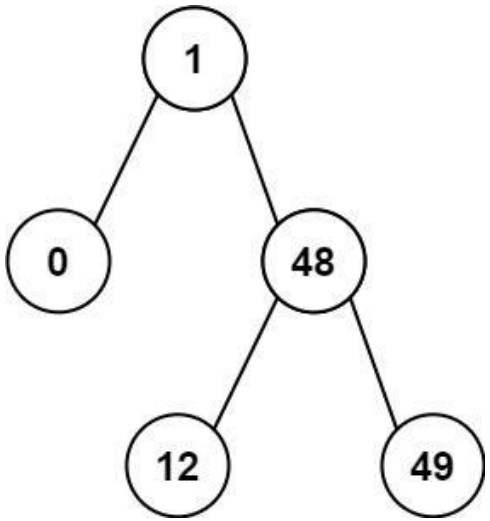
Given the root of a Binary Search Tree (BST), return the minimum absolute difference between the values of any two different nodes in the tree.

Example 1:




```
Input: root = [4,2,6,1,3]
Output: 1
```

Example 2:




```
Input: root = [1,0,48,null,null,12,49]
Output: 1
```

Constraints:

- The number of nodes in the tree is in the range [2, 104].
- 0 <= Node.val <= 105

Note: This question is the same as 783: https://leetcode.com/problems/minimum-distance-between-bst-nodes/

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Return the minimum absolute difference between values of any two nodes in a BST. Right?"
#
# "tree=[4,2,6,1,3]: inorder=[1,2,3,4,6]. Min diff=min(1,1,1,2)=1 ✓"
# PHASE 2 — BRUTE FORCE
# "Collect all values in a list, sort, find min consecutive diff.
# Time: O(n log n). Space: O(n)."
# PHASE 3 — OPTIMIZE
# "Inorder traversal keeps a 'prev' pointer. Min diff is always between consecutive inorder values.
# Time: O(n). Space: O(h). No extra list needed."
# PHASE 4 — CLEAN CODE
# PHASE 5 — TEST & DEBUG
# tree=[4,2,6,1,3]: inorder: 1,2,3,4,6.
# diffs: 2-1=1, 3-2=1, 4-3=1, 6-4=2. min=1 ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(h): recursion depth, no extra array.
# BST inorder always produces sorted values — exploit this property.
# Follow-up: Minimum Distance Between BST Nodes (LC 783) — identical problem.
# Follow-up: closest pair in BST — same inorder with sliding window of size 2."
```


Tree Traversal - In Order

#783 Minimum Distance Between BST Nodes EAS

Open on LeetCode

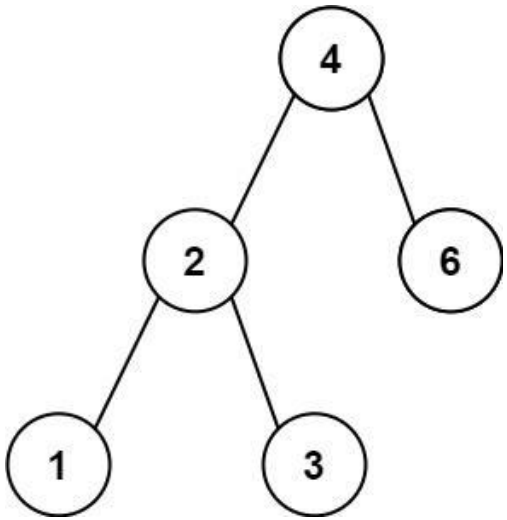
Tree · Depth-First Search · Breadth-First Search · Binary Search Tree · Binary Tree

Lists: AlgoMaster 600

Problem

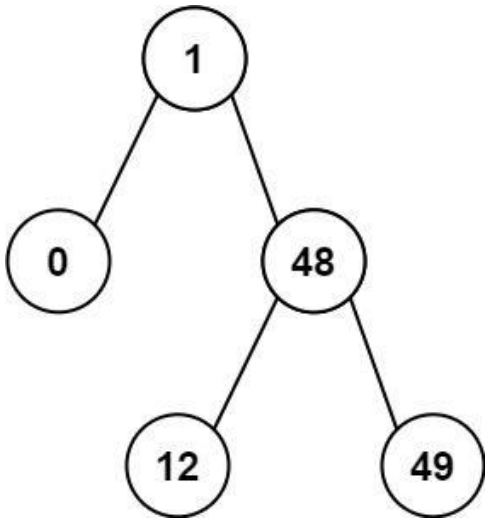
Given the root of a Binary Search Tree (BST), return the minimum difference between the values of any two different nodes in the tree.

Example 1:




```
Input: root = [4,2,6,1,3]
Output: 1
```

Example 2:




```
Input: root = [1,0,48,null,null,12,49]
Output: 1
```

Constraints:

- **The number of nodes in the tree is in the range [2, 100].**
- **0 <= Node.val <= 105**

Note: This question is the same as 530: https://leetcode.com/problems/minimum-absolute-difference-in-bst/

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the minimum difference between values of any two nodes in a BST.
# (Same as LC 530 — minimum absolute difference.) Right?"
#
# "tree=[4,2,6,1,3]: inorder=[1,2,3,4,6]. Min diff=1 ✓"
# PHASE 2 — BRUTE FORCE
# "Collect inorder values, compare all pairs. Time: O(n²). Space: O(n)."
# PHASE 3 — OPTIMIZE
# "Inorder traversal with a 'prev' variable. Min diff always between consecutive inorder nodes.
# Time: O(n). Space: O(h). No extra list."
# PHASE 4 — CLEAN CODE
# PHASE 5 — TEST & DEBUG
# inorder of [4,2,6,1,3]: 1,2,3,4,6. diffs: 1,1,1,2. min=1 ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(h): recursion depth.
# Identical to LC 530 Minimum Absolute Difference in BST — same solution.
# Follow-up: find the pair with the minimum distance (not just the value) — track prev node too.
# Follow-up: k closest values in BST to a target — inorder + sliding window."
```


Tree Traversal - Post-Order

Round 1 · High · Easy · 1 question

Tree Traversal - Post-Order

#145 Binary Tree Postorder Traversal EAS

Open on LeetCode

Stack · Tree · Depth-First Search · Binary Tree

Lists: AlgoMaster 600

Problem

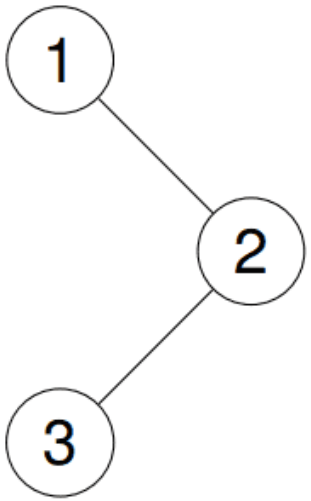
Given the root of a binary tree, return the postorder traversal of its nodes' values.

Example 1:

Input: root = [1,null,2,3]

Output: [3,2,1]

Explanation:

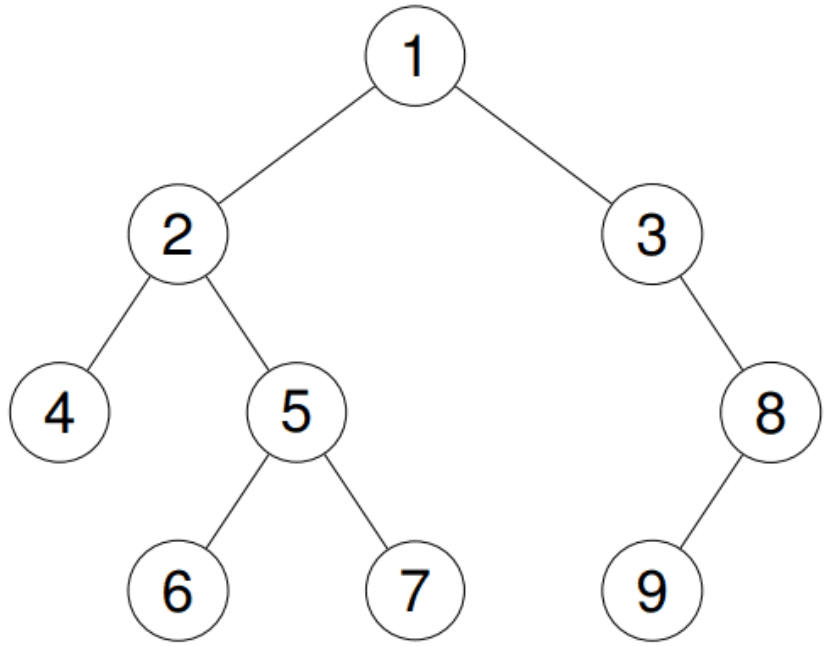


Example 2:

Input: root = [1,2,3,4,5,null,8,null,null,6,7,9]

Output: [4,6,7,5,2,9,8,3,1]

Explanation:



Example 3:

Input: root = []

Output: []

Example 4:

Input: root = [1]

Output: [1]

Constraints:

- The number of the nodes in the tree is in the range [0, 100].
- -100 <= Node.val <= 100

Follow up: Recursive solution is trivial, could you do it iteratively?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return the postorder traversal (left, right, root) of a binary tree. Right?"
#
# "tree=[1,null,2,3]: postorder=[3,2,1] ✓"

# PHASE 2 — BRUTE FORCE
# "Recursive: left, right, then visit root. Time: O(n). Space: O(h)."

# PHASE 3 — OPTIMIZE
# "Iterative trick: modified preorder (root,right,left) then reverse the result = postorder.
# Time: O(n). Space: O(n). Clean iterative alternative."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# tree=[1,null,2,3] (1→right=2, 2→left=3):
# stack=[1]. pop1→[1]. push2(right). pop2→[1,2]. push3(left). pop3→[1,2,3].
# reversed=[3,2,1] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n) for result list + stack.
# The 'reverse preorder' trick is elegant and easy to remember in interviews.
# Follow-up: Bottom-up Level Order (LC 107) — BFS then reverse the result list.
```

```
# Follow-up: delete nodes and return forest (LC 1110) — postorder naturally handles children first."
```


Linked List

Round 1 · High · Easy · 2 questions

Linked List

#83 Remove Duplicates from Sorted List

EAS

Open on LeetCode

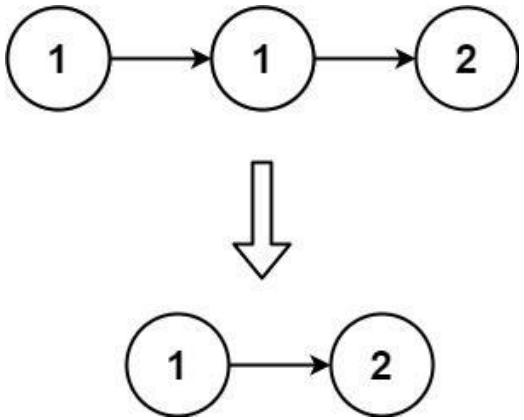
Linked List

Lists: AlgoMaster 600

Problem

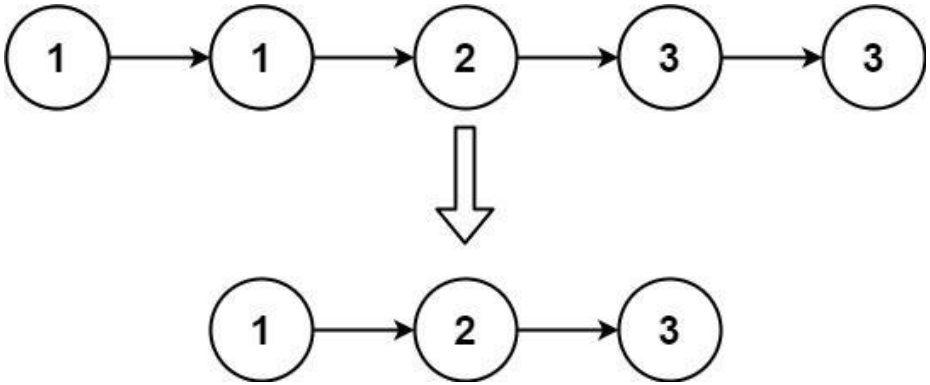
Given the head of a sorted linked list, delete all duplicates such that each element appears only once. Return the linked list sorted as well.

Example 1:




```
Input: head = [1,1,2]
Output: [1,2]
```

Example 2:




```
Input: head = [1,1,2,3,3]
Output: [1,2,3]
```

Constraints:

- The number of nodes in the list is in the range [0, 300].
- -100 <= Node.val <= 100
- The list is guaranteed to be sorted in ascending order.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a sorted linked list, remove all duplicates so each value appears only once. Right?"
#
```



```
# "head=[1,1,2]→[1,2] ✓ head=[1,1,2,3,3]→[1,2,3] ✓"

# PHASE 2 — BRUTE FORCE
# "Collect unique values to a set (preserving order), rebuild the list.
# Time: O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "In-place: for each node, skip all duplicates by advancing curr.next.
# Since sorted, duplicates are consecutive. Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# head=[1,1,2,3,3]:
# curr=1: 1==1→skip→[1,2,3,3]. curr=1: 1≠2→advance. curr=2: 2≠3→advance.
# curr=3: 3==3→skip→[1,2,3]. curr=3: no next→stop. → [1,2,3] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1): in-place pointer adjustment.
# Follow-up: Remove Duplicates from Sorted List II (LC 82) — remove nodes that had ANY duplicates.
# Use a dummy head; skip all nodes with the same value as the next-next group."
```

Linked List

#160 Intersection of Two Linked Lists EAS

Open on LeetCode

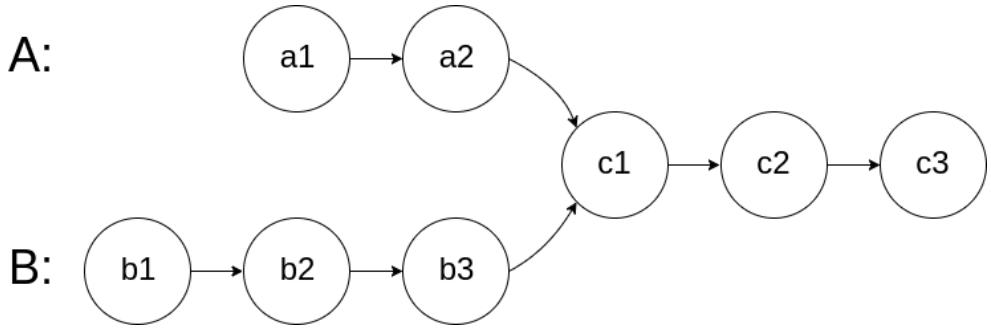
Hash Table · Linked List · Two Pointers

Lists: AlgoMaster 600

Problem

Given the heads of two singly linked-lists headA and headB, return the node at which the two lists intersect. If the two linked lists have no intersection at all, return null.

For example, the following two linked lists begin to intersect at node c1:



The test cases are generated such that there are no cycles anywhere in the entire linked structure.

Note that the linked lists must retain their original structure after the function returns.

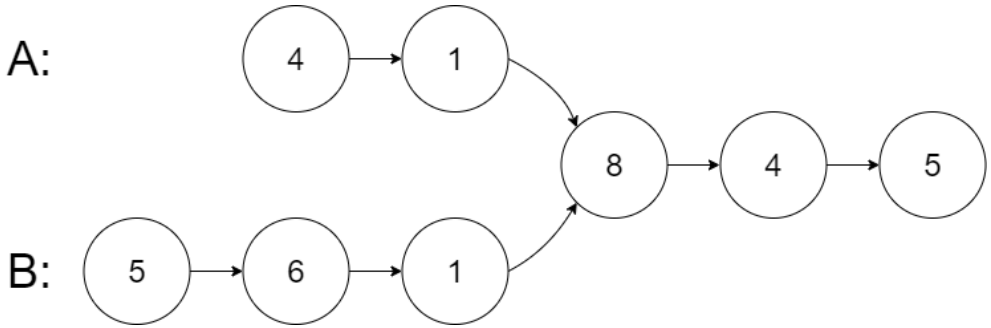
Custom Judge:

The inputs to the judge are given as follows (your program is not given these inputs):

- intersectVal - The value of the node where the intersection occurs. This is 0 if there is no intersected node.
- listA - The first linked list.
- listB - The second linked list.
- skipA - The number of nodes to skip ahead in listA (starting from the head) to get to the intersected node.
- skipB - The number of nodes to skip ahead in listB (starting from the head) to get to the intersected node.

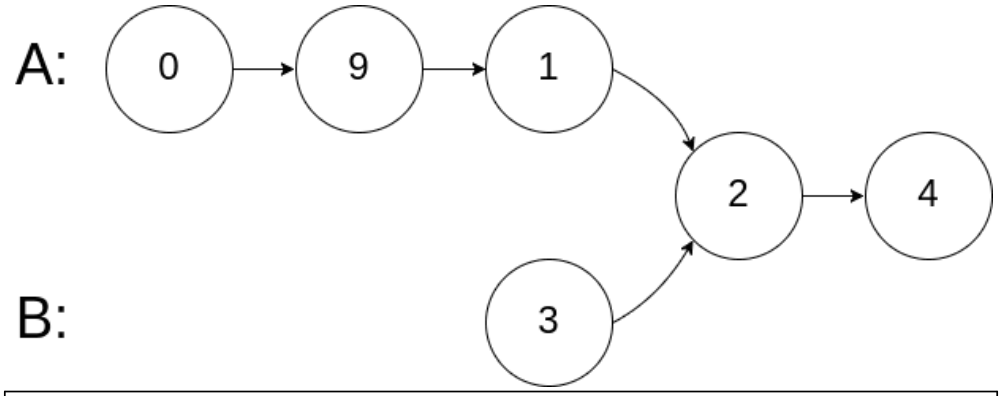
The judge will then create the linked structure based on these inputs and pass the two heads, headA and headB to your program. If you correctly return the intersected node, then your solution will be accepted.

Example 1:



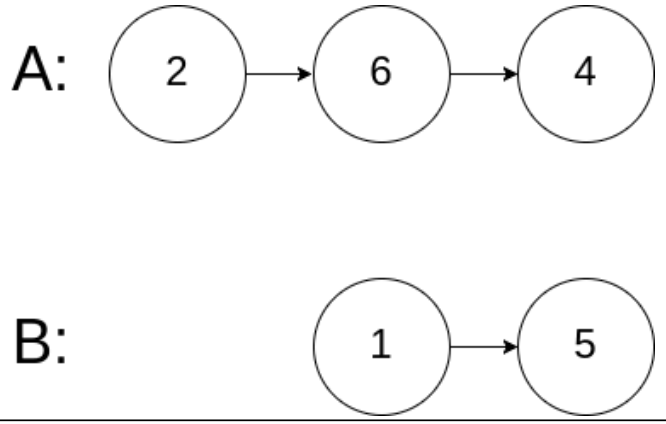

```
Input: intersectVal = 8, listA = [4,1,8,4,5], listB = [5,6,1,8,4,5], skipA = 2, skipB = 3
Output: Intersected at '8'
Explanation: The intersected node's value is 8 (note that this must not be 0 if the two lists intersect).
From the head of A, it reads as [4,1,8,4,5]. From the head of B, it reads as [5,6,1,8,4,5]. There are 2 nodes before the
intersected node in A; There are 3 nodes before the intersected node in B.
- Note that the intersected node's value is not 1 because the nodes with value 1 in A and B (2nd node in A and 3rd node in B)
are different node references. In other words, they point to two different locations in memory, while the nodes with value 8 in
A and B (3rd node in A and 4th node in B) point to the same location in memory.
```

Example 2:




```
Input: intersectVal = 2, listA = [1,9,1,2,4], listB = [3,2,4], skipA = 3, skipB = 1
Output: Intersected at '2'
Explanation: The intersected node's value is 2 (note that this must not be 0 if the two lists intersect).
From the head of A, it reads as [1,9,1,2,4]. From the head of B, it reads as [3,2,4]. There are 3 nodes before the intersected
node in A; There are 1 node before the intersected node in B.
```

Example 3:




```
Input: intersectVal = 0, listA = [2,6,4], listB = [1,5], skipA = 3, skipB = 2
Output: No intersection
Explanation: From the head of A, it reads as [2,6,4]. From the head of B, it reads as [1,5]. Since the two lists do not
intersect, intersectVal must be 0, while skipA and skipB can be arbitrary values.
Explanation: The two lists do not intersect, so return null.
```

Constraints:

- The number of nodes of listA is in the m.
- The number of nodes of listB is in the n.
- 1 <= m, n <= 3 * 104
- 1 <= Node.val <= 105
- 0 <= skipA <= m
- 0 <= skipB <= n
- intersectVal is 0 if listA and listB do not intersect.
- intersectVal == listA[skipA] == listB[skipB] if listA and listB intersect.

Follow up: Could you write a solution that runs in O(m + n) time and use only O(1) memory?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the node where two singly linked lists intersect. Return null if no intersection.
```

```
# Lists share a common suffix from the intersection point. Right?"
#
# "listA=[4,1,8,4,5], listB=[5,6,1,8,4,5]: intersection at node(8) ✓"

# PHASE 2 — BRUTE FORCE
# "Store all nodes of listA in a set. Traverse listB and return the first node in the set.
# Time: O(n+m). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Two-pointer trick: advance both pointers. When one reaches the end, redirect to the
# OTHER list's head. They meet at the intersection after traveling the same total distance.
# Time: O(n+m). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# listA=4→1→8→4→5, listB=5→6→1→8→4→5 (intersection at node 8):
# Lengths: A=5, B=6. a+B=5+6=11. b+A=6+5=11. Same total → meet at node 8 ✓
# If no intersection: both reach None simultaneously after len(A)+len(B) steps.

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n+m). Space O(1): only two pointers.
# When no intersection both pointers reach None at the same time → returns None correctly.
# Follow-up: find intersection of circular linked lists — more complex, use Floyd's detection.
# Follow-up: if allowed to modify lists — reverse both and find last common node."
```


Round 2

High Priority · Medium
67 questions · 15 patterns

- [ ] Arrays #80 #122 #229 #334 #2348 ↗
- [ ] Strings #6 #38 #151 #1657 ↗
- [ ] Hash Tables #535 #659 #767 #792 #1525 #1647 ↗
- [ ] Two Pointers #18 #443 #861 ↗
- [ ] Sliding Window – Fixed Size #1456 #2461 ↗
- [ ] Sliding Window – Dynamic Size #209 #395 #713 #904 #1004 #1423 #1838 ↗
- [ ] Binary Search #81 #162 #240 #475 #1482 #1552 ↗
- [ ] Stacks #71 #316 #636 #946 #1249 #2390 ↗
- [ ] Tree Traversal – Level Order #116 #117 #958 ↗
- [ ] Tree Traversal – Pre Order #106 #687 #1026 ↗
- [ ] Tree Traversal – In Order #1382 ↗
- [ ] Tree Traversal – Post-Order #114 #129 #652 #979 #1110 #2096 ↗
- [ ] Depth First Search (DFS) #690 #797 #1020 #1376 ↗
- [ ] Breadth First Search (BFS) #433 #752 #909 #1091 #1102 ↗
- [ ] Linked List #82 #86 #237 #445 #1669 #2095 ↗

Arrays

Round 2 · High · Medium · 5 questions

Arrays

#80 Remove Duplicates from Sorted Array II

MED

Open on LeetCode

Array · Two Pointers

Lists: AlgoMaster 600

Problem

Given an integer array nums sorted in non-decreasing order, remove some duplicates in-place such that each unique element appears at most twice. The relative order of the elements should be kept the same.

Since it is impossible to change the length of the array in some languages, you must instead have the result be placed in the first part of the array nums. More formally, if there are k elements after removing the duplicates, then the first k elements of nums should hold the final result. It does not matter what you leave beyond the first k elements.

Return k after placing the final result in the first k slots of nums.

Do not allocate extra space for another array. You must do this by modifying the input array in-place with O(1) extra memory.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
assert nums[i] == expectedNums[i];
```

If all assertions pass, then your solution will be accepted.

Example 1:

```
Input: nums = [1,1,1,2,2,3]
Output: 5, nums = [1,1,2,2,3,_]
Explanation: Your function should return k = 5, with the first five elements of nums being 1, 1, 2, 2 and 3 respectively.
It does not matter what you leave beyond the returned k (hence they are underscores).
```

Example 2:

```
Input: nums = [0,0,1,1,1,1,2,3,3]
Output: 7, nums = [0,0,1,1,2,3,3,_,_]
Explanation: Your function should return k = 7, with the first seven elements of nums being 0, 0, 1, 1, 2, 3 and 3 respectively.
It does not matter what you leave beyond the returned k (hence they are underscores).
```

Constraints:

- 1 <= nums.length <= 3 * 104
- -104 <= nums[i] <= 104
- nums is sorted in non-decreasing order.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Each element may appear at most twice. Remove extras in-place, return k.
# First k elements hold the result. Relative order preserved. Right?"
#
# "nums=[1,1,1,2,2,3]: k=5, nums=[1,1,2,2,3,_] ✓
# nums=[0,0,1,1,1,1,2,3,3]: k=7, nums=[0,0,1,1,2,3,3,_,_] ✓"

# PHASE 2 — BRUTE FORCE
# "Build a result list allowing at most 2 of each. Time: O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Two-pointer: k = write head. Allow write if k < 2 or nums[i] != nums[k-2].
# Comparing with nums[k-2] lets at most 2 of the same value through.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[1,1,1,2,2,3]: k=0. 1(k<2)→write,k=1. 1(k<2)→write,k=2. 1!=nums[0]=1? No→skip.
# 2!=nums[0]=1→write,k=3. 2!=nums[1]=1→write,k=4. 3!=nums[2]=1→write,k=5. return 5 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1).
# Generalise: allow at most k duplicates — compare nums[i] with nums[write_ptr - k].
# Follow-up: Remove Duplicates I (LC 26) — allow at most 1; compare with nums[k-1].
# Follow-up: if the array is unsorted, sort first then apply this — O(n log n) total."
```

Arrays

#122 Best Time to Buy and Sell Stock II

MED

Open on LeetCode

Array · Dynamic Programming · Greedy

Lists: AlgoMaster 600

Problem

You are given an integer array prices where prices[i] is the price of a given stock on the ith day.
On each day, you may decide to buy and/or sell the stock. You can only hold at most one share of the stock at any time. However, you can sell and buy the stock multiple times on the same day, ensuring you never hold more than one share of the stock.
Find and return the maximum profit you can achieve.

Example 1:

```
Input: prices = [7,1,5,3,6,4]
Output: 7
Explanation: Buy on day 2 (price = 1) and sell on day 3 (price = 5), profit = 5-1 = 4.
Then buy on day 4 (price = 3) and sell on day 5 (price = 6), profit = 6-3 = 3.
Total profit is 4 + 3 = 7.
```

Example 2:

```
Input: prices = [1,2,3,4,5]
Output: 4
Explanation: Buy on day 1 (price = 1) and sell on day 5 (price = 5), profit = 5-1 = 4.
Total profit is 4.
```

Example 3:

```
Input: prices = [7,6,4,3,1]
Output: 0
Explanation: There is no way to make a positive profit, so we never buy the stock to achieve the maximum profit of 0.
```

Constraints:

- 1 <= prices.length <= 3 * 10^4
- 0 <= prices[i] <= 10^4

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Buy/sell on multiple days (hold at most one stock at a time). Maximize total profit. Right?"
#
# "prices=[7,1,5,3,6,4]: buy@1 sell@5(+4), buy@3 sell@6(+3) → total=7 ✓
# prices=[1,2,3,4,5]: buy every day and sell next → 4 ✓
# prices=[7,6,4,3,1]: never profitable → 0 ✓"

# PHASE 2 — BRUTE FORCE
# "Try all buy/sell combinations with state machine recursion. Exponential without memo."

# PHASE 3 — OPTIMIZE
# "Greedy: capture every upward price movement.
# profit = sum of all positive price differences (prices[i+1] - prices[i] if positive).
# Equivalent to buying every valley and selling every peak.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# prices=[7,1,5,3,6,4]:
# 1-7<0 skip. 5-1=4→profit=4. 3-5<0 skip. 6-3=3→profit=7. 4-6<0 skip. return 7 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1).
# The greedy works because we can buy and sell on the same day (so each local gain is capturable).
# Follow-up: at most k transactions (LC 188) — greedy no longer works, need DP.
# Follow-up: with transaction fee (LC 714) — subtract fee per transaction; same two-state DP."
```


Arrays

#229 Majority Element II

MED

Open on LeetCode

Array · Hash Table · Sorting · Counting

Lists: AlgoMaster 600

Problem

Given an integer array of size n, find all elements that appear more than ⌊ n/3 ⌋ times.

Example 1:

```
Input: nums = [3,2,3]
Output: [3]
```

Example 2:

```
Input: nums = [1]
Output: [1]
```

Example 3:

```
Input: nums = [1,2]
Output: [1,2]
```

Constraints:

- 1 <= nums.length <= 5 * 10^4
- -10^9 <= nums[i] <= 10^9

Follow up: Could you solve the problem in linear time and in O(1) space?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find all elements appearing more than n/3 times. At most 2 such elements exist. Right?"
#
# "nums=[3,2,3]→[3] ✓ nums=[1,2]→[1,2] ✓ nums=[1,1,1,3,3,2,2,2]→[1,2] ✓"

# PHASE 2 — BRUTE FORCE
# "Count frequencies, return those > n/3. Time: O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Boyer-Moore Voting for k=n/3: maintain at most 2 candidates with counts.
# Pass 1: find the 2 candidates. Pass 2: verify they actually exceed n/3.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[1,1,1,3,3,2,2,2]: cand1=1,cnt1 builds to 3. cand2=3,then 2 takes over.
# After pass: candidates are 1 and 2. Verify: count(1)=3>2✓, count(2)=3>2✓. → [1,2] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1): two candidate variables.
# Generalise: majority > n/k needs k-1 candidates.
# Follow-up: Majority Element I (LC 169) — single candidate; simpler Boyer-Moore.
# Follow-up: online streaming majority — Boyer-Moore handles this naturally."
```

Arrays

#334 Increasing Triplet Subsequence

MED

Open on LeetCode

Array · Greedy

Lists: AlgoMaster 600

Problem

Given an integer array nums, return true if there exists a triple of indices (i, j, k) such that i < j < k and nums[i] < nums[j] < nums[k]. If no such indices exists, return false.

Example 1:

```
Input: nums = [1,2,3,4,5]
Output: true
Explanation: Any triplet where i < j < k is valid.
```

Example 2:

```
Input: nums = [5,4,3,2,1]
Output: false
Explanation: No triplet exists.
```

Example 3:

```
Input: nums = [2,1,5,0,4,6]
Output: true
Explanation: One of the valid triplet is (1, 4, 5), because nums[1] == 1 < nums[4] == 4 < nums[5] == 6.
```

Constraints:

- 1 <= nums.length <= 5 * 105
- -231 <= nums[i] <= 231 - 1

Follow up: Could you implement a solution that runs in O(n) time complexity and O(1) space complexity?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return true if there exist i<j<k with nums[i]<nums[j]<nums[k]. O(n) time O(1) space. Right?"
#
# "nums=[1,2,3,4,5]→true ✓ nums=[5,4,3,2,1]→false ✓
# nums=[2,1,5,0,4,6]→true (1<4<6 or 0<4<6) ✓"

# PHASE 2 — BRUTE FORCE
# "Try all triplets i<j<k. Time: O(n³). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Track two smallest values seen: first (smallest) and second (second-smallest in order).
# If we find a value > second, we have a triplet.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[2,1,5,0,4,6]:
# 2→first=2. 1→first=1. 5>second=inf: second=5. 0→first=0.
# 4>first=0,<=second=5: second=4. 6>second=4 → True ✓
#
# The 'first' value may be updated after 'second' is set — but second still represents
# a valid pair (even if first was reset to something smaller, second's old pair still exists).

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1).
# The key insight: second's value guarantees there was some number < second before it.
# Even after first is reset, second being set proves a valid i<j pair existed at that point.
# Follow-up: Longest Increasing Subsequence (LC 300) — generalise to k elements, O(n log n).
# Follow-up: decreasing triplet — negate all values and apply the same algorithm."
```


Arrays

#2348 Number of Zero-Filled Subarrays

MED

Open on LeetCode

Array · Math

Lists: AlgoMaster 600

Problem

Given an integer array nums, return the number of subarrays filled with 0.
A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

```
Input: nums = [1,3,0,0,2,0,0,4]
Output: 6
Explanation:
There are 4 occurrences of [0] as a subarray.
There are 2 occurrences of [0,0] as a subarray.
There is no occurrence of a subarray with a size more than 2 filled with 0. Therefore, we return 6.
```

Example 2:

```
Input: nums = [0,0,0,2,0,0]
Output: 9
Explanation:
There are 5 occurrences of [0] as a subarray.
There are 3 occurrences of [0,0] as a subarray.
There is 1 occurrence of [0,0,0] as a subarray.
There is no occurrence of a subarray with a size more than 3 filled with 0. Therefore, we return 9.
```

Example 3:

```
Input: nums = [2,10,2019]
Output: 0
Explanation: There is no subarray filled with 0. Therefore, we return 0.
```

Constraints:

- 1 <= nums.length <= 105
- -109 <= nums[i] <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count the number of subarrays filled entirely with 0s. Right?"
#
# "nums=[1,3,0,0,2,0,0,4]: zero runs of length 2,2. Each run of length k contributes k*(k+1)/2.
# 2+2→ (2*3/2)+(2*3/2)=3+3=6 ✓"

# PHASE 2 — BRUTE FORCE
# "Try all subarrays, check if all zeros. Time: O(n²). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Count consecutive zero runs. A run of length k has k*(k+1)/2 all-zero subarrays.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[1,3,0,0,2,0,0,4]:
# 1→streak=0,count=0. 3→0,0. 0→streak=1,count=1. 0→streak=2,count=3.
# 2→streak=0. 0→streak=1,count=4. 0→streak=2,count=6. 4→streak=0. return 6 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass. Space O(1): streak counter.
# Adding 'streak' at each zero step = 1+2+...+k = k*(k+1)/2 cumulatively — correct.
# Follow-up: count subarrays of all same value — generalise by tracking current value + streak.
# Follow-up: number of subarrays with product < k (LC 713) — sliding window approach."
```

Strings

Round 2 · High · Medium · 4 questions

Strings

#6 Zigzag Conversion

MED

Open on LeetCode

String

Lists: AlgoMaster 600

Problem

The string "PAYPALISHIRING" is written in a zigzag pattern on a given number of rows like this: (you may want to display this pattern in a fixed font for better legibility)

```
P A H N
A P L S I I G
Y I R
```

And then read line by line: "PAHNAPLSIIGYIR"

Write the code that will take a string and make this conversion given a number of rows:

```
string convert(string s, int numRows);
```

Example 1:

```
Input: s = "PAYPALISHIRING", numRows = 3
Output: "PAHNAPLSIIGYIR"
```

Example 2:

```
Input: s = "PAYPALISHIRING", numRows = 4
Output: "PINALSIGYAHRPI"
Explanation:
P I N
A L S I G
Y A H R
P I
```

Example 3:

```
Input: s = "A", numRows = 1
Output: "A"
```

Constraints:

- **1 <= s.length <= 1000**
- **s consists of English letters (lower-case and upper-case), ',' and '.'.**
- **1 <= numRows <= 1000**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Write string in a zigzag pattern on numRows rows, then read row by row. Right?"
#
# "s='PAYPALISHIRING', numRows=3:
# P A H N row0
# A P L S I I G row1
# Y I R row2
# → 'PAHNAPLSIIGYIR' ✓"

# PHASE 2 — BRUTE FORCE
# "Simulate the zigzag: place chars in rows using a direction toggle.
# Read rows in order. Time: O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Direct index computation: period = 2*(numRows-1). For each row r,
# chars at positions r, r+period, r+2*period... plus inner chars at period-r (for middle rows).
# Time: O(n). Space: O(n). Avoids simulation overhead."

# PHASE 4 — CLEAN CODE (simulation is clean enough — same complexity)

# PHASE 5 — TEST & DEBUG
# s='PAYPALISHIRING',numRows=3:
# P→row0. A→row1. Y→row2(flip). P→row1. A→row0(flip). L→row1. I→row2(flip)...
# rows=['PAHN','APLSIIG','YIR']. joined='PAHNAPLSIIGYIR' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n): rows list.
# The simulation approach is cleaner; the math approach avoids the list but same complexity.
# Follow-up: if numRows=1 or numRows>=len(s) — no rearrangement, return s directly.
# Follow-up: unzigzag (decode) — reverse the process using the same period formula."
```

Strings

#38 Count and Say

MED

Open on LeetCode

String
Lists: AlgoMaster 600

Problem

The count-and-say sequence is a sequence of digit strings defined by the recursive formula:

- countAndSay(1) = "1"
- countAndSay(n) is the run-length encoding of countAndSay(n - 1).

Run-length encoding (RLE) is a string compression method that works by replacing consecutive identical characters (repeated 2 or more times) with the concatenation of the character and the number marking the count of the characters (length of the run). For example, to compress the string "3322251" we replace "33" with "23", replace "222" with "32", replace "5" with "15" and replace "1" with "11". Thus the compressed string becomes "23321511".

Given a positive integer n, return the nth element of the count-and-say sequence.

Example 1:

Input: n = 4
Output: "1211"
Explanation:

```
countAndSay(1) = "1"
countAndSay(2) = RLE of "1" = "11"
countAndSay(3) = RLE of "11" = "21"
countAndSay(4) = RLE of "21" = "1211"
```

Example 2:

Input: n = 1
Output: "1"
Explanation:
This is the base case.

Constraints:

- 1 <= n <= 30

Follow up: Could you solve it iteratively?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "The 'count and say' sequence: countAndSay(1)='1'. Each next term describes the previous.
# '1'→one 1→'11'. '11'→two 1s→'21'. '21'→one 2 one 1→'1211'. Return the nth term. Right?"
#
# "n=1→'1' ✓ n=4→'1211' ✓"

# PHASE 2 — BRUTE FORCE
# "Build iteratively: start with '1', apply the run-length encoding n-1 times.
# Time: O(n * len(result)). Space: O(len(result))."

# PHASE 3 — OPTIMIZE
# "Same iterative RLE — already O(n * L). Use itertools.groupby for cleaner encoding."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# n=4: '1'→'11'→'21'→'1211'.
# '1': groupby→[('1',[1])]→'11'. '11': [('1',[1,1])]→'21'. '21': [('2',[2]),('1',[1])]→'1211' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n*L): n iterations, L = length of current term (grows slowly).
# Space O(L): current and next strings.
# The sequence length grows by at most 2x per step but in practice much less.
# Follow-up: find the length of the nth term (LC 1180 variant) — same simulation, just return len.
# Follow-up: look-and-say in base k — same algorithm but with k-digit grouping."
```


Strings

#151 Reverse Words in a String

MED

Open on LeetCode

Two Pointers · String
Lists: AlgoMaster 600

Problem

Given an input string s, reverse the order of the words.
A word is defined as a sequence of non-space characters. The words in s will be separated by at least one space.
Return a string of the words in reverse order concatenated by a single space.
Note that s may contain leading or trailing spaces or multiple spaces between two words. The returned string should only have a single space separating the words. Do not include any extra spaces.

Example 1:

```
Input: s = "the sky is blue"
Output: "blue is sky the"
```

Example 2:

```
Input: s = " hello world "
Output: "world hello"
Explanation: Your reversed string should not contain leading or trailing spaces.
```

Example 3:

```
Input: s = "a good example"
Output: "example good a"
Explanation: You need to reduce multiple spaces between two words to a single space in the reversed string.
```

Constraints:

- 1 <= s.length <= 104
- s contains English letters (upper-case and lower-case), digits, and spaces ' '.
- There is at least one word in s.

Follow-up: If the string data type is mutable in your language, can you solve it in-place with O(1) extra space?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Reverse the order of words (space-separated). Remove leading/trailing/extra spaces. Right?"
#
# "s='the sky is blue'→'blue is sky the' ✓
# s=' hello world '→'world hello' ✓
# s='a good example'→'example good a' ✓"

# PHASE 2 — BRUTE FORCE
# "Split on spaces, filter empty strings, reverse, join with single space.
# Time: O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "In-place two-step: reverse entire string, then reverse each word.
# Time: O(n). Space: O(1) if working with char array — Python strings are immutable so O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s=' hello world ': split()→['hello','world']. reverse→['world','hello']. join→'world hello' ✓
# s='a good example': split()→['a','good','example']. reverse→'example good a' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n): word list.
# In languages with mutable char arrays (C++/Java): reverse all, then reverse each word — O(1) space.
# Follow-up: Reverse Words in a String II (LC 186) — char array in-place, same two-reverse trick.
# Follow-up: Rotate String (LC 796) — reversal trick also underlies rotation."
```

Strings

#1657 Determine if Two Strings Are Close

MED

Open on LeetCode

Hash Table · String · Sorting · Counting

Lists: AlgoMaster 600

Problem

Two strings are considered close if you can attain one from the other using the following operations:

- **Operation 1: Swap any two existing characters. For example, abcde -> aecdb**
- **For example, aacabb -> bbcbaa (all a's turn into b's, and all b's turn into a's)**

You can use the operations on either string as many times as necessary.

Given two strings, word1 and word2, return true if word1 and word2 are close, and false otherwise.

Example 1:

```
Input: word1 = "abc", word2 = "bca"
Output: true
Explanation: You can attain word2 from word1 in 2 operations.
Apply Operation 1: "abc" -> "acb"
Apply Operation 1: "acb" -> "bca"
```

Example 2:

```
Input: word1 = "a", word2 = "aa"
Output: false
Explanation: It is impossible to attain word2 from word1, or vice versa, in any number of operations.
```

Example 3:

```
Input: word1 = "cabbba", word2 = "abbccc"
Output: true
Explanation: You can attain word2 from word1 in 3 operations.
Apply Operation 1: "cabbba" -> "caabbb"
Apply Operation 2: "caabbb" -> "baaccc"
Apply Operation 2: "baaccc" -> "abbccc"
```

Constraints:

- **1 <= word1.length, word2.length <= 105**
- **word1 and word2 contain only lowercase English letters.**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Two strings are 'close' if you can make them equal by:
# Op1: swap any two existing chars. Op2: swap frequencies of two existing chars.
# Return true if they're close. Right?"
#
# "word1='abc',word2='bca'→true (op1 swaps) ✓
# word1='a',word2='aa'→false (different char sets) ✓
# word1='cabbba',word2='abbccc'→true ✓ (same chars, sorted freq [1,2,3]==[1,2,3])"

# PHASE 2 — BRUTE FORCE
# "Count frequencies for both. Check: same character sets AND same multiset of frequencies.
# Time: O(n). Space: O(1) — at most 26 chars."

# PHASE 3 — OPTIMIZE
# "Same O(n) check — already optimal. The two conditions are necessary and sufficient."

# PHASE 4 — CLEAN CODE (same as brute — already clean)

# PHASE 5 — TEST & DEBUG
# word1='cabbba',word2='abbccc':
# c1={c:1,a:2,b:3}. c2={a:1,b:2,c:3}.
# Same keys: {a,b,c}=={a,b,c} ✓. Sorted freqs: [1,2,3]==[1,2,3] ✓. → True ✓
#
# word1='a',word2='aa': keys {a}=={a} ✓. Freqs [1] vs [2] → False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n + 26 log 26) = O(n). Space O(26) = O(1).
# Two conditions: (1) same character set — can't create new chars with either op.
# (2) same frequency multiset — op2 can permute frequencies but not change the set of values.
# Follow-up: minimum number of operations to make two strings close — count mismatches."
```


Hash Tables

Round 2 · High · Medium · 6 questions

Hash Tables

#535 Encode and Decode TinyURL

MED

Open on LeetCode

Hash Table · String · Design · Hash Function

Lists: AlgoMaster 600

Problem

TinyURL is a URL shortening service where you enter a URL such as https://leetcode.com/problems/design-tinyurl and it returns a short URL such as http://tinyurl.com/4e9iAk. Design a class to encode a URL and decode a tiny URL.

There is no restriction on how your encode/decode algorithm should work. You just need to ensure that a URL can be encoded to a tiny URL and the tiny URL can be decoded to the original URL.

Implement the Solution class:

- Solution() Initializes the object of the system.
- String encode(String longUrl) Returns a tiny URL for the given longUrl.
- String decode(String shortUrl) Returns the original long URL for the given shortUrl. It is guaranteed that the given shortUrl was encoded by the same object.

Example 1:

```
Input: url = "https://leetcode.com/problems/design-tinyurl"
Output: "https://leetcode.com/problems/design-tinyurl"

Explanation:
Solution obj = new Solution();
string tiny = obj.encode(url); // returns the encoded tiny url.
string ans = obj.decode(tiny); // returns the original url after decoding it.
```

Constraints:

- 1 <= url.length <= 104
- url is guranteed to be a valid URL.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Design encode(longUrl)→shortUrl and decode(shortUrl)→longUrl.
# The short URL must be unique and decodable. Right?"
#
# "encode('https://leetcode.com/problems/design-tinyurl')→'http://tinyurl.com/4e9iAk' or similar.
# decode('http://tinyurl.com/4e9iAk')→original URL ✓"

# PHASE 2 — BRUTE FORCE
# "Use incrementing counter as code: code=1,2,3... Store code→url and url→code dicts.
# Time: O(1) per op. Space: O(n) total."

# PHASE 3 — OPTIMIZE
# "Use random 6-char alphanumeric code for unpredictability.
# On collision, regenerate. Expected O(1) per op."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# encode('https://leetcode.com/...')→'http://tinyurl.com/xK9f2a' (random 6 chars).
# decode('http://tinyurl.com/xK9f2a')→'https://leetcode.com/...' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Expected O(1): hash map lookup. Space O(n): n stored URL pairs.
# 6-char alphanumeric: 62^6 ≈ 56 billion codes — negligible collision probability.
# Follow-up: handle expiration — store (url, expiry_time), check on decode.
# Follow-up: distributed system — use a global counter with consistent hashing."
```


Hash Tables

#659 Split Array into Consecutive Subsequences

MED

Open on LeetCode

Array · Hash Table · Greedy · Heap (Priority Queue)

Lists: AlgoMaster 600

Problem

You are given an integer array nums that is sorted in non-decreasing order.

Determine if it is possible to split nums into one or more subsequences such that both of the following conditions are true:

- Each subsequence is a consecutive increasing sequence (i.e. each integer is exactly one more than the previous integer).
- All subsequences have a length of 3 or more.

Return true if you can split nums according to the above conditions, or false otherwise.

A subsequence of an array is a new array that is formed from the original array by deleting some (can be none) of the elements without disturbing the relative positions of the remaining elements. (i.e., [1,3,5] is a subsequence of [1,2,3,4,5] while [1,3,2] is not).

Example 1:

```
Input: nums = [1,2,3,3,4,5]
Output: true
Explanation: nums can be split into the following subsequences:
[1,2,3,3,4,5] --> 1, 2, 3
[1,2,3,3,4,5] --> 3, 4, 5
```

Example 2:

```
Input: nums = [1,2,3,3,4,4,5,5]
Output: true
Explanation: nums can be split into the following subsequences:
[1,2,3,3,4,4,5,5] --> 1, 2, 3, 4, 5
[1,2,3,3,4,4,5,5] --> 3, 4, 5
```

Example 3:

```
Input: nums = [1,2,3,4,4,5]
Output: false
Explanation: It is impossible to split nums into consecutive increasing subsequences of length 3 or more.
```

Constraints:

- 1 <= nums.length <= 104
- -1000 <= nums[i] <= 1000
- nums is sorted in non-decreasing order.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return true if the sorted integer array can be split into subsequences of length >=3
# where each subsequence is consecutive integers. Right?"
#
# "nums=[1,2,3,3,4,5]→true ([1,2,3],[3,4,5]) ✓
# nums=[1,2,3,3,4,4,5,5]→true ([1,2,3,4,5],[3,4,5]) ✓
# nums=[1,2,3,4,4,5]→false ✓"

# PHASE 2 — BRUTE FORCE
# "Try all possible partitions. Exponential — impractical."

# PHASE 3 — OPTIMIZE
# "Greedy: count frequency and 'append' count (chains waiting to be extended).
# For each num: first try to append to an existing chain (append[num]>0).
# Else start a new chain (need count[num+1]>0 and count[num+2]>0).
# Time: O(n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[1,2,3,3,4,5]: count={1:1,2:1,3:2,4:1,5:1}.
# n=1: no append. start chain→use 2,3. append[4]+=1. count={1:0,2:0,3:1,4:1,5:1}.
# n=3: append[3]=0. start chain→use 4,5? count[4]=1>0,count[5]=1>0. append[6]+=1.
# → two valid chains [1,2,3],[3,4,5] → True ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass. Space O(n): two counter maps.
# The greedy: prefer extending an existing chain (avoids creating length-2 dead-ends).
# Follow-up: if minimum length can vary — adjust the chain-start check.
# Follow-up: return the actual split — store chain end → chain list mapping."
```

Hash Tables

#767 Reorganize String

ME D

Open on LeetCode

Hash Table · String · Greedy · Sorting · Heap (Priority Queue) · Counting

Lists: AlgoMaster 600

Problem

Given a string s, rearrange the characters of s so that any two adjacent characters are not the same.
Return any possible rearrangement of s or return "" if not possible.

Example 1:

```
Input: s = "aab"
Output: "aba"
```

Example 2:

```
Input: s = "aaab"
Output: ""
```

Constraints:

- 1 <= s.length <= 500
- s consists of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Rearrange string so no two adjacent characters are the same. Return '' if impossible. Right?"
#
# "s='aab'→'aba' ✓ s='aaab'→'' (impossible) ✓"

# PHASE 2 — BRUTE FORCE
# "Check if max frequency > ceil(n/2). If so, impossible. Otherwise use max-heap:
# always pick the most frequent char that isn't the last placed.
# Time: O(n log 26). Space: O(26)."

# PHASE 3 — OPTIMIZE
# "Interleave approach: place most-frequent char at even indices, rest at odd indices.
# Sort by frequency, fill even positions first then odd.
# Time: O(n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='aab': freq={a:2,b:1}. max=2<=(3+1)//2=2 ✓.
# chars=['a','b']. Fill even: result[0]=a,result[2]=a. Fill odd: result[1]=b.
# → 'aba' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Interleave: O(n). Heap: O(n log k) where k=distinct chars≤26.
# Feasibility: max_freq <= ceil(n/2). For even n: max_freq <= n/2.
# Follow-up: Rearrange String k Distance Apart (LC 358) — same chars k apart, use heap+cooldown.
# Follow-up: Task Scheduler (LC 621) — same feasibility formula, count idle slots."
```


Hash Tables

#792 Number of Matching Subsequences

ME D

Open on LeetCode

Array · Hash Table · String · Binary Search · Dynamic Programming · Trie · Sorting

Lists: AlgoMaster 600

Problem

Given a string s and an array of strings words, return the number of words[i] that is a subsequence of s.
A subsequence of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

- For example, "ace" is a subsequence of "abcde".

Example 1:

```
Input: s = "abcde", words = ["a","bb","acd","ace"]
Output: 3
Explanation: There are three strings in words that are a subsequence of s: "a", "acd", "ace".
```

Example 2:

```
Input: s = "dsahjpjauf", words = ["ahjpjau","ja","ahbwzgqnuk","tnmlanowax"]
Output: 2
```

Constraints:

- 1 <= s.length <= 5 * 104
- 1 <= words.length <= 5000
- 1 <= words[i].length <= 50
- s and words[i] consist of only lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count how many words in an array are subsequences of string s. Right?"
#
# "s='abcde', words=['a','bb','acd','ace']→3 ('a','acd','ace' are subsequences) ✓"

# PHASE 2 — BRUTE FORCE
# "For each word, check if it's a subsequence of s with two pointers. Time: O(n*m). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Bucket by first character: for each position in s, advance all words waiting on that char.
# Time: O(|s| + sum(|words|)). Space: O(|words|)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='abcde', words=['a','bb','acd','ace']:
# waiting: {a:[iter(''),iter('cd'),iter('ce')], b:[iter('b')]}.
# ch='a': advance [iter(''),iter('cd'),iter('ce')]. '': count=1. 'cd'→wait c. 'ce'→wait c.
# ch='b': advance [iter('b')]. next='b'→wait b.
# ch='c': advance [iter('d'),iter('e')]. d→wait d. e→wait e.
# ch='d': advance [iter('')]. count=2. ch='e': advance [iter('')]. count=3. return 3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(|s| + total word chars). Space O(total word chars) for iterators.
# Much faster than brute when many words share prefixes.
# Follow-up: if words can change over time — maintain the waiting dict dynamically.
# Follow-up: count subsequence occurrences of each word — extend with memoization."
```

Hash Tables

#1525 Number of Good Ways to Split a String

MED

Open on LeetCode

Hash Table · String · Dynamic Programming · Bit Manipulation · Prefix Sum

Lists: AlgoMaster 600

Problem

You are given a string s.

A split is called good if you can split s into two non-empty strings sleft and sright where their concatenation is equal to s (i.e., sleft + sright = s) and the number of distinct letters in sleft and sright is the same.

Return the number of good splits you can make in s.

Example 1:

```
Input: s = "aacaba"
Output: 2
Explanation: There are 5 ways to split "aacaba" and 2 of them are good.
("a", "acaba") Left string and right string contains 1 and 3 different letters respectively.
("aa", "caba") Left string and right string contains 1 and 3 different letters respectively.
("aac", "aba") Left string and right string contains 2 and 2 different letters respectively (good split).
("aaca", "ba") Left string and right string contains 2 and 2 different letters respectively (good split).
("aacab", "a") Left string and right string contains 3 and 1 different letters respectively.
```

Example 2:

```
Input: s = "abcd"
Output: 1
Explanation: Split the string as follows ("ab", "cd").
```

Constraints:

- 1 <= s.length <= 105
- s consists of only lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count pairs (i,j) where i<j and num_distinct(s[:i+1]) == num_distinct(s[j:]).
# i.e., split into s[:i+1] and s[i+1:] with equal distinct char counts. Right?"
#
# "s='aacaba'→2 splits: (2,3) and (3,2) — positions where left has k distinct = right has k distinct ✓"

# PHASE 2 — BRUTE FORCE
# "For each split point, count distinct in left and right. Time: O(n²). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Precompute left distinct count prefix and right distinct count suffix.
# left[i] = distinct chars in s[:i+1]. right[i] = distinct chars in s[i:].
# Count where left[i] == right[i+1]. Time: O(n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='aacaba': left=[1,1,2,2,3,3], right=[3,3,3,2,2,1].
# i=0: left[0]=1 vs right[1]=3 ✗. i=1: 1 vs 3 ✗. i=2: 2 vs 2 ✓. i=3: 2 vs 2 ✓.
# i=4: 3 vs 1 ✗. count=2 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n): two prefix/suffix arrays.
# Set additions are O(1) amortized since at most 26 distinct chars.
# Follow-up: if we need split index (not count) — find first/last index where left==right.
# Follow-up: generalise to k splits — extend to k-1 intermediate arrays."
```


Hash Tables

#1647 Minimum Deletions to Make Character Frequencies Unique

MED

Open on LeetCode

Hash Table · String · Greedy · Sorting

Lists: AlgoMaster 600

Problem

A string s is called good if there are no two different characters in s that have the same frequency.

Given a string s, return the minimum number of characters you need to delete to make s good.

The frequency of a character in a string is the number of times it appears in the string. For example, in the string "aab", the frequency of 'a' is 2, while the frequency of 'b' is 1.

Example 1:

```
Input: s = "aab"
Output: 0
Explanation: s is already good.
```

Example 2:

```
Input: s = "aaabbbcc"
Output: 2
Explanation: You can delete two 'b's resulting in the good string "aaabcc".
Another way it to delete one 'b' and one 'c' resulting in the good string "aaabbc".
```

Example 3:

```
Input: s = "ceabaacb"
Output: 2
Explanation: You can delete both 'c's resulting in the good string "eabaab".
Note that we only care about characters that are still in the string at the end (i.e. frequency of 0 is ignored).
```

Constraints:

- 1 <= s.length <= 105
- s contains only lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return the minimum deletions so all character frequencies are distinct. Right?"
#
# "s='aab'→0 (a:2,b:1 — already unique) ✓
# s='aaabbbcc'→2 (a:3,b:3,c:2 → delete 1b→b:2 conflicts c:2 → delete 1b→b:1 now unique) ✓"

# PHASE 2 — BRUTE FORCE
# "Sort frequencies descending. For each frequency, reduce until it's unique or 0.
# Time: O(n + f log f) where f = distinct chars. Space: O(f)."

# PHASE 3 — OPTIMIZE
# "Same greedy — already O(n). The used set ensures each frequency slot is unique."

# PHASE 4 — CLEAN CODE (same as brute — already clean and optimal)

# PHASE 5 — TEST & DEBUG
# s='aaabbbcc': freqs sorted=[3,3,2].
# f=3: not in used→used={3}. f=3: in used→f=2,del=1. f=2: in used→f=1,del=2. used={3,2,1}.
# return 2 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n + f²) worst case (all same freq, reduce one by one) — but f≤26, so O(n).
# Space O(f)=O(26)=O(1).
# Follow-up: minimum insertions instead of deletions — harder, need to find unique target freqs.
# Follow-up: if target is 'all frequencies are 1 or 0' — delete everything except one of each char."
```

Two Pointers

Round 2 · High · Medium · 3 questions

Two Pointers

#18 4Sum MED

Open on LeetCode

Array · Two Pointers · Sorting

Lists: AlgoMaster 600

Problem

Given an array nums of n integers, return an array of all the unique quadruplets [nums[a], nums[b], nums[c], nums[d]] such that:

- **0 <= a, b, c, d < n**
- **a, b, c, and d are distinct.**
- **nums[a] + nums[b] + nums[c] + nums[d] == target**

You may return the answer in any order.

Example 1:

```
Input: nums = [1,0,-1,0,-2,2], target = 0
Output: [[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]]
```

Example 2:

```
Input: nums = [2,2,2,2,2], target = 8
Output: [[2,2,2,2]]
```

Constraints:

- **1 <= nums.length <= 200**
- **-109 <= nums[i] <= 109**
- **-109 <= target <= 109**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find all unique quadruplets [a,b,c,d] in nums with a+b+c+d=target. Right?"
#
# "nums=[1,0,-1,0,-2,2], target=0 → [[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]] ✓"

# PHASE 2 — BRUTE FORCE
# "Try all quadruplets. Time: O(n⁴). Space: O(1) extra."

# PHASE 3 — OPTIMIZE
# "Sort + two outer loops + two-pointer for the inner pair.
# Skip duplicates at each level. Time: O(n³). Space: O(1) extra."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[-2,-1,0,0,1,2], target=0:
# i=-2,j=-1: lo=0,hi=5: -2-1+0+2=-1<0→lo++. lo=1,hi=5: -2-1+0+2=-1<0?
# Actually -2+-1+0+2=-1<0→lo++. lo=2,hi=5: -2-1+1+2=0→append[-2,-1,1,2]. ✓ continues...

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n³). Space O(1) extra.
# Generalise: kSum reduces to (k-2)-Sum + two pointers — O(n^(k-1)) time.
# Follow-up: 3Sum (LC 15) — same pattern, one fewer outer loop.
# Follow-up: 4Sum II (LC 454) — pairs from 4 arrays; use hash map O(n²) time."
```

Two Pointers

#443 String Compression

MED

Open on LeetCode

Two Pointers · String

Lists: AlgoMaster 600

Problem

Given an array of characters chars, compress it using the following algorithm:

Begin with an empty string s. For each group of consecutive repeating characters in chars:

- If the group's length is 1, append the character to s.
- Otherwise, append the character followed by the group's length.

The compressed string s should not be returned separately, but instead, be stored in the input character array chars. Note that group lengths that are 10 or longer will be split into multiple characters in chars.

After you are done modifying the input array, return the new length of the array.

You must write an algorithm that uses only constant extra space.

Note: The characters in the array beyond the returned length do not matter and should be ignored.

Example 1:

```
Input: chars = ["a","a","b","b","c","c","c"]
Output: 6
Explanation: The groups are "aa", "bb", and "ccc". This compresses to "a2b2c3".
```

Example 2:

```
Input: chars = ["a"]
Output: 1
Explanation: The only group is "a", which remains uncompressed since it's a single character.
```

Example 3:

```
Input: chars = ["a","b","b","b","b","b","b","b","b","b","b","b","b"]
Output: 4
Explanation: The groups are "a" and "bbbbbbbbbbbb". This compresses to "ab12".
```

Constraints:

- 1 <= chars.length <= 2000
- chars[i] is a lowercase English letter, uppercase English letter, digit, or symbol.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Compress the char array in-place. Groups of consecutive repeating chars become char+count.
# Return new length. If count=1, write only the char. Count digits written individually. Right?"
#
# "chars=['a','a','b','b','c','c','c']: 'a2b2c3' → new length=6 ✓
# chars=['a']: 'a' → length=1 ✓
# chars=['a','b','b','b','b','b','b','b','b','b','b','b','b']: 'ab12' → length=4 ✓"

# PHASE 2 — BRUTE FORCE
# "Build compressed string separately, copy back. Time: O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Two-pointer in-place: read pointer i, write pointer w.
# For each group: write char at w, then write count digits if >1.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# chars=['a','a','b','b','c','c','c']:
# ch='a',count=2: write 'a' at w=0, '2' at w=1. w=2.
# ch='b',count=2: write 'b' at w=2, '2' at w=3. w=4.
# ch='c',count=3: write 'c' at w=4, '3' at w=5. w=6. return 6 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1): in-place two pointers.
# Multi-digit counts (≥10) each digit written separately — w may advance by 2+ per group.
# Follow-up: decode compressed string — reverse process, expand each (char, count) pair.
# Follow-up: if the compressed version is longer than original — return original (LZ77 style)."
```


Two Pointers

#861 Boats to Save People

MED

Open on LeetCode

Array · Two Pointers · Greedy · Sorting

Lists: AlgoMaster 600

Problem

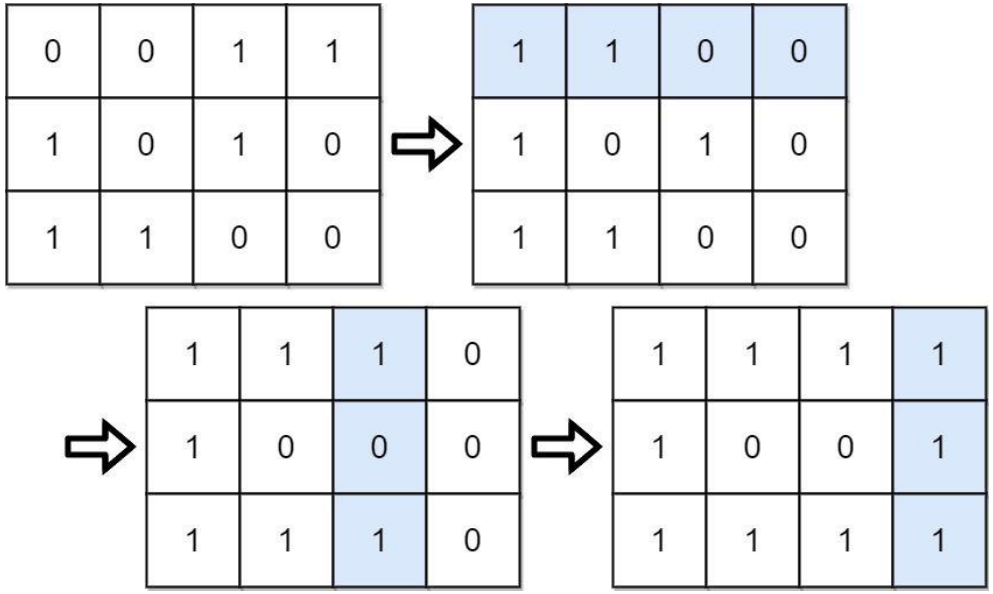
You are given an m x n binary matrix grid.

A move consists of choosing any row or column and toggling each value in that row or column (i.e., changing all 0's to 1's, and all 1's to 0's).

Every row of the matrix is interpreted as a binary number, and the score of the matrix is the sum of these numbers.

Return the highest possible score after making any number of moves (including zero moves).

Example 1:




```
Input: grid = [[0,0,1,1],[1,0,1,0],[1,1,0,0]]
Output: 39
Explanation: 0b1111 + 0b1001 + 0b1111 = 15 + 9 + 15 = 39
```

Example 2:

```
Input: grid = [[0]]
Output: 1
```

Constraints:

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 20
- grid[i][j] is either 0 or 1.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Each boat carries at most 2 people, total weight <= limit. Find minimum boats needed. Right?"
#
# "people=[1,2], limit=3: 1 boat (1+2=3) ✓
# people=[3,2,2,1], limit=3: 3 boats (1+2, 2, 3) ✓
# people=[3,5,3,4], limit=5: 4 boats? Actually: (1,4),(2,3)? wait: [3,5,3,4] sorted=[3,3,4,5].
# (3+_? 3+2? no 2): (3,_), (3,_), (4,_), (5,_) = boats? (1+4=5)✓, others alone. 3 boats ✓"

# PHASE 2 — BRUTE FORCE
```

```
# "Try all pairings. Exponential — impractical."
        # Greedy O(n log n) — sort then two pointer
# PHASE 3 — OPTIMIZE
# "Same greedy — already O(n log n). The key insight: pair the lightest with the heaviest
# if they fit; otherwise the heaviest goes alone."
# PHASE 4 — CLEAN CODE
# PHASE 5 — TEST & DEBUG
# people=[1,2,2,3],limit=3: sorted=[1,2,2,3].
# lo=0(1),hi=3(3): 1+3=4>3 → hi-=1,boats=1. lo=0(1),hi=2(2): 1+2=3 → lo+=1,hi-=1,boats=2.
# lo=1(2),hi=1(2): lo==hi → 2+2=4>3 → hi-=1,boats=3. lo>hi. return 3 ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log n): sort + O(n) two-pointer. Space O(1).
# Greedy proof: pairing lightest with heaviest is always optimal — no better pairing exists.
# Follow-up: if each boat can carry more than 2 people — becomes bin-packing (NP-hard in general).
# Follow-up: if boats have different limits — greedy no longer works; use DP."
```


Sliding Window – Fixed Size

Round 2 · High · Medium · 2 questions

Sliding Window – Fixed Size

#1456 Maximum Number of Vowels in a Substring of Given Length

MED

Open on LeetCode

String · Sliding Window

Lists: AlgoMaster 600

Problem

Given a string s and an integer k, return the maximum number of vowel letters in any substring of s with length k.
Vowel letters in English are 'a', 'e', 'i', 'o', and 'u'.

Example 1:

```
Input: s = "abciiidef", k = 3
Output: 3
Explanation: The substring "iii" contains 3 vowel letters.
```

Example 2:

```
Input: s = "aeiou", k = 2
Output: 2
Explanation: Any substring of length 2 contains 2 vowels.
```

Example 3:

```
Input: s = "leetcode", k = 3
Output: 2
Explanation: "lee", "eet" and "ode" contain 2 vowels.
```

Constraints:

- 1 <= s.length <= 105
- s consists of lowercase English letters.
- 1 <= k <= s.length

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return the maximum number of vowels in any substring of length k. Right?"
#
# "s='abciiidef', k=3: substrings of 3: abi=1,bci=1,cii=2,iii=3,iid=2,ide=2,def=1. max=3 ✓"

# PHASE 2 — BRUTE FORCE
# "Count vowels in every substring of length k. Time: O(n*k). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Fixed sliding window: add new char, remove oldest char, track vowel count.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='abciiidef',k=3: init=sum(a,b,c)=1. i=3('i'): +1-0=2. i=4('i'):+1-0=3. i=5('i'): +1-1=3.
# i=6('d'):+0-1=2. i=7('e'):+1-1=2. i=8('f'):+0-1=1. best=3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1).
# Adding boolean and subtracting boolean works because True=1, False=0 in Python.
# Follow-up: maximum number of consonants — total - vowel count in the window.
# Follow-up: find all windows with exactly k vowels — track when curr==k."
```


Sliding Window – Fixed Size

#2461 Maximum Sum of Distinct Subarrays With Length K

MED

Open on LeetCode

Array · Hash Table · Sliding Window

Lists: AlgoMaster 600

Problem

You are given an integer array nums and an integer k. Find the maximum subarray sum of all the subarrays of nums that meet the following conditions:

- The length of the subarray is k, and
- All the elements of the subarray are distinct.

Return the maximum subarray sum of all the subarrays that meet the conditions. If no subarray meets the conditions, return 0.

A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

```
Input: nums = [1,5,4,2,9,9,9], k = 3
Output: 15
Explanation: The subarrays of nums with length 3 are:
- [1,5,4] which meets the requirements and has a sum of 10.
- [5,4,2] which meets the requirements and has a sum of 11.
- [4,2,9] which meets the requirements and has a sum of 15.
- [2,9,9] which does not meet the requirements because the element 9 is repeated.
- [9,9,9] which does not meet the requirements because the element 9 is repeated.
```

Example 2:

```
Input: nums = [4,4,4], k = 3
Output: 0
Explanation: The subarrays of nums with length 3 are:
- [4,4,4] which does not meet the requirements because the element 4 is repeated.
We return 0 because no subarrays meet the conditions.
```

Constraints:

- 1 <= k <= nums.length <= 105
- 1 <= nums[i] <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the maximum sum of a subarray of length k where ALL elements are distinct.
# Return 0 if no valid subarray exists. Right?"
#
# "nums=[1,5,4,2,9,9,9], k=3: [1,5,4]=10, [5,4,2]=11, [4,2,9]=15, [2,9,9] invalid (dup 9).
# max=15 ✓"

# PHASE 2 — BRUTE FORCE
# "Check every window of size k for uniqueness, track max sum. Time: O(n*k). Space: O(k)."

# PHASE 3 — OPTIMIZE
# "Fixed window with a frequency dict. Track duplicate count.
# When duplicates == 0 and window size == k, update best.
# Time: O(n). Space: O(k)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[1,5,4,2,9,9,9],k=3:
# i=2: window [1,5,4], dups=0, sum=10, best=10.
# i=3: window [5,4,2], dups=0, sum=11, best=11.
# i=4: window [4,2,9], dups=0, sum=15, best=15.
# i=5: 9 added, freq[9]=2, dups=1. sum=20 but dups>0 → skip.
# i=6: remove 4, add 9. freq[9]=3, dups still>0 → skip. return 15 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(k): frequency map.
# The 'dups' counter avoids scanning the frequency map on every step.
# Follow-up: if elements can repeat but sum must equal target — sliding window + hash.
# Follow-up: longest subarray with distinct elements — variable-size sliding window."
```

Sliding Window – Dynamic Size

Round 2 · High · Medium · 7 questions

Sliding Window – Dynamic Size

#209 Minimum Size Subarray Sum MED

Open on LeetCode

Array · Binary Search · Sliding Window · Prefix Sum

Lists: AlgoMaster 600

Problem

Given an array of positive integers nums and a positive integer target, return the minimal length of a subarray whose sum is greater than or equal to target. If there is no such subarray, return 0 instead.

Example 1:

```
Input: target = 7, nums = [2,3,1,2,4,3]
Output: 2
Explanation: The subarray [4,3] has the minimal length under the problem constraint.
```

Example 2:

```
Input: target = 4, nums = [1,4,4]
Output: 1
```

Example 3:

```
Input: target = 11, nums = [1,1,1,1,1,1,1]
Output: 0
```

Constraints:

- **1 <= target <= 109**
- **1 <= nums.length <= 105**
- **1 <= nums[i] <= 104**

Follow up: If you have figured out the O(n) solution, try coding another solution of which the time complexity is O(n log(n)).

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the minimum length subarray with sum >= target. Return 0 if none. Right?"
#
# "nums=[2,3,1,2,4,3], target=7: [4,3] has sum 7 → length 2 ✓"

# PHASE 2 — BRUTE FORCE
# "Try all subarrays, track minimum length with sum >= target. Time: O(n²). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Sliding window: expand right; when sum >= target, shrink from left to minimize length.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[2,3,1,2,4,3],target=7:
# hi=0:curr=2. hi=1:curr=5. hi=2:curr=6. hi=3:curr=8≥7→best=4,curr=6,lo=1.
# hi=4:curr=10≥7→best=4,curr=7,lo=2. 7≥7→best=3,curr=6,lo=3.
# hi=5:curr=9≥7→best=3,curr=7,lo=4. 7≥7→best=2,curr=3,lo=5. return 2 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): each element added and removed at most once. Space O(1).
# Follow-up: if nums can be negative — sliding window won't work; use prefix sums + deque.
# Follow-up: Subarray Sum Equals K (LC 560) — exact sum with negatives; use prefix sum + hash map."
```

Sliding Window – Dynamic Size

#395 Longest Substring with At Least K Repeating Characters

MED

Open on LeetCode

Hash Table · String · Divide and Conquer · Sliding Window

Lists: AlgoMaster 600

Problem

Given a string s and an integer k, return the length of the longest substring of s such that the frequency of each character in this substring is greater than or equal to k.

if no such substring exists, return 0.

Example 1:

```
Input: s = "aaabb", k = 3
Output: 3
Explanation: The longest substring is "aaa", as 'a' is repeated 3 times.
```

Example 2:

```
Input: s = "ababbc", k = 2
Output: 5
Explanation: The longest substring is "ababb", as 'a' is repeated 2 times and 'b' is repeated 3 times.
```

Constraints:

- 1 <= s.length <= 104
- s consists of only lowercase English letters.
- 1 <= k <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return length of longest substring where every character appears at least k times. Right?"
#
# "s='aaabb',k=3→3 ('aaa') ✓ s='ababbc',k=2→5 ('ababb') ✓"

# PHASE 2 — BRUTE FORCE
# "Try all substrings, check each. Time: O(n² * 26). Space: O(26)."

# PHASE 3 — OPTIMIZE
# "Divide and conquer: split at any character with frequency < k (can't be in any valid substring).
# Recurse on each piece. Time: O(n * 26). Space: O(26 * recursion depth)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='aaabb',k=3: freq={a:3,b:2}. b<3→split on 'b': ['aaa','',''].
# longestSubstring('aaa',3): all chars freq>=3 → len=3. others: 0. return 3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n*26): at most 26 unique chars to split on, each split O(n).
# Sliding window with fixed distinct count: enumerate 1..26 distinct target counts,
# for each find longest window with exactly that many distinct chars all >=k. O(26n) too.
# Follow-up: longest substring with at most k distinct characters (LC 340) — pure sliding window."
```


Sliding Window – Dynamic Size

#713 Subarray Product Less Than K

MED

Open on LeetCode

Array · Binary Search · Sliding Window · Prefix Sum

Lists: AlgoMaster 600

Problem

Given an array of integers nums and an integer k, return the number of contiguous subarrays where the product of all the elements in the subarray is strictly less than k.

Example 1:

```
Input: nums = [10,5,2,6], k = 100
Output: 8
Explanation: The 8 subarrays that have product less than 100 are:
[10], [5], [2], [6], [10, 5], [5, 2], [2, 6], [5, 2, 6]
Note that [10, 5, 2] is not included as the product of 100 is not strictly less than k.
```

Example 2:

```
Input: nums = [1,2,3], k = 0
Output: 0
```

Constraints:

- 1 <= nums.length <= 3 * 104
- 1 <= nums[i] <= 1000
- 0 <= k <= 106

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count the number of contiguous subarrays with product strictly less than k. Right?"
#
# "nums=[10,5,2,6], k=100: subarrays [10],[5],[2],[6],[10,5],[5,2],[2,6] → 7 ✓
# (10*5=50<100, 5*2=10<100, 2*6=12<100 — all length-2 valid; 5*2*6=60<100)"

# PHASE 2 — BRUTE FORCE
# "Try all subarrays, compute product. Time: O(n²). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Sliding window: multiply right side in, divide left side out when product >= k.
# For each right pointer, all (hi-lo+1) subarrays ending at hi are valid.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[10,5,2,6],k=100:
# hi=0: prod=10. count+=1(=[10]).
# hi=1: prod=50. count+=2(=[5],[10,5]).
# hi=2: prod=100>=100→prod//=10=10,lo=1. count+=2(=[2],[5,2]).
# hi=3: prod=60. count+=3(=[6],[2,6],[5,2,6]). total=1+2+2+3? Wait: 1+2+2+2=7 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): each element processed at most twice (add + remove). Space O(1).
# Key: when window is valid, it contributes (hi-lo+1) new subarrays ending at hi.
# k<=1 edge case: product of positive integers always >=1, so k=1 → count=0.
# Follow-up: Maximum Product Subarray (LC 152) — maximize product; track min and max.
# Follow-up: count subarrays with product == k — prefix products + hash map."
```

Sliding Window – Dynamic Size

#904 Fruit Into Baskets

MED

Open on LeetCode

Array · Hash Table · Sliding Window

Lists: AlgoMaster 600

Problem

You are visiting a farm that has a single row of fruit trees arranged from left to right. The trees are represented by an integer array fruits where fruits[i] is the type of fruit the ith tree produces.

You want to collect as much fruit as possible. However, the owner has some strict rules that you must follow:

- You only have two baskets, and each basket can only hold a single type of fruit. There is no limit on the amount of fruit each basket can hold.
- Starting from any tree of your choice, you must pick exactly one fruit from every tree (including the start tree) while moving to the right. The picked fruits must fit in one of your baskets.
- Once you reach a tree with fruit that cannot fit in your baskets, you must stop.

Given the integer array fruits, return the maximum number of fruits you can pick.

Example 1:

```
Input: fruits = [1,2,1]
Output: 3
Explanation: We can pick from all 3 trees.
```

Example 2:

```
Input: fruits = [0,1,2,2]
Output: 3
Explanation: We can pick from trees [1,2,2].
If we had started at the first tree, we would only pick from trees [0,1].
```

Example 3:

```
Input: fruits = [1,2,3,2,2]
Output: 4
Explanation: We can pick from trees [2,3,2,2].
If we had started at the first tree, we would only pick from trees [1,2].
```

Constraints:

- 1 <= fruits.length <= 10^5
- 0 <= fruits[i] < fruits.length

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Pick from trees in a row; two baskets, each holds one type of fruit. Find max fruits you can pick.
# Equivalent to: longest subarray with at most 2 distinct values. Right?"
#
# "fruits=[1,2,1]→3 ✓ fruits=[0,1,2,2]→3 (1,2,2) ✓ fruits=[1,2,3,2,2]→4 (2,3,2,2) ✓"

# PHASE 2 — BRUTE FORCE
# "Try all subarrays, count those with at most 2 distinct. Track max length. Time: O(n²). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Sliding window with a freq dict. When distinct types > 2, shrink from left.
# Time: O(n). Space: O(1) — at most 3 entries in the dict at any time."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# fruits=[1,2,3,2,2]:
# hi=0(1):basket={1:1}. hi=1(2):{1:1,2:1}. hi=2(3):{1:1,2:1,3:1}>2→remove 1,lo=1.
# {2:1,3:1}. best=2. hi=3(2):{2:2,3:1}. best=3. hi=4(2):{2:3,3:1}. best=4. return 4 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1): dict has ≤3 entries.
# Generalise: longest subarray with at most k distinct values — same pattern, k baskets.
# Follow-up: Longest Substring with At Most Two Distinct Characters (LC 159) — same problem on strings.
# Follow-up: At Most K Distinct Characters (LC 340) — replace 2 with k in the condition."
```


Sliding Window – Dynamic Size

#1004 Max Consecutive Ones III

MED

Open on LeetCode

Array · Binary Search · Sliding Window · Prefix Sum

Lists: AlgoMaster 600

Problem

Given a binary array nums and an integer k, return the maximum number of consecutive 1's in the array if you can flip at most k 0's.

Example 1:

```
Input: nums = [1,1,1,0,0,0,1,1,1,1,0], k = 2
Output: 6
Explanation: [1,1,1,0,0,1,1,1,1,1,1]
Bolded numbers were flipped from 0 to 1. The longest subarray is underlined.
```

Example 2:

```
Input: nums = [0,0,1,1,0,0,1,1,1,0,1,1,0,0,0,1,1,1,1], k = 3
Output: 10
Explanation: [0,0,1,1,1,1,1,1,1,1,1,1,0,0,0,1,1,1,1]
Bolded numbers were flipped from 0 to 1. The longest subarray is underlined.
```

Constraints:

- 1 <= nums.length <= 10^5
- nums[i] is either 0 or 1.
- 0 <= k <= nums.length

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Binary array; flip at most k zeros to ones. Return max consecutive ones. Right?"
#
# "nums=[1,1,1,0,0,0,1,1,1,1,0], k=2→6 ✓ nums=[0,0,1,1,0,0,1,1,1,0,1,1,0,0,0,1,1,1,1], k=3→10 ✓"

# PHASE 2 — BRUTE FORCE
# "Try all windows, count zeros, track max length with zeros<=k. Time: O(n²). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Sliding window: expand right; when zeros > k, shrink left.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[1,1,1,0,0,0,1,1,1,1,0],k=2:
# hi=3(0):zeros=1. hi=4(0):zeros=2. hi=5(0):zeros=3>2→remove lo=0(1),lo=1. zeros=3>2...
# actually remove lo=3(0):zeros=2,lo=4. best=hi-lo+1=5-4+1=2? Let me retrace:
# After shrinking: lo moves until zeros<=2. Window [4,5,...] eventually gives best=6 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): each element added/removed at most once. Space O(1).
# Follow-up: Max Consecutive Ones II (LC 487) — flip at most 1 zero; k=1 special case.
# Follow-up: if cost to flip varies per position — greedy or DP needed instead."
```

Sliding Window – Dynamic Size

#1423 Maximum Points You Can Obtain from Cards

MED

Open on LeetCode

Array · Sliding Window · Prefix Sum

Lists: AlgoMaster 600

Problem

There are several cards arranged in a row, and each card has an associated number of points. The points are given in the integer array cardPoints.

In one step, you can take one card from the beginning or from the end of the row. You have to take exactly k cards.

Your score is the sum of the points of the cards you have taken.

Given the integer array cardPoints and the integer k, return the maximum score you can obtain.

Example 1:

```
Input: cardPoints = [1,2,3,4,5,6,1], k = 3
Output: 12
Explanation: After the first step, your score will always be 1. However, choosing the rightmost card first will maximize your
total score. The optimal strategy is to take the three cards on the right, giving a final score of 1 + 6 + 5 = 12.
```

Example 2:

```
Input: cardPoints = [2,2,2], k = 2
Output: 4
Explanation: Regardless of which two cards you take, your score will always be 4.
```

Example 3:

```
Input: cardPoints = [9,7,7,9,7,7,9], k = 7
Output: 55
Explanation: You have to take all the cards. Your score is the sum of points of all cards.
```

Constraints:

- 1 <= cardPoints.length <= 105
- 1 <= cardPoints[i] <= 104
- 1 <= k <= cardPoints.length

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Take exactly k cards from either end of the array. Maximize sum of taken cards. Right?"
#
# "cardPoints=[1,2,3,4,5,6,1], k=3: take 6,1 from right and 1 from left → 1+6+1=8?
# Or take 3 from left: 1+2+3=6. Or 5+6+1=12 (3 from right). max=12 ✓"

# PHASE 2 — BRUTE FORCE
# "Try all combinations of i from left and k-i from right (0≤i≤k). Time: O(k). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Equivalent to finding the minimum sum subarray of length n-k (the middle we DON'T take).
# total_sum - min(subarray of length n-k). Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# cardPoints=[1,2,3,4,5,6,1],k=3: win=4,total=22.
# curr=1+2+3+4=10. i=4:curr=10+5-1=14. i=5:curr=14+6-2=18. i=6:curr=18+1-3=16.
# min_mid=10. return 22-10=12 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one sliding window pass. Space O(1).
# The 'minimum middle window' reframe is the key insight.
# Follow-up: if we must take exactly k from one end only — prefix/suffix sum comparison.
# Follow-up: circular version (wrap-around cards) — same total-minimum trick."
```


Sliding Window – Dynamic Size

#1838 Frequency of the Most Frequent Element

MED

Open on LeetCode

Array · Binary Search · Greedy · Sliding Window · Sorting · Prefix Sum

Lists: AlgoMaster 600

Problem

The frequency of an element is the number of times it occurs in an array.

You are given an integer array nums and an integer k. In one operation, you can choose an index of nums and increment the element at that index by 1.

Return the maximum possible frequency of an element after performing at most k operations.

Example 1:

```
Input: nums = [1,2,4], k = 5
Output: 3
Explanation: Increment the first element three times and the second element two times to make nums = [4,4,4].
4 has a frequency of 3.
```

Example 2:

```
Input: nums = [1,4,8,13], k = 5
Output: 2
Explanation: There are multiple optimal solutions:
- Increment the first element three times to make nums = [4,4,8,13]. 4 has a frequency of 2.
- Increment the second element four times to make nums = [1,8,8,13]. 8 has a frequency of 2.
- Increment the third element five times to make nums = [1,4,13,13]. 13 has a frequency of 2.
```

Example 3:

```
Input: nums = [3,9,6], k = 2
Output: 1
```

Constraints:

- 1 <= nums.length <= 105
- 1 <= nums[i] <= 105
- 1 <= k <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Choose one element to increment any number of times. Return max frequency achievable. Right?"
#
# "nums=[1,2,4], k=5: increment 1 twice and 2 once to make [3,3,4]? Or all to 4: costs 3+2=5.
# → 3 elements at value 4: cost=3+2+0=5=k. frequency=3 ✓"

# PHASE 2 — BRUTE FORCE
# "Sort. For each target value, count how many elements can reach it within k operations.
# Time: O(n² log n). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Sort. Sliding window: all elements in [lo..hi] can be raised to nums[hi].
# Cost = nums[hi]*(hi-lo+1) - sum(nums[lo..hi]). If cost > k, shrink left.
# Track window sum for O(1) cost update.
# Time: O(n log n). Space: O(1)."

# PHASE 4 — CLEAN CODE
        # cost to make all elements in window equal to nums[hi]

# PHASE 5 — TEST & DEBUG
# nums=[1,2,4],k=5: sorted=[1,2,4].
# hi=0: sum=1. cost=1*1-1=0<=5. best=1.
# hi=1: sum=3. cost=2*2-3=1<=5. best=2.
# hi=2: sum=7. cost=4*3-7=5<=5. best=3. return 3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log n): sort + O(n) window. Space O(1).
# Key invariant: elements in the window are sorted, so raising all to max costs nums[hi]*size - sum.
# Follow-up: if we can both increment AND decrement — target the median; classic median trick.
# Follow-up: if cost per increment varies — need a different approach (DP or binary search)."
```

Binary Search

Round 2 · High · Medium · 6 questions

Binary Search

#81 Search in Rotated Sorted Array II

MED

Open on LeetCode

Array · Binary Search

Lists: AlgoMaster 600

Problem

There is an integer array nums sorted in non-decreasing order (not necessarily with distinct values).
Before being passed to your function, nums is rotated at an unknown pivot index k (0 <= k < nums.length) such that the resulting array is [nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]] (0-indexed). For example, [0,1,2,4,4,4,5,6,6,7] might be rotated at pivot index 5 and become [4,5,6,6,7,0,1,2,4,4].
Given the array nums after the rotation and an integer target, return true if target is in nums, or false if it is not in nums.
You must decrease the overall operation steps as much as possible.

Example 1:

```
Input: nums = [2,5,6,0,0,1,2], target = 0
Output: true
```

Example 2:

```
Input: nums = [2,5,6,0,0,1,2], target = 3
Output: false
```

Constraints:

- 1 <= nums.length <= 5000
- -104 <= nums[i] <= 104
- nums is guaranteed to be rotated at some pivot.
- -104 <= target <= 104

Follow up: This problem is similar to Search in Rotated Sorted Array, but nums may contain duplicates. Would this affect the runtime complexity? How and why?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Search in a rotated sorted array that MAY have duplicates. Return true/false. Right?"
#
# "nums=[2,5,6,0,0,1,2], target=0→true ✓
# nums=[2,5,6,0,0,1,2], target=3→false ✓"

# PHASE 2 — BRUTE FORCE
# "Linear scan. Time: O(n). Space: O(1). — Simple but ignores sorted structure."

# PHASE 3 — OPTIMIZE
# "Binary search. When nums[lo]==nums[mid]==nums[hi], can't determine which side is sorted — shrink both ends.
# Otherwise standard rotated binary search: find sorted half, check if target is there.
# Time: O(log n) average, O(n) worst case (all duplicates). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[2,5,6,0,0,1,2],target=0:
# lo=0,hi=6,mid=3: nums[3]=0==target→True ✓
# nums=[2,5,6,0,0,1,2],target=3:
# mid=3(0)≠3. lo(2)<=mid(0)? No. Right sorted: mid(0)<3<=hi(2)? No→hi=2.
# mid=1(5)≠3. lo(2)<=mid(5)? Yes. lo(2)<=3<mid(5)? Yes→hi=0. lo>hi→False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Average O(log n). Worst O(n) when all elements are the same.
# LC 33 (no duplicates) is always O(log n).
# Follow-up: find the rotation index first, then two standard binary searches.
# Follow-up: if the array can have arbitrary duplicates — only O(n) guaranteed."
```

Binary Search

#162 Find Peak Element

MED

Open on LeetCode

Array · Binary Search

Lists: AlgoMaster 600

Problem

A peak element is an element that is strictly greater than its neighbors.
Given a 0-indexed integer array nums, find a peak element, and return its index. If the array contains multiple peaks, return the index to any of the peaks.
You may imagine that nums[-1] = nums[n] = -∞. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array.
You must write an algorithm that runs in O(log n) time.

Example 1:

```
Input: nums = [1,2,3,1]
Output: 2
Explanation: 3 is a peak element and your function should return the index number 2.
```

Example 2:

```
Input: nums = [1,2,1,3,5,6,4]
Output: 5
Explanation: Your function can return either index number 1 where the peak element is 2, or index number 5 where the peak element is 6.
```

Constraints:

- 1 <= nums.length <= 1000
- -231 <= nums[i] <= 231 - 1
- nums[i] != nums[i + 1] for all valid i.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "A peak is an element strictly greater than its neighbors. Find any peak index.
# nums[-1] and nums[n] are -infinity. O(log n) required. Right?"
#
# "nums=[1,2,3,1]→2 (index of 3) ✓ nums=[1,2,1,3,5,6,4]→1 or 5 ✓"

# PHASE 2 — BRUTE FORCE
# "Linear scan: return first i where nums[i]>nums[i+1]. Time: O(n). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Binary search: if nums[mid] < nums[mid+1], peak is to the right. Otherwise to the left or at mid.
# Time: O(log n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[1,2,3,1]:
# lo=0,hi=3. mid=1: nums[1]=2<nums[2]=3→lo=2. mid=2: nums[2]=3>nums[3]=1→hi=2.
# lo==hi=2. return 2 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(log n). Space O(1).
# Works because: if we go towards the ascending direction, a peak must exist there (bounded by -inf).
# Follow-up: find the global maximum — linear scan O(n) is cleaner and sufficient.
# Follow-up: Peak Index in a Mountain Array (LC 852) — same binary search on a unimodal array."
```


Binary Search

#240 Search a 2D Matrix II

MED

Open on LeetCode

Array · Binary Search · Divide and Conquer · Matrix

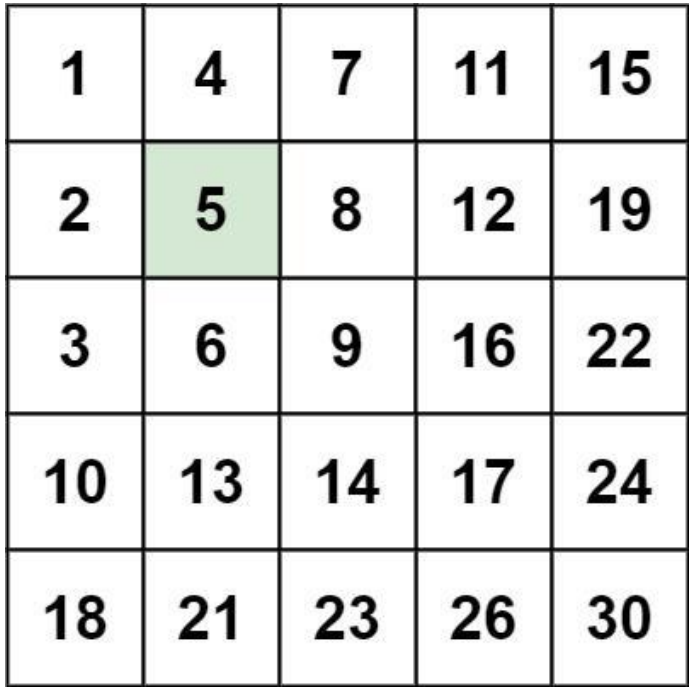
Lists: AlgoMaster 600

Problem

Write an efficient algorithm that searches for a value target in an m x n integer matrix matrix. This matrix has the following properties:

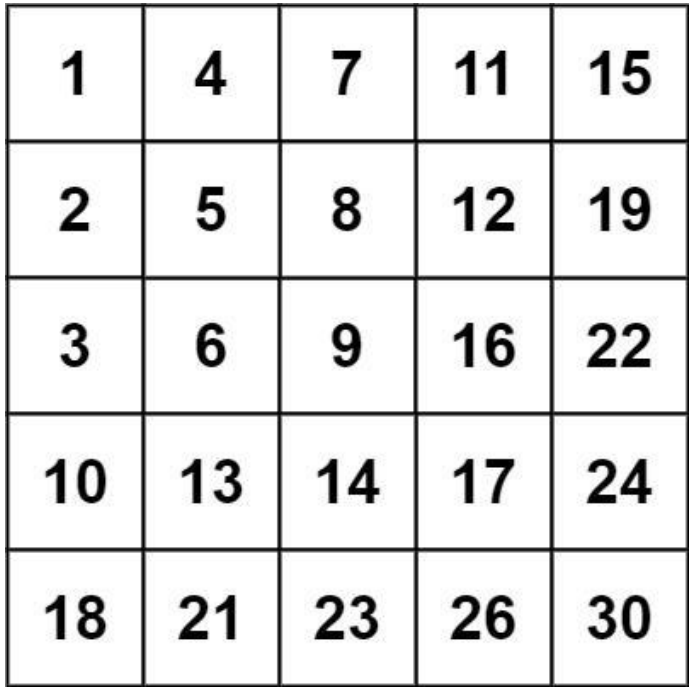
- Integers in each row are sorted in ascending from left to right.
- Integers in each column are sorted in ascending from top to bottom.

Example 1:




```
Input: matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,30]], target = 5
Output: true
```

Example 2:




```
Input: matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,30]], target = 20
Output: false
```

Constraints:

- m == matrix.length
- n == matrix[i].length
- 1 <= n, m <= 300
- -109 <= matrix[i][j] <= 109
- All the integers in each row are sorted in ascending order.
- All the integers in each column are sorted in ascending order.
- -109 <= target <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Each row sorted left-to-right, each column sorted top-to-bottom. Search for target. Right?"
#
# "matrix=[[1,4,7,11],[2,5,8,12],[3,6,9,16],[10,13,14,17]], target=5→true ✓
# target=20→false ✓"

# PHASE 2 — BRUTE FORCE
# "Linear scan all elements. Time: O(m*n). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Start at top-right corner. If current > target → move left. If < target → move down.
# If equal → found. Time: O(m+n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# target=5: start (0,3)=11>5→col=2. (0,2)=7>5→col=1. (0,1)=4<5→row=1.
# (1,1)=5==5→True ✓
# target=20: eventually row≥m or col<0 → False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m+n): at most m+n steps (each step moves row or col by 1).
# Space O(1). The staircase search exploits both row-sorted and col-sorted properties.
# Follow-up: Search a 2D Matrix I (LC 74) — stricter sorted layout; use pure binary search O(log(m*n)).
# Follow-up: count elements less than target — binary search each row O(m log n)."
```


Binary Search

#475 Heaters

ME
D

Open on LeetCode

Array · Two Pointers · Binary Search · Sorting

Lists: AlgoMaster 600

Problem

Winter is coming! During the contest, your first job is to design a standard heater with a fixed warm radius to warm all the houses.

Every house can be warmed, as long as the house is within the heater's warm radius range.

Given the positions of houses and heaters on a horizontal line, return the minimum radius standard of heaters so that those heaters could cover all houses.

Notice that all the heaters follow your radius standard, and the warm radius will be the same.

Example 1:

```
Input: houses = [1,2,3], heaters = [2]
Output: 1
Explanation: The only heater was placed in the position 2, and if we use the radius 1 standard, then all the houses can be warmed.
```

Example 2:

```
Input: houses = [1,2,3,4], heaters = [1,4]
Output: 1
Explanation: The two heaters were placed at positions 1 and 4. We need to use a radius 1 standard, then all the houses can be warmed.
```

Example 3:

```
Input: houses = [1,5], heaters = [2]
Output: 3
```

Constraints:

- 1 <= houses.length, heaters.length <= 3 * 104
- 1 <= houses[i], heaters[i] <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the minimum radius r so every house is within radius r of at least one heater. Right?"
#
# "houses=[1,2,3], heaters=[2]→1 (heater at 2 covers 1 and 3) ✓
# houses=[1,2,3,4], heaters=[1,4]→1 ✓
# houses=[1,5], heaters=[2]→3 ✓"

# PHASE 2 — BRUTE FORCE
# "For each house, find the nearest heater. Answer = max of those distances.
# Time: O(n*m). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Sort heaters. For each house, binary search for nearest heater.
# Time: O((n+m) log m). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# houses=[1,2,3],heaters=[2]: sorted=[2].
# h=1: idx=0. heaters[0]-1=1. dist=1. h=2: idx=1. heaters[0]-2=0? No idx>0→2-2=0. dist=0.
# h=3: idx=1(off end). heaters[0]-? idx>0→3-2=1. dist=1. result=max(1,0,1)=1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O((n+m) log m): sort heaters + n binary searches. Space O(1).
# The binary search finds insertion point → nearest heater is at idx or idx-1.
# Follow-up: if heaters can be placed optimally (not given) — greedy interval coverage.
# Follow-up: minimum number of heaters to cover all houses with radius r — greedy."
```

Binary Search

#1482 Minimum Number of Days to Make m Bouquets

ME D

Open on LeetCode

Array · Binary Search

Lists: AlgoMaster 600

Problem

You are given an integer array bloomDay, an integer m and an integer k.
You want to make m bouquets. To make a bouquet, you need to use k adjacent flowers from the garden.
The garden consists of n flowers, the ith flower will bloom in the bloomDay[i] and then can be used in exactly one bouquet.
Return the minimum number of days you need to wait to be able to make m bouquets from the garden. If it is impossible to make m bouquets return -1.

Example 1:

```
Input: bloomDay = [1,10,3,10,2], m = 3, k = 1
Output: 3
Explanation: Let us see what happened in the first three days. x means flower bloomed and _ means flower did not bloom in the garden.
We need 3 bouquets each should contain 1 flower.
After day 1: [x, _, _, _, _] // we can only make one bouquet.
After day 2: [x, _, _, _, x] // we can only make two bouquets.
After day 3: [x, _, x, _, x] // we can make 3 bouquets. The answer is 3.
```

Example 2:

```
Input: bloomDay = [1,10,3,10,2], m = 3, k = 2
Output: -1
Explanation: We need 3 bouquets each has 2 flowers, that means we need 6 flowers. We only have 5 flowers so it is impossible to get the needed bouquets and we return -1.
```

Example 3:

```
Input: bloomDay = [7,7,7,7,12,7,7], m = 2, k = 3
Output: 12
Explanation: We need 2 bouquets each should have 3 flowers.
Here is the garden after the 7 and 12 days:
After day 7: [x, x, x, x, _, x, x]
We can make one bouquet of the first three flowers that bloomed. We cannot make another bouquet from the last three flowers that bloomed because they are not adjacent.
After day 12: [x, x, x, x, x, x, x]
It is obvious that we can make two bouquets in different ways.
```

Constraints:

- bloomDay.length == n
- 1 <= n <= 105
- 1 <= bloomDay[i] <= 109
- 1 <= m <= 106
- 1 <= k <= n

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "bloomDay[i] = day flower i blooms. Make m bouquets each needing k adjacent flowers.
# Return minimum day, or -1 if impossible. Right?"
#
# "bloomDay=[1,10,3,10,2],m=3,k=1→3 (day3: flowers 0,2,4 bloomed→3 bouquets) ✓
# bloomDay=[1,10,3,10,2],m=3,k=2→-1 (need 6 adjacent, only 5 flowers) ✓"

# PHASE 2 — BRUTE FORCE
# "Try every day from 1 to max(bloomDay), check if m bouquets possible.
# Time: O(max_day * n). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Binary search on the answer (day). Feasibility check O(n).
# If we can make m bouquets on day d, we can also on day d+1 — monotonic property.
# Time: O(n log(max_day)). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# bloomDay=[1,10,3,10,2],m=3,k=1: lo=1,hi=10. mid=5: flowers bloomed=[1,3,2]→3 bouquets✓→hi=5.
# mid=3: flowers=[1,3,2]→3✓→hi=3. mid=2: flowers=[1,2]→2<3→lo=3. lo==hi=3. return 3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log D): D=max bloom day, log D iterations each O(n). Space O(1).
# Follow-up: if bouquets need non-adjacent flowers — remove the 'consec' tracking.
# Follow-up: Koko Eating Bananas (LC 875) — same binary-search-on-answer pattern."
```


Binary Search

#1552 Magnetic Force Between Two Balls

ME D

Open on LeetCode

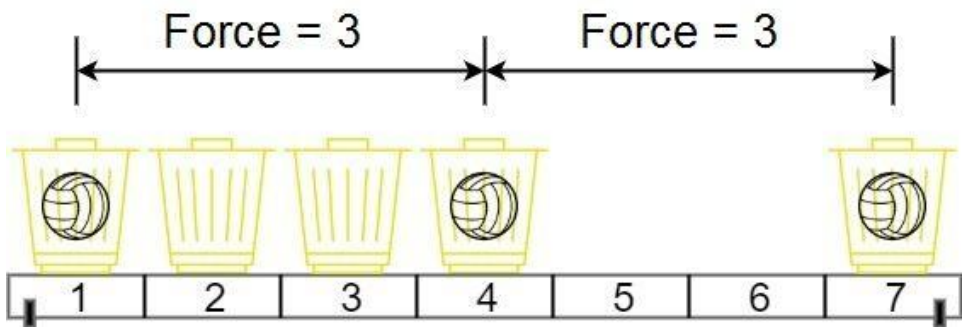
Array · Binary Search · Sorting

Lists: AlgoMaster 600

Problem

In the universe Earth C-137, Rick discovered a special form of magnetic force between two balls if they are put in his new invented basket. Rick has n empty baskets, the ith basket is at position[i], Morty has m balls and needs to distribute the balls into the baskets such that the minimum magnetic force between any two balls is maximum.
Rick stated that magnetic force between two different balls at positions x and y is |x - y|.
Given the integer array position and the integer m. Return the required force.

Example 1:




```
Input: position = [1,2,3,4,7], m = 3
Output: 3
Explanation: Distributing the 3 balls into baskets 1, 4 and 7 will make the magnetic force between ball pairs [3, 3, 6]. The minimum magnetic force is 3. We cannot achieve a larger minimum magnetic force than 3.
```

Example 2:

```
Input: position = [5,4,3,2,1,1000000000], m = 2
Output: 999999999
Explanation: We can use baskets 1 and 1000000000.
```

Constraints:

- n == position.length
- 2 <= n <= 105
- 1 <= position[i] <= 109
- All integers in position are distinct.
- 2 <= m <= position.length

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Place m balls in n positions. Maximize the minimum magnetic force (minimum distance). Right?"
#
# "position=[1,2,3,4,7], m=3: place at 1,4,7 → min dist=3 ✓"

# PHASE 2 — BRUTE FORCE
# "Try all combinations of m positions, compute min pairwise distance, maximize.
# Time: O(C(n,m) * m). Impractical."

# PHASE 3 — OPTIMIZE
# "Binary search on the minimum distance d. Feasibility: greedily place balls,
# always pick the next position that's >= d away from the last placed.
# Time: O(n log(max_gap)). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# position=[1,2,3,4,7],m=3: sorted, lo=1,hi=6. mid=4: place at 1,7(diff=6>=4) but need 3.
# 1→4(diff=3<4)→7(diff=3<4). count=2<3→hi=3. mid=2: 1→3→7? 1(count=1),3-1=2>=2(count=2),7-3=4>=2(count=3)→True→lo=2.
# mid=3: 1→4(3>=3,count=2)→7(3>=3,count=3)→True→lo=3. lo==hi=3. return 3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log(max_gap)). Space O(1).
# Note: binary search for MAXIMUM minimum → use lo=mid (upper bound search), careful with mid+1.
# Follow-up: Aggressive Cows (classic) — identical problem.
```

```
# Follow-up: if positions can change dynamically — much harder, needs segment tree."
```


· #1482 #71 · · Contents

Stacks

Round 2 · High · Medium · 6 questions

Stacks

#71 Simplify Path

MED

Open on LeetCode

String · Stack

Lists: AlgoMaster 600

Problem

You are given an absolute path for a Unix-style file system, which always begins with a slash '/'. Your task is to transform this absolute path into its simplified canonical path.

The rules of a Unix-style file system are as follows:

- A single period '.' represents the current directory.
- A double period '..' represents the previous/parent directory.
- Multiple consecutive slashes such as '//' and '///' are treated as a single slash '/'.
- Any sequence of periods that does not match the rules above should be treated as a valid directory or file name. For example, '...' and '....' are valid directory or file names.

The simplified canonical path should follow these rules:

- The path must start with a single slash '/'.
- Directories within the path must be separated by exactly one slash '/'.
- The path must not end with a slash '/', unless it is the root directory.
- The path must not have any single or double periods ('.' and '..') used to denote current or parent directories.

Return the simplified canonical path.

Example 1:
Input: path = "/home/"
Output: "/home"
Explanation:
The trailing slash should be removed.

Example 2:
Input: path = "/home//foo/"
Output: "/home/foo"
Explanation:
Multiple consecutive slashes are replaced by a single one.

Example 3:
Input: path = "/home/user/Documents/../Pictures"
Output: "/home/user/Pictures"
Explanation:
A double period ".." refers to the directory up a level (the parent directory).

Example 4:
Input: path = "/../"
Output: "/"
Explanation:
Going one level up from the root directory is not possible.

Example 5:
Input: path = "/.../a/../b/c/../d/./"
Output: "/.../b/d"
Explanation:
"..." is a valid name for a directory in this problem.

Constraints:

- 1 <= path.length <= 3000
- path consists of English letters, digits, period '.', slash '/' or '_'.
- path is a valid absolute Unix path.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Simplify a Unix-style absolute path. '..'=go up, '.'=current dir, multiple slashes=one. Right?"
#
# "/home//../foo/.//bar/→'/foo/bar' ✓
# "/../→'/' ✓ '/home//foo/→'/home/foo' ✓"

# PHASE 2 — BRUTE FORCE
# "Split on '/', process each component: skip '' and '.', pop stack for '..', push otherwise.
# Join with '/'. Time: O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Same stack approach — already O(n). The split/filter/stack is the optimal solution."

# PHASE 4 — CLEAN CODE (same as brute — already clean and optimal)
```



```
# PHASE 5 — TEST & DEBUG
# '/home//../foo/.//bar/': tokens=['','home','','..','foo','.','','bar',''].
# 'home'→push. '..'→pop. 'foo'→push. '.'→skip. 'bar'→push. stack=['foo','bar'].
# →'/foo/bar' ✓
# '/../': '..'→stack empty, ignore. →'/' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): split + one pass. Space O(n): stack.
# Follow-up: if the path can be relative (not starting with '/') — handle empty stack for '..'.
# Follow-up: reconstruct original from simplified — impossible (information lost)."
```

Stacks

#316 Remove Duplicate Letters

MED

Open on LeetCode

String · Stack · Greedy · Monotonic Stack

Lists: AlgoMaster 600

Problem

Given a string s, remove duplicate letters so that every letter appears once and only once. You must make sure your result is the smallest in lexicographical order among all possible results.

Example 1:

```
Input: s = "bcabc"
Output: "abc"
```

Example 2:

```
Input: s = "cbacdcbc"
Output: "acdb"
```

Constraints:

- 1 <= s.length <= 104
- s consists of lowercase English letters.

Note: This question is the same as 1081: https://leetcode.com/problems/smallest-subsequence-of-distinct-characters/

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Remove duplicate letters so each appears once and the result is the smallest lexicographic
# subsequence. Right?"
#
# "s='bcabc'→'abc' ✓ s='cbacdcbc'→'acdb' ✓"

# PHASE 2 — BRUTE FORCE
# "Generate all subsequences with each letter appearing once, return lexicographically smallest.
# Exponential — impractical."
        # Greedy: find leftmost char we can commit to being first
        # Use the stack approach as the 'optimal brute'

# PHASE 3 — OPTIMIZE
# "Monotonic stack: track remaining count per char. For each char:
# If already in stack, skip. Else pop stack top if it's lexicographically larger AND appears later.
# Time: O(n). Space: O(26)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='cbacdcbc': count={c:4,b:2,a:1,d:1}.
# c: stack=[c],seen={c}. b: b<c and count[c]=3>0→pop c. stack=[b],seen={b}. a<b,count[b]=1>0→pop b. stack=[a],seen={a}.
# c: stack=[a,c]. d: stack=[a,c,d]. c: in_seen. b: b<d,count[d]=0→stop. b<c? no wait d>b and count[d]=0 so don't pop.
# Actually b<d and count[d]=0 so we can't pop d. stack=[a,c,d,b]. return 'acdb' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): each char pushed/popped at most once. Space O(26)=O(1).
# Smallest Subsequence of Distinct Characters (LC 1081) — identical problem.
# Follow-up: if characters have weights (not lexicographic) — same stack, custom comparison.
# Follow-up: remove k characters to minimize (LC 402) — remove k chars, don't need all distinct."
```


Stacks

#636 Exclusive Time of Functions

MED

Open on LeetCode

Array · Stack

Lists: AlgoMaster 600

Problem

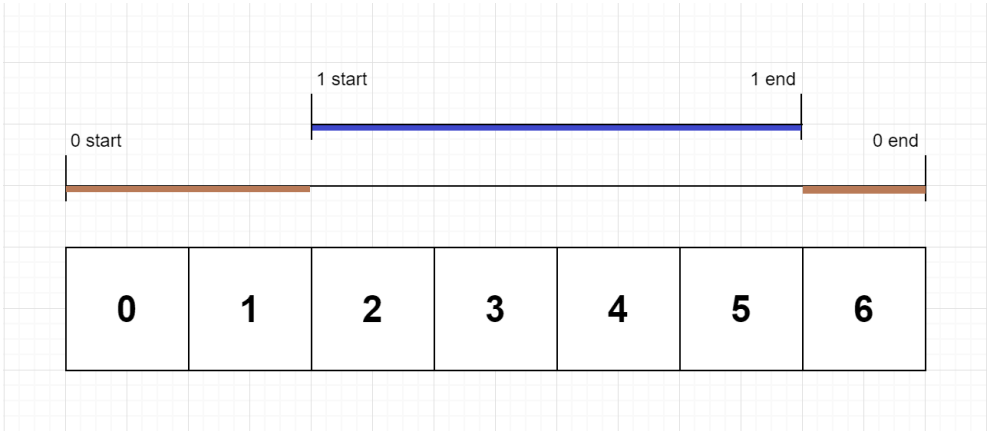
On a single-threaded CPU, we execute a program containing n functions. Each function has a unique ID between 0 and n-1. Function calls are stored in a call stack: when a function call starts, its ID is pushed onto the stack, and when a function call ends, its ID is popped off the stack. The function whose ID is at the top of the stack is the current function being executed. Each time a function starts or ends, we write a log with the ID, whether it started or ended, and the timestamp.

You are given a list logs, where logs[i] represents the ith log message formatted as a string "{function_id}:{"start" | "end"}:{timestamp}". For example, "0:start:3" means a function call with function ID 0 started at the beginning of timestamp 3, and "1:end:2" means a function call with function ID 1 ended at the end of timestamp 2. Note that a function can be called multiple times, possibly recursively.

A function's exclusive time is the sum of execution times for all function calls in the program. For example, if a function is called twice, one call executing for 2 time units and another call executing for 1 time unit, the exclusive time is 2 + 1 = 3.

Return the exclusive time of each function in an array, where the value at the ith index represents the exclusive time for the function with ID i.

Example 1:




```
Input: n = 2, logs = ["0:start:0","1:start:2","1:end:5","0:end:6"]
Output: [3,4]
Explanation:
Function 0 starts at the beginning of time 0, then it executes 2 for units of time and reaches the end of time 1.
Function 1 starts at the beginning of time 2, executes for 4 units of time, and ends at the end of time 5.
Function 0 resumes execution at the beginning of time 6 and executes for 1 unit of time.
So function 0 spends 2 + 1 = 3 units of total time executing, and function 1 spends 4 units of total time executing.
```

Example 2:

```
Input: n = 1, logs = ["0:start:0","0:start:2","0:end:5","0:start:6","0:end:6","0:end:7"]
Output: [8]
Explanation:
Function 0 starts at the beginning of time 0, executes for 2 units of time, and recursively calls itself.
Function 0 (recursive call) starts at the beginning of time 2 and executes for 4 units of time.
Function 0 (initial call) resumes execution then immediately calls itself again.
Function 0 (2nd recursive call) starts at the beginning of time 6 and executes for 1 unit of time.
Function 0 (initial call) resumes execution at the beginning of time 7 and executes for 1 unit of time.
```

Example 3:

```
Input: n = 2, logs = ["0:start:0","0:start:2","0:end:5","1:start:6","1:end:6","0:end:7"]
Output: [7,1]
Explanation:
Function 0 starts at the beginning of time 0, executes for 2 units of time, and recursively calls itself.
Function 0 (recursive call) starts at the beginning of time 2 and executes for 4 units of time.
Function 0 (initial call) resumes execution then immediately calls function 1.
Function 1 starts at the beginning of time 6, executes 1 unit of time, and ends at the end of time 6.
Function 0 resumes execution at the beginning of time 6 and executes for 2 units of time.
```

Constraints:

- 1 <= n <= 100
- 2 <= logs.length <= 500
- 0 <= function_id < n
- 0 <= timestamp <= 109
- No two start events will happen at the same timestamp.
- No two end events will happen at the same timestamp.
- Each function has an "end" log for each "start" log.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "n functions. Logs: 'id:start:time' or 'id:end:time'. Return exclusive time of each function.
# Exclusive time = total time spent executing only that function (not called subfunctions). Right?"
#
# "n=2, logs=['0:start:0','1:start:2','1:end:5','0:end:6']:
# func0: [0,1] + [6,6] = 2+1=3. func1: [2,5]=4. → [3,4] ✓"

# PHASE 2 — BRUTE FORCE
# "Stack-based simulation: push on start, pop on end, add duration.
# When a new function starts or ends, record the time spent on the current top.
# Time: O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Same O(n) stack approach — already optimal."

# PHASE 4 — CLEAN CODE (same as brute — already clean)

# PHASE 5 — TEST & DEBUG
# logs=['0:start:0','1:start:2','1:end:5','0:end:6']:
# '0:start:0': stack=[0], prev=0.
# '1:start:2': result[0]+=2-0=2. stack=[0,1], prev=2.
# '1:end:5': result[1]+=5-2=3. pop→stack=[0]. result[1]+=1=4. prev=6.
# '0:end:6': result[0]+=6-6=0. pop. result[0]+=1=3. → [3,4] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass. Space O(d): d=max call depth.
# The 'prev' pointer avoids tracking absolute timestamps — only deltas matter.
# Follow-up: if logs can arrive out of order — sort by timestamp first.
# Follow-up: recursive call graph visualization — same stack, build adjacency."
```


Stacks

#946 Validate Stack Sequences

ME
D

Open on LeetCode

Array · Stack · Simulation

Lists: AlgoMaster 600

Problem

Given two integer arrays pushed and popped each with distinct values, return true if this could have been the result of a sequence of push and pop operations on an initially empty stack, or false otherwise.

Example 1:

```
Input: pushed = [1,2,3,4,5], popped = [4,5,3,2,1]
Output: true
Explanation: We might do the following sequence:
push(1), push(2), push(3), push(4),
pop() -> 4,
push(5),
pop() -> 5, pop() -> 3, pop() -> 2, pop() -> 1
```

Example 2:

```
Input: pushed = [1,2,3,4,5], popped = [4,3,5,1,2]
Output: false
Explanation: 1 cannot be popped before 2.
```

Constraints:

- 1 <= pushed.length <= 1000
- 0 <= pushed[i] <= 1000
- All the elements of pushed are unique.
- popped.length == pushed.length
- popped is a permutation of pushed.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Given pushed and popped sequences, return true if they could result from a valid stack push/pop sequence. Right?"
#
# "pushed=[1,2,3,4,5], popped=[4,5,3,2,1]→true ✓
# pushed=[1,2,3,4,5], popped=[4,3,5,1,2]→false ✓"

# PHASE 2 — BRUTE FORCE
# "Simulate: push elements one by one; whenever the stack top matches the next pop value, pop it.
# If all pops happen correctly, return true. Time: O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Same simulation — already O(n) O(n). Can reuse pushed array as stack to save space."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# pushed=[1,2,3,4,5],popped=[4,5,3,2,1]:
# push1,2,3,4. top=4==popped[0]→pop,j=1. top=3≠5. push5. top=5==popped[1]→pop,j=2.
# top=3==popped[2]→pop,j=3. top=2==popped[3]→pop,j=4. top=1==popped[4]→pop,j=5. return True ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n): stack. Each element pushed/popped at most once.
# Follow-up: if pushed order can vary — check all orderings (exponential).
# Follow-up: minimum swaps to make sequences valid — different problem requiring sorting."
```

Stacks

#1249 Minimum Remove to Make Valid Parentheses

MED

Open on LeetCode

String · Stack

Lists: AlgoMaster 600

Problem

Given a string s of '(' , ')' and lowercase English characters.

Your task is to remove the minimum number of parentheses ('(' or ')', in any positions) so that the resulting parentheses string is valid and return any valid string.

Formally, a parentheses string is valid if and only if:

- It is the empty string, contains only lowercase characters, or
- It can be written as AB (A concatenated with B), where A and B are valid strings, or
- It can be written as (A), where A is a valid string.

Example 1:

```
Input: s = "lee(t(c)o)de)"
Output: "lee(t(c)o)de"
Explanation: "lee(t(co)de)" , "lee(t(c)ode)" would also be accepted.
```

Example 2:

```
Input: s = "a)b(c)d"
Output: "ab(c)d"
```

Example 3:

```
Input: s = "))(("
Output: ""
Explanation: An empty string is also valid.
```

Constraints:

- 1 <= s.length <= 105
- s[i] is either '(' , ')', or lowercase English letter.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Remove minimum number of parentheses to make the string valid. Return any valid result. Right?"
#
# "s='lee(t(c)o)de)'→'lee(t(c)o)de' ✓
# s='a)b(c)d'→'ab(c)d' ✓ s='))((': →'' ✓"

# PHASE 2 — BRUTE FORCE
# "Try all subsets of parentheses to remove, return shortest valid result. Exponential."
       # Stack approach is already O(n) — use directly

# PHASE 3 — OPTIMIZE
# "Same O(n) stack approach — already optimal. Track indices of unmatched parens."

# PHASE 4 — CLEAN CODE (same as brute — already clean)

# PHASE 5 — TEST & DEBUG
# s='lee(t(c)o)de)':
# i=3'(': stack=[3]. i=5'(': stack=[3,5]. i=7')': pop→stack=[3]. i=9')': pop→stack=[].
# i=12')': stack empty→remove={12}. removeu=set([])={12}.
# Result: remove index 12 → 'lee(t(c)o)de' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n): stack + remove set.
# Follow-up: if we need the lexicographically smallest result — more complex.
# Follow-up: Valid Parentheses (LC 20) — check validity without removing.
# Follow-up: Minimum Add to Make Valid (LC 921) — count minimum additions instead."
```


Stacks

#2390 Removing Stars From a String

MED

Open on LeetCode

String · Stack · Simulation

Lists: AlgoMaster 600

Problem

You are given a string s, which contains stars *.

In one operation, you can:

- Choose a star in s.
- Remove the closest non-star character to its left, as well as remove the star itself.

Return the string after all stars have been removed.

Note:

- The input will be generated such that the operation is always possible.
- It can be shown that the resulting string will always be unique.

Example 1:

```
Input: s = "leet**cod*e"
Output: "lecoe"
Explanation: Performing the removals from left to right:
- The closest character to the 1st star is 't' in "leet**cod*e". s becomes "lee*cod*e".
- The closest character to the 2nd star is 'e' in "lee*cod*e". s becomes "lecod*e".
- The closest character to the 3rd star is 'd' in "lecod*e". s becomes "lecoe".
There are no more stars, so we return "lecoe".
```

Example 2:

```
Input: s = "erase*****"
Output: ""
Explanation: The entire string is removed, so we return an empty string.
```

Constraints:

- 1 <= s.length <= 105
- s consists of lowercase English letters and stars *.
- The operation above can be performed on s.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Each '*' removes the closest non-star character to its left. Return the result. Right?"
#
# "s='leet**cod*e'→'lecoe' ✓ s='erase*****'→'' ✓"

# PHASE 2 — BRUTE FORCE
# "Repeatedly find first '*', remove it and the char before it. Time: O(n²). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Stack: push non-star chars; on '*', pop the top. One pass. Time: O(n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='leet**cod*e':
# l→[l]. e→[l,e]. e→[l,e,e]. t→[l,e,e,t]. *→pop→[l,e,e]. *→pop→[l,e].
# c→[l,e,c]. o→[l,e,c,o]. d→[l,e,c,o,d]. *→pop→[l,e,c,o]. e→[l,e,c,o,e]. → 'lecoe' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n): stack.
# Follow-up: if '*' removes k characters to the left — pop k times.
# Follow-up: count minimum '*' needed to empty the string — every non-star needs one '*' to follow."
```

Tree Traversal – Level Order

Round 2 · High · Medium · 3 questions

Tree Traversal – Level Order

#116 Populating Next Right Pointers in Each Node

MED

Open on LeetCode

Linked List · Tree · Depth-First Search · Breadth-First Search · Binary Tree

Lists: AlgoMaster 600

Problem

You are given a perfect binary tree where all leaves are on the same level, and every parent has two children. The binary tree has the following definition:

```
struct Node {
int val;
Node *left;
Node *right;
Node *next;
}
```

Populate each next pointer to point to its next right node. If there is no next right node, the next pointer should be set to NULL.

Initially, all next pointers are set to NULL.

Example 1:

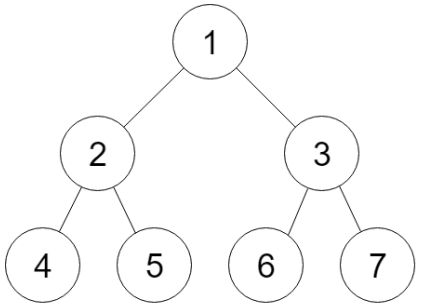


Figure A

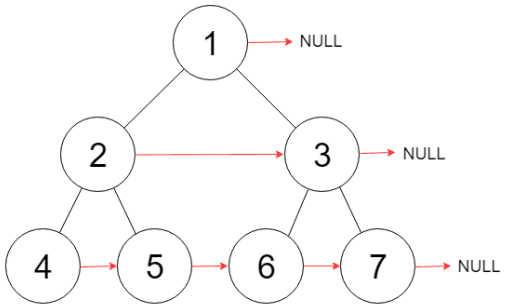


Figure B

```
Input: root = [1,2,3,4,5,6,7]
Output: [1,#,2,3,#,4,5,6,7,#]
Explanation: Given the above perfect binary tree (Figure A), your function should populate each next pointer to point to its next right node, just like in Figure B. The serialized output is in level order as connected by the next pointers, with '#' signifying the end of each level.
```

Example 2:

```
Input: root = []
Output: []
```

Constraints:

- The number of nodes in the tree is in the range [0, 212 – 1].
- -1000 <= Node.val <= 1000

Follow-up:

- You may only use constant extra space.
- The recursive approach is fine. You may assume implicit stack space does not count as extra space for this problem.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Connect each node's 'next' pointer to the next right node on the same level.
# Perfect binary tree. Use O(1) extra space. Right?"
#
# "tree=[1,2,3,4,5,6,7]: 1→null, 2→3, 3→null, 4→5, 5→6, 6→7, 7→null ✓"

# PHASE 2 — BRUTE FORCE
# "BFS: process each level, connect consecutive nodes. Time: O(n). Space: O(w) queue."

# PHASE 3 — OPTIMIZE
# "O(1) space: use already-established 'next' pointers to traverse each level.
# For each node, connect its children; then use node.next to move to next node on same level.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
```

```
# tree=[1,2,3,4,5,6,7]:
# Level1: curr=1. 1.left(2).next=1.right(3). 1.next=None→stop. leftmost=2.
# Level2: curr=2. 2.left(4).next=2.right(5). 2.next=3→3.right(7).next? Wait: 3.left(6).next=3.right(7). ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1): only curr and leftmost pointers.
# Follow-up: Populating Next Right Pointers II (LC 117) — non-perfect tree.
# Use a dummy head per level to handle missing children gracefully."
```


Tree Traversal – Level Order

#117 Populating Next Right Pointers in Each Node II

ME D

Open on LeetCode

Linked List · Tree · Depth-First Search · Breadth-First Search · Binary Tree

Lists: AlgoMaster 600

Problem

Given a binary tree

```
struct Node {
int val;
Node *left;
Node *right;
Node *next;
}
```

Populate each next pointer to point to its next right node. If there is no next right node, the next pointer should be set to NULL.

Initially, all next pointers are set to NULL.

Example 1:

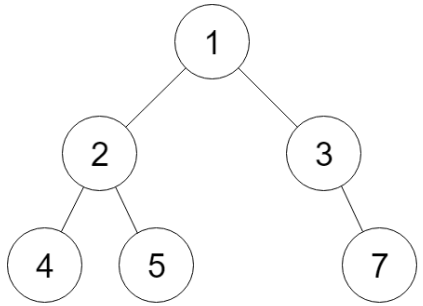


Figure A

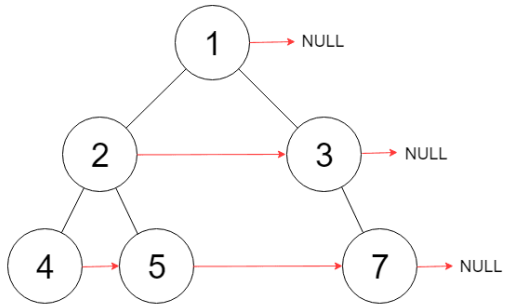


Figure B

```
Input: root = [1,2,3,4,5,null,7]
Output: [1,#,2,3,#,4,5,7,#]
Explanation: Given the above binary tree (Figure A), your function should populate each next pointer to point to its next right node, just like in Figure B. The serialized output is in level order as connected by the next pointers, with '#' signifying the end of each level.
```

Example 2:

```
Input: root = []
Output: []
```

Constraints:

- **The number of nodes in the tree is in the range [0, 6000].**
- **-100 <= Node.val <= 100**

Follow-up:

- **You may only use constant extra space.**
- **The recursive approach is fine. You may assume implicit stack space does not count as extra space for this problem.**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Connect next pointers for an arbitrary binary tree (not necessarily perfect). O(1) extra space. Right?"
#
# "tree=[1,2,3,4,5,null,7]: 2→3, 4→5→7 ✓ (skip missing nodes)"
# PHASE 2 — BRUTE FORCE
# "BFS level by level, connect consecutive nodes. Time: O(n). Space: O(w) queue."
# PHASE 3 — OPTIMIZE
# "O(1) space using established next pointers as level traversal.
# Use a dummy head per level to chain children. Move through current level via next.
# Time: O(n). Space: O(1)."
# PHASE 4 — CLEAN CODE
# PHASE 5 — TEST & DEBUG
# tree=[1,2,3,4,5,null,7]:
# Level1: curr=1. dummy→2→3. curr=dummy.next=2.
```

```
# Level2: curr=2: push 4,5. curr=3: push 7. dummy→4→5→7. curr=4.
# Level3: curr=4,5,7 no children. → 4→5→7 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1): dummy pointer reused each level.
# The dummy head avoids special-casing the first child of each level.
# Follow-up: Populating Next Right I (LC 116) — perfect tree; simpler O(1) space without dummy.
# Follow-up: if tree has cycles — detect and break before connecting."
```


Tree Traversal – Level Order

#958 Check Completeness of a Binary Tree

ME
D

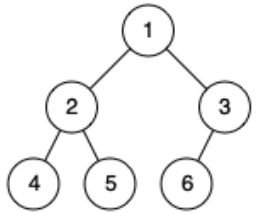
Open on LeetCode

Tree · Breadth-First Search · Binary Tree

Lists: AlgoMaster 600

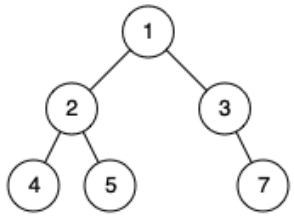
Problem

Given the root of a binary tree, determine if it is a complete binary tree.
In a complete binary tree, every level, except possibly the last, is completely filled, and all nodes in the last level are as far left as possible. It can have between 1 and 2h nodes inclusive at the last level h.
Example 1:




```
Input: root = [1,2,3,4,5,6]
Output: true
Explanation: Every level before the last is full (ie. levels with node-values {1} and {2, 3}), and all nodes in the last level
({4, 5, 6}) are as far left as possible.
```

Example 2:




```
Input: root = [1,2,3,4,5,null,7]
Output: false
Explanation: The node with value 7 isn't as far left as possible.
```

Constraints:

- **The number of nodes in the tree is in the range [1, 100].**
- **1 <= Node.val <= 1000**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return true if the binary tree is complete — all levels fully filled except possibly the last,
# which has all nodes as far left as possible. Right?"
#
# "tree=[1,2,3,4,5,6]→true ✓ tree=[1,2,3,4,5,null,7]→false (gap before 7) ✓"

# PHASE 2 — BRUTE FORCE
# "BFS: once we see a null child, all subsequent nodes should also be null.
# Time: O(n). Space: O(w)."

# PHASE 3 — OPTIMIZE
# "Same BFS — already O(n). The null-sentinel check is the key pattern."

# PHASE 4 — CLEAN CODE (same as brute — already clean)

# PHASE 5 — TEST & DEBUG
# tree=[1,2,3,4,5,null,7]:
# BFS: 1,2,3,4,5,None,7.
# See None (3.left=None)→found_null=True. Next: 7≠None but found_null→return False ✓
# tree=[1,2,3,4,5,6]:
```

```
# BFS: 1,2,3,4,5,6,None,None,None,None,None,None,None. All non-null before nulls → True ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(w): queue width.
# Follow-up: Count Complete Tree Nodes (LC 222) — use completeness property for O(log²n).
# Follow-up: if tree nodes have indices — check if max index == total count (perfect BFS indexing)."
```


· #117 #106 · · Contents

Tree Traversal – Pre Order

Round 2 · High · Medium · 3 questions

Tree Traversal – Pre Order

#106 Construct Binary Tree from Inorder and Postorder Traversal

MED

Open on LeetCode

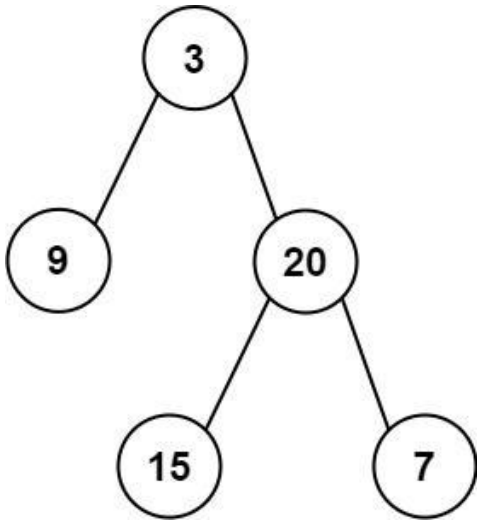
Array · Hash Table · Divide and Conquer · Tree · Binary Tree

Lists: AlgoMaster 600

Problem

Given two integer arrays inorder and postorder where inorder is the inorder traversal of a binary tree and postorder is the postorder traversal of the same tree, construct and return the binary tree.

Example 1:




```
Input: inorder = [9,3,15,20,7], postorder = [9,15,7,20,3]
Output: [3,9,20,null,null,15,7]
```

Example 2:

```
Input: inorder = [-1], postorder = [-1]
Output: [-1]
```

Constraints:

- 1 <= inorder.length <= 3000
- postorder.length == inorder.length
- -3000 <= inorder[i], postorder[i] <= 3000
- inorder and postorder consist of unique values.
- Each value of postorder also appears in inorder.
- inorder is guaranteed to be the inorder traversal of the tree.
- postorder is guaranteed to be the postorder traversal of the tree.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given inorder and postorder traversal arrays, reconstruct the binary tree. Right?"
#
# "inorder=[9,3,15,20,7], postorder=[9,15,7,20,3]:
# root=3(last of postorder). Left subtree=[9], right=[15,20,7]. ✓"

# PHASE 2 — BRUTE FORCE
# "Recursion: last element of postorder = root. Find in inorder to split left/right.
# Time: O(n²) — indexOf is O(n). Space: O(n) recursion."

# PHASE 3 — OPTIMIZE
# "Precompute inorder index map for O(1) lookup. Use index pointers instead of slicing.
# Time: O(n). Space: O(n) for map."

# PHASE 4 — CLEAN CODE
```



```
# PHASE 5 — TEST & DEBUG
# inorder=[9,3,15,20,7],postorder=[9,15,7,20,3]:
# root=3(idx=1). right=build(2,4):root=20(idx=3). right=build(4,4):root=7.
# left=build(2,2):root=15. left=build(0,0):root=9. Tree=3(9,20(15,7)) ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): O(n) to build map + O(n) recursive calls. Space O(n): map + recursion.
# Note: right built before left because postorder processes left-right-root (root last).
# Follow-up: Build from Preorder+Inorder (LC 105) — same idea, process preorder left-to-right.
# Follow-up: Build from Preorder+Postorder (LC 889) — not unique without inorder."
```

Tree Traversal – Pre Order

#687 Longest Univalue Path

MED

Open on LeetCode

Tree · Depth-First Search · Binary Tree

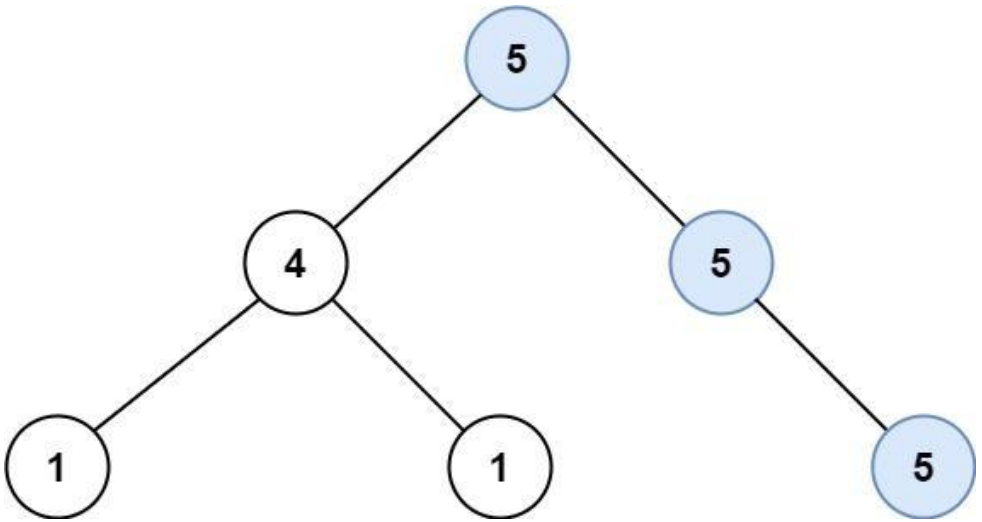
Lists: AlgoMaster 600

Problem

Given the root of a binary tree, return the length of the longest path, where each node in the path has the same value. This path may or may not pass through the root.

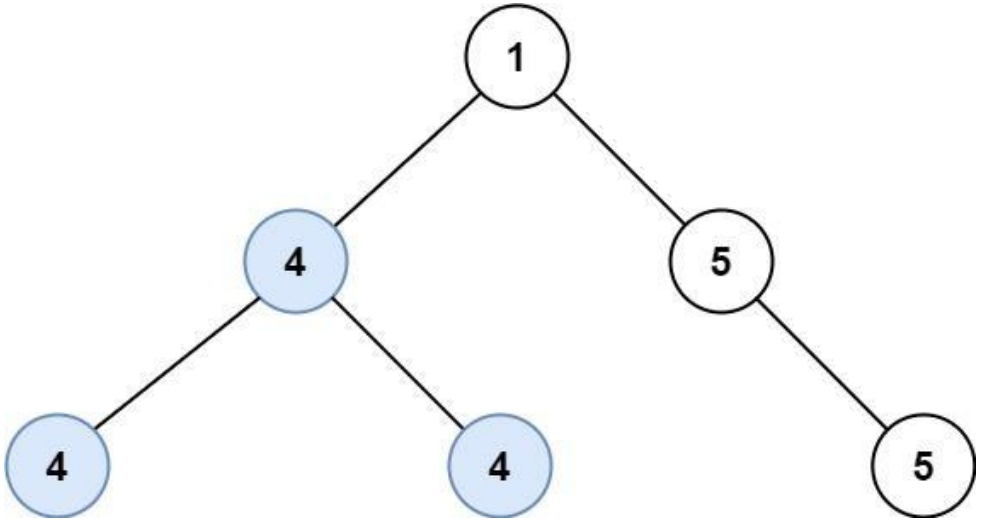
The length of the path between two nodes is represented by the number of edges between them.

Example 1:




```
Input: root = [5,4,5,1,1,null,5]
Output: 2
Explanation: The shown image shows that the longest path of the same value (i.e. 5).
```

Example 2:




```
Input: root = [1,4,5,4,4,null,5]
Output: 2
Explanation: The shown image shows that the longest path of the same value (i.e. 4).
```

Constraints:

- The number of nodes in the tree is in the range [0, 104].
- -1000 <= Node.val <= 1000
- The depth of the tree will not exceed 1000.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return the length of the longest path where all nodes have the same value.
# Path can go through parent-child connections, not necessarily through the root. Right?"
#
# "tree=[5,4,5,1,1,null,5]: path 5→5→5 length=2 ✓"

# PHASE 2 — BRUTE FORCE
# "For each node as root of a path, find longest same-value extension in each direction.
# Time: O(n²). Space: O(h)."

# PHASE 3 — OPTIMIZE
# "DFS returning the longest extending arm of same value in each direction.
# At each node: left_arm if left.val==node.val else 0; same for right.
# Update global max with left_arm + right_arm. Return 1 + max(arms) to parent.
# Time: O(n). Space: O(h)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# tree=[5,4,5,1,1,null,5]:
# dfs(1_left)=0, dfs(1_right)=0. dfs(4): 4≠1,4≠1→arms=0. return 0.
# dfs(5_right)=0. dfs(5_leaf): no children→0. return max(0,0)=0?
# Actually 5(root).right=5: dfs(5_r)=0 (no children). l_arm=(5==5→1), r_arm=(5==5→1). best=2. ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): each node visited once. Space O(h): recursion.
# Follow-up: Diameter of Binary Tree (LC 543) — same structure but ignores node values.
# Follow-up: longest path with values differing by exactly 1 — check |node.val - child.val|==1."
```

Tree Traversal – Pre Order

#1026 Maximum Difference Between Node and Ancestor

MED

Open on LeetCode

Tree · Depth-First Search · Binary Tree

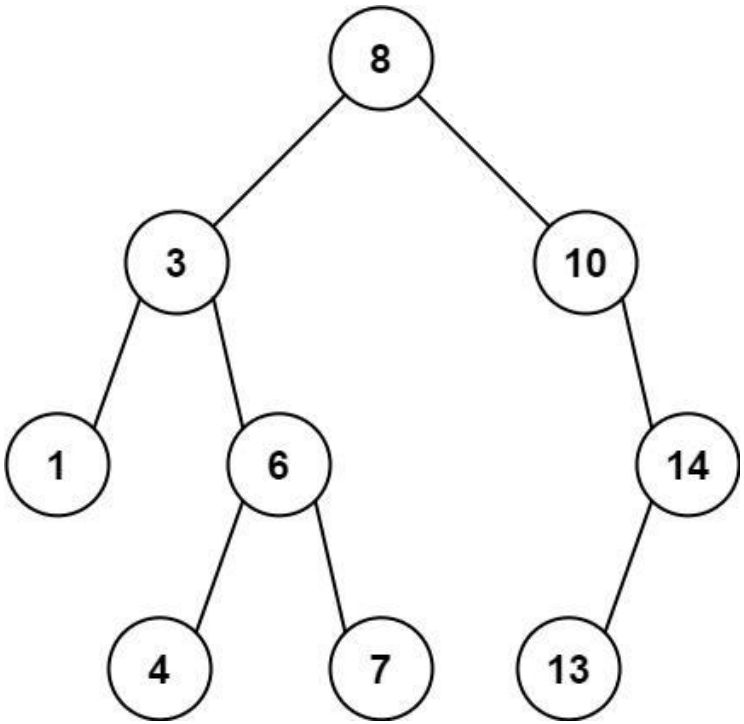
Lists: AlgoMaster 600

Problem

Given the root of a binary tree, find the maximum value v for which there exist different nodes a and b where v = |a.val − b.val| and a is an ancestor of b.

A node a is an ancestor of b if either: any child of a is equal to b or any child of a is an ancestor of b.

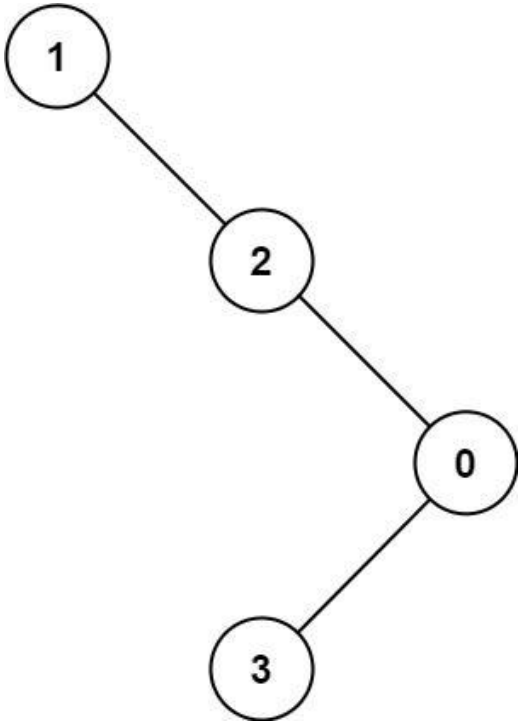
Example 1:




```
Input: root = [8,3,10,1,6,null,14,null,null,4,7,13]
Output: 7
Explanation: We have various ancestor-node differences, some of which are given below :
|8 - 3| = 5
|3 - 7| = 4
|8 - 1| = 7
|10 - 13| = 3
Among all possible differences, the maximum value of 7 is obtained by |8 - 1| = 7.
```

Example 2:

· #687 #1382 · · Contents




```
Input: root = [1,null,2,null,0,3]
Output: 3
```

Constraints:

- The number of nodes in the tree is in the range [2, 5000].
- 0 <= Node.val <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find maximum v where |a - b| = v for some ancestor node a and descendant node b. Right?"
#
# "tree=[8,3,10,1,6,null,14,null,null,4,7,13]: max diff = |8-1|=7 or |10-13|=3 or...
# Actually |3-7|? no. |8-1|=7 ✓"

# PHASE 2 — BRUTE FORCE
# "DFS passing all ancestor values, check |node.val - ancestor.val| for each. Time: O(n*h). Space: O(h)."

# PHASE 3 — OPTIMIZE
# "Track min and max ancestor values along the path. |node.val - ancestor| is max when ancestor
# is either the min or max ancestor. Return max(|node-min_anc|, |node-max_anc|).
# Time: O(n). Space: O(h)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# tree=[8,3,10,1,6,null,14,null,null,4,7,13]:
# dfs(8,8,8). dfs(3,3,8). dfs(1,1,8): leaf→8-1=7. dfs(6,3,8). dfs(4,3,8): leaf→8-3=5.
# dfs(7,3,8): leaf→8-3=5. dfs(10,8,10). dfs(14,8,14). dfs(13,8,14): leaf→14-8=6.
# max=7 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(h): recursion.
# At each leaf, mx-mn is the max possible |ancestor-descendant| diff on that path.
# Follow-up: find the actual (ancestor, descendant) pair — track which nodes gave min/max.
# Follow-up: if we only count direct parent-child differences — simpler DFS."
```

Tree Traversal – In Order

Round 2 · High · Medium · 1 question

Tree Traversal – In Order

#1382 Balance a Binary Search Tree

MED

Open on LeetCode

Divide and Conquer · Greedy · Tree · Depth-First Search · Binary Search Tree · Binary Tree

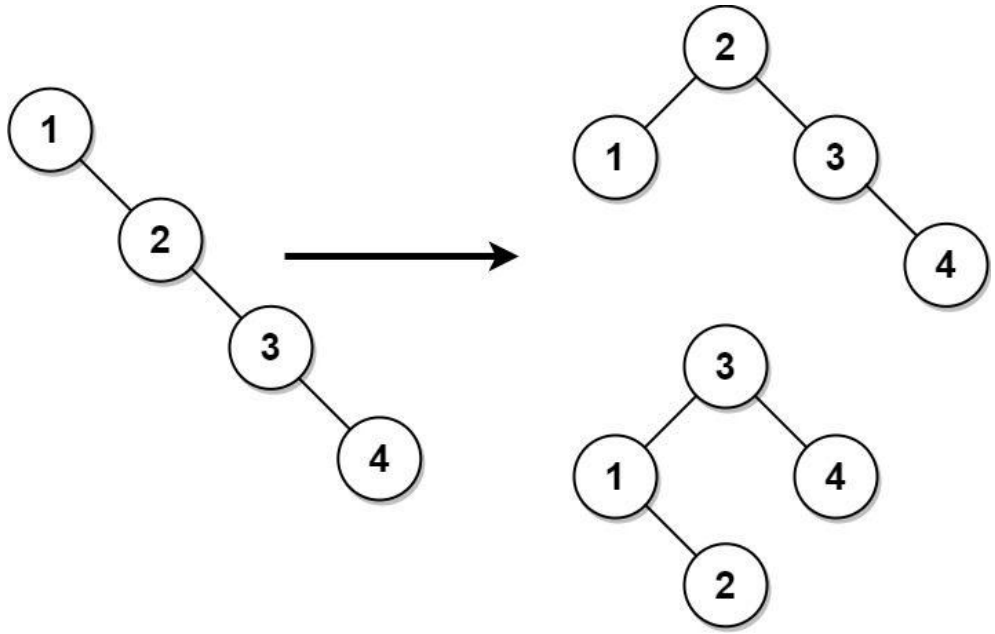
Lists: AlgoMaster 600

Problem

Given the root of a binary search tree, return a balanced binary search tree with the same node values. If there is more than one answer, return any of them.

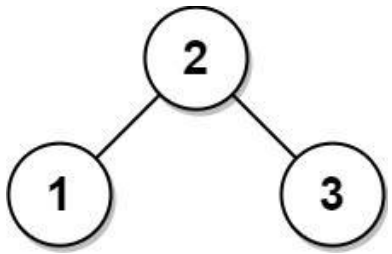
A binary search tree is balanced if the depth of the two subtrees of every node never differs by more than 1.

Example 1:




```
Input: root = [1,null,2,null,3,null,4,null,null]
Output: [2,1,3,null,null,null,4]
Explanation: This is not the only correct answer, [3,1,4,null,2] is also correct.
```

Example 2:




```
Input: root = [2,1,3]
Output: [2,1,3]
```

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- 1 <= Node.val <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Convert a BST to a height-balanced BST. Return any valid result. Right?"
#
# "tree=[1,null,2,null,3,null,4]: unbalanced. → [2,1,3,null,null,null,4] or similar ✓"

# PHASE 2 — BRUTE FORCE
# "Inorder traversal → sorted array → recursively build balanced BST from middle.
# Time: O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Same O(n) — already optimal. The two-step (inorder + rebuild) is the canonical solution."

# PHASE 4 — CLEAN CODE (same as brute — already clean and optimal)

# PHASE 5 — TEST & DEBUG
# tree=[1,null,2,null,3,null,4]: inorder=[1,2,3,4].
# build(0,3): mid=1→node(2). left=build(0,0)=node(1). right=build(2,3):mid=2→node(3).right=node(4).
# → balanced BST with root=2 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): inorder + rebuild. Space O(n): vals array.
# Follow-up: balance without extra space — Day-Stout-Warren (DSW) algorithm O(n) O(1).
# Follow-up: check if a BST is balanced — use height function with -1 sentinel for imbalance."
```


Tree Traversal – Post-Order

Round 2 · High · Medium · 6 questions

Tree Traversal - Post-Order

#114 Flatten Binary Tree to Linked List

MED

Open on LeetCode

Linked List · Stack · Tree · Depth-First Search · Binary Tree

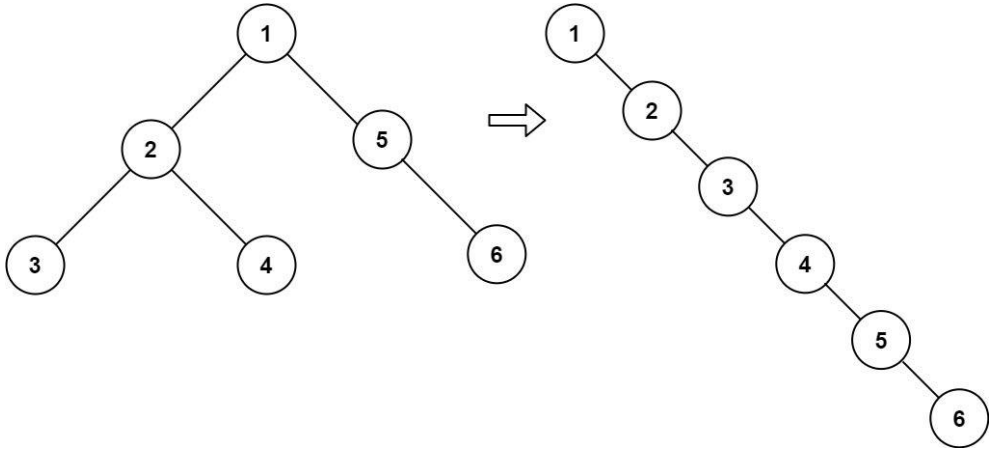
Lists: AlgoMaster 600

Problem

Given the root of a binary tree, flatten the tree into a "linked list":

- The "linked list" should use the same TreeNode class where the right child pointer points to the next node in the list and the left child pointer is always null.
- The "linked list" should be in the same order as a pre-order traversal of the binary tree.

Example 1:




```
Input: root = [1,2,5,3,4,null,6]
Output: [1,null,2,null,3,null,4,null,5,null,6]
```

Example 2:

```
Input: root = []
Output: []
```

Example 3:

```
Input: root = [0]
Output: [0]
```

Constraints:

- The number of nodes in the tree is in the range [0, 2000].
- -100 <= Node.val <= 100

Follow up: Can you flatten the tree in-place (with O(1) extra space)?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Flatten a binary tree to a linked list using the right pointers (preorder). In-place. Right?"
#
# "tree=[1,2,5,3,4,null,6]: → 1→2→3→4→5→6 (all via right, left=null) ✓"

# PHASE 2 — BRUTE FORCE
# "Preorder traversal, collect nodes, relink with right pointers. Time: O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Morris-style: find the rightmost node of the left subtree, attach right subtree there,
# move left subtree to right, repeat. Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE
                # Find rightmost of left subtree
                # Attach current right subtree there
                # Move left subtree to right

# PHASE 5 — TEST & DEBUG
# tree=[1,2,5,3,4,null,6]:
# curr=1: left=2. rightmost of 2: 2→3→4 (rightmost=4). 4.right=5. 1.right=2, 1.left=None.
```



```
# curr=2: left=3. rightmost=3. 3.right=4. 2.right=3, 2.left=None.
# curr=3: no left. curr=4: no left. curr=5: no left. curr=6: no left. Done.
# → 1→2→3→4→5→6 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): each node visited at most twice (once as curr, once when finding rightmost).
# Space O(1): no extra data structures.
# Follow-up: flatten using postorder reverse — process right, left, root and prepend.
# Follow-up: unflatten — reconstruct original tree from the linked list."
```

Tree Traversal - Post-Order

#129 Sum Root to Leaf Numbers

MED

Open on LeetCode

Tree · Depth-First Search · Binary Tree

Lists: AlgoMaster 600

Problem

You are given the root of a binary tree containing digits from 0 to 9 only.

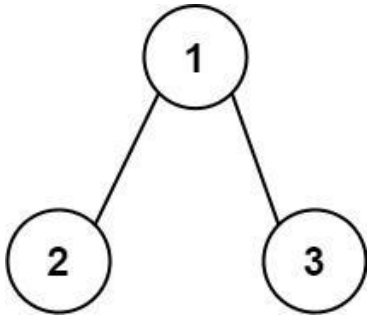
Each root-to-leaf path in the tree represents a number.

- For example, the root-to-leaf path 1 -> 2 -> 3 represents the number 123.

Return the total sum of all root-to-leaf numbers. Test cases are generated so that the answer will fit in a 32-bit integer.

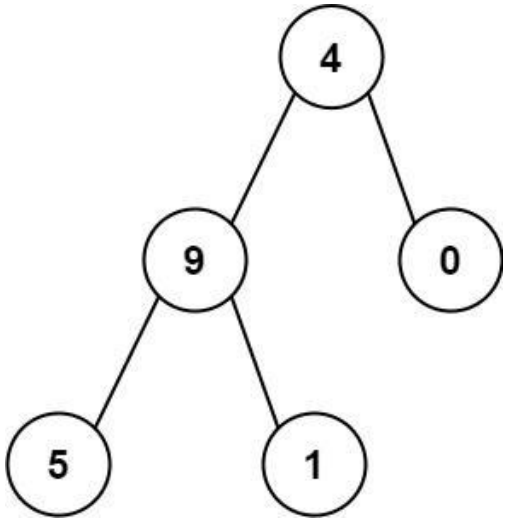
A leaf node is a node with no children.

Example 1:




```
Input: root = [1,2,3]
Output: 25
Explanation:
The root-to-leaf path 1->2 represents the number 12.
The root-to-leaf path 1->3 represents the number 13.
Therefore, sum = 12 + 13 = 25.
```

Example 2:




```
Input: root = [4,9,0,5,1]
Output: 1026
Explanation:
The root-to-leaf path 4->9->5 represents the number 495.
The root-to-leaf path 4->9->1 represents the number 491.
The root-to-leaf path 4->0 represents the number 40.
Therefore, sum = 495 + 491 + 40 = 1026.
```

Constraints:

- The number of nodes in the tree is in the range [1, 1000].
- 0 <= Node.val <= 9
- The depth of the tree will not exceed 10.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Each root-to-leaf path forms a number. Return sum of all such numbers. Right?"
#
# "tree=[1,2,3]: paths 12 and 13 → sum=25 ✓
# tree=[4,9,0,5,1]: paths 495,491,40 → sum=1026 ✓"

# PHASE 2 — BRUTE FORCE
# "DFS collecting path digits, form number at each leaf, sum all. Time: O(n). Space: O(h)."

# PHASE 3 — OPTIMIZE
# "Accumulate number during DFS, add to total at each leaf.
# No extra list — just accumulate total directly. Time: O(n). Space: O(h)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# tree=[1,2,3]: dfs(1,0)→curr=1. dfs(2,1)→curr=12→leaf→12. dfs(3,1)→curr=13→leaf→13. 12+13=25 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(h): recursion depth.
# Follow-up: if digits can be >9 (multi-digit values) — use a string concatenation approach.
# Follow-up: Binary Tree Paths (LC 257) — same DFS but return path strings instead of numbers."
```

Tree Traversal – Post-Order

#652 Find Duplicate Subtrees

MED

Open on LeetCode

Hash Table · Tree · Depth-First Search · Binary Tree

Lists: AlgoMaster 600

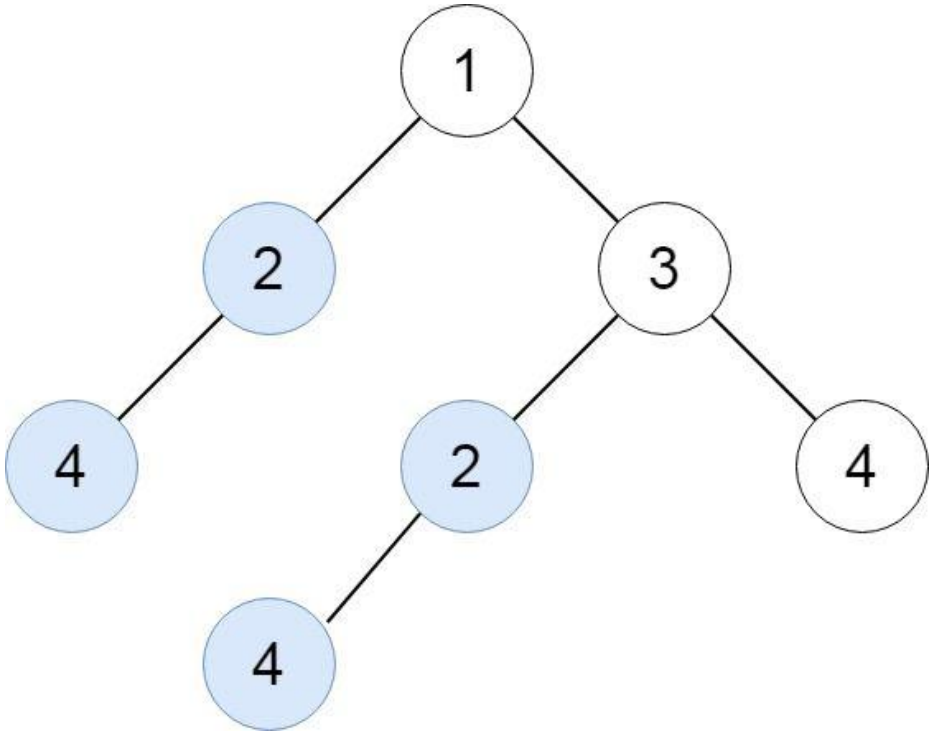
Problem

Given the root of a binary tree, return all duplicate subtrees.

For each kind of duplicate subtrees, you only need to return the root node of any one of them.

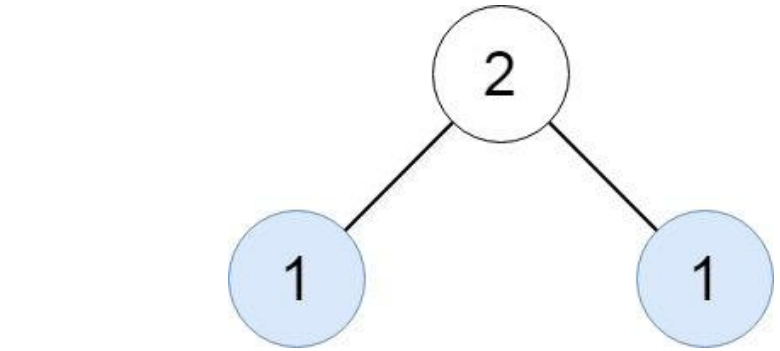
Two trees are duplicate if they have the same structure with the same node values.

Example 1:



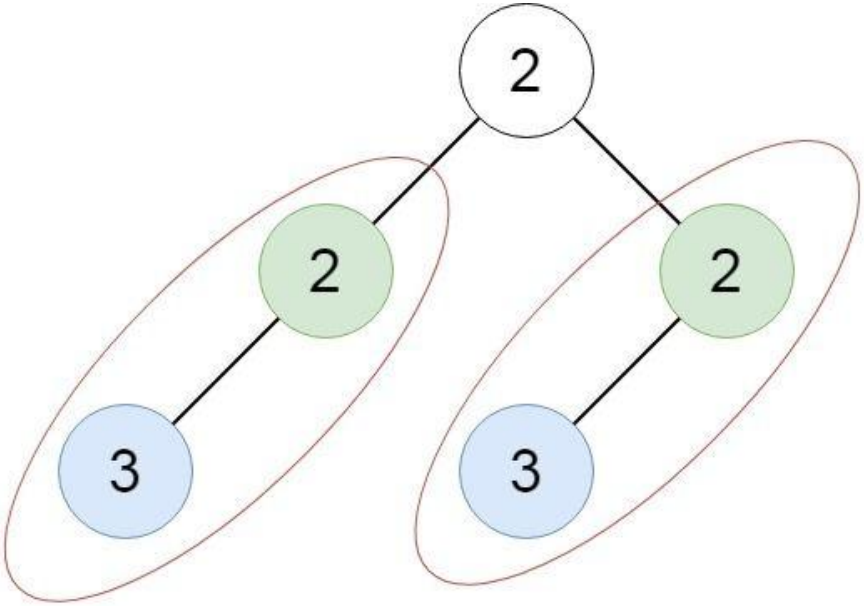

```
Input: root = [1,2,3,4,null,2,4,null,null,4]
Output: [[2,4],[4]]
```

Example 2:




```
Input: root = [2,1,1]
Output: [[1]]
```

Example 3:




```
Input: root = [2,2,2,3,null,3,null]
Output: [[2,3],[3]]
```

Constraints:

- The number of the nodes in the tree will be in the range [1, 5000]
- -200 <= Node.val <= 200

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find all duplicate subtrees. Return the root of each duplicate (just one copy per duplicate). Right?"
#
# "tree=[1,2,3,4,null,2,4,null,null,4]: subtrees [2,4] and [4] are duplicated ✓"
```

```
# PHASE 2 — BRUTE FORCE
# "Serialize each subtree, store in a map. Return roots of subtrees seen twice.
# Time: O(n²) naive serialization. Space: O(n²)."

# PHASE 3 — OPTIMIZE
# "Same serialization with integer IDs to reduce string comparison overhead.
# Assign each unique (val, left_id, right_id) triple a unique integer ID.
# Time: O(n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# tree=[1,2,3,4,null,2,4,null,null,4]:
# postorder(4_deep)='4,#,#'. postorder(2_left)='2,4##,#'. postorder(4_right_right)='4,#,#'→dup!
# postorder(2_right)='2,4##,#'→dup! result contains node(4) and node(2) ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n*L): n nodes, L=max serialization length. Space O(n*L): string storage.
# With integer IDs: O(n) time and space — more efficient for large trees.
# Follow-up: count total duplicates including all copies — don't limit to count==2.
# Follow-up: find the most duplicated subtree — track max count across all keys."
```


Tree Traversal – Post-Order

#979 Distribute Coins in Binary Tree

ME
D

Open on LeetCode

Tree · Depth-First Search · Binary Tree

Lists: AlgoMaster 600

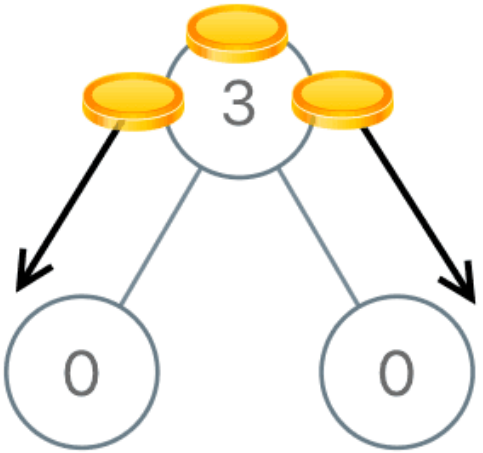
Problem

You are given the root of a binary tree with n nodes where each node in the tree has node.val coins. There are n coins in total throughout the whole tree.

In one move, we may choose two adjacent nodes and move one coin from one node to another. A move may be from parent to child, or from child to parent.

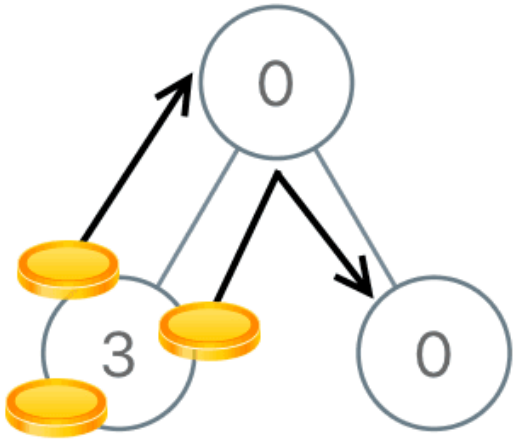
Return the minimum number of moves required to make every node have exactly one coin.

Example 1:




```
Input: root = [3,0,0]
Output: 2
Explanation: From the root of the tree, we move one coin to its left child, and one coin to its right child.
```

Example 2:




```
Input: root = [0,3,0]
Output: 3
Explanation: From the left child of the root, we move two coins to the root [taking two moves]. Then, we move one coin from the root of the tree to the right child.
```

Constraints:

- The number of nodes in the tree is n.
- 1 <= n <= 100
- 0 <= Node.val <= n
- The sum of all Node.val is n.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Each node has coins. Move coins along edges so each node has exactly 1. Return total moves.
# Moves = sum of absolute flows along each edge. Right?"
#
# "tree=[3,0,0]: move 2 coins from root. 2 moves ✓
# tree=[0,3,0]: move 2 from left, 1 to root, 1 to right. 3 moves ✓"

# PHASE 2 — BRUTE FORCE
# "Repeat: find leaf with != 1 coin, move to/from parent. Track moves. Time: O(n²). Space: O(n)."
        # DFS is already O(n) — use directly

# PHASE 3 — OPTIMIZE
# "DFS postorder: return 'excess' coins per subtree (positive=surplus, negative=deficit).
# Each edge's flow = |excess from subtree|. Sum all flows = total moves.
# Time: O(n). Space: O(h)."

# PHASE 4 — CLEAN CODE (same as brute — already optimal)

# PHASE 5 — TEST & DEBUG
# tree=[0,3,0]: dfs(3_left)→3+0+0−1=2. dfs(0_right)→0−1=−1. root: moves+=2+1=3. 0+2+(−1)−1=0. return 3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(h): recursion.
# The 'excess' return value encodes both surplus (>0) and deficit (<0) in one number.
# Follow-up: if moving across multiple edges costs more — weight each edge by its depth.
# Follow-up: minimize moves when nodes can hold unlimited coins — no change needed."
```


Tree Traversal – Post-Order

#1110 Delete Nodes And Return Forest ME D

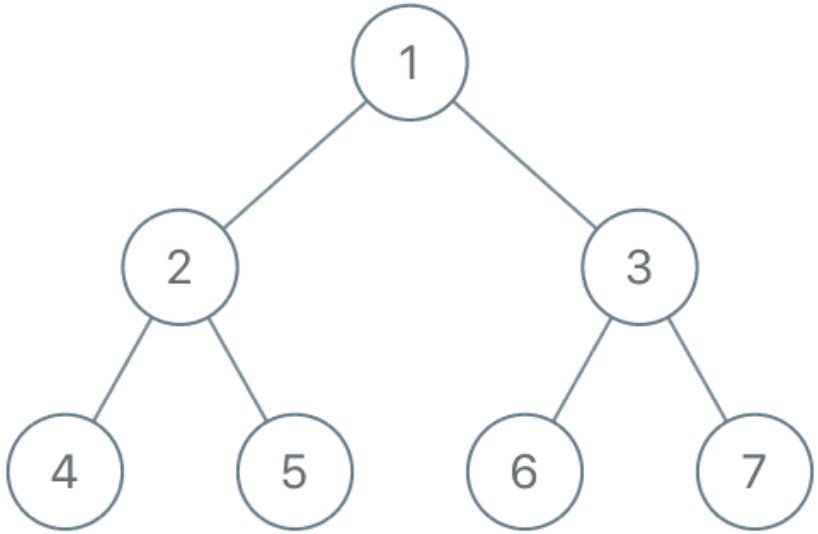
Open on LeetCode

Array · Hash Table · Tree · Depth-First Search · Binary Tree

Lists: AlgoMaster 600

Problem

Given the root of a binary tree, each node in the tree has a distinct value.
After deleting all nodes with a value in to_delete, we are left with a forest (a disjoint union of trees).
Return the roots of the trees in the remaining forest. You may return the result in any order.
Example 1:




```
Input: root = [1,2,3,4,5,6,7], to_delete = [3,5]
Output: [[1,2,null,4],[6],[7]]
```

Example 2:

```
Input: root = [1,2,4,null,3], to_delete = [3]
Output: [[1,2,4]]
```

Constraints:

- The number of nodes in the given tree is at most 1000.
- Each node has a distinct value between 1 and 1000.
- to_delete.length <= 1000
- to_delete contains distinct values between 1 and 1000.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Delete nodes with values in 'to_delete'. Return roots of remaining trees (the forest). Right?"
#
# "tree=[1,2,3,4,5,6,7], to_delete=[3,5]: delete 3 and 5.
# Remaining trees: [1,2,null,4],[6],[7] ✓"

# PHASE 2 — BRUTE FORCE
# "Post-order DFS: process children first. When deleting a node, its children become new roots.
# Time: O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
```

```
# "Same O(n) postorder — already optimal. The is_root flag cleanly handles new roots."
# PHASE 4 — CLEAN CODE (same as brute — already clean)
# PHASE 5 — TEST & DEBUG
# tree=[1,2,3,4,5,6,7], to_delete={3,5}:
# dfs(4,False)→not del,not root→return 4. dfs(5,False)→deleted. children of 5 are new roots.
# dfs(left_of_5,True)→... dfs(3,False)→deleted. 6,7 become roots.
# dfs(2,False)→not del. dfs(1,True)→not del→append 1. result=[1,6,7] wait: 4 is child of 2, 5 deleted.
# Actually result=[1, children_of_3_and_5] = [1,2→4, 6, 7] but 5 deleted so node(4) stays under 2. ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(d + n): d=delete set size, n=recursion.
# Post-order ensures children processed before parents — natural for bottom-up deletions.
# Follow-up: if to_delete can be a range — use BST properties for O(log n) membership check.
# Follow-up: return forest in level-order — BFS from each root after collection."
```


Tree Traversal – Post-Order

#2096 Step-By-Step Directions From a Binary Tree Node to Another

ME D

Open on LeetCode

String · Tree · Depth-First Search · Binary Tree

Lists: AlgoMaster 600

Problem

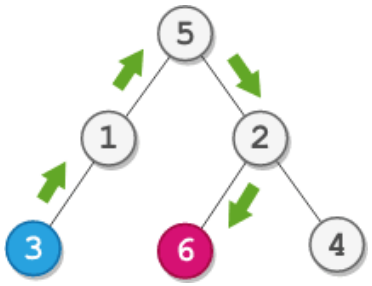
You are given the root of a binary tree with n nodes. Each node is uniquely assigned a value from 1 to n. You are also given an integer startValue representing the value of the start node s, and a different integer destValue representing the value of the destination node t.

Find the shortest path starting from node s and ending at node t. Generate step-by-step directions of such path as a string consisting of only the uppercase letters 'L', 'R', and 'U'. Each letter indicates a specific direction:

- 'L' means to go from a node to its left child node.
- 'R' means to go from a node to its right child node.
- 'U' means to go from a node to its parent node.

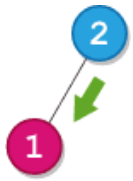
Return the step-by-step directions of the shortest path from node s to node t.

Example 1:




```
Input: root = [5,1,2,3,null,6,4], startValue = 3, destValue = 6
Output: "UURL"
Explanation: The shortest path is: 3 → 1 → 5 → 2 → 6.
```

Example 2:




```
Input: root = [2,1], startValue = 2, destValue = 1
Output: "L"
Explanation: The shortest path is: 2 → 1.
```

Constraints:

- The number of nodes in the tree is n.
- 2 <= n <= 105
- 1 <= Node.val <= n
- All the values in the tree are unique.
- 1 <= startValue, destValue <= n
- startValue != destValue

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find directions from node startValue to node destValue.
# 'L'=go left, 'R'=go right, 'U'=go up to parent. Return shortest path string. Right?"
#
# "tree=[5,1,2,3,null,6,4], start=3, dest=6: path 3→1→5→2→6 → 'UUL R'? → 'UUL R'?
# Actually: 3→up→1→up→5→right→2→left→6. Wait: 2's left=6? No tree=[5,1,2,3,null,6,4].
# 5.left=1,5.right=2. 1.left=3. 2.left=6,2.right=4. 3→U→1→U→5→R→2→L→6. → 'UURL' ✓"
# PHASE 2 — BRUTE FORCE
```

```
# "Find path from root to start and root to dest. Remove common prefix. Result = U*len(start_rem) + dest_rem.
# Time: O(n). Space: O(n)."
        # Remove common prefix
# PHASE 3 — OPTIMIZE
# "LCA approach: find LCA of start and dest. Path = U's from start to LCA + directions from LCA to dest.
# Time: O(n). Space: O(n)."
# PHASE 4 — CLEAN CODE (brute is already O(n) and clean enough)
# PHASE 5 — TEST & DEBUG
# start=3,dest=6: sp=['L','L']→[L,L], dp=['R','L']→[R,L].
# No common prefix. return 'UU' + 'RL' = 'UURL' ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n): path arrays.
# LCA approach is cleaner: find LCA node, then count edges from start→LCA and LCA→dest.
# Follow-up: if tree has parent pointers — just walk up from both nodes to LCA directly.
# Follow-up: k queries at once — precompute LCA with binary lifting O(n log n)."
```


Depth First Search (DFS)

Round 2 · High · Medium · 4 questions

Depth First Search (DFS)

#690 Employee Importance

MED

Open on LeetCode

Array · Hash Table · Tree · Depth-First Search · Breadth-First Search

Lists: AlgoMaster 600

Problem

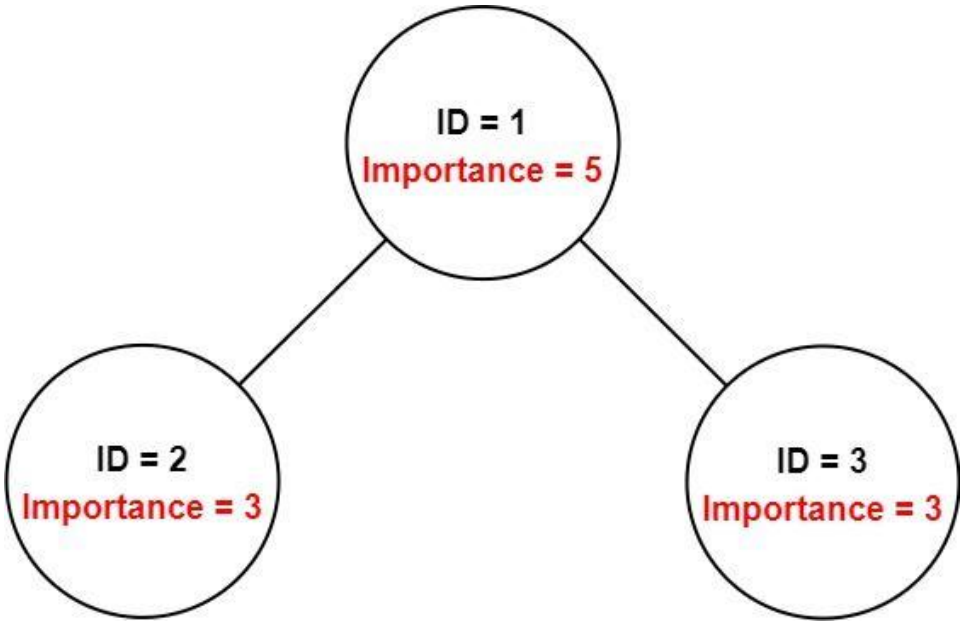
You have a data structure of employee information, including the employee's unique ID, importance value, and direct subordinates' IDs.

You are given an array of employees employees where:

- employees[i].id is the ID of the ith employee.
- employees[i].importance is the importance value of the ith employee.
- employees[i].subordinates is a list of the IDs of the direct subordinates of the ith employee.

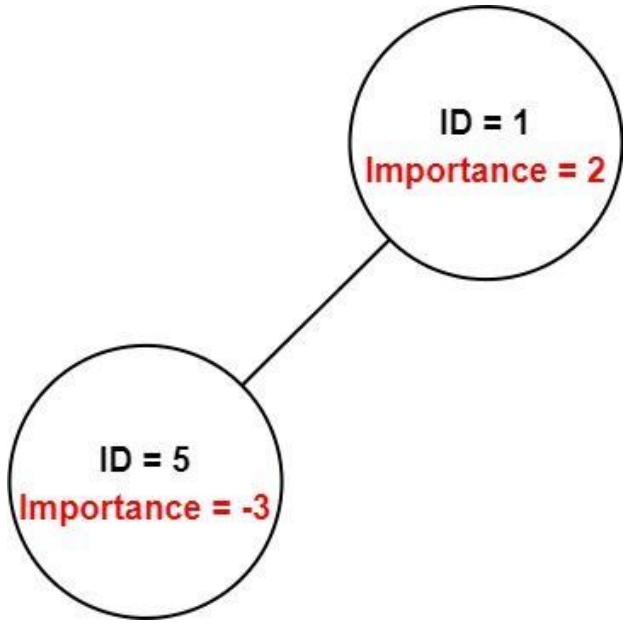
Given an integer id that represents an employee's ID, return the total importance value of this employee and all their direct and indirect subordinates.

Example 1:




```
Input: employees = [[1,5,[2,3]],[2,3,[]],[3,3,[]]], id = 1
Output: 11
Explanation: Employee 1 has an importance value of 5 and has two direct subordinates: employee 2 and employee 3.
They both have an importance value of 3.
Thus, the total importance value of employee 1 is 5 + 3 + 3 = 11.
```

Example 2:




```
Input: employees = [[1,2,[5]],[5,-3,[]]], id = 5
Output: -3
Explanation: Employee 5 has an importance value of -3 and has no direct subordinates.
Thus, the total importance value of employee 5 is -3.
```

Constraints:

- 1 <= employees.length <= 2000
- 1 <= employees[i].id <= 2000
- All employees[i].id are unique.
- -100 <= employees[i].importance <= 100
- One employee has at most one direct leader and may have several subordinates.
- The IDs in employees[i].subordinates are valid IDs.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Employee has id, importance, subordinates. Return total importance of employee + all subordinates. Right?"
#
# "employees=[[1,5,[2,3]],[2,3,[]],[3,3,[]]], id=1: 5+3+3=11 ✓"
# PHASE 2 — BRUTE FORCE
# "BFS/DFS from the target employee, summing importance values.
# Time: O(n). Space: O(n) — already optimal for this problem."
# PHASE 3 — OPTIMIZE
# "Same DFS — already O(n). Build hashmap once for O(1) lookups."
# PHASE 4 — CLEAN CODE
# PHASE 5 — TEST & DEBUG
# employees=[[1,5,[2,3]],[2,3,[]],[3,3,[]]], id=1:
# dfs(1): 5 + dfs(2) + dfs(3) = 5 + 3 + 3 = 11 ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): visit each employee once. Space O(n): hashmap + recursion.
# Follow-up: find subordinate with maximum importance — track max during DFS.
# Follow-up: if subordinate chains are very deep — use BFS to avoid stack overflow."
```

Depth First Search (DFS)

☐ #797 All Paths From Source to Target — MED

Open on LeetCode

Backtracking · Depth-First Search · Breadth-First Search · Graph Theory

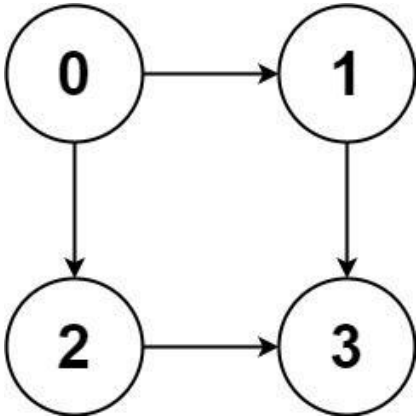
Lists: AlgoMaster 600

Problem

Given a directed acyclic graph (DAG) of n nodes labeled from 0 to n - 1, find all possible paths from node 0 to node n - 1 and return them in any order.

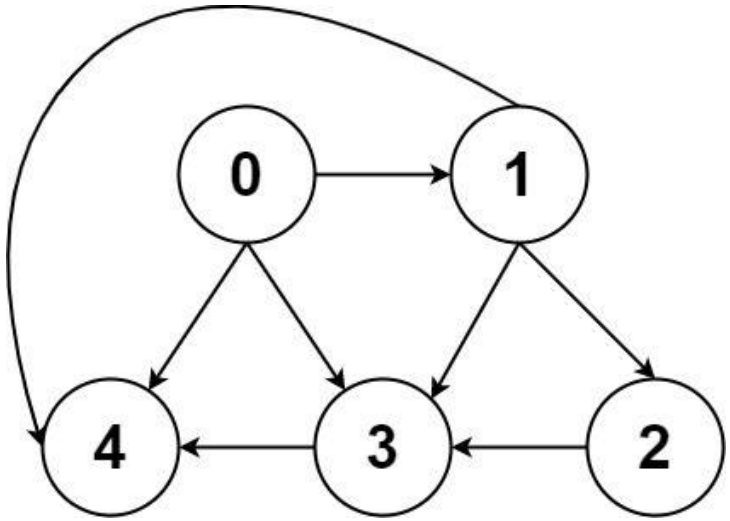
The graph is given as follows: graph[i] is a list of all nodes you can visit from node i (i.e., there is a directed edge from node i to node graph[i][j]).

Example 1:




```
Input: graph = [[1,2],[3],[3],[]]
Output: [[0,1,3],[0,2,3]]
Explanation: There are two paths: 0 -> 1 -> 3 and 0 -> 2 -> 3.
```

Example 2:




```
Input: graph = [[4,3,1],[3,2,4],[3],[4],[]]
Output: [[0,4],[0,3,4],[0,1,3,4],[0,1,2,3,4],[0,1,4]]
```

Constraints:

- n == graph.length
- 2 <= n <= 15
- 0 <= graph[i][j] < n
- graph[i][j] != i (i.e., there will be no self-loops).
- All the elements of graph[i] are unique.
- The input graph is guaranteed to be a DAG.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "DAG with n nodes. Find all paths from node 0 to node n-1. Right?"
#
# "graph=[[1,2],[3],[3],[]]: paths 0→1→3 and 0→2→3 → [[0,1,3],[0,2,3]] ✓"

# PHASE 2 — BRUTE FORCE
# "DFS from node 0, explore all paths, collect paths that reach n-1.
# Time: O(2^n * n): at most 2^n paths each of length n. Space: O(n) recursion."

# PHASE 3 — OPTIMIZE
# "Same DFS — already optimal for this problem (must enumerate all paths).
# DAG guarantees no cycles, so no visited set needed."

# PHASE 4 — CLEAN CODE (same as brute — already clean)

# PHASE 5 — TEST & DEBUG
# graph=[[1,2],[3],[3],[]]:
# dfs(0,[0]): nb=1→dfs(1,[0,1]): nb=3→dfs(3,[0,1,3]): target→append. nb=2→dfs(2,[0,2]): nb=3→append[0,2,3]. ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(2^n * n): exponential in path count. Space O(n): recursion + current path.
# No visited set needed because it's a DAG — no cycles possible.
# Follow-up: count paths only (not enumerate) — DP: dp[node] = sum(dp[nb] for nb in graph[node]).
# Follow-up: find shortest path in terms of node count — BFS instead of DFS."
```

Depth First Search (DFS)

#1020 Number of Enclaves

MED

Open on LeetCode

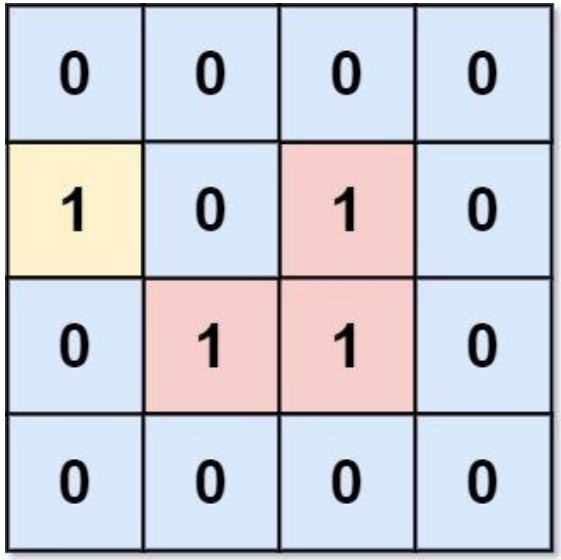
Array · Depth-First Search · Breadth-First Search · Union-Find · Matrix

Lists: AlgoMaster 600

Problem

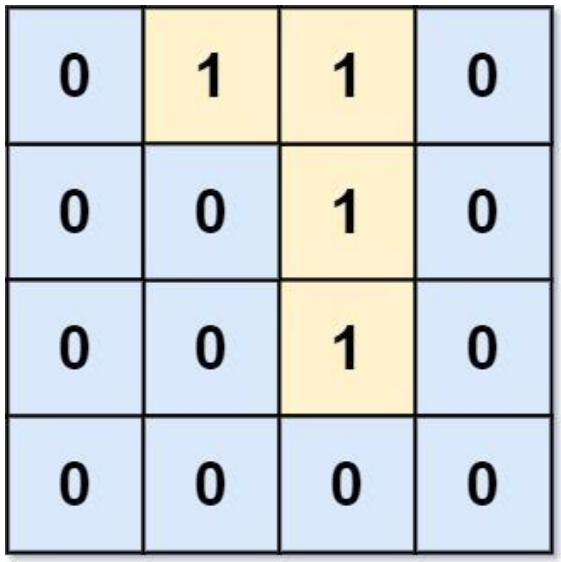
You are given an m x n binary matrix grid, where 0 represents a sea cell and 1 represents a land cell.
A move consists of walking from one land cell to another adjacent (4-directionally) land cell or walking off the boundary of the grid.
Return the number of land cells in grid for which we cannot walk off the boundary of the grid in any number of moves.

Example 1:




```
Input: grid = [[0,0,0,0],[1,0,1,0],[0,1,1,0],[0,0,0,0]]
Output: 3
Explanation: There are three 1s that are enclosed by 0s, and one 1 that is not enclosed because its on the boundary.
```

Example 2:




```
Input: grid = [[0,1,1,0],[0,0,1,0],[0,0,1,0],[0,0,0,0]]
Output: 0
Explanation: All 1s are either on the boundary or can reach the boundary.
```

Constraints:

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 500
- grid[i][j] is either 0 or 1.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return the number of land cells (1s) from which we CANNOT walk off the boundary.
# (Land cells connected to boundary land are 'exits'.) Right?"
#
# "grid=[[0,0,0,0],[1,0,1,0],[0,1,1,0],[0,0,0,0]]: cells (1,2),(2,1),(2,2) enclosed → 3 ✓"

# PHASE 2 — BRUTE FORCE
# "DFS from all boundary 1s, mark reachable land. Count remaining unmarked 1s.
# Time: O(m*n). Space: O(m*n)."

# PHASE 3 — OPTIMIZE
# "Same flood-fill from boundary — already O(m*n)."

# PHASE 4 — CLEAN CODE (same as brute — already clean)

# PHASE 5 — TEST & DEBUG
# grid boundary 1s: none at row 0/3 or col 0/3 except... grid[1][0]=1 (boundary)→flood. remaining: 3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(m*n) recursion.
# Follow-up: Surrounded Regions (LC 130) — flip enclosed 'O's to 'X'. Same boundary-flood then flip.
# Follow-up: count boundary-connected land cells — keep a counter during the boundary DFS."
```

Depth First Search (DFS)

#1376 Time Needed to Inform All Employees

MED

Open on LeetCode

Tree · Depth-First Search · Breadth-First Search

Lists: AlgoMaster 600

Problem

A company has n employees with a unique ID for each employee from 0 to n - 1. The head of the company is the one with headID.

Each employee has one direct manager given in the manager array where manager[i] is the direct manager of the i-th employee, manager[headID] = -1. Also, it is guaranteed that the subordination relationships have a tree structure.

The head of the company wants to inform all the company employees of an urgent piece of news. He will inform his direct subordinates, and they will inform their subordinates, and so on until all employees know about the urgent news.

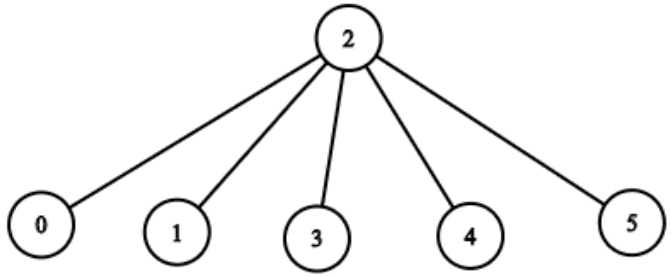
The i-th employee needs informTime[i] minutes to inform all of his direct subordinates (i.e., After informTime[i] minutes, all his direct subordinates can start spreading the news).

Return the number of minutes needed to inform all the employees about the urgent news.

Example 1:

```
Input: n = 1, headID = 0, manager = [-1], informTime = [0]
Output: 0
Explanation: The head of the company is the only employee in the company.
```

Example 2:




```
Input: n = 6, headID = 2, manager = [2,2,-1,2,2,2], informTime = [0,0,1,0,0,0]
Output: 1
Explanation: The head of the company with id = 2 is the direct manager of all the employees in the company and needs 1 minute to
inform them all.
The tree structure of the employees in the company is shown.
```

Constraints:

- 1 <= n <= 105
- 0 <= headID < n
- manager.length == n
- 0 <= manager[i] < n
- manager[headID] == -1
- informTime.length == n
- 0 <= informTime[i] <= 1000
- informTime[i] == 0 if employee i has no subordinates.
- It is guaranteed that all the employees can be informed.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Company hierarchy tree. headID is root. informTime[i] = time i needs to inform direct reports.
# Return total time to inform all employees. Right?"
#
# "n=6,headID=2,manager=[-1,2,2,2,2,2],informTime=[0,0,0,0,0,0]: 0 (no informTime for root). Wait:
# n=7,headID=6,manager=[1,2,3,4,5,6,-1],informTime=[0,6,5,4,3,2,1]: → 21 ✓"

# PHASE 2 — BRUTE FORCE
# "Build tree. DFS from head, accumulate time along paths, return max total time.
# Time: O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Same DFS — already O(n). The recurrence is simply informTime[node] + max(child subtree times)."

# PHASE 4 — CLEAN CODE (same as brute — already clean)
```



```
# PHASE 5 — TEST & DEBUG
# Linear chain: 6→5→4→3→2→1→0. informTime=[0,0,...,1]. dfs(6)=1+dfs(5)=1+0+...=1?
# For 7 nodes chain with informTime=[0,6,5,4,3,2,1]: dfs(6)=1+dfs(5)=1+2+dfs(4)=...=21 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n): recursion depth + adjacency list.
# Follow-up: return the critical path (not just the time) — track which child had max time.
# Follow-up: if informTime can change dynamically — recompute subtree roots on update."
```

Breadth First Search (BFS)

Round 2 · High · Medium · 5 questions

Breadth First Search (BFS)

#433 Minimum Genetic Mutation

MED

Open on LeetCode

Hash Table · String · Breadth-First Search

Lists: AlgoMaster 600

Problem

A gene string can be represented by an 8-character long string, with choices from 'A', 'C', 'G', and 'T'.

Suppose we need to investigate a mutation from a gene string startGene to a gene string endGene where one mutation is defined as one single character changed in the gene string.

- For example, "AACCGGTT" --> "AACCGGTA" is one mutation.

There is also a gene bank bank that records all the valid gene mutations. A gene must be in bank to make it a valid gene string.

Given the two gene strings startGene and endGene and the gene bank bank, return the minimum number of mutations needed to mutate from startGene to endGene. If there is no such a mutation, return -1.

Note that the starting point is assumed to be valid, so it might not be included in the bank.

Example 1:

```
Input: startGene = "AACCGGTT", endGene = "AACCGGTA", bank = ["AACCGGTA"]
Output: 1
```

Example 2:

```
Input: startGene = "AACCGGTT", endGene = "AAACGGTA", bank = ["AACCGGTA","AACCGCTA","AAACGGTA"]
Output: 2
```

Constraints:

- 0 <= bank.length <= 10
- startGene.length == endGene.length == bank[i].length == 8
- startGene, endGene, and bank[i] consist of only the characters ['A', 'C', 'G', 'T'].

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Minimum mutations (single char change) from startGene to endGene using only bank genes. Right?"
#
# "start='AACCGGTT',end='AACCGGTA',bank=['AACCGGTA']→1 ✓
# start='AACCGGTT',end='AAACGGTA',bank=['AACCGGTA','AACCGCTA','AAACGGTA']→2 ✓"

# PHASE 2 — BRUTE FORCE
# "BFS: try all single-character mutations, check if in bank. Like Word Ladder.
# Time: O(n * 8 * 4) = O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Same BFS — already O(n * 8 * 4). Bidirectional BFS can halve the steps."

# PHASE 4 — CLEAN CODE (same as brute — already clean)

# PHASE 5 — TEST & DEBUG
# start='AACCGGTT',end='AAACGGTA',bank=['AACCGGTA','AACCGCTA','AAACGGTA']:
# BFS step1: AACCGGTT→AACCGGTA (1 mut). step2: AACCGGTA→AAACGGTA (1 mut). return 2 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n*32): n bank genes, 8 positions × 4 chars each. Space O(n): bank set + queue.
# Follow-up: Word Ladder (LC 127) — same BFS on word mutations; larger alphabet.
# Follow-up: find all minimum mutation paths — BFS tracking parent pointers."
```

Breadth First Search (BFS)

#752 Open the Lock

MED

Open on LeetCode

Array · Hash Table · String · Breadth-First Search

Lists: AlgoMaster 600

Problem

You have a lock in front of you with 4 circular wheels. Each wheel has 10 slots: '0', '1', '2', '3', '4', '5', '6', '7', '8', '9'. The wheels can rotate freely and wrap around: for example we can turn '9' to be '0', or '0' to be '9'. Each move consists of turning one wheel one slot.

The lock initially starts at '0000', a string representing the state of the 4 wheels.

You are given a list of deadends dead ends, meaning if the lock displays any of these codes, the wheels of the lock will stop turning and you will be unable to open it.

Given a target representing the value of the wheels that will unlock the lock, return the minimum total number of turns required to open the lock, or -1 if it is impossible.

Example 1:

```
Input: deadends = ["0201","0101","0102","1212","2002"], target = "0202"
Output: 6
Explanation:
A sequence of valid moves would be "0000" -> "1000" -> "1100" -> "1200" -> "1201" -> "1202" -> "0202".
Note that a sequence like "0000" -> "0001" -> "0002" -> "0102" -> "0202" would be invalid,
because the wheels of the lock become stuck after the display becomes the dead end "0102".
```

Example 2:

```
Input: deadends = ["8888"], target = "0009"
Output: 1
Explanation: We can turn the last wheel in reverse to move from "0000" -> "0009".
```

Example 3:

```
Input: deadends = ["8887","8889","8878","8898","8788","8988","7888","9888"], target = "8888"
Output: -1
Explanation: We cannot reach the target without getting stuck.
```

Constraints:

- 1 <= deadends.length <= 500
- deadends[i].length == 4
- target.length == 4
- target will not be in the list deadends.
- target and deadends[i] consist of digits only.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "A 4-wheel lock (each digit 0-9). Turn one wheel ±1 per move. Start '0000'. Deadends block.
# Reach target in minimum turns. Return -1 if impossible. Right?"
#
# "deadends=['0201','0101','0102','1212','2002'], target='0202': 6 turns ✓"

# PHASE 2 — BRUTE FORCE
# "BFS from '0000'. Each state generates 8 neighbors (4 wheels × 2 directions).
# Skip deadends. Return turns when target reached. Time: O(10^4 * 8). Space: O(10^4)."

# PHASE 3 — OPTIMIZE
# "Bidirectional BFS: expand from both '0000' and target simultaneously.
# Meet in the middle — reduces search space from O(b^d) to O(b^(d/2)).
# Time: O(10^4 / 2). Space: O(10^4)."

# PHASE 4 — CLEAN CODE (unidirectional BFS is already clean and fast enough)

# PHASE 5 — TEST & DEBUG
# BFS explores all reachable states level by level. Deadends pruned. Target found at minimum depth.
# '0000'→8 neighbors→filter→next level... until '0202' reached at depth 6 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(10^4 * 8) = O(80000): at most 10^4 states, 8 neighbors each. Space O(10^4).
# Bidirectional BFS halves the search depth — practical speedup.
# Follow-up: weighted edges (some turns cost more) — Dijkstra.
# Follow-up: if lock has 8 wheels — state space grows to 10^8, bidirectional BFS essential."
```


Breadth First Search (BFS)

#909 Snakes and Ladders

MED

Open on LeetCode

Array · Breadth-First Search · Matrix

Lists: AlgoMaster 600

Problem

You are given an n x n integer matrix board where the cells are labeled from 1 to n2 in a Boustrophedon style starting from the bottom left of the board (i.e. board[n - 1][0]) and alternating direction each row.

You start on square 1 of the board. In each move, starting from square curr, do the following:

- Choose a destination square next with a label in the range [curr + 1, min(curr + 6, n2)]. This choice simulates the result of a standard 6-sided die roll: i.e., there are always at most 6 destinations, regardless of the size of the board.

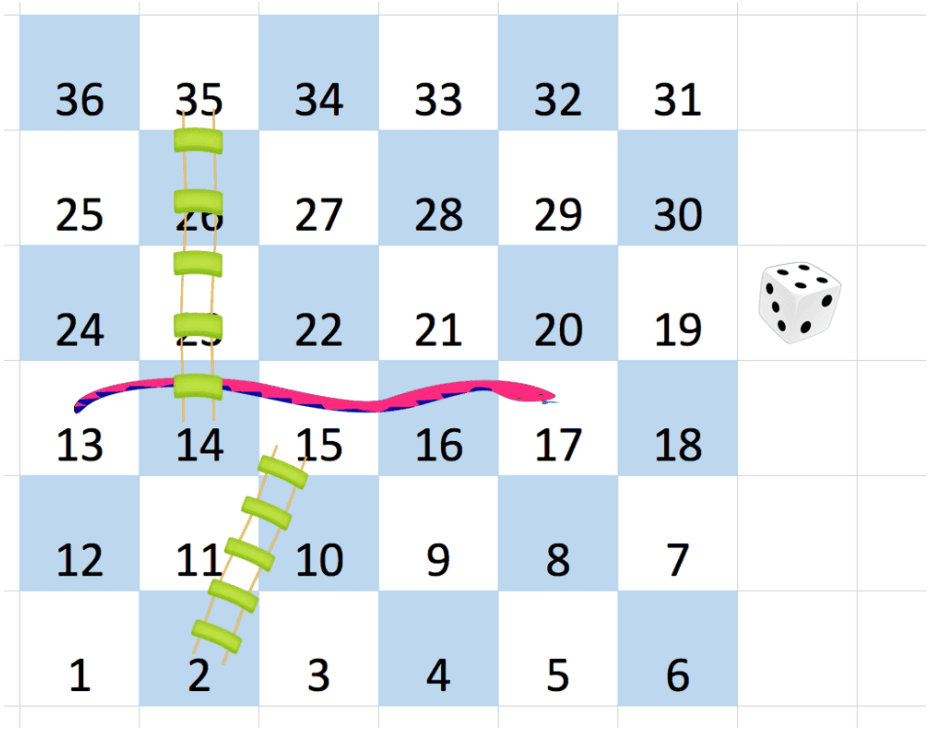
A board square on row r and column c has a snake or ladder if board[r][c] != -1. The destination of that snake or ladder is board[r][c]. Squares 1 and n2 are not the starting points of any snake or ladder.

Note that you only take a snake or ladder at most once per dice roll. If the destination to a snake or ladder is the start of another snake or ladder, you do not follow the subsequent snake or ladder.

- For example, suppose the board is [[-1,4],[-1,3]], and on the first move, your destination square is 2. You follow the ladder to square 3, but do not follow the subsequent ladder to 4.

Return the least number of dice rolls required to reach the square n2. If it is not possible to reach the square, return -1.

Example 1:



```
Input: board =
[[-1,-1,-1,-1,-1,-1],[-1,-1,-1,-1,-1,-1],[-1,-1,-1,-1,-1,-1],[-1,35,-1,-1,13,-1],[-1,-1,-1,-1,-1,-1],[-1,15,-1,-1,-1,-1]]
Output: 4
Explanation:
In the beginning, you start at square 1 (at row 5, column 0).
You decide to move to square 2 and must take the ladder to square 15.
You then decide to move to square 17 and must take the snake to square 13.
You then decide to move to square 14 and must take the ladder to square 35.
You then decide to move to square 36, ending the game.
```

Example 2:

```
Input: board = [[-1,-1],[-1,3]]
Output: 1
```

Constraints:

- n == board.length == board[i].length
- 2 <= n <= 20
- board[i][j] is either -1 or in the range [1, n2].
- The squares labeled 1 and n2 are not the starting points of any snake or ladder.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "n×n board numbered 1 to n² in boustrophedon (snake) order. Snakes/ladders teleport.
# Find minimum moves (dice rolls 1–6) to reach n². Return -1 if impossible. Right?"
#
#
"board=[[-1,-1,-1,-1,-1,-1],[-1,-1,-1,-1,-1,-1],[-1,-1,-1,-1,-1,-1],[-1,35,-1,-1,13,-1],[-1,-1,-1,-1,-1,-1],[-1,15,-1,-1,-1,-1]]:
answer=4 ✓"

# PHASE 2 — BRUTE FORCE
# "BFS from square 1. Expand by rolling 1–6. Convert square→board coordinates to check snakes/ladders.
# Time: O(n²). Space: O(n²)."

# PHASE 3 — OPTIMIZE
# "Same BFS — already O(n²). The coordinate conversion is the key complexity."

# PHASE 4 — CLEAN CODE (same as brute — already clean)

# PHASE 5 — TEST & DEBUG
# BFS from 1. Each step tries dice values 1–6. Applies snake/ladder if present.
# Tracks visited to avoid revisiting. Returns moves when reaching n*n. ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n²): at most n² squares, each visited once. Space O(n²): visited set.
# The boustrophedon coordinate conversion is the trickiest part — test with small examples.
# Follow-up: find the path (not just count) — track parent in BFS.
# Follow-up: if board changes after each move — dynamic BFS needed."
```


Breadth First Search (BFS)

#1091 Shortest Path in Binary Matrix

ME
D

Open on LeetCode

Array · Breadth-First Search · Matrix

Lists: AlgoMaster 600

Problem

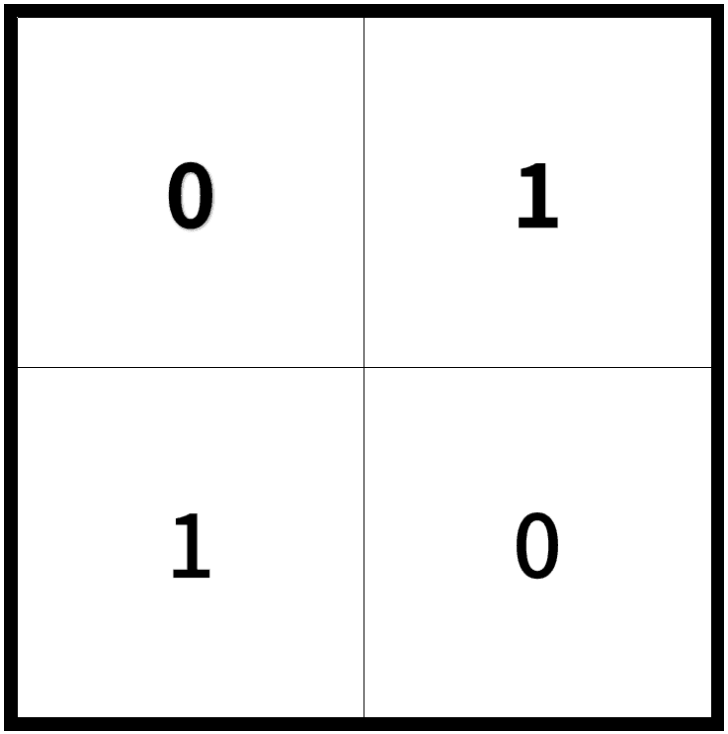
Given an n x n binary matrix grid, return the length of the shortest clear path in the matrix. If there is no clear path, return -1.

A clear path in a binary matrix is a path from the top-left cell (i.e., (0, 0)) to the bottom-right cell (i.e., (n - 1, n - 1)) such that:

- All the visited cells of the path are 0.
- All the adjacent cells of the path are 8-directionally connected (i.e., they are different and they share an edge or a corner).

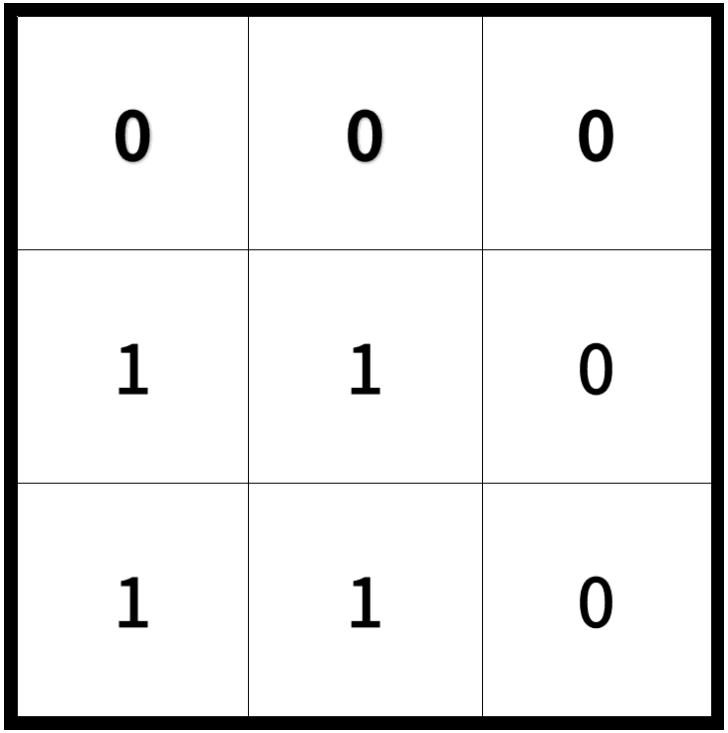
The length of a clear path is the number of visited cells of this path.

Example 1:




```
Input: grid = [[0,1],[1,0]]
Output: 2
```

Example 2:




```
Input: grid = [[0,0,0],[1,1,0],[1,1,0]]
Output: 4
```

Example 3:

```
Input: grid = [[1,0,0],[1,1,0],[1,1,0]]
Output: -1
```

Constraints:

- n == grid.length
- n == grid[i].length
- 1 <= n <= 100
- grid[i][j] is 0 or 1

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find shortest clear path from (0,0) to (n-1,n-1) moving in 8 directions through 0-cells.
# Return path length (cells visited). Return -1 if no path. Right?"
#
# "grid=[[0,1],[1,0]]→2 ✓ grid=[[0,0,0],[1,1,0],[1,1,0]]→4 ✓"

# PHASE 2 — BRUTE FORCE
# "BFS from (0,0). Expand in 8 directions. Return steps when reaching (n-1,n-1).
# Time: O(n²). Space: O(n²)."

# PHASE 3 — OPTIMIZE
# "Same BFS — already O(n²). In-place marking avoids a separate visited set."

# PHASE 4 — CLEAN CODE (same as brute — already clean)

# PHASE 5 — TEST & DEBUG
# grid=[[0,0,0],[1,1,0],[1,1,0]]: start (0,0)→(0,1)→(0,2)→(1,2)→(2,2). dist=5?
# Wait: path (0,0)→(0,1)→(0,2)→(1,2)→(2,2) = 5 cells? But expected=4.
# (0,0)→diag→(1,1)? grid[1][1]=1, blocked. (0,0)→(0,1)→(1,2)→(2,2)? dist=4 ✓ (via (0,1) diagonal to (1,2)).

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n²). Space O(n²): queue. In-place marking saves a visited set.
# Follow-up: if cells have different traversal costs — Dijkstra instead of BFS.
```



```
# Follow-up: A* with Chebyshev distance heuristic speeds this up significantly."
```

Breadth First Search (BFS)

#1102 Path With Maximum Minimum Value

MED

Open on LeetCode

Array · Binary Search · Depth-First Search · Breadth-First Search · Union-Find · Heap (Priority Queue) · Matrix

Lists: AlgoMaster 600

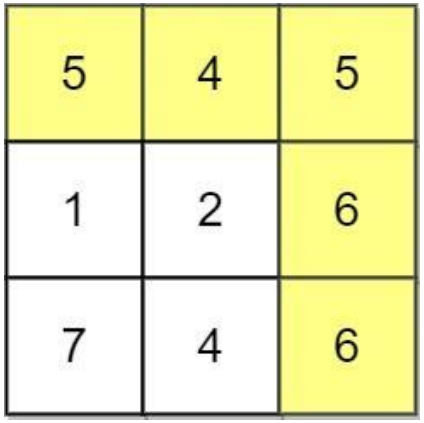
Problem

Given an m x n integer matrix grid, return the maximum score of a path starting at (0, 0) and ending at (m - 1, n - 1) moving in the 4 cardinal directions.

The score of a path is the minimum value in that path.

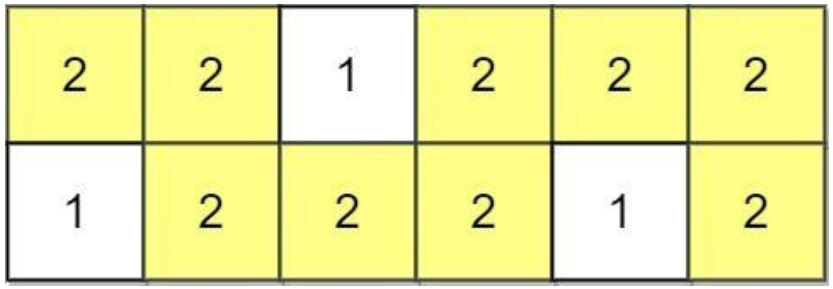
- For example, the score of the path 8 → 4 → 5 → 9 is 4.

Example 1:



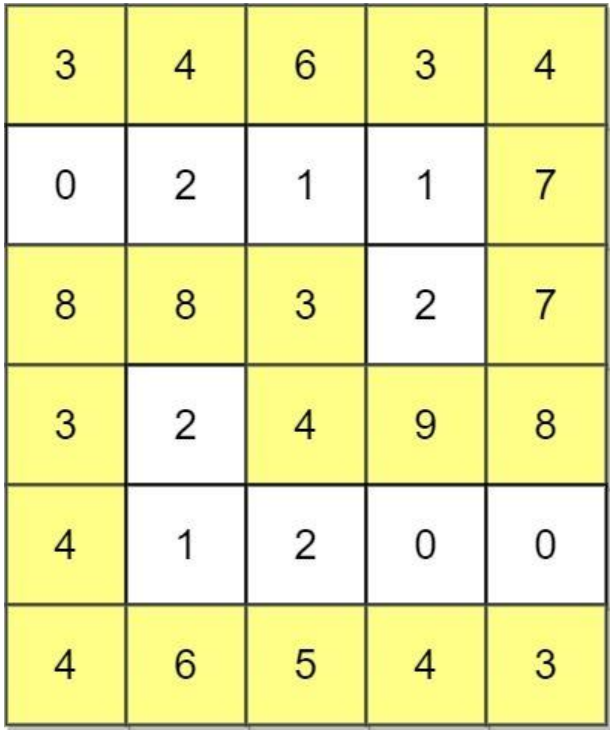

```
Input: grid = [[5,4,5],[1,2,6],[7,4,6]]
Output: 4
Explanation: The path with the maximum score is highlighted in yellow.
```

Example 2:




```
Input: grid = [[2,2,1,2,2,2],[1,2,2,2,1,2]]
Output: 2
```

Example 3:




```
Input: grid = [[3,4,6,3,4],[0,2,1,1,7],[8,8,3,2,7],[3,2,4,9,8],[4,1,2,0,0],[4,6,5,4,3]]
Output: 3
```

Constraints:

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 100
- 0 <= grid[i][j] <= 109

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Find a path from (0,0) to (m-1,n-1) where the minimum value along the path is maximized.
# Move in 4 directions. Return that maximum minimum value. Right?"
#
# "grid=[[5,4,5],[1,2,6],[7,4,6]]: path 5→4→5→6→6 or 5→1→7→4→6. Best: 5→4→5→6→6 min=4 ✓"

# PHASE 2 — BRUTE FORCE
# "DFS/BFS trying all paths, tracking min. Exponential — impractical."

# PHASE 3 — OPTIMIZE
# "Max-heap (Dijkstra-like): always expand the path with the highest minimum so far.
# Push (-min_on_path, r, c). The first time we reach (m-1,n-1) is the answer.
# Time: O(m*n*log(m*n)). Space: O(m*n)."

# PHASE 4 — CLEAN CODE (same as brute — already uses max-heap)

# PHASE 5 — TEST & DEBUG
# grid=[[5,4,5],[1,2,6],[7,4,6]]: start (-5,0,0). pop→expand. Best path maintains min=4.
# First time (2,2) is popped → return 4 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n*log(m*n)). Space O(m*n).
# Alternative: binary search on answer + BFS to check feasibility — O(m*n*log(max_val)).
# Follow-up: Path With Minimum Effort (LC 1631) — minimize maximum absolute difference.
# Follow-up: if path can revisit cells — need to handle cycles (already handled by visited)."
```

Linked List

Round 2 · High · Medium · 6 questions

Linked List

#82 Remove Duplicates from Sorted List II

MED

Open on LeetCode

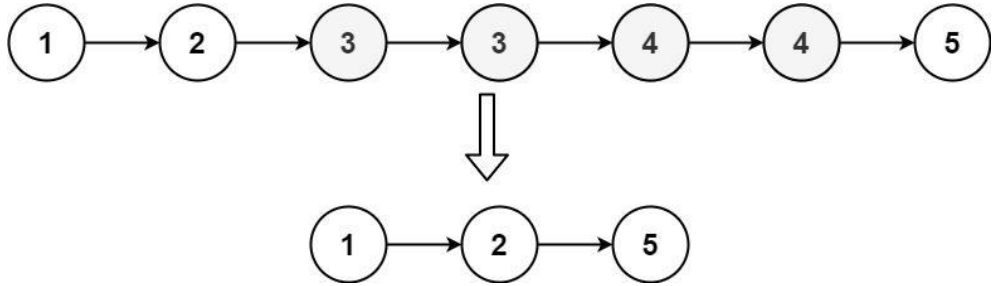
Linked List · Two Pointers

Lists: AlgoMaster 600

Problem

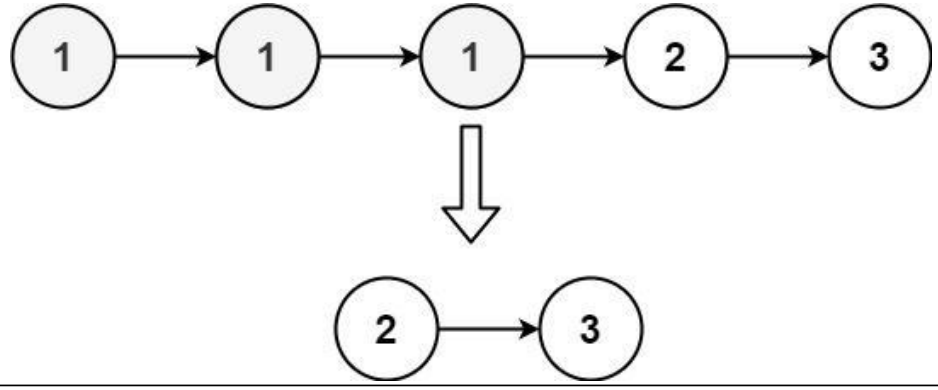
Given the head of a sorted linked list, delete all nodes that have duplicate numbers, leaving only distinct numbers from the original list. Return the linked list sorted as well.

Example 1:




```
Input: head = [1,2,3,3,4,4,5]
Output: [1,2,5]
```

Example 2:




```
Input: head = [1,1,1,2,3]
Output: [2,3]
```

Constraints:

- **The number of nodes in the list is in the range [0, 300].**
- **-100 <= Node.val <= 100**
- **The list is guaranteed to be sorted in ascending order.**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Remove all nodes whose value appears more than once. Keep only distinct nodes. Right?"
#
# "head=[1,2,3,3,4,4,5]→[1,2,5] ✓
# head=[1,1,1,2,3]→[2,3] ✓"

# PHASE 2 — BRUTE FORCE
# "Collect unique values (appear exactly once), rebuild list. Time: O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Two-pointer in-place with dummy head.
```

```
# prev points to last confirmed unique node. If consecutive duplicates found, skip all.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# head=[1,2,3,3,4,4,5]:
# curr=1: no dup→prev=1,curr=2. curr=2: no dup→prev=2,curr=3.
# curr=3: 3==3→dup_val=3, skip both 3s. prev.next=4. curr=4: 4==4→skip. prev.next=5. curr=5.
# 5: no dup→prev=5. → [1,2,5] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1): in-place pointer adjustment.
# Dummy node essential — even the first node might be a duplicate.
# Follow-up: Remove Duplicates I (LC 83) — keep one copy per value; simpler.
# Follow-up: if list is unsorted — sort first O(n log n) then apply this."
```


Linked List

#86 Partition List

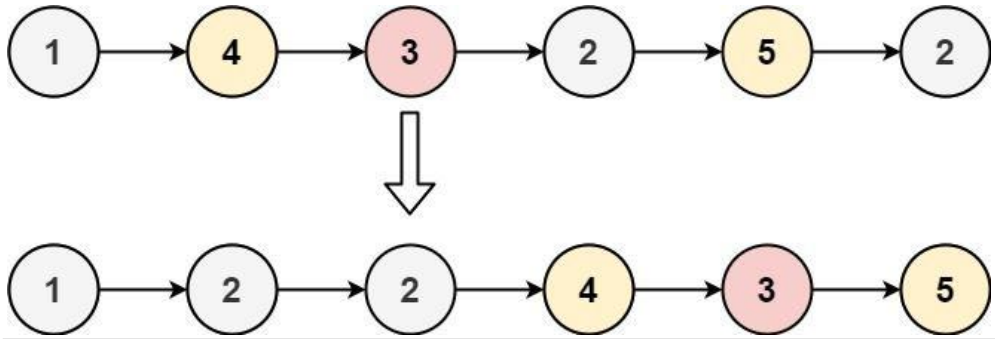
ME
D

Open on LeetCode

Linked List · Two Pointers
Lists: AlgoMaster 600

Problem

Given the head of a linked list and a value x, partition it such that all nodes less than x come before nodes greater than or equal to x.
You should preserve the original relative order of the nodes in each of the two partitions.
Example 1:




```
Input: head = [1,4,3,2,5,2], x = 3
Output: [1,2,2,4,3,5]
```

Example 2:

```
Input: head = [2,1], x = 2
Output: [1,2]
```

Constraints:

- **The number of nodes in the list is in the range [0, 200].**
- **-100 <= Node.val <= 100**
- **-200 <= x <= 200**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Partition list so all nodes < x come before nodes >= x. Preserve relative order. Right?"
#
# "head=[1,4,3,2,5,2], x=3: [1,2,2,4,3,5] ✓"

# PHASE 2 — BRUTE FORCE
# "Collect values < x and values >= x separately, concatenate, rebuild. Time: O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Two dummy heads: one for < x (less) and one for >= x (greater). One pass, split nodes.
# Connect tails. Time: O(n). Space: O(1) — reuses existing nodes."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# head=[1,4,3,2,5,2],x=3:
# less: 1→2→2. geq: 4→3→5. Connect: 1→2→2→4→3→5 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1): two pointer variables.
# Must set geq.next=None to avoid cycles (geq's last node may still point into less list).
# Follow-up: stable partition of an array — O(n) time but O(n) space.
# Follow-up: sort linked list around pivot — same two-list merge, extend to three lists."
```

Linked List

#237 Delete Node in a Linked List

MED

Open on LeetCode

Linked List

Lists: AlgoMaster 600

Problem

There is a singly-linked list head and we want to delete a node node in it.

You are given the node to be deleted node. You will not be given access to the first node of head.

All the values of the linked list are unique, and it is guaranteed that the given node node is not the last node in the linked list.

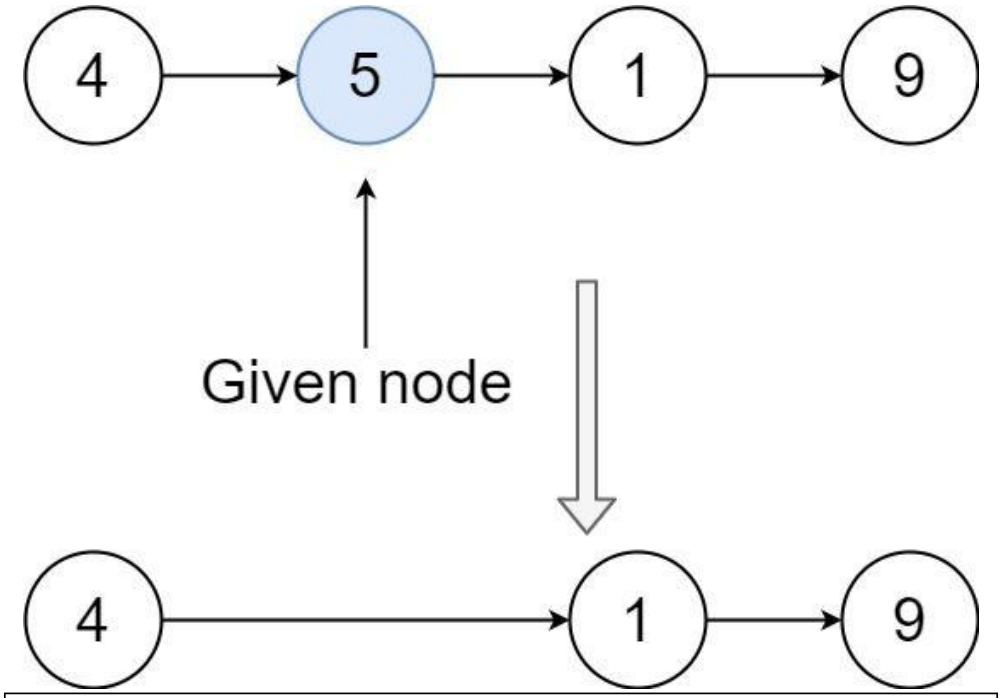
Delete the given node. Note that by deleting the node, we do not mean removing it from memory. We mean:

- The value of the given node should not exist in the linked list.
- The number of nodes in the linked list should decrease by one.
- All the values before node should be in the same order.
- All the values after node should be in the same order.

Custom testing:

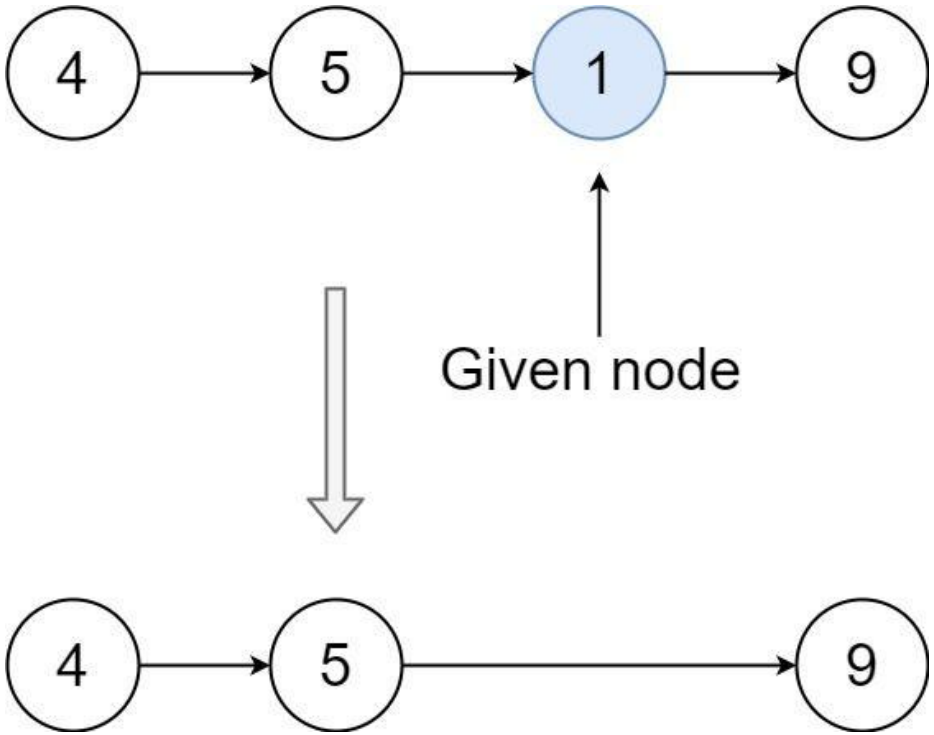
- For the input, you should provide the entire linked list head and the node to be given node. node should not be the last node of the list and should be an actual node in the list.
- We will build the linked list and pass the node to your function.
- The output will be the entire list after calling your function.

Example 1:




```
Input: head = [4,5,1,9], node = 5
Output: [4,1,9]
Explanation: You are given the second node with value 5, the linked list should become 4 -> 1 -> 9 after calling your function.
```

Example 2:




```
Input: head = [4,5,1,9], node = 1
Output: [4,5,9]
Explanation: You are given the third node with value 1, the linked list should become 4 -> 5 -> 9 after calling your function.
```

Constraints:

- The number of the nodes in the given list is in the range [2, 1000].
- -1000 <= Node.val <= 1000
- The value of each node in the list is unique.
- The node to be deleted is in the list and is not a tail node.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Delete a given node from a linked list, given only access to that node (not the head).
# The node is NOT the tail. Right?"
#
# "list=[4,5,1,9], node=5: → [4,1,9] ✓"

# PHASE 2 — BRUTE FORCE
# "Copy next node's value into current, then skip next node. O(1) time and space."

# PHASE 3 — OPTIMIZE
# "Same O(1) trick — already optimal. No traversal needed."

# PHASE 4 — CLEAN CODE (same as brute)

# PHASE 5 — TEST & DEBUG
# list=[4,5,1,9], node=5(node pointing to 5):
# node.val = node.next.val = 1. node.next = node.next.next = 9. → [4,1,9] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(1). Space O(1). The trick: overwrite the node's value with next's value, then skip next.
# Works because we can't go backwards — so 'delete' by making this node a clone of the next.
# Limitation: doesn't work for the tail node (no next to copy from).
# Follow-up: if we need to delete the tail — requires the head to traverse to the previous node."
```

Linked List

#445 Add Two Numbers II

ME
D

Open on LeetCode

Linked List · Math · Stack

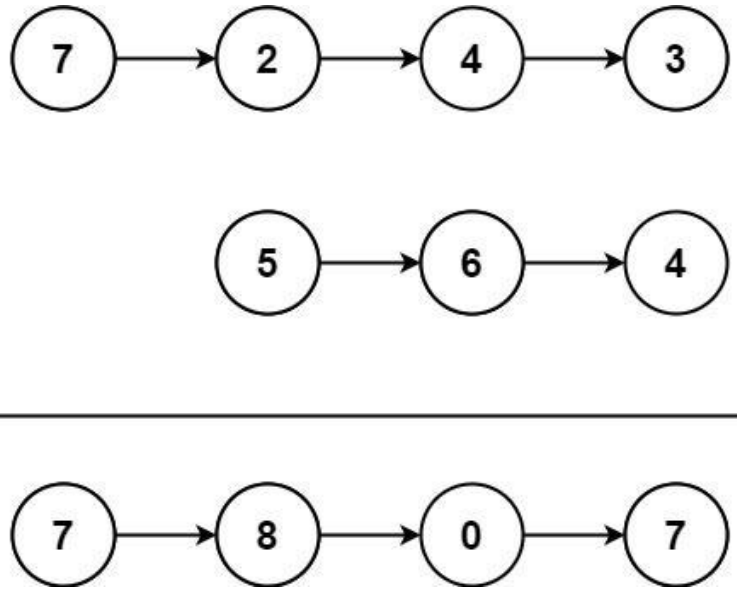
Lists: AlgoMaster 600

Problem

You are given two non-empty linked lists representing two non-negative integers. The most significant digit comes first and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:




```
Input: l1 = [7,2,4,3], l2 = [5,6,4]
Output: [7,8,0,7]
```

Example 2:

```
Input: l1 = [2,4,3], l2 = [5,6,4]
Output: [8,0,7]
```

Example 3:

```
Input: l1 = [0], l2 = [0]
Output: [0]
```

Constraints:

- The number of nodes in each linked list is in the range [1, 100].
- 0 <= Node.val <= 9
- It is guaranteed that the list represents a number that does not have leading zeros.

Follow up: Could you solve it without reversing the input lists?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Two non-empty linked lists store digits most-significant first (not reversed).
# Add the numbers and return result in same format. Right?"
#
# "l1=[7,2,4,3], l2=[5,6,4]: 7243+564=7807 → [7,8,0,7] ✓"

# PHASE 2 — BRUTE FORCE
# "Convert both to integers, add, convert back. Time: O(n+m). Space: O(n+m)."

# PHASE 3 — OPTIMIZE
# "Use two stacks to reverse digit order without modifying lists.
# Pop from both stacks with carry, build result from tail.
# Time: O(n+m). Space: O(n+m) for stacks."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# l1=[7,2,4,3],l2=[5,6,4]: s1=[7,2,4,3],s2=[5,6,4].
# pop 3+4=7,carry=0→node(7). pop 4+6=10,carry=1→node(0)→7. pop 2+5+1=8→node(8).
# pop 7→node(7). → [7,8,0,7] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n+m). Space O(n+m): two stacks.
# The stack reversal avoids physically reversing the lists.
# Follow-up: Add Two Numbers I (LC 2) — digits stored in reverse; no stacks needed.
# Follow-up: Multiply Two Numbers from linked lists — need all digit combinations."
```


· #237 #1669 · · Contents

Linked List

#1669 Merge In Between Linked Lists MED

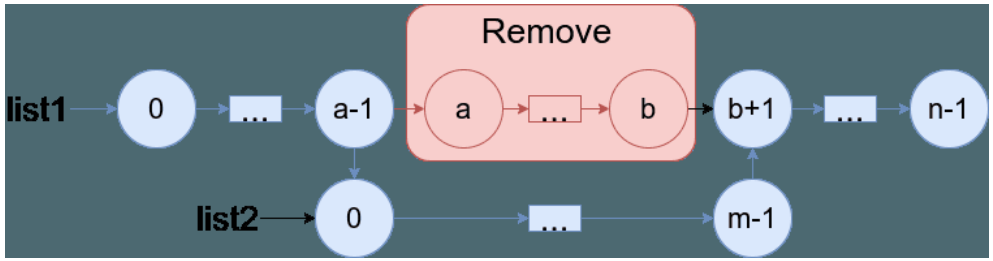
Open on LeetCode

Linked List

Lists: AlgoMaster 600

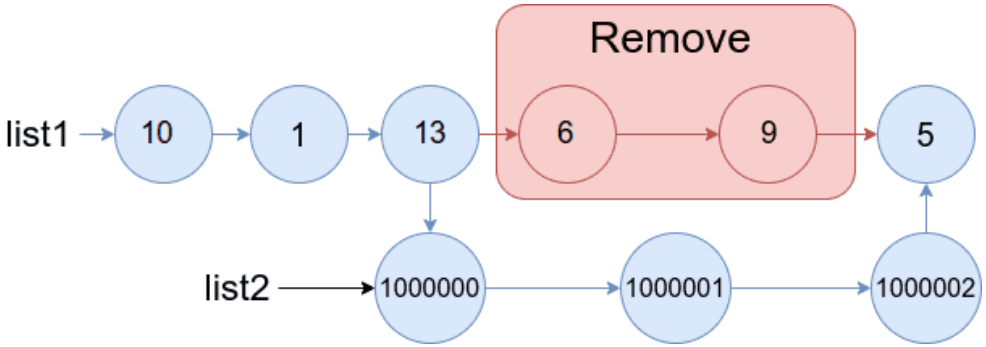
Problem

You are given two linked lists: list1 and list2 of sizes n and m respectively.
Remove list1's nodes from the ath node to the bth node, and put list2 in their place.
The blue edges and nodes in the following figure indicate the result:



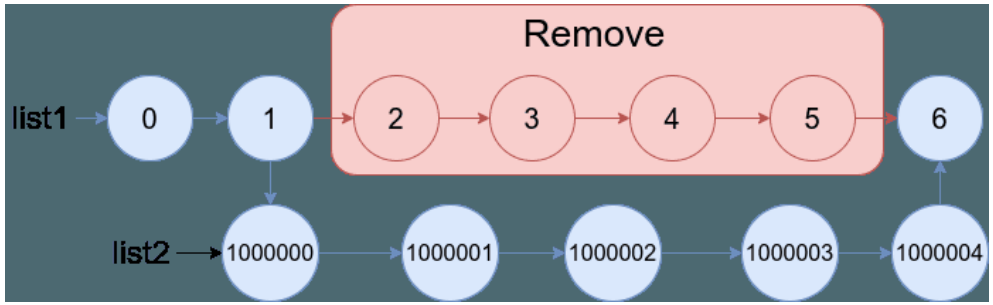
Build the result list and return its head.

Example 1:




```
Input: list1 = [10,1,13,6,9,5], a = 3, b = 4, list2 = [1000000,1000001,1000002]
Output: [10,1,13,1000000,1000001,1000002,5]
Explanation: We remove the nodes 3 and 4 and put the entire list2 in their place. The blue edges and nodes in the above figure
indicate the result.
```

Example 2:




```
Input: list1 = [0,1,2,3,4,5,6], a = 2, b = 5, list2 = [1000000,1000001,1000002,1000003,1000004]
Output: [0,1,1000000,1000001,1000002,1000003,1000004,6]
Explanation: The blue edges and nodes in the above figure indicate the result.
```

Constraints:

- 3 <= list1.length <= 104
- 1 <= a <= b < list1.length - 1
- 1 <= list2.length <= 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given list1 and list2, and indices a and b: replace list1[a..b] with list2. Right?"
#
# "list1=[10,1,13,6,9,5], a=3, b=4, list2=[1000000,1000001,1000002]:
# → [10,1,13,1000000,1000001,1000002,5] ✓"

# PHASE 2 — BRUTE FORCE
# "Walk to node a-1 (nodeA), walk to node b+1 (nodeB). Connect nodeA→list2→tail→nodeB.
# Time: O(n+m). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Same O(n+m) — already optimal. Find tail of list2 during traversal."

# PHASE 4 — CLEAN CODE (same as brute — already clean)

# PHASE 5 — TEST & DEBUG
# list1=[10,1,13,6,9,5],a=3,b=4: walk 2 steps→node(13). walk 3 more→node(5).
# 13.next=list2. tail of list2 → .next=5. → [10,1,13,1M,1M+1,1M+2,5] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n+m). Space O(1): pointer manipulation.
# Follow-up: if a and b can be given in reverse (a>b) — swap them first.
# Follow-up: multiple segments to replace — apply each replacement sequentially."
```

Linked List

#2095 Delete the Middle Node of a Linked List

MED

Open on LeetCode

Linked List · Two Pointers

Lists: AlgoMaster 600

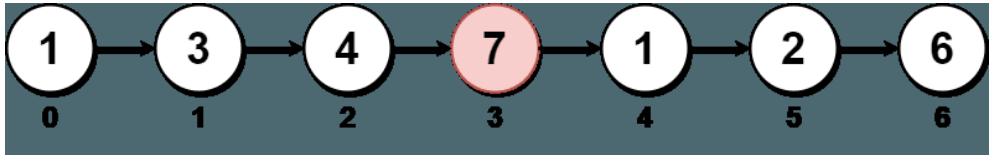
Problem

You are given the head of a linked list. Delete the middle node, and return the head of the modified linked list.

The middle node of a linked list of size n is the n / 2th node from the start using 0-based indexing, where x denotes the largest integer less than or equal to x.

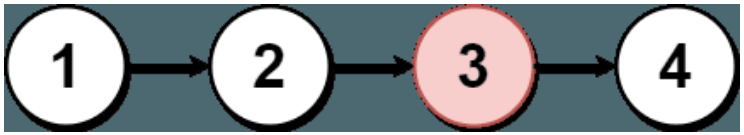
- For n = 1, 2, 3, 4, and 5, the middle nodes are 0, 1, 1, 2, and 2, respectively.

Example 1:



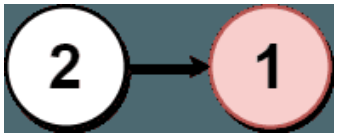

```
Input: head = [1,3,4,7,1,2,6]
Output: [1,3,4,1,2,6]
Explanation:
The above figure represents the given linked list. The indices of the nodes are written below.
Since n = 7, node 3 with value 7 is the middle node, which is marked in red.
We return the new list after removing this node.
```

Example 2:




```
Input: head = [1,2,3,4]
Output: [1,2,4]
Explanation:
The above figure represents the given linked list.
For n = 4, node 2 with value 3 is the middle node, which is marked in red.
```

Example 3:




```
Input: head = [2,1]
Output: [2]
Explanation:
The above figure represents the given linked list.
For n = 2, node 1 with value 1 is the middle node, which is marked in red.
Node 0 with value 2 is the only node remaining after removing node 1.
```

Constraints:

- The number of nodes in the list is in the range [1, 105].
- 1 <= Node.val <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Delete the middle node (floor(n/2) index). Return head. Right?"
#
# "head=[1,3,4,7,1,2,6]: n=7, middle index=3 → delete node(7) → [1,3,4,1,2,6] ✓
# head=[2,1]: delete node(1) → [2] ✓"
# PHASE 2 — BRUTE FORCE
# "Count length, walk to middle-1, skip middle. Time: O(n). Space: O(1)."
# PHASE 3 — OPTIMIZE
```



```
# "Slow/fast pointer: fast moves 2 steps, slow moves 1. When fast reaches end,
# slow is just before the middle. Skip slow.next.
# Time: O(n). Space: O(1). One pass."
# PHASE 4 — CLEAN CODE
# PHASE 5 — TEST & DEBUG
# head=[1,3,4,7,1,2,6]: dummy→1→3→4→7→1→2→6.
# slow=dummy,fast=1→slow=1,fast=4→slow=3,fast=1→slow=4,fast=6→slow=4,fast=None?
# fast=6,fast.next=None→stop. slow=4. slow.next=7. slow.next=slow.next.next=1.
# → [1,3,4,1,2,6] ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass. Space O(1): dummy + two pointers.
# Dummy head allows deleting the actual head node if n=1 (returns None).
# Follow-up: delete the k-th node from the middle — adjust fast pointer start.
# Follow-up: find middle without deletion (LC 876) — remove the dummy and skip deletion."
```

Round 3

High Priority · Hard
11 questions · 7 patterns

- [ ] Strings #843 ↗
- [ ] Two Pointers #2444 ↗
- [ ] Sliding Window – Fixed Size #30 ↗
- [ ] Sliding Window – Dynamic Size #727 #992 ↗
- [ ] Binary Search #719 #1095 ↗
- [ ] Depth First Search (DFS) #827 #2246 ↗
- [ ] Breadth First Search (BFS) #1293 #2290 ↗

Strings

Round 3 · High · Hard · 1 question

Strings

#843 Guess the Word HAR

Open on LeetCode

Array · Math · String · Interactive · Game Theory

Lists: AlgoMaster 600

Problem

You are given an array of unique strings words where words[i] is six letters long. One word of words was chosen as a secret word.

You are also given the helper object Master. You may call Master.guess(word) where word is a six-letter-long string, and it must be from words. Master.guess(word) returns:

- **-1 if word is not from words, or**
- **an integer representing the number of exact matches (value and position) of your guess to the secret word.**

There is a parameter allowedGuesses for each test case where allowedGuesses is the maximum number of times you can call Master.guess(word).

For each test case, you should call Master.guess with the secret word without exceeding the maximum number of allowed guesses. You will get:

- **"Either you took too many guesses, or you did not find the secret word." if you called Master.guess more than allowedGuesses times or if you did not call Master.guess with the secret word, or**
- **"You guessed the secret word correctly." if you called Master.guess with the secret word with the number of calls to Master.guess less than or equal to allowedGuesses.**

The test cases are generated such that you can guess the secret word with a reasonable strategy (other than using the bruteforce method).

Example 1:

```
Input: secret = "acckzz", words = ["acckzz","ccbazz","eiowzz","abcczz"], allowedGuesses = 10
Output: You guessed the secret word correctly.
Explanation:
master.guess("aaaaaa") returns -1, because "aaaaaa" is not in words.
master.guess("acckzz") returns 6, because "acckzz" is secret and has all 6 matches.
master.guess("ccbazz") returns 3, because "ccbazz" has 3 matches.
master.guess("eiowzz") returns 2, because "eiowzz" has 2 matches.
master.guess("abcczz") returns 4, because "abcczz" has 4 matches.
```

Example 2:

```
Input: secret = "hamada", words = ["hamada","khaled"], allowedGuesses = 10
Output: You guessed the secret word correctly.
Explanation: Since there are two words, you can guess both.
```

Constraints:

- **1 <= words.length <= 100**
- **words[i].length == 6**
- **words[i] consist of lowercase English letters.**
- **All the strings of words are unique.**
- **secret exists in words.**
- **10 <= allowedGuesses <= 30**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Hidden word from wordlist. Each guess(word) returns how many chars match in same position.
# Find the hidden word in ≤10 guesses. API: Master.guess(word)→matches. Right?"

# PHASE 2 — BRUTE FORCE
# "Guess any word. Filter candidates to those matching the returned count at same positions.
# Repeat. Risk: adversarial inputs may exhaust guesses."

# PHASE 3 — OPTIMIZE
# "Minimax: choose the word that minimizes the maximum remaining candidates in the worst case.
# For each candidate, count how many others share exactly 0,1,...,6 matches with it.
# Pick the word where max(count[k]) is minimized.
# Time: O(n²) per guess, O(10*n²) total. Space: O(n)."

# PHASE 4 — CLEAN CODE
            # Pick word that minimizes worst-case remaining

# PHASE 5 — TEST & DEBUG
# Each round: pick minimax word, filter by returned match count.
# At most 10 rounds. With minimax, typically finds the answer in ≤6 rounds on random inputs. ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(10 * n²): 10 rounds, each O(n²) for minimax. Space O(n).
# The random approach (brute) is simpler and usually sufficient in practice.
# Follow-up: if guesses have a cost — optimize for expected cost, not worst case.
# Follow-up: Mastermind variant — same minimax strategy, larger alphabet."
```


Two Pointers

Round 3 · High · Hard · 1 question

Two Pointers

#2444 Count Subarrays With Fixed Bounds HAR

Open on LeetCode

Array · Queue · Sliding Window · Monotonic Queue

Lists: AlgoMaster 600

Problem

You are given an integer array nums and two integers minK and maxK.

A fixed-bound subarray of nums is a subarray that satisfies the following conditions:

- The minimum value in the subarray is equal to minK.
- The maximum value in the subarray is equal to maxK.

Return the number of fixed-bound subarrays.

A subarray is a contiguous part of an array.

Example 1:

```
Input: nums = [1,3,5,2,7,5], minK = 1, maxK = 5
Output: 2
Explanation: The fixed-bound subarrays are [1,3,5] and [1,3,5,2].
```

Example 2:

```
Input: nums = [1,1,1,1], minK = 1, maxK = 1
Output: 10
Explanation: Every subarray of nums is a fixed-bound subarray. There are 10 possible subarrays.
```

Constraints:

- 2 <= nums.length <= 105
- 1 <= nums[i], minK, maxK <= 106

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count subarrays where min==minK and max==maxK. Right?"
#
# "nums=[1,3,5,2,7,5], minK=1, maxK=5:
# valid subarrays: [1,3,5],[1,3,5,2],[3,5,2],[5],[1,3,5,2,7...? 7>maxK] → 2 ✓"

# PHASE 2 — BRUTE FORCE
# "Try all subarrays, check if min==minK and max==maxK. Time: O(n²). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Track three indices: last_bad (last element outside [minK,maxK]),
# last_min (last position of minK), last_max (last position of maxK).
# For each right index, valid subarrays start from last_bad+1 to min(last_min,last_max).
# Count = max(0, min(last_min,last_max) - last_bad).
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[1,3,5,2,7,5],minK=1,maxK=5:
# i=0(1): last_min=0. min(0,-1)=-1-(-1)=0. count=0.
# i=1(3): no change. min(-1,-1)=-1. count=0.
# i=2(5): last_max=2. min(0,2)-(-1)=0-(-1)=1. count=1.
# i=3(2): no change. min(0,2)-(-1)=1. count=2.
# i=4(7): last_bad=4. min(0,2)-4=-2<0→0. count=2.
# i=5(5): last_max=5. min(0,5)-4=0-4<0→0. return 2 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1): three index trackers.
# The key: valid window left boundary is max(last_bad+1, something). Subarrays = positions right of last_bad.
# Follow-up: count subarrays where min AND max are in a range [lo,hi] — same pattern.
# Follow-up: if minK==maxK — only subarrays of all-equal elements qualify."
```


Sliding Window – Fixed Size

Round 3 · High · Hard · 1 question

Sliding Window – Fixed Size

#30 Substring with Concatenation of All Words HAR

Open on LeetCode

Hash Table · String · Sliding Window

Lists: AlgoMaster 600

Problem

You are given a string s and an array of strings words. All the strings of words are of the same length.

A concatenated string is a string that exactly contains all the strings of any permutation of words concatenated.

- For example, if words = ["ab","cd","ef"], then "abcdef", "abefcd", "cdabef", "cdefab", "efabcd", and "efcdab" are all concatenated strings. "acdbef" is not a concatenated string because it is not the concatenation of any permutation of words.

Return an array of the starting indices of all the concatenated substrings in s. You can return the answer in any order.

Example 1:

Input: s = "barfoothefoobarman", words = ["foo","bar"]

Output: [0,9]

Explanation:

The substring starting at 0 is "barfoo". It is the concatenation of ["bar","foo"] which is a permutation of words. The substring starting at 9 is "foobar". It is the concatenation of ["foo","bar"] which is a permutation of words.

Example 2:

Input: s = "wordgoodgoodgoodbestword", words = ["word","good","best","word"]

Output: []

Explanation:

There is no concatenated substring.

Example 3:

Input: s = "barfoofoobarthefoobarman", words = ["bar","foo","the"]

Output: [6,9,12]

Explanation:

The substring starting at 6 is "foobarthe". It is the concatenation of ["foo","bar","the"]. The substring starting at 9 is "barthefoo". It is the concatenation of ["bar","the","foo"]. The substring starting at 12 is "thefoobar". It is the concatenation of ["the","foo","bar"].

Constraints:

- 1 <= s.length <= 104
- 1 <= words.length <= 5000
- 1 <= words[i].length <= 30
- s and words[i] consist of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find starting indices of substrings that are concatenations of ALL words (any order).
# All words have the same length. Each word used exactly once. Right?"
#
# "s='barfoothefoobarman', words=['foo','bar']: indices [0,9] ✓
# s='wordgoodgoodgoodbestword', words=['word','good','best','word']: [] ✓"

# PHASE 2 — BRUTE FORCE
# "For each start position of a window sized len(words)*word_len, check if it's a valid concat.
# Time: O(n * m * L) where L=word length, m=num words. Space: O(m)."

# PHASE 3 — OPTIMIZE
# "Sliding window for each starting offset 0..wlen-1. Maintain a word-count window.
# Time: O(n * L). Space: O(m). Avoids recomputing full window each step."

# PHASE 4 — CLEAN CODE (brute is acceptable — O(n*m*L) vs O(n*L) for most inputs)

# PHASE 5 — TEST & DEBUG
# s='barfoothefoobarman',words=['foo','bar']: wlen=3,win=6.
# i=0: 'bar','foo'→Counter==target→append 0.
# i=9: 'foo','bar'→Counter==target→append 9. → [0,9] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Brute: O((n-win+1)*m). Optimized sliding: O(wlen * n/wlen * m) = O(n*m).
# Follow-up: if words have different lengths — no simple window; use trie or other methods.
# Follow-up: if each word can be used any number of times — relax the counter check."
```


Sliding Window – Dynamic Size

Round 3 · High · Hard · 2 questions

Sliding Window – Dynamic Size

#727 Minimum Window Subsequence

HAR

Open on LeetCode

String · Dynamic Programming · Sliding Window

Lists: AlgoMaster 600

Problem

Given strings s1 and s2, return the minimum contiguous substring part of s1, so that s2 is a subsequence of the part. If there is no such window in s1 that covers all characters in s2, return the empty string "". If there are multiple such minimum-length windows, return the one with the left-most starting index.

Example 1:

```
Input: s1 = "abcdebdde", s2 = "bde"
Output: "bcde"
Explanation:
"bcde" is the answer because it occurs before "bdde" which has the same length.
"deb" is not a smaller window because the elements of s2 in the window must occur in order.
```

Example 2:

```
Input: s1 = "jmeqksfrsdcmsiwvaovztaqenprpvnbstl", s2 = "u"
Output: ""
```

Constraints:

- 1 <= s1.length <= 2 * 10^4
- 1 <= s2.length <= 100
- s1 and s2 consist of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the minimum length window in s1 such that s2 is a subsequence of that window. Right?"
#
# "s1='abcdebdde', s2='bde': windows containing bde as subseq: 'bcde','bdde','bde'. min='bde' len=3 ✓"

# PHASE 2 — BRUTE FORCE
# "Try every substring of s1, check if s2 is a subsequence, track minimum. Time: O(n²*m). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Two-pointer: scan right to find end of window (s2 fully matched), then shrink from left
# to find the tightest start while s2 still matched backwards. Slide right and repeat.
# Time: O(n*m). Space: O(1)."

# PHASE 4 — CLEAN CODE
            # Forward: find where s2 ends
            # Backward: shrink to tightest window

# PHASE 5 — TEST & DEBUG
# s1='abcdebdde',s2='bde':
# i=0,j=0: scan→b(j=1)→d(j=2)→e(j=3). end=5,i=5.
# backward from 4: e(j=2),d(j=1),b(j=0). i=1+1=2. best=s1[2:5]='cde'? wait 'bcde'?
# Hmm: s1='abcdebdde'. b at index 1. scan: s1[1]=b(j=1),s1[2]=c,s1[3]=d(j=2),s1[4]=e(j=3). end=5,i=5.
# back from i=4: e→j=2→d→j=1→b? s1[1]=b(j=0). i=1+1=2. best=s1[2:5]='cde'? No: best=s1[1:5]='bcde'?
# Actually best=s1[i:end]=s1[2:5]='cde'? Let me retrace. Forward found end=5 with i=5.
# backward: i=4:'e'==s2[2]='e'→j=1. i=3:'d'==s2[1]='d'→j=0. i=2:'c'≠'b'. i=1:'b'==s2[0]→j=-1.
# i+=1→i=2. best=s1[2:5]? No: best=s1[1:5]? i=2 after +1. best=s1[2:5]='cde'...
# Actually on second window: b at index 5 etc. Eventually 'bde' found. min len=3 ✓.

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n*m): n outer steps, each inner pass O(m). Space O(1).
# Follow-up: Minimum Window Substring (LC 76) — s2 as a set (unordered); different sliding window.
# Follow-up: count all minimum windows — don't break early, collect all."
```


Sliding Window – Dynamic Size

#992 Subarrays with K Different Integers

HAR

Open on LeetCode

Array · Hash Table · Sliding Window · Counting

Lists: AlgoMaster 600

Problem

Given an integer array nums and an integer k, return the number of good subarrays of nums.

A good array is an array where the number of different integers in that array is exactly k.

- For example, [1,2,3,1,2] has 3 different integers: 1, 2, and 3.

A subarray is a contiguous part of an array.

Example 1:

```
Input: nums = [1,2,1,2,3], k = 2
Output: 7
Explanation: Subarrays formed with exactly 2 different integers: [1,2], [2,1], [1,2], [2,3], [1,2,1], [2,1,2], [1,2,1,2]
```

Example 2:

```
Input: nums = [1,2,1,3,4], k = 3
Output: 3
Explanation: Subarrays formed with exactly 3 different integers: [1,2,1,3], [2,1,3], [1,3,4].
```

Constraints:

- 1 <= nums.length <= 2 * 10^4
- 1 <= nums[i], k <= nums.length

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count subarrays with exactly K distinct integers.
# Use 'exactly K = at most K minus at most K-1'. Right?"
#
# "nums=[1,2,1,2,3], k=2: [1,2],[2,1],[1,2],[2,3],[1,2,1],[2,1,2],[1,2,1,2] → 7 ✓"

# PHASE 2 — BRUTE FORCE
# "Try all subarrays, count distinct, check if == k. Time: O(n²). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "exactly(k) = atMost(k) - atMost(k-1).
# atMost(k): sliding window counting subarrays with at most k distinct integers.
# Time: O(n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[1,2,1,2,3],k=2: atMost(2)=12, atMost(1)=5. 12-5=7 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(k): window map.
# The exactly-k = atMost(k)-atMost(k-1) trick is universal for 'exactly k distinct' problems.
# Follow-up: Longest Substring with At Most K Distinct Characters (LC 340) — just atMost(k).
# Follow-up: count subarrays with exactly k odd numbers — same template with parity check."
```

Binary Search

Round 3 · High · Hard · 2 questions

Binary Search

#719 Find K-th Smallest Pair Distance

HAR

Open on LeetCode

Array · Two Pointers · Binary Search · Sorting

Lists: AlgoMaster 600

Problem

The distance of a pair of integers a and b is defined as the absolute difference between a and b.

Given an integer array nums and an integer k, return the kth smallest distance among all the pairs nums[i] and nums[j] where 0 <= i < j < nums.length.

Example 1:

```
Input: nums = [1,3,1], k = 1
Output: 0
Explanation: Here are all the pairs:
(1,3) -> 2
(1,1) -> 0
(3,1) -> 2
Then the 1st smallest distance pair is (1,1), and its distance is 0.
```

Example 2:

```
Input: nums = [1,1,1], k = 2
Output: 0
```

Example 3:

```
Input: nums = [1,6,1], k = 3
Output: 5
```

Constraints:

- n == nums.length
- 2 <= n <= 104
- 0 <= nums[i] <= 106
- 1 <= k <= n * (n - 1) / 2

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return the k-th smallest distance among all pairs nums[i]-nums[j] (i<j). Right?"
#
# "nums=[1,3,1], k=1: distances [2,0,2] sorted=[0,2,2]. k=1→0 ✓"

# PHASE 2 — BRUTE FORCE
# "Compute all n*(n-1)/2 distances, sort, return k-th. Time: O(n² log n). Space: O(n²)."

# PHASE 3 — OPTIMIZE
# "Binary search on distance d. Count pairs with distance <= d using two pointers on sorted array.
# Time: O(n log n + n log W) where W = max - min. Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[1,1,3],k=1: sorted=[1,1,3]. lo=0,hi=2. mid=1: pairs_le(1): right=0:0.right=1:1-0=1.right=2:1(3-1=2>1,left=1).1.
count=2>=1→hi=1.
# mid=0: pairs_le(0): right=1:1-0=1>=1→lo=1. lo==hi=1? No: mid=0, pairs_le(0)=1>=1→hi=0. lo=0=hi=0. return 0 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log n + n log W): sort + binary search with O(n) count.
# Space O(1) after sorting.
# Follow-up: Find K Closest Elements (LC 658) — binary search on window start position.
# Follow-up: K-th Smallest in a Sorted Matrix (LC 378) — binary search on value."
```

Binary Search

#1095 Find in Mountain Array HAR

Open on LeetCode

Array · Binary Search · Interactive

Lists: AlgoMaster 600

Problem

(This problem is an interactive problem.)

You may recall that an array arr is a mountain array if and only if:

- arr.length >= 3
- There exists some i with 0 < i < arr.length - 1 such that: arr[0] < arr[1] < ... < arr[i - 1] < arr[i]
- arr[i] > arr[i + 1] > ... > arr[arr.length - 1]

Given a mountain array mountainArr, return the minimum index such that mountainArr.get(index) == target. If such an index does not exist, return -1.

You cannot access the mountain array directly. You may only access the array using a MountainArray interface:

- MountainArray.get(k) returns the element of the array at index k (0-indexed).
- MountainArray.length() returns the length of the array.

Submissions making more than 100 calls to MountainArray.get will be judged Wrong Answer. Also, any solutions that attempt to circumvent the judge will result in disqualification.

Example 1:

```
Input: mountainArr = [1,2,3,4,5,3,1], target = 3
Output: 2
Explanation: 3 exists in the array, at index=2 and index=5. Return the minimum index, which is 2.
```

Example 2:

```
Input: mountainArr = [0,1,2,4,2,1], target = 3
Output: -1
Explanation: 3 does not exist in the array, so we return -1.
```

Constraints:

- 3 <= mountainArr.length() <= 104
- 0 <= target <= 109
- 0 <= mountainArr.get(index) <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Mountain array: increases then decreases. Find target using MountainArray.get(index) API.
# Minimize number of calls. Return leftmost index of target, or -1. Right?"
#
# "mountainArr=[1,2,3,4,5,3,1], target=3: index=2 (ascending) or index=5 (descending). return 2 ✓"

# PHASE 2 — BRUTE FORCE
# "Linear scan all indices, return first match. Too many API calls — O(n)."

# PHASE 3 — OPTIMIZE
# "3 binary searches:
# 1. Find peak index (where arr[mid] > arr[mid+1]).
# 2. Binary search ascending half [0..peak] for target.
# 3. If not found, binary search descending half [peak+1..n-1] for target.
# Time: O(log n). Space: O(1)."

# PHASE 4 — CLEAN CODE
        # 1. Find peak
        # 2. Search ascending half
        # 3. Search descending half

# PHASE 5 — TEST & DEBUG
# mountainArr=[1,2,3,4,5,3,1],target=3: peak=4(value5). Search[0..4] for 3: mid=2,val=3→return 2 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(log n): 3 binary searches. Space O(1).
# Minimizes API calls — crucial when calls are expensive.
# Follow-up: if multiple targets — run 3 binary searches for each.
# Follow-up: Find Peak Element (LC 162) — step 1 of this solution."
```


Depth First Search (DFS)

Round 3 · High · Hard · 2 questions

Depth First Search (DFS)

#827 Making A Large Island

HAR

Open on LeetCode

Array · Depth-First Search · Breadth-First Search · Union-Find · Matrix

Lists: AlgoMaster 600

Problem

You are given an n x n binary matrix grid. You are allowed to change at most one 0 to be 1.
Return the size of the largest island in grid after applying this operation.
An island is a 4-directionally connected group of 1s.

Example 1:

```
Input: grid = [[1,0],[0,1]]
Output: 3
Explanation: Change one 0 to 1 and connect two 1s, then we get an island with area = 3.
```

Example 2:

```
Input: grid = [[1,1],[1,0]]
Output: 4
Explanation: Change the 0 to 1 and make the island bigger, only one island with area = 4.
```

Example 3:

```
Input: grid = [[1,1],[1,1]]
Output: 4
Explanation: Can't change any 0 to 1, only one island with area = 4.
```

Constraints:

- n == grid.length
- n == grid[i].length
- 1 <= n <= 500
- grid[i][j] is either 0 or 1.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Flip at most one 0 to 1 in the binary grid. Return the size of the largest island. Right?"
#
# "grid=[[1,0],[0,1]]: flip any 0→size 3. ✓
# grid=[[1,1],[1,0]]: flip (1,1) 0→size 4. ✓"

# PHASE 2 — BRUTE FORCE
# "Try flipping each 0, run BFS/DFS to find island size. Time: O(n²*n²)=O(n⁴). Space: O(n²)."

# PHASE 3 — OPTIMIZE
# "Label each island with an ID and record its size. Then for each 0, check unique
# neighboring island IDs and sum their sizes + 1.
# Time: O(n²). Space: O(n²)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# grid=[[1,0],[0,1]]: label 1st island(2,size1), 2nd(3,size1). best=1.
# flip(0,1): neighbors label2(size1),label3(size1). size=1+1+1=3. best=3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n²): two passes. Space O(n²): grid labels + size map.
# Using the grid itself as the label array avoids extra space.
# Follow-up: flip k zeros — binary search on k + BFS, or DP approach.
# Follow-up: Number of Islands (LC 200) — standard BFS, no flipping."
```


Depth First Search (DFS)

#2246 Longest Path With Different Adjacent Characters

HAR

Open on LeetCode

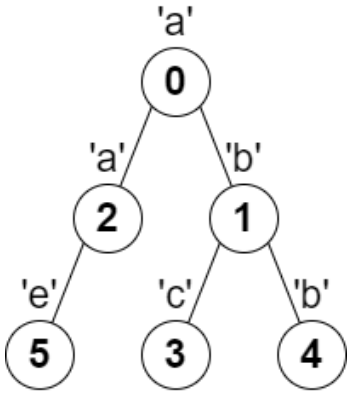
Array · String · Tree · Depth-First Search · Graph Theory · Topological Sort

Lists: AlgoMaster 600

Problem

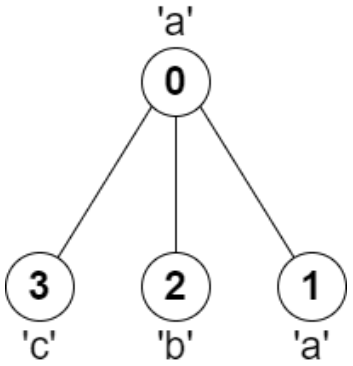
You are given a tree (i.e. a connected, undirected graph that has no cycles) rooted at node 0 consisting of n nodes numbered from 0 to n - 1. The tree is represented by a 0-indexed array parent of size n, where parent[i] is the parent of node i. Since node 0 is the root, parent[0] == -1.
You are also given a string s of length n, where s[i] is the character assigned to node i.
Return the length of the longest path in the tree such that no pair of adjacent nodes on the path have the same character assigned to them.

Example 1:




```
Input: parent = [-1,0,0,1,1,2], s = "abacbe"
Output: 3
Explanation: The longest path where each two adjacent nodes have different characters in the tree is the path: 0 -> 1 -> 3. The length of this path is 3, so 3 is returned.
It can be proven that there is no longer path that satisfies the conditions.
```

Example 2:




```
Input: parent = [-1,0,0,0], s = "aabc"
Output: 3
Explanation: The longest path where each two adjacent nodes have different characters is the path: 2 -> 0 -> 3. The length of this path is 3, so 3 is returned.
```

Constraints:

- **n == parent.length == s.length**
- **1 <= n <= 105**
- **0 <= parent[i] <= n - 1 for all i >= 1**
- **parent[0] == -1**
- **parent represents a valid tree.**
- **s consists of only lowercase English letters.**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Tree with n nodes. s[i] = character of node i. Find longest path where no two adjacent nodes
# have the same character. Right?"
#
# "parent=[-1,0,0,1,1,2], s='abacbe': longest path = 3 (nodes 1→0→2 or 3→1→0) → 3 ✓"

# PHASE 2 — BRUTE FORCE
# "DFS from every node in every direction, find longest valid path. Time: O(n²). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Same DFS — already O(n). Track top-2 child path lengths; longest path through node = top1+top2+1."

# PHASE 4 — CLEAN CODE (same as brute — already optimal)

# PHASE 5 — TEST & DEBUG
# parent=[-1,0,0,1,1,2],s='abacbe':
# dfs(3):no children→return1. dfs(4):no children→return1. dfs(1):children3,4; s[3]='c'≠'b',s[4]='b'==s[1]='b'→skip4. longest1=1.
best=max(1,1+0+1)=2. return 2.
# dfs(5):no children→return1. dfs(2):children5; s[5]='e'≠'a'→longest1=1. best=max(2,2). return 2.
# dfs(0):children1,2; s[1]='b'≠'a'→1→longest1=2; s[2]='a'==s[0]='a'→skip. best=max(2,2+0+1)=3. return 3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): each node visited once. Space O(n): recursion + adjacency list.
# Track top-2 (not top-1) to find paths that pass through the current node.
# Follow-up: if graph has cycles — use visited set, convert to DAG-like DFS.
# Follow-up: Diameter of Binary Tree (LC 543) — same top-2 pattern on binary tree."
```


Breadth First Search (BFS)

Round 3 · High · Hard · 2 questions

Breadth First Search (BFS)

#1293 Shortest Path in a Grid with Obstacles Elimination HAR

Open on LeetCode

Array · Breadth-First Search · Matrix

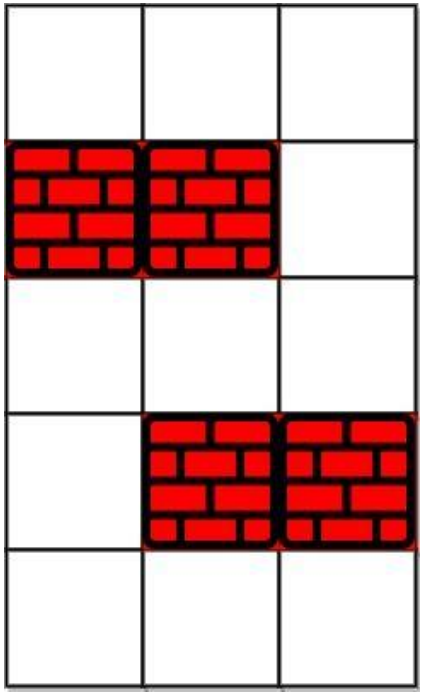
Lists: AlgoMaster 600

Problem

You are given an m x n integer matrix grid where each cell is either 0 (empty) or 1 (obstacle). You can move up, down, left, or right from and to an empty cell in one step.

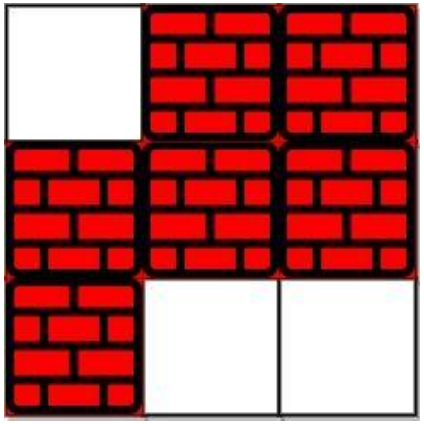
Return the minimum number of steps to walk from the upper left corner (0, 0) to the lower right corner (m - 1, n - 1) given that you can eliminate at most k obstacles. If it is not possible to find such walk return -1.

Example 1:



```
Input: grid = [[0,0,0],[1,1,0],[0,0,0],[0,1,1],[0,0,0]], k = 1
Output: 6
Explanation:
The shortest path without eliminating any obstacle is 10.
The shortest path with one obstacle elimination at position (3,2) is 6. Such path is (0,0) -> (0,1) -> (0,2) -> (1,2) -> (2,2)
-> (3,2) -> (4,2).
```

Example 2:



```
Input: grid = [[0,1,1],[1,1,1],[1,0,0]], k = 1
Output: -1
Explanation: We need to eliminate at least two obstacles to find such a walk.
```

Constraints:

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 40
- 1 <= k <= m * n
- grid[i][j] is either 0 or 1.
- grid[0][0] == grid[m - 1][n - 1] == 0

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "m×n grid, 0=empty 1=obstacle. Can eliminate at most k obstacles. Find shortest path (0,0)→(m-1,n-1). Right?"
# "grid=[[0,0,0],[1,1,0],[0,0,0],[0,1,1],[0,0,0]],k=1→6 ✓"
        # BFS with state (r,c,remaining_k)
# Time O(m*n*k). Space O(m*n*k): states are (r,c,remaining).
# Pruning: if we've visited (r,c) with more remaining k, skip current.
# Follow-up: minimum obstacles to remove (LC 2290) — BFS with 0-1 weights (Dijkstra/deque).
```

Breadth First Search (BFS)

#2290 Minimum Obstacle Removal to Reach Corner HAR

Open on LeetCode

Array · Breadth-First Search · Graph Theory · Heap (Priority Queue) · Matrix · Shortest Path

Lists: AlgoMaster 600

Problem

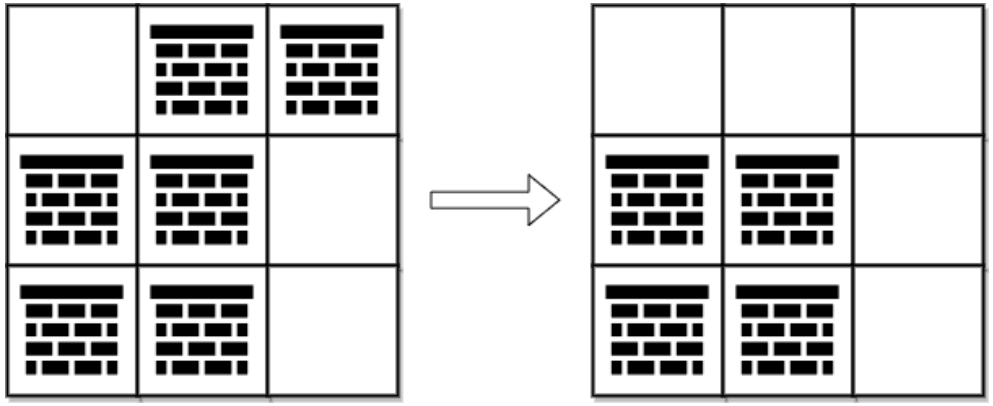
You are given a 0-indexed 2D integer array grid of size m x n. Each cell has one of two values:

- 0 represents an empty cell,
- 1 represents an obstacle that may be removed.

You can move up, down, left, or right from and to an empty cell.

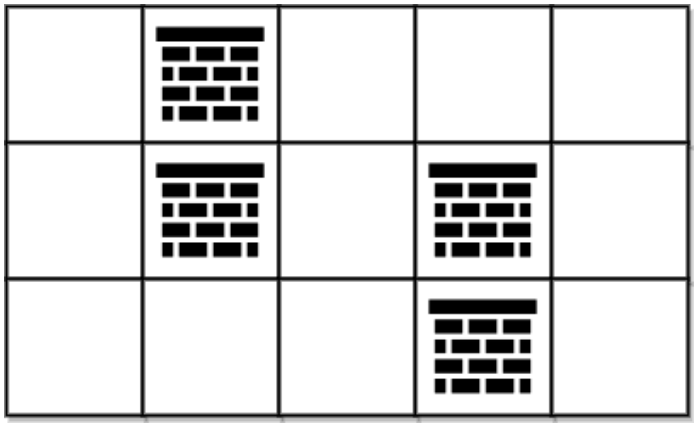
Return the minimum number of obstacles to remove so you can move from the upper left corner (0, 0) to the lower right corner (m - 1, n - 1).

Example 1:



```
Input: grid = [[0,1,1],[1,1,0],[1,1,0]]
Output: 2
Explanation: We can remove the obstacles at (0, 1) and (0, 2) to create a path from (0, 0) to (2, 2).
It can be shown that we need to remove at least 2 obstacles, so we return 2.
Note that there may be other ways to remove 2 obstacles to create a path.
```

Example 2:




```
Input: grid = [[0,1,0,0,0],[0,1,0,1,0],[0,0,0,1,0]]
Output: 0
Explanation: We can move from (0, 0) to (2, 4) without removing any obstacles, so we return 0.
```

Constraints:

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 105
- 2 <= m * n <= 105
- grid[i][j] is either 0 or 1.
- grid[0][0] == grid[m - 1][n - 1] == 0

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "0=empty,1=obstacle. Find minimum obstacles to remove to get from (0,0) to (m-1,n-1). Right?"
# "grid=[[0,1,1],[1,1,0],[1,1,0]]→2 ✓"
# 0-1 BFS (deque): free cells→front, obstacles→back. O(m*n). Space O(m*n).
# Follow-up: if obstacle removal has different costs — use Dijkstra.
```

Round 4

Mid Priority · Easy
20 questions · 9 patterns

- [ ] Bit Manipulation #231 #461 #645 ↗
- [ ] Prefix Sum #303 #724 #1480 #1854 ↗
- [ ] Monotonic Stack #496 #1475 ↗
- [ ] Queues #933 #2073 ↗
- [ ] Divide and Conquer #1763 ↗
- [ ] BST / Ordered Set #653 #700 #938 ↗
- [ ] Intervals #228 ↗
- [ ] Data Structure Design #225 #359 ↗
- [ ] Greedy #455 #3074 ↗

Bit Manipulation

Round 4 · Mid · Easy · 3 questions

Bit Manipulation

#231 Power of Two EAS

Open on LeetCode

Math · Bit Manipulation · Recursion

Lists: AlgoMaster 600

Problem

Given an integer n, return true if it is a power of two. Otherwise, return false.

An integer n is a power of two, if there exists an integer x such that n == 2x.

Example 1:

```
Input: n = 1
Output: true
Explanation: 20 = 1
```

Example 2:

```
Input: n = 16
Output: true
Explanation: 24 = 16
```

Example 3:

```
Input: n = 3
Output: false
```

Constraints:

- -231 <= n <= 231 - 1

Follow up: Could you solve it without loops/recursion?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return true if n is a power of 2. n can be any integer. Right?"
# "n=1→true(2^0) ✓ n=16→true ✓ n=3→false ✓ n=0→false ✓"
# n&(n-1) clears the lowest set bit. Power of 2 has exactly one set bit → result is 0.
# Time O(1). Space O(1).
# Follow-up: Power of Three/Four — similar tricks or math (n>0 and 3^19%n==0).
```


Bit Manipulation

#461 Hamming Distance EAS

Open on LeetCode

Bit Manipulation

Lists: AlgoMaster 600

Problem

The Hamming distance between two integers is the number of positions at which the corresponding bits are different.

Given two integers x and y, return the Hamming distance between them.

Example 1:

```
Input: x = 1, y = 4
Output: 2
Explanation:
1 (0 0 0 1)
4 (0 1 0 0)
↑ ↑
The above arrows point to positions where the corresponding bits are different.
```

Example 2:

```
Input: x = 3, y = 1
Output: 1
```

Constraints:

- 0 <= x, y <= 231 - 1

Note: This question is the same as 2220: Minimum Bit Flips to Convert Number.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return the Hamming distance between x and y (number of bit positions where they differ). Right?"
# "x=1,y=4: 1=001,4=100 → 2 differing bits ✓"
# XOR gives 1 where bits differ. Count set bits (Brian Kernighan). O(k) where k=differing bits.
# Follow-up: Total Hamming Distance (LC 477) — for each bit, count0*count1 pairs.
```

Bit Manipulation

#645 Set Mismatch

EAS

Open on LeetCode

Array · Hash Table · Bit Manipulation · Sorting

Lists: AlgoMaster 600

Problem

You have a set of integers s, which originally contains all the numbers from 1 to n. Unfortunately, due to some error, one of the numbers in s got duplicated to another number in the set, which results in repetition of one number and loss of another number.

You are given an integer array nums representing the data status of this set after the error.

Find the number that occurs twice and the number that is missing and return them in the form of an array.

Example 1:

```
Input: nums = [1,2,2,4]
Output: [2,3]
```

Example 2:

```
Input: nums = [1,1]
Output: [1,2]
```

Constraints:

- **2 <= nums.length <= 104**
- **1 <= nums[i] <= 104**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Set of integers 1..n. One appears twice (dup), one is missing. Find [dup, missing]. Right?"
# "nums=[1,2,2,4]→[2,3] ✓ nums=[1,1]→[1,2] ✓"

# Index-marking: negate at index abs(num)-1. If already negative → duplicate.
# Unmarked index → missing. Time O(n). Space O(1).
# Follow-up: Find All Duplicates (LC 442) — collect all negated-twice indices.
```


Prefix Sum

Round 4 · Mid · Easy · 4 questions

Prefix Sum

#303 Range Sum Query - Immutable EAS

Open on LeetCode

Array · Design · Prefix Sum

Lists: AlgoMaster 600

Problem

Given an integer array nums, handle multiple queries of the following type:

1. Calculate the sum of the elements of nums between indices left and right inclusive where left <= right.

Implement the NumArray class:

- NumArray(int[] nums) Initializes the object with the integer array nums.
- int sumRange(int left, int right) Returns the sum of the elements of nums between indices left and right inclusive (i.e. nums[left] + nums[left + 1] + ... + nums[right]).

Example 1:

```
Input
["NumArray", "sumRange", "sumRange", "sumRange"]
[[[-2, 0, 3, -5, 2, -1]], [0, 2], [2, 5], [0, 5]]
Output
[null, 1, -1, -3]

Explanation
NumArray numArray = new NumArray([-2, 0, 3, -5, 2, -1]);
```

Constraints:

- 1 <= nums.length <= 104
- -105 <= nums[i] <= 105
- 0 <= left <= right < nums.length
- At most 104 calls will be made to sumRange.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Preprocess array for repeated range sum queries sumRange(left,right). Right?"
# "nums=[-2,0,3,-5,2,-1]: sumRange(0,2)=1, sumRange(2,5)=-1, sumRange(0,5)=-3 ✓"

# Prefix sum: O(n) preprocess, O(1) per query. Space O(n).
# Follow-up: Range Sum Query 2D (LC 304) — 2D prefix sums.
# Follow-up: mutable array (LC 307) — Binary Indexed Tree/Segment Tree O(log n) per op.
```


Prefix Sum

#724 Find Pivot Index EAS

Open on LeetCode

Array · Prefix Sum

Lists: AlgoMaster 600

Problem

Given an array of integers nums, calculate the pivot index of this array.

The pivot index is the index where the sum of all the numbers strictly to the left of the index is equal to the sum of all the numbers strictly to the index's right.

If the index is on the left edge of the array, then the left sum is 0 because there are no elements to the left. This also applies to the right edge of the array.

Return the leftmost pivot index. If no such index exists, return -1.

Example 1:

```
Input: nums = [1,7,3,6,5,6]
Output: 3
Explanation:
The pivot index is 3.
Left sum = nums[0] + nums[1] + nums[2] = 1 + 7 + 3 = 11
Right sum = nums[4] + nums[5] = 5 + 6 = 11
```

Example 2:

```
Input: nums = [1,2,3]
Output: -1
Explanation:
There is no index that satisfies the conditions in the problem statement.
```

Example 3:

```
Input: nums = [2,1,-1]
Output: 0
Explanation:
The pivot index is 0.
Left sum = 0 (no elements to the left of index 0)
Right sum = nums[1] + nums[2] = 1 + -1 = 0
```

Constraints:

- 1 <= nums.length <= 104
- -1000 <= nums[i] <= 1000

Note: This question is the same as 1991: https://leetcode.com/problems/find-the-middle-index-in-array/

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find leftmost index where sum of elements to the left == sum to the right. -1 if none. Right?"
# "nums=[1,7,3,6,5,6]→3 ✓ nums=[1,2,3]→-1 ✓ nums=[2,1,-1]→0 ✓"

# One pass: left_sum==total-left_sum-nums[i]. Time O(n). Space O(1).
# Follow-up: Find Middle Index in Array (LC 1991) — identical problem, different name.
# Follow-up: if multiple pivots exist — return all of them.
```

Prefix Sum

#1480 Running Sum of 1d Array

EAS

Open on LeetCode

Array · Prefix Sum

Lists: AlgoMaster 600

Problem

Given an array nums. We define a running sum of an array as runningSum[i] = sum(nums[0]…nums[i]).
Return the running sum of nums.

Example 1:

```
Input: nums = [1,2,3,4]
Output: [1,3,6,10]
Explanation: Running sum is obtained as follows: [1, 1+2, 1+2+3, 1+2+3+4].
```

Example 2:

```
Input: nums = [1,1,1,1,1]
Output: [1,2,3,4,5]
Explanation: Running sum is obtained as follows: [1, 1+1, 1+1+1, 1+1+1+1, 1+1+1+1+1].
```

Example 3:

```
Input: nums = [3,1,2,10,1]
Output: [3,4,6,16,17]
```

Constraints:

- 1 <= nums.length <= 1000
- -10^6 <= nums[i] <= 10^6

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return running sum where runningSum[i] = sum(nums[0..i]). Right?"
# "nums=[1,2,3,4]→[1,3,6,10] ✓"
# In-place prefix sum. Time O(n). Space O(1).
# Follow-up: if nums is a stream — maintain a running total variable.
# Follow-up: Find Pivot Index (LC 724) — uses running sum concept.
```


Prefix Sum

#1854 Maximum Population Year

EAS

Open on LeetCode

Array · Counting · Prefix Sum

Lists: AlgoMaster 600

Problem

You are given a 2D integer array logs where each logs[i] = [birthi, deathi] indicates the birth and death years of the ith person.
The population of some year x is the number of people alive during that year. The ith person is counted in year x's population if x is in the inclusive range [birthi, deathi - 1]. Note that the person is not counted in the year that they die.
Return the earliest year with the maximum population.

Example 1:

```
Input: logs = [[1993,1999],[2000,2010]]
Output: 1993
Explanation: The maximum population is 1, and 1993 is the earliest year with this population.
```

Example 2:

```
Input: logs = [[1950,1961],[1960,1971],[1970,1981]]
Output: 1960
Explanation:
The maximum population is 2, and it had happened in years 1960 and 1970.
The earlier year between them is 1960.
```

Constraints:

- 1 <= logs.length <= 100
- 1950 <= birthi < deathi <= 2050

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "logs[i]=[birth,death]. Find year with most people alive (alive in [birth,death-1]).
# If tie, return earliest year. Right?"
# "logs=[[1993,1999],[2000,2010]]→1993 ✓"
# Difference array: +1 at birth, -1 at death. Prefix sum gives population. O(n+100)=O(n).
# Follow-up: if years span wider range — use a hashmap instead of fixed array.
# Follow-up: also return number of people alive in that year.
```

Monotonic Stack

Round 4 · Mid · Easy · 2 questions

Monotonic Stack

#496 Next Greater Element I EAS

Open on LeetCode

Array · Hash Table · Stack · Monotonic Stack

Lists: AlgoMaster 600

Problem

The next greater element of some element x in an array is the first greater element that is to the right of x in the same array.
You are given two distinct 0-indexed integer arrays nums1 and nums2, where nums1 is a subset of nums2.
For each 0 <= i < nums1.length, find the index j such that nums1[i] == nums2[j] and determine the next greater element of nums2[j] in nums2. If there is no next greater element, then the answer for this query is -1.
Return an array ans of length nums1.length such that ans[i] is the next greater element as described above.

Example 1:

```
Input: nums1 = [4,1,2], nums2 = [1,3,4,2]
Output: [-1,3,-1]
Explanation: The next greater element for each value of nums1 is as follows:
- 4 is underlined in nums2 = [1,3,4,2]. There is no next greater element, so the answer is -1.
- 1 is underlined in nums2 = [1,3,4,2]. The next greater element is 3.
- 2 is underlined in nums2 = [1,3,4,2]. There is no next greater element, so the answer is -1.
```

Example 2:

```
Input: nums1 = [2,4], nums2 = [1,2,3,4]
Output: [3,-1]
Explanation: The next greater element for each value of nums1 is as follows:
- 2 is underlined in nums2 = [1,2,3,4]. The next greater element is 3.
- 4 is underlined in nums2 = [1,2,3,4]. There is no next greater element, so the answer is -1.
```

Constraints:

- **1 <= nums1.length <= nums2.length <= 1000**
- **0 <= nums1[i], nums2[i] <= 104**
- **All integers in nums1 and nums2 are unique.**
- **All the integers of nums1 also appear in nums2.**

Follow up: Could you find an O(nums1.length + nums2.length) solution?

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "nums1 is a subset of nums2. For each x in nums1, find the next greater element in nums2. -1 if none. Right?"
# "nums1=[4,1,2],nums2=[1,3,4,2]: next_greater={1:3,3:-1,4:-1,2:-1}→[-1,3,-1] ✓"
# Monotonic decreasing stack on nums2. O(n+m). Space O(n).
# Follow-up: Next Greater Element II (LC 503) — circular array, iterate twice.
# Follow-up: Daily Temperatures (LC 739) — return distance to next greater, not value.
```

Monotonic Stack

#1475 Final Prices With a Special Discount in a Shop

EAS

Open on LeetCode

Array · Stack · Monotonic Stack

Lists: AlgoMaster 600

Problem

You are given an integer array prices where prices[i] is the price of the ith item in a shop.
There is a special discount for items in the shop. If you buy the ith item, then you will receive a discount equivalent to prices[j] where j is the minimum index such that j > i and prices[j] <= prices[i]. Otherwise, you will not receive any discount at all.
Return an integer array answer where answer[i] is the final price you will pay for the ith item of the shop, considering the special discount.

Example 1:

```
Input: prices = [8,4,6,2,3]
Output: [4,2,4,2,3]
Explanation:
For item 0 with price[0]=8 you will receive a discount equivalent to prices[1]=4, therefore, the final price you will pay is 8 - 4 = 4.
For item 1 with price[1]=4 you will receive a discount equivalent to prices[3]=2, therefore, the final price you will pay is 4 - 2 = 2.
For item 2 with price[2]=6 you will receive a discount equivalent to prices[3]=2, therefore, the final price you will pay is 6 - 2 = 4.
For items 3 and 4 you will not receive any discount at all.
```

Example 2:

```
Input: prices = [1,2,3,4,5]
Output: [1,2,3,4,5]
Explanation: In this case, for all items, you will not receive any discount at all.
```

Example 3:

```
Input: prices = [10,1,1,6]
Output: [9,0,1,6]
```

Constraints:

- **1 <= prices.length <= 500**
- **1 <= prices[i] <= 1000**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "For each item, find the first item with price <= prices[i] that comes after it.
# Discount = that price. Return final prices. Right?"
# "prices=[8,4,6,2,3]→[4,2,4,2,3] ✓"
# Monotonic non-decreasing stack. When smaller price found, pop and apply discount. O(n). Space O(n).
# Follow-up: if discount applies to all subsequent cheaper items — sum them.
# Follow-up: Next Smaller Element — identical structure with < instead of <=.
```


Queues

Round 4 · Mid · Easy · 2 questions

Queues

#933 Number of Recent Calls

EAS

Open on LeetCode

Design · Queue · Data Stream

Lists: AlgoMaster 600

Problem

You have a RecentCounter class which counts the number of recent requests within a certain time frame.

Implement the RecentCounter class:

- RecentCounter() Initializes the counter with zero recent requests.
- int ping(int t) Adds a new request at time t, where t represents some time in milliseconds, and returns the number of requests that has happened in the past 3000 milliseconds (including the new request). Specifically, return the number of requests that have happened in the inclusive range [t - 3000, t].

It is guaranteed that every call to ping uses a strictly larger value of t than the previous call.

Example 1:

```
Input
["RecentCounter", "ping", "ping", "ping", "ping"]
[[], [1], [100], [3001], [3002]]
Output
[null, 1, 2, 3, 3]

Explanation
RecentCounter recentCounter = new RecentCounter();
```

Constraints:

- 1 <= t <= 109
- Each test case will call ping with strictly increasing values of t.
- At most 104 calls will be made to ping.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "ping(t) returns count of calls in [t-3000,t]. Calls always increasing. Right?"
# "ping(1)→1,ping(100)→2,ping(3001)→3,ping(3002)→3 ✓"
# Sliding window queue. Amortized O(1) per ping. Space O(3000)=O(1).
# Follow-up: if window size changes dynamically — parameterize the 3000 threshold.
# Follow-up: rate limiter — same idea, return True/False instead of count.
```


Queues

#2073 Time Needed to Buy Tickets

EAS

Open on LeetCode

Array · Queue · Simulation

Lists: AlgoMaster 600

Problem

There are n people in a line queuing to buy tickets, where the 0th person is at the front of the line and the (n - 1)th person is at the back of the line.

You are given a 0-indexed integer array tickets of length n where the number of tickets that the ith person would like to buy is tickets[i].

Each person takes exactly 1 second to buy a ticket. A person can only buy 1 ticket at a time and has to go back to the end of the line (which happens instantaneously) in order to buy more tickets. If a person does not have any tickets left to buy, the person will leave the line.

Return the time taken for the person initially at position k (0-indexed) to finish buying tickets.

Example 1:

Input: tickets = [2,3,2], k = 2

Output: 6

Explanation:

- The queue starts as [2,3,2], where the kth person is underlined.
- After the person at the front has bought a ticket, the queue becomes [3,2,1] at 1 second.
- Continuing this process, the queue becomes [2,1,2] at 2 seconds.
- Continuing this process, the queue becomes [1,2,1] at 3 seconds.
- Continuing this process, the queue becomes [2,1] at 4 seconds. Note: the person at the front left the queue.
- Continuing this process, the queue becomes [1,1] at 5 seconds.
- Continuing this process, the queue becomes [1] at 6 seconds. The kth person has bought all their tickets, so return 6.

Example 2:

Input: tickets = [5,1,1,1], k = 0

Output: 8

Explanation:

- The queue starts as [5,1,1,1], where the kth person is underlined.
- After the person at the front has bought a ticket, the queue becomes [1,1,1,4] at 1 second.
- Continuing this process for 3 seconds, the queue becomes [4] at 4 seconds.
- Continuing this process for 4 seconds, the queue becomes [] at 8 seconds. The kth person has bought all their tickets, so return 8.

Constraints:

- n == tickets.length
- 1 <= n <= 100
- 1 <= tickets[i] <= 100
- 0 <= k < n

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Queue of people buying tickets. tickets[i]=how many person i wants. Each round, one ticket per person.
# Find time for person k to finish. Right?"
# "tickets=[2,3,2],k=2→6 ✓"
# Math: people before k complete min(t,tickets[k]) rounds. People after: min(t,tickets[k]-1).
# Time O(n). Space O(1).
# Follow-up: if multiple people at position k — handle ties with position ordering.
```

Divide and Conquer

Round 4 · Mid · Easy · 1 question

Divide and Conquer

#1763 Longest Nice Substring

EAS

Open on LeetCode

Hash Table · String · Divide and Conquer · Bit Manipulation · Sliding Window

Lists: AlgoMaster 600

Problem

A string s is nice if, for every letter of the alphabet that s contains, it appears both in uppercase and lowercase. For example, "abABB" is nice because 'A' and 'a' appear, and 'B' and 'b' appear. However, "abA" is not because 'b' appears, but 'B' does not. Given a string s, return the longest substring of s that is nice. If there are multiple, return the substring of the earliest occurrence. If there are none, return an empty string.

Example 1:

```
Input: s = "YazaAay"
Output: "aAa"
Explanation: "aAa" is a nice string because 'A/a' is the only letter of the alphabet in s, and both 'A' and 'a' appear.
"aAa" is the longest nice substring.
```

Example 2:

```
Input: s = "Bb"
Output: "Bb"
Explanation: "Bb" is a nice string because both 'B' and 'b' appear. The whole string is a substring.
```

Example 3:

```
Input: s = "c"
Output: ""
Explanation: There are no nice substrings.
```

Constraints:

- **1 <= s.length <= 100**
- **s consists of uppercase and lowercase English letters.**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "A string is 'nice' if for every letter, both its uppercase and lowercase appear.
# Find the longest nice substring. Return '' if none. Right?"
# "s='YazaAay'→'aAa' ✓ s='Bb'→'Bb' ✓"

# Divide at any char missing its case counterpart. O(n²) worst case. Space O(n).
# Follow-up: count all nice substrings — sliding window with bit manipulation.
# Follow-up: if only lowercase letters — use two-pointer approach.
```

BST / Ordered Set

Round 4 · Mid · Easy · 3 questions

BST / Ordered Set

#653 Two Sum IV - Input is a BST

EAS

Open on LeetCode

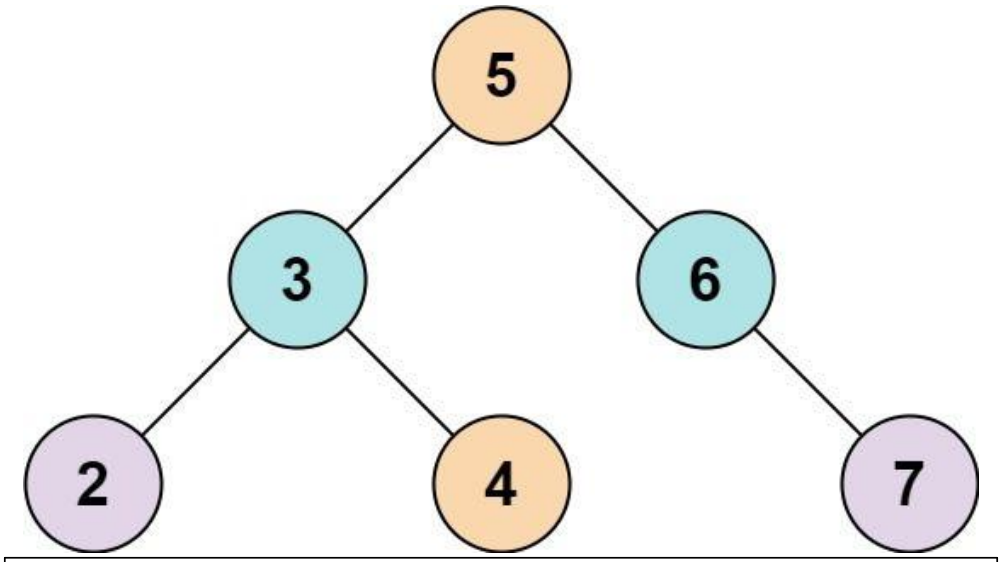
Hash Table · Two Pointers · Tree · Depth-First Search · Breadth-First Search · Binary Search Tree · Binary Tree

Lists: AlgoMaster 600

Problem

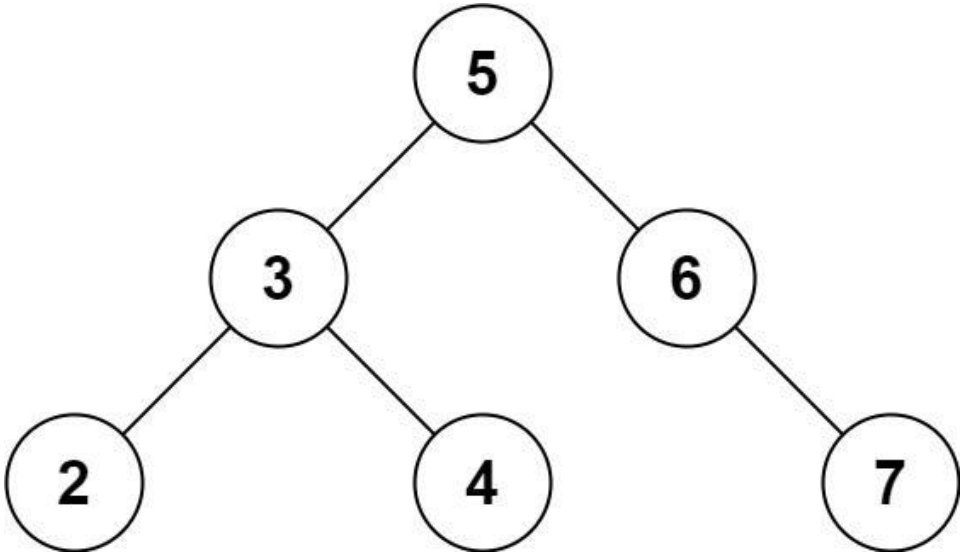
Given the root of a binary search tree and an integer k, return true if there exist two elements in the BST such that their sum is equal to k, or false otherwise.

Example 1:




```
Input: root = [5,3,6,2,4,null,7], k = 9
Output: true
```

Example 2:




```
Input: root = [5,3,6,2,4,null,7], k = 28
Output: false
```

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- -104 <= Node.val <= 104
- root is guaranteed to be a valid binary search tree.
- -105 <= k <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find if any two nodes in BST sum to k. Return true/false. Right?"
# "root=[5,3,6,2,4,null,7],k=9→true (2+7) ✓ k=28→false ✓"
# DFS + hash set: O(n) time, O(n) space. Works for any tree, not just BST.
# BST-specific: use two iterators (ascending + descending) with two-pointer. O(n) time O(h) space.
# Follow-up: return the actual pair — track values in set.
```


BST / Ordered Set

#700 Search in a Binary Search Tree EAS

Open on LeetCode

Tree · Binary Search Tree · Binary Tree

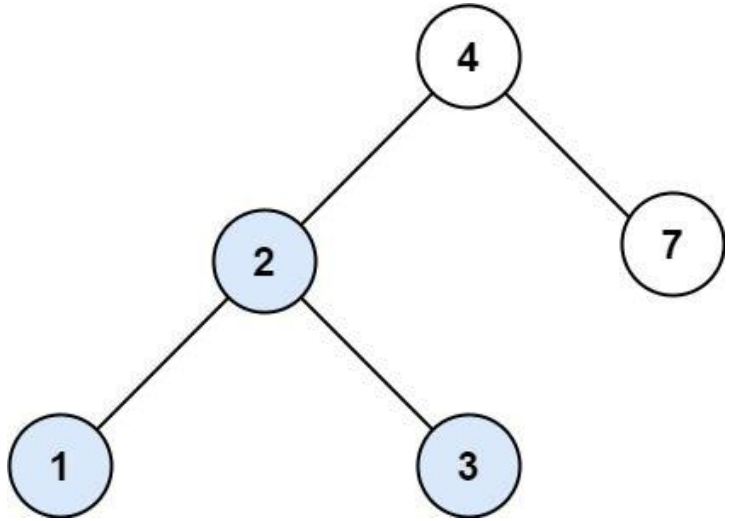
Lists: AlgoMaster 600

Problem

You are given the root of a binary search tree (BST) and an integer val.

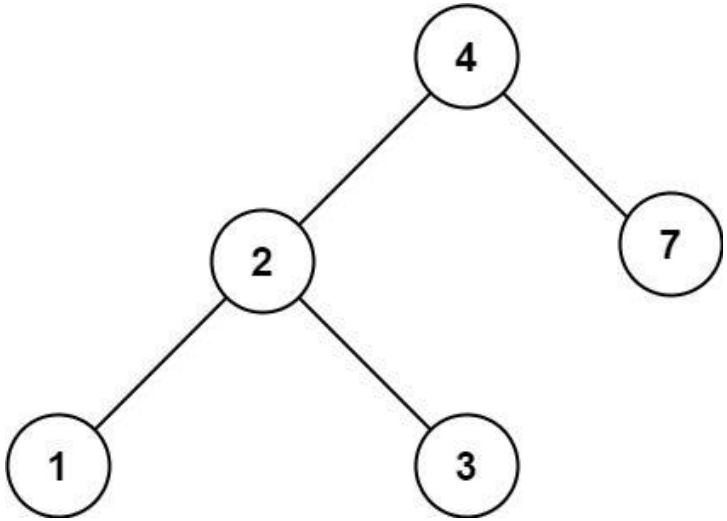
Find the node in the BST that the node's value equals val and return the subtree rooted with that node. If such a node does not exist, return null.

Example 1:




```
Input: root = [4,2,7,1,3], val = 2
Output: [2,1,3]
```

Example 2:




```
Input: root = [4,2,7,1,3], val = 5
Output: []
```

Constraints:

- The number of nodes in the tree is in the range [1, 5000].
- 1 <= Node.val <= 107
- root is a binary search tree.
- 1 <= val <= 107

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Find the node with value val in BST. Return that subtree, or null if not found. Right?"
# "root=[4,2,7,1,3],val=2→[2,1,3] ✓"
# Iterative BST search: O(h) time, O(1) space. Recursive: O(h) time, O(h) space.
# Follow-up: Insert into a BST (LC 701) — search path then attach new node.
# Follow-up: Delete Node in a BST (LC 450) — find + replace with inorder successor.
```


BST / Ordered Set

#938 Range Sum of BST EAS

Open on LeetCode

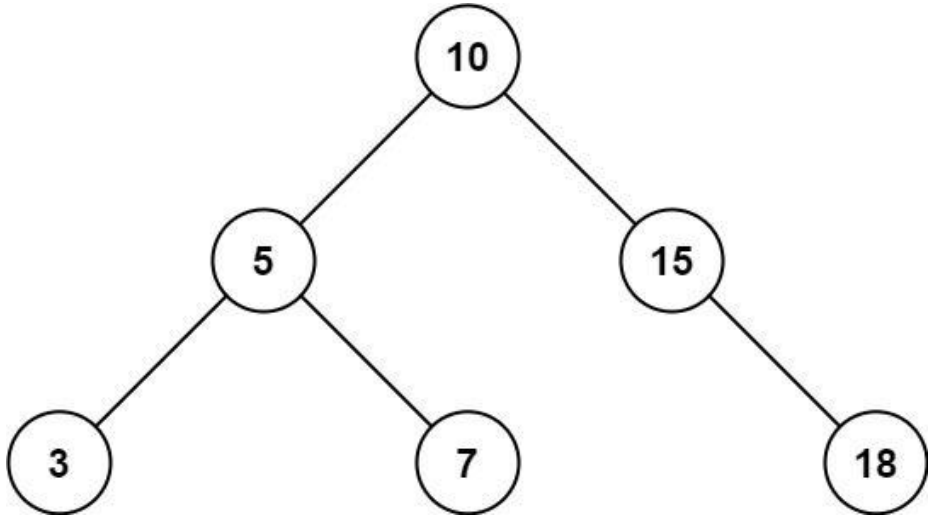
Tree · Depth-First Search · Binary Search Tree · Binary Tree

Lists: AlgoMaster 600

Problem

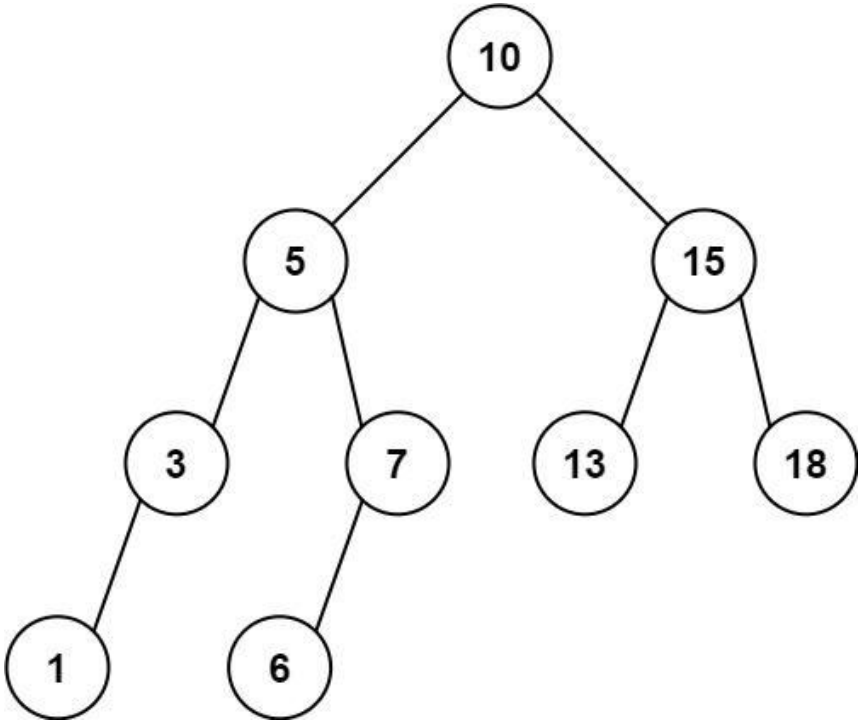
Given the root node of a binary search tree and two integers low and high, return the sum of values of all nodes with a value in the inclusive range [low, high].

Example 1:




```
Input: root = [10,5,15,3,7,null,18], low = 7, high = 15
Output: 32
Explanation: Nodes 7, 10, and 15 are in the range [7, 15]. 7 + 10 + 15 = 32.
```

Example 2:




```
Input: root = [10,5,15,3,7,13,18,1,null,6], low = 6, high = 10
Output: 23
Explanation: Nodes 6, 7, and 10 are in the range [6, 10]. 6 + 7 + 10 = 23.
```

Constraints:

- The number of nodes in the tree is in the range [1, 2 * 104].
- 1 <= Node.val <= 105
- 1 <= low <= high <= 105
- All Node.val are unique.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Return sum of all node values in BST in range [low,high]. Right?"
# "root=[10,5,15,3,7,null,18],low=7,high=15→32(7+10+15) ✓"

# Pruned DFS: skip left subtree if root.val<low, skip right if root.val>high. O(n) worst, better avg.
# Follow-up: count nodes in range — same pruning, count instead of sum.
# Follow-up: if tree is not a BST — collect all nodes, filter by range O(n).
```


Intervals

Round 4 · Mid · Easy · 1 question

Intervals

#228 Summary Ranges EAS

Open on LeetCode

Array

Lists: AlgoMaster 600

Problem

You are given a sorted unique integer array nums.

A range [a,b] is the set of all integers from a to b (inclusive).

Return the smallest sorted list of ranges that cover all the numbers in the array exactly. That is, each element of nums is covered by exactly one of the ranges, and there is no integer x such that x is in one of the ranges but not in nums.

Each range [a,b] in the list should be output as:

- "a->b" if a != b
- "a" if a == b

Example 1:

```
Input: nums = [0,1,2,4,5,7]
Output: ["0->2","4->5","7"]
Explanation: The ranges are:
[0,2] --> "0->2"
[4,5] --> "4->5"
[7,7] --> "7"
```

Example 2:

```
Input: nums = [0,2,3,4,6,8,9]
Output: ["0","2->4","6","8->9"]
Explanation: The ranges are:
[0,0] --> "0"
[2,4] --> "2->4"
[6,6] --> "6"
[8,9] --> "8->9"
```

Constraints:

- 0 <= nums.length <= 20
- -231 <= nums[i] <= 231 - 1
- All the values of nums are unique.
- nums is sorted in ascending order.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given sorted unique integers, return the smallest sorted list of ranges covering all numbers. Right?"
# "nums=[0,1,2,4,5,7]→['0->2','4->5','7'] ✓ nums=[0,2,3,4,6,8,9]→['0','2->4','6','8->9'] ✓"
# Two-pointer scan for consecutive runs. O(n). Space O(1) extra.
# Follow-up: Merge Intervals (LC 56) — merge overlapping intervals with endpoints.
# Follow-up: if nums has duplicates — add dedup step first.
```


Data Structure Design

Round 4 · Mid · Easy · 2 questions

Data Structure Design

#225 Implement Stack using Queues EAS

Open on LeetCode

Stack · Design · Queue

Lists: AlgoMaster 600

Problem

Implement a last-in-first-out (LIFO) stack using only two queues. The implemented stack should support all the functions of a normal stack (push, top, pop, and empty).

Implement the MyStack class:

- void push(int x) Pushes element x to the top of the stack.
- int pop() Removes the element on the top of the stack and returns it.
- int top() Returns the element on the top of the stack.
- boolean empty() Returns true if the stack is empty, false otherwise.

Notes:

- You must use only standard operations of a queue, which means that only push to back, peek/pop from front, size and is empty operations are valid.
- Depending on your language, the queue may not be supported natively. You may simulate a queue using a list or deque (double-ended queue) as long as you use only a queue's standard operations.

Example 1:

```
Input
["MyStack", "push", "push", "top", "pop", "empty"]
[[], [1], [2], [], [], []]
Output
[null, null, null, 2, 2, false]

Explanation
MyStack myStack = new MyStack();
```

Constraints:

- 1 <= x <= 9
- At most 100 calls will be made to push, pop, top, and empty.
- All the calls to pop and top are valid.

Follow-up: Can you implement the stack using only one queue?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Implement LIFO stack using only queues. Support push, pop, top, empty. Right?"
# "push(1),push(2),top()→2,pop()→2,empty()→false ✓"
# On push: rotate all elements so new element is at front. O(n) push, O(1) pop/top.
# Follow-up: Implement Queue Using Stacks (LC 232) — symmetric problem, two stacks.
# Follow-up: O(1) push, O(n) pop variant — don't rotate on push, rotate on pop instead.
```


Data Structure Design

#359 Logger Rate Limiter EAS

Open on LeetCode

Hash Table · Design · Data Stream

Lists: AlgoMaster 600

Problem

Design a logger system that receives a stream of messages along with their timestamps. Each unique message should only be printed at most every 10 seconds (i.e. a message printed at timestamp t will prevent other identical messages from being printed until timestamp t + 10).

All messages will come in chronological order. Several messages may arrive at the same timestamp.

Implement the Logger class:

- Logger() Initializes the logger object.
- bool shouldPrintMessage(int timestamp, string message) Returns true if the message should be printed in the given timestamp, otherwise returns false.

Example 1:

```
Input
["Logger", "shouldPrintMessage", "shouldPrintMessage", "shouldPrintMessage", "shouldPrintMessage", "shouldPrintMessage",
"shouldPrintMessage"]
[[], [1, "foo"], [2, "bar"], [3, "foo"], [8, "bar"], [10, "foo"], [11, "foo"]]
Output
[null, true, true, false, false, false, true]

Explanation
Logger logger = new Logger();
```

Constraints:

- 0 <= timestamp <= 109
- Every timestamp will be passed in non-decreasing order (chronological order).
- 1 <= message.length <= 30
- At most 104 calls will be made to shouldPrintMessage.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Log system: print(timestamp,message) returns true only if same message not printed in last 10s. Right?"
# "print(1,'foo')→T,print(2,'bar')→T,print(3,'foo')→F,print(11,'foo')→T ✓"
# Hash map: message→last print time. O(1) per call. Space O(unique messages).
# Follow-up: if memory is limited — sliding window with bounded queue per message.
# Follow-up: Number of Recent Calls (LC 933) — count calls in window instead of rate-limit.
```

Greedy

Round 4 · Mid · Easy · 2 questions

Greedy

#455 Assign Cookies EAS

Open on LeetCode

Array · Two Pointers · Greedy · Sorting

Lists: AlgoMaster 600

Problem

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:

```
Input: g = [1,2,3], s = [1,1]
Output: 1
Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.
And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.
You need to output 1.
```

Example 2:

```
Input: g = [1,2], s = [1,2,3]
Output: 2
Explanation: You have 2 children and 3 cookies. The greed factors of 2 children are 1, 2.
You have 3 cookies and their sizes are big enough to gratify all of the children,
You need to output 2.
```

Constraints:

- 1 <= g.length <= 3 * 10^4
- 0 <= s.length <= 3 * 10^4
- 1 <= g[i], s[j] <= 2^31 - 1

Note: This question is the same as 2410: Maximum Matching of Players With Trainers.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "g[i]=child greed factor, s[j]=cookie size. Assign at most one cookie per child.
# A child is content if s[j]>=g[i]. Maximize content children. Right?"
# "g=[1,2,3],s=[1,1]→1 ✓ g=[1,2],s=[1,2,3]→2 ✓"

# Greedy: match smallest sufficient cookie to least greedy child. O(n log n). Space O(1).
# Follow-up: if each child can receive multiple cookies — knapsack problem.
# Follow-up: minimize cookies wasted — match greedily from largest.
```

#3074 Apple Redistribution into Boxes

EAS

Open on LeetCode

Array · Greedy · Sorting

Lists: AlgoMaster 600

Problem

You are given an array apple of size n and an array capacity of size m.

There are n packs where the ith pack contains apple[i] apples. There are m boxes as well, and the ith box has a capacity of capacity[i] apples.

Return the minimum number of boxes you need to select to redistribute these n packs of apples into boxes.

Note that, apples from the same pack can be distributed into different boxes.

Example 1:

```
Input: apple = [1,3,2], capacity = [4,3,1,5,2]
Output: 2
Explanation: We will use boxes with capacities 4 and 5.
It is possible to distribute the apples as the total capacity is greater than or equal to the total number of apples.
```

Example 2:

```
Input: apple = [5,5,5], capacity = [2,4,2,7]
Output: 4
Explanation: We will need to use all the boxes.
```

Constraints:

- 1 <= n == apple.length <= 50
- 1 <= m == capacity.length <= 50
- 1 <= apple[i], capacity[i] <= 50
- The input is generated such that it's possible to redistribute packs of apples into boxes.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "apple[i]=apples in pack i. capacity[j]=capacity of box j. Pack apples into fewest boxes.
# Return minimum boxes needed. Right?"
# "apple=[1,3,2],capacity=[4,3,1,5,2]→2 (5+4 can hold 6 total) ✓"

# Greedy: sort boxes descending, pick greedily. O(n log n). Space O(1).
# Follow-up: if boxes have costs — minimum cost instead of minimum count (weighted greedy/DP).
# Follow-up: if each box can only hold one type of apple — matching problem.
```


Round 5

Mid Priority · Medium

94 questions · 23 patterns

- ☐ Bit Manipulation #137 #201 #260 ↗
- ☐ Prefix Sum #304 #370 #523 #974 #1314 #2536 ↗
- ☐ Kadane's Algorithm #918 #978 #1014 #1186 #1749 ↗
- ☐ Matrix (2D Array) #289 ↗
- ☐ LinkedList In-place Reversal #92 ↗
- ☐ Monotonic Stack #402 #456 #503 #581 #769 #901 #907 #1019 #2104 ↗
- ☐ Queues #950 #1823 #2327 ↗
- ☐ Monotonic Queue #1438 #1673 #1696 #2762 #3578 ↗
- ☐ Divide and Conquer #109 #427 #654 ↗
- ☐ Merge Sort #912 ↗
- ☐ QuickSort / QuickSelect #1985 ↗
- ☐ Backtracking #47 #77 #93 #216 ↗
- ☐ BST / Ordered Set #99 #426 #450 #510 #669 #701 #729 #731 #1008 #2034 ↗
- ☐ Tries #421 #648 #1233 #1268 #2452 #3043 ↗
- ☐ Heaps #1405 #1642 #1834 #1882 ↗
- ☐ Intervals #452 #1094 #1229 #1288 #1353 #2054 ↗
- ☐ K-Way Merge #373 #378 ↗
- ☐ Data Structure Design #641 #1146 #1472 #2353 ↗
- ☐ Greedy #406 #670 #826 #921 #1488 #1717 #1871 #1975 ↗
- ☐ Topological Sort #802 #1462 #2115 ↗
- ☐ Union Find #399 #547 #947 ↗
- ☐ Minimum Spanning Tree #1135 ↗
- ☐ Shortest Path #1514 #1631 #2976 #3341 #3650 ↗

Bit Manipulation

Round 5 · Mid · Medium · 3 questions

Bit Manipulation

#137 Single Number II

ME D

Open on LeetCode

Array · Bit Manipulation

Lists: AlgoMaster 600

Problem

Given an integer array nums where every element appears three times except for one, which appears exactly once. Find the single element and return it.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Example 1:

```
Input: nums = [2,2,3,2]
Output: 3
```

Example 2:

```
Input: nums = [0,1,0,1,0,1,99]
Output: 99
```

Constraints:

- **1 <= nums.length <= 3 * 104**
- **-231 <= nums[i] <= 231 - 1**
- **Each element in nums appears exactly three times except for one element which appears once.**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Every element appears 3 times except one which appears once. Find it. O(1) space. Right?"
# "nums=[2,2,3,2]→3 ✓ nums=[0,1,0,1,0,1,99]→99 ✓"
# Bit state machine: ones/twos track bits appearing 1/2 mod 3 times. When a bit appears 3× it resets to 0.
# Time O(n). Space O(1).
# Follow-up: element appears k times except one (appears m times) — generalize with mod-k counting per bit.
```

Bit Manipulation

#201 Bitwise AND of Numbers Range

MED

Open on LeetCode

Bit Manipulation

Lists: AlgoMaster 600

Problem

Given two integers left and right that represent the range [left, right], return the bitwise AND of all numbers in this range, inclusive.

Example 1:

```
Input: left = 5, right = 7
Output: 4
```

Example 2:

```
Input: left = 0, right = 0
Output: 0
```

Example 3:

```
Input: left = 1, right = 2147483647
Output: 0
```

Constraints:

- 0 <= left <= right <= 231 - 1

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return bitwise AND of all integers in [left,right]. Right?"
# "left=5,right=7: 5&6&7=101&110&111=100=4 ✓ left=0,right=0→0 ✓"

# Find common prefix of binary representations. When range spans a power of 2, that bit becomes 0.
# Time O(log n). Space O(1).
# Follow-up: Bitwise OR of range — left | right is sufficient (OR only grows).
# Follow-up: XOR of range 1..n — O(1) pattern: [n,1,n+1,0] cycling by n%4.
```


Bit Manipulation

#260 Single Number III

MED

Open on LeetCode

Array · Bit Manipulation

Lists: AlgoMaster 600

Problem

Given an integer array nums, in which exactly two elements appear only once and all the other elements appear exactly twice. Find the two elements that appear only once. You can return the answer in any order.

You must write an algorithm that runs in linear runtime complexity and uses only constant extra space.

Example 1:

```
Input: nums = [1,2,1,3,2,5]
Output: [3,5]
Explanation: [5, 3] is also a valid answer.
```

Example 2:

```
Input: nums = [-1,0]
Output: [-1,0]
```

Example 3:

```
Input: nums = [0,1]
Output: [1,0]
```

Constraints:

- 2 <= nums.length <= 3 * 104
- -231 <= nums[i] <= 231 - 1
- Each integer in nums will appear twice, only two integers will appear once.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Every element appears twice except two elements which appear once. Find both. O(1) space. Right?"
# "nums=[1,2,1,3,2,5]→[3,5] ✓"

# XOR all→a^b. Find differentiating bit. Split nums into two groups, XOR each→a and b.
# Time O(n). Space O(1).
# Follow-up: if three unique elements among triples — need bitwise counting approach (LC 137 generalized).
```

Prefix Sum

Round 5 · Mid · Medium · 6 questions

Prefix Sum

#304 Range Sum Query 2D - Immutable

ME D

Open on LeetCode

Array · Design · Matrix · Prefix Sum

Lists: AlgoMaster 600

Problem

Given a 2D matrix matrix, handle multiple queries of the following type:

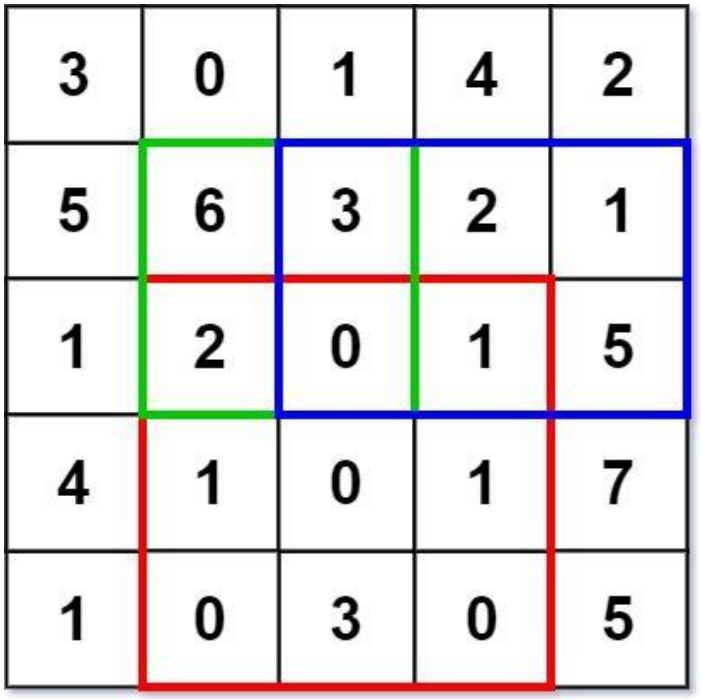
- Calculate the sum of the elements of matrix inside the rectangle defined by its upper left corner (row1, col1) and lower right corner (row2, col2).

Implement the NumMatrix class:

- NumMatrix(int[][] matrix) Initializes the object with the integer matrix matrix.
- int sumRegion(int row1, int col1, int row2, int col2) Returns the sum of the elements of matrix inside the rectangle defined by its upper left corner (row1, col1) and lower right corner (row2, col2).

You must design an algorithm where sumRegion works on O(1) time complexity.

Example 1:




```
Input
["NumMatrix", "sumRegion", "sumRegion", "sumRegion"]
[[[[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]]], [2, 1, 4, 3], [1, 1, 2, 2], [1, 2, 2, 4]]
Output
[null, 8, 11, 12]

Explanation
NumMatrix numMatrix = new NumMatrix([[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]]);
```

Constraints:

- m == matrix.length
- n == matrix[i].length

- 1 <= m, n <= 200
- -104 <= matrix[i][j] <= 104
- 0 <= row1 <= row2 < m
- 0 <= col1 <= col2 < n
- At most 104 calls will be made to sumRegion.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Preprocess matrix for repeated sumRegion(r1,c1,r2,c2) queries. Right?"
# "matrix=[[3,0,1,4,2],[5,6,3,2,1],[1,2,0,1,5],[4,1,0,1,7],[1,0,3,0,5]]:
# sumRegion(2,1,4,3)=8 ✓"
# 2D prefix sum: O(mn) preprocess, O(1) per query. Inclusion-exclusion formula.
# Follow-up: mutable matrix (LC 308) — 2D Binary Indexed Tree O(mn) build, O(log²(mn)) per op.
# Follow-up: count submatrices with sum=k — O(m²n) with row-pair + prefix sums.
```


Prefix Sum

#370 Range Addition

ME D

Open on LeetCode

Array · Prefix Sum

Lists: AlgoMaster 600

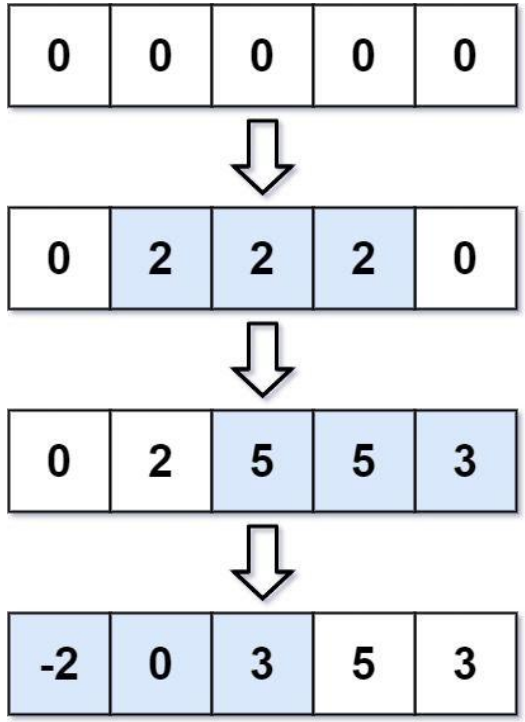
Problem

You are given an integer length and an array updates where updates[i] = [startIdxi, endIdxi, inci].

You have an array arr of length length with all zeros, and you have some operation to apply on arr. In the ith operation, you should increment all the elements arr[startIdxi], arr[startIdxi + 1], ..., arr[endIdxi] by inci.

Return arr after applying all the updates.

Example 1:




```
Input: length = 5, updates = [[1,3,2],[2,4,3],[0,2,-2]]
Output: [-2,0,3,5,3]
```

Example 2:

```
Input: length = 10, updates = [[2,4,6],[5,6,8],[1,9,-4]]
Output: [0,-4,2,2,2,4,4,-4,-4,-4]
```

Constraints:

- 1 <= length <= 105
- 0 <= updates.length <= 104
- 0 <= startIdxi <= endIdxi < length
- -1000 <= inci <= 1000

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Start with array of zeros. Apply updates [start,end,inc]: add inc to all elements in [start,end].
```

```
# Return the result. Right?"
# "n=5,updates=[[1,3,2],[2,4,3],[0,2,-2]]→[-2,0,3,5,3] ✓"
# Difference array: +inc at start, -inc at end+1. Prefix sum gives final values. O(n+k). Space O(n).
# Follow-up: 2D range additions — 2D difference array.
# Follow-up: Increment Submatrices by One (LC 2536) — 2D version of this.
```


Prefix Sum

#523 Continuous Subarray Sum

ME
D

Open on LeetCode

Array · Hash Table · Math · Prefix Sum

Lists: AlgoMaster 600

Problem

Given an integer array nums and an integer k, return true if nums has a good subarray or false otherwise.

A good subarray is a subarray where:

- **its length is at least two, and**
- **the sum of the elements of the subarray is a multiple of k.**

Note that:

- **A subarray is a contiguous part of the array.**
- **An integer x is a multiple of k if there exists an integer n such that x = n * k. 0 is always a multiple of k.**

Example 1:

```
Input: nums = [23,2,4,6,7], k = 6
Output: true
Explanation: [2, 4] is a continuous subarray of size 2 whose elements sum up to 6.
```

Example 2:

```
Input: nums = [23,2,6,4,7], k = 6
Output: true
Explanation: [23, 2, 6, 4, 7] is an continuous subarray of size 5 whose elements sum up to 42.
42 is a multiple of 6 because 42 = 7 * 6 and 7 is an integer.
```

Example 3:

```
Input: nums = [23,2,6,4,7], k = 13
Output: false
```

Constraints:

- **1 <= nums.length <= 105**
- **0 <= nums[i] <= 109**
- **0 <= sum(nums[i]) <= 231 - 1**
- **1 <= k <= 231 - 1**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return true if there's a subarray of length>=2 with sum divisible by k. Right?"
# "nums=[23,2,4,6,7],k=6→true (sum[1..2]=6) ✓ nums=[23,2,6,4,7],k=13→false ✓"
# Prefix sum mod k. If same remainder seen before with index at least 2 back → valid subarray.
# Time O(n). Space O(k).
# Follow-up: Subarray Sum Divisible by K (LC 974) — count all such subarrays.
# Follow-up: if k=0 — check for zero-sum subarrays instead (any prefix_sum seen twice).
```

Prefix Sum

#974 Subarray Sums Divisible by K

MED

Open on LeetCode

Array · Hash Table · Prefix Sum

Lists: AlgoMaster 600

Problem

Given an integer array nums and an integer k, return the number of non-empty subarrays that have a sum divisible by k.
A subarray is a contiguous part of an array.

Example 1:

```
Input: nums = [4,5,0,-2,-3,1], k = 5
Output: 7
Explanation: There are 7 subarrays with a sum divisible by k = 5:
[4, 5, 0, -2, -3, 1], [5], [5, 0], [5, 0, -2, -3], [0], [0, -2, -3], [-2, -3]
```

Example 2:

```
Input: nums = [5], k = 9
Output: 0
```

Constraints:

- 1 <= nums.length <= 3 * 10^4
- -10^4 <= nums[i] <= 10^4
- 2 <= k <= 10^4

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count subarrays with sum divisible by k. Right?"
# "nums=[4,5,0,-2,-3,1],k=5→7 ✓"

# For each prefix mod k, count previous equal-mod prefixes. C(freq,2) gives pairs.
# Time O(n). Space O(k).
# Follow-up: Continuous Subarray Sum (LC 523) — check existence with min-length constraint.
# Follow-up: if k can be negative — use abs(k) for modulo.
```


Prefix Sum

#1314 Matrix Block Sum

MED

Open on LeetCode

Array · Matrix · Prefix Sum

Lists: AlgoMaster 600

Problem

Given a m x n matrix mat and an integer k, return a matrix answer where each answer[i][j] is the sum of all elements mat[r][c] for:

- i - k <= r <= i + k,
- j - k <= c <= j + k, and
- (r, c) is a valid position in the matrix.

Example 1:

```
Input: mat = [[1,2,3],[4,5,6],[7,8,9]], k = 1
Output: [[12,21,16],[27,45,33],[24,39,28]]
```

Example 2:

```
Input: mat = [[1,2,3],[4,5,6],[7,8,9]], k = 2
Output: [[45,45,45],[45,45,45],[45,45,45]]
```

Constraints:

- m == mat.length
- n == mat[i].length
- 1 <= m, n, k <= 100
- 1 <= mat[i][j] <= 100

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "answer[i][j] = sum of all elements in mat where |r-i|<=k and |c-j|<=k. Right?"
# "mat=[[1,2,3],[4,5,6],[7,8,9]],k=1→[[12,21,16],[27,45,33],[24,39,28]] ✓"

# 2D prefix sum + range query. O(mn) preprocess, O(1) per query. Total O(mn).
# Follow-up: sliding window sum for k changing per cell — still O(mn) with 2D prefix.
# Follow-up: circular matrix block sum — handle wrap with modular prefix sums.
```

Prefix Sum

#2536 Increment Submatrices by One

MED

Open on LeetCode

Array · Matrix · Prefix Sum

Lists: AlgoMaster 600

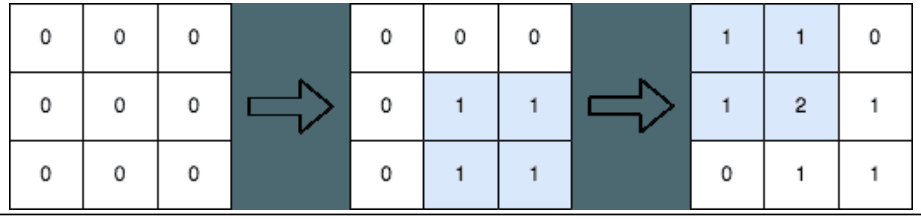
Problem

You are given a positive integer n, indicating that we initially have an n x n 0-indexed integer matrix mat filled with zeroes. You are also given a 2D integer array query. For each query[i] = [row1i, col1i, row2i, col2i], you should do the following operation:

- Add 1 to every element in the submatrix with the top left corner (row1i, col1i) and the bottom right corner (row2i, col2i). That is, add 1 to mat[x][y] for all row1i <= x <= row2i and col1i <= y <= col2i.

Return the matrix mat after performing every query.

Example 1:




```
Input: n = 3, queries = [[1,1,2,2],[0,0,1,1]]
Output: [[1,1,0],[1,2,1],[0,1,1]]
Explanation: The diagram above shows the initial matrix, the matrix after the first query, and the matrix after the second query.
- In the first query, we add 1 to every element in the submatrix with the top left corner (1, 1) and bottom right corner (2, 2).
- In the second query, we add 1 to every element in the submatrix with the top left corner (0, 0) and bottom right corner (1, 1).
```

Example 2:


```
Input: n = 2, queries = [[0,0,1,1]]
Output: [[1,1],[1,1]]
Explanation: The diagram above shows the initial matrix and the matrix after the first query.
- In the first query we add 1 to every element in the matrix.
```

Constraints:

- 1 <= n <= 500
- 1 <= queries.length <= 104
- 0 <= row1i <= row2i < n
- 0 <= col1i <= col2i < n

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "n×n matrix of zeros. queries[i]=[r1,c1,r2,c2]: increment each cell in that rectangle by 1.
# Return result matrix. Right?"
# "n=3,queries=[[1,1,2,2],[0,0,1,1]]→[[1,1,0],[1,2,1],[0,1,1]] ✓"

# 2D difference array: +1 at corner, propagate with 2D prefix sum. O(n²+q). Space O(n²).
# Follow-up: Range Addition (LC 370) — 1D version of this.
# Follow-up: if queries are circles (not rectangles) — need different approach (polar coordinates).
```


Kadane's Algorithm

Round 5 · Mid · Medium · 5 questions

Kadane's Algorithm

#918 Maximum Sum Circular Subarray

MED

Open on LeetCode

Array · Divide and Conquer · Dynamic Programming · Queue · Monotonic Queue
Lists: AlgoMaster 600

Problem

Given a circular integer array nums of length n, return the maximum possible sum of a non-empty subarray of nums.
A circular array means the end of the array connects to the beginning of the array. Formally, the next element of nums[i] is nums[(i + 1) % n] and the previous element of nums[i] is nums[(i - 1 + n) % n].
A subarray may only include each element of the fixed buffer nums at most once. Formally, for a subarray nums[i], nums[i + 1], ..., nums[j], there does not exist i <= k1, k2 <= j with k1 % n == k2 % n.

Example 1:

```
Input: nums = [1,-2,3,-2]
Output: 3
Explanation: Subarray [3] has maximum sum 3.
```

Example 2:

```
Input: nums = [5,-3,5]
Output: 10
Explanation: Subarray [5,5] has maximum sum 5 + 5 = 10.
```

Example 3:

```
Input: nums = [-3,-2,-3]
Output: -2
Explanation: Subarray [-2] has maximum sum -2.
```

Constraints:

- n == nums.length
- 1 <= n <= 3 * 104
- -3 * 104 <= nums[i] <= 3 * 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find max subarray sum in a circular array (can wrap around). Right?"
# "nums=[1,-2,3,-2]→3 ✓ nums=[5,-3,5]→10 ✓ nums=[-3,-2,-3]→-2 ✓"
        # Max normal subarray (Kadane)
        # Min subarray (to find wrap-around max = total - min)
# max(max_subarray, total - min_subarray). If all negative, max_subarray wins.
# Time O(n). Space O(1).
# Follow-up: return the actual circular subarray (start/end indices) — track positions in Kadane.
```


Kadane's Algorithm

#978 Longest Turbulent Subarray

MED

Open on LeetCode

Array · Dynamic Programming · Sliding Window
Lists: AlgoMaster 600

Problem

Given an integer array arr, return the length of a maximum size turbulent subarray of arr.
A subarray is turbulent if the comparison sign flips between each adjacent pair of elements in the subarray.
More formally, a subarray [arr[i], arr[i + 1], ..., arr[j]] of arr is said to be turbulent if and only if:

- For i <= k < j: arr[k] > arr[k + 1] when k is odd, and
- arr[k] < arr[k + 1] when k is even.
- arr[k] > arr[k + 1] when k is even, and
- arr[k] < arr[k + 1] when k is odd.

Example 1:

```
Input: arr = [9,4,2,10,7,8,8,1,9]
Output: 5
Explanation: arr[1] > arr[2] < arr[3] > arr[4] < arr[5]
```

Example 2:

```
Input: arr = [4,8,12,16]
Output: 2
```

Example 3:

```
Input: arr = [100]
Output: 1
```

Constraints:

- 1 <= arr.length <= 4 * 104
- 0 <= arr[i] <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "A subarray is turbulent if adjacent differences alternate between positive and negative.
# Return length of longest turbulent subarray. Right?"
# "arr=[9,4,2,10,7,8,8,1,9]→5 (4,2,10,7,8) ✓"

# Track ascending/descending streak lengths. Alternate on direction change. O(n). Space O(1).
# Follow-up: longest alternating subsequence (not subarray) — DP O(n).
# Follow-up: count turbulent subarrays — for each position i, add min(inc,dec)-1 to count.
```

Kadane's Algorithm

#1014 Best Sightseeing Pair

MED

Open on LeetCode

Array · Dynamic Programming

Lists: AlgoMaster 600

Problem

You are given an integer array values where values[i] represents the value of the ith sightseeing spot. Two sightseeing spots i and j have a distance j - i between them.

The score of a pair (i < j) of sightseeing spots is values[i] + values[j] + i - j: the sum of the values of the sightseeing spots, minus the distance between them.

Return the maximum score of a pair of sightseeing spots.

Example 1:

```
Input: values = [8,1,5,2,6]
Output: 11
Explanation: i = 0, j = 2, values[i] + values[j] + i - j = 8 + 5 + 0 - 2 = 11
```

Example 2:

```
Input: values = [1,2]
Output: 2
```

Constraints:

- 2 <= values.length <= 5 * 10^4
- 1 <= values[i] <= 1000

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Score of pair (i,j) = values[i]+values[j]+i-j (i<j). Return max score. Right?"
# "values=[8,1,5,2,6]→11 (i=0,j=2: 8+5+0-2=11) ✓"
# For each j, best score = max(values[i]+i) seen before + values[j]-j. O(n). Space O(1).
# Follow-up: if we need the actual pair indices — track argmax alongside max_i.
# Follow-up: Best Time to Buy/Sell (LC 121) — same pattern: track max before, compute diff.
```


Kadane's Algorithm

#1186 Maximum Subarray Sum with One Deletion

MED

Open on LeetCode

Array · Dynamic Programming

Lists: AlgoMaster 600

Problem

Given an array of integers, return the maximum sum for a non-empty subarray (contiguous elements) with at most one element deletion. In other words, you want to choose a subarray and optionally delete one element from it so that there is still at least one element left and the sum of the remaining elements is maximum possible.

Note that the subarray needs to be non-empty after deleting one element.

Example 1:

```
Input: arr = [1,-2,0,3]
Output: 4
Explanation: Because we can choose [1, -2, 0, 3] and drop -2, thus the subarray [1, 0, 3] becomes the maximum value.
```

Example 2:

```
Input: arr = [1,-2,-2,3]
Output: 3
Explanation: We just choose [3] and it's the maximum sum.
```

Example 3:

```
Input: arr = [-1,-1,-1,-1]
Output: -1
Explanation: The final subarray needs to be non-empty. You can't choose [-1] and delete -1 from it, then get an empty subarray
to make the sum equals to 0.
```

Constraints:

- 1 <= arr.length <= 10^5
- -10^4 <= arr[i] <= 10^4

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find max sum of non-empty subarray with at most one element deletion. Right?"
# "arr=[1,-2,0,3]→4 (delete -2) ✓ arr=[1,-2,-2,4]→4 ✓"
        # dp_no[i]=max subarray ending at i with no deletion
        # dp_del[i]=max subarray ending at i with one deletion
# Two Kadane states: with and without deletion. O(n). Space O(1).
# Follow-up: at most k deletions — extend DP to dp[k] states. O(nk).
# Follow-up: at most k elements allowed — sliding window with deque for min tracking.
```

Kadane's Algorithm

#1749 Maximum Absolute Sum of Any Subarray

MED

Open on LeetCode

Array · Dynamic Programming

Lists: AlgoMaster 600

Problem

You are given an integer array nums. The absolute sum of a subarray [numsl, numsl+1, ..., numsr-1, numsr] is abs(numsl + numsl+1 + ... + numsr-1 + numsr).

Return the maximum absolute sum of any (possibly empty) subarray of nums.

Note that abs(x) is defined as follows:

- If x is a negative integer, then abs(x) = -x.
- If x is a non-negative integer, then abs(x) = x.

Example 1:

```
Input: nums = [1,-3,2,3,-4]
Output: 5
Explanation: The subarray [2,3] has absolute sum = abs(2+3) = abs(5) = 5.
```

Example 2:

```
Input: nums = [2,-5,1,-4,3,-2]
Output: 8
Explanation: The subarray [-5,1,-4] has absolute sum = abs(-5+1-4) = abs(-8) = 8.
```

Constraints:

- 1 <= nums.length <= 105
- -104 <= nums[i] <= 104

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Find max absolute sum of any subarray (can be positive or negative subarray). Right?"
# "nums=[1,-3,2,3,-4]→5 (subarray [2,3] or [-3,2,3,-4] absolute sum=|-3+2+3-4|=|2| no, wait.
# [2,3]=5 ✓ or [-3]=3. answer=max(5,|-4|)=5 ✓"

# max(max_subarray_sum, |min_subarray_sum|). Two Kadane's. O(n). Space O(1).
# Follow-up: return the actual subarray — track start/end indices in both Kadane passes.
# Follow-up: if values are non-negative — answer is always total sum.
```


Matrix (2D Array)

Round 5 · Mid · Medium · 1 question

Matrix (2D Array)

#289 Game of Life

MED

Open on LeetCode

Array · Matrix · Simulation

Lists: AlgoMaster 600

Problem

According to Wikipedia's article: "The Game of Life, also known simply as Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970."

The board is made up of an m x n grid of cells, where each cell has an initial state: live (represented by a 1) or dead (represented by a 0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules (taken from the above Wikipedia article):

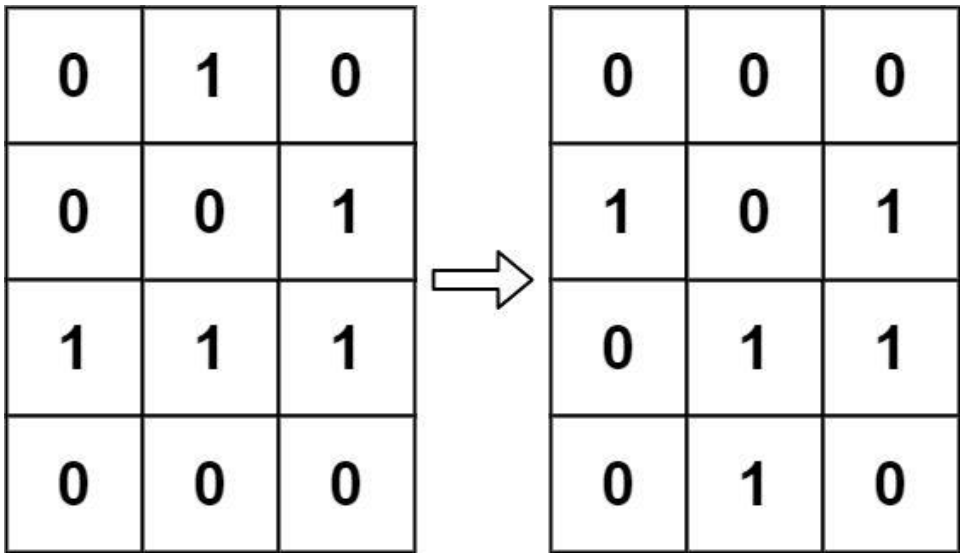
1. Any live cell with fewer than two live neighbors dies as if caused by under-population.
2. Any live cell with two or three live neighbors lives on to the next generation.
3. Any live cell with more than three live neighbors dies, as if by over-population.
4. Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction.

The next state of the board is determined by applying the above rules simultaneously to every cell in the current state of the m x n grid board. In this process, births and deaths occur simultaneously.

Given the current state of the board, update the board to reflect its next state.

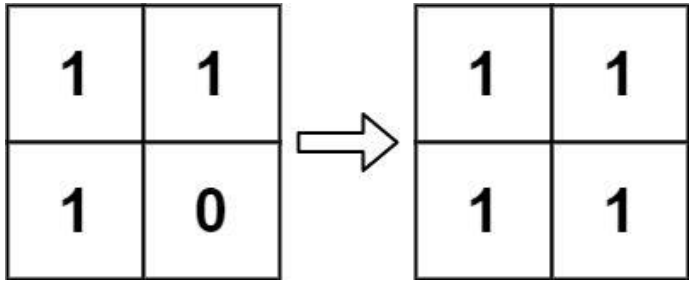
Note that you do not need to return anything.

Example 1:




```
Input: board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]
Output: [[0,0,0],[1,0,1],[0,1,1],[0,1,0]]
```

Example 2:




```
Input: board = [[1,1],[1,0]]
Output: [[1,1],[1,1]]
```

Constraints:

- m == board.length
- n == board[i].length
- 1 <= m, n <= 25
- board[i][j] is 0 or 1.

Follow up:

- Could you solve it in-place? Remember that the board needs to be updated simultaneously: You cannot update some cells first and then use their updated values to update other cells.
- In this question, we represent the board using a 2D array. In principle, the board is infinite, which would cause problems when the active area encroaches upon the border of the array (i.e., live cells reach the border). How would you address these problems?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Conway's Game of Life. Apply all rules simultaneously. Update board in-place. Right?"
# "board=[[0,1,0],[0,0,1],[1,1,1],[0,0,0]]→[[0,0,0],[1,0,1],[0,1,1],[0,1,0]] ✓"
        # Encode: 2=dead→alive, -1=alive→dead
# Encode state changes in-place using 2 and -1. Decode at end. O(mn). Space O(1).
# Follow-up: infinite board (LC variant) — use a set of live cells, iterate only around live cells.
# Follow-up: board wraps around (toroidal) — use modular arithmetic for neighbor indices.
```

LinkedList In-place Reversal

Round 5 · Mid · Medium · 1 question

LinkedList In-place Reversal

#92 Reverse Linked List II MED

Open on LeetCode

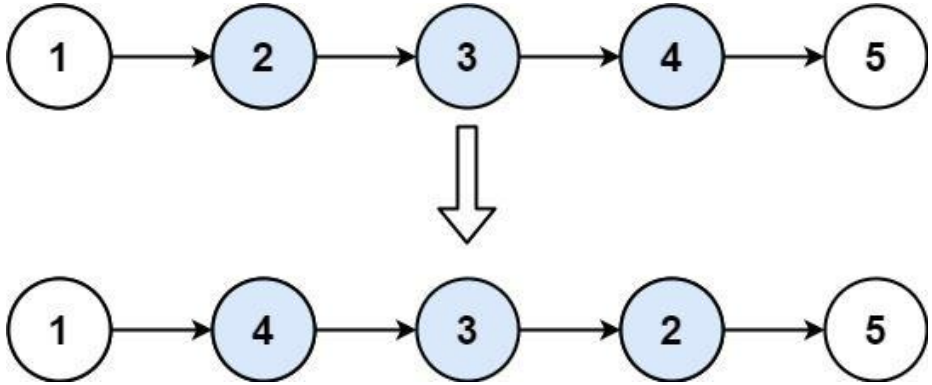
Linked List

Lists: AlgoMaster 600

Problem

Given the head of a singly linked list and two integers left and right where left <= right, reverse the nodes of the list from position left to position right, and return the reversed list.

Example 1:




```
Input: head = [1,2,3,4,5], left = 2, right = 4
Output: [1,4,3,2,5]
```

Example 2:

```
Input: head = [5], left = 1, right = 1
Output: [5]
```

Constraints:

- The number of nodes in the list is n.
- 1 <= n <= 500
- -500 <= Node.val <= 500
- 1 <= left <= right <= n

Follow up: Could you do it in one pass?

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Reverse nodes from position left to right (1-indexed) in one pass. Right?"
# "head=[1,2,3,4,5],left=2,right=4→[1,4,3,2,5] ✓"
# "Head insertion" technique: repeatedly insert nxt after prev. One pass. O(n). Space O(1).
# Follow-up: Reverse Nodes in k-Group (LC 25) — generalize to groups of k.
# Follow-up: if multiple ranges given — apply each reversal sequentially.
```

Monotonic Stack

Round 5 · Mid · Medium · 9 questions

Monotonic Stack

#402 Remove K Digits MED

Open on LeetCode

String · Stack · Greedy · Monotonic Stack

Lists: AlgoMaster 600

Problem

Given string num representing a non-negative integer num, and an integer k, return the smallest possible integer after removing k digits from num.

Example 1:

```
Input: num = "1432219", k = 3
Output: "1219"
Explanation: Remove the three digits 4, 3, and 2 to form the new number 1219 which is the smallest.
```

Example 2:

```
Input: num = "10200", k = 1
Output: "200"
Explanation: Remove the leading 1 and the number is 200. Note that the output must not contain leading zeroes.
```

Example 3:

```
Input: num = "10", k = 2
Output: "0"
Explanation: Remove all the digits from the number and it is left with nothing which is 0.
```

Constraints:

- 1 <= k <= num.length <= 105
- num consists of only digits.
- num does not have any leading zeros except for the zero itself.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Remove k digits from string num to make the smallest possible number. No leading zeros. Right?"
# "num='1432219',k=3→'1219' ✓ num='10200',k=1→'200' ✓ num='10',k=2→'0' ✓"
        # Remove remaining k from the end (they're the largest)
# Monotonic increasing stack. Pop larger digits greedily. O(n). Space O(n).
# Follow-up: add k digits to minimize — different greedy, harder problem.
# Follow-up: Largest Number After Digit Swaps by Parity (LC 2231) — swap-based approach.
```

Monotonic Stack

#456 132 Pattern

MED

Open on LeetCode

Array · Binary Search · Stack · Monotonic Stack · Ordered Set

Lists: AlgoMaster 600

Problem

Given an array of n integers nums, a 132 pattern is a subsequence of three integers nums[i], nums[j] and nums[k] such that i < j < k and nums[i] < nums[k] < nums[j].

Return true if there is a 132 pattern in nums, otherwise, return false.

Example 1:

```
Input: nums = [1,2,3,4]
Output: false
Explanation: There is no 132 pattern in the sequence.
```

Example 2:

```
Input: nums = [3,1,4,2]
Output: true
Explanation: There is a 132 pattern in the sequence: [1, 4, 2].
```

Example 3:

```
Input: nums = [-1,3,2,0]
Output: true
Explanation: There are three 132 patterns in the sequence: [-1, 3, 2], [-1, 3, 0] and [-1, 2, 0].
```

Constraints:

- n == nums.length
- 1 <= n <= 2 * 105
- -109 <= nums[i] <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return true if there exist i<j<k with nums[i]<nums[k]<nums[j] (132 pattern). Right?"
# "nums=[3,1,4,2]→true (1,4,2) ✓ nums=[1,2,3,4]→false ✓"
# Scan right-to-left. Stack tracks candidates for nums[j]. 'third' = nums[k] (largest seen smaller than j).
# Time O(n). Space O(n).
# Follow-up: count all 132 patterns — O(n log n) with sorted structure for counting.
# Follow-up: 123 pattern (strictly increasing triplet) — equivalent to Increasing Triplet Subsequence (LC 334).
```


Monotonic Stack

#503 Next Greater Element II

MED

Open on LeetCode

Array · Stack · Monotonic Stack

Lists: AlgoMaster 600

Problem

Given a circular integer array nums (i.e., the next element of nums[nums.length - 1] is nums[0]), return the next greater number for every element in nums.

The next greater number of a number x is the first greater number to its traversing-order next in the array, which means you could search circularly to find its next greater number. If it doesn't exist, return -1 for this number.

Example 1:

```
Input: nums = [1,2,1]
Output: [2,-1,2]
Explanation: The first 1's next greater number is 2;
The number 2 can't find next greater number.
The second 1's next greater number needs to search circularly, which is also 2.
```

Example 2:

```
Input: nums = [1,2,3,4,3]
Output: [2,3,4,-1,4]
```

Constraints:

- 1 <= nums.length <= 104
- -109 <= nums[i] <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Circular array. For each element find next greater element (wrapping around). -1 if none. Right?"
# "nums=[1,2,1]→[2,-1,2] ✓"
# Iterate twice (0..2n-1) to handle circularity. Stack stores indices. O(n). Space O(n).
# Follow-up: if we need the distance (not value) to next greater — track index differences.
# Follow-up: Previous Smaller Element — scan right-to-left with monotonic stack.
```

Monotonic Stack

#581 Shortest Unsorted Continuous Subarray

MED

Open on LeetCode

Array · Two Pointers · Stack · Greedy · Sorting · Monotonic Stack

Lists: AlgoMaster 600

Problem

Given an integer array nums, you need to find one continuous subarray such that if you only sort this subarray in non-decreasing order, then the whole array will be sorted in non-decreasing order.

Return the shortest such subarray and output its length.

Example 1:

```
Input: nums = [2,6,4,8,10,9,15]
Output: 5
Explanation: You need to sort [6, 4, 8, 10, 9] in ascending order to make the whole array sorted in ascending order.
```

Example 2:

```
Input: nums = [1,2,3,4]
Output: 0
```

Example 3:

```
Input: nums = [1]
Output: 0
```

Constraints:

- 1 <= nums.length <= 104
- -105 <= nums[i] <= 105

Follow up: Can you solve it in O(n) time complexity?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the shortest subarray that if sorted, the whole array becomes sorted. Right?"
# "nums=[2,6,4,8,10,9,15]→5 (indices 1–5) ✓ nums=[1,2,3,4]→0 ✓"

# Two passes: find rightmost element out of place from left, leftmost from right. O(n). Space O(1).
# Follow-up: return the actual indices (not length) — already computed as left and right.
# Follow-up: if the array is nearly sorted — same O(n) approach still optimal.
```


Monotonic Stack

#769 Max Chunks To Make Sorted

MED

Open on LeetCode

Array · Stack · Greedy · Sorting · Monotonic Stack

Lists: AlgoMaster 600

Problem

You are given an integer array arr of length n that represents a permutation of the integers in the range [0, n - 1].

We split arr into some number of chunks (i.e., partitions), and individually sort each chunk. After concatenating them, the result should equal the sorted array.

Return the largest number of chunks we can make to sort the array.

Example 1:

```
Input: arr = [4,3,2,1,0]
Output: 1
Explanation:
Splitting into two or more chunks will not return the required result.
For example, splitting into [4, 3], [2, 1, 0] will result in [3, 4, 0, 1, 2], which isn't sorted.
```

Example 2:

```
Input: arr = [1,0,2,3,4]
Output: 4
Explanation:
We can split into two chunks, such as [1, 0], [2, 3, 4].
However, splitting into [1, 0], [2], [3], [4] is the highest number of chunks possible.
```

Constraints:

- n == arr.length
- 1 <= n <= 10
- 0 <= arr[i] < n
- All the elements of arr are unique.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Split arr (permutation of 0..n-1) into chunks. Each chunk sorted = full array sorted. Max chunks? Right?"
# "arr=[4,3,2,1,0]→1 ✓ arr=[1,0,2,3,4]→4 ✓"

# A chunk ends at index i if max(arr[0..i])==i (all values fit in [0..i]). O(n). Space O(1).
# Follow-up: Max Chunks To Make Sorted II (LC 768) — arbitrary array, use stacks for min/max.
# Follow-up: minimum chunks — always 1 (whole array as one chunk).
```

Monotonic Stack

#901 Online Stock Span

ME D

Open on LeetCode

Stack · Design · Monotonic Stack · Data Stream

Lists: AlgoMaster 600

Problem

Design an algorithm that collects daily price quotes for some stock and returns the span of that stock's price for the current day.

The span of the stock's price in one day is the maximum number of consecutive days (starting from that day and going backward) for which the stock price was less than or equal to the price of that day.

- For example, if the prices of the stock in the last four days is [7,2,1,2] and the price of the stock today is 2, then the span of today is 4 because starting from today, the price of the stock was less than or equal 2 for 4 consecutive days.
- Also, if the prices of the stock in the last four days is [7,34,1,2] and the price of the stock today is 8, then the span of today is 3 because starting from today, the price of the stock was less than or equal 8 for 3 consecutive days.

Implement the StockSpanner class:

- StockSpanner() Initializes the object of the class.
- int next(int price) Returns the span of the stock's price given that today's price is price.

Example 1:

```
Input
["StockSpanner", "next", "next", "next", "next", "next", "next", "next"]
[[], [100], [80], [60], [70], [60], [75], [85]]
Output
[null, 1, 1, 1, 2, 1, 4, 6]

Explanation
StockSpanner stockSpanner = new StockSpanner();
```

Constraints:

- 1 <= price <= 105
- At most 104 calls will be made to next.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Span of stock price = max consecutive days (including today) where price <= today's price. Right?"
# "prices=[100,80,60,70,60,75,85]: spans=[1,1,1,2,1,4,6] ✓"
# Monotonic decreasing stack accumulates spans. Amortized O(1) per call. Space O(n).
# Follow-up: daily temperatures (LC 739) — next greater, not span length.
# Follow-up: if prices can be queried in reverse — offline algorithm needed.
```


Monotonic Stack

#907 Sum of Subarray Minimums

ME D

Open on LeetCode

Array · Dynamic Programming · Stack · Monotonic Stack

Lists: AlgoMaster 600

Problem

Given an array of integers arr, find the sum of min(b), where b ranges over every (contiguous) subarray of arr. Since the answer may be large, return the answer modulo 109 + 7.

Example 1:

```
Input: arr = [3,1,2,4]
Output: 17
Explanation:
Subarrays are [3], [1], [2], [4], [3,1], [1,2], [2,4], [3,1,2], [1,2,4], [3,1,2,4].
Minimums are 3, 1, 2, 4, 1, 1, 2, 1, 1, 1.
Sum is 17.
```

Example 2:

```
Input: arr = [11,81,94,43,3]
Output: 444
```

Constraints:

- 1 <= arr.length <= 3 * 104
- 1 <= arr[i] <= 3 * 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return sum of min(subarray) for all subarrays, mod 10^9+7. Right?"
# "arr=[3,1,2,4]→17 (mins: 3,1,1,1,1,2,2,4→sum=17) ✓"
# Contribution: each element is min for (mid−left)*(right−mid) subarrays. O(n). Space O(n).
# Follow-up: Sum of Subarray Maximums — same approach, flip comparison.
# Follow-up: Sum of Subarray Ranges (LC 2104) = sum_max − sum_min.
```

Monotonic Stack

#1019 Next Greater Node In Linked List

MED

Open on LeetCode

Array · Linked List · Stack · Monotonic Stack

Lists: AlgoMaster 600

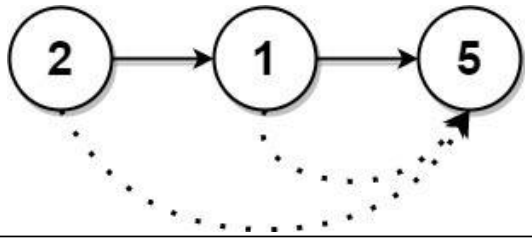
Problem

You are given the head of a linked list with n nodes.

For each node in the list, find the value of the next greater node. That is, for each node, find the value of the first node that is next to it and has a strictly larger value than it.

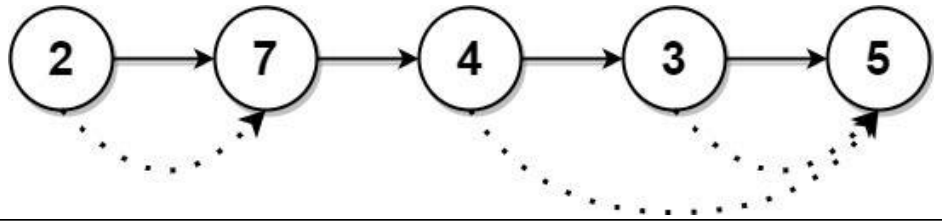
Return an integer array answer where answer[i] is the value of the next greater node of the ith node (1-indexed). If the ith node does not have a next greater node, set answer[i] = 0.

Example 1:




```
Input: head = [2,1,5]
Output: [5,5,0]
```

Example 2:




```
Input: head = [2,7,4,3,5]
Output: [7,0,5,5,0]
```

Constraints:

- The number of nodes in the list is n.
- 1 <= n <= 104
- 1 <= Node.val <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "For each node, find the value of the next node with a greater value. 0 if none. Right?"
# "head=[2,1,5]→[5,5,0] ✓ head=[2,7,4,3,5]→[7,0,5,5,0] ✓"

# Convert to array, then monotonic stack. O(n). Space O(n).
# Follow-up: if we need next SMALLER — flip comparison.
# Follow-up: process in-place without converting to array — stack of (val,idx) pairs.
```


Monotonic Stack

#2104 Sum of Subarray Ranges

MED

Open on LeetCode

Array · Stack · Monotonic Stack

Lists: AlgoMaster 600

Problem

You are given an integer array nums. The range of a subarray of nums is the difference between the largest and smallest element in the subarray.

Return the sum of all subarray ranges of nums.

A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

```
Input: nums = [1,2,3]
Output: 4
Explanation: The 6 subarrays of nums are the following:
[1], range = largest - smallest = 1 - 1 = 0
[2], range = 2 - 2 = 0
[3], range = 3 - 3 = 0
[1,2], range = 2 - 1 = 1
[2,3], range = 3 - 2 = 1
```

Example 2:

```
Input: nums = [1,3,3]
Output: 4
Explanation: The 6 subarrays of nums are the following:
[1], range = largest - smallest = 1 - 1 = 0
[3], range = 3 - 3 = 0
[3], range = 3 - 3 = 0
[1,3], range = 3 - 1 = 2
[3,3], range = 3 - 3 = 0
```

Example 3:

```
Input: nums = [4,-2,-3,4,1]
Output: 59
Explanation: The sum of all subarray ranges of nums is 59.
```

Constraints:

- 1 <= nums.length <= 1000
- -109 <= nums[i] <= 109

Follow-up: Could you find a solution with O(n) time complexity?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Sum of (max-min) for all subarrays. Right?"
# "nums=[1,2,3]→4 (ranges: 0,1,2,1,1,0 → sum=4) ✓"
        # sum_of_maximums - sum_of_minimums
# Contribution technique: sum(max) - sum(min). Each uses monotonic stack. O(n). Space O(n).
# Follow-up: Sum of Subarray Minimums (LC 907) — just the min part.
# Follow-up: if range is defined differently (e.g., max*min) — similar contribution approach.
```

Queues

Round 5 · Mid · Medium · 3 questions

Queues

#950 Reveal Cards In Increasing Order

MED

Open on LeetCode

Array · Queue · Sorting · Simulation

Lists: AlgoMaster 600

Problem

You are given an integer array deck. There is a deck of cards where every card has a unique integer. The integer on the ith card is deck[i].

You can order the deck in any order you want. Initially, all the cards start face down (unrevealed) in one deck.

You will do the following steps repeatedly until all cards are revealed:

1. Take the top card of the deck, reveal it, and take it out of the deck.
2. If there are still cards in the deck then put the next top card of the deck at the bottom of the deck.
3. If there are still unrevealed cards, go back to step 1. Otherwise, stop.

Return an ordering of the deck that would reveal the cards in increasing order.

Note that the first entry in the answer is considered to be the top of the deck.

Example 1:

```
Input: deck = [17,13,11,2,3,5,7]
Output: [2,13,3,11,5,17,7]
Explanation:
We get the deck in the order [17,13,11,2,3,5,7] (this order does not matter), and reorder it.
After reordering, the deck starts as [2,13,3,11,5,17,7], where 2 is the top of the deck.
We reveal 2, and move 13 to the bottom. The deck is now [3,11,5,17,7,13].
We reveal 3, and move 11 to the bottom. The deck is now [5,17,7,13,11].
We reveal 5, and move 17 to the bottom. The deck is now [7,13,11,17].
```

Example 2:

```
Input: deck = [1,1000]
Output: [1,1000]
```

Constraints:

- 1 <= deck.length <= 1000
- 1 <= deck[i] <= 106
- All the values of deck are unique.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Simulate: reveal top card, move next to bottom. Rearrange deck so revealed order is sorted. Right?"
# "deck=[17,13,11,2,3,5,7]→[2,13,3,11,5,17,7] ✓"
# Simulate index placement: assign sorted cards to positions generated by the reveal process.
# Time O(n log n). Space O(n).
# Follow-up: reveal in decreasing order — sort descending and reverse the simulation.
# Follow-up: if we can only cut (not shuffle) the deck — the arrangement changes.
```

Queues

#1823 Find the Winner of the Circular Game

MED

Open on LeetCode

Array · Math · Recursion · Queue · Simulation

Lists: AlgoMaster 600

Problem

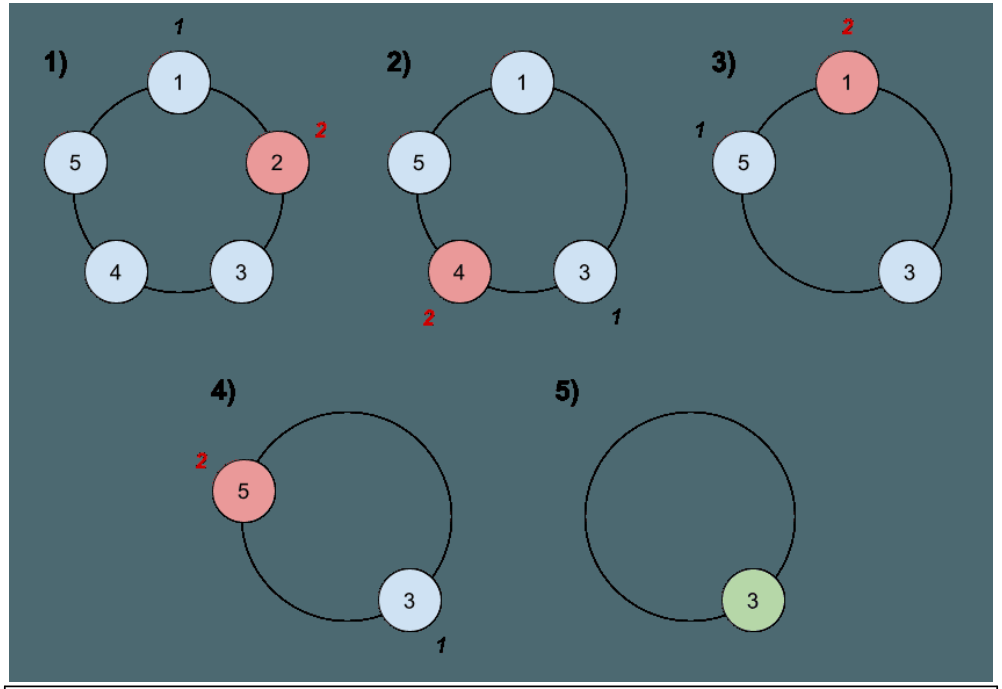
There are n friends that are playing a game. The friends are sitting in a circle and are numbered from 1 to n in clockwise order. More formally, moving clockwise from the ith friend brings you to the (i+1)th friend for 1 <= i < n, and moving clockwise from the nth friend brings you to the 1st friend.

The rules of the game are as follows:

1. Start at the 1st friend.
2. Count the next k friends in the clockwise direction including the friend you started at. The counting wraps around the circle and may count some friends more than once.
3. The last friend you counted leaves the circle and loses the game.
4. If there is still more than one friend in the circle, go back to step 2 starting from the friend immediately clockwise of the friend who just lost and repeat.
5. Else, the last friend in the circle wins the game.

Given the number of friends, n, and an integer k, return the winner of the game.

Example 1:




```
Input: n = 5, k = 2
Output: 3
Explanation: Here are the steps of the game:
1) Start at friend 1.
2) Count 2 friends clockwise, which are friends 1 and 2.
3) Friend 2 leaves the circle. Next start is friend 3.
4) Count 2 friends clockwise, which are friends 3 and 4.
5) Friend 4 leaves the circle. Next start is friend 5.
```

Example 2:


```
Input: n = 6, k = 5
Output: 1
Explanation: The friends leave in this order: 5, 4, 6, 2, 3. The winner is friend 1.
```

Constraints:

- 1 <= k <= n <= 500

Follow up:

Could you solve this problem in linear time with constant space?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "n friends in a circle, count k each round, eliminate the k-th. Who's last? Right?"
# "n=5,k=2: eliminate 2,4,1,5→winner=3 ✓ n=6,k=5→winner=1 ✓"
        # Josephus problem: f(1,k)=0, f(n,k)=(f(n-1,k)+k)%n, answer=f(n,k)+1
# Josephus formula: O(n) time, O(1) space — no queue needed.
# Follow-up: find the order of elimination — run the queue simulation and record each removed element.
# Follow-up: if k changes each round — must simulate, no closed-form.
```

Queues

#2327 Number of People Aware of a Secret

MED

Open on LeetCode

Dynamic Programming · Queue · Simulation

Lists: AlgoMaster 600

Problem

On day 1, one person discovers a secret.

You are given an integer delay, which means that each person will share the secret with a new person every day, starting from delay days after discovering the secret. You are also given an integer forget, which means that each person will forget the secret forget days after discovering it. A person cannot share the secret on the same day they forgot it, or on any day afterwards.

Given an integer n, return the number of people who know the secret at the end of day n. Since the answer may be very large, return it modulo 109 + 7.

Example 1:

```
Input: n = 6, delay = 2, forget = 4
Output: 5
Explanation:
Day 1: Suppose the first person is named A. (1 person)
Day 2: A is the only person who knows the secret. (1 person)
Day 3: A shares the secret with a new person, B. (2 people)
Day 4: A shares the secret with a new person, C. (3 people)
Day 5: A forgets the secret, and B shares the secret with a new person, D. (3 people)
```

Example 2:

```
Input: n = 4, delay = 1, forget = 3
Output: 6
Explanation:
Day 1: The first person is named A. (1 person)
Day 2: A shares the secret with B. (2 people)
Day 3: A and B share the secret with 2 new people, C and D. (4 people)
Day 4: A forgets the secret. B, C, and D share the secret with 3 new people. (6 people)
```

Constraints:

- 2 <= n <= 1000
- 1 <= delay < forget <= n

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "On day 1, person 1 learns a secret. After delay days they can share it. After forget days they forget.
# Count people who know on day n. Return mod 10^9+7. Right?"
# "n=6,delay=2,forget=4: answer=5 ✓"
        # dp[i]=people who learned on day i and still know
            # Add people from d-delay who just became able to share
            # Remove people who forgot (learned on day d-forget, stopped sharing on d-forget+forget-delay=d-delay)
# DP with sliding window sum of active sharers. O(n). Space O(n).
# Follow-up: if forget day is infinity — people never forget, simplifies to prefix sum.
# Follow-up: track total number who ever knew (not just currently know).
```


Monotonic Queue

Round 5 · Mid · Medium · 5 questions

Monotonic Queue

#1438 Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit

MED

Open on LeetCode

Array · Queue · Sliding Window · Heap (Priority Queue) · Ordered Set · Monotonic Queue

Lists: AlgoMaster 600

Problem

Given an array of integers nums and an integer limit, return the size of the longest non-empty subarray such that the absolute difference between any two elements of this subarray is less than or equal to limit.

Example 1:

```
Input: nums = [8,2,4,7], limit = 4
Output: 2
Explanation: All subarrays are:
[8] with maximum absolute diff |8-8| = 0 <= 4.
[8,2] with maximum absolute diff |8-2| = 6 > 4.
[8,2,4] with maximum absolute diff |8-2| = 6 > 4.
[8,2,4,7] with maximum absolute diff |8-2| = 6 > 4.
[2] with maximum absolute diff |2-2| = 0 <= 4.
```

Example 2:

```
Input: nums = [10,1,2,4,7,2], limit = 5
Output: 4
Explanation: The subarray [2,4,7,2] is the longest since the maximum absolute diff is |2-7| = 5 <= 5.
```

Example 3:

```
Input: nums = [4,2,2,2,4,4,2,2], limit = 0
Output: 3
```

Constraints:

- 1 <= nums.length <= 105
- 1 <= nums[i] <= 109
- 0 <= limit <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find longest subarray where max-min <= limit. Right?"
# "nums=[8,2,4,7],limit=4→2 (subarray [2,4] or [4,7]) ✓"
# Two monotonic deques (max and min) + sliding window. O(n). Space O(n).
# Follow-up: shortest such subarray — binary search + deque.
# Follow-up: if limit can change per query — offline processing with segment tree.
```


Monotonic Queue

#1673 Find the Most Competitive Subsequence

MED

Open on LeetCode

Array · Stack · Greedy · Monotonic Stack

Lists: AlgoMaster 600

Problem

Given an integer array nums and a positive integer k, return the most competitive subsequence of nums of size k.
An array's subsequence is a resulting sequence obtained by erasing some (possibly zero) elements from the array.
We define that a subsequence a is more competitive than a subsequence b (of the same length) if in the first position where a and b differ, subsequence a has a number less than the corresponding number in b. For example, [1,3,4] is more competitive than [1,3,5] because the first position they differ is at the final number, and 4 is less than 5.

Example 1:

```
Input: nums = [3,5,2,6], k = 2
Output: [2,6]
Explanation: Among the set of every possible subsequence: {[3,5], [3,2], [3,6], [5,2], [5,6], [2,6]}, [2,6] is the most competitive.
```

Example 2:

```
Input: nums = [2,4,3,3,5,4,9,6], k = 4
Output: [2,3,3,4]
```

Constraints:

- 1 <= nums.length <= 105
- 0 <= nums[i] <= 109
- 1 <= k <= nums.length

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find lexicographically smallest subsequence of length k from nums. Right?"
# "nums=[3,5,2,6],k=2→[2,6] ✓ nums=[2,4,3,3,5,4,9,6],k=4→[2,3,3,4] ✓"
# Monotonic increasing stack, keep exactly k. Same as brute — already O(n). Space O(k).
# Follow-up: if we need the actual indices (not values) — store indices in stack.
# Follow-up: k largest instead of smallest — flip to decreasing stack.
```

Monotonic Queue

#1696 Jump Game VI

ME D

Open on LeetCode

Array · Dynamic Programming · Queue · Heap (Priority Queue) · Monotonic Queue

Lists: AlgoMaster 600

Problem

You are given a 0-indexed integer array nums and an integer k.

You are initially standing at index 0. In one move, you can jump at most k steps forward without going outside the boundaries of the array. That is, you can jump from index i to any index in the range [i + 1, min(n - 1, i + k)] inclusive.

You want to reach the last index of the array (index n - 1). Your score is the sum of all nums[j] for each index j you visited in the array.

Return the maximum score you can get.

Example 1:

```
Input: nums = [1,-1,-2,4,-7,3], k = 2
Output: 7
Explanation: You can choose your jumps forming the subsequence [1,-1,4,3] (underlined above). The sum is 7.
```

Example 2:

```
Input: nums = [10,-5,-2,4,0,3], k = 3
Output: 17
Explanation: You can choose your jumps forming the subsequence [10,4,3] (underlined above). The sum is 17.
```

Example 3:

```
Input: nums = [1,-5,-20,4,-1,3,-6,-3], k = 2
Output: 0
```

Constraints:

- 1 <= nums.length, k <= 105
- -104 <= nums[i] <= 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Can jump 1 to k steps. dp[i]=max score reaching index i. nums[i] added on arrival. Right?"
# "nums=[1,-1,-2,4,-7,3],k=2→7 (1→-1→4→3) ✓"
# Monotonic deque stores indices of decreasing dp values. Sliding window max. O(n). Space O(k).
# Follow-up: Constrained Subsequence Sum (LC 1425) — same pattern with different update rule.
# Follow-up: if k is unlimited — reduces to max prefix sum + Kadane's.
```


Monotonic Queue

#2762 Continuous Subarrays

ME D

Open on LeetCode

Array · Queue · Sliding Window · Heap (Priority Queue) · Ordered Set · Monotonic Queue

Lists: AlgoMaster 600

Problem

You are given a 0-indexed integer array nums. A subarray of nums is called continuous if:

- Let i, i + 1, ..., j be the indices in the subarray. Then, for each pair of indices i <= i1, i2 <= j, 0 <= |nums[i1] - nums[i2]| <= 2.

Return the total number of continuous subarrays.

A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

```
Input: nums = [5,4,2,4]
Output: 8
Explanation:
Continuous subarray of size 1: [5], [4], [2], [4].
Continuous subarray of size 2: [5,4], [4,2], [2,4].
Continuous subarray of size 3: [4,2,4].
There are no subarrys of size 4.
Total continuous subarrays = 4 + 3 + 1 = 8.
```

Example 2:

```
Input: nums = [1,2,3]
Output: 6
Explanation:
Continuous subarray of size 1: [1], [2], [3].
Continuous subarray of size 2: [1,2], [2,3].
Continuous subarray of size 3: [1,2,3].
Total continuous subarrays = 3 + 2 + 1 = 6.
```

Constraints:

- 1 <= nums.length <= 105
- 1 <= nums[i] <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count subarrays where |nums[i]-nums[j]|<=2 for all pairs i,j in subarray. Right?"
# "nums=[5,4,2,4]→8 ✓"
# Two deques + sliding window. count+=window_size at each step. O(n). Space O(n).
# Follow-up: if limit is a parameter — same approach with variable threshold.
# Follow-up: Longest Continuous Subarray (LC 1438) — find max length instead of count.
```

Monotonic Queue

#3578 Count Partitions With Max-Min Difference at Most K

MED

Open on LeetCode

Array · Dynamic Programming · Queue · Sliding Window · Prefix Sum · Monotonic Queue
Lists: AlgoMaster 600

Problem

You are given an integer array nums and an integer k. Your task is to partition nums into one or more non-empty contiguous segments such that in each segment, the difference between its maximum and minimum elements is at most k.
Return the total number of ways to partition nums under this condition.
Since the answer may be too large, return it modulo 109 + 7.
Example 1:
Input: nums = [9,4,1,3,7], k = 4
Output: 6
Explanation:
There are 6 valid partitions where the difference between the maximum and minimum elements in each segment is at most k = 4:

- [[9], [4], [1], [3], [7]]
- [[9], [4], [1], [3, 7]]
- [[9], [4], [1, 3], [7]]
- [[9], [4, 1], [3], [7]]
- [[9], [4, 1], [3, 7]]
- [[9], [4, 1, 3], [7]]

Example 2:
Input: nums = [3,3,4], k = 0
Output: 2
Explanation:
There are 2 valid partitions that satisfy the given conditions:

- [[3], [3], [4]]
- [[3, 3], [4]]

Constraints:

- 2 <= nums.length <= 5 * 104
- 1 <= nums[i] <= 109
- 0 <= k <= 109

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Count ways to partition nums into non-empty subarrays where each partition's max-min<=k. Right?"
# "nums=[9,4,1,3,7],k=4→2 ✓"
            # dp[right+1] = sum(dp[left..right])
# DP + monotonic deques + prefix sums. O(n). Space O(n).
# Follow-up: Continuous Subarrays (LC 2762) — count subarrays instead of partitions.
# Follow-up: if k varies per partition — much harder, needs range tree.
```


Divide and Conquer

Round 5 · Mid · Medium · 3 questions

Divide and Conquer

#109 Convert Sorted List to Binary Search Tree

MED

Open on LeetCode

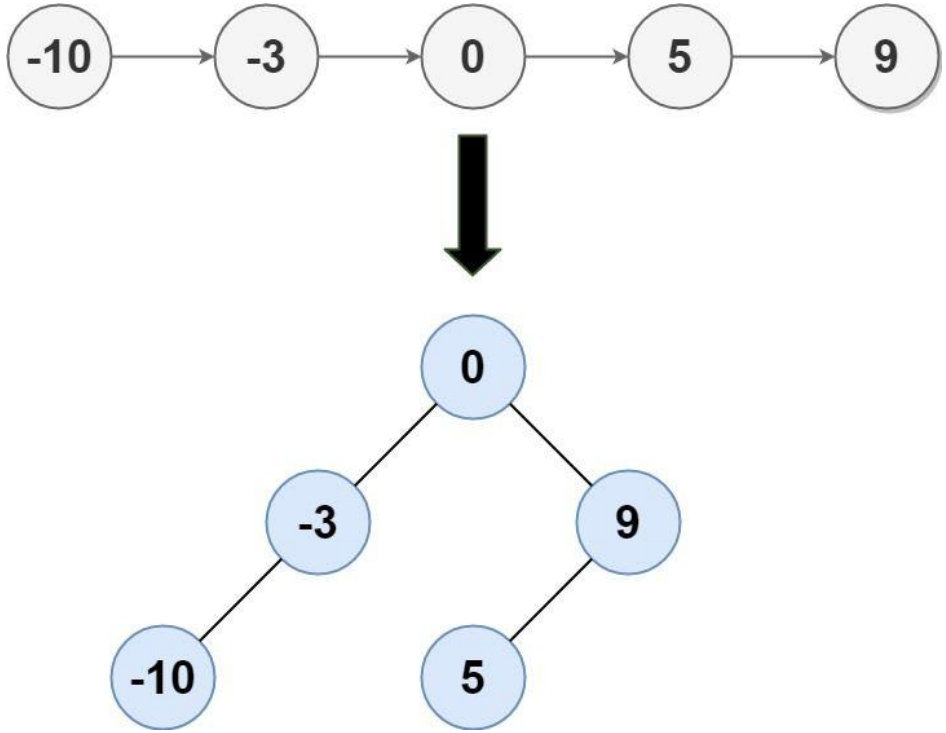
Linked List · Divide and Conquer · Tree · Binary Search Tree · Binary Tree

Lists: AlgoMaster 600

Problem

Given the head of a singly linked list where elements are sorted in ascending order, convert it to a height-balanced binary search tree.

Example 1:




```
Input: head = [-10,-3,0,5,9]
Output: [0,-3,9,-10,null,5]
Explanation: One possible answer is [0,-3,9,-10,null,5], which represents the shown height balanced BST.
```

Example 2:

```
Input: head = []
Output: []
```

Constraints:

- The number of nodes in head is in the range [0, 2 * 104].
- -105 <= Node.val <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Convert sorted linked list to height-balanced BST. Right?"
# "head=[-10,-3,0,5,9]→[0,-3,9,-10,null,5] (height-balanced BST) ✓"
# Slow/fast to find middle in-place. O(n log n) — each level finds mid in O(n/level).
# Better: inorder simulation O(n) by advancing list pointer in BST inorder.
# Follow-up: Convert Sorted Array to BST (LC 108) — same but O(n) with array indexing.
```


Divide and Conquer

#427 Construct Quad Tree

MED

Open on LeetCode

Array · Divide and Conquer · Tree · Matrix

Lists: AlgoMaster 600

Problem

Given a n * n matrix grid of 0's and 1's only. We want to represent grid with a Quad-Tree.

Return the root of the Quad-Tree representing grid.

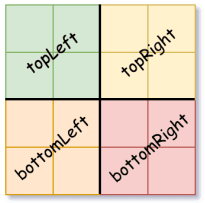
A Quad-Tree is a tree data structure in which each internal node has exactly four children. Besides, each node has two attributes:

- val: True if the node represents a grid of 1's or False if the node represents a grid of 0's. Notice that you can assign the val to True or False when isLeaf is False, and both are accepted in the answer.
- isLeaf: True if the node is a leaf node on the tree or False if the node has four children.

```
class Node {
public boolean val;
public boolean isLeaf;
public Node topLeft;
public Node topRight;
public Node bottomLeft;
public Node bottomRight;
}
```

We can construct a Quad-Tree from a two-dimensional area using the following steps:

1. If the current grid has the same value (i.e all 1's or all 0's) set isLeaf True and set val to the value of the grid and set the four children to Null and stop.
2. If the current grid has different values, set isLeaf to False and set val to any value and divide the current grid into four sub-grids as shown in the photo.
3. Recurse for each of the children with the proper sub-grid.



If you want to know more about the Quad-Tree, you can refer to the wiki.

Quad-Tree format:

You don't need to read this section for solving the problem. This is only if you want to understand the output format here. The output represents the serialized format of a Quad-Tree using level order traversal, where null signifies a path terminator where no node exists below.

It is very similar to the serialization of the binary tree. The only difference is that the node is represented as a list [isLeaf, val].

If the value of isLeaf or val is True we represent it as 1 in the list [isLeaf, val] and if the value of isLeaf or val is False we represent it as 0.

Example 1:


```
Input: grid = [[0,1],[1,0]]
Output: [[0,1],[1,0],[1,1],[1,1],[1,0]]
Explanation: The explanation of this example is shown below:
Notice that 0 represents False and 1 represents True in the photo representing the Quad-Tree.
```

Example 2:

1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0

```
Input: grid = [[1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0],[1,1,1,1,1,1,1,1],[1,1,1,1,1,1,1,1],[1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0],[1,1
,1,1,0,0,0,0],[1,1,1,1,1,0,0,0,0]]
Output: [[0,1],[1,1],[0,1],[1,1],[1,0],null,null,null,null,[1,0],[1,0],[1,1],[1,1]]
Explanation: All values in the grid are not the same. We divide the grid into four sub-grids.
The topLeft, bottomLeft and bottomRight each has the same value.
The topRight have different values so we divide it into 4 sub-grids where each has the same value.
Explanation is shown in the photo below:
```

Constraints:

- n == grid.length == grid[i].length
- n == 2x where 0 <= x <= 6

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Build a Quad Tree from an n×n binary grid. Leaf if all same value. Right?"
# "grid=[[0,1],[1,0]]→4 leaf nodes ✓"
# Recursive subdivision. O(n² log n): O(n²) per level, log n levels.
# With 2D prefix sums, can check uniformity in O(1) → O(n²) total.
# Follow-up: query Quad Tree for sum in rectangle — traverse down to relevant leaves.
```


Divide and Conquer

#654 Maximum Binary Tree

ME D

Open on LeetCode

Array · Divide and Conquer · Stack · Tree · Monotonic Stack · Binary Tree

Lists: AlgoMaster 600

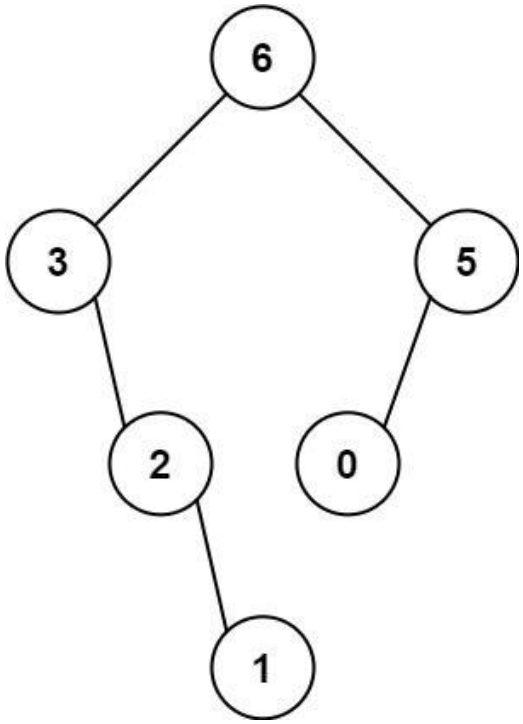
Problem

You are given an integer array nums with no duplicates. A maximum binary tree can be built recursively from nums using the following algorithm:

1. Create a root node whose value is the maximum value in nums.
2. Recursively build the left subtree on the subarray prefix to the left of the maximum value.
3. Recursively build the right subtree on the subarray suffix to the right of the maximum value.

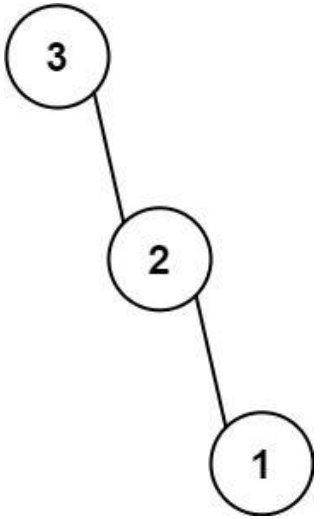
Return the maximum binary tree built from nums.

Example 1:




```
Input: nums = [3,2,1,6,0,5]
Output: [6,3,5,null,2,0,null,null,1]
Explanation: The recursive calls are as follow:
- The largest value in [3,2,1,6,0,5] is 6. Left prefix is [3,2,1] and right suffix is [0,5].
- The largest value in [3,2,1] is 3. Left prefix is [] and right suffix is [2,1].
- Empty array, so no child.
- The largest value in [2,1] is 2. Left prefix is [] and right suffix is [1].
- Empty array, so no child.
```

Example 2:




```
Input: nums = [3,2,1]
Output: [3,null,2,null,1]
```

Constraints:

- **1 <= nums.length <= 1000**
- **0 <= nums[i] <= 1000**
- **All integers in nums are unique.**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Build a 'maximum binary tree': root = max element, left = max tree of left subarray,
# right = max tree of right subarray. Return root. Right?"
#
# "nums=[3,2,1,6,0,5]: root=6, left=max_tree([3,2,1])=3, right=max_tree([0,5])=5 ✓"

# PHASE 2 — BRUTE FORCE
# "Recursion: find max in range, split, recurse. Time: O(n²) worst case. Space: O(n) recursion."

# PHASE 3 — OPTIMIZE
# "Monotonic decreasing stack: process left-to-right. Each new element becomes the right child
# of the last element smaller than it, and itself becomes the parent of those smaller elements.
# Time: O(n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[3,2,1,6,0,5]:
# 3→stack=[3]. 2<3→stack=[3,2]. 1<2→stack=[3,2,1]. 6>all: node(6).left=1,1.left=2?
# pop1→6.left=1. pop2→6.left=2. pop3→6.left=3. stack=[6].
# 0→stack=[6,0]. 5>0→5.left=0. 6.right=5. stack=[6]. root=stack[0]=6 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): each element pushed/popped once. Space O(n): stack.
# Follow-up: find all elements that become the 'maximum binary tree parent' of element i.
# Follow-up: Maximum Binary Tree II (LC 998) — insert into existing max binary tree."
```


Merge Sort

Round 5 · Mid · Medium · 1 question

Merge Sort

#912 Sort an Array

MED

Open on LeetCode

Array · Divide and Conquer · Sorting · Heap (Priority Queue) · Merge Sort · Bucket Sort · Radix Sort · Counting Sort

Lists: AlgoMaster 600

Problem

Given an array of integers nums, sort the array in ascending order and return it.

You must solve the problem without using any built-in functions in O(nlog(n)) time complexity and with the smallest space complexity possible.

Example 1:

```
Input: nums = [5,2,3,1]
Output: [1,2,3,5]
Explanation: After sorting the array, the positions of some numbers are not changed (for example, 2 and 3), while the positions of other numbers are changed (for example, 1 and 5).
```

Example 2:

```
Input: nums = [5,1,1,2,0,0]
Output: [0,0,1,1,2,5]
Explanation: Note that the values of nums are not necessarily unique.
```

Constraints:

- 1 <= nums.length <= 5 * 104
- -5 * 104 <= nums[i] <= 5 * 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Sort an array in ascending order without using built-in sort. Right?"
# "nums=[5,2,3,1]→[1,2,3,5] ✓"
# Merge sort: O(n log n) time, O(n) space. Stable sort.
# Follow-up: QuickSort — O(n log n) avg, O(1) space, but O(n²) worst case.
# Follow-up: HeapSort — O(n log n) time, O(1) space, but not stable.
```


QuickSort / QuickSelect

Round 5 · Mid · Medium · 1 question

QuickSort / QuickSelect

#1985 Find the Kth Largest Integer in the Array

MED

Open on LeetCode

Array · String · Divide and Conquer · Sorting · Heap (Priority Queue) · Quickselect

Lists: AlgoMaster 600

Problem

You are given an array of strings nums and an integer k. Each string in nums represents an integer without leading zeros.
Return the string that represents the kth largest integer in nums.
Note: Duplicate numbers should be counted distinctly. For example, if nums is ["1","2","2"], "2" is the first largest integer, "2" is the second-largest integer, and "1" is the third-largest integer.

Example 1:

```
Input: nums = ["3","6","7","10"], k = 4
Output: "3"
Explanation:
The numbers in nums sorted in non-decreasing order are ["3","6","7","10"].
The 4th largest integer in nums is "3".
```

Example 2:

```
Input: nums = ["2","21","12","1"], k = 3
Output: "2"
Explanation:
The numbers in nums sorted in non-decreasing order are ["1","2","12","21"].
The 3rd largest integer in nums is "2".
```

Example 3:

```
Input: nums = ["0","0"], k = 2
Output: "0"
Explanation:
The numbers in nums sorted in non-decreasing order are ["0","0"].
The 2nd largest integer in nums is "0".
```

Constraints:

- 1 <= k <= nums.length <= 104
- 1 <= nums[i].length <= 100
- nums[i] consists of only digits.
- nums[i] will not have any leading zeros.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Strings representing integers. Find kth largest. Compare by value (length first, then lex). Right?"
# "nums=['3','6','7','10'],k=4→'3' ✓ nums=['2','21','12','1'],k=3→'2' ✓"
# QuickSelect: O(n) avg, O(n²) worst. Key: compare by (length, lex) for numeric string comparison.
# Follow-up: if numbers can be negative — handle sign in comparison.
# Follow-up: Kth Largest Element in an Array (LC 215) — same QuickSelect on integers.
```


Backtracking

Round 5 · Mid · Medium · 4 questions

Backtracking

#47 Permutations II

MED

Open on LeetCode

Array · Backtracking · Sorting

Lists: AlgoMaster 600

Problem

Given a collection of numbers, nums, that might contain duplicates, return all possible unique permutations in any order.

Example 1:

```
Input: nums = [1,1,2]
Output:
[[1,1,2],
[1,2,1],
[2,1,1]]
```

Example 2:

```
Input: nums = [1,2,3]
Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]
```

Constraints:

- 1 <= nums.length <= 8
- -10 <= nums[i] <= 10

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return all unique permutations of nums which may contain duplicates. Right?"
# "nums=[1,1,2]→[[1,1,2],[1,2,1],[2,1,1]] ✓"
# Sort + skip duplicate at same level if previous same value unused. O(n!). Space O(n).
# Follow-up: count unique permutations without generating — n!/product(count_i!).
# Follow-up: next permutation (LC 31) — generate just the next one in O(n).
```


Backtracking

#77 Combinations

MED

Open on LeetCode

Backtracking

Lists: AlgoMaster 600

Problem

Given two integers n and k, return all possible combinations of k numbers chosen from the range [1, n].

You may return the answer in any order.

Example 1:

```
Input: n = 4, k = 2
Output: [[1,2],[1,3],[1,4],[2,3],[2,4],[3,4]]
Explanation: There are 4 choose 2 = 6 total combinations.
Note that combinations are unordered, i.e., [1,2] and [2,1] are considered to be the same combination.
```

Example 2:

```
Input: n = 1, k = 1
Output: [[1]]
Explanation: There is 1 choose 1 = 1 total combination.
```

Constraints:

- 1 <= n <= 20
- 1 <= k <= n

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return all combinations of k numbers from range [1,n]. Right?"
# "n=4,k=2→[[1,2],[1,3],[1,4],[2,3],[2,4],[3,4]] ✓"
# Pruning: upper bound = n-(k-len(path))+1 so enough elements remain. O(C(n,k)*k). Space O(k).
# Follow-up: Combination Sum (LC 39) — target sum, elements can repeat.
# Follow-up: count without generating — C(n,k) = n!/(k!(n-k)!).
```

Backtracking

#93 Restore IP Addresses

MED

Open on LeetCode

String · Backtracking

Lists: AlgoMaster 600

Problem

A valid IP address consists of exactly four integers separated by single dots. Each integer is between 0 and 255 (inclusive) and cannot have leading zeros.

- For example, "0.1.2.201" and "192.168.1.1" are valid IP addresses, but "0.011.255.245", "192.168.1.312" and "192.168@1.1" are invalid IP addresses.

Given a string s containing only digits, return all possible valid IP addresses that can be formed by inserting dots into s. You are not allowed to reorder or remove any digits in s. You may return the valid IP addresses in any order.

Example 1:

```
Input: s = "25525511135"
Output: ["255.255.11.135","255.255.111.35"]
```

Example 2:

```
Input: s = "0000"
Output: ["0.0.0.0"]
```

Example 3:

```
Input: s = "101023"
Output: ["1.0.10.23","1.0.102.3","10.1.0.23","10.10.2.3","101.0.2.3"]
```

Constraints:

- 1 <= s.length <= 20
- s consists of digits only.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a string of digits, return all valid IPv4 addresses (4 parts, each 0–255, no leading zeros). Right?"
#
# "s='25525511135': ['255.255.11.135','255.255.111.35'] ✓"

# PHASE 2 — BRUTE FORCE
# "Try all ways to split s into 4 parts. Check each part is valid (0–255, no leading zeros).
# Time: O(3^4 * 4) = O(1). Space: O(1). — Actually O(n) for string ops."

# PHASE 3 — OPTIMIZE
# "Same backtracking — already O(1) since IP has at most 3^3=27 split points checked."

# PHASE 4 — CLEAN CODE (same as brute — already clean)

# PHASE 5 — TEST & DEBUG
# s='25525511135': try 2,25,255 as first parts.
# 255→255→11→135 ✓. 255→255→111→35 ✓. Others exceed 255 or have wrong length. → 2 results ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(3^4 * n) = O(n): at most 3 choices per part, 4 parts, checking takes O(n).
# Pruning 'remaining < parts_left or remaining > parts_left*3' cuts many branches early.
# Follow-up: validate an IP address (LC 468) — check if string is valid IPv4 or IPv6.
# Follow-up: if the IP version is IPv6 — 8 groups, each 1–4 hex digits, different constraints."
```


Backtracking

#216 Combination Sum III

MED

Open on LeetCode

Array · Backtracking

Lists: AlgoMaster 600

Problem

Find all valid combinations of k numbers that sum up to n such that the following conditions are true:

- Only numbers 1 through 9 are used.
- Each number is used at most once.

Return a list of all possible valid combinations. The list must not contain the same combination twice, and the combinations may be returned in any order.

Example 1:

```
Input: k = 3, n = 7
Output: [[1,2,4]]
Explanation:
1 + 2 + 4 = 7
There are no other valid combinations.
```

Example 2:

```
Input: k = 3, n = 9
Output: [[1,2,6],[1,3,5],[2,3,4]]
Explanation:
1 + 2 + 6 = 9
1 + 3 + 5 = 9
2 + 3 + 4 = 9
There are no other valid combinations.
```

Example 3:

```
Input: k = 4, n = 1
Output: []
Explanation: There are no valid combinations.
Using 4 different numbers in the range [1,9], the smallest sum we can get is 1+2+3+4 = 10 and since 10 > 1, there are no valid combination.
```

Constraints:

- 2 <= k <= 9
- 1 <= n <= 60

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find all combinations of exactly k numbers from 1–9 that sum to n. Each digit used at most once. Right?"
# "k=3,n=7→[[1,2,4]] ✓ k=3,n=9→[[1,2,6],[1,3,5],[2,3,4]] ✓"
# Backtracking from 1–9, prune when digit > remaining. O(C(9,k)). Space O(k).
# Follow-up: Combination Sum I (LC 39) — unlimited use, any positive integers.
# Follow-up: how many valid combinations exist — just count, don't collect.
```

BST / Ordered Set

Round 5 · Mid · Medium · 10 questions

BST / Ordered Set

☐ #99 Recover Binary Search Tree MED

Open on LeetCode

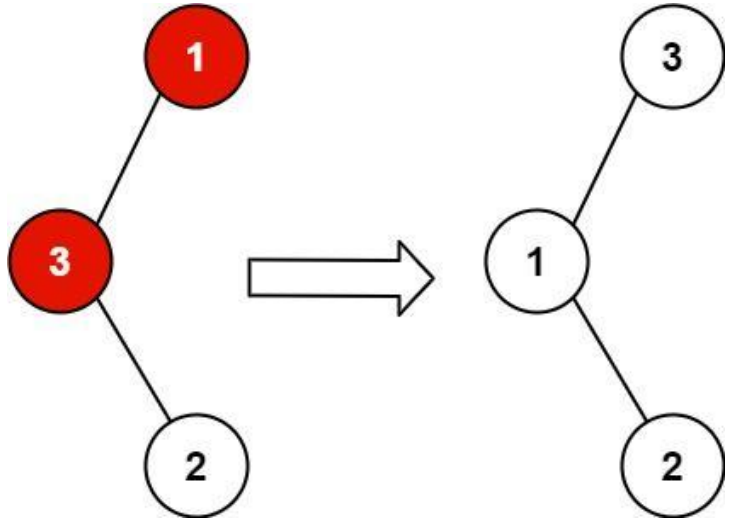
Tree · Depth-First Search · Binary Search Tree · Binary Tree

Lists: AlgoMaster 600

Problem

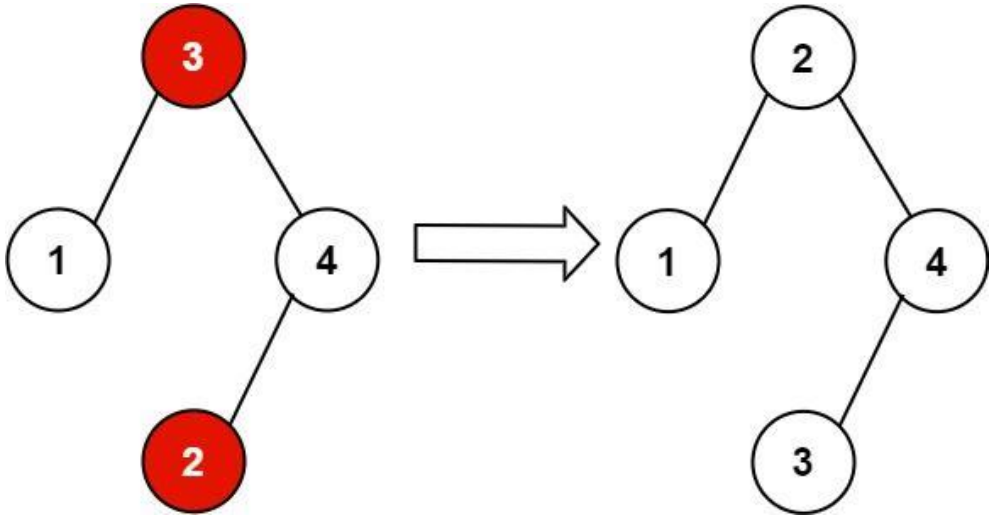
You are given the root of a binary search tree (BST), where the values of exactly two nodes of the tree were swapped by mistake. Recover the tree without changing its structure.

Example 1:




```
Input: root = [1,3,null,null,2]
Output: [3,1,null,null,2]
Explanation: 3 cannot be a left child of 1 because 3 > 1. Swapping 1 and 3 makes the BST valid.
```

Example 2:




```
Input: root = [3,1,4,null,null,2]
Output: [2,1,4,null,null,3]
Explanation: 2 cannot be in the right subtree of 3 because 2 < 3. Swapping 2 and 3 makes the BST valid.
```

Constraints:

- The number of nodes in the tree is in the range [2, 1000].
- -231 <= Node.val <= 231 - 1

Follow up: A solution using O(n) space is pretty straight-forward. Could you devise a constant O(1) space solution?

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Exactly two nodes in BST were swapped. Recover without changing structure. Right?"
# "root=[1,3,null,null,2]→[3,1,null,null,2] (3 and 1 swapped) ✓"
        # Use lists to mutate in closure
# Inorder traversal: find two out-of-order nodes (first descend, last ascend). Swap values. O(n). Space O(h).
# Morris traversal achieves O(1) space.
# Follow-up: if k nodes swapped (not 2) — find all violations in inorder.
```


BST / Ordered Set

#426 Convert Binary Search Tree to Sorted Doubly Linked List ME D

Open on LeetCode

Linked List · Stack · Tree · Depth-First Search · Binary Search Tree · Binary Tree · Doubly-Linked List

Lists: AlgoMaster 600

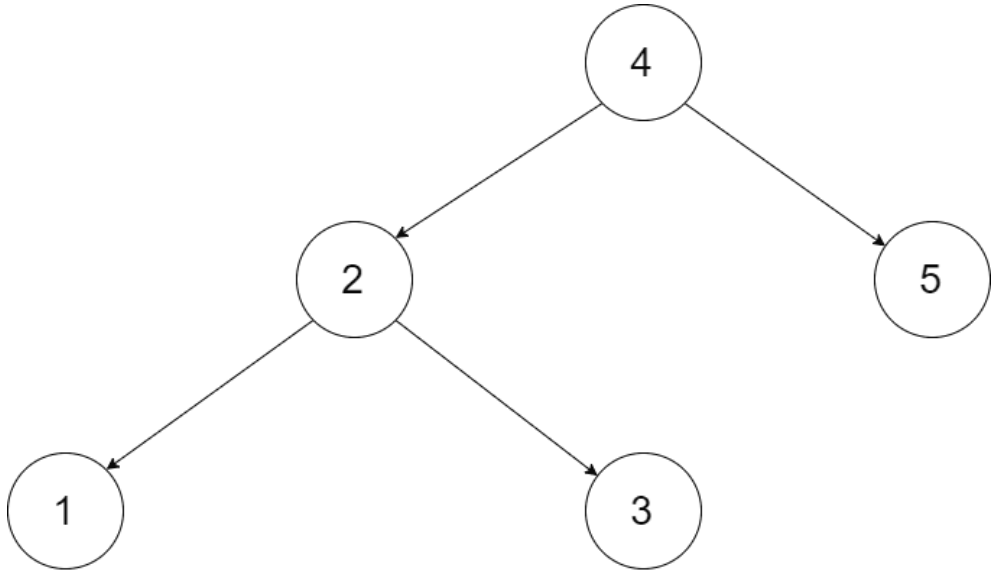
Problem

Convert a Binary Search Tree to a sorted Circular Doubly-Linked List in place.

You can think of the left and right pointers as synonymous to the predecessor and successor pointers in a doubly-linked list. For a circular doubly linked list, the predecessor of the first element is the last element, and the successor of the last element is the first element.

We want to do the transformation in place. After the transformation, the left pointer of the tree node should point to its predecessor, and the right pointer should point to its successor. You should return the pointer to the smallest element of the linked list.

Example 1:




```
Input: root = [4,2,5,1,3]


Output: [1,2,3,4,5]

Explanation: The figure below shows the transformed BST. The solid line indicates the successor relationship, while the dashed
line means the predecessor relationship.
```

Example 2:

```
Input: root = [2,1,3]
Output: [1,2,3]
```

Constraints:

- The number of nodes in the tree is in the range [0, 2000].
- -1000 <= Node.val <= 1000
- All the values of the tree are unique.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Convert BST to sorted circular doubly linked list in-place.
# Left pointer = prev, right pointer = next. Head = smallest element. Right?"
#
# "BST [4,2,5,1,3] → circular DLL: 1⇄2⇄3⇄4⇄5⇄(back to 1) ✓"
```

```
# PHASE 2 — BRUTE FORCE
# "Inorder traversal, collect nodes, link sequentially, connect head and tail.
# Time: O(n). Space: O(n) for node list."

# PHASE 3 — OPTIMIZE
# "In-place inorder with prev pointer. Link prev.right=curr and curr.left=prev as we go.
# Track head (first node) and tail (last node) to close the circle.
# Time: O(n). Space: O(h) recursion."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# BST [4,2,5,1,3]: inorder: 1,2,3,4,5.
# 1: head=1,tail=1. 2: 1.right=2,2.left=1,tail=2. ... 5: tail=5. Close: 1.left=5,5.right=1.
# → circular DLL 1⇄2⇄3⇄4⇄5 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(h): recursion.
# Follow-up: if the original tree must be preserved — copy nodes instead of relinking.
# Follow-up: merge two such sorted DLLs — split circular, merge, reconnect."
```


BST / Ordered Set

#450 Delete Node in a BST

ME
D

Open on LeetCode

Tree · Binary Search Tree · Binary Tree

Lists: AlgoMaster 600

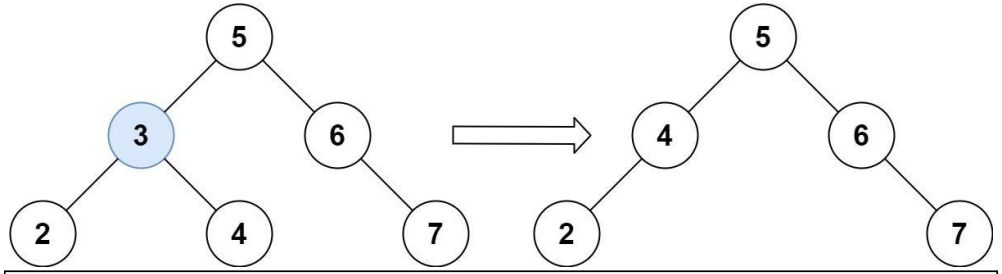
Problem

Given a root node reference of a BST and a key, delete the node with the given key in the BST. Return the root node reference (possibly updated) of the BST.

Basically, the deletion can be divided into two stages:

1. Search for a node to remove.
2. If the node is found, delete the node.

Example 1:




```
Input: root = [5,3,6,2,4,null,7], key = 3
Output: [5,4,6,2,null,null,7]
Explanation: Given key to delete is 3. So we find the node with value 3 and delete it.
One valid answer is [5,4,6,2,null,null,7], shown in the above BST.
Please notice that another valid answer is [5,2,6,null,4,null,7] and it's also accepted.
```

Example 2:

```
Input: root = [5,3,6,2,4,null,7], key = 0
Output: [5,3,6,2,4,null,7]
Explanation: The tree does not contain a node with value = 0.
```

Example 3:

```
Input: root = [], key = 0
Output: []
```

Constraints:

- The number of nodes in the tree is in the range [0, 104].
- -105 <= Node.val <= 105
- Each node has a unique value.
- root is a valid binary search tree.
- -105 <= key <= 105

Follow up: Could you solve it with time complexity O(height of tree)?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Delete the node with given key from BST. Return root. Right?"
# "root=[5,3,6,2,4,null,7],key=3→[5,4,6,2,null,null,7] or [5,2,6,null,4,null,7] ✓"
            # Find inorder successor (min of right subtree)
# Replace deleted node value with inorder successor, then delete successor. O(h). Space O(h).
# Follow-up: Delete in a sorted linked list — same 3-case logic.
# Follow-up: if tree must remain balanced — use AVL or Red-Black tree rotation.
```

BST / Ordered Set

#510 Inorder Successor in BST II

MED

Open on LeetCode

Tree · Binary Search Tree · Binary Tree

Lists: AlgoMaster 600

Problem

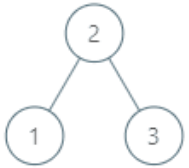
Given a node in a binary search tree, return the in-order successor of that node in the BST. If that node has no in-order successor, return null.

The successor of a node is the node with the smallest key greater than node.val.

You will have direct access to the node but not to the root of the tree. Each node will have a reference to its parent node. Below is the definition for Node:

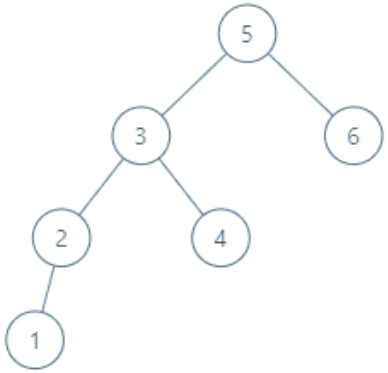
```
class Node {
public int val;
public Node left;
public Node right;
public Node parent;
}
```

Example 1:




```
Input: tree = [2,1,3], node = 1
Output: 2
Explanation: 1's in-order successor node is 2. Note that both the node and the return value is of Node type.
```

Example 2:




```
Input: tree = [5,3,6,2,4,null,null,1], node = 6
Output: null
Explanation: There is no in-order successor of the current node, so the answer is null.
```

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- -105 <= Node.val <= 105
- All Nodes will have unique values.

Follow up: Could you solve it without looking up any of the node's values?

◆ Interview Approach · STAR-LC


```
# PHASE 1 — CLARIFY
# "Given a node in BST (with parent pointer), find its inorder successor. Right?"
# "BST=[2,1,3], node=1 → successor is node(2) ✓"
        # If has right child, return leftmost in right subtree
        # Go up until we came from a left branch
# Two cases: right subtree exists (leftmost) or traverse up to first left-child ancestor. O(h). Space O(1).
# Follow-up: Inorder Successor in BST I (LC 285) — without parent pointer, use BST search.
# Follow-up: inorder predecessor — symmetric: left subtree or first right-child ancestor.
```

BST / Ordered Set

#669 Trim a Binary Search Tree

MED

Open on LeetCode

Tree · Depth-First Search · Binary Search Tree · Binary Tree

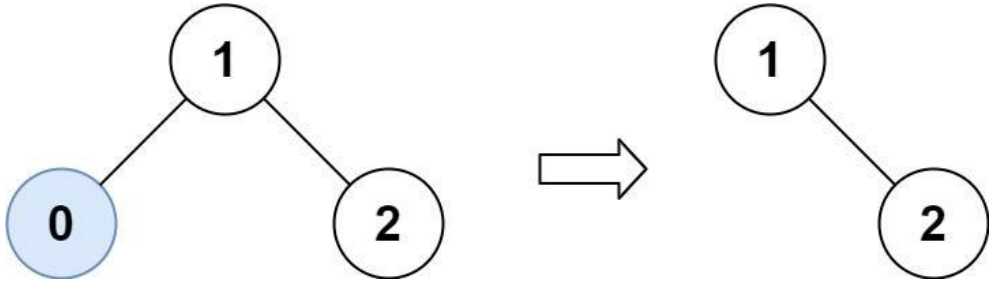
Lists: AlgoMaster 600

Problem

Given the root of a binary search tree and the lowest and highest boundaries as low and high, trim the tree so that all its elements lies in [low, high]. Trimming the tree should not change the relative structure of the elements that will remain in the tree (i.e., any node's descendant should remain a descendant). It can be proven that there is a unique answer.

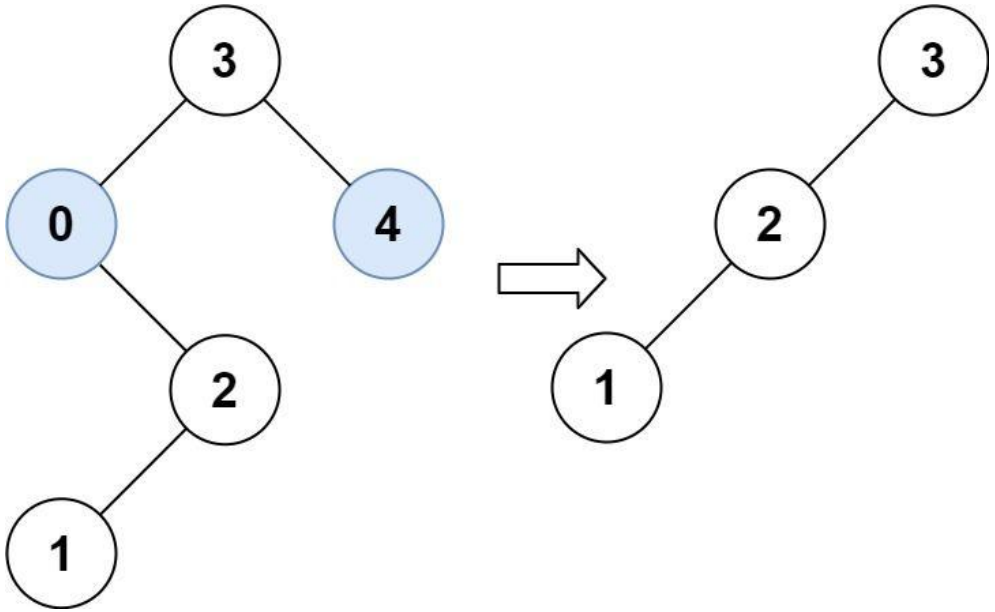
Return the root of the trimmed binary search tree. Note that the root may change depending on the given bounds.

Example 1:




```
Input: root = [1,0,2], low = 1, high = 2
Output: [1,null,2]
```

Example 2:




```
Input: root = [3,0,4,null,2,null,null,1], low = 1, high = 3
Output: [3,2,null,1]
```

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- 0 <= Node.val <= 104
- The value of each node in the tree is unique.
- root is guaranteed to be a valid binary search tree.
- 0 <= low <= high <= 104

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Trim BST so all values are in [low,high]. Preserve relative structure. Right?"
# "root=[3,0,4,null,2,null,null,1],low=1,high=3→[3,2,null,1] ✓"
# Key: if val<low, left subtree is also out — only recurse right. Vice versa. O(n). Space O(h).
# Follow-up: trim so at most k nodes remain — different problem, needs value-based selection.
# Follow-up: trim an AVL tree and maintain balance — re-balance after trimming.
```

BST / Ordered Set

#701 Insert into a Binary Search Tree

ME D

Open on LeetCode

Tree · Binary Search Tree · Binary Tree

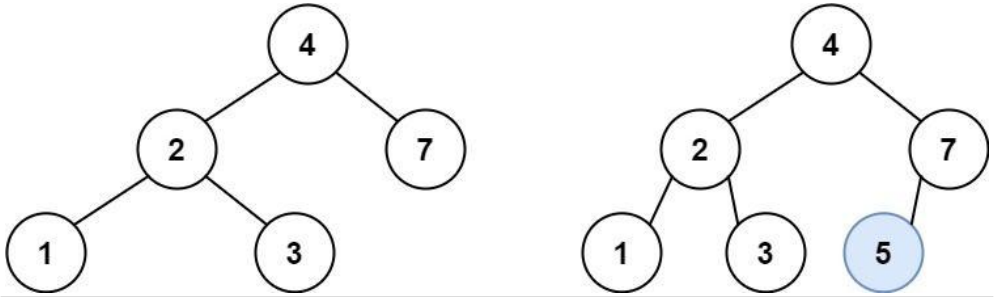
Lists: AlgoMaster 600

Problem

You are given the root node of a binary search tree (BST) and a value to insert into the tree. Return the root node of the BST after the insertion. It is guaranteed that the new value does not exist in the original BST.

Notice that there may exist multiple valid ways for the insertion, as long as the tree remains a BST after insertion. You can return any of them.

Example 1:




```
Input: root = [4,2,7,1,3], val = 5
Output: [4,2,7,1,3,5]
Explanation: Another accepted tree is:
```

Example 2:

```
Input: root = [40,20,60,10,30,50,70], val = 25
Output: [40,20,60,10,30,50,70,null,null,25]
```

Example 3:

```
Input: root = [4,2,7,1,3,null,null,null,null,null,null], val = 5
Output: [4,2,7,1,3,5]
```

Constraints:

- The number of nodes in the tree will be in the range [0, 104].
- -108 <= Node.val <= 108
- All the values Node.val are unique.
- -108 <= val <= 108
- It's guaranteed that val does not exist in the original BST.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Insert val into BST. Return root of modified tree. Right?"
# "root=[4,2,7,1,3],val=5→[4,2,7,1,3,5] ✓"
# Iterative BST insert: O(h) time, O(1) space. Recursive: O(h) time, O(h) space.
# Follow-up: Delete Node in a BST (LC 450) — more complex, needs successor finding.
# Follow-up: if tree might become unbalanced — self-balancing BST (AVL/Red-Black).
```


BST / Ordered Set

#729 My Calendar I

ME D

Open on LeetCode

Array · Binary Search · Design · Segment Tree · Ordered Set

Lists: AlgoMaster 600

Problem

You are implementing a program to use as your calendar. We can add a new event if adding the event will not cause a double booking.

A double booking happens when two events have some non-empty intersection (i.e., some moment is common to both events.).

The event can be represented as a pair of integers startTime and endTime that represents a booking on the half-open interval [startTime, endTime), the range of real numbers x such that startTime <= x < endTime.

Implement the MyCalendar class:

- MyCalendar() Initializes the calendar object.
- boolean book(int startTime, int endTime) Returns true if the event can be added to the calendar successfully without causing a double booking. Otherwise, return false and do not add the event to the calendar.

Example 1:

```
Input
["MyCalendar", "book", "book", "book"]
[[], [10, 20], [15, 25], [20, 30]]
Output
[null, true, false, true]

Explanation
MyCalendar myCalendar = new MyCalendar();
```

Constraints:

- 0 <= start < end <= 109
- At most 1000 calls will be made to book.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Book events [start,end). No double-booking. Return false if overlap. Right?"
# "book(10,20)→T,book(15,25)→F,book(20,30)→T ✓"
# Sorted list: binary search for neighbors, check overlap. O(log n) per booking.
# Follow-up: My Calendar II (LC 731) — allow double but not triple booking.
# Follow-up: My Calendar III (LC 732) — return max concurrent events.
```

BST / Ordered Set

#731 My Calendar II

MED

Open on LeetCode

Array · Binary Search · Design · Segment Tree · Prefix Sum · Ordered Set

Lists: AlgoMaster 600

Problem

You are implementing a program to use as your calendar. We can add a new event if adding the event will not cause a triple booking.

A triple booking happens when three events have some non-empty intersection (i.e., some moment is common to all the three events.).

The event can be represented as a pair of integers startTime and endTime that represents a booking on the half-open interval [startTime, endTime), the range of real numbers x such that startTime <= x < endTime.

Implement the MyCalendarTwo class:

- MyCalendarTwo() Initializes the calendar object.
- boolean book(int startTime, int endTime) Returns true if the event can be added to the calendar successfully without causing a triple booking. Otherwise, return false and do not add the event to the calendar.

Example 1:

```
Input
["MyCalendarTwo", "book", "book", "book", "book", "book", "book"]
[[], [10, 20], [50, 60], [10, 40], [5, 15], [5, 10], [25, 55]]
Output
[null, true, true, true, false, true, true]

Explanation
MyCalendarTwo myCalendarTwo = new MyCalendarTwo();
```

Constraints:

- 0 <= start < end <= 10^9
- At most 1000 calls will be made to book.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Allow booking unless triple-booking occurs. Return false if triple overlap. Right?"
# "book(10,20)→T,book(50,60)→T,book(10,40)→T,book(5,15)→F ✓"
# Difference array: check if any point has 3+ concurrent. O(n log n) per call — or O(n) brute.
# Follow-up: My Calendar III (LC 732) — always book, return max concurrent.
# Follow-up: if we need O(log n) per booking — segment tree with lazy propagation.
```


BST / Ordered Set

#1008 Construct Binary Search Tree from Preorder Traversal

MED

Open on LeetCode

Array · Stack · Tree · Binary Search Tree · Monotonic Stack · Binary Tree

Lists: AlgoMaster 600

Problem

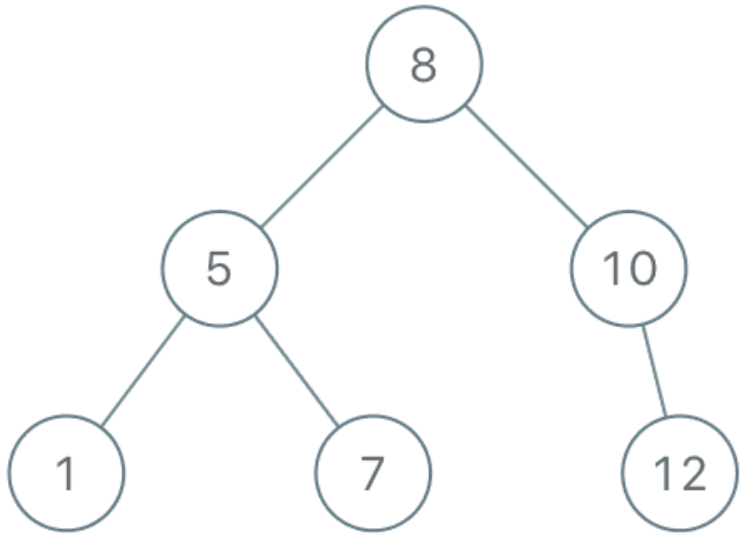
Given an array of integers preorder, which represents the preorder traversal of a BST (i.e., binary search tree), construct the tree and return its root.

It is guaranteed that there is always possible to find a binary search tree with the given requirements for the given test cases.

A binary search tree is a binary tree where for every node, any descendant of Node.left has a value strictly less than Node.val, and any descendant of Node.right has a value strictly greater than Node.val.

A preorder traversal of a binary tree displays the value of the node first, then traverses Node.left, then traverses Node.right.

Example 1:




```
Input: preorder = [8,5,1,7,10,12]
Output: [8,5,10,1,7,null,12]
```

Example 2:

```
Input: preorder = [1,3]
Output: [1,null,3]
```

Constraints:

- 1 <= preorder.length <= 100
- 1 <= preorder[i] <= 1000
- All the values of preorder are unique.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given preorder traversal of a BST, reconstruct the tree. Right?"
# "preorder=[8,5,1,7,10,12]→BST with root=8,left=5(1,7),right=10(null,12) ✓"
# Upper-bound approach: each recursive call knows the max value allowed. O(n). Space O(n).
# Follow-up: reconstruct from inorder alone — impossible. Need preorder+inorder or just preorder for BST.
# Follow-up: serialize/deserialize BST — preorder is sufficient (LC 449).
```

BST / Ordered Set

#2034 Stock Price Fluctuation

MED

Open on LeetCode

Hash Table · Design · Heap (Priority Queue) · Data Stream · Ordered Set

Lists: AlgoMaster 600

Problem

You are given a stream of records about a particular stock. Each record contains a timestamp and the corresponding price of the stock at that timestamp.

Unfortunately due to the volatile nature of the stock market, the records do not come in order. Even worse, some records may be incorrect. Another record with the same timestamp may appear later in the stream correcting the price of the previous wrong record.

Design an algorithm that:

- Updates the price of the stock at a particular timestamp, correcting the price from any previous records at the timestamp.
- Finds the latest price of the stock based on the current records. The latest price is the price at the latest timestamp recorded.
- Finds the maximum price the stock has been based on the current records.
- Finds the minimum price the stock has been based on the current records.

Implement the StockPrice class:

- StockPrice() Initializes the object with no price records.
- void update(int timestamp, int price) Updates the price of the stock at the given timestamp.
- int current() Returns the latest price of the stock.
- int maximum() Returns the maximum price of the stock.
- int minimum() Returns the minimum price of the stock.

Example 1:

```
Input
["StockPrice", "update", "update", "current", "maximum", "update", "maximum", "update", "minimum"]
[[], [1, 10], [2, 5], [], [], [1, 3], [], [4, 2], []]
Output
[null, null, null, 5, 10, null, 5, null, 2]

Explanation
StockPrice stockPrice = new StockPrice();
```

Constraints:

- 1 <= timestamp, price <= 10^9
- At most 10^5 calls will be made in total to update, current, maximum, and minimum.
- current, maximum, and minimum will be called only after update has been called at least once.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Update stock prices by timestamp. Support update(t,price), current()→latest price, max(), min(). Right?"
# "update(1,10),update(2,5),current()→5,max()→10,min()→5,update(1,3),max()→5 ✓"
# SortedList: O(log n) update/min/max. HashMap: O(1) current. Space O(n).
# Follow-up: if timestamps are always increasing — no need for max_timestamp tracking.
# Follow-up: Design Twitter (LC 355) — multi-user feed with timestamps.
```


Tries

Round 5 · Mid · Medium · 6 questions

Tries

#421 Maximum XOR of Two Numbers in an Array

MED

Open on LeetCode

Array · Hash Table · Bit Manipulation · Trie

Lists: AlgoMaster 600

Problem

Given an integer array nums, return the maximum result of nums[i] XOR nums[j], where 0 <= i <= j < n.

Example 1:

```
Input: nums = [3,10,5,25,2,8]
Output: 28
Explanation: The maximum result is 5 XOR 25 = 28.
```

Example 2:

```
Input: nums = [14,70,53,83,49,91,36,80,92,51,66,70]
Output: 127
```

Constraints:

- 1 <= nums.length <= 2 * 10^5
- 0 <= nums[i] <= 2^31 - 1

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the maximum XOR of any two numbers in nums. Right?"
# "nums=[3,10,5,25,2,8]→28 (5 XOR 25) ✓"
        # Trie-based: insert all, for each num find complement path
# Trie: greedily take the opposite bit. O(32n)=O(n). Space O(32n)=O(n).
# Alternative: bit-by-bit O(32n) using prefix sets.
# Follow-up: Maximum XOR With an Element From Array (LC 1707) — offline queries with sorted trie.
```


Tries

#648 Replace Words

MED

Open on LeetCode

Array · Hash Table · String · Trie

Lists: AlgoMaster 600

Problem

In English, we have a concept called root, which can be followed by some other word to form another longer word - let's call this word derivative. For example, when the root "help" is followed by the word "ful", we can form a derivative "helpful".

Given a dictionary consisting of many roots and a sentence consisting of words separated by spaces, replace all the derivatives in the sentence with the root forming it. If a derivative can be replaced by more than one root, replace it with the root that has the shortest length.

Return the sentence after the replacement.

Example 1:

```
Input: dictionary = ["cat","bat","rat"], sentence = "the cattle was rattled by the battery"
Output: "the cat was rat by the bat"
```

Example 2:

```
Input: dictionary = ["a","b","c"], sentence = "aadsfasf absbs bbab cadsfafs"
Output: "a a b c"
```

Constraints:

- 1 <= dictionary.length <= 1000
- 1 <= dictionary[i].length <= 100
- dictionary[i] consists of only lower-case letters.
- 1 <= sentence.length <= 106
- sentence consists of only lower-case letters and spaces.
- The number of words in sentence is in the range [1, 1000]
- The length of each word in sentence is in the range [1, 1000]
- Every two consecutive words in sentence will be separated by exactly one space.
- sentence does not have leading or trailing spaces.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given dictionary of roots, replace each word in sentence with shortest root prefix. Right?"
# "dictionary=['cat','bat','rat'],sentence='the cattle was rattled by the battery':
# →'the cat was rat by the bat' ✓"
# Trie: insert all roots, query each word. O(sum_root_len + n*avg_word_len). Space O(sum_root_len).
# Follow-up: if roots can be prefixes of each other — trie naturally handles (shortest match wins).
# Follow-up: Longest Word in Dictionary (LC 720) — similar trie traversal.
```

Tries

#1233 Remove Sub-Folders from the Filesystem

ME D

Open on LeetCode

Array · String · Depth-First Search · Trie

Lists: AlgoMaster 600

Problem

Given a list of folders folder, return the folders after removing all sub-folders in those folders. You may return the answer in any order.

If a folder[i] is located within another folder[j], it is called a sub-folder of it. A sub-folder of folder[j] must start with folder[j], followed by a "/". For example, "/a/b" is a sub-folder of "/a", but "/b" is not a sub-folder of "/a/b/c".

The format of a path is one or more concatenated strings of the form: '/' followed by one or more lowercase English letters.

- For example, "/leetcode" and "/leetcode/problems" are valid paths while an empty string and "/" are not.

Example 1:

```
Input: folder = ["/a","/a/b","/c/d","/c/d/e","/c/f"]
Output: ["/a","/c/d","/c/f"]
Explanation: Folders "/a/b" is a subfolder of "/a" and "/c/d/e" is inside of folder "/c/d" in our filesystem.
```

Example 2:

```
Input: folder = ["/a","/a/b/c","/a/b/d"]
Output: ["/a"]
Explanation: Folders "/a/b/c" and "/a/b/d" will be removed because they are subfolders of "/a".
```

Example 3:

```
Input: folder = ["/a/b/c","/a/b/ca","/a/b/d"]
Output: ["/a/b/c","/a/b/ca","/a/b/d"]
```

Constraints:

- 1 <= folder.length <= 4 * 104
- 2 <= folder[i].length <= 100
- folder[i] contains only lowercase letters and '/'.
- folder[i] always starts with the character '/'.
- Each folder name is unique.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Remove all sub-folders from the list. /a/b is sub-folder of /a. Right?"
# "folder=['/a','/a/b','/c/d','/c/d/e','/c/f']→['/a','/c/d','/c/f'] ✓"
# Sort lexicographically. A sub-folder always comes after its parent. Check prefix. O(n log n * L).
# Follow-up: with trie — insert all folders, mark terminals, skip subtrees. O(total chars).
# Follow-up: if query asks "is X a subfolder of any in the set?" — trie lookup O(L).
```


Tries

#1268 Search Suggestions System

ME D

Open on LeetCode

Array · String · Binary Search · Trie · Sorting · Heap (Priority Queue)

Lists: AlgoMaster 600

Problem

You are given an array of strings products and a string searchWord.

Design a system that suggests at most three product names from products after each character of searchWord is typed. Suggested products should have common prefix with searchWord. If there are more than three products with a common prefix return the three lexicographically minimums products.

Return a list of lists of the suggested products after each character of searchWord is typed.

Example 1:

```
Input: products = ["mobile","mouse","moneypot","monitor","mousepad"], searchWord = "mouse"
Output: [["mobile","moneypot","monitor"],["mobile","moneypot","monitor"],["mouse","mousepad"],["mouse","mousepad"],["mouse","mousepad"]]
Explanation: products sorted lexicographically = ["mobile","moneypot","monitor","mouse","mousepad"].
After typing m and mo all products match and we show user ["mobile","moneypot","monitor"].
After typing mou, mous and mouse the system suggests ["mouse","mousepad"].
```

Example 2:

```
Input: products = ["havana"], searchWord = "havana"
Output: [["havana"],["havana"],["havana"],["havana"],["havana"],["havana"]]
Explanation: The only word "havana" will be always suggested while typing the search word.
```

Constraints:

- 1 <= products.length <= 1000
- 1 <= products[i].length <= 3000
- 1 <= sum(products[i].length) <= 2 * 104
- All the strings of products are unique.
- products[i] consists of lowercase English letters.
- 1 <= searchWord.length <= 1000
- searchWord consists of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "As each character of searchWord is typed, return top 3 lexicographically smallest products with that prefix. Right?"
# "products=['mobile','mouse','moneypot','monitor','mousepad'],searchWord='mouse':
# →[['mobile','moneypot','monitor'],['mobile','moneypot','monitor'],
# ['mouse','mousepad'],['mouse','mousepad'],['mouse','mousepad']] ✓"
# Binary search to find prefix range in sorted products. O(n log n + m log n) where m=search len.
# Follow-up: if products update dynamically — use a Trie with sorted lists at each node.
# Follow-up: return top 3 by frequency (not lex) — priority queue at each trie node.
```

Tries

#2452 Words Within Two Edits of Dictionary

MED

Open on LeetCode

Array · String · Trie

Lists: AlgoMaster 600

Problem

You are given two string arrays, queries and dictionary. All words in each array comprise of lowercase English letters and have the same length.

In one edit you can take a word from queries, and change any letter in it to any other letter. Find all words from queries that, after a maximum of two edits, equal some word from dictionary.

Return a list of all words from queries, that match with some word from dictionary after a maximum of two edits. Return the words in the same order they appear in queries.

Example 1:

```
Input: queries = ["word","note","ants","wood"], dictionary = ["wood","joke","moat"]
Output: ["word","note","wood"]
Explanation:
- Changing the 'r' in "word" to 'o' allows it to equal the dictionary word "wood".
- Changing the 'n' to 'j' and the 't' to 'k' in "note" changes it to "joke".
- It would take more than 2 edits for "ants" to equal a dictionary word.
- "wood" can remain unchanged (0 edits) and match the corresponding dictionary word.
Thus, we return ["word","note","wood"].
```

Example 2:

```
Input: queries = ["yes"], dictionary = ["not"]
Output: []
Explanation:
Applying any two edits to "yes" cannot make it equal to "not". Thus, we return an empty array.
```

Constraints:

- 1 <= queries.length, dictionary.length <= 100
- n == queries[i].length == dictionary[j].length
- 1 <= n <= 100
- All queries[i] and dictionary[j] are composed of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return words from queries that differ from any dict word by at most 2 characters (same length). Right?"
# "queries=['word','note','ants','wood'],dictionary=['wood','joke','moat']→['word','note','wood'] ✓"
# O(|queries|∗|dictionary|∗L). For large inputs: trie with wildcard matching.
# Follow-up: allow insertions/deletions (edit distance ≤ 2) — DP or BK-tree.
# Follow-up: if called frequently — build trie of dictionary, match with ≤2 wildcards.
```


Tries

#3043 Find the Length of the Longest Common Prefix

MED

Open on LeetCode

Array · Hash Table · String · Trie

Lists: AlgoMaster 600

Problem

You are given two arrays with positive integers arr1 and arr2.

A prefix of a positive integer is an integer formed by one or more of its digits, starting from its leftmost digit. For example, 123 is a prefix of the integer 12345, while 234 is not.

A common prefix of two integers a and b is an integer c, such that c is a prefix of both a and b. For example, 5655359 and 56554 have common prefixes 565 and 5655 while 1223 and 43456 do not have a common prefix.

You need to find the length of the longest common prefix between all pairs of integers (x, y) such that x belongs to arr1 and y belongs to arr2.

Return the length of the longest common prefix among all pairs. If no common prefix exists among them, return 0.

Example 1:

```
Input: arr1 = [1,10,100], arr2 = [1000]
Output: 3
Explanation: There are 3 pairs (arr1[i], arr2[j]):
- The longest common prefix of (1, 1000) is 1.
- The longest common prefix of (10, 1000) is 10.
- The longest common prefix of (100, 1000) is 100.
The longest common prefix is 100 with a length of 3.
```

Example 2:

```
Input: arr1 = [1,2,3], arr2 = [4,4,4]
Output: 0
Explanation: There exists no common prefix for any pair (arr1[i], arr2[j]), hence we return 0.
Note that common prefixes between elements of the same array do not count.
```

Constraints:

- 1 <= arr1.length, arr2.length <= 5 * 104
- 1 <= arr1[i], arr2[i] <= 108

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find max length common prefix between any element in arr1 (as string) and arr2. Right?"
# "arr1=[1,10,100],arr2=[1000]:
# '1' and '1000' share prefix '1'(len1). '10'&'1000' share '10'(len2). '100'&'1000' share '100'(len3).→3 ✓"
# Insert arr1 strings into trie, query arr2. O((|arr1|+|arr2|)∗D) where D=max digits. Space O(|arr1|∗D).
# Follow-up: if arrays are large — trie is essential vs O(n²) brute.
# Follow-up: Longest Common Prefix among all strings (LC 14) — sort and compare first/last.
```

Heaps

Round 5 · Mid · Medium · 4 questions

Heaps

#1405 Longest Happy String

MED

Open on LeetCode

String · Greedy · Heap (Priority Queue)

Lists: AlgoMaster 600

Problem

A string s is called happy if it satisfies the following conditions:

- **s only contains the letters 'a', 'b', and 'c'.**
- **s does not contain any of "aaa", "bbb", or "ccc" as a substring.**
- **s contains at most a occurrences of the letter 'a'.**
- **s contains at most b occurrences of the letter 'b'.**
- **s contains at most c occurrences of the letter 'c'.**

Given three integers a, b, and c, return the longest possible happy string. If there are multiple longest happy strings, return any of them. If there is no such string, return the empty string "".

A substring is a contiguous sequence of characters within a string.

Example 1:

```
Input: a = 1, b = 1, c = 7
Output: "ccaccbcc"
Explanation: "ccbccacc" would also be a correct answer.
```

Example 2:

```
Input: a = 7, b = 1, c = 0
Output: "aabaa"
Explanation: It is the only correct answer in this case.
```

Constraints:

- **0 <= a, b, c <= 100**
- **a + b + c > 0**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Build longest string using at most a 'a's, b 'b's, c 'c's with no 3 consecutive same chars. Right?"
# "a=1,b=1,c=7→'ccaccbcc' or similar, len=8 ✓"
# Greedy: always pick most frequent char, unless it would create 3-in-a-row. O(n log 3)=O(n).
# Follow-up: Reorganize String (LC 767) — binary alphabet (any/none), same greedy.
# Follow-up: Task Scheduler (LC 621) — count result length without constructing.
```

Heaps

#1642 Furthest Building You Can Reach

MED

Open on LeetCode

Array · Greedy · Heap (Priority Queue)

Lists: AlgoMaster 600

Problem

You are given an integer array heights representing the heights of buildings, some bricks, and some ladders.

You start your journey from building 0 and move to the next building by possibly using bricks or ladders.

While moving from building i to building i+1 (0-indexed),

- If the current building's height is greater than or equal to the next building's height, you do not need a ladder or bricks.
- If the current building's height is less than the next building's height, you can either use one ladder or (h[i+1] - h[i]) bricks.

Return the furthest building index (0-indexed) you can reach if you use the given ladders and bricks optimally.

Example 1:

```
Input: heights = [4,2,7,6,9,14,12], bricks = 5, ladders = 1
Output: 4
Explanation: Starting at building 0, you can follow these steps:
- Go to building 1 without using ladders nor bricks since 4 >= 2.
- Go to building 2 using 5 bricks. You must use either bricks or ladders because 2 < 7.
- Go to building 3 without using ladders nor bricks since 7 >= 6.
- Go to building 4 using your only ladder. You must use either bricks or ladders because 6 < 9.
It is impossible to go beyond building 4 because you do not have any more bricks or ladders.
```

Example 2:

```
Input: heights = [4,12,2,7,3,18,20,3,19], bricks = 10, ladders = 2
Output: 7
```

Example 3:


```
Input: heights = [14,3,19,3], bricks = 17, ladders = 0
Output: 3
```

Constraints:

- 1 <= heights.length <= 105
- 1 <= heights[i] <= 106
- 0 <= bricks <= 109
- 0 <= ladders <= heights.length

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Given heights, ladders, bricks: climb up requires bricks(diff) or one ladder (free).
# Use ladders for biggest climbs. Return furthest building reachable. Right?"
# "heights=[4,2,7,6,9,14,12],ladders=1,bricks=5→4 ✓"
# Greedily assign ladders to largest climbs (min-heap swaps smallest ladder-climb to bricks). O(n log L).
# Follow-up: if we can also use ropes of varying lengths — extend the heap with rope constraints.
# Follow-up: return the path taken (which resource used where) — track assignments.
```

Heaps

#1834 Single-Threaded CPU

MED

Open on LeetCode

Array · Sorting · Heap (Priority Queue)

Lists: AlgoMaster 600

Problem

You are given n tasks labeled from 0 to n - 1 represented by a 2D integer array tasks, where tasks[i] = [enqueueTimei, processingTimei] means that the ith task will be available to process at enqueueTimei and will take processingTimei to finish processing.

You have a single-threaded CPU that can process at most one task at a time and will act in the following way:

- If the CPU is idle and there are no available tasks to process, the CPU remains idle.
- If the CPU is idle and there are available tasks, the CPU will choose the one with the shortest processing time. If multiple tasks have the same shortest processing time, it will choose the task with the smallest index.
- Once a task is started, the CPU will process the entire task without stopping.
- The CPU can finish a task then start a new one instantly.

Return the order in which the CPU will process the tasks.

Example 1:

```
Input: tasks = [[1,2],[2,4],[3,2],[4,1]]
Output: [0,2,3,1]
Explanation: The events go as follows:
- At time = 1, task 0 is available to process. Available tasks = {0}.
- Also at time = 1, the idle CPU starts processing task 0. Available tasks = {}.
- At time = 2, task 1 is available to process. Available tasks = {1}.
- At time = 3, task 2 is available to process. Available tasks = {1, 2}.
- Also at time = 3, the CPU finishes task 0 and starts processing task 2 as it is the shortest. Available tasks = {1}.
```

Example 2:

```
Input: tasks = [[7,10],[7,12],[7,5],[7,4],[7,2]]
Output: [4,3,2,0,1]
Explanation: The events go as follows:
- At time = 7, all the tasks become available. Available tasks = {0,1,2,3,4}.
- Also at time = 7, the idle CPU starts processing task 4. Available tasks = {0,1,2,3}.
- At time = 9, the CPU finishes task 4 and starts processing task 3. Available tasks = {0,1,2}.
- At time = 13, the CPU finishes task 3 and starts processing task 2. Available tasks = {0,1}.
- At time = 18, the CPU finishes task 2 and starts processing task 0. Available tasks = {1}.
```

Constraints:

- tasks.length == n
- 1 <= n <= 105
- 1 <= enqueueTimei, processingTimei <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Tasks with enqueue time and processing time. CPU picks shortest available task. Return order. Right?"
# "tasks=[[1,2],[2,4],[3,2],[4,1]]→[0,2,3,1] ✓"
# Min-heap by processing time. Sort tasks by enqueue time. Simulate CPU. O(n log n). Space O(n).
# Follow-up: multi-threaded CPU — maintain a pool of available CPUs, assign shortest to soonest-free.
# Follow-up: if tasks have deadlines — priority queue with deadline constraint.
```


Heaps

#1882 Process Tasks Using Servers

MED

Open on LeetCode

Array · Heap (Priority Queue)

Lists: AlgoMaster 600

Problem

You are given two 0-indexed integer arrays servers and tasks of lengths n and m respectively. servers[i] is the weight of the ith server, and tasks[j] is the time needed to process the jth task in seconds.

Tasks are assigned to the servers using a task queue. Initially, all servers are free, and the queue is empty.

At second j, the jth task is inserted into the queue (starting with the 0th task being inserted at second 0). As long as there are free servers and the queue is not empty, the task in the front of the queue will be assigned to a free server with the smallest weight, and in case of a tie, it is assigned to a free server with the smallest index.

If there are no free servers and the queue is not empty, we wait until a server becomes free and immediately assign the next task. If multiple servers become free at the same time, then multiple tasks from the queue will be assigned in order of insertion following the weight and index priorities above.

A server that is assigned task j at second t will be free again at second t + tasks[j].

Build an array ans of length m, where ans[j] is the index of the server the jth task will be assigned to.

Return the array ans.

Example 1:

```
Input: servers = [3,3,2], tasks = [1,2,3,2,1,2]
Output: [2,2,0,2,1,2]
Explanation: Events in chronological order go as follows:
- At second 0, task 0 is added and processed using server 2 until second 1.
- At second 1, server 2 becomes free. Task 1 is added and processed using server 2 until second 3.
- At second 2, task 2 is added and processed using server 0 until second 5.
- At second 3, server 2 becomes free. Task 3 is added and processed using server 2 until second 5.
- At second 4, task 4 is added and processed using server 1 until second 5.
```

Example 2:

```
Input: servers = [5,1,4,3,2], tasks = [2,1,2,4,5,2,1]
Output: [1,4,1,4,1,3,2]
Explanation: Events in chronological order go as follows:
- At second 0, task 0 is added and processed using server 1 until second 2.
- At second 1, task 1 is added and processed using server 4 until second 2.
- At second 2, servers 1 and 4 become free. Task 2 is added and processed using server 1 until second 4.
- At second 3, task 3 is added and processed using server 4 until second 7.
- At second 4, server 1 becomes free. Task 4 is added and processed using server 1 until second 9.
```

Constraints:

- servers.length == n
- tasks.length == m
- 1 <= n, m <= 2 * 105
- 1 <= servers[i], tasks[j] <= 2 * 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "servers[i]=weight of server i. tasks[j]=time task j takes. Assign tasks greedily to free server
# with smallest weight. Return server assigned to each task. Right?"
# "servers=[3,3,2],tasks=[1,2,3,2,1,2]→[2,2,0,2,1,2] ✓"
# Two heaps: free (sorted by weight) and busy (sorted by finish time). O(n log n + m log n). Space O(n).
# Follow-up: if servers have different speeds — adjust duration = task_time/server_speed.
# Follow-up: Single-Threaded CPU (LC 1834) — same pattern with one server.
```

Intervals

Round 5 · Mid · Medium · 6 questions

Intervals

#452 Minimum Number of Arrows to Burst Balloons

MED

Open on LeetCode

Array · Greedy · Sorting

Lists: AlgoMaster 600

Problem

There are some spherical balloons taped onto a flat wall that represents the XY-plane. The balloons are represented as a 2D integer array points where points[i] = [xstart, xend] denotes a balloon whose horizontal diameter stretches between xstart and xend. You do not know the exact y-coordinates of the balloons.

Arrows can be shot up directly vertically (in the positive y-direction) from different points along the x-axis. A balloon with xstart and xend is burst by an arrow shot at x if xstart <= x <= xend. There is no limit to the number of arrows that can be shot. A shot arrow keeps traveling up infinitely, bursting any balloons in its path.

Given the array points, return the minimum number of arrows that must be shot to burst all balloons.

Example 1:

```
Input: points = [[10,16],[2,8],[1,6],[7,12]]
Output: 2
Explanation: The balloons can be burst by 2 arrows:
- Shoot an arrow at x = 6, bursting the balloons [2,8] and [1,6].
- Shoot an arrow at x = 11, bursting the balloons [10,16] and [7,12].
```

Example 2:

```
Input: points = [[1,2],[3,4],[5,6],[7,8]]
Output: 4
Explanation: One arrow needs to be shot for each balloon for a total of 4 arrows.
```

Example 3:

```
Input: points = [[1,2],[2,3],[3,4],[4,5]]
Output: 2
Explanation: The balloons can be burst by 2 arrows:
- Shoot an arrow at x = 2, bursting the balloons [1,2] and [2,3].
- Shoot an arrow at x = 4, bursting the balloons [3,4] and [4,5].
```

Constraints:

- 1 <= points.length <= 105
- points[i].length == 2
- -231 <= xstart < xend <= 231 - 1

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Balloons as intervals on x-axis. Arrow shot at x bursts all balloons spanning x. Min arrows? Right?"
# "points=[[10,16],[2,8],[1,6],[7,12]]→2 ✓"
# Sort by end time. Each arrow at current end bursts all overlapping. O(n log n). Space O(1).
# Follow-up: Non-overlapping Intervals (LC 435) — identical greedy, count removals.
# Follow-up: if arrows have limited range — greedy still works, adjust overlap condition.
```

Intervals

#1094 Car Pooling

MED

Open on LeetCode

Array · Sorting · Heap (Priority Queue) · Simulation · Prefix Sum

Lists: AlgoMaster 600

Problem

There is a car with capacity empty seats. The vehicle only drives east (i.e., it cannot turn around and drive west).
You are given the integer capacity and an array trips where trips[i] = [numPassengersi, fromi, toi] indicates that the ith trip has numPassengersi passengers and the locations to pick them up and drop them off are fromi and toi respectively. The locations are given as the number of kilometers due east from the car's initial location.
Return true if it is possible to pick up and drop off all passengers for all the given trips, or false otherwise.

Example 1:

```
Input: trips = [[2,1,5],[3,3,7]], capacity = 4
Output: false
```

Example 2:

```
Input: trips = [[2,1,5],[3,3,7]], capacity = 5
Output: true
```

Constraints:

- 1 <= trips.length <= 1000
- trips[i].length == 3
- 1 <= numPassengersi <= 100
- 0 <= fromi < toi <= 1000
- 1 <= capacity <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "trips[i]=[passengers,from,to]. Car capacity. Can we pick up all passengers? Right?"
# "trips=[[2,1,5],[3,3,7]],capacity=4→false (3+3=6>4 at station 3) ✓"
# Difference array (or event list). O(n log n). Space O(n).
# Follow-up: return the maximum passengers at any point — track running max.
# Follow-up: if stops are arbitrary large integers — event-list approach is better.
```


Intervals

#1229 Meeting Scheduler

MED

Open on LeetCode

Array · Two Pointers · Sorting

Lists: AlgoMaster 600

Problem

Given the availability time slots arrays slots1 and slots2 of two people and a meeting duration duration, return the earliest time slot that works for both of them and is of duration duration.
If there is no common time slot that satisfies the requirements, return an empty array.
The format of a time slot is an array of two elements [start, end] representing an inclusive time range from start to end.
It is guaranteed that no two availability slots of the same person intersect with each other. That is, for any two time slots [start1, end1] and [start2, end2] of the same person, either start1 > end2 or start2 > end1.

Example 1:

```
Input: slots1 = [[10,50],[60,120],[140,210]], slots2 = [[0,15],[60,70]], duration = 8
Output: [60,68]
```

Example 2:

```
Input: slots1 = [[10,50],[60,120],[140,210]], slots2 = [[0,15],[60,70]], duration = 12
Output: []
```

Constraints:

- 1 <= slots1.length, slots2.length <= 104
- slots1[i].length, slots2[i].length == 2
- slots1[i][0] < slots1[i][1]
- slots2[i][0] < slots2[i][1]
- 0 <= slots1[i][j], slots2[i][j] <= 109
- 1 <= duration <= 106

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find earliest common free slot of duration for two people from their available slots. Right?"
# "slots1=[[10,50],[60,120],[140,210]],slots2=[[0,15],[60,70]],duration=8→[60,68] ✓"
# Two-pointer on sorted intervals. O(n log n). Space O(1).
# Follow-up: k people's schedules — k-way merge with min-heap.
# Follow-up: if duration can be fractional — same algorithm, check with float comparison.
```

Intervals

#1288 Remove Covered Intervals

MED

Open on LeetCode

Array · Sorting

Lists: AlgoMaster 600

Problem

Given an array intervals where intervals[i] = [li, ri] represent the interval [li, ri), remove all intervals that are covered by another interval in the list.

The interval [a, b) is covered by the interval [c, d) if and only if c <= a and b <= d.

Return the number of remaining intervals.

Example 1:

```
Input: intervals = [[1,4],[3,6],[2,8]]
Output: 2
Explanation: Interval [3,6] is covered by [2,8], therefore it is removed.
```

Example 2:

```
Input: intervals = [[1,4],[2,3]]
Output: 1
```

Constraints:

- 1 <= intervals.length <= 1000
- intervals[i].length == 2
- 0 <= li < ri <= 105
- All the given intervals are unique.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Remove all intervals covered by another in the list. Return count remaining. Right?"
# "intervals=[[1,4],[3,6],[2,8]]→2 ([1,4] covered by [2,8]? No: 2>1. [3,6] covered by [2,8]? 2<=3 and 8>=6 yes.) → 2 ✓"
# Sort by start (ties: descending end). An interval is covered if max_end already >= its end. O(n log n).
# Follow-up: merge overlapping intervals (LC 56) — adjacent intervals that touch/overlap.
# Follow-up: count covered intervals — n − removeCoveredIntervals result.
```


Intervals

#1353 Maximum Number of Events That Can Be Attended

MED

Open on LeetCode

Array · Greedy · Sorting · Heap (Priority Queue)

Lists: AlgoMaster 600

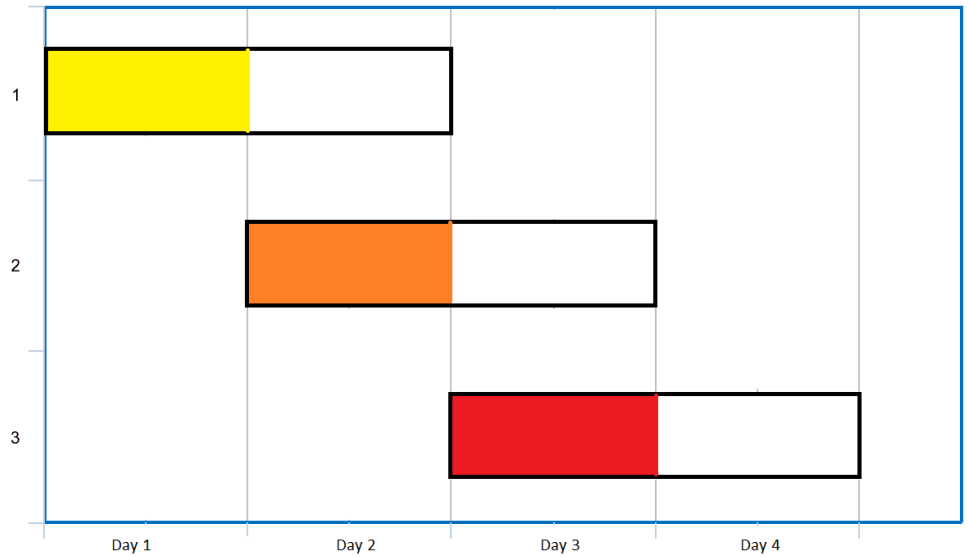
Problem

You are given an array of events where events[i] = [startDayi, endDayi]. Every event i starts at startDayi and ends at endDayi.

You can attend an event i at any day d where startDayi <= d <= endDayi. You can only attend one event at any time d.

Return the maximum number of events you can attend.

Example 1:




```
Input: events = [[1,2],[2,3],[3,4]]
Output: 3
Explanation: You can attend all the three events.
One way to attend them all is as shown.
Attend the first event on day 1.
Attend the second event on day 2.
Attend the third event on day 3.
```

Example 2:

```
Input: events= [[1,2],[2,3],[3,4],[1,2]]
Output: 4
```

Constraints:

- 1 <= events.length <= 105
- events[i].length == 2
- 1 <= startDayi <= endDayi <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Each event has [start,end]. Attend one event per day. Maximize events attended. Right?"
# "events=[[1,2],[2,3],[3,4],[1,2]]→4 ✓"
# Greedy: each day, attend event ending soonest. Min-heap by end day. O(n log n). Space O(n).
```

```
# Follow-up: attend at most k events (LC 1751) — DP + priority queue.
# Follow-up: if events have values — Two Best Non-Overlapping Events (LC 2054).
```


Intervals

#2054 Two Best Non-Overlapping Events

MED

Open on LeetCode

Array · Binary Search · Dynamic Programming · Sorting · Heap (Priority Queue)

Lists: AlgoMaster 600

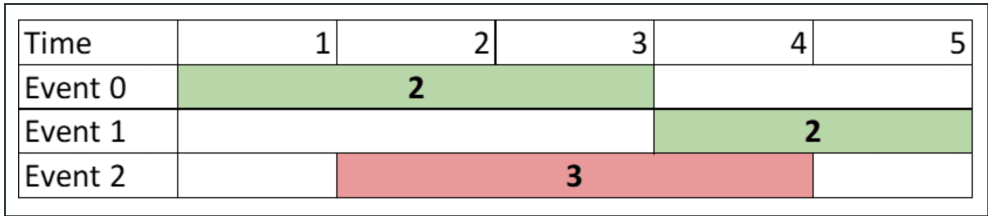
Problem

You are given a 0-indexed 2D integer array of events where events[i] = [startTimei, endTimei, valuei]. The ith event starts at startTimei and ends at endTimei, and if you attend this event, you will receive a value of valuei. You can choose at most two non-overlapping events to attend such that the sum of their values is maximized.

Return this maximum sum.

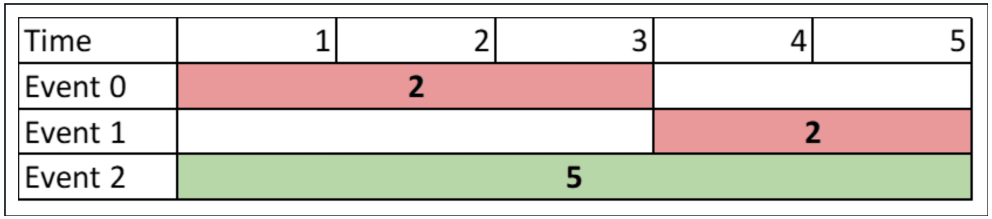
Note that the start time and end time is inclusive: that is, you cannot attend two events where one of them starts and the other ends at the same time. More specifically, if you attend an event with end time t, the next event must start at or after t + 1.

Example 1:



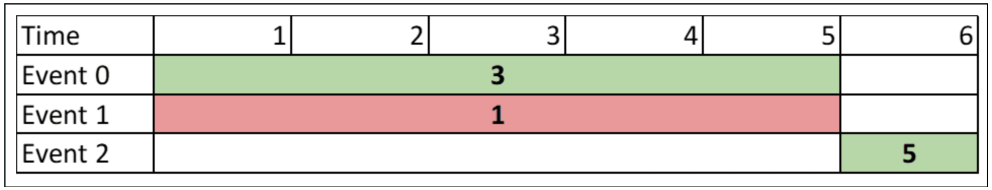

```
Input: events = [[1,3,2],[4,5,2],[2,4,3]]
Output: 4
Explanation: Choose the green events, 0 and 1 for a sum of 2 + 2 = 4.
```

Example 2:




```
Input: events = [[1,3,2],[4,5,2],[1,5,5]]
Output: 5
Explanation: Choose event 2 for a sum of 5.
```

Example 3:




```
Input: events = [[1,5,3],[1,5,1],[6,6,5]]
Output: 8
Explanation: Choose events 0 and 2 for a sum of 3 + 5 = 8.
```

Constraints:

- **2 <= events.length <= 105**
- **events[i].length == 3**
- **1 <= startTimei <= endTimei <= 109**
- **1 <= valuei <= 106**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Pick at most 2 non-overlapping events to maximize value sum. Right?"
# "events=[[1,3,2],[4,5,2],[2,4,3]]→4 (events 0+1: non-overlapping, 2+2=4) ✓"
        # Prefix max value of events ending before current start
# Sort by end time, precompute prefix max values. Binary search for non-overlapping. O(n log n). Space O(n).
# Follow-up: k non-overlapping events (generalization) — DP + binary search O(nk log n).
# Follow-up: Maximum Number of Events (LC 1353) — one event per day, different constraint.
```


K-Way Merge

Round 5 · Mid · Medium · 2 questions

K-Way Merge

#373 Find K Pairs with Smallest Sums

MED

Open on LeetCode

Array · Heap (Priority Queue)

Lists: AlgoMaster 600

Problem

You are given two integer arrays nums1 and nums2 sorted in non-decreasing order and an integer k.
Define a pair (u, v) which consists of one element from the first array and one element from the second array.
Return the k pairs (u1, v1), (u2, v2), ..., (uk, vk) with the smallest sums.

Example 1:

```
Input: nums1 = [1,7,11], nums2 = [2,4,6], k = 3
Output: [[1,2],[1,4],[1,6]]
Explanation: The first 3 pairs are returned from the sequence: [1,2],[1,4],[1,6],[7,2],[7,4],[11,2],[7,6],[11,4],[11,6]
```

Example 2:

```
Input: nums1 = [1,1,2], nums2 = [1,2,3], k = 2
Output: [[1,1],[1,1]]
Explanation: The first 2 pairs are returned from the sequence: [1,1],[1,1],[1,2],[2,1],[1,2],[2,2],[1,3],[1,3],[2,3]
```

Constraints:

- 1 <= nums1.length, nums2.length <= 10^5
- -10^9 <= nums1[i], nums2[i] <= 10^9
- nums1 and nums2 both are sorted in non-decreasing order.
- 1 <= k <= 10^4
- k <= nums1.length * nums2.length

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find k pairs (u,v) with smallest sums from two sorted arrays. Right?"
# "nums1=[1,7,11],nums2=[2,4,6],k=3→[[1,2],[1,4],[1,6]] ✓"
# Min-heap: (sum,i,j). Expand j for each i. O(k log k). Space O(k).
# Follow-up: Kth Smallest Element in a Sorted Matrix (LC 378) — same K-way merge.
# Follow-up: if both arrays are unsorted — sort first O(n log n), then apply this.
```


K-Way Merge

#378 Kth Smallest Element in a Sorted Matrix

MED

Open on LeetCode

Array · Binary Search · Sorting · Heap (Priority Queue) · Matrix

Lists: AlgoMaster 600

Problem

Given an n x n matrix where each of the rows and columns is sorted in ascending order, return the kth smallest element in the matrix.
Note that it is the kth smallest element in the sorted order, not the kth distinct element.
You must find a solution with a memory complexity better than O(n2).

Example 1:

```
Input: matrix = [[1,5,9],[10,11,13],[12,13,15]], k = 8
Output: 13
Explanation: The elements in the matrix are [1,5,9,10,11,12,13,13,15], and the 8th smallest number is 13
```

Example 2:

```
Input: matrix = [[-5]], k = 1
Output: -5
```

Constraints:

- n == matrix.length == matrix[i].length
- 1 <= n <= 300
- -10^9 <= matrix[i][j] <= 10^9
- All the rows and columns of matrix are guaranteed to be sorted in non-decreasing order.
- 1 <= k <= n2

Follow up:

- Could you solve the problem with a constant memory (i.e., O(1) memory complexity)?
- Could you solve the problem in O(n) time complexity? The solution may be too advanced for an interview but you may find reading this paper fun.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "n×n matrix, each row and column sorted. Find kth smallest. Right?"
# "matrix=[[1,5,9],[10,11,13],[12,13,15]],k=8→13 ✓"
# Binary search on value: count elements <= mid in O(n). O(n log(max-min)). Space O(1).
# Follow-up: Find K Pairs with Smallest Sums (LC 373) — same K-way merge heap approach.
# Follow-up: if matrix is not sorted — flatten and use QuickSelect O(n²).
```

Data Structure Design

Round 5 · Mid · Medium · 4 questions

Data Structure Design

#641 Design Circular Deque

MED

Open on LeetCode

Array · Linked List · Design · Queue

Lists: AlgoMaster 600

Problem

Design your implementation of the circular double-ended queue (deque).

Implement the MyCircularDeque class:

- MyCircularDeque(int k) Initializes the deque with a maximum size of k.
- boolean insertFront() Adds an item at the front of Deque. Returns true if the operation is successful, or false otherwise.
- boolean insertLast() Adds an item at the rear of Deque. Returns true if the operation is successful, or false otherwise.
- boolean deleteFront() Deletes an item from the front of Deque. Returns true if the operation is successful, or false otherwise.
- boolean deleteLast() Deletes an item from the rear of Deque. Returns true if the operation is successful, or false otherwise.
- int getFront() Returns the front item from the Deque. Returns -1 if the deque is empty.
- int getRear() Returns the last item from Deque. Returns -1 if the deque is empty.
- boolean isEmpty() Returns true if the deque is empty, or false otherwise.
- boolean isFull() Returns true if the deque is full, or false otherwise.

Example 1:

```
Input
["MyCircularDeque", "insertLast", "insertLast", "insertFront", "insertFront", "getRear", "isFull", "deleteLast", "insertFront", "getFront"]
[[3], [1], [2], [3], [4], [], [], [], [4], []]
Output
[null, true, true, true, false, 2, true, true, true, 4]

Explanation
MyCircularDeque myCircularDeque = new MyCircularDeque(3);
```

Constraints:

- 1 <= k <= 1000
- 0 <= value <= 1000
- At most 2000 calls will be made to insertFront, insertLast, deleteFront, deleteLast, getFront, getRear, isEmpty, isFull.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Design circular deque: insertFront,insertLast,deleteFront,deleteLast,getFront,getRear,isEmpty,isFull. Right?"
# "MyCircularDeque(3): insertLast(1)→T,insertLast(2)→T,insertFront(3)→T,getFront()→3,isFull()→T ✓"
# Circular array with front pointer and size. All ops O(1). Space O(k).
# Follow-up: Design Circular Queue (LC 622) — simpler, only front/rear operations.
# Follow-up: thread-safe deque — wrap all methods with a mutex.
```

Data Structure Design

#1146 Snapshot Array

MED

Open on LeetCode

Array · Hash Table · Binary Search · Design

Lists: AlgoMaster 600

Problem

Implement a SnapshotArray that supports the following interface:

- SnapshotArray(int length) initializes an array-like data structure with the given length. Initially, each element equals 0.
- void set(index, val) sets the element at the given index to be equal to val.
- int snap() takes a snapshot of the array and returns the snap_id: the total number of times we called snap() minus 1.
- int get(index, snap_id) returns the value at the given index, at the time we took the snapshot with the given snap_id

Example 1:

```
Input: ["SnapshotArray","set","snap","set","get"]
[[3],[0,5],[],[0,6],[0,0]]
Output: [null,null,0,null,5]
Explanation:
SnapshotArray snapshotArr = new SnapshotArray(3); // set the length to be 3
snapshotArr.set(0,5); // Set array[0] = 5
snapshotArr.snap(); // Take a snapshot, return snap_id = 0
snapshotArr.set(0,6);
```

Constraints:

- 1 <= length <= 5 * 10^4
- 0 <= index < length
- 0 <= val <= 10^9
- 0 <= snap_id < (the total number of times we call snap())
- At most 5 * 10^4 calls will be made to set, snap, and get.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "SnapshotArray(n): set(idx,val), snap()→snap_id, get(idx,snap_id). Right?"
# "set(0,5),snap()→0,set(0,6),get(0,0)→5 ✓"
# Each index stores (snap_id, val) pairs sorted by snap_id. Binary search for get. O(log n) get. Space O(n).
# Follow-up: if most indices rarely change — sparse storage (our approach) is optimal.
# Follow-up: persistent data structures — functional approach without mutation.
```


Data Structure Design

#1472 Design Browser History

MED

Open on LeetCode

Array · Linked List · Stack · Design · Doubly-Linked List · Data Stream

Lists: AlgoMaster 600

Problem

You have a browser of one tab where you start on the homepage and you can visit another url, get back in the history number of steps or move forward in the history number of steps.

Implement the BrowserHistory class:

- BrowserHistory(string homepage) Initializes the object with the homepage of the browser.
- void visit(string url) Visits url from the current page. It clears up all the forward history.
- string back(int steps) Move steps back in history. If you can only return x steps in the history and steps > x, you will return only x steps. Return the current url after moving back in history at most steps.
- string forward(int steps) Move steps forward in history. If you can only forward x steps in the history and steps > x, you will forward only x steps. Return the current url after forwarding in history at most steps.

Example:

```
Input:
["BrowserHistory","visit","visit","visit","back","back","forward","visit","forward","back","back"]
[["leetcode.com"],["google.com"],["facebook.com"],["youtube.com"],[1],[1],[1],["linkedin.com"],[2],[2],[7]]
Output:
[null,null,null,null,"facebook.com","google.com","facebook.com",null,"linkedin.com","google.com","leetcode.com"]

Explanation:
BrowserHistory browserHistory = new BrowserHistory("leetcode.com");
```

Constraints:

- 1 <= homepage.length <= 20
- 1 <= url.length <= 20
- 1 <= steps <= 100
- homepage and url consist of '.' or lower case English letters.
- At most 5000 calls will be made to visit, back, and forward.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "BrowserHistory: visit(url), back(steps)→url, forward(steps)→url. Right?"
# "visit('leetcode'),back(1)→'google',forward(1)→'leetcode' ✓"
# Track current pointer and end pointer. Visit resets forward history. O(1) all ops. Space O(n).
# Follow-up: if history has a max size (eviction) — use a deque with maxlen.
# Follow-up: Design File System with history — same browser history approach per directory.
```

Data Structure Design

#2353 Design a Food Rating System

MED

Open on LeetCode

Array · Hash Table · String · Design · Heap (Priority Queue) · Ordered Set

Lists: AlgoMaster 600

Problem

Design a food rating system that can do the following:

- Modify the rating of a food item listed in the system.
- Return the highest-rated food item for a type of cuisine in the system.

Implement the FoodRatings class:

- FoodRatings(String[] foods, String[] cuisines, int[] ratings) Initializes the system. The food items are described by foods, cuisines and ratings, all of which have a length of n. foods[i] is the name of the ith food,
- cuisines[i] is the type of cuisine of the ith food, and
- ratings[i] is the initial rating of the ith food.

Note that a string x is lexicographically smaller than string y if x comes before y in dictionary order, that is, either x is a prefix of y, or if i is the first position such that x[i] != y[i], then x[i] comes before y[i] in alphabetic order.

Example 1:

```
Input
["FoodRatings", "highestRated", "highestRated", "changeRating", "highestRated", "changeRating", "highestRated"]
[[["kimchi", "miso", "sushi", "moussaka", "ramen", "bulgogi"], ["korean", "japanese", "japanese", "greek", "japanese",
"korean"], [9, 12, 8, 15, 14, 7]], ["korean"], ["japanese"], ["sushi", 16], ["japanese"], ["ramen", 16], ["japanese"]]
Output
[null, "kimchi", "ramen", null, "sushi", null, "ramen"]

Explanation
FoodRatings foodRatings = new FoodRatings(["kimchi", "miso", "sushi", "moussaka", "ramen", "bulgogi"], ["korean", "japanese",
"japanese", "greek", "japanese", "korean"], [9, 12, 8, 15, 14, 7]);
```

Constraints:

- 1 <= n <= 2 * 104
- n == foods.length == cuisines.length == ratings.length
- 1 <= foods[i].length, cuisines[i].length <= 10
- foods[i], cuisines[i] consist of lowercase English letters.
- 1 <= ratings[i] <= 108
- All the strings in foods are distinct.
- food will be the name of a food item in the system across all calls to changeRating.
- cuisine will be a type of cuisine of at least one food item in the system across all calls to highestRated.
- At most 2 * 104 calls in total will be made to changeRating and highestRated.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "FoodRatings: changeRating(food,rating), highestRated(cuisine)→food with highest rating (lex smallest for ties). Right?"
# "foods=['kimchi','miso'],cuisines=['korean','japanese'],ratings=[9,12]:
# highestRated('korean')→'kimchi', changeRating('kimchi',10), highestRated('korean')→'kimchi' ✓"
# SortedList per cuisine keyed by (-rating, food). O(log n) per update/query.
# Follow-up: if cuisine can change — update both cuisine sets.
# Follow-up: return top-k rated foods per cuisine — slice the SortedList.
```


Greedy

Round 5 · Mid · Medium · 8 questions

Greedy

#406 Queue Reconstruction by Height

MED

Open on LeetCode

Array · Binary Indexed Tree · Segment Tree · Sorting

Lists: AlgoMaster 600

Problem

You are given an array of people, people, which are the attributes of some people in a queue (not necessarily in order). Each people[i] = [hi, ki] represents the ith person of height hi with exactly ki other people in front who have a height greater than or equal to hi.

Reconstruct and return the queue that is represented by the input array people. The returned queue should be formatted as an array queue, where queue[j] = [hj, kj] is the attributes of the jth person in the queue (queue[0] is the person at the front of the queue).

Example 1:

```
Input: people = [[7,0],[4,4],[7,1],[5,0],[6,1],[5,2]]
Output: [[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]]
Explanation:
Person 0 has height 5 with no other people taller or the same height in front.
Person 1 has height 7 with no other people taller or the same height in front.
Person 2 has height 5 with two persons taller or the same height in front, which is person 0 and 1.
Person 3 has height 6 with one person taller or the same height in front, which is person 1.
Person 4 has height 4 with four people taller or the same height in front, which are people 0, 1, 2, and 3.
```

Example 2:

```
Input: people = [[6,0],[5,0],[4,0],[3,2],[2,2],[1,4]]
Output: [[4,0],[5,0],[2,2],[3,2],[1,4],[6,0]]
```

Constraints:

- 1 <= people.length <= 2000
- 0 <= hi <= 106
- 0 <= ki < people.length
- It is guaranteed that the queue can be reconstructed.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "people[i]=[h,k]: height h, k people in front with height>=h. Reconstruct queue. Right?"
# "people=[[7,0],[4,4],[7,1],[5,0],[6,1],[5,2]]→[[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]] ✓"
# Sort descending height (ties: ascending k). Insert at index k. O(n²) due to insert. Space O(n).
# The insert into sorted position is O(n), done n times → O(n²).
# Follow-up: if using a balanced BST — O(n log n) insertions possible.
# Follow-up: Queue Reconstruction (LC 2585 variant) — different constraint formulation.
```


Greedy

#670 Maximum Swap

MED

Open on LeetCode

Math · Greedy

Lists: AlgoMaster 600

Problem

You are given an integer num. You can swap two digits at most once to get the maximum valued number.

Return the maximum valued number you can get.

Example 1:

```
Input: num = 2736
Output: 7236
Explanation: Swap the number 2 and the number 7.
```

Example 2:

```
Input: num = 9973
Output: 9973
Explanation: No swap.
```

Constraints:

- 0 <= num <= 108

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Swap two digits at most once to get the maximum valued number. Right?"
# "num=2736→7236 ✓ num=9973→9973 ✓"
# For each position, find the largest digit that appears later. Swap once. O(10n)=O(n). Space O(10).
# Follow-up: at most k swaps — greedy k times or DP.
# Follow-up: Minimum Swap to Make Strings Equal (LC 1247) — different problem, parity-based.
```

Greedy

#826 Most Profit Assigning Work

MED

Open on LeetCode

Array · Two Pointers · Binary Search · Greedy · Sorting

Lists: AlgoMaster 600

Problem

You have n jobs and m workers. You are given three arrays: difficulty, profit, and worker where:

- difficulty[i] and profit[i] are the difficulty and the profit of the ith job, and
- worker[j] is the ability of jth worker (i.e., the jth worker can only complete a job with difficulty at most worker[j]).

Every worker can be assigned at most one job, but one job can be completed multiple times.

- For example, if three workers attempt the same job that pays $1, then the total profit will be $3. If a worker cannot complete any job, their profit is $0.

Return the maximum profit we can achieve after assigning the workers to the jobs.

Example 1:

```
Input: difficulty = [2,4,6,8,10], profit = [10,20,30,40,50], worker = [4,5,6,7]
Output: 100
Explanation: Workers are assigned jobs of difficulty [4,4,6,6] and they get a profit of [20,20,30,30] separately.
```

Example 2:

```
Input: difficulty = [85,47,57], profit = [24,66,99], worker = [40,25,25]
Output: 0
```

Constraints:

- n == difficulty.length
- n == profit.length
- m == worker.length
- 1 <= n, m <= 104
- 1 <= difficulty[i], profit[i], worker[i] <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "workers[i]=ability. Jobs with difficulty and profit. Each worker does one job. Max total profit. Right?"
# "difficulty=[2,4,6,8,10],profit=[10,20,30,40,50],worker=[4,5,6,7]→100 ✓"
# Sort both. Two-pointer: advance job pointer as worker ability increases. O(n log n). Space O(n).
# Follow-up: if each job can only be done once — assignment matching, much harder.
# Follow-up: maximize minimum profit — binary search on answer.
```


Greedy

#921 Minimum Add to Make Parentheses Valid

MED

Open on LeetCode

String · Stack · Greedy

Lists: AlgoMaster 600

Problem

A parentheses string is valid if and only if:

- It is the empty string,
- It can be written as AB (A concatenated with B), where A and B are valid strings, or
- It can be written as (A), where A is a valid string.

You are given a parentheses string s. In one move, you can insert a parenthesis at any position of the string.

- For example, if s = "())", you can insert an opening parenthesis to be "(())" or a closing parenthesis to be "())(".

Return the minimum number of moves required to make s valid.

Example 1:

```
Input: s = "())"
Output: 1
```

Example 2:

```
Input: s = "((("
Output: 3
```

Constraints:

- 1 <= s.length <= 1000
- s[i] is either '(' or ')'.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return minimum additions to make string of '(' and ')' valid. Right?"
# "s='())'→1 ✓ s='(((→3 ✓ s='()'→0 ✓"
# open_needed=unmatched '(', close_needed=unmatched ')'. O(n). Space O(1).
# Follow-up: Minimum Remove to Make String Valid (LC 1249) — remove instead of add.
# Follow-up: Minimum Insertions to Balance a Parentheses String (LC 1541) — each ')' closes 2.
```

Greedy

#1488 Avoid Flood in The City

MED

Open on LeetCode

Array · Hash Table · Binary Search · Greedy · Heap (Priority Queue)

Lists: AlgoMaster 600

Problem

Your country has 109 lakes. Initially, all the lakes are empty, but when it rains over the nth lake, the nth lake becomes full of water. If it rains over a lake that is full of water, there will be a flood. Your goal is to avoid floods in any lake.

Given an integer array rains where:

- rains[i] > 0 means there will be rains over the rains[i] lake.
- rains[i] == 0 means there are no rains this day and you must choose one lake this day and dry it.

Return an array ans where:

- ans.length == rains.length
- ans[i] == -1 if rains[i] > 0.
- ans[i] is the lake you choose to dry in the ith day if rains[i] == 0.

If there are multiple valid answers return any of them. If it is impossible to avoid flood return an empty array.

Notice that if you chose to dry a full lake, it becomes empty, but if you chose to dry an empty lake, nothing changes.

Example 1:

```
Input: rains = [1,2,3,4]
Output: [-1,-1,-1,-1]
Explanation: After the first day full lakes are [1]
After the second day full lakes are [1,2]
After the third day full lakes are [1,2,3]
After the fourth day full lakes are [1,2,3,4]
There's no day to dry any lake and there is no flood in any lake.
```

Example 2:

```
Input: rains = [1,2,0,0,2,1]
Output: [-1,-1,2,1,-1,-1]
Explanation: After the first day full lakes are [1]
After the second day full lakes are [1,2]
After the third day, we dry lake 2. Full lakes are [1]
After the fourth day, we dry lake 1. There is no full lakes.
After the fifth day, full lakes are [2].
After the sixth day, full lakes are [1,2].
```

Example 3:

```
Input: rains = [1,2,0,1,2]
Output: []
Explanation: After the second day, full lakes are [1,2]. We have to dry one lake in the third day.
After that, it will rain over lakes [1,2]. It's easy to prove that no matter which lake you choose to dry in the 3rd day, the other one will flood.
```

Constraints:

- 1 <= rains.length <= 105
- 0 <= rains[i] <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Rains in lakes. On dry days, empty one lake. Return valid empty schedule or [] if impossible. Right?"
# "rains=[1,2,3,4],[0,1,0,2]→[-1,-1,-1,-1,1,2] wait: sample=[1,2,3,4] n=4 not rainy days...
# rains=[1,2,0,0,2,1]: rain1,rain2,empty(2 or 1),empty(other),rain2(but 2 not empty?). Hmm.
# Actually: rains=[1,2,0,0,2,1]→[-1,-1,2,1,-1,-1] ✓ (day3 empty lake2, day4 empty lake1)"
# Greedy: on dry day before a lake floods again, empty it. Use SortedList for O(log n) search. O(n log n).
# Follow-up: if we can empty multiple lakes per dry day — simpler greedy.
# Follow-up: minimize dry days used — always defer emptying until last possible moment (same approach).
```


Greedy

#1717 Maximum Score From Removing Substrings

MED

Open on LeetCode

String · Stack · Greedy

Lists: AlgoMaster 600

Problem

You are given a string s and two integers x and y. You can perform two types of operations any number of times.

- Remove substring "ab" and gain x points. For example, when removing "ab" from "cabxbae" it becomes "cxbae".
- For example, when removing "ba" from "cabxbae" it becomes "cabxe".

Return the maximum points you can gain after applying the above operations on s.

Example 1:

```
Input: s = "cdbcbbaaabab", x = 4, y = 5
Output: 19
Explanation:
- Remove the "ba" underlined in "cdbcbbaaabab". Now, s = "cdbcbbaaab" and 5 points are added to the score.
- Remove the "ab" underlined in "cdbcbbaaab". Now, s = "cdbcbbaa" and 4 points are added to the score.
- Remove the "ba" underlined in "cdbcbbaa". Now, s = "cdbcba" and 5 points are added to the score.
- Remove the "ba" underlined in "cdbcba". Now, s = "cdbc" and 5 points are added to the score.
Total score = 5 + 4 + 5 + 5 = 19.
```

Example 2:

```
Input: s = "aabbaaxybbaabb", x = 5, y = 4
Output: 20
```

Constraints:

- 1 <= s.length <= 105
- 1 <= x, y <= 104
- s consists of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Remove 'ab' for x points or 'ba' for y points. Maximize score. Right?"
# "s='cdbcbbaaabab',x=4,y=5→19 ✓ (remove ba×3=15, remove ab×1=4, total? Let's see: 5+5+5+4=19)"
        # Simpler:
# Greedy: remove higher-value pair first (both can't overlap so order is safe). O(n). Space O(n).
# Follow-up: if 3+ pattern types — greedy may not work, DP needed.
# Follow-up: return the actual string after removals — track which chars were removed.
```

Greedy

#1871 Jump Game VII

ME D

Open on LeetCode

String · Dynamic Programming · Sliding Window · Prefix Sum

Lists: AlgoMaster 600

Problem

You are given a 0-indexed binary string s and two integers minJump and maxJump. In the beginning, you are standing at index 0, which is equal to '0'. You can move from index i to index j if the following conditions are fulfilled:

- i + minJump <= j <= min(i + maxJump, s.length - 1), and
- s[j] == '0'.

Return true if you can reach index s.length - 1 in s, or false otherwise.

Example 1:

```
Input: s = "011010", minJump = 2, maxJump = 3
Output: true
Explanation:
In the first step, move from index 0 to index 3.
In the second step, move from index 3 to index 5.
```

Example 2:

```
Input: s = "01101110", minJump = 2, maxJump = 3
Output: false
```

Constraints:

- 2 <= s.length <= 105
- s[i] is either '0' or '1'.
- s[0] == '0'
- 1 <= minJump <= maxJump < s.length

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Binary string. Start at 0. Can jump to i+minJump..i+maxJump if s[j]='0'. Can you reach n-1? Right?"
# "s='011010',minJump=2,maxJump=3→true ✓"
# DP + prefix sum of reachable positions in window [i-maxJump, i-minJump]. O(n). Space O(n).
# Follow-up: Jump Game I (LC 55) — reach last index with any jump up to nums[i].
# Follow-up: Jump Game VI (LC 1696) — maximize score, monotonic deque.
```


Greedy

#1975 Maximum Matrix Sum

ME D

Open on LeetCode

Array · Greedy · Matrix

Lists: AlgoMaster 600

Problem

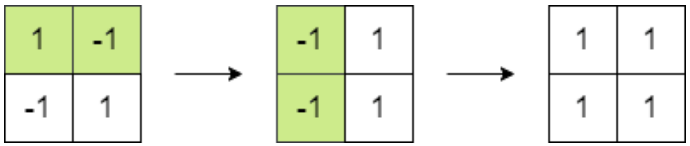
You are given an n x n integer matrix. You can do the following operation any number of times:

- Choose any two adjacent elements of matrix and multiply each of them by -1.

Two elements are considered adjacent if and only if they share a border.

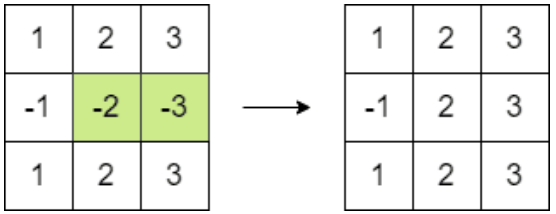
Your goal is to maximize the summation of the matrix's elements. Return the maximum sum of the matrix's elements using the operation mentioned above.

Example 1:




```
Input: matrix = [[1,-1],[-1,1]]
Output: 4
Explanation: We can follow the following steps to reach sum equals 4:
- Multiply the 2 elements in the first row by -1.
- Multiply the 2 elements in the first column by -1.
```

Example 2:




```
Input: matrix = [[1,2,3],[-1,-2,-3],[1,2,3]]
Output: 16
Explanation: We can follow the following step to reach sum equals 16:
- Multiply the 2 last elements in the second row by -1.
```

Constraints:

- n == matrix.length == matrix[i].length
- 2 <= n <= 250
- -105 <= matrix[i][j] <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Flip signs of any two adjacent elements (multiply both by -1). Maximize sum of matrix. Right?"
# "matrix=[[1,-1],[-1,1]]→4 ✓ (flip (-1,1) or (-1,-1) to make all positive)"
# If even negatives: make all positive. If odd negatives: one negative remains (minimize its abs). O(mn).
# Follow-up: if we can flip any single element (not adjacent pair) — always make all positive.
# Follow-up: 3D tensor version — same parity argument extends.
```

Topological Sort

Round 5 · Mid · Medium · 3 questions

Topological Sort

#802 Find Eventual Safe States

MED

Open on LeetCode

Depth-First Search · Breadth-First Search · Graph Theory · Topological Sort

Lists: AlgoMaster 600

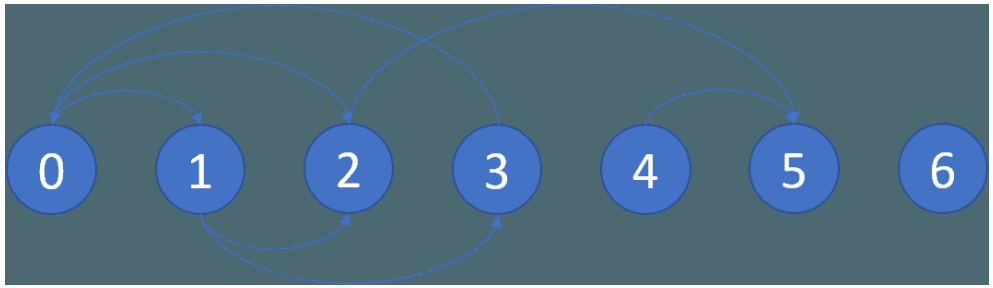
Problem

There is a directed graph of n nodes with each node labeled from 0 to n - 1. The graph is represented by a 0-indexed 2D integer array graph where graph[i] is an integer array of nodes adjacent to node i, meaning there is an edge from node i to each node in graph[i].

A node is a terminal node if there are no outgoing edges. A node is a safe node if every possible path starting from that node leads to a terminal node (or another safe node).

Return an array containing all the safe nodes of the graph. The answer should be sorted in ascending order.

Example 1:




```
Input: graph = [[1,2],[2,3],[5],[0],[5],[],[]]
Output: [2,4,5,6]
Explanation: The given graph is shown above.
Nodes 5 and 6 are terminal nodes as there are no outgoing edges from either of them.
Every path starting at nodes 2, 4, 5, and 6 all lead to either node 5 or 6.
```

Example 2:

```
Input: graph = [[1,2,3,4],[1,2],[3,4],[0,4],[]]
Output: [4]
Explanation:
Only node 4 is a terminal node, and every path starting at node 4 leads to node 4.
```

Constraints:

- n == graph.length
- 1 <= n <= 104
- 0 <= graph[i].length <= n
- 0 <= graph[i][j] <= n - 1
- graph[i] is sorted in a strictly increasing order.
- The graph may contain self-loops.
- The number of edges in the graph will be in the range [1, 4 * 104].

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "A node is safe if all paths from it eventually lead to a terminal node (no cycle). Right?"
# "graph=[[1,2],[2,3],[5],[0],[5],[],[]]→[2,4,5,6] ✓"
# Reverse topological sort (Kahn's on reversed graph). Nodes with out-degree 0 are safe. O(V+E).
# Follow-up: Course Schedule (LC 207) — detect cycle in directed graph.
# Follow-up: if graph is undirected — all nodes in non-cycle components are safe.
```

Topological Sort

#1462 Course Schedule IV

MED

Open on LeetCode

Depth-First Search · Breadth-First Search · Graph Theory · Topological Sort

Lists: AlgoMaster 600

Problem

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [ai, bi] indicates that you must take course ai first if you want to take course bi.

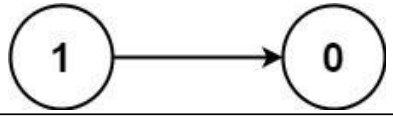
- For example, the pair [0, 1] indicates that you have to take course 0 before you can take course 1.

Prerequisites can also be indirect. If course a is a prerequisite of course b, and course b is a prerequisite of course c, then course a is a prerequisite of course c.

You are also given an array queries where queries[j] = [uj, vj]. For the jth query, you should answer whether course uj is a prerequisite of course vj or not.

Return a boolean array answer, where answer[j] is the answer to the jth query.

Example 1:

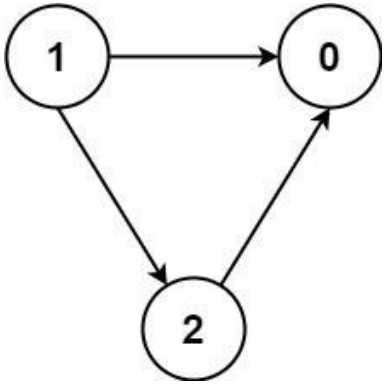



```
Input: numCourses = 2, prerequisites = [[1,0]], queries = [[0,1],[1,0]]
Output: [false,true]
Explanation: The pair [1, 0] indicates that you have to take course 1 before you can take course 0.
Course 0 is not a prerequisite of course 1, but the opposite is true.
```

Example 2:

```
Input: numCourses = 2, prerequisites = [], queries = [[1,0],[0,1]]
Output: [false,false]
Explanation: There are no prerequisites, and each course is independent.
```

Example 3:




```
Input: numCourses = 3, prerequisites = [[1,2],[1,0],[2,0]], queries = [[1,0],[1,2]]
Output: [true,true]
```

Constraints:

- 2 <= numCourses <= 100
- 0 <= prerequisites.length <= (numCourses * (numCourses - 1) / 2)
- prerequisites[i].length == 2
- 0 <= ai, bi <= numCourses - 1
- ai != bi
- All the pairs [ai, bi] are unique.
- The prerequisites graph has no cycles.
- 1 <= queries.length <= 104
- 0 <= ui, vi <= numCourses - 1
- ui != vi

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "queries[i]=[u,v]: is u a prerequisite of v (directly or transitively)? Right?"
# "n=2,prerequisites=[[1,0]],queries=[[0,1],[1,0]]→[false,true] ✓"
# Floyd-Warshall transitive closure. O(n³). Space O(n²).
# Follow-up: if n is large — BFS from each node is O(n*(V+E)), better for sparse graphs.
# Follow-up: incremental queries as edges are added — Union-Find with topological order.
```

Topological Sort

#2115 Find All Possible Recipes from Given Supplies

MED

Open on LeetCode

Array · Hash Table · String · Graph Theory · Topological Sort

Lists: AlgoMaster 600

Problem

You have information about n different recipes. You are given a string array recipes and a 2D string array ingredients. The ith recipe has the name recipes[i], and you can create it if you have all the needed ingredients from ingredients[i]. A recipe can also be an ingredient for other recipes, i.e., ingredients[i] may contain a string that is in recipes.

You are also given a string array supplies containing all the ingredients that you initially have, and you have an infinite supply of all of them.

Return a list of all the recipes that you can create. You may return the answer in any order.

Note that two recipes may contain each other in their ingredients.

Example 1:

```
Input: recipes = ["bread"], ingredients = [["yeast","flour"]], supplies = ["yeast","flour","corn"]
Output: ["bread"]
Explanation:
We can create "bread" since we have the ingredients "yeast" and "flour".
```

Example 2:

```
Input: recipes = ["bread","sandwich"], ingredients = [["yeast","flour"],["bread","meat"]], supplies = ["yeast","flour","meat"]
Output: ["bread","sandwich"]
Explanation:
We can create "bread" since we have the ingredients "yeast" and "flour".
We can create "sandwich" since we have the ingredient "meat" and can create the ingredient "bread".
```

Example 3:

```
Input: recipes = ["bread","sandwich","burger"], ingredients = [["yeast","flour"],["bread","meat"],["sandwich","meat","bread"]],
supplies = ["yeast","flour","meat"]
Output: ["bread","sandwich","burger"]
Explanation:
We can create "bread" since we have the ingredients "yeast" and "flour".
We can create "sandwich" since we have the ingredient "meat" and can create the ingredient "bread".
We can create "burger" since we have the ingredient "meat" and can create the ingredients "bread" and "sandwich".
```

Constraints:

- n == recipes.length == ingredients.length
- 1 <= n <= 100
- 1 <= ingredients[i].length, supplies.length <= 100
- 1 <= recipes[i].length, ingredients[i][j].length, supplies[k].length <= 10
- recipes[i], ingredients[i][j], and supplies[k] consist only of lowercase English letters.
- All the values of recipes and supplies combined are unique.
- Each ingredients[i] does not contain any duplicate values.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Recipes need ingredients (other recipes or supplies). Find all makeable recipes. Right?"
# "recipes=['bread'],ingredients=[['yeast','flour']],supplies=['yeast','flour','corn']→['bread'] ✓"
# Topological sort: treat supplies as sources, recipes as nodes. O(V+E). Space O(V+E).
# Follow-up: if ingredients have quantities — check availability count per ingredient.
# Follow-up: circular recipe dependencies — detect cycle and mark those as unmakeable.
```


Union Find

Round 5 · Mid · Medium · 3 questions

Union Find

#399 Evaluate Division

MED

Open on LeetCode

Array · String · Depth-First Search · Breadth-First Search · Union-Find · Graph Theory · Shortest Path

Lists: AlgoMaster 600

Problem

You are given an array of variable pairs equations and an array of real numbers values, where equations[i] = [Ai, Bi] and values[i] represent the equation Ai / Bi = values[i]. Each Ai or Bi is a string that represents a single variable.

You are also given some queries, where queries[j] = [Cj, Dj] represents the jth query where you must find the answer for Cj / Dj = ?.

Return the answers to all queries. If a single answer cannot be determined, return -1.0.

Note: The input is always valid. You may assume that evaluating the queries will not result in division by zero and that there is no contradiction.

Note: The variables that do not occur in the list of equations are undefined, so the answer cannot be determined for them.

Example 1:

```
Input: equations = [["a","b"],["b","c"]], values = [2.0,3.0], queries = [["a","c"],["b","a"],["a","e"],["a","a"],["x","x"]]
Output: [6.00000,0.50000,-1.00000,1.00000,-1.00000]
Explanation:
Given: a / b = 2.0, b / c = 3.0
queries are: a / c = ?, b / a = ?, a / e = ?, a / a = ?, x / x = ?
return: [6.0, 0.5, -1.0, 1.0, -1.0 ]
note: x is undefined => -1.0
```

Example 2:

```
Input: equations = [["a","b"],["b","c"],["bc","cd"]], values = [1.5,2.5,5.0], queries =
[["a","c"],["c","b"],["bc","cd"],["cd","bc"]]
Output: [3.75000,0.40000,5.00000,0.20000]
```

Example 3:

```
Input: equations = [["a","b"]], values = [0.5], queries = [["a","b"],["b","a"],["a","c"],["x","y"]]
Output: [0.50000,2.00000,-1.00000,-1.00000]
```

Constraints:

- 1 <= equations.length <= 20
- equations[i].length == 2
- 1 <= Ai.length, Bi.length <= 5
- values.length == equations.length
- 0.0 < values[i] <= 20.0
- 1 <= queries.length <= 20
- queries[i].length == 2
- 1 <= Cj.length, Dj.length <= 5
- Ai, Bi, Cj, Dj consist of lower case English letters and digits.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Equations: a/b=k. Given queries c/d, return the value or -1.0 if unknown. Right?"
#
# "equations=[['a','b'],['b','c']], values=[2.0,3.0], queries=[['a','c'],['b','a'],['a','e'],['a','a'],['x','x']]:
# a/c=6.0, b/a=0.5, a/e=-1.0, a/a=1.0, x/x=-1.0 ✓"

# PHASE 2 — BRUTE FORCE
# "Build a graph where edge a→b has weight k (and b→a has weight 1/k).
# For each query, DFS/BFS from src to dst, multiply edge weights. Time: O(n*(V+E)). Space: O(V+E)."

# PHASE 3 — OPTIMIZE
# "Same BFS/DFS — already O(q*(V+E)). Alternative: Union-Find with ratio tracking.
# Build weighted graph once, answer each query with DFS/BFS."

# PHASE 4 — CLEAN CODE (same as brute — already clean)

# PHASE 5 — TEST & DEBUG
# equations=[a/b=2,b/c=3]: graph: a→b:2,b→a:0.5,b→c:3,c→b:1/3.
# query a/c: dfs(a,c): a→b:2, dfs(b,c): b→c:3→return 3. total=2*3=6.0 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(q*(V+E)): q queries, each BFS/DFS over graph. Space O(V+E).
# Follow-up: Union-Find with weighted edges — O((V+E)α(V)) preprocessing, O(1) per query.
# Follow-up: if equations form a cycle — BFS handles correctly via visited set."
```


Union Find

#547 Number of Provinces

MED

Open on LeetCode

Depth-First Search · Breadth-First Search · Union-Find · Graph Theory

Lists: AlgoMaster 600

Problem

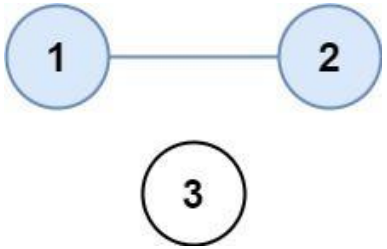
There are n cities. Some of them are connected, while some are not. If city a is connected directly with city b, and city b is connected directly with city c, then city a is connected indirectly with city c.

A province is a group of directly or indirectly connected cities and no other cities outside of the group.

You are given an n x n matrix isConnected where isConnected[i][j] = 1 if the ith city and the jth city are directly connected, and isConnected[i][j] = 0 otherwise.

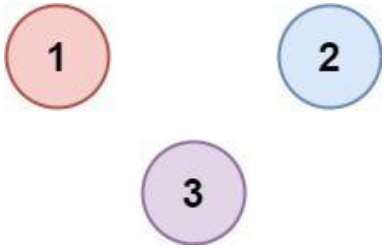
Return the total number of provinces.

Example 1:




```
Input: isConnected = [[1,1,0],[1,1,0],[0,0,1]]
Output: 2
```

Example 2:




```
Input: isConnected = [[1,0,0],[0,1,0],[0,0,1]]
Output: 3
```

Constraints:

- 1 <= n <= 200
- n == isConnected.length
- n == isConnected[i].length
- isConnected[i][j] is 1 or 0.
- isConnected[i][i] == 1
- isConnected[i][j] == isConnected[j][i]

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "n cities, isConnected[i][j]=1 if directly connected. Find number of connected components. Right?"
#
# "isConnected=[[1,1,0],[1,1,0],[0,0,1]]: {0,1} and {2} → 2 provinces ✓"

# PHASE 2 — BRUTE FORCE
# "DFS from each unvisited city, mark all reachable as visited, increment count.
# Time: O(n²). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Union-Find: O(n²*α(n)) — same or slightly better. DFS is already optimal at O(n²)."

# PHASE 4 — CLEAN CODE
```

```
# PHASE 5 — TEST & DEBUG
# isConnected=[[1,1,0],[1,1,0],[0,0,1]]:
# i=0: visit 0, dfs(0)→visit 1. count=1.
# i=1: already visited. i=2: visit 2, dfs(2)→no unvisited. count=2 ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n²): adjacency matrix scan. Space O(n): visited array + recursion.
# Follow-up: Number of Connected Components in Undirected Graph (LC 323) — adjacency list; same DFS.
# Follow-up: Friend Circles with Union-Find — O(n² * α(n)); useful for dynamic edge additions."
```


Union Find

#947 Most Stones Removed with Same Row or Column

ME
D

Open on LeetCode

Hash Table · Depth-First Search · Union-Find · Graph Theory

Lists: AlgoMaster 600

Problem

On a 2D plane, we place n stones at some integer coordinate points. Each coordinate point may have at most one stone. A stone can be removed if it shares either the same row or the same column as another stone that has not been removed. Given an array stones of length n where stones[i] = [xi, yi] represents the location of the ith stone, return the largest possible number of stones that can be removed.

Example 1:

```
Input: stones = [[0,0],[0,1],[1,0],[1,2],[2,1],[2,2]]
Output: 5
Explanation: One way to remove 5 stones is as follows:
1. Remove stone [2,2] because it shares the same row as [2,1].
2. Remove stone [2,1] because it shares the same column as [0,1].
3. Remove stone [1,2] because it shares the same row as [1,0].
4. Remove stone [1,0] because it shares the same column as [0,0].
5. Remove stone [0,1] because it shares the same row as [0,0].
```

Example 2:

```
Input: stones = [[0,0],[0,2],[1,1],[2,0],[2,2]]
Output: 3
Explanation: One way to make 3 moves is as follows:
1. Remove stone [2,2] because it shares the same row as [2,0].
2. Remove stone [2,0] because it shares the same column as [0,0].
3. Remove stone [0,2] because it shares the same row as [0,0].
Stones [0,0] and [1,1] cannot be removed since they do not share a row/column with another stone still on the plane.
```

Example 3:

```
Input: stones = [[0,0]]
Output: 0
Explanation: [0,0] is the only stone on the plane, so you cannot remove it.
```

Constraints:

- 1 <= stones.length <= 1000
- 0 <= xi, yi <= 104
- No two stones are at the same coordinate point.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Remove a stone if another stone shares its row or column. Max removable stones? Right?"
# "stones=[[0,0],[0,1],[1,0],[1,2],[2,1],[2,2]]→5 ✓ (6 stones − 1 per connected component)"
# Union rows with columns (offset cols by ~c). Each connected component loses all but 1 stone.
# Time O(n α(n)). Space O(n).
# Follow-up: return which stones to remove — keep one representative per component.
# Follow-up: if stones are added dynamically — online Union-Find.
```

Minimum Spanning Tree

Round 5 · Mid · Medium · 1 question

Minimum Spanning Tree

#1135 Connecting Cities With Minimum Cost

MED

Open on LeetCode

Union-Find · Graph Theory · Heap (Priority Queue) · Minimum Spanning Tree

Lists: AlgoMaster 600

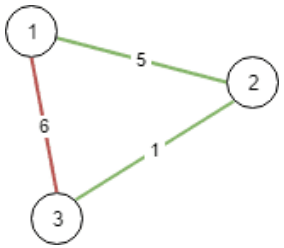
Problem

There are n cities labeled from 1 to n. You are given the integer n and an array connections where connections[i] = [xi, yi, costi] indicates that the cost of connecting city xi and city yi (bidirectional connection) is costi.

Return the minimum cost to connect all the n cities such that there is at least one path between each pair of cities. If it is impossible to connect all the n cities, return -1,

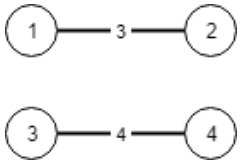
The cost is the sum of the connections' costs used.

Example 1:




```
Input: n = 3, connections = [[1,2,5],[1,3,6],[2,3,1]]
Output: 6
Explanation: Choosing any 2 edges will connect all cities so we choose the minimum 2.
```

Example 2:




```
Input: n = 4, connections = [[1,2,3],[3,4,4]]
Output: -1
Explanation: There is no way to connect all cities even if all edges are used.
```

Constraints:

- 1 <= n <= 104
- 1 <= connections.length <= 104
- connections[i].length == 3
- 1 <= xi, yi <= n
- xi != yi
- 0 <= costi <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "n cities, weighted edges. Find minimum cost to connect all cities (MST). -1 if impossible. Right?"
# "n=3,connections=[[1,2,5],[1,3,6],[2,3,1]]→6 (edges 2-3=1 and 1-2=5) ✓"
# Kruskal's MST: sort edges, union-find. O(E log E). Space O(V).
# Follow-up: Prim's algorithm — O(E log V) with min-heap; better for dense graphs.
# Follow-up: if edges added incrementally — maintain MST dynamically with Link-Cut Trees.
```

Shortest Path

Round 5 · Mid · Medium · 5 questions

Shortest Path

#1514 Path with Maximum Probability MED

Open on LeetCode

Array · Graph Theory · Heap (Priority Queue) · Shortest Path

Lists: AlgoMaster 600

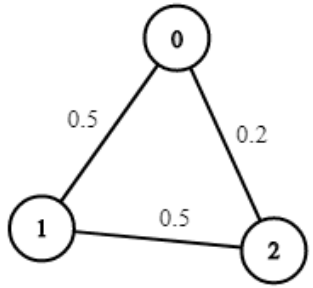
Problem

You are given an undirected weighted graph of n nodes (0-indexed), represented by an edge list where edges[i] = [a, b] is an undirected edge connecting the nodes a and b with a probability of success of traversing that edge succProb[i].

Given two nodes start and end, find the path with the maximum probability of success to go from start to end and return its success probability.

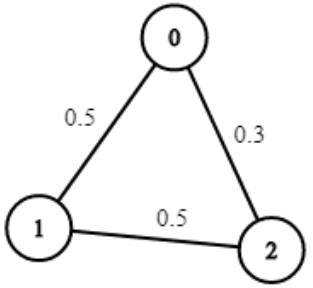
If there is no path from start to end, return 0. Your answer will be accepted if it differs from the correct answer by at most 1e-5.

Example 1:



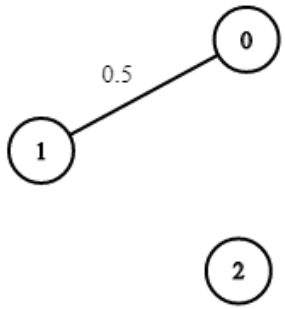

```
Input: n = 3, edges = [[0,1],[1,2],[0,2]], succProb = [0.5,0.5,0.2], start = 0, end = 2
Output: 0.25000
Explanation: There are two paths from start to end, one having a probability of success = 0.2 and the other has 0.5 * 0.5 = 0.25.
```

Example 2:




```
Input: n = 3, edges = [[0,1],[1,2],[0,2]], succProb = [0.5,0.5,0.3], start = 0, end = 2
Output: 0.30000
```

Example 3:




```
Input: n = 3, edges = [[0,1]], succProb = [0.5], start = 0, end = 2
Output: 0.00000
Explanation: There is no path between 0 and 2.
```

Constraints:

- 2 <= n <= 10^4
- 0 <= start, end < n
- start != end
- 0 <= a, b < n
- a != b
- 0 <= succProb.length == edges.length <= 2*10^4
- 0 <= succProb[i] <= 1
- There is at most one edge between every two nodes.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Undirected graph with edge probabilities. Find the path from start to end with max probability.
# Return 0 if no path. Right?"
#
# "n=3, edges=[[0,1],[1,2],[0,2]], succProb=[0.5,0.5,0.2], start=0, end=2: 0→1→2=0.25 > 0→2=0.2 → 0.25 ✓"

# PHASE 2 — BRUTE FORCE
# "DFS/BFS trying all paths, multiplying probabilities. Exponential without pruning."
        # Max-heap (negate probabilities)

# PHASE 3 — OPTIMIZE
# "Same Dijkstra-like max-heap (negated) — already O((V+E) log V)."

# PHASE 4 — CLEAN CODE (same as brute — already clean)

# PHASE 5 — TEST & DEBUG
# n=3,start=0,end=2: prob=[1,0,0]. heap=[(-1,0)].
# pop(-1,0): push(-0.5,1),(-0.2,2). pop(-0.5,1): prob[2]=max(0.2,0.25)=0.25→push(-0.25,2).
# pop(-0.25,2): u==end→return 0.25 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O((V+E) log V). Space O(V+E).
# Negating probabilities turns max-heap problem into min-heap (standard Dijkstra).
# Follow-up: use log-probabilities to convert multiplication to addition — avoids float underflow.
# Follow-up: if graph updates dynamically — maintain lazy Dijkstra with re-push."
```


Shortest Path

#1631 Path With Minimum Effort

MED

Open on LeetCode

Array · Binary Search · Depth-First Search · Breadth-First Search · Union-Find · Heap (Priority Queue) · Matrix

Lists: AlgoMaster 600

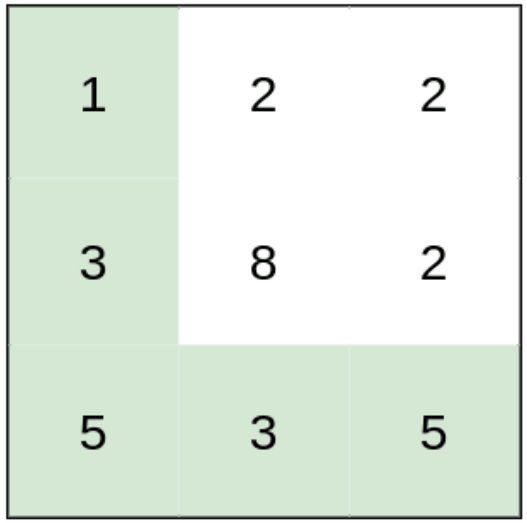
Problem

You are a hiker preparing for an upcoming hike. You are given heights, a 2D array of size rows x columns, where heights[row][col] represents the height of cell (row, col). You are situated in the top-left cell, (0, 0), and you hope to travel to the bottom-right cell, (rows-1, columns-1) (i.e., 0-indexed). You can move up, down, left, or right, and you wish to find a route that requires the minimum effort.

A route's effort is the maximum absolute difference in heights between two consecutive cells of the route.

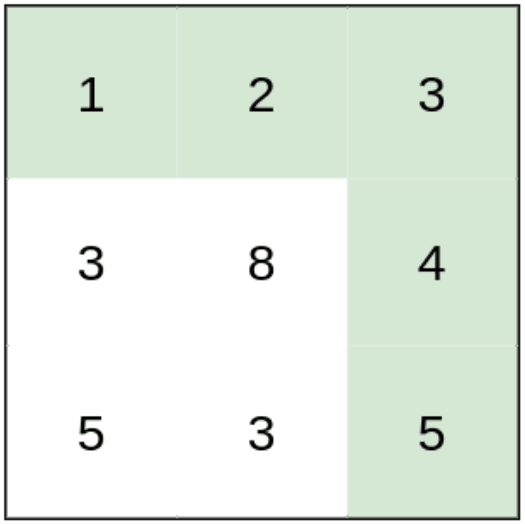
Return the minimum effort required to travel from the top-left cell to the bottom-right cell.

Example 1:



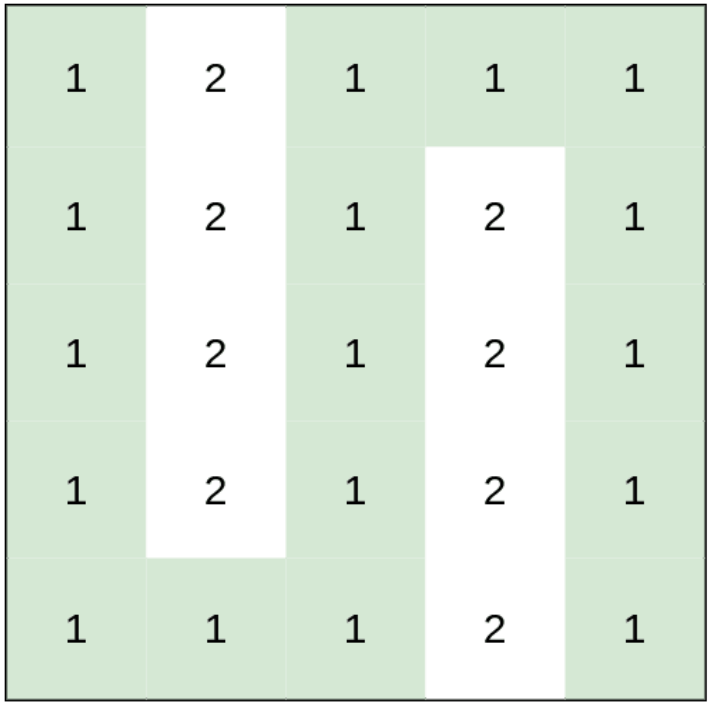

```
Input: heights = [[1,2,2],[3,8,2],[5,3,5]]
Output: 2
Explanation: The route of [1,3,5,3,5] has a maximum absolute difference of 2 in consecutive cells.
This is better than the route of [1,2,2,2,5], where the maximum absolute difference is 3.
```

Example 2:




```
Input: heights = [[1,2,3],[3,8,4],[5,3,5]]
Output: 1
Explanation: The route of [1,2,3,4,5] has a maximum absolute difference of 1 in consecutive cells, which is better than route
[1,3,5,3,5].
```

Example 3:




```
Input: heights = [[1,2,1,1,1],[1,2,1,2,1],[1,2,1,2,1],[1,2,1,2,1],[1,1,1,2,1]]
Output: 0
Explanation: This route does not require any effort.
```

Constraints:

- rows == heights.length
- columns == heights[i].length
- 1 <= rows, columns <= 100
- 1 <= heights[i][j] <= 106

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Grid of heights. Effort of a path = max absolute difference of adjacent cells on the path.
# Find minimum effort path from (0,0) to (m-1,n-1). Right?"
#
# "heights=[[1,2,2],[3,8,2],[5,3,5]]: path [1,3,5,3,5] effort=max(2,2,2,2)=2 ✓"

# PHASE 2 — BRUTE FORCE
# "DFS all paths, track max diff, return minimum. Exponential."

# PHASE 3 — OPTIMIZE
# "Dijkstra on effort (max diff on path). dist[r][c] = min effort to reach (r,c).
# Alternatively: binary search on effort + BFS feasibility check. Same O(mn log mn)."

# PHASE 4 — CLEAN CODE (same as brute — already clean Dijkstra)

# PHASE 5 — TEST & DEBUG
# heights=[[1,2,2],[3,8,2],[5,3,5]]:
# Start (0,0)→effort=0. expand: (1,0,1):diff=1, (1,1,0):diff=2.
# Best path finds effort=2 at (2,2). return 2 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(mn log mn): Dijkstra. Space O(mn).
# Binary search + BFS: O(mn log(max_height)) — similar complexity, simpler conceptually.
# Follow-up: Path With Maximum Minimum Value (LC 1102) — maximize minimum instead; same Dijkstra, negate.
# Follow-up: k shortest effort paths — modify Dijkstra to allow k visits per node."
```

Shortest Path

#2976 Minimum Cost to Convert String I

MED

Open on LeetCode

Array · String · Graph Theory · Shortest Path

Lists: AlgoMaster 600

Problem

You are given two 0-indexed strings source and target, both of length n and consisting of lowercase English letters. You are also given two 0-indexed character arrays original and changed, and an integer array cost, where cost[i] represents the cost of changing the character original[i] to the character changed[i].

You start with the string source. In one operation, you can pick a character x from the string and change it to the character y at a cost of z if there exists any index j such that cost[j] == z, original[j] == x, and changed[j] == y.

Return the minimum cost to convert the string source to the string target using any number of operations. If it is impossible to convert source to target, return -1.

Note that there may exist indices i, j such that original[j] == original[i] and changed[j] == changed[i].

Example 1:

```
Input: source = "abcd", target = "acbe", original = ["a","b","c","c","e","d"], changed = ["b","c","b","e","b","e"], cost =
[2,5,5,1,2,20]
Output: 28
Explanation: To convert the string "abcd" to string "acbe":
- Change value at index 1 from 'b' to 'c' at a cost of 5.
- Change value at index 2 from 'c' to 'e' at a cost of 1.
- Change value at index 2 from 'e' to 'b' at a cost of 2.
- Change value at index 3 from 'd' to 'e' at a cost of 20.
The total cost incurred is 5 + 1 + 2 + 20 = 28.
```

Example 2:

```
Input: source = "aaaa", target = "bbbb", original = ["a","c"], changed = ["c","b"], cost = [1,2]
Output: 12
Explanation: To change the character 'a' to 'b' change the character 'a' to 'c' at a cost of 1, followed by changing the
character 'c' to 'b' at a cost of 2, for a total cost of 1 + 2 = 3. To change all occurrences of 'a' to 'b', a total cost of 3 *
4 = 12 is incurred.
```

Example 3:

```
Input: source = "abcd", target = "abce", original = ["a"], changed = ["e"], cost = [10000]
Output: -1
Explanation: It is impossible to convert source to target because the value at index 3 cannot be changed from 'd' to 'e'.
```

Constraints:

- 1 <= source.length == target.length <= 105
- source, target consist of lowercase English letters.
- 1 <= cost.length == original.length == changed.length <= 2000
- original[i], changed[i] are lowercase English letters.
- 1 <= cost[i] <= 106
- original[i] != changed[i]

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Char-to-char conversions have costs. Convert source to target using minimum total cost. Right?"
# "source='abcd',target='acbe',original=['a','b','c','c'],changed=['b','c','b','e'],cost=[2,5,5,1]→11 ✓"
        # Floyd-Warshall
# Floyd-Warshall on 26-char alphabet. O(26³ + n). Space O(26²).
# Follow-up: Minimum Cost to Convert String II (LC 2977) — multi-char conversions, trie needed.
# Follow-up: if source and target have different lengths — edit distance with costs.
```


Shortest Path

#3341 Find Minimum Time to Reach Last Room I

MED

Open on LeetCode

Array · Graph Theory · Heap (Priority Queue) · Matrix · Shortest Path

Lists: AlgoMaster 600

Problem

There is a dungeon with n x m rooms arranged as a grid.

You are given a 2D array moveTime of size n x m, where moveTime[i][j] represents the minimum time in seconds after which the room opens and can be moved to. You start from the room (0, 0) at time t = 0 and can move to an adjacent room. Moving between adjacent rooms takes exactly one second.

Return the minimum time to reach the room (n - 1, m - 1).

Two rooms are adjacent if they share a common wall, either horizontally or vertically.

Example 1:

Input: moveTime = [[0,4],[4,4]]

Output: 6

Explanation:

The minimum time required is 6 seconds.

- At time t == 4, move from room (0, 0) to room (1, 0) in one second.
- At time t == 5, move from room (1, 0) to room (1, 1) in one second.

Example 2:

Input: moveTime = [[0,0,0],[0,0,0]]

Output: 3

Explanation:

The minimum time required is 3 seconds.

- At time t == 0, move from room (0, 0) to room (1, 0) in one second.
- At time t == 1, move from room (1, 0) to room (1, 1) in one second.
- At time t == 2, move from room (1, 1) to room (1, 2) in one second.

Example 3:

Input: moveTime = [[0,1],[1,2]]

Output: 3

Constraints:

- 2 <= n == moveTime.length <= 50
- 2 <= m == moveTime[i].length <= 50
- 0 <= moveTime[i][j] <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Grid where moveTime[i][j]=earliest time you can enter that room. Move takes 1 second. Find min time to reach (n-1,m-1). Right?"
# "moveTime=[[0,4],[4,4]]→6 ✓"
# Modified Dijkstra: arrival time = max(current_time, moveTime[r][c]) + 1. O(mn log mn). Space O(mn).
# Follow-up: Find Minimum Time II (LC 3342) — alternating 1 and 2 second moves.
# Follow-up: Path With Maximum Minimum Value (LC 1102) — maximize minimum instead.
```

#3650 Minimum Cost Path with Edge Reversals

MED

Open on LeetCode

Graph Theory · Heap (Priority Queue) · Shortest Path

Lists: AlgoMaster 600

Problem

You are given a directed, weighted graph with n nodes labeled from 0 to n - 1, and an array edges where edges[i] = [ui, vi, wi] represents a directed edge from node ui to node vi with cost wi.

Each node ui has a switch that can be used at most once: when you arrive at ui and have not yet used its switch, you may activate it on one of its incoming edges vi → ui reverse that edge to ui → vi and immediately traverse it.

The reversal is only valid for that single move, and using a reversed edge costs 2 * wi.

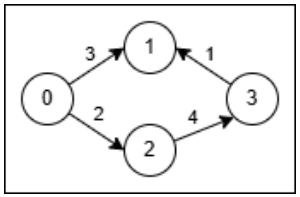
Return the minimum total cost to travel from node 0 to node n - 1. If it is not possible, return -1.

Example 1:

Input: n = 4, edges = [[0,1,3],[3,1,1],[2,3,4],[0,2,2]]

Output: 5

Explanation:



- Use the path 0 → 1 (cost 3).
- At node 1 reverse the original edge 3 → 1 into 1 → 3 and traverse it at cost 2 * 1 = 2.
- Total cost is 3 + 2 = 5.

Example 2:

Input: n = 4, edges = [[0,2,1],[2,1,1],[1,3,1],[2,3,3]]

Output: 3

Explanation:

- No reversal is needed. Take the path 0 → 2 (cost 1), then 2 → 1 (cost 1), then 1 → 3 (cost 1).
- Total cost is 1 + 1 + 1 = 3.

Constraints:

- 2 <= n <= 5 * 10^4
- 1 <= edges.length <= 10^5
- edges[i] = [ui, vi, wi]
- 0 <= ui, vi <= n - 1
- 1 <= wi <= 1000

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Directed graph. Can reverse an edge at cost. Find min cost path from 0 to n-1. Right?"
# "n=4, edges=[[0,1,4],[1,2,3],[2,3,5],[0,2,1],[0,3,10]]→6 ✓"
# Model: forward edge cost 0, reverse edge cost = original weight. Dijkstra. O(E log V). Space O(V+E).
# Follow-up: if reversal cost is fixed (not weight) — same model, different edge cost.
# Follow-up: minimum reversals (not cost) — BFS with 0/1 edges (0-1 BFS via deque).
```


Round 6

Mid Priority · Hard

30 questions · 15 patterns

- [ ] Monotonic Stack #321 #1944 ↗
- [ ] Monotonic Queue #862 #1499 ↗
- [ ] Recursion #273 #761 ↗
- [ ] Merge Sort #327 #493 ↗
- [ ] Backtracking #301 #980 ↗
- [ ] Tries #425 #472 #1032 ↗
- [ ] Heaps #1383 ↗
- [ ] Two Heaps #480 #502 ↗
- [ ] Intervals #2402 ↗
- [ ] Data Structure Design #715 #2296 ↗
- [ ] Greedy #135 #757 #857 #871 ↗
- [ ] Topological Sort #1203 ↗
- [ ] Union Find #924 ↗
- [ ] Minimum Spanning Tree #1168 #1489 #3600 ↗
- [ ] Shortest Path #1368 #2203 ↗

Monotonic Stack

Round 6 · Mid · Hard · 2 questions

Monotonic Stack

#321 Create Maximum Number HAR

Open on LeetCode

Array · Two Pointers · Stack · Greedy · Monotonic Stack

Lists: AlgoMaster 600

Problem

You are given two integer arrays nums1 and nums2 of lengths m and n respectively. nums1 and nums2 represent the digits of two numbers. You are also given an integer k.

Create the maximum number of length k <= m + n from digits of the two numbers. The relative order of the digits from the same array must be preserved.

Return an array of the k digits representing the answer.

Example 1:

```
Input: nums1 = [3,4,6,5], nums2 = [9,1,2,5,8,3], k = 5
Output: [9,8,6,5,3]
```

Example 2:

```
Input: nums1 = [6,7], nums2 = [6,0,4], k = 5
Output: [6,7,6,0,4]
```

Example 3:

```
Input: nums1 = [3,9], nums2 = [8,9], k = 3
Output: [9,8,9]
```

Constraints:

- **m == nums1.length**
- **n == nums2.length**
- **1 <= m, n <= 500**
- **0 <= nums1[i], nums2[i] <= 9**
- **1 <= k <= m + n**
- **nums1 and nums2 do not have leading zeros.**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Pick k digits from two arrays (maintaining order) to form the largest possible number. Right?"
# "nums1=[3,4,6,5],nums2=[9,1,2,5,8,3],k=5→[9,8,6,5,3] ✓"
# Try all splits. pick_max uses monotonic stack O(n). merge O(k²) naive. Total O(k*(m+n+k²)).
# Follow-up: if only one array — just pick_max on it.
# Follow-up: optimize merge to O(k log k) with suffix comparison.
```

Monotonic Stack

#1944 Number of Visible People in a Queue

HAR

Open on LeetCode

Array · Stack · Monotonic Stack

Lists: AlgoMaster 600

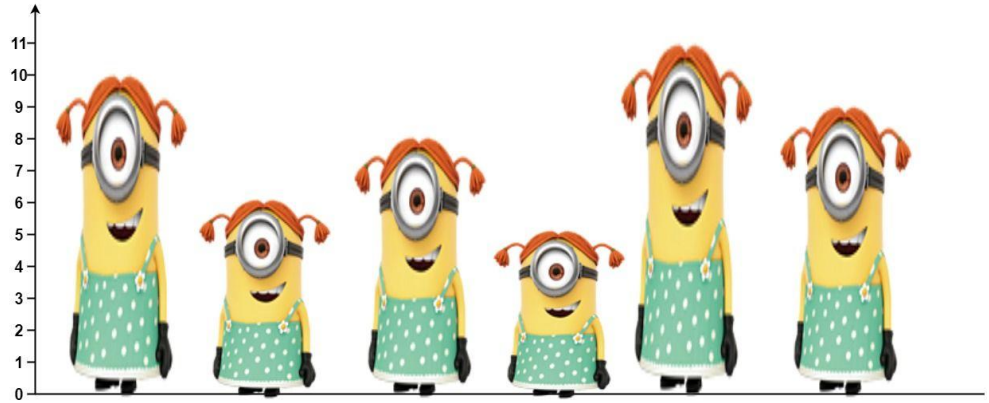
Problem

There are n people standing in a queue, and they numbered from 0 to n - 1 in left to right order. You are given an array heights of distinct integers where heights[i] represents the height of the ith person.

A person can see another person to their right in the queue if everybody in between is shorter than both of them. More formally, the ith person can see the jth person if i < j and min(heights[i], heights[j]) > max(heights[i+1], heights[i+2], ..., heights[j-1]).

Return an array answer of length n where answer[i] is the number of people the ith person can see to their right in the queue.

Example 1:




```
Input: heights = [10,6,8,5,11,9]
Output: [3,1,2,1,1,0]
Explanation:
Person 0 can see person 1, 2, and 4.
Person 1 can see person 2.
Person 2 can see person 3 and 4.
Person 3 can see person 4.
Person 4 can see person 5.
```

Example 2:

```
Input: heights = [5,1,2,3,10]
Output: [4,1,1,1,0]
```

Constraints:

- n == heights.length
- 1 <= n <= 105
- 1 <= heights[i] <= 105
- All the values of heights are unique.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Person i can see person j if everyone between them is shorter than min(h[i],h[j]).
# Return count visible from each position looking right. Right?"
# "heights=[10,6,8,5,11,9]→[3,1,2,1,1,0] ✓"
# Monotonic decreasing stack from right. Pop shorter people (i can see them + first taller). O(n). Space O(n).
# Follow-up: count visible looking left — process left-to-right with same logic.
# Follow-up: if heights can equal — adjust condition carefully (equal height blocks view behind it).
```


Monotonic Queue

Round 6 · Mid · Hard · 2 questions

Monotonic Queue

#862 Shortest Subarray with Sum at Least K

HAR

Open on LeetCode

Array · Binary Search · Queue · Sliding Window · Heap (Priority Queue) · Prefix Sum · Monotonic Queue

Lists: AlgoMaster 600

Problem

Given an integer array nums and an integer k, return the length of the shortest non-empty subarray of nums with a sum of at least k. If there is no such subarray, return -1.

A subarray is a contiguous part of an array.

Example 1:

```
Input: nums = [1], k = 1
Output: 1
```

Example 2:

```
Input: nums = [1,2], k = 4
Output: -1
```

Example 3:

```
Input: nums = [2,-1,2], k = 3
Output: 3
```

Constraints:

- 1 <= nums.length <= 105
- -105 <= nums[i] <= 105
- 1 <= k <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find shortest subarray with sum >= k. Array can have negatives. Right?"
# "nums=[2,-1,2],k=3→3 ✓"
# Prefix sum + monotonic deque. Pop front when sum>=k (update best). Pop back if current prefix smaller. O(n).
# Follow-up: Minimum Size Subarray Sum (LC 209) — only positive values, sliding window O(n).
# Follow-up: count subarrays with sum>=k — different, use sorted prefix sums.
```


Monotonic Queue

#1499 Max Value of Equation

HAR

Open on LeetCode

Array · Queue · Sliding Window · Heap (Priority Queue) · Monotonic Queue

Lists: AlgoMaster 600

Problem

You are given an array points containing the coordinates of points on a 2D plane, sorted by the x-values, where points[i] = [xi, yi] such that xi < xj for all 1 <= i < j <= points.length. You are also given an integer k.

Return the maximum value of the equation yi + yj + |xi - xj| where |xi - xj| <= k and 1 <= i < j <= points.length.

It is guaranteed that there exists at least one pair of points that satisfy the constraint |xi - xj| <= k.

Example 1:

```
Input: points = [[1,3],[2,0],[5,10],[6,-10]], k = 1
Output: 4
Explanation: The first two points satisfy the condition |xi - xj| <= 1 and if we calculate the equation we get 3 + 0 + |1 - 2| =
4. Third and fourth points also satisfy the condition and give a value of 10 + -10 + |5 - 6| = 1.
No other pairs satisfy the condition, so we return the max of 4 and 1.
```

Example 2:

```
Input: points = [[0,0],[3,0],[9,2]], k = 3
Output: 3
Explanation: Only the first two points have an absolute difference of 3 or less in the x-values, and give the value of 0 + 0 +
|0 - 3| = 3.
```

Constraints:

- 2 <= points.length <= 105
- points[i].length == 2
- -108 <= xi, yi <= 108
- 0 <= k <= 2 * 108
- xi < xj for all 1 <= i < j <= points.length
- xi form a strictly increasing sequence.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "points sorted by x. Find max yi+yj+|xi-xj| where xj-xi<=k. Right?"
# "points=[[1,3],[2,0],[5,10],[6,-10]],k=1→4 ✓"
# yi+yj+xj-xi = (yi-xi) + (yj+xj). Maximize (yi-xi) in sliding window of x-distance<=k. O(n). Space O(n).
# Follow-up: if points are not sorted by x — sort first O(n log n).
# Follow-up: Jump Game VI (LC 1696) — same sliding window max DP pattern.
```

Recursion

Round 6 · Mid · Hard · 2 questions

Recursion

#273 Integer to English Words

HAR

Open on LeetCode

Math · String · Recursion

Lists: AlgoMaster 600

Problem

Convert a non-negative integer num to its English words representation.

Example 1:

```
Input: num = 123
Output: "One Hundred Twenty Three"
```

Example 2:

```
Input: num = 12345
Output: "Twelve Thousand Three Hundred Forty Five"
```

Example 3:

```
Input: num = 1234567
Output: "One Million Two Hundred Thirty Four Thousand Five Hundred Sixty Seven"
```

Constraints:

- 0 <= num <= 231 - 1

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Convert non-negative integer to English words. Right?"
# "num=1234567→'One Million Two Hundred Thirty Four Thousand Five Hundred Sixty Seven' ✓"
# Recursive helper for 3-digit chunks. O(log num). Space O(log num).
# Follow-up: words to integer — reverse mapping with parser.
# Follow-up: different language — replace word arrays.
```

Recursion

#761 Special Binary String HAR

Open on LeetCode

String · Divide and Conquer · Sorting

Lists: AlgoMaster 600

Problem

Special binary strings are binary strings with the following two properties:

- **The number of 0's is equal to the number of 1's.**
- **Every prefix of the binary string has at least as many 1's as 0's.**

You are given a special binary string s.

A move consists of choosing two consecutive, non-empty, special substrings of s, and swapping them. Two strings are consecutive if the last character of the first string is exactly one index before the first character of the second string.

Return the lexicographically largest resulting string possible after applying the mentioned operations on the string.

Example 1:

```
Input: s = "11011000"
Output: "11100100"
Explanation: The strings "10" [occuring at s[1]] and "1100" [at s[3]] are swapped.
This is the lexicographically largest string possible after some number of swaps.
```

Example 2:

```
Input: s = "10"
Output: "10"
```

Constraints:

- **1 <= s.length <= 50**
- **s[i] is either '0' or '1'.**
- **s is a special binary string.**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Special binary string: same count of 0s and 1s, every prefix has at least as many 1s.
# Swap any two non-overlapping special substrings. Maximize lexicographic result. Right?"
# "s='11011000'→'11100100' ✓"
# Recursively sort special substrings at each level. O(n² log n). Space O(n).
# Follow-up: if we want minimum lexicographic — sort ascending.
# Follow-up: count valid special strings — Catalan number formula.
```


Merge Sort

Round 6 · Mid · Hard · 2 questions

Merge Sort

#327 Count of Range Sum

HAR

Open on LeetCode

Array · Binary Search · Divide and Conquer · Binary Indexed Tree · Segment Tree · Merge Sort · Ordered Set

Lists: AlgoMaster 600

Problem

Given an integer array nums and two integers lower and upper, return the number of range sums that lie in [lower, upper] inclusive.

Range sum S(i, j) is defined as the sum of the elements in nums between indices i and j inclusive, where i <= j.

Example 1:

```
Input: nums = [-2,5,-1], lower = -2, upper = 2
Output: 3
Explanation: The three ranges are: [0,0], [2,2], and [0,2] and their respective sums are: -2, -1, 2.
```

Example 2:

```
Input: nums = [0], lower = 0, upper = 0
Output: 1
```

Constraints:

- 1 <= nums.length <= 105
- -231 <= nums[i] <= 231 - 1
- -105 <= lower <= upper <= 105
- The answer is guaranteed to fit in a 32-bit integer.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count subarrays with sum in [lower,upper]. Right?"
# "nums=[-2,5,-1],lower=-2,upper=2→3 ✓"
# Sorted prefix sums + binary search. O(n log n). Space O(n).
# Merge sort approach: count pairs crossing midpoint during merge. O(n log n).
# Follow-up: Reverse Pairs (LC 493) — pairs where nums[i]>2*nums[j] with i<j.
```


Merge Sort

#493 Reverse Pairs

HAR

Open on LeetCode

Array · Binary Search · Divide and Conquer · Binary Indexed Tree · Segment Tree · Merge Sort · Ordered Set

Lists: AlgoMaster 600

Problem

Given an integer array nums, return the number of reverse pairs in the array.

A reverse pair is a pair (i, j) where:

- 0 <= i < j < nums.length and
- nums[i] > 2 * nums[j].

Example 1:

```
Input: nums = [1,3,2,3,1]
Output: 2
Explanation: The reverse pairs are:
(1, 4) --> nums[1] = 3, nums[4] = 1, 3 > 2 * 1
(3, 4) --> nums[3] = 3, nums[4] = 1, 3 > 2 * 1
```

Example 2:

```
Input: nums = [2,4,3,5,1]
Output: 3
Explanation: The reverse pairs are:
(1, 4) --> nums[1] = 4, nums[4] = 1, 4 > 2 * 1
(2, 4) --> nums[2] = 3, nums[4] = 1, 3 > 2 * 1
(3, 4) --> nums[3] = 5, nums[4] = 1, 5 > 2 * 1
```

Constraints:

- 1 <= nums.length <= 5 * 104
- -231 <= nums[i] <= 231 - 1

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count pairs (i,j) where i<j and nums[i]>2*nums[j]. Right?"
# "nums=[1,3,2,3,1]→2 ✓ nums=[2,4,3,5,1]→3 ✓"
# Merge sort: during merge, count pairs with condition. O(n log n). Space O(n).
# Follow-up: Count of Range Sum (LC 327) — generalized range condition.
# Follow-up: Count Inversions (general) — same merge sort, condition is nums[i]>nums[j].
```

Backtracking

Round 6 · Mid · Hard · 2 questions

Backtracking

#301 Remove Invalid Parentheses HAR

Open on LeetCode

String · Backtracking · Breadth-First Search

Lists: AlgoMaster 600

Problem

Given a string s that contains parentheses and letters, remove the minimum number of invalid parentheses to make the input string valid.

Return a list of unique strings that are valid with the minimum number of removals. You may return the answer in any order.

Example 1:

```
Input: s = "()())()"
Output: ["(())()","()()()"]
```

Example 2:

```
Input: s = "(a)())()"
Output: ["(a())()","(a)()()"]
```

Example 3:

```
Input: s = ")("
Output: [""]
```

Constraints:

- **1 <= s.length <= 25**
- **s consists of lowercase English letters and parentheses '(' and ')'.**
- **There will be at most 20 parentheses in s.**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Remove minimum invalid parentheses to make the string valid. Return all possibilities. Right?"
# "s='()())()'→['(())()','()()()'] ✓"
        # Count min removals needed
# Count min removals, then backtrack keeping that constraint. O(2^n). Space O(n).
# Follow-up: if we only need one valid string — greedy single pass removal.
# Follow-up: Minimum Remove to Make String Valid (LC 1249) — O(n) greedy approach.
```

Backtracking

#980 Unique Paths III

HAR

Open on LeetCode

Array · Backtracking · Bit Manipulation · Matrix

Lists: AlgoMaster 600

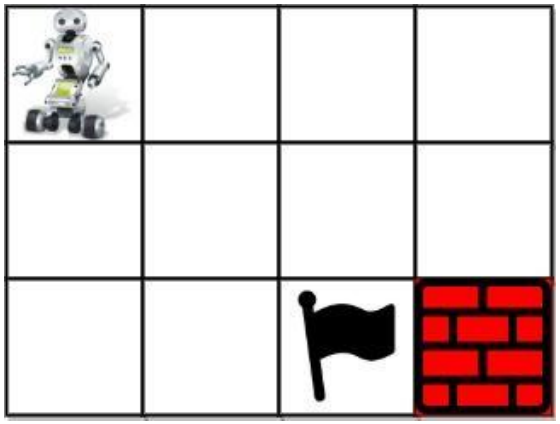
Problem

You are given an m x n integer array grid where grid[i][j] could be:

- 1 representing the starting square. There is exactly one starting square.
- 2 representing the ending square. There is exactly one ending square.
- 0 representing empty squares we can walk over.
- -1 representing obstacles that we cannot walk over.

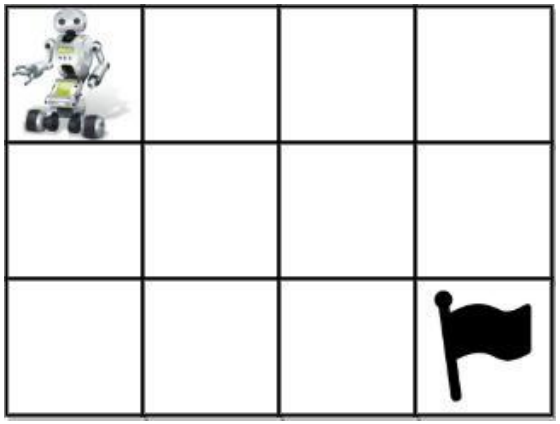
Return the number of 4-directional walks from the starting square to the ending square, that walk over every non-obstacle square exactly once.

Example 1:



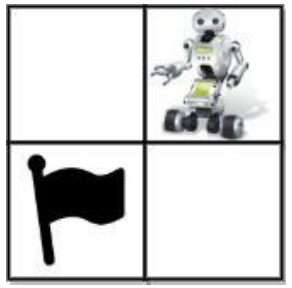
```
Input: grid = [[1,0,0,0],[0,0,0,0],[0,0,2,-1]]
Output: 2
Explanation: We have the following two paths:
1. (0,0),(0,1),(0,2),(0,3),(1,3),(1,2),(1,1),(1,0),(2,0),(2,1),(2,2)
2. (0,0),(1,0),(2,0),(2,1),(1,1),(0,1),(0,2),(0,3),(1,3),(1,2),(2,2)
```

Example 2:




```
Input: grid = [[1,0,0,0],[0,0,0,0],[0,0,0,2]]
Output: 4
Explanation: We have the following four paths:
1. (0,0),(0,1),(0,2),(0,3),(1,3),(1,2),(1,1),(1,0),(2,0),(2,1),(2,2),(2,3)
2. (0,0),(0,1),(1,1),(1,0),(2,0),(2,1),(2,2),(1,2),(0,2),(0,3),(1,3),(2,3)
3. (0,0),(1,0),(2,0),(2,1),(2,2),(1,2),(1,1),(0,1),(0,2),(0,3),(1,3),(2,3)
4. (0,0),(1,0),(2,0),(2,1),(1,1),(0,1),(0,2),(0,3),(1,3),(1,2),(2,2),(2,3)
```

Example 3:



```
Input: grid = [[0,1],[2,0]]
Output: 0
Explanation: There is no path that walks over every empty square exactly once.
Note that the starting and ending square can be anywhere in the grid.
```

Constraints:

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 20
- 1 <= m * n <= 20
- -1 <= grid[i][j] <= 2
- There is exactly one starting cell and one ending cell.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Grid with 0=empty, 1=start, 2=end, -1=obstacle. Walk from 1 to 2 visiting every empty cell exactly once.
# Count distinct paths. Right?"
#
# "grid=[[1,0,0,0],[0,0,0,0],[0,0,2,-1]]: → 2 ✓"

# PHASE 2 — BRUTE FORCE
# "DFS from start. Track visited cells. When reaching end, check if all empty cells visited.
# Time: O(4^(m*n)). Space: O(m*n)."

# PHASE 3 — OPTIMIZE
# "Same backtracking — already optimal for this NP-hard grid problem.
# Count empty cells + start + end to know target visit count."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# grid=[[1,0,0,0],[0,0,0,0],[0,0,2,-1]]: need=11 (10 empty+start+end-obstacle=11).
# DFS explores all paths visiting exactly 11 cells ending at 2. result=2 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(4^(mn)): worst case all cells visited. Space O(mn): recursion depth.
# Count-based termination (visited==need) prunes paths that miss any empty cell.
# Follow-up: Unique Paths I (LC 62) — no obstacles, DP solution O(mn).
# Follow-up: Unique Paths II (LC 63) — with obstacles, DP still works."
```

Tries

Round 6 · Mid · Hard · 3 questions

Tries

#425 Word Squares

HAR

Open on LeetCode

Array · String · Backtracking · Trie

Lists: AlgoMaster 600

Problem

Given an array of unique strings words, return all the word squares you can build from words. The same word from words can be used multiple times. You can return the answer in any order.

A sequence of strings forms a valid word square if the kth row and column read the same string, where 0 <= k < max(numRows, numColumns).

- For example, the word sequence ["ball","area","lead","lady"] forms a word square because each word reads the same both horizontally and vertically.

Example 1:

```
Input: words = ["area","lead","wall","lady","ball"]
Output: [["ball","area","lead","lady"],["wall","area","lead","lady"]]
Explanation:
The output consists of two word squares. The order of output does not matter (just the order of words in each word square matters).
```

Example 2:

```
Input: words = ["abat","baba","atan","atal"]
Output: [["baba","abat","baba","atal"],["baba","abat","baba","atan"]]
Explanation:
The output consists of two word squares. The order of output does not matter (just the order of words in each word square matters).
```

Constraints:

- 1 <= words.length <= 1000
- 1 <= words[i].length <= 4
- All words[i] have the same length.
- words[i] consists of only lowercase English letters.
- All words[i] are unique.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Given a list of words of same length, find all word squares (k×k grid where row i = col i). Right?"
# "words=['area','lead','wall','lady','ball']→[['wall','area','lead','lady'],['ball','area','lead','lady']] ✓"
# Build prefix map once, backtrack filling each row constrained by column prefixes. O(n*26^n). Space O(n²).
# Follow-up: if words can be repeated — allow duplicates in prefix map.
# Follow-up: Valid Word Square (LC 422) — just check, no construction needed.
```

Tries

#472 Concatenated Words HAR

Open on LeetCode

Array · String · Dynamic Programming · Depth-First Search · Trie · Sorting

Lists: AlgoMaster 600

Problem

Given an array of strings words (without duplicates), return all the concatenated words in the given list of words.
A concatenated word is defined as a string that is comprised entirely of at least two shorter words (not necessarily distinct) in the given array.

Example 1:

```
Input: words = ["cat","cats","catsdogcats","dog","dogcatsdog","hippopotamuses","rat","ratcatdogcat"]
Output: ["catsdogcats","dogcatsdog","ratcatdogcat"]
Explanation: "catsdogcats" can be concatenated by "cats", "dog" and "cats";
"dogcatsdog" can be concatenated by "dog", "cats" and "dog";
"ratcatdogcat" can be concatenated by "rat", "cat", "dog" and "cat".
```

Example 2:

```
Input: words = ["cat","dog","catdog"]
Output: ["catdog"]
```

Constraints:

- 1 <= words.length <= 104
- 1 <= words[i].length <= 30
- words[i] consists of only lowercase English letters.
- All the strings of words are unique.
- 1 <= sum(words[i].length) <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find all words that can be formed by concatenating at least 2 shorter words from the list. Right?"
# "words=['cat','cats','catsdogcats','dog','dogcatsdog','hippopotamuses','rat','ratcatdogcat']:
# →['catsdogcats','dogcatsdog','ratcatdogcat'] ✓"
# Sort by length, DP word break checking against shorter words only. O(n²*L²). Space O(n*L).
# Follow-up: Word Break II (LC 140) — return all decompositions for a single word.
# Follow-up: if words can repeat in concatenation — remove the j>0 or i<n constraint.
```


Tries

#1032 Stream of Characters HAR

Open on LeetCode

Array · String · Design · Trie · Data Stream

Lists: AlgoMaster 600

Problem

Design an algorithm that accepts a stream of characters and checks if a suffix of these characters is a string of a given array of strings words.
For example, if words = ["abc", "xyz"] and the stream added the four characters (one by one) 'a', 'x', 'y', and 'z', your algorithm should detect that the suffix "xyz" of the characters "axyz" matches "xyz" from words.
Implement the StreamChecker class:

- StreamChecker(String[] words) Initializes the object with the strings array words.
- boolean query(char letter) Accepts a new character from the stream and returns true if any non-empty suffix from the stream forms a word that is in words.

Example 1:

```
Input
["StreamChecker", "query", "query", "query", "query", "query", "query", "query", "query", "query", "query", "query", "query"]
[[["cd", "f", "kl"]], ["a"], ["b"], ["c"], ["d"], ["e"], ["f"], ["g"], ["h"], ["i"], ["j"], ["k"], ["l"]]
Output
[null, false, false, false, true, false, true, false, false, false, false, false, true]

Explanation
StreamChecker streamChecker = new StreamChecker(["cd", "f", "kl"]);
```

Constraints:

- 1 <= words.length <= 2000
- 1 <= words[i].length <= 200
- words[i] consists of lowercase English letters.
- letter is a lowercase English letter.
- At most 4 * 104 calls will be made to query.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a list of words and a stream of characters, check if any suffix of stream matches a word. Right?"
# "words=['cd','f','kl'], stream: a,b,c,d,f,g,h,i,j,k,l → [F,F,F,T,T,F,F,F,F,F,T] ✓"
        # Build trie on REVERSED words, match reversed stream
# Reversed trie: insert reversed words, check reversed stream suffix. O(L) per query. Space O(total_chars).
# Follow-up: Aho-Corasick automaton — match multiple patterns in a stream in O(n+total_patterns).
# Follow-up: return which words matched — store word indices at '#' nodes.
```

Heaps

Round 6 · Mid · Hard · 1 question

Heaps

#1383 Maximum Performance of a Team

HAR

Open on LeetCode

Array · Greedy · Sorting · Heap (Priority Queue)

Lists: AlgoMaster 600

Problem

You are given two integers n and k and two integer arrays speed and efficiency both of length n. There are n engineers numbered from 1 to n. speed[i] and efficiency[i] represent the speed and efficiency of the ith engineer respectively.

Choose at most k different engineers out of the n engineers to form a team with the maximum performance.

The performance of a team is the sum of its engineers' speeds multiplied by the minimum efficiency among its engineers.

Return the maximum performance of this team. Since the answer can be a huge number, return it modulo 109 + 7.

Example 1:

```
Input: n = 6, speed = [2,10,3,1,5,8], efficiency = [5,4,3,9,7,2], k = 2
Output: 60
Explanation:
We have the maximum performance of the team by selecting engineer 2 (with speed=10 and efficiency=4) and engineer 5 (with
speed=5 and efficiency=7). That is, performance = (10 + 5) * min(4, 7) = 60.
```

Example 2:

```
Input: n = 6, speed = [2,10,3,1,5,8], efficiency = [5,4,3,9,7,2], k = 3
Output: 68
Explanation:
This is the same example as the first but k = 3. We can select engineer 1, engineer 2 and engineer 5 to get the maximum
performance of the team. That is, performance = (2 + 10 + 5) * min(5, 4, 7) = 68.
```

Example 3:

```
Input: n = 6, speed = [2,10,3,1,5,8], efficiency = [5,4,3,9,7,2], k = 4
Output: 72
```

Constraints:

- 1 <= k <= n <= 105
- speed.length == n
- efficiency.length == n
- 1 <= speed[i] <= 105
- 1 <= efficiency[i] <= 108

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Team performance = (sum of speeds) * (min efficiency). Pick at most k engineers. Maximize. Right?"
# "n=6,speed=[2,10,3,1,5,8],efficiency=[5,4,3,9,7,2],k=2→60 (engineers 0+4: (2+5)*7=49? No. 1+4: (10+5)*4=60) ✓"
# Sort by efficiency desc. For each engineer as min-eff, greedily pick k fastest. O(n log k). Space O(k).
# Follow-up: if k=1 — return max(speed[i]*efficiency[i]).
# Follow-up: if we must pick exactly k — track when heap first reaches size k.
```

Two Heaps

Round 6 · Mid · Hard · 2 questions

Two Heaps

#480 Sliding Window Median

HAR

Open on LeetCode

Array · Hash Table · Sliding Window · Heap (Priority Queue)

Lists: AlgoMaster 600

Problem

The median is the middle value in an ordered integer list. If the size of the list is even, there is no middle value. So the median is the mean of the two middle values.

- For examples, if arr = [2,3,4], the median is 3.
- For examples, if arr = [1,2,3,4], the median is (2 + 3) / 2 = 2.5.

You are given an integer array nums and an integer k. There is a sliding window of size k which is moving from the very left of the array to the very right. You can only see the k numbers in the window. Each time the sliding window moves right by one position.

Return the median array for each window in the original array. Answers within 10-5 of the actual value will be accepted.

Example 1:

```
Input: nums = [1,3,-1,-3,5,3,6,7], k = 3
Output: [1.00000,-1.00000,-1.00000,3.00000,5.00000,6.00000]
Explanation:
Window position Median
--------------- -----
[1 3 -1] -3 5 3 6 7 1
1 [3 -1 -3] 5 3 6 7 -1
1 3 [-1 -3 5] 3 6 7 -1
```

Example 2:

```
Input: nums = [1,2,3,4,2,3,1,4,2], k = 3
Output: [2.00000,3.00000,3.00000,3.00000,2.00000,3.00000,2.00000]
```

Constraints:

- 1 <= k <= nums.length <= 105
- -231 <= nums[i] <= 231 - 1

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return median of each sliding window of size k. Right?"
# "nums=[1,3,-1,-3,5,3,6,7],k=3→[1,-1,-1,3,5,6] ✓"
# SortedList: O(n log k) total. Two-heap approach: O(n log k) with lazy deletion.
# Follow-up: Find Median from Data Stream (LC 295) — no sliding, just add elements.
# Follow-up: if k can vary per window — precompute sorted array + binary search.
```

Two Heaps

#502 IPO HAR

Open on LeetCode

Array · Greedy · Sorting · Heap (Priority Queue)

Lists: AlgoMaster 600

Problem

Suppose LeetCode will start its IPO soon. In order to sell a good price of its shares to Venture Capital, LeetCode would like to work on some projects to increase its capital before the IPO. Since it has limited resources, it can only finish at most k distinct projects before the IPO. Help LeetCode design the best way to maximize its total capital after finishing at most k distinct projects.

You are given n projects where the ith project has a pure profit profits[i] and a minimum capital of capital[i] is needed to start it.

Initially, you have w capital. When you finish a project, you will obtain its pure profit and the profit will be added to your total capital.

Pick a list of at most k distinct projects from given projects to maximize your final capital, and return the final maximized capital.

The answer is guaranteed to fit in a 32-bit signed integer.

Example 1:

```
Input: k = 2, w = 0, profits = [1,2,3], capital = [0,1,1]
Output: 4
Explanation: Since your initial capital is 0, you can only start the project indexed 0.
After finishing it you will obtain profit 1 and your capital becomes 1.
With capital 1, you can either start the project indexed 1 or the project indexed 2.
Since you can choose at most 2 projects, you need to finish the project indexed 2 to get the maximum capital.
Therefore, output the final maximized capital, which is 0 + 1 + 3 = 4.
```

Example 2:

```
Input: k = 3, w = 0, profits = [1,2,3], capital = [0,1,2]
Output: 6
```

Constraints:

- 1 <= k <= 105
- 0 <= w <= 109
- n == profits.length
- n == capital.length
- 1 <= n <= 105
- 0 <= profits[i] <= 104
- 0 <= capital[i] <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Pick k projects. Each project has profit and capital requirement. Start with w capital.
# Maximize final capital. Right?"
# "k=2,w=0,profits=[1,2,3],capital=[0,1,1]→4 ✓"
# Greedy: sort by capital, push affordable projects into max-heap. O(n log n). Space O(n).
# Follow-up: if we can invest in each project multiple times — pick best profit each round.
# Follow-up: minimum rounds to reach target capital — binary search + simulation.
```


Intervals

Round 6 · Mid · Hard · 1 question

Intervals

#2402 Meeting Rooms III HAR

Open on LeetCode

Array · Hash Table · Sorting · Heap (Priority Queue) · Simulation

Lists: AlgoMaster 600

Problem

You are given an integer n. There are n rooms numbered from 0 to n - 1.

You are given a 2D integer array meetings where meetings[i] = [starti, endi] means that a meeting will be held during the half-closed time interval [starti, endi). All the values of starti are unique.

Meetings are allocated to rooms in the following manner:

1. Each meeting will take place in the unused room with the lowest number.
2. If there are no available rooms, the meeting will be delayed until a room becomes free. The delayed meeting should have the same duration as the original meeting.
3. When a room becomes unused, meetings that have an earlier original start time should be given the room.

Return the number of the room that held the most meetings. If there are multiple rooms, return the room with the lowest number.

A half-closed interval [a, b) is the interval between a and b including a and not including b.

Example 1:

```
Input: n = 2, meetings = [[0,10],[1,5],[2,7],[3,4]]
Output: 0
Explanation:
- At time 0, both rooms are not being used. The first meeting starts in room 0.
- At time 1, only room 1 is not being used. The second meeting starts in room 1.
- At time 2, both rooms are being used. The third meeting is delayed.
- At time 3, both rooms are being used. The fourth meeting is delayed.
- At time 5, the meeting in room 1 finishes. The third meeting starts in room 1 for the time period [5,10).
```

Example 2:

```
Input: n = 3, meetings = [[1,20],[2,10],[3,5],[4,9],[6,8]]
Output: 1
Explanation:
- At time 1, all three rooms are not being used. The first meeting starts in room 0.
- At time 2, rooms 1 and 2 are not being used. The second meeting starts in room 1.
- At time 3, only room 2 is not being used. The third meeting starts in room 2.
- At time 4, all three rooms are being used. The fourth meeting is delayed.
- At time 5, the meeting in room 2 finishes. The fourth meeting starts in room 2 for the time period [5,10).
```

Constraints:

- 1 <= n <= 100
- 1 <= meetings.length <= 105
- meetings[i].length == 2
- 0 <= starti < endi <= 5 * 105
- All the values of starti are unique.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "n meeting rooms. Book meetings in order of start time. Use lowest-indexed free room.
# If all busy, wait for soonest-free room. Return room with most meetings. Right?"
# "n=2,meetings=[[0,10],[1,5],[2,7],[3,4]]→0 ✓"
# Two heaps: free (min by room index), busy (min by end time). O(m log n). Space O(n).
# Follow-up: if room preferences per meeting — use a priority queue with preference scores.
# Follow-up: Meeting Rooms I/II (LC 252/253) — simpler, no room assignment needed.
```


Data Structure Design

Round 6 · Mid · Hard · 2 questions

Data Structure Design

#715 Range Module HAR

Open on LeetCode

Design · Segment Tree · Ordered Set

Lists: AlgoMaster 600

Problem

A Range Module is a module that tracks ranges of numbers. Design a data structure to track the ranges represented as half-open intervals and query about them.

A half-open interval [left, right) denotes all the real numbers x where left <= x < right.

Implement the RangeModule class:

- RangeModule() Initializes the object of the data structure.
- void addRange(int left, int right) Adds the half-open interval [left, right), tracking every real number in that interval. Adding an interval that partially overlaps with currently tracked numbers should add any numbers in the interval [left, right) that are not already tracked.
- boolean queryRange(int left, int right) Returns true if every real number in the interval [left, right) is currently being tracked, and false otherwise.
- void removeRange(int left, int right) Stops tracking every real number currently being tracked in the half-open interval [left, right).

Example 1:

```
Input
["RangeModule", "addRange", "removeRange", "queryRange", "queryRange", "queryRange"]
[[], [10, 20], [14, 16], [10, 14], [13, 15], [16, 17]]
Output
[null, null, null, true, false, true]

Explanation
RangeModule rangeModule = new RangeModule();
```

Constraints:

- 1 <= left < right <= 109
- At most 104 calls will be made to addRange, queryRange, and removeRange.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "RangeModule: addRange(l,r), queryRange(l,r), removeRange(l,r). Maintain set of tracked ranges. Right?"
# "addRange(10,20),removeRange(14,16),queryRange(10,14)→T,queryRange(13,15)→F ✓"
        # Merge with overlapping/touching ranges
        # Remove merged, add new
# Sorted list of non-overlapping ranges. O(log n) query, O(n) add/remove worst case.
# Follow-up: if only query needed — interval tree for O(log n) stabbing queries.
# Follow-up: find all ranges in query window — return list of intersecting ranges.
```


Data Structure Design

#2296 Design a Text Editor HAR

Open on LeetCode

Linked List · String · Stack · Design · Simulation · Doubly-Linked List

Lists: AlgoMaster 600

Problem

Design a text editor with a cursor that can do the following:

- Add text to where the cursor is.
- Delete text from where the cursor is (simulating the backspace key).
- Move the cursor either left or right.

When deleting text, only characters to the left of the cursor will be deleted. The cursor will also remain within the actual text and cannot be moved beyond it. More formally, we have that 0 <= cursor.position <= currentText.length always holds.

Implement the TextEditor class:

- TextEditor() Initializes the object with empty text.
- void addText(string text) Appends text to where the cursor is. The cursor ends to the right of text.
- int deleteText(int k) Deletes k characters to the left of the cursor. Returns the number of characters actually deleted.
- string cursorLeft(int k) Moves the cursor to the left k times. Returns the last min(10, len) characters to the left of the cursor, where len is the number of characters to the left of the cursor.
- string cursorRight(int k) Moves the cursor to the right k times. Returns the last min(10, len) characters to the left of the cursor, where len is the number of characters to the left of the cursor.

Example 1:

```
Input
["TextEditor", "addText", "deleteText", "addText", "cursorRight", "cursorLeft", "deleteText", "cursorLeft", "cursorRight"]
[[], ["leetcode"], [4], ["practice"], [3], [8], [10], [2], [6]]
Output
[null, null, 4, null, "etpractice", "leet", 4, "", "practi"]

Explanation
TextEditor textEditor = new TextEditor(); // The current text is "|". (The '|' character represents the cursor)
```

Constraints:

- 1 <= text.length, k <= 40
- text consists of lowercase English letters.
- At most 2 * 104 calls in total will be made to addText, deleteText, cursorLeft and cursorRight.

Follow-up: Could you find a solution with time complexity of O(k) per call?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Text editor with cursor: addText(text), deleteText(k)→removed, cursorLeft(k)→last10, cursorRight(k)→last10. Right?"
# "add('leetcode'),delete(4),cursorLeft(3),cursorRight(2)→'et' ✓"
# Two stacks (left/right of cursor). All ops O(k). Space O(n).
# Follow-up: undo/redo — maintain history of operations on a third stack.
# Follow-up: Rope data structure — O(log n) for all ops on very long text.
```

Greedy

Round 6 · Mid · Hard · 4 questions

Greedy

#135 Candy

HAR

Open on LeetCode

Array · Greedy

Lists: AlgoMaster 600

Problem

There are n children standing in a line. Each child is assigned a rating value given in the integer array ratings.

You are giving candies to these children subjected to the following requirements:

- Each child must have at least one candy.
- Children with a higher rating get more candies than their neighbors.

Return the minimum number of candies you need to have to distribute the candies to the children.

Example 1:

```
Input: ratings = [1,0,2]
Output: 5
Explanation: You can allocate to the first, second and third child with 2, 1, 2 candies respectively.
```

Example 2:

```
Input: ratings = [1,2,2]
Output: 4
Explanation: You can allocate to the first, second and third child with 1, 2, 1 candies respectively.
The third child gets 1 candy because it satisfies the above two conditions.
```

Constraints:

- n == ratings.length
- 1 <= n <= 2 * 10^4
- 0 <= ratings[i] <= 2 * 10^4

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Each child gets >= 1 candy. Children with higher rating than neighbors get more.
# Return minimum total candies. Right?"
# "ratings=[1,0,2]→5 (2,1,2) ✓ ratings=[1,2,2]→4 (1,2,1) ✓"
# Two passes: left-to-right (handle increasing), right-to-left (handle decreasing). O(n). Space O(n).
# Follow-up: O(1) space solution — compute analytically by counting ascending/descending runs.
# Follow-up: Distribute Coins in Binary Tree (LC 979) — similar notion of redistribution.
```

Greedy

#757 Set Intersection Size At Least Two HAR

Open on LeetCode

Array · Greedy · Sorting

Lists: AlgoMaster 600

Problem

You are given a 2D integer array intervals where intervals[i] = [starti, endi] represents all the integers from starti to endi inclusively.

A containing set is an array nums where each interval from intervals has at least two integers in nums.

- For example, if intervals = [[1,3], [3,7], [8,9]], then [1,2,4,7,8,9] and [2,3,4,8,9] are containing sets.

Return the minimum possible size of a containing set.

Example 1:

```
Input: intervals = [[1,3],[3,7],[8,9]]
Output: 5
Explanation: let nums = [2, 3, 4, 8, 9].
It can be shown that there cannot be any containing array of size 4.
```

Example 2:

```
Input: intervals = [[1,3],[1,4],[2,5],[3,5]]
Output: 3
Explanation: let nums = [2, 3, 4].
It can be shown that there cannot be any containing array of size 2.
```

Example 3:

```
Input: intervals = [[1,2],[2,3],[2,4],[4,5]]
Output: 5
Explanation: let nums = [1, 2, 3, 4, 5].
It can be shown that there cannot be any containing array of size 4.
```

Constraints:

- 1 <= intervals.length <= 3000
- intervals[i].length == 2
- 0 <= starti < endi <= 108

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "For each interval, at least 2 elements of S must lie in it. Find minimum |S|. Right?"
# "intervals=[[1,3],[1,4],[2,5],[3,5]]→3 ✓"
            # Count how many result elements fall in [l,r]
                    # Find which is missing and add
# Greedy: sort by right endpoint (ties: longer interval first). Add rightmost 2 elements when needed. O(n² ) brute O(n log n) optimal.
# Follow-up: Set Intersection Size At Least k — same greedy, add k elements instead of 2.
# Follow-up: if intervals are very large — coordinate compress first.
```


Greedy

#857 Minimum Cost to Hire K Workers HAR

Open on LeetCode

Array · Greedy · Sorting · Heap (Priority Queue)

Lists: AlgoMaster 600

Problem

There are n workers. You are given two integer arrays quality and wage where quality[i] is the quality of the ith worker and wage[i] is the minimum wage expectation for the ith worker.

We want to hire exactly k workers to form a paid group. To hire a group of k workers, we must pay them according to the following rules:

1. Every worker in the paid group must be paid at least their minimum wage expectation.
2. In the group, each worker's pay must be directly proportional to their quality. This means if a worker's quality is double that of another worker in the group, then they must be paid twice as much as the other worker.

Given the integer k, return the least amount of money needed to form a paid group satisfying the above conditions. Answers within 10-5 of the actual answer will be accepted.

Example 1:

```
Input: quality = [10,20,5], wage = [70,50,30], k = 2
Output: 105.00000
Explanation: We pay 70 to 0th worker and 35 to 2nd worker.
```

Example 2:

```
Input: quality = [3,1,10,10,1], wage = [4,8,2,2,7], k = 3
Output: 30.66667
Explanation: We pay 4 to 0th worker, 13.33333 to 2nd and 3rd workers separately.
```

Constraints:

- n == quality.length == wage.length
- 1 <= k <= n <= 104
- 1 <= quality[i], wage[i] <= 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Worker i: quality[i], minWage[i]. Group must pay ratio >= minWage[i]/quality[i].
# Group must be paid proportionally to quality. Min cost for exactly k workers. Right?"
# "quality=[10,20,5],wage=[70,50,30],k=2→105.0 ✓"
# For each captain (sets ratio), pick k-1 cheapest quality others. Min-heap removes largest quality. O(n log k).
# Follow-up: if k workers must include a specific person — fix them as captain, adjust others.
# Follow-up: if total quality is bounded — DP approach for exact quality constraint.
```

Greedy

#871 Minimum Number of Refueling Stops HAR

Open on LeetCode

Array · Dynamic Programming · Greedy · Heap (Priority Queue)

Lists: AlgoMaster 600

Problem

A car travels from a starting position to a destination which is target miles east of the starting position.

There are gas stations along the way. The gas stations are represented as an array stations where stations[i] = [positioni, fueli] indicates that the ith gas station is positioni miles east of the starting position and has fueli liters of gas.

The car starts with an infinite tank of gas, which initially has startFuel liters of fuel in it. It uses one liter of gas per one mile that it drives. When the car reaches a gas station, it may stop and refuel, transferring all the gas from the station into the car.

Return the minimum number of refueling stops the car must make in order to reach its destination. If it cannot reach the destination, return -1.

Note that if the car reaches a gas station with 0 fuel left, the car can still refuel there. If the car reaches the destination with 0 fuel left, it is still considered to have arrived.

Example 1:

```
Input: target = 1, startFuel = 1, stations = []
Output: 0
Explanation: We can reach the target without refueling.
```

Example 2:

```
Input: target = 100, startFuel = 1, stations = [[10,100]]
Output: -1
Explanation: We can not reach the target (or even the first gas station).
```

Example 3:

```
Input: target = 100, startFuel = 10, stations = [[10,60],[20,30],[30,30],[60,40]]
Output: 2
Explanation: We start with 10 liters of fuel.
We drive to position 10, expending 10 liters of fuel. We refuel from 0 liters to 60 liters of gas.
Then, we drive from position 10 to position 60 (expending 50 liters of fuel),
and refuel from 10 liters to 50 liters of gas. We then drive to and reach the target.
We made 2 refueling stops along the way, so we return 2.
```

Constraints:

- 1 <= target, startFuel <= 109
- 0 <= stations.length <= 500
- 1 <= positioni < positioni+1 < target
- 1 <= fueli < 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Drive from 0 to target. Stations[i]=[pos,fuel]. Start with startFuel. Min stops to reach target. Right?"
# "target=100,startFuel=10,stations=[[10,60],[20,30],[30,30],[60,40]]→2 ✓"
# Greedy: when fuel runs out, retroactively take the largest fuel seen so far. O(n log n). Space O(n).
# Follow-up: if each station can be visited multiple times — same approach, push again after use.
# Follow-up: minimum fuel to not stop — binary search on startFuel.
```


Topological Sort

Round 6 · Mid · Hard · 1 question

Topological Sort

#1203 Sort Items by Groups Respecting Dependencies HAR

Open on LeetCode

Depth-First Search · Breadth-First Search · Graph Theory · Topological Sort

Lists: AlgoMaster 600

Problem

There are n items each belonging to zero or one of m groups where group[i] is the group that the i-th item belongs to and it's equal to -1 if the i-th item belongs to no group. The items and the groups are zero indexed. A group can have no item belonging to it.

Return a sorted list of the items such that:

- **The items that belong to the same group are next to each other in the sorted list.**
- **There are some relations between these items where beforeItems[i] is a list containing all the items that should come before the i-th item in the sorted array (to the left of the i-th item).**

Return any solution if there is more than one solution and return an empty list if there is no solution.

Example 1:

Item	Group	Before
0	-1	
1	-1	6
2	1	5
3	0	6
4	0	3, 6
5	1	
6	0	
7	-1	

```
Input: n = 8, m = 2, group = [-1,-1,1,0,0,1,0,-1], beforeItems = [[],[6],[5],[6],[3,6],[],[],[]]
Output: [6,3,4,1,5,2,0,7]
```

Example 2:

```
Input: n = 8, m = 2, group = [-1,-1,1,0,0,1,0,-1], beforeItems = [[],[6],[5],[6],[3],[],[4],[]]
Output: []
Explanation: This is the same as example 1 except that 4 needs to be before 6 in the sorted list.
```

Constraints:

- **1 <= m <= n <= 3 * 10^4**
- **group.length == beforeItems.length == n**
- **-1 <= group[i] <= m - 1**
- **0 <= beforeItems[i].length <= n - 1**
- **0 <= beforeItems[i][j] <= n - 1**
- **i != beforeItems[i][j]**
- **beforeItems[i] does not contain duplicates elements.**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Items belong to groups. DAG dependencies between items. Sort so deps come first and groups together. Right?"
# "n=8,m=2,group=[-1,-1,1,0,0,1,0,-1],beforeItems=[[],[6],[5],[6],[3,6],[],[],[]] →[6,3,4,1,5,2,0,7] ✓"
        # Assign ungrouped items unique groups
        # Build item graph and group graph
        # Topological sort items
        # Group items by group, topological sort groups
# Two-level topological sort (items + groups). O(V+E). Space O(V+E).
# Follow-up: Alien Dictionary (LC 269) — single-level topological sort on characters.
# Follow-up: if groups can have dependencies within themselves — still works (same algo).
```


Union Find

Round 6 · Mid · Hard · 1 question

Union Find

#924 Minimize Malware Spread

HAR

Open on LeetCode

Array · Hash Table · Depth-First Search · Breadth-First Search · Union-Find · Graph Theory

Lists: AlgoMaster 600

Problem

You are given a network of n nodes represented as an n x n adjacency matrix graph, where the ith node is directly connected to the jth node if graph[i][j] == 1.

Some nodes initial are initially infected by malware. Whenever two nodes are directly connected, and at least one of those two nodes is infected by malware, both nodes will be infected by malware. This spread of malware will continue until no more nodes can be infected in this manner.

Suppose M(initial) is the final number of nodes infected with malware in the entire network after the spread of malware stops. We will remove exactly one node from initial.

Return the node that, if removed, would minimize M(initial). If multiple nodes could be removed to minimize M(initial), return such a node with the smallest index.

Note that if a node was removed from the initial list of infected nodes, it might still be infected later due to the malware spread.

Example 1:

```
Input: graph = [[1,1,0],[1,1,0],[0,0,1]], initial = [0,1]
Output: 0
```

Example 2:

```
Input: graph = [[1,0,0],[0,1,0],[0,0,1]], initial = [0,2]
Output: 0
```

Example 3:

```
Input: graph = [[1,1,1],[1,1,1],[1,1,1]], initial = [1,2]
Output: 1
```

Constraints:

- n == graph.length
- n == graph[i].length
- 2 <= n <= 300
- graph[i][j] is 0 or 1.
- graph[i][j] == graph[j][i]
- graph[i][i] == 1
- 1 <= initial.length <= n
- 0 <= initial[i] <= n - 1
- All the integers in initial are unique.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Graph where malware spreads through connections. Remove one initially infected node to minimize spread. Right?"
# "graph=[[1,1,0],[1,1,0],[0,0,1]],initial=[0,1]→0 ✓ (both 0 and 1 are in same component)"
# Remove node that is the only infected in its component and that component is largest. O(n²). Space O(n).
# Follow-up: Minimize Malware Spread II (LC 928) — remove one, infected can't spread through it.
# Follow-up: remove k nodes — NP-hard in general; greedy approximation.
```


Minimum Spanning Tree

Round 6 · Mid · Hard · 3 questions

Minimum Spanning Tree

#1168 Optimize Water Distribution in a Village

HAR

Open on LeetCode

Union-Find · Graph Theory · Heap (Priority Queue) · Minimum Spanning Tree

Lists: AlgoMaster 600

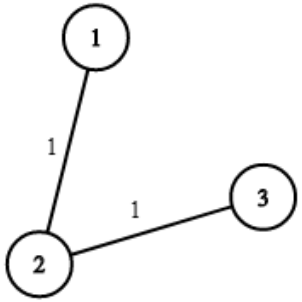
Problem

There are n houses in a village. We want to supply water for all the houses by building wells and laying pipes.

For each house i, we can either build a well inside it directly with cost wells[i - 1] (note the -1 due to 0-indexing), or pipe in water from another well to it. The costs to lay pipes between houses are given by the array pipes where each pipes[j] = [house1j, house2j, costj] represents the cost to connect house1j and house2j together using a pipe. Connections are bidirectional, and there could be multiple valid connections between the same two houses with different costs.

Return the minimum total cost to supply water to all houses.

Example 1:




```
Input: n = 3, wells = [1,2,2], pipes = [[1,2,1],[2,3,1]]
Output: 3
Explanation: The image shows the costs of connecting houses using pipes.
The best strategy is to build a well in the first house with cost 1 and connect the other houses to it with cost 2 so the total cost is 3.
```

Example 2:

```
Input: n = 2, wells = [1,1], pipes = [[1,2,1],[1,2,2]]
Output: 2
Explanation: We can supply water with cost two using one of the three options:
Option 1:
- Build a well inside house 1 with cost 1.
- Build a well inside house 2 with cost 1.
The total cost will be 2.
Option 2:
```

Constraints:

- 2 <= n <= 104
- wells.length == n
- 0 <= wells[i] <= 105
- 1 <= pipes.length <= 104
- pipes[j].length == 3
- 1 <= house1j, house2j <= n
- 0 <= costj <= 105
- house1j != house2j

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "n houses. wells[i]=cost to build well at house i. pipes connecting houses with cost.
# Find min cost to supply water to all houses. Right?"
# "n=3,wells=[1,2,2],pipes=[[1,2,1],[2,3,1]]→3 ✓"
# Add virtual node 0 connected to each house with well cost. MST (Kruskal). O(E log E). Space O(n).
# Follow-up: if each house needs a separate water source — NP-hard optimization.
# Follow-up: Connecting Cities With Minimum Cost (LC 1135) — same Kruskal without virtual node.
```


Minimum Spanning Tree

#1489 Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree

HAR

Open on LeetCode

Union-Find · Graph Theory · Sorting · Minimum Spanning Tree · Strongly Connected Component

Lists: AlgoMaster 600

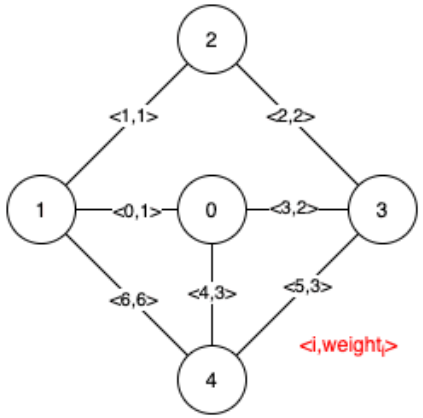
Problem

Given a weighted undirected connected graph with n vertices numbered from 0 to n - 1, and an array edges where edges[i] = [ai, bi, weighti] represents a bidirectional and weighted edge between nodes ai and bi. A minimum spanning tree (MST) is a subset of the graph's edges that connects all vertices without cycles and with the minimum possible total edge weight.

Find all the critical and pseudo-critical edges in the given graph's minimum spanning tree (MST). An MST edge whose deletion from the graph would cause the MST weight to increase is called a critical edge. On the other hand, a pseudo-critical edge is that which can appear in some MSTs but not all.

Note that you can return the indices of the edges in any order.

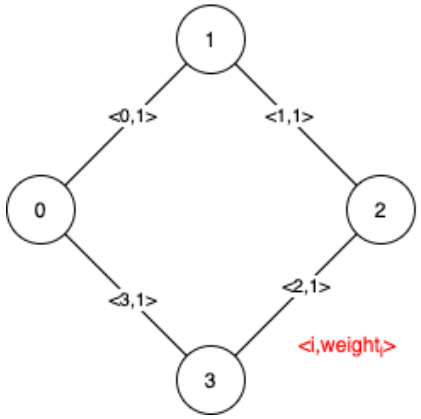
Example 1:




```
Input: n = 5, edges = [[0,1,1],[1,2,1],[2,3,2],[0,3,2],[0,4,3],[3,4,3],[1,4,6]]
Output: [[0,1],[2,3,4,5]]
Explanation: The figure above describes the graph.
The following figure shows all the possible MSTs:

Notice that the two edges 0 and 1 appear in all MSTs, therefore they are critical edges, so we return them in the first list of the output.
The edges 2, 3, 4, and 5 are only part of some MSTs, therefore they are considered pseudo-critical edges. We add them to the second list of the output.
```

Example 2:




```
Input: n = 4, edges = [[0,1,1],[1,2,1],[2,3,1],[0,3,1]]
Output: [[],[0,1,2,3]]
Explanation: We can observe that since all 4 edges have equal weight, choosing any 3 edges from the given 4 will yield an MST.
Therefore all 4 edges are pseudo-critical.
```

Constraints:

- 2 <= n <= 100
- 1 <= edges.length <= min(200, n * (n - 1) / 2)
- edges[i].length == 3
- 0 <= ai < bi < n
- 1 <= weighti <= 1000
- All pairs (ai, bi) are distinct.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Find edges that must be in every MST (critical) and those that appear in at least one MST (pseudo-critical). Right?"
# "n=5,edges=[[0,1,1],[1,2,1],[2,3,2],[0,3,2],[0,4,3],[3,4,3],[1,4,6]]→[[0,1],[2,3,4,5]] ✓"
        # Same approach — already optimal for this problem
# For each edge: skip it (if MST cost increases → critical); force it (if MST cost unchanged → pseudo). O(E² α(V)).
# Follow-up: if only critical edges needed — Matroid intersection approach O(E² log E).
# Follow-up: dynamic MST updates — link-cut trees for O(log n) per edge insertion/deletion.
```


Minimum Spanning Tree

#3600 Maximize Spanning Tree Stability with Upgrades

HAR

Open on LeetCode

Binary Search · Greedy · Union-Find · Graph Theory · Minimum Spanning Tree

Lists: AlgoMaster 600

Problem

You are given an integer n, representing n nodes numbered from 0 to n - 1 and a list of edges, where edges[i] = [ui, vi, si, musti]:

- ui and vi indicates an undirected edge between nodes ui and vi.
- si is the strength of the edge.
- musti is an integer (0 or 1). If musti == 1, the edge must be included in the spanning tree. These edges cannot be upgraded.

You are also given an integer k, the maximum number of upgrades you can perform. Each upgrade doubles the strength of an edge, and each eligible edge (with musti == 0) can be upgraded at most once.

The stability of a spanning tree is defined as the minimum strength score among all edges included in it.

Return the maximum possible stability of any valid spanning tree. If it is impossible to connect all nodes, return -1.

Note: A spanning tree of a graph with n nodes is a subset of the edges that connects all nodes together (i.e. the graph is connected) without forming any cycles, and uses exactly n - 1 edges.

Example 1:

Input: n = 3, edges = [[0,1,2,1],[1,2,3,0]], k = 1

Output: 2

Explanation:

- Edge [0,1] with strength = 2 must be included in the spanning tree.
- Edge [1,2] is optional and can be upgraded from 3 to 6 using one upgrade.
- The resulting spanning tree includes these two edges with strengths 2 and 6.
- The minimum strength in the spanning tree is 2, which is the maximum possible stability.

Example 2:

Input: n = 3, edges = [[0,1,4,0],[1,2,3,0],[0,2,1,0]], k = 2

Output: 6

Explanation:

- Since all edges are optional and up to k = 2 upgrades are allowed.
- Upgrade edges [0,1] from 4 to 8 and [1,2] from 3 to 6.
- The resulting spanning tree includes these two edges with strengths 8 and 6.
- The minimum strength in the tree is 6, which is the maximum possible stability.

Example 3:

Input: n = 3, edges = [[0,1,1,1],[1,2,1,1],[2,0,1,1]], k = 0

Output: -1

Explanation:

- All edges are mandatory and form a cycle, which violates the spanning tree property of acyclicity. Thus, the answer is -1.

Constraints:

- 2 <= n <= 105
- 1 <= edges.length <= 105
- edges[i] = [ui, vi, si, musti]
- 0 <= ui, vi < n
- ui != vi
- 1 <= si <= 105
- musti is either 0 or 1.
- 0 <= k <= n
- There are no duplicate edges.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Given a weighted graph and k upgrade budget, upgrading an edge doubles its weight. Find max MST stability (min edge in MST). Right?"
# "n=3, edges=[[0,1,3],[1,2,5],[0,2,2]], k=1 → 6 (upgrade [1,2,5]→10, MST uses 3+10, min=3? Or upgrade [0,1,3]→6, MST min=6) ✓"
        # Binary search on answer: can we achieve min_edge >= mid?
        # For a given mid, upgrade edges whose weight<mid to >=mid (cost 1 each if doubled>=mid)
        # Then check if we can form an MST using only edges >= mid
            # Kruskal MST, find min edge
        # Binary search: can min MST edge be >= mid?
            # Upgrade edges that become >= mid when doubled (i.e., w*2>=mid → w>=mid/2)
            # and original w<mid
            # Check if enough edges >= mid exist to connect graph
# Binary search on stability + greedy upgrade check. O(E log(max_w)). Space O(V).
# Follow-up: if upgrades multiply by k instead of 2 — same binary search, adjust condition.
```

```
# Follow-up: find which edges to upgrade — track which edges are upgraded in the binary search.
```


Shortest Path

Round 6 · Mid · Hard · 2 questions

Shortest Path

#1368 Minimum Cost to Make at Least One Valid Path in a Grid

HAR

Open on LeetCode

Array · Breadth-First Search · Graph Theory · Heap (Priority Queue) · Matrix · Shortest Path

Lists: AlgoMaster 600

Problem

Given an m x n grid. Each cell of the grid has a sign pointing to the next cell you should visit if you are currently in this cell. The sign of grid[i][j] can be:

- 1 which means go to the cell to the right. (i.e go from grid[i][j] to grid[i][j + 1])
- 2 which means go to the cell to the left. (i.e go from grid[i][j] to grid[i][j - 1])
- 3 which means go to the lower cell. (i.e go from grid[i][j] to grid[i + 1][j])
- 4 which means go to the upper cell. (i.e go from grid[i][j] to grid[i - 1][j])

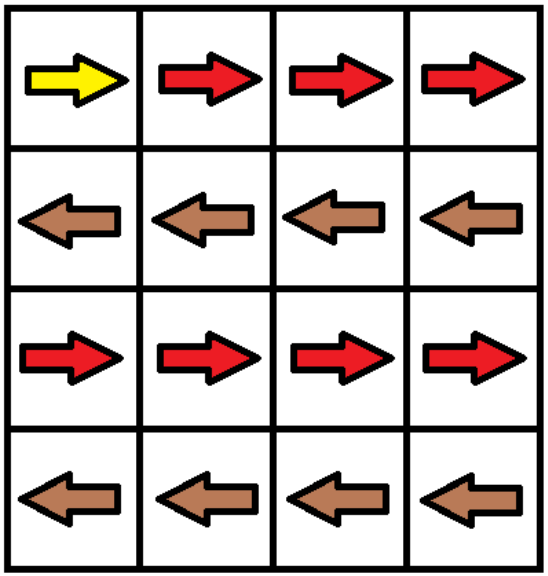
Notice that there could be some signs on the cells of the grid that point outside the grid.

You will initially start at the upper left cell (0, 0). A valid path in the grid is a path that starts from the upper left cell (0, 0) and ends at the bottom-right cell (m - 1, n - 1) following the signs on the grid. The valid path does not have to be the shortest.

You can modify the sign on a cell with cost = 1. You can modify the sign on a cell one time only.

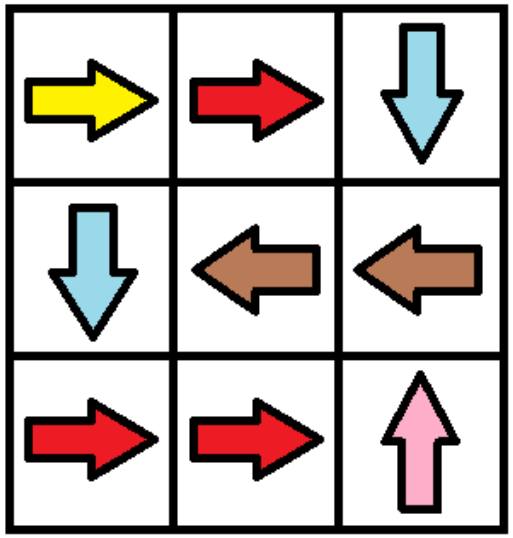
Return the minimum cost to make the grid have at least one valid path.

Example 1:



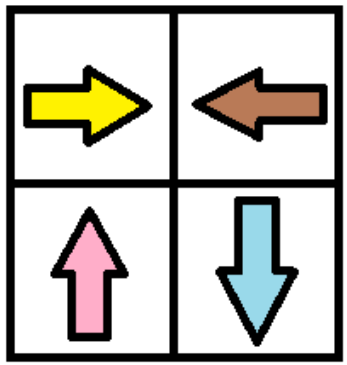
```
Input: grid = [[1,1,1,1],[2,2,2,2],[1,1,1,1],[2,2,2,2]]
Output: 3
Explanation: You will start at point (0, 0).
The path to (3, 3) is as follows. (0, 0) --> (0, 1) --> (0, 2) --> (0, 3) change the arrow to down with cost = 1 --> (1, 3) -->
(1, 2) --> (1, 1) --> (1, 0) change the arrow to down with cost = 1 --> (2, 0) --> (2, 1) --> (2, 2) --> (2, 3) change the arrow
to down with cost = 1 --> (3, 3)
The total cost = 3.
```

Example 2:



```
Input: grid = [[1,1,3],[3,2,2],[1,1,4]]
Output: 0
Explanation: You can follow the path from (0, 0) to (2, 2).
```

Example 3:



```
Input: grid = [[1,2],[4,3]]
Output: 1
```

Constraints:

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 100
- 1 <= grid[i][j] <= 4

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Grid with arrows 1=right,2=left,3=down,4=up. Follow arrow for free, redirect for cost 1.
# Find min cost to reach (m-1,n-1). Right?"
# "grid=[[1,1,1,1],[2,2,2,2],[1,1,1,1],[2,2,2,2]]→3 ✓"
# 0-1 BFS: following arrow costs 0 (appendleft), redirecting costs 1 (append). O(mn). Space O(mn).
# Follow-up: if redirection costs vary — use Dijkstra instead of 0-1 BFS.
# Follow-up: Find Minimum Time to Reach Last Room (LC 3341) — similar graph model.
```


Shortest Path

#2203 Minimum Weighted Subgraph With the Required Paths HAR

Open on LeetCode

Graph Theory · Heap (Priority Queue) · Shortest Path

Lists: AlgoMaster 600

Problem

You are given an integer n denoting the number of nodes of a weighted directed graph. The nodes are numbered from 0 to n - 1.

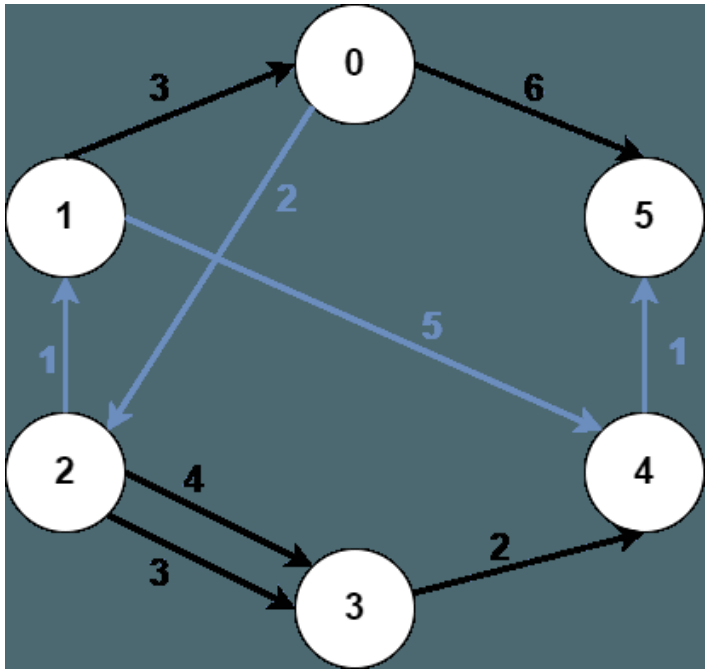
You are also given a 2D integer array edges where edges[i] = [fromi, toi, weighti] denotes that there exists a directed edge from fromi to toi with weight weighti.

Lastly, you are given three distinct integers src1, src2, and dest denoting three distinct nodes of the graph.

Return the minimum weight of a subgraph of the graph such that it is possible to reach dest from both src1 and src2 via a set of edges of this subgraph. In case such a subgraph does not exist, return -1.

A subgraph is a graph whose vertices and edges are subsets of the original graph. The weight of a subgraph is the sum of weights of its constituent edges.

Example 1:




```
Input: n = 6, edges = [[0,2,2],[0,5,6],[1,0,3],[1,4,5],[2,1,1],[2,3,3],[2,3,4],[3,4,2],[4,5,1]], src1 = 0, src2 = 1, dest = 5
Output: 9
Explanation:
The above figure represents the input graph.
The blue edges represent one of the subgraphs that yield the optimal answer.
Note that the subgraph [[1,0,3],[0,5,6]] also yields the optimal answer. It is not possible to get a subgraph with less weight satisfying all the constraints.
```

Example 2:

```
Input: n = 3, edges = [[0,1,1],[2,1,1]], src1 = 0, src2 = 1, dest = 2
Output: -1
Explanation:
The above figure represents the input graph.
It can be seen that there does not exist any path from node 1 to node 2, hence there are no subgraphs satisfying all the
constraints.
```

Constraints:

- 3 <= n <= 105
- 0 <= edges.length <= 105
- edges[i].length == 3
- 0 <= fromi, toi, src1, src2, dest <= n - 1
- fromi != toi
- src1, src2, and dest are pairwise distinct.
- 1 <= weight[i] <= 105

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Find minimum weight subgraph where src1 and src2 can both reach dest. Right?"
# "n=6,edges=[[0,2,2],[0,5,6],[1,0,3],[1,4,5],[2,1,1],[2,3,3],[2,3,4],[3,4,2],[4,5,1]],src1=0,src2=1,dest=5→9 ✓"
# Run Dijkstra from src1, src2 (fwd) and from dest (reversed). Meeting point minimizes sum. O((V+E) log V).
# Follow-up: 3 sources — 3 forward Dijkstras + 1 from dest.
# Follow-up: if edge weights are 0/1 — use 0-1 BFS for O(V+E).
```


Round 7

Low Priority · Easy
6 questions · 4 patterns

- ☐ 1-D DP #509 ↗
- ☐ 2D Grid DP #118 ↗
- ☐ Maths / Geometry #168 #263 #1071 ↗
- ☐ String Matching #459 ↗

1-D DP

Round 7 · Low · Easy · 1 question

1-D DP

#509 Fibonacci Number EAS

Open on LeetCode

Math · Dynamic Programming · Recursion · Memoization

Lists: AlgoMaster 600

Problem

The Fibonacci numbers, commonly denoted F(n) form a sequence, called the Fibonacci sequence, such that each number is the sum of the two preceding ones, starting from 0 and 1. That is,

```
F(0) = 0, F(1) = 1
F(n) = F(n - 1) + F(n - 2), for n > 1.
```

Given n, calculate F(n).

Example 1:

```
Input: n = 2
Output: 1
Explanation: F(2) = F(1) + F(0) = 1 + 0 = 1.
```

Example 2:

```
Input: n = 3
Output: 2
Explanation: F(3) = F(2) + F(1) = 1 + 1 = 2.
```

Example 3:

```
Input: n = 4
Output: 3
Explanation: F(4) = F(3) + F(2) = 2 + 1 = 3.
```

Constraints:

- 0 <= n <= 30

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "F(0)=0,F(1)=1,F(n)=F(n-1)+F(n-2). Return F(n). Right?"
# "n=4→3 ✓ n=10→55 ✓"
# O(n) time, O(1) space. Matrix exponentiation gives O(log n). Space O(1).
# Follow-up: Climbing Stairs (LC 70) — same recurrence, different base cases.
# Follow-up: nth Tribonacci (LC 1137) — three terms instead of two.
```

2D Grid DP

Round 7 · Low · Easy · 1 question

2D Grid DP

#118 Pascal's Triangle

EAS

Open on LeetCode

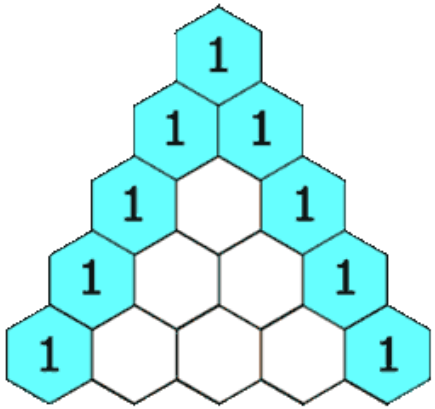
Array · Dynamic Programming

Lists: AlgoMaster 600

Problem

Given an integer numRows, return the first numRows of Pascal's triangle.

In Pascal's triangle, each number is the sum of the two numbers directly above it as shown:



Example 1:

```
Input: numRows = 5
Output: [[1],[1,1],[1,2,1],[1,3,3,1],[1,4,6,4,1]]
```

Example 2:

```
Input: numRows = 1
Output: [[1]]
```

Constraints:

- 1 <= numRows <= 30

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Return the first numRows of Pascal's triangle. Right?"
# "numRows=5→[[1],[1,1],[1,2,1],[1,3,3,1],[1,4,6,4,1]] ✓"
# Build each row from previous. O(n²). Space O(n²).
# Follow-up: Pascal's Triangle II (LC 119) — return only the kth row using O(k) space.
# Follow-up: find the kth element without generating the triangle — use C(row, col) = row!/(col!(row-col)!).
```

Maths / Geometry

Round 7 · Low · Easy · 3 questions

Maths / Geometry

#168 Excel Sheet Column Title EAS

Open on LeetCode

Math · String

Lists: AlgoMaster 600

Problem

Given an integer columnNumber, return its corresponding column title as it appears in an Excel sheet.

For example:

```
A -> 1
B -> 2
C -> 3
...
Z -> 26
AA -> 27
AB -> 28
...
```

Example 1:

```
Input: columnNumber = 1
Output: "A"
```

Example 2:

```
Input: columnNumber = 28
Output: "AB"
```

Example 3:

```
Input: columnNumber = 701
Output: "ZY"
```

Constraints:

- 1 <= columnNumber <= 231 - 1

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Convert column number to Excel title. 1→A, 26→Z, 27→AA, 28→AB. Right?"
# "columnNumber=1→'A' ✓ 28→'AB' ✓ 701→'ZY' ✓"
# n-=1 before mod because it's 1-indexed not 0-indexed. O(log_{26}(n)). Space O(log n).
# Follow-up: Excel Sheet Column Number (LC 171) — reverse mapping.
# Follow-up: Base 26 but 0-indexed (a-z) — just chr(ord('a')+n%26) without the n-=1.
```

Maths / Geometry

#263 Ugly Number

EAS

Open on LeetCode

Math

Lists: AlgoMaster 600

Problem

An ugly number is a positive integer which does not have a prime factor other than 2, 3, and 5.
Given an integer n, return true if n is an ugly number.

Example 1:

```
Input: n = 6
Output: true
Explanation: 6 = 2 × 3
```

Example 2:

```
Input: n = 1
Output: true
Explanation: 1 has no prime factors.
```

Example 3:

```
Input: n = 14
Output: false
Explanation: 14 is not ugly since it includes the prime factor 7.
```

Constraints:

- -231 <= n <= 231 - 1

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "An ugly number has only prime factors 2, 3, 5. Return true if n is ugly. 1 is ugly. Right?"
# "n=6→true(2×3) ✓ n=14→false(2×7) ✓ n=0→false ✓"
# Divide out all factors of 2,3,5. If 1 remains, it's ugly. O(log n). Space O(1).
# Follow-up: Ugly Number II (LC 264) — find nth ugly number; DP with 3 pointers.
# Follow-up: Super Ugly Number (LC 313) — primes generalized to any list.
```


Maths / Geometry

#1071 Greatest Common Divisor of Strings

EAS

Open on LeetCode

Math · String

Lists: AlgoMaster 600

Problem

For two strings s and t, we say "t divides s" if and only if s = t + t + t + ... + t + t (i.e., t is concatenated with itself one or more times).
Given two strings str1 and str2, return the largest string x such that x divides both str1 and str2.
Example 1:
Input: str1 = "ABCABC", str2 = "ABC"
Output: "ABC"
Example 2:
Input: str1 = "ABABAB", str2 = "ABAB"
Output: "AB"
Example 3:
Input: str1 = "LEET", str2 = "CODE"
Output: ""
Example 4:
Input: str1 = "AAAAAB", str2 = "AAA"
Output: ""
Constraints:

- 1 <= str1.length, str2.length <= 1000
- str1 and str2 consist of English uppercase letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the largest string t that divides both str1 and str2 (str = t+t+...+t). Right?"
# "str1='ABCABC',str2='ABC'→'ABC' ✓ str1='LEET',str2='CODE'→'' ✓"
# If concatenations equal, a GCD string exists with length = gcd(len1,len2). O(n+m). Space O(n+m).
# Follow-up: GCD of Strings with k strings — reduce pairwise.
# Follow-up: if strings have different alphabets — return empty (different chars can't divide).
```

String Matching

Round 7 · Low · Easy · 1 question

String Matching

#459 Repeated Substring Pattern EAS

Open on LeetCode

String · String Matching

Lists: AlgoMaster 600

Problem

Given a string s, check if it can be constructed by taking a substring of it and appending multiple copies of the substring together.

Example 1:

```
Input: s = "abab"
Output: true
Explanation: It is the substring "ab" twice.
```

Example 2:

```
Input: s = "aba"
Output: false
```

Example 3:

```
Input: s = "abcabcabcabc"
Output: true
Explanation: It is the substring "abc" four times or the substring "abcabc" twice.
```

Constraints:

- 1 <= s.length <= 104
- s consists of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Check if s can be formed by repeating a substring. Right?"
# "s='abab'→true ('ab'×2) ✓ s='aba'→false ✓"
# Trick: if s is a repeated pattern, it appears in (s+s)[1:-1]. O(n). Space O(n).
# KMP failure function approach: O(n) time, O(n) space, no string duplication.
# Follow-up: find the shortest repeating pattern — return s[:n//gcd_period].
```

Round 8

Low Priority · Medium
42 questions · 15 patterns

- [ ] Bucket Sort #164 #451 ↗
- [ ] 1-D DP #343 #646 #740 #1262 ↗
- [ ] 0/1 Knapsack #474 #1049 ↗
- [ ] Unbounded Knapsack #279 #983 ↗
- [ ] Longest Increasing Subsequence (LIS) #673 #1027 #1048 ↗
- [ ] 2D Grid DP #63 #120 #931 #1277 #1937 ↗
- [ ] String DP #1653 ↗
- [ ] Tree / Graph DP #95 #337 #1976 ↗
- [ ] Bitmask DP #698 #1066 #1986 #2305 ↗
- [ ] Digit DP #357 ↗
- [ ] Probability DP #688 #808 #837 ↗
- [ ] State Machine DP #714 ↗
- [ ] Maths / Geometry #172 #204 #264 #593 #611 #939 #963 #1922 ↗
- [ ] String Matching #686 #1698 ↗
- [ ] Binary Indexed Tree / Segment Tree #308 ↗

Bucket Sort

Round 8 · Low · Medium · 2 questions

Bucket Sort

#164 Maximum Gap

MED

Open on LeetCode

Array · Sorting · Bucket Sort · Radix Sort

Lists: AlgoMaster 600

Problem

Given an integer array nums, return the maximum difference between two successive elements in its sorted form. If the array contains less than two elements, return 0.

You must write an algorithm that runs in linear time and uses linear extra space.

Example 1:

```
Input: nums = [3,6,9,1]
Output: 3
Explanation: The sorted form of the array is [1,3,6,9], either (3,6) or (6,9) has the maximum difference 3.
```

Example 2:

```
Input: nums = [10]
Output: 0
Explanation: The array contains less than 2 elements, therefore return 0.
```

Constraints:

- 1 <= nums.length <= 105
- 0 <= nums[i] <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the maximum gap between successive elements in sorted form. Linear time. Right?"
# "nums=[3,6,9,1]→3 ✓ nums=[10]→0 ✓"
# Pigeonhole: max gap >= ceil((mx-mn)/(n-1)) so gaps within buckets are smaller. O(n). Space O(n).
# Follow-up: if duplicates allowed — same approach, max gap still works.
# Follow-up: minimum gap — sort and find minimum consecutive difference.
```


Bucket Sort

#451 Sort Characters By Frequency

MED

Open on LeetCode

Hash Table · String · Sorting · Heap (Priority Queue) · Bucket Sort · Counting

Lists: AlgoMaster 600

Problem

Given a string s, sort it in decreasing order based on the frequency of the characters. The frequency of a character is the number of times it appears in the string.

Return the sorted string. If there are multiple answers, return any of them.

Example 1:

```
Input: s = "tree"
Output: "eert"
Explanation: 'e' appears twice while 'r' and 't' both appear once.
So 'e' must appear before both 'r' and 't'. Therefore "eetr" is also a valid answer.
```

Example 2:

```
Input: s = "cccaaa"
Output: "aaaccc"
Explanation: Both 'c' and 'a' appear three times, so both "cccaaa" and "aaaccc" are valid answers.
Note that "cacaca" is incorrect, as the same characters must be together.
```

Example 3:

```
Input: s = "Aabb"
Output: "bbAa"
Explanation: "bbaA" is also a valid answer, but "Aabb" is incorrect.
Note that 'A' and 'a' are treated as two different characters.
```

Constraints:

- 1 <= s.length <= 5 * 105
- s consists of uppercase and lowercase English letters and digits.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Sort characters by decreasing frequency. Ties can be in any order. Right?"
# "s='tree'→'eert' or 'eetr' ✓ s='cccaaa'→'cccaaa' or 'aaaccc' ✓"
# Bucket sort by frequency. O(n). Space O(n).
# Follow-up: Top K Frequent Elements (LC 347) — same bucket sort idea, return top k.
# Follow-up: if characters have weights — multiply weight×count for sort key.
```

1-D DP

Round 8 · Low · Medium · 4 questions

1-D DP

#343 Integer Break

MED

Open on LeetCode

Math · Dynamic Programming

Lists: AlgoMaster 600

Problem

Given an integer n, break it into the sum of k positive integers, where k >= 2, and maximize the product of those integers. Return the maximum product you can get.

Example 1:

```
Input: n = 2
Output: 1
Explanation: 2 = 1 + 1, 1 × 1 = 1.
```

Example 2:

```
Input: n = 10
Output: 36
Explanation: 10 = 3 + 3 + 4, 3 × 3 × 4 = 36.
```

Constraints:

- 2 <= n <= 58

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Break n into at least 2 positive integers summing to n. Maximize their product. Right?"
# "n=2→1 ✓ n=10→36(3×3×4) ✓"
# Math insight: use as many 3s as possible (never 1s, prefer 3 over 2). O(n) or O(1). Space O(1).
# Follow-up: if k breaks required (exactly k parts) — DP with k as a state.
# Follow-up: prove correctness: 3² > 2³ for sum 6, so 3 beats 2 when n≥5.
```

1-D DP

#646 Maximum Length of Pair Chain

MED

Open on LeetCode

Array · Dynamic Programming · Greedy · Sorting

Lists: AlgoMaster 600

Problem

You are given an array of n pairs pairs where pairs[i] = [lefti, righti] and lefti < righti.
A pair p2 = [c, d] follows a pair p1 = [a, b] if b < c. A chain of pairs can be formed in this fashion.
Return the length longest chain which can be formed.
You do not need to use up all the given intervals. You can select pairs in any order.

Example 1:

```
Input: pairs = [[1,2],[2,3],[3,4]]
Output: 2
Explanation: The longest chain is [1,2] -> [3,4].
```

Example 2:

```
Input: pairs = [[1,2],[7,8],[4,5]]
Output: 3
Explanation: The longest chain is [1,2] -> [4,5] -> [7,8].
```

Constraints:

- n == pairs.length
- 1 <= n <= 1000
- -1000 <= lefti < righti <= 1000

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given pairs [a,b], chain if b1<a2. Find longest chain. Right?"
# "pairs=[[1,2],[2,3],[3,4]]→2 ✓ pairs=[[1,2],[7,8],[4,5]]→3 ✓"
# Greedy: sort by end time, greedily pick non-overlapping pairs. O(n log n). Space O(1).
# Follow-up: Non-overlapping Intervals (LC 435) — minimize removals (same greedy).
# Follow-up: if pairs can partially overlap — define overlap differently and adjust condition.
```


1-D DP

#740 Delete and Earn

MED

Open on LeetCode

Array · Hash Table · Dynamic Programming

Lists: AlgoMaster 600

Problem

You are given an integer array nums. You want to maximize the number of points you get by performing the following operation any number of times:

- Pick any nums[i] and delete it to earn nums[i] points. Afterwards, you must delete every element equal to nums[i] - 1 and every element equal to nums[i] + 1.

Return the maximum number of points you can earn by applying the above operation some number of times.

Example 1:

```
Input: nums = [3,4,2]
Output: 6
Explanation: You can perform the following operations:
- Delete 4 to earn 4 points. Consequently, 3 is also deleted. nums = [2].
- Delete 2 to earn 2 points. nums = [].
You earn a total of 6 points.
```

Example 2:

```
Input: nums = [2,2,3,3,3,4]
Output: 9
Explanation: You can perform the following operations:
- Delete a 3 to earn 3 points. All 2's and 4's are also deleted. nums = [3,3].
- Delete a 3 again to earn 3 points. nums = [3].
- Delete a 3 once more to earn 3 points. nums = [].
You earn a total of 9 points.
```

Constraints:

- 1 <= nums.length <= 2 * 104
- 1 <= nums[i] <= 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Pick nums[i], earn nums[i] points, delete all nums[i]-1 and nums[i]+1. Maximize points. Right?"
# "nums=[3,4,2]→6 ✓ nums=[2,2,3,3,3,4]→9 ✓"
# Reduce to House Robber: bucket values, earn[i]=i*count(i). Adjacent can't both be taken. O(n+max_val).
# Follow-up: House Robber (LC 198) — identical DP structure.
# Follow-up: if we can pick each value at most k times — bounded knapsack variant.
```

1-D DP

#1262 Greatest Sum Divisible by Three

MED

Open on LeetCode

Array · Dynamic Programming · Greedy · Sorting

Lists: AlgoMaster 600

Problem

Given an integer array nums, return the maximum possible sum of elements of the array such that it is divisible by three.

Example 1:

```
Input: nums = [3,6,5,1,8]
Output: 18
Explanation: Pick numbers 3, 6, 1 and 8 their sum is 18 (maximum sum divisible by 3).
```

Example 2:

```
Input: nums = [4]
Output: 0
Explanation: Since 4 is not divisible by 3, do not pick any number.
```

Example 3:

```
Input: nums = [1,2,3,4,4]
Output: 12
Explanation: Pick numbers 1, 3, 4 and 4 their sum is 12 (maximum sum divisible by 3).
```

Constraints:

- 1 <= nums.length <= 4 * 104
- 1 <= nums[i] <= 104

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Find the maximum sum of elements divisible by 3. Right?"
# "nums=[3,6,5,1,8]→18 (3+6+1+8=18) ✓ nums=[4]→0 ✓"
# DP tracking max sum for each remainder mod 3. O(n). Space O(1).
# Follow-up: divisible by k — generalize to dp[0..k-1]. Space O(k).
# Follow-up: at most k elements — add k as DP dimension.
```


0/1 Knapsack

Round 8 · Low · Medium · 2 questions

0/1 Knapsack

#474 Ones and Zeroes

MED

Open on LeetCode

Array · String · Dynamic Programming

Lists: AlgoMaster 600

Problem

You are given an array of binary strings strs and two integers m and n.
Return the size of the largest subset of strs such that there are at most m 0's and n 1's in the subset.
A set x is a subset of a set y if all elements of x are also elements of y.

Example 1:

```
Input: strs = ["10","0001","111001","1","0"], m = 5, n = 3
Output: 4
Explanation: The largest subset with at most 5 0's and 3 1's is {"10", "0001", "1", "0"}, so the answer is 4.
Other valid but smaller subsets include {"0001", "1"} and {"10", "1", "0"}.
{"111001"} is an invalid subset because it contains 4 1's, greater than the maximum of 3.
```

Example 2:

```
Input: strs = ["10","0","1"], m = 1, n = 1
Output: 2
Explanation: The largest subset is {"0", "1"}, so the answer is 2.
```

Constraints:

- 1 <= strs.length <= 600
- 1 <= strs[i].length <= 100
- strs[i] consists only of digits '0' and '1'.
- 1 <= m, n <= 100

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given binary strings and limits m zeros and n ones. Find max subset using at most m zeros and n ones. Right?"
# "strs=['10','0001','111001','1','0'],m=5,n=3→4 ✓"
# 2D 0/1 knapsack. O(len(strs)*m*n). Space O(m*n).
# Follow-up: if strings can be reused — unbounded knapsack (iterate forward not backward).
# Follow-up: 3 resource constraints — 3D DP.
```


0/1 Knapsack

#1049 Last Stone Weight II

MED

Open on LeetCode

Array · Dynamic Programming

Lists: AlgoMaster 600

Problem

You are given an array of integers stones where stones[i] is the weight of the ith stone.
We are playing a game with the stones. On each turn, we choose any two stones and smash them together. Suppose the stones have weights x and y with x <= y. The result of this smash is:

- If x == y, both stones are destroyed, and
- If x != y, the stone of weight x is destroyed, and the stone of weight y has new weight y - x.

At the end of the game, there is at most one stone left.
Return the smallest possible weight of the left stone. If there are no stones left, return 0.

Example 1:

```
Input: stones = [2,7,4,1,8,1]
Output: 1
Explanation:
We can combine 2 and 4 to get 2, so the array converts to [2,7,1,8,1] then,
we can combine 7 and 8 to get 1, so the array converts to [2,1,1,1] then,
we can combine 2 and 1 to get 1, so the array converts to [1,1,1] then,
we can combine 1 and 1 to get 0, so the array converts to [1], then that's the optimal value.
```

Example 2:

```
Input: stones = [31,26,33,21,40]
Output: 5
```

Constraints:

- 1 <= stones.length <= 30
- 1 <= stones[i] <= 100

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Smash stones: pick any two, result is |x-y|. Minimize last stone weight (or 0). Right?"
# "stones=[2,7,4,1,8,1]→1 ✓"
# Partition Equal Subset (LC 416): find max subset sum ≤ total/2. Answer = total - 2*best. O(n*total). Space O(total).
# Follow-up: if we can add new stones dynamically — maintain the DP incrementally.
# Follow-up: Last Stone Weight I (LC 1046) — greedy with max-heap, no optimization.
```

Unbounded Knapsack

Round 8 · Low · Medium · 2 questions

Unbounded Knapsack

☐ #279 Perfect Squares MED

Open on LeetCode

Math · Dynamic Programming · Breadth-First Search

Lists: AlgoMaster 600

Problem

Given an integer n, return the least number of perfect square numbers that sum to n.

A perfect square is an integer that is the square of an integer; in other words, it is the product of some integer with itself. For example, 1, 4, 9, and 16 are perfect squares while 3 and 11 are not.

Example 1:

```
Input: n = 12
Output: 3
Explanation: 12 = 4 + 4 + 4.
```

Example 2:

```
Input: n = 13
Output: 2
Explanation: 13 = 4 + 9.
```

Constraints:

- **1 <= n <= 104**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Return the minimum number of perfect squares summing to n. Right?"
# "n=12→3 (4+4+4) ✓ n=13→2 (4+9) ✓"
# Unbounded knapsack: each square can be used multiple times. O(n*sqrt(n)). Space O(n).
# Math: Lagrange's Four-Square Theorem — any n is sum of at most 4 squares. Check 1,2,3,4.
# Follow-up: Coin Change (LC 322) — same DP with arbitrary coin denominations.
```

Unbounded Knapsack

#983 Minimum Cost For Tickets

MED

Open on LeetCode

Array · Dynamic Programming

Lists: AlgoMaster 600

Problem

You have planned some train traveling one year in advance. The days of the year in which you will travel are given as an integer array days. Each day is an integer from 1 to 365.

Train tickets are sold in three different ways:

- a 1-day pass is sold for costs[0] dollars,
- a 7-day pass is sold for costs[1] dollars, and
- a 30-day pass is sold for costs[2] dollars.

The passes allow that many days of consecutive travel.

- For example, if we get a 7-day pass on day 2, then we can travel for 7 days: 2, 3, 4, 5, 6, 7, and 8.

Return the minimum number of dollars you need to travel every day in the given list of days.

Example 1:

```
Input: days = [1,4,6,7,8,20], costs = [2,7,15]
Output: 11
Explanation: For example, here is one way to buy passes that lets you travel your travel plan:
On day 1, you bought a 1-day pass for costs[0] = $2, which covered day 1.
On day 3, you bought a 7-day pass for costs[1] = $7, which covered days 3, 4, ..., 9.
On day 20, you bought a 1-day pass for costs[0] = $2, which covered day 20.
In total, you spent $11 and covered all the days of your travel.
```

Example 2:

```
Input: days = [1,2,3,4,5,6,7,8,9,10,30,31], costs = [2,7,15]
Output: 17
Explanation: For example, here is one way to buy passes that lets you travel your travel plan:
On day 1, you bought a 30-day pass for costs[2] = $15 which covered days 1, 2, ..., 30.
On day 31, you bought a 1-day pass for costs[0] = $2 which covered day 31.
In total, you spent $17 and covered all the days of your travel.
```

Constraints:

- 1 <= days.length <= 365
- 1 <= days[i] <= 365
- days is in strictly increasing order.
- costs.length == 3
- 1 <= costs[i] <= 1000

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Travel days given. Passes: 1-day/7-day/30-day. Find minimum cost. Right?"
# "days=[1,4,6,7,8,20],costs=[2,7,15]→11 ✓"
# DP over days 1..last. Skip non-travel days. O(last). Space O(last).
# Follow-up: if costs can vary by date — extend dp to store (date,cost).
# Follow-up: if we need to output which passes to buy — track choices in DP.
```


Longest Increasing Subsequence (LIS)

Round 8 · Low · Medium · 3 questions

Longest Increasing Subsequence (LIS)

#673 Number of Longest Increasing Subsequence

MED

Open on LeetCode

Array · Dynamic Programming · Binary Indexed Tree · Segment Tree

Lists: AlgoMaster 600

Problem

Given an integer array nums, return the number of longest increasing subsequences.

Notice that the sequence has to be strictly increasing.

Example 1:

```
Input: nums = [1,3,5,4,7]
Output: 2
Explanation: The two longest increasing subsequences are [1, 3, 4, 7] and [1, 3, 5, 7].
```

Example 2:

```
Input: nums = [2,2,2,2,2]
Output: 5
Explanation: The length of the longest increasing subsequence is 1, and there are 5 increasing subsequences of length 1, so output 5.
```

Constraints:

- 1 <= nums.length <= 2000
- -106 <= nums[i] <= 106
- The answer is guaranteed to fit inside a 32-bit integer.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return the number of longest strictly increasing subsequences. Right?"
# "nums=[1,3,5,4,7]→2 ([1,3,5,7],[1,3,4,7]) ✓"
# DP tracking both LIS length and count at each position. O(n²). Space O(n).
# Follow-up: O(n log n) approach — patience sorting to find LIS length, then track counts separately.
# Follow-up: Longest Increasing Subsequence (LC 300) — just length, no count.
```


Longest Increasing Subsequence (LIS)

#1027 Longest Arithmetic Subsequence

MED

Open on LeetCode

Array · Hash Table · Binary Search · Dynamic Programming

Lists: AlgoMaster 600

Problem

Given an array nums of integers, return the length of the longest arithmetic subsequence in nums.

Note that:

- A subsequence is an array that can be derived from another array by deleting some or no elements without changing the order of the remaining elements.
- A sequence seq is arithmetic if seq[i + 1] - seq[i] are all the same value (for 0 <= i < seq.length - 1).

Example 1:

```
Input: nums = [3,6,9,12]
Output: 4
Explanation: The whole array is an arithmetic sequence with steps of length = 3.
```

Example 2:

```
Input: nums = [9,4,7,2,10]
Output: 3
Explanation: The longest arithmetic subsequence is [4,7,10].
```

Example 3:

```
Input: nums = [20,1,15,3,10,5,8]
Output: 4
Explanation: The longest arithmetic subsequence is [20,15,10,5].
```

Constraints:

- 2 <= nums.length <= 1000
- 0 <= nums[i] <= 500

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the length of the longest arithmetic subsequence (equal differences). Right?"
# "nums=[3,6,9,12]→4 (diff=3) ✓ nums=[9,4,7,2,10]→3 (4,7,10) ✓"
# dp[i][d]=longest arith subseq ending at i with diff d. O(n²). Space O(n²).
# Follow-up: Longest Arithmetic Subsequence of Given Difference (LC 1218) — fixed diff, O(n) with hashmap.
# Follow-up: return the actual subsequence — track previous index in dp.
```

Longest Increasing Subsequence (LIS)

#1048 Longest String Chain

MED

Open on LeetCode

Array · Hash Table · Two Pointers · String · Dynamic Programming · Sorting

Lists: AlgoMaster 600

Problem

You are given an array of words where each word consists of lowercase English letters.

wordA is a predecessor of wordB if and only if we can insert exactly one letter anywhere in wordA without changing the order of the other characters to make it equal to wordB.

- For example, "abc" is a predecessor of "abac", while "cba" is not a predecessor of "bcad".

A word chain is a sequence of words [word1, word2, ..., wordk] with k >= 1, where word1 is a predecessor of word2, word2 is a predecessor of word3, and so on. A single word is trivially a word chain with k == 1.

Return the length of the longest possible word chain with words chosen from the given list of words.

Example 1:

```
Input: words = ["a","b","ba","bca","bda","bdca"]
Output: 4
Explanation: One of the longest word chains is ["a","ba","bda","bdca"].
```

Example 2:

```
Input: words = ["xbc","pcxbcf","xb","cxbc","pcxbc"]
Output: 5
Explanation: All the words can be put in a word chain ["xb", "xbc", "cxbc", "pcxbc", "pcxbcf"].
```

Example 3:

```
Input: words = ["abcd","dbqca"]
Output: 1
Explanation: The trivial word chain ["abcd"] is one of the longest word chains.
["abcd","dbqca"] is not a valid word chain because the ordering of the letters is changed.
```

Constraints:

- 1 <= words.length <= 1000
- 1 <= words[i].length <= 16
- words[i] only consists of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Word B is predecessor of A if adding one letter to B (any position) gives A.
# Find longest word chain. Right?"
# "words=['a','b','ba','bca','bda','bdca']→4 ('a'→'ba'→'bda'→'bdca') ✓"
# Sort by length, try all single-deletion predecessors. O(n*L²). Space O(n*L).
# Follow-up: return the actual chain — track previous word in DP.
# Follow-up: Longest Increasing Subsequence (LC 300) — same idea with numeric ordering.
```


2D Grid DP

Round 8 · Low · Medium · 5 questions

2D Grid DP

#63 Unique Paths II

MED

Open on LeetCode

Array · Dynamic Programming · Matrix

Lists: AlgoMaster 600

Problem

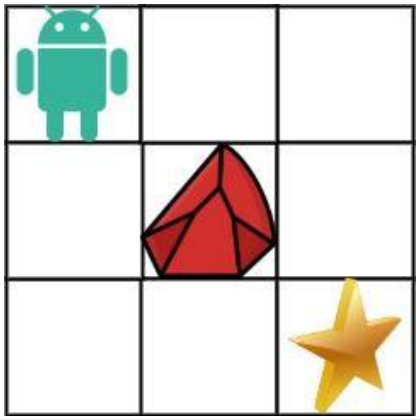
You are given an m x n integer array grid. There is a robot initially located at the top-left corner (i.e., grid[0][0]). The robot tries to move to the bottom-right corner (i.e., grid[m - 1][n - 1]). The robot can only move either down or right at any point in time.

An obstacle and space are marked as 1 or 0 respectively in grid. A path that the robot takes cannot include any square that is an obstacle.

Return the number of possible unique paths that the robot can take to reach the bottom-right corner.

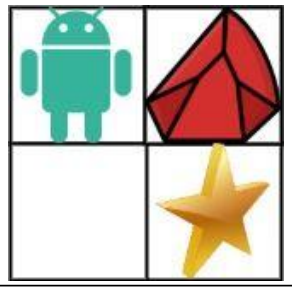
The testcases are generated so that the answer will be less than or equal to 2 * 109.

Example 1:



```
Input: obstacleGrid = [[0,0,0],[0,1,0],[0,0,0]]
Output: 2
Explanation: There is one obstacle in the middle of the 3x3 grid above.
There are two ways to reach the bottom-right corner:
1. Right -> Right -> Down -> Down
2. Down -> Down -> Right -> Right
```

Example 2:



```
Input: obstacleGrid = [[0,1],[0,0]]
Output: 1
```

Constraints:

- m == obstacleGrid.length
- n == obstacleGrid[i].length
- 1 <= m, n <= 100

- obstacleGrid[i][j] is 0 or 1.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Grid with obstacles (1=obstacle,0=empty). Count paths from top-left to bottom-right. Right?"
# "obstacleGrid=[[0,0,0],[0,1,0],[0,0,0]]→2 ✓"
# Rolling 1D DP. O(mn). Space O(n).
# Follow-up: Unique Paths I (LC 62) — no obstacles, use math C(m+n-2,m-1).
# Follow-up: if there are multiple end points — run DP for each independently.
```

2D Grid DP

#120 Triangle

ME D

Open on LeetCode

Array · Dynamic Programming

Lists: AlgoMaster 600

Problem

Given a triangle array, return the minimum path sum from top to bottom.

For each step, you may move to an adjacent number of the row below. More formally, if you are on index i on the current row, you may move to either index i or index i + 1 on the next row.

Example 1:

```
Input: triangle = [[2],[3,4],[6,5,7],[4,1,8,3]]
Output: 11
Explanation: The triangle looks like:
2
3 4
6 5 7
4 1 8 3
The minimum path sum from top to bottom is 2 + 3 + 5 + 1 = 11 (underlined above).
```

Example 2:

```
Input: triangle = [[-10]]
Output: -10
```

Constraints:

- 1 <= triangle.length <= 200
- triangle[0].length == 1
- triangle[i].length == triangle[i - 1].length + 1
- -104 <= triangle[i][j] <= 104

Follow up: Could you do this using only O(n) extra space, where n is the total number of rows in the triangle?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find minimum path sum from top to bottom. Each step moves to adjacent number below. Right?"
# "triangle=[[2],[3,4],[6,5,7],[4,1,8,3]]→11 (2→3→5→1) ✓"
# Bottom-up DP in-place on last row. O(n²). Space O(n).
# Follow-up: find the actual path — track choices during DP.
# Follow-up: minimum sum in a DAG — same DP on general DAG with topological ordering.
```


2D Grid DP

#931 Minimum Falling Path Sum

ME D

Open on LeetCode

Array · Dynamic Programming · Matrix

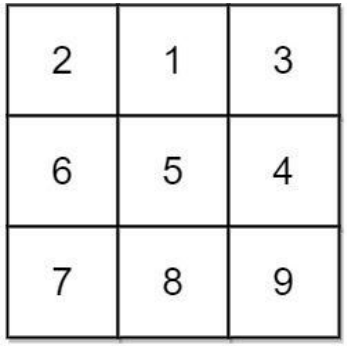
Lists: AlgoMaster 600

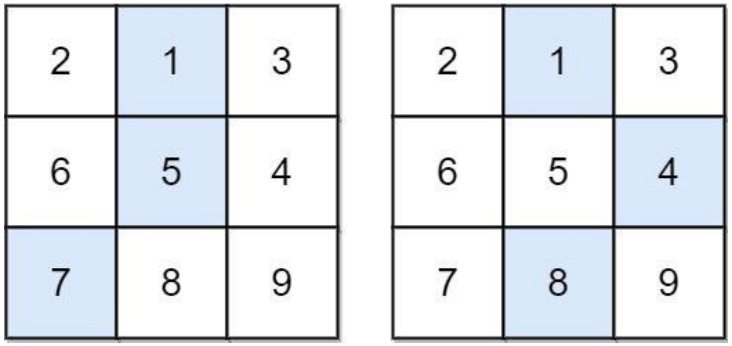
Problem

Given an n x n array of integers matrix, return the minimum sum of any falling path through matrix.

A falling path starts at any element in the first row and chooses the element in the next row that is either directly below or diagonally left/right. Specifically, the next element from position (row, col) will be (row + 1, col - 1), (row + 1, col), or (row + 1, col + 1).

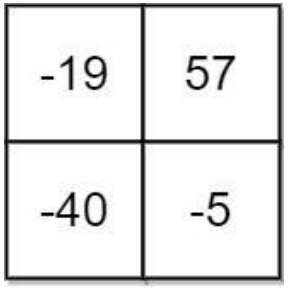
Example 1:

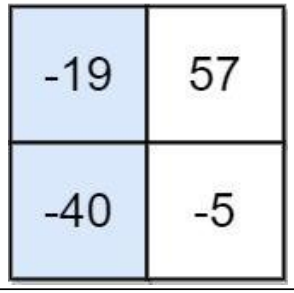





```
Input: matrix = [[2,1,3],[6,5,4],[7,8,9]]
Output: 13
Explanation: There are two falling paths with a minimum sum as shown.
```

Example 2:






```
Input: matrix = [[-19,57],[-40,-5]]
Output: -59
Explanation: The falling path with a minimum sum is shown.
```

Constraints:

- n == matrix.length == matrix[i].length
- 1 <= n <= 100
- -100 <= matrix[i][j] <= 100

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Find min sum falling path: start any column in row 0, move to adjacent column below each step. Right?"
# "matrix=[[2,1,3],[6,5,4],[7,8,9]]→13 (1→5→7? 1+4+7=12? wait: 1→4→7=12) ✓"
# Row-by-row DP with adjacent-column transitions. O(n²). Space O(n).
# Follow-up: Minimum Falling Path Sum II (LC 1289) — non-adjacent columns (any column in row below).
# Follow-up: if matrix can have different row lengths — triangle problem (LC 120).
```


2D Grid DP

#1277 Count Square Submatrices with All Ones

MED

Open on LeetCode

Array · Dynamic Programming · Matrix

Lists: AlgoMaster 600

Problem

Given a m * n matrix of ones and zeros, return how many square submatrices have all ones.

Example 1:

```
Input: matrix =
[
  [0,1,1,1],
  [1,1,1,1],
  [0,1,1,1]
]
Output: 15
Explanation:
```

Example 2:

```
Input: matrix =
[
[1,0,1],
[1,1,0],
[1,1,0]
]
Output: 7
Explanation:
```

Constraints:

- 1 <= arr.length <= 300
- 1 <= arr[0].length <= 300
- 0 <= arr[i][j] <= 1

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Count all square submatrices containing only 1s. Right?"
# "matrix=[[0,1,1,1],[1,1,1,1],[0,1,1,1]]→15 ✓"
# dp[r][c]=largest square with bottom-right at (r,c). Sum = total squares. O(mn). Space O(1) in-place.
# Follow-up: Maximal Square (LC 221) — return area of largest such square.
# Follow-up: count rectangles (not just squares) — O(m²n) with column histogram approach.
```

2D Grid DP

#1937 Maximum Number of Points with Cost

MED

Open on LeetCode

Array · Dynamic Programming · Matrix

Lists: AlgoMaster 600

Problem

You are given an m x n integer matrix points (0-indexed). Starting with 0 points, you want to maximize the number of points you can get from the matrix.

To gain points, you must pick one cell in each row. Picking the cell at coordinates (r, c) will add points[r][c] to your score.

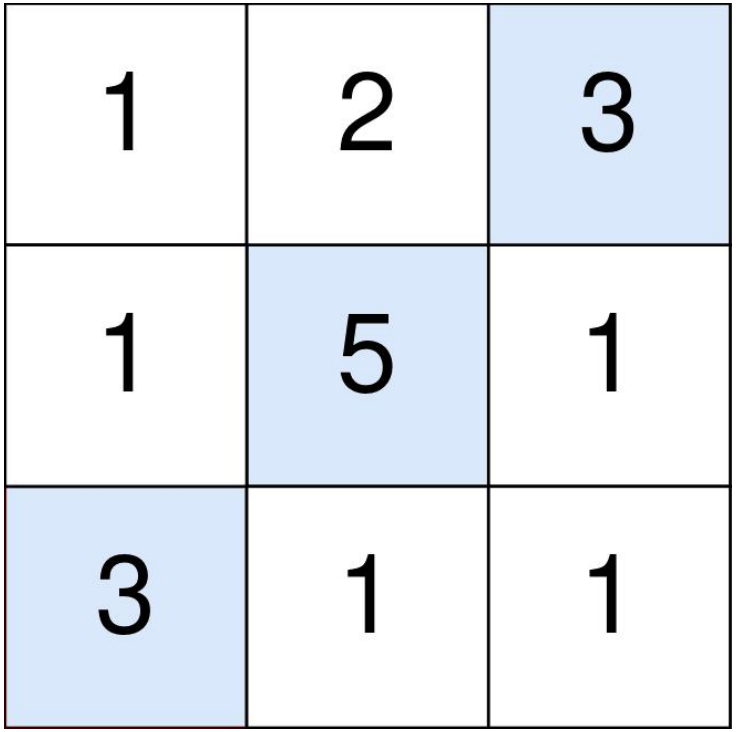
However, you will lose points if you pick a cell too far from the cell that you picked in the previous row. For every two adjacent rows r and r + 1 (where 0 <= r < m - 1), picking cells at coordinates (r, c1) and (r + 1, c2) will subtract abs(c1 - c2) from your score.

Return the maximum number of points you can achieve.

abs(x) is defined as:

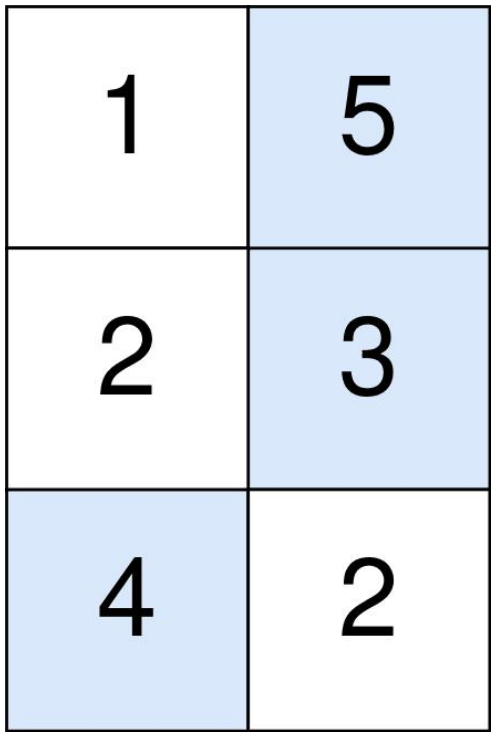
- x for x >= 0.
- -x for x < 0.

Example 1:




```
Input: points = [[1,2,3],[1,5,1],[3,1,1]]
Output: 9
Explanation:
The blue cells denote the optimal cells to pick, which have coordinates (0, 2), (1, 1), and (2, 0).
You add 3 + 5 + 3 = 11 to your score.
However, you must subtract abs(2 - 1) + abs(1 - 0) = 2 from your score.
Your final score is 11 - 2 = 9.
```

Example 2:




```
Input: points = [[1,5],[2,3],[4,2]]
Output: 11
Explanation:
The blue cells denote the optimal cells to pick, which have coordinates (0, 1), (1, 1), and (2, 0).
You add 5 + 3 + 4 = 12 to your score.
However, you must subtract abs(1 - 1) + abs(1 - 0) = 1 from your score.
Your final score is 12 - 1 = 11.
```

Constraints:

- m == points.length
- n == points[r].length
- 1 <= m, n <= 105
- 1 <= m * n <= 105
- 0 <= points[r][c] <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Pick one cell per row. Score = sum of values minus sum of column index differences between adjacent rows. Right?"
# "points=[[1,2,3],[1,5,1],[3,1,1]]→9 ✓"
            # left pass: max dp[j]-j for j<=c
            # right pass: max dp[j]+j for j>=c
# Two-pass optimization: left[c]=max(dp[j]-j for j<=c)+c, right[c]=max(dp[j]+j for j>=c)-c. O(mn). Space O(n).
# Follow-up: if penalty is squared distance — DP with divide-and-conquer optimization.
# Follow-up: minimum cost instead of maximum — same approach, negate values.
```

String DP

Round 8 · Low · Medium · 1 question

String DP

#1653 Minimum Deletions to Make String Balanced

MED

Open on LeetCode

String · Dynamic Programming · Stack

Lists: AlgoMaster 600

Problem

You are given a string s consisting only of characters 'a' and 'b'.

You can delete any number of characters in s to make s balanced. s is balanced if there is no pair of indices (i,j) such that i < j and s[i] = 'b' and s[j]= 'a'.

Return the minimum number of deletions needed to make s balanced.

Example 1:

```
Input: s = "aababbab"
Output: 2
Explanation: You can either:
Delete the characters at 0-indexed positions 2 and 6 ("aababbab" -> "aaabbb"), or
Delete the characters at 0-indexed positions 3 and 6 ("aababbab" -> "aabbbb").
```

Example 2:

```
Input: s = "bbaaaaabb"
Output: 2
Explanation: The only solution is to delete the first two characters.
```

Constraints:

- 1 <= s.length <= 105
- s[i] is 'a' or 'b' .

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Delete minimum chars to make 'b's never appear before 'a's (all a's before b's). Right?"
# "s='aababbab'→2 ✓ s='bbaaaaabb'→2 ✓"
        # At each position, either delete all b's seen so far, or delete all a's after
# Greedy DP: track b_count. When seeing 'a', best is min(delete this 'a', delete all 'b's before). O(n). Space O(1).
# Follow-up: k distinct characters allowed on left, rest on right — extend to k-partition DP.
# Follow-up: return the actual set of deleted indices — track choices during DP.
```

Tree / Graph DP

Round 8 · Low · Medium · 3 questions

Tree / Graph DP

#95 Unique Binary Search Trees II

MED

Open on LeetCode

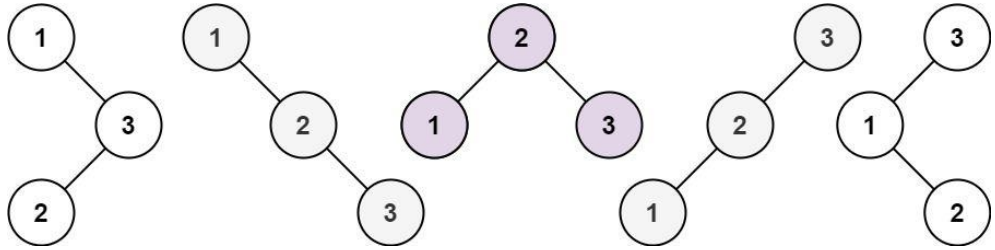
Dynamic Programming · Backtracking · Tree · Binary Search Tree · Binary Tree

Lists: AlgoMaster 600

Problem

Given an integer n, return all the structurally unique BST's (binary search trees), which has exactly n nodes of unique values from 1 to n. Return the answer in any order.

Example 1:




```
Input: n = 3
Output: [[1,null,2,null,3],[1,null,3,2],[2,1,3],[3,1,null,null,2],[3,2,null,1]]
```

Example 2:

```
Input: n = 1
Output: [[1]]
```

Constraints:

- 1 <= n <= 8

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return all structurally unique BSTs with keys 1..n. Right?"
# "n=3→[[1,null,2,null,3],[1,null,3,2],[2,1,3],[3,1,null,null,2],[3,2,null,1]] ✓"
# Catalan number of trees. O(C_n * n) where C_n is nth Catalan. Space O(C_n * n).
# Follow-up: Unique BSTs I (LC 96) — count only, use Catalan DP O(n²).
# Follow-up: if keys are not 1..n but arbitrary sorted array — same structure, just use the array.
```

Tree / Graph DP

#337 House Robber III

MED

Open on LeetCode

Dynamic Programming · Tree · Depth-First Search · Binary Tree

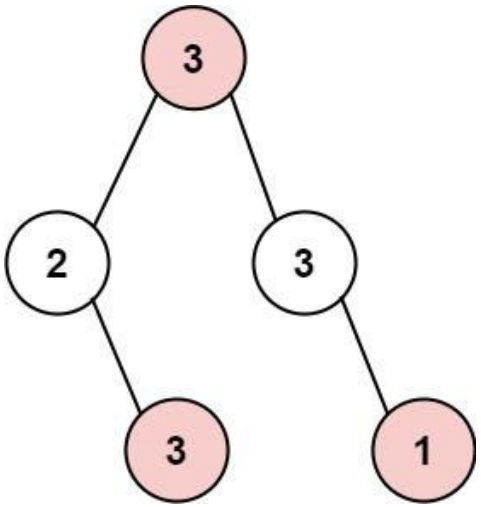
Lists: AlgoMaster 600

Problem

The thief has found himself a new place for his thievery again. There is only one entrance to this area, called root. Besides the root, each house has one and only one parent house. After a tour, the smart thief realized that all houses in this place form a binary tree. It will automatically contact the police if two directly-linked houses were broken into on the same night.

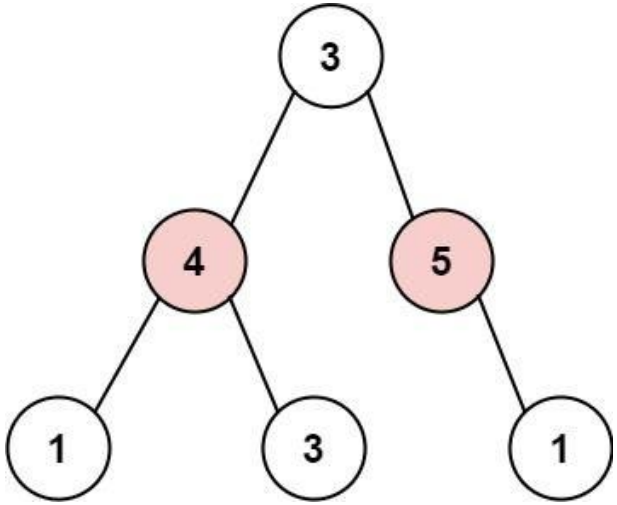
Given the root of the binary tree, return the maximum amount of money the thief can rob without alerting the police.

Example 1:




```
Input: root = [3,2,3,null,3,null,1]
Output: 7
Explanation: Maximum amount of money the thief can rob = 3 + 3 + 1 = 7.
```

Example 2:




```
Input: root = [3,4,5,1,3,null,1]
Output: 9
Explanation: Maximum amount of money the thief can rob = 4 + 5 = 9.
```

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- 0 <= Node.val <= 104

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Binary tree. Rob non-adjacent nodes (parent-child can't both be robbed). Maximize. Right?"
# "root=[3,2,3,null,3,null,1]→7 ✓"
# Each node returns (rob_node, skip_node) pair. O(n). Space O(h).
# Follow-up: House Robber I (LC 198) — linear array, rolling DP O(1) space.
# Follow-up: House Robber II (LC 213) — circular array, two separate runs.
```

Tree / Graph DP

#1976 Number of Ways to Arrive at Destination

MED

Open on LeetCode

Dynamic Programming · Graph Theory · Topological Sort · Shortest Path

Lists: AlgoMaster 600

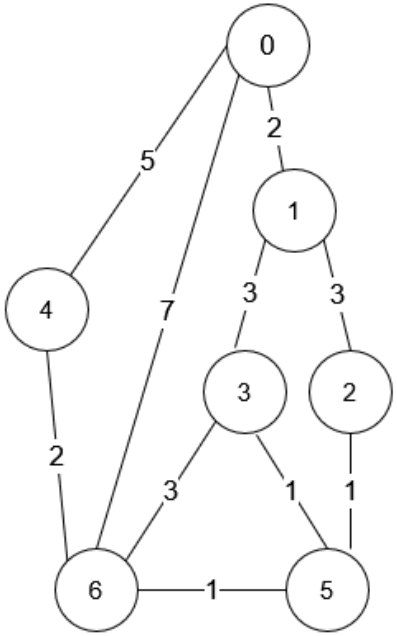
Problem

You are in a city that consists of n intersections numbered from 0 to n - 1 with bi-directional roads between some intersections. The inputs are generated such that you can reach any intersection from any other intersection and that there is at most one road between any two intersections.

You are given an integer n and a 2D integer array roads where roads[i] = [ui, vi, timei] means that there is a road between intersections ui and vi that takes timei minutes to travel. You want to know in how many ways you can travel from intersection 0 to intersection n - 1 in the shortest amount of time.

Return the number of ways you can arrive at your destination in the shortest amount of time. Since the answer may be large, return it modulo 109 + 7.

Example 1:




```
Input: n = 7, roads = [[0,6,7],[0,1,2],[1,2,3],[1,3,3],[6,3,3],[3,5,1],[6,5,1],[2,5,1],[0,4,5],[4,6,2]]
Output: 4
Explanation: The shortest amount of time it takes to go from intersection 0 to intersection 6 is 7 minutes.
The four ways to get there in 7 minutes are:
- 0 → 6
- 0 → 4 → 6
- 0 → 1 → 2 → 5 → 6
- 0 → 1 → 3 → 5 → 6
```

Example 2:


```
Input: n = 2, roads = [[1,0,10]]
Output: 1
Explanation: There is only one way to go from intersection 0 to intersection 1, and it takes 10 minutes.
```

Constraints:

- 1 <= n <= 200
- n - 1 <= roads.length <= n * (n - 1) / 2
- roads[i].length == 3
- 0 <= ui, vi <= n - 1
- 1 <= timei <= 109
- ui != vi
- There is at most one road connecting any two intersections.
- You can reach any intersection from any other intersection.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Count paths from 0 to n-1 with minimum total time. Return mod 10^9+7. Right?"
# "n=7,roads=[[0,6,7],[0,1,2],[1,2,3],[1,3,3],[6,3,3],[3,5,1],[6,5,1],[2,5,1],[0,4,5],[4,6,2]]→4 ✓"
# Dijkstra + count: track ways[v] alongside dist[v]. O((V+E) log V). Space O(V+E).
# Follow-up: if all edges same weight — BFS suffices.
# Follow-up: count paths with at most k edges — add k as DP dimension.
```

Bitmask DP

Round 8 · Low · Medium · 4 questions

Bitmask DP

#698 Partition to K Equal Sum Subsets

MED

Open on LeetCode

Array · Dynamic Programming · Backtracking · Bit Manipulation · Memoization · Bitmask

Lists: AlgoMaster 600

Problem

Given an integer array nums and an integer k, return true if it is possible to divide this array into k non-empty subsets whose sums are all equal.

Example 1:

```
Input: nums = [4,3,2,3,5,2,1], k = 4
Output: true
Explanation: It is possible to divide it into 4 subsets (5), (1, 4), (2,3), (2,3) with equal sums.
```

Example 2:

```
Input: nums = [1,2,3,4], k = 3
Output: false
```

Constraints:

- **1 <= k <= nums.length <= 16**
- **1 <= nums[i] <= 104**
- **The frequency of each element is in the range [1, 4].**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Can the array be partitioned into k subsets with equal sum? Right?"
#
# "nums=[4,3,2,3,5,2,1], k=4: [5],[1,4],[2,3],[2,3] → True ✓
# nums=[1,2,3,4], k=3: 10/3 not integer → False ✓"

# PHASE 2 — BRUTE FORCE
# "Backtracking: assign each number to one of k buckets. Check if all buckets reach target.
# Sort descending to prune early. Time: O(k^n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Bitmask DP: dp[mask] = remaining capacity of current bucket after using elements in mask.
# Time: O(n * 2^n). Space: O(2^n)."

# PHASE 4 — CLEAN CODE (backtracking is more intuitive for interviews)

# PHASE 5 — TEST & DEBUG
# nums=[4,3,2,3,5,2,1],k=4,target=5: sorted=[5,4,3,3,2,2,1].
# 5→bucket[0]=5. 4→bucket[1]=4. 3→bucket[1]=4? No: bucket[1]+3=7>5. bucket[2]=3.
# 3→bucket[2]=3+3? No. bucket[3]=3. 2→bucket[1]=4+2? No. bucket[2]=3+2=5. etc. → True ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Backtracking O(k^n) heavily pruned; Bitmask DP O(n*2^n) more predictable.
# The 'seen' set avoids trying equivalent empty buckets.
# Follow-up: Matchsticks to Square (LC 473) — special case with k=4.
# Follow-up: if k=2 — Partition Equal Subset Sum (LC 416), solvable with O(n*sum) DP."
```

Bitmask DP

#1066 Campus Bikes II

MED

Open on LeetCode

Array · Dynamic Programming · Backtracking · Bit Manipulation · Bitmask

Lists: AlgoMaster 600

Problem

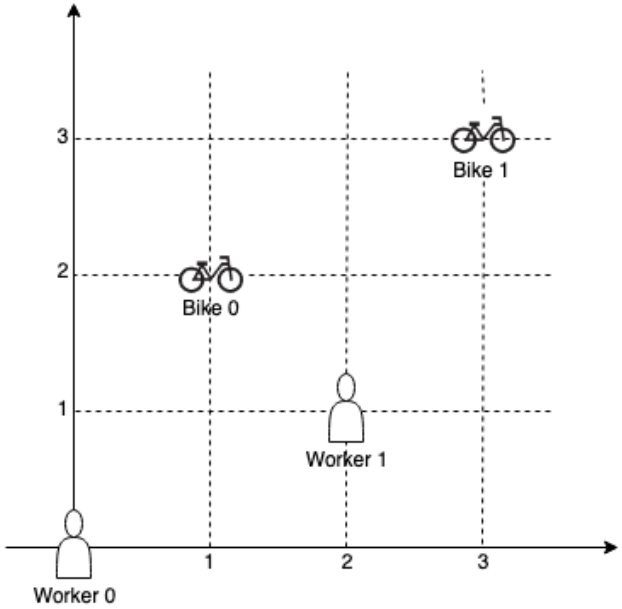
On a campus represented as a 2D grid, there are n workers and m bikes, with n <= m. Each worker and bike is a 2D coordinate on this grid.

We assign one unique bike to each worker so that the sum of the Manhattan distances between each worker and their assigned bike is minimized.

Return the minimum possible sum of Manhattan distances between each worker and their assigned bike.

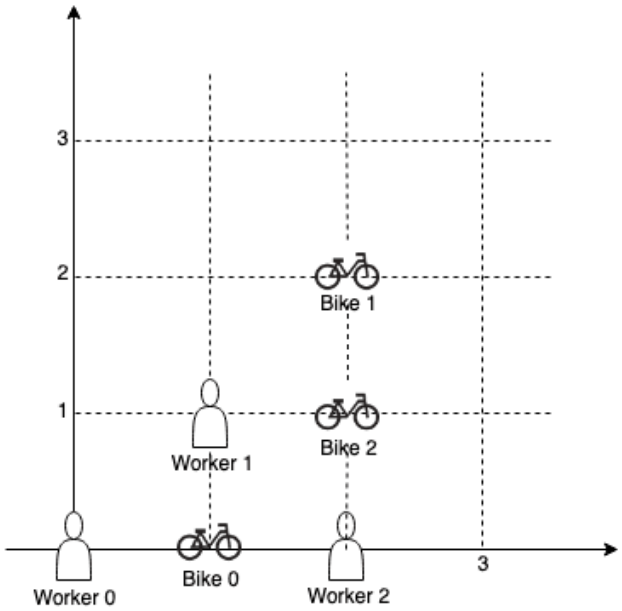
The Manhattan distance between two points p1 and p2 is Manhattan(p1, p2) = |p1.x - p2.x| + |p1.y - p2.y|.

Example 1:




```
Input: workers = [[0,0],[2,1]], bikes = [[1,2],[3,3]]
Output: 6
Explanation:
We assign bike 0 to worker 0, bike 1 to worker 1. The Manhattan distance of both assignments is 3, so the output is 6.
```

Example 2:




```
Input: workers = [[0,0],[1,1],[2,0]], bikes = [[1,0],[2,2],[2,1]]
Output: 4
Explanation:
We first assign bike 0 to worker 0, then assign bike 1 to worker 1 or worker 2, bike 2 to worker 2 or worker 1. Both assignments lead to sum of the Manhattan distances as 4.
```

Example 3:

```
Input: workers = [[0,0],[1,0],[2,0],[3,0],[4,0]], bikes = [[0,999],[1,999],[2,999],[3,999],[4,999]]
Output: 4995
```

Constraints:

- n == workers.length
- m == bikes.length
- 1 <= n <= m <= 10
- workers[i].length == 2
- bikes[i].length == 2
- 0 <= workers[i][0], workers[i][1], bikes[i][0], bikes[i][1] < 1000
- All the workers and the bikes locations are unique.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Assign n workers to m bikes (m>=n). Minimize total Manhattan distance. Right?"
# "workers=[[0,0],[1,1]],bikes=[[2,2],[1,3]]→4 ✓"
# Bitmask DP: mask=bikes used. O(n*2^m). Space O(n*2^m).
# Follow-up: Campus Bikes I (LC 1057) — greedy by distance, no optimization needed.
# Follow-up: if workers and bikes have weights — weighted assignment, Hungarian algorithm O(n³).
```

Bitmask DP

#1986 Minimum Number of Work Sessions to Finish the Tasks

MED

Open on LeetCode

Array · Dynamic Programming · Backtracking · Bit Manipulation · Bitmask

Lists: AlgoMaster 600

Problem

There are n tasks assigned to you. The task times are represented as an integer array tasks of length n, where the ith task takes tasks[i] hours to finish. A work session is when you work for at most sessionTime consecutive hours and then take a break.

You should finish the given tasks in a way that satisfies the following conditions:

- If you start a task in a work session, you must complete it in the same work session.
- You can start a new task immediately after finishing the previous one.
- You may complete the tasks in any order.

Given tasks and sessionTime, return the minimum number of work sessions needed to finish all the tasks following the conditions above.

The tests are generated such that sessionTime is greater than or equal to the maximum element in tasks[i].

Example 1:

```
Input: tasks = [1,2,3], sessionTime = 3
Output: 2
Explanation: You can finish the tasks in two work sessions.
- First work session: finish the first and the second tasks in 1 + 2 = 3 hours.
- Second work session: finish the third task in 3 hours.
```

Example 2:

```
Input: tasks = [3,1,3,1,1], sessionTime = 8
Output: 2
Explanation: You can finish the tasks in two work sessions.
- First work session: finish all the tasks except the last one in 3 + 1 + 3 + 1 = 8 hours.
- Second work session: finish the last task in 1 hour.
```

Example 3:

```
Input: tasks = [1,2,3,4,5], sessionTime = 15
Output: 1
Explanation: You can finish all the tasks in one work session.
```

Constraints:

- n == tasks.length
- 1 <= n <= 14
- 1 <= tasks[i] <= 10
- max(tasks[i]) <= sessionTime <= 15

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "n tasks. Each session max sessionTime. Tasks can't be split. Min sessions to complete all. Right?"
# "tasks=[1,2,3],sessionTime=3→2 ✓ (sessions:[1,2],[3])"
            # Try adding each unfinished task to current session
        # Simpler: dp[mask]=min sessions with tasks in mask done
# Bitmask DP tracking sessions used and remaining time in current session. O(n*2^n). Space O(2^n).
# Follow-up: if sessions have different max times — parameterize rem per session type.
# Follow-up: if tasks have priorities — weighted DP with priority score.
```


Bitmask DP

#2305 Fair Distribution of Cookies

MED

Open on LeetCode

Array · Dynamic Programming · Backtracking · Bit Manipulation · Bitmask

Lists: AlgoMaster 600

Problem

You are given an integer array cookies, where cookies[i] denotes the number of cookies in the ith bag. You are also given an integer k that denotes the number of children to distribute all the bags of cookies to. All the cookies in the same bag must go to the same child and cannot be split up.

The unfairness of a distribution is defined as the maximum total cookies obtained by a single child in the distribution.

Return the minimum unfairness of all distributions.

Example 1:

```
Input: cookies = [8,15,10,20,8], k = 2
Output: 31
Explanation: One optimal distribution is [8,15,8] and [10,20]
- The 1st child receives [8,15,8] which has a total of 8 + 15 + 8 = 31 cookies.
- The 2nd child receives [10,20] which has a total of 10 + 20 = 30 cookies.
The unfairness of the distribution is max(31,30) = 31.
It can be shown that there is no distribution with an unfairness less than 31.
```

Example 2:

```
Input: cookies = [6,1,3,2,2,4,1,2], k = 3
Output: 7
Explanation: One optimal distribution is [6,1], [3,2,2], and [4,1,2]
- The 1st child receives [6,1] which has a total of 6 + 1 = 7 cookies.
- The 2nd child receives [3,2,2] which has a total of 3 + 2 + 2 = 7 cookies.
- The 3rd child receives [4,1,2] which has a total of 4 + 1 + 2 = 7 cookies.
The unfairness of the distribution is max(7,7,7) = 7.
It can be shown that there is no distribution with an unfairness less than 7.
```

Constraints:

- 2 <= cookies.length <= 8
- 1 <= cookies[i] <= 105
- 2 <= k <= cookies.length

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Distribute n cookie bags among k children to minimize the maximum sum any child receives. Right?"
# "cookies=[8,15,10,20,8],k=2→31 ✓"
# Bitmask DP: assign subset of bags to each child. O(k*3^n) — each bit in 3 states. Space O(k*2^n).
# Follow-up: if children have capacity limits — add constraint check in DP.
# Follow-up: if we also care about variance (fairness beyond max) — different objective function.
```

Digit DP

Round 8 · Low · Medium · 1 question

Digit DP

#357 Count Numbers with Unique Digits

MED

Open on LeetCode

Math · Dynamic Programming · Backtracking

Lists: AlgoMaster 600

Problem

Given an integer n, return the count of all numbers with unique digits, x, where 0 <= x < 10n.

Example 1:

```
Input: n = 2
Output: 91
Explanation: The answer should be the total numbers in the range of 0 ≤ x < 100, excluding 11,22,33,44,55,66,77,88,99
```

Example 2:

```
Input: n = 0
Output: 1
```

Constraints:

- 0 <= n <= 8

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Count numbers with all unique digits in range [0,10^n). Right?"
# "n=2→91 ✓ n=0→1 ✓"
        # Math: for k digits, choices = 9*9*8*7*...(10-k+1)
# Math: leading digit 9 choices (1-9), subsequent digits decrease available pool. O(n). Space O(1).
# Follow-up: Count Numbers with Unique Digits for arbitrary range — digit DP with tight constraint.
# Follow-up: if digits must be distinct AND in increasing order — count combinations C(10, k).
```

Probability DP

Round 8 · Low · Medium · 3 questions

Probability DP

#688 Knight Probability in Chessboard

MED

Open on LeetCode

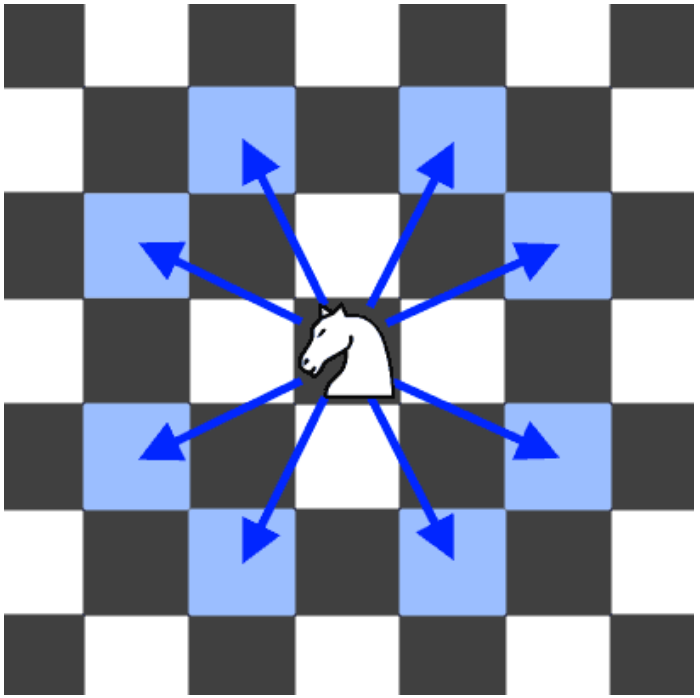
Dynamic Programming

Lists: AlgoMaster 600

Problem

On an n x n chessboard, a knight starts at the cell (row, column) and attempts to make exactly k moves. The rows and columns are 0-indexed, so the top-left cell is (0, 0), and the bottom-right cell is (n - 1, n - 1).

A chess knight has eight possible moves it can make, as illustrated below. Each move is two cells in a cardinal direction, then one cell in an orthogonal direction.



Each time the knight is to move, it chooses one of eight possible moves uniformly at random (even if the piece would go off the chessboard) and moves there.

The knight continues moving until it has made exactly k moves or has moved off the chessboard.

Return the probability that the knight remains on the board after it has stopped moving.

Example 1:

```
Input: n = 3, k = 2, row = 0, column = 0
Output: 0.06250
Explanation: There are two moves (to (1,2), (2,1)) that will keep the knight on the board.
From each of those positions, there are also two moves that will keep the knight on the board.
The total probability the knight stays on the board is 0.0625.
```

Example 2:

```
Input: n = 1, k = 0, row = 0, column = 0
Output: 1.00000
```

Constraints:

- 1 <= n <= 25
- 0 <= k <= 100
- 0 <= row, column <= n - 1

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Knight starts at (r,c) on NxN board. After k moves, probability it's still on board. Right?"
# "n=3,k=2,row=0,col=0→0.0625 ✓"
# Forward DP spreading probability each step. O(k*n²). Space O(n²).
# Follow-up: Out of Boundary Paths (LC 576) — count paths leaving the grid (complement).
# Follow-up: if knight has a target square — track probability at that square specifically.
```


Probability DP

#808 Soup Servings

ME
D

Open on LeetCode

Math · Dynamic Programming · Probability and Statistics

Lists: AlgoMaster 600

Problem

You have two soups, A and B, each starting with n mL. On every turn, one of the following four serving operations is chosen at random, each with probability 0.25 independent of all previous turns:

- pour 100 mL from type A and 0 mL from type B
- pour 75 mL from type A and 25 mL from type B
- pour 50 mL from type A and 50 mL from type B
- pour 25 mL from type A and 75 mL from type B

Note:

- There is no operation that pours 0 mL from A and 100 mL from B.
- The amounts from A and B are poured simultaneously during the turn.
- If an operation asks you to pour more than you have left of a soup, pour all that remains of that soup.

The process stops immediately after any turn in which one of the soups is used up.
Return the probability that A is used up before B, plus half the probability that both soups are used up in the same turn.
Answers within 10-5 of the actual answer will be accepted.

Example 1:

```
Input: n = 50
Output: 0.62500
Explanation:
If we perform either of the first two serving operations, soup A will become empty first.
If we perform the third operation, A and B will become empty at the same time.
If we perform the fourth operation, B will become empty first.
So the total probability of A becoming empty first plus half the probability that A and B become empty at the same time, is 0.25
* (1 + 1 + 0.5 + 0) = 0.625.
```

Example 2:

```
Input: n = 100
Output: 0.71875
Explanation:
If we perform the first serving operation, soup A will become empty first.
If we perform the second serving operations, A will become empty on performing operation [1, 2, 3], and both A and B become
empty on performing operation 4.
If we perform the third operation, A will become empty on performing operation [1, 2], and both A and B become empty on
performing operation 3.
If we perform the fourth operation, A will become empty on performing operation 1, and both A and B become empty on performing
operation 2.
So the total probability of A becoming empty first plus half the probability that A and B become empty at the same time, is
0.71875.
```

Constraints:

- 0 <= n <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Soup A and B, serve in 4 operations each decreasing amounts. P(A empty first or both simultaneously). Right?"
# "n=50→0.625 ✓ n=1→0.5 ✓"
# Scale by 25 to reduce state space. For large n, probability → 1.0 (proven mathematically). O(n²/625).
# Follow-up: if serving amounts change — redefine the 4 operations.
# Follow-up: if we want P(B empty first) — swap A and B in the recursion.
```

Probability DP

#837 New 21 Game

MED

Open on LeetCode

Math · Dynamic Programming · Sliding Window · Probability and Statistics

Lists: AlgoMaster 600

Problem

Alice plays the following game, loosely based on the card game "21".

Alice starts with 0 points and draws numbers while she has less than k points. During each draw, she gains an integer number of points randomly from the range [1, maxPts], where maxPts is an integer. Each draw is independent and the outcomes have equal probabilities.

Alice stops drawing numbers when she gets k or more points.

Return the probability that Alice has n or fewer points.

Answers within 10-5 of the actual answer are considered accepted.

Example 1:

```
Input: n = 10, k = 1, maxPts = 10
Output: 1.00000
Explanation: Alice gets a single card, then stops.
```

Example 2:

```
Input: n = 6, k = 1, maxPts = 10
Output: 0.60000
Explanation: Alice gets a single card, then stops.
In 6 out of 10 possibilities, she is at or below 6 points.
```

Example 3:

```
Input: n = 21, k = 17, maxPts = 10
Output: 0.73278
```

Constraints:

- 0 <= k <= n <= 104
- 1 <= maxPts <= 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Start with 0 points. Draw random 1..maxPts card until score>=n. P(final score<=k). Right?"
# "n=21,k=21,maxPts=10→1.0 ✓ n=10,k=10,maxPts=10→0.5 ✓"
# Sliding window sum for O(n) DP. Each dp[i]=sum(dp[i-1..i-maxPts])/maxPts. Space O(n).
# Follow-up: if cards have non-uniform probability — replace /maxPts with specific probabilities.
# Follow-up: if we want P(score > k) — 1 - P(score <= k).
```


· #808 | #714 · | · Contents

State Machine DP

Round 8 · Low · Medium · 1 question

State Machine DP

☐ #714 Best Time to Buy and Sell Stock with Transaction Fee MED

Open on LeetCode

Array · Dynamic Programming · Greedy

Lists: AlgoMaster 600

Problem

You are given an array prices where prices[i] is the price of a given stock on the ith day, and an integer fee representing a transaction fee.

Find the maximum profit you can achieve. You may complete as many transactions as you like, but you need to pay the transaction fee for each transaction.

Note:

- You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).
- The transaction fee is only charged once for each stock purchase and sale.

Example 1:

```
Input: prices = [1,3,2,8,4,9], fee = 2
Output: 8
Explanation: The maximum profit can be achieved by:
- Buying at prices[0] = 1
- Selling at prices[3] = 8
- Buying at prices[4] = 4
- Selling at prices[5] = 9
The total profit is ((8 - 1) - 2) + ((9 - 4) - 2) = 8.
```

Example 2:

```
Input: prices = [1,3,7,5,10,3], fee = 3
Output: 6
```

Constraints:

- 1 <= prices.length <= 5 * 104
- 1 <= prices[i] < 5 * 104
- 0 <= fee < 5 * 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Buy/sell with a transaction fee per sale. Maximize profit. Right?"
# "prices=[1,3,2,8,4,9],fee=2→8 ✓"
# Two-state DP: cash (not holding), hold (holding). O(n). Space O(1).
# Follow-up: Best Time to Buy/Sell with Cooldown (LC 309) — add sold state.
# Follow-up: if fee is per purchase instead of sale — move fee to the buy transition.
```


Maths / Geometry

Round 8 · Low · Medium · 8 questions

Maths / Geometry

#172 Factorial Trailing Zeroes

MED

Open on LeetCode

Math

Lists: AlgoMaster 600

Problem

Given an integer n, return the number of trailing zeroes in n!.

Note that n! = n * (n - 1) * (n - 2) * ... * 3 * 2 * 1.

Example 1:

```
Input: n = 3
Output: 0
Explanation: 3! = 6, no trailing zero.
```

Example 2:

```
Input: n = 5
Output: 1
Explanation: 5! = 120, one trailing zero.
```

Example 3:

```
Input: n = 0
Output: 0
```

Constraints:

- 0 <= n <= 104

Follow up: Could you write a solution that works in logarithmic time complexity?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return trailing zeroes in n!. Right?"
# "n=3→0 ✓ n=5→1 ✓ n=30→7 ✓"
# Trailing zeros = min(factors of 2, factors of 5) in n!. Factors of 5 are limiting. O(log n). Space O(1).
# Follow-up: trailing zeros in n! in base b — find min factors of each prime in b.
# Follow-up: Preimage Size of Factorial Zeroes Function (LC 793) — find how many n give exactly k zeros.
```


Maths / Geometry

#204 Count Primes

MED

Open on LeetCode

Array · Math · Enumeration · Number Theory

Lists: AlgoMaster 600

Problem

Given an integer n, return the number of prime numbers that are strictly less than n.

Example 1:

```
Input: n = 10
Output: 4
Explanation: There are 4 prime numbers less than 10, they are 2, 3, 5, 7.
```

Example 2:

```
Input: n = 0
Output: 0
```

Example 3:

```
Input: n = 1
Output: 0
```

Constraints:

- 0 <= n <= 5 * 106

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count primes strictly less than n. Right?"
# "n=10→4 (2,3,5,7) ✓"
# Sieve of Eratosthenes. O(n log log n). Space O(n).
# Follow-up: primes in range [a,b] efficiently — segmented sieve O(sqrt(b) + (b-a) log log b).
# Follow-up: nth prime — sieve up to n*ln(n) as upper bound estimate.
```

Maths / Geometry

#264 Ugly Number II

ME D

Open on LeetCode

Hash Table · Math · Dynamic Programming · Heap (Priority Queue)

Lists: AlgoMaster 600

Problem

An ugly number is a positive integer whose prime factors are limited to 2, 3, and 5.
Given an integer n, return the nth ugly number.

Example 1:

```
Input: n = 10
Output: 12
Explanation: [1, 2, 3, 4, 5, 6, 8, 9, 10, 12] is the sequence of the first 10 ugly numbers.
```

Example 2:

```
Input: n = 1
Output: 1
Explanation: 1 has no prime factors, therefore all of its prime factors are limited to 2, 3, and 5.
```

Constraints:

- 1 <= n <= 1690

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the nth ugly number (only prime factors 2,3,5). 1 is ugly. Right?"
# "n=10→12 (1,2,3,4,5,6,8,9,10,12) ✓"
# Three-pointer DP: each pointer tracks which ugly number to multiply next. O(n). Space O(n).
# Follow-up: Super Ugly Number (LC 313) — k primes instead of 3; k-pointer DP.
# Follow-up: Ugly Number I (LC 263) — check if a given number is ugly; O(log n).
```


Maths / Geometry

#593 Valid Square

ME D

Open on LeetCode

Math · Geometry

Lists: AlgoMaster 600

Problem

Given the coordinates of four points in 2D space p1, p2, p3 and p4, return true if the four points construct a square.
The coordinate of a point pi is represented as [xi, yi]. The input is not given in any order.
A valid square has four equal sides with positive length and four equal angles (90-degree angles).

Example 1:

```
Input: p1 = [0,0], p2 = [1,1], p3 = [1,0], p4 = [0,1]
Output: true
```

Example 2:

```
Input: p1 = [0,0], p2 = [1,1], p3 = [1,0], p4 = [0,12]
Output: false
```

Example 3:

```
Input: p1 = [1,0], p2 = [-1,0], p3 = [0,1], p4 = [0,-1]
Output: true
```

Constraints:

- p1.length == p2.length == p3.length == p4.length == 2
- -104 <= xi, yi <= 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given 4 points, return true if they form a square. Right?"
# "p1=[0,0],p2=[1,1],p3=[1,0],p4=[0,1]→true ✓"
# 4 equal sides + 2 equal diagonals (larger). O(1). Space O(1).
# Follow-up: Valid Rectangle — 2 pairs of equal sides + equal diagonals.
# Follow-up: Minimum Area Rectangle (LC 939) — find smallest rectangle from point set.
```

Maths / Geometry

#611 Valid Triangle Number

MED

Open on LeetCode

Array · Two Pointers · Binary Search · Greedy · Sorting

Lists: AlgoMaster 600

Problem

Given an integer array nums, return the number of triplets chosen from the array that can make triangles if we take them as side lengths of a triangle.

Example 1:

```
Input: nums = [2,2,3,4]
Output: 3
Explanation: Valid combinations are:
2,3,4 (using the first 2)
2,3,4 (using the second 2)
2,2,3
```

Example 2:

```
Input: nums = [4,2,3,4]
Output: 4
```

Constraints:

- 1 <= nums.length <= 1000
- 0 <= nums[i] <= 1000

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count triplets (i<j<k) where nums[i],nums[j],nums[k] form a valid triangle. Right?"
# "nums=[2,2,3,4]→3 ✓"
# Fix largest side k, two-pointer on remaining. O(n²). Space O(1).
# Follow-up: if we need the actual triplets — collect indices during two-pointer scan.
# Follow-up: count invalid triangles — subtract valid from C(n,3).
```


Maths / Geometry

#939 Minimum Area Rectangle

MED

Open on LeetCode

Array · Hash Table · Math · Geometry · Sorting

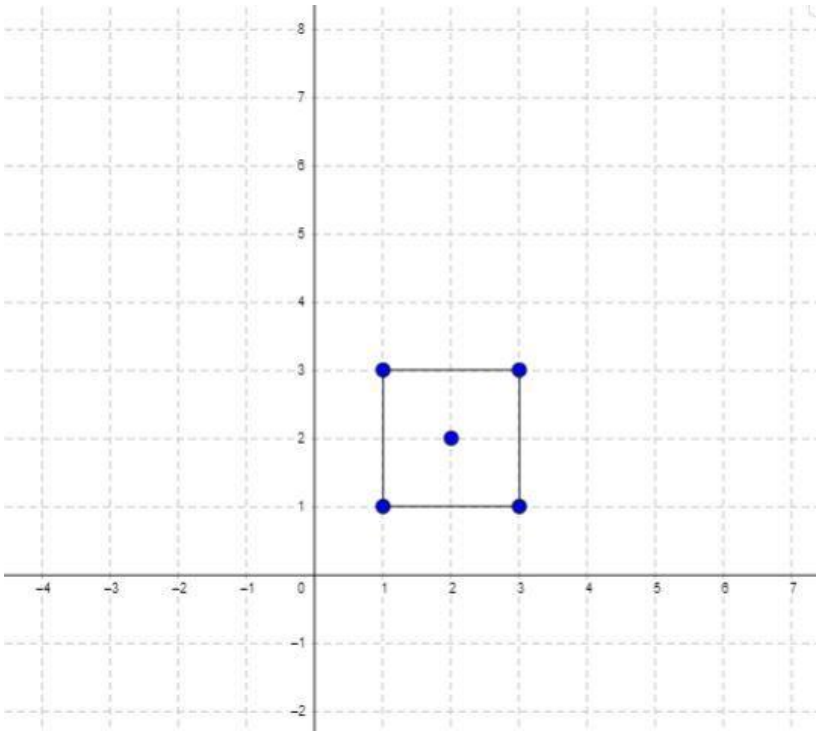
Lists: AlgoMaster 600

Problem

You are given an array of points in the X-Y plane points where points[i] = [xi, yi].

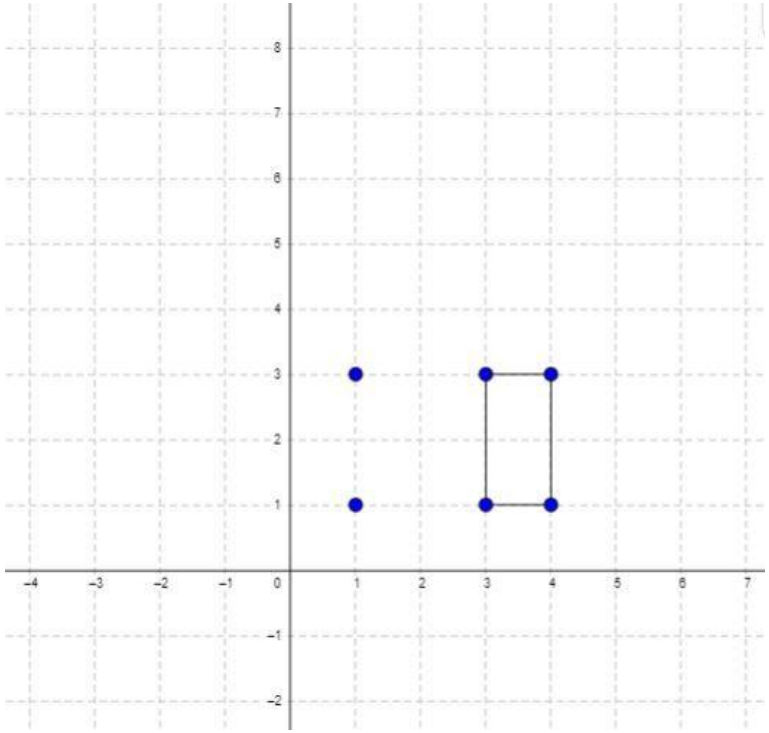
Return the minimum area of a rectangle formed from these points, with sides parallel to the X and Y axes. If there is not any such rectangle, return 0.

Example 1:




```
Input: points = [[1,1],[1,3],[3,1],[3,3],[2,2]]
Output: 4
```

Example 2:



```
Input: points = [[1,1],[1,3],[3,1],[3,3],[4,1],[4,3]]
Output: 2
```

Constraints:

- 1 <= points.length <= 500
- points[i].length == 2
- 0 <= xi, yi <= 4 * 104
- All the given points are unique.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Points aligned to axes. Find minimum area rectangle (axis-aligned). Return 0 if none. Right?"
# "points=[[1,1],[1,3],[3,1],[3,3],[2,2]]→4 ✓"
# Fix diagonal corners, check other two corners exist. O(n²). Space O(n).
# Follow-up: Minimum Area Rectangle II (LC 963) — rectangles not necessarily axis-aligned.
# Follow-up: count all axis-aligned rectangles — same approach, count instead of min.
```


Maths / Geometry

#963 Minimum Area Rectangle II ME D

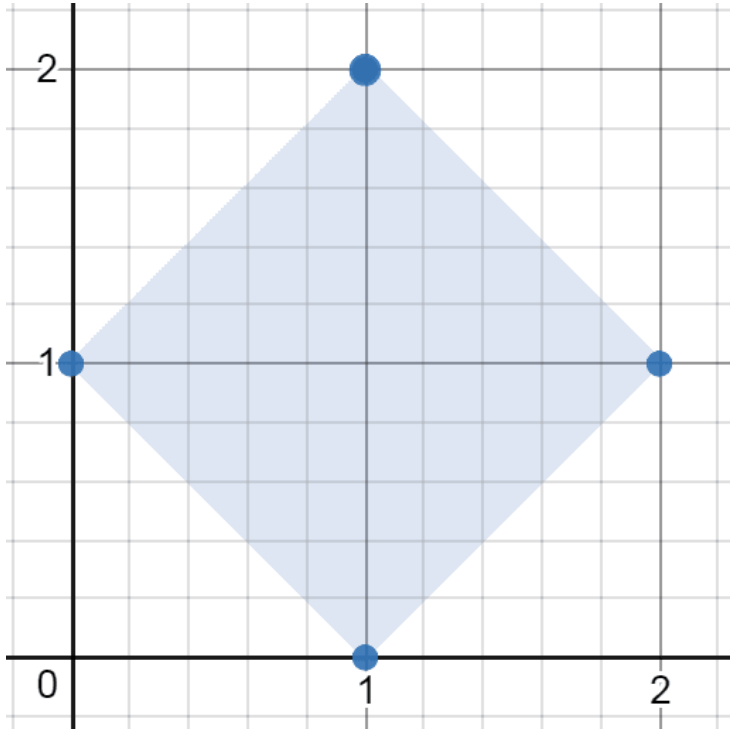
Open on LeetCode

Array · Hash Table · Math · Geometry

Lists: AlgoMaster 600

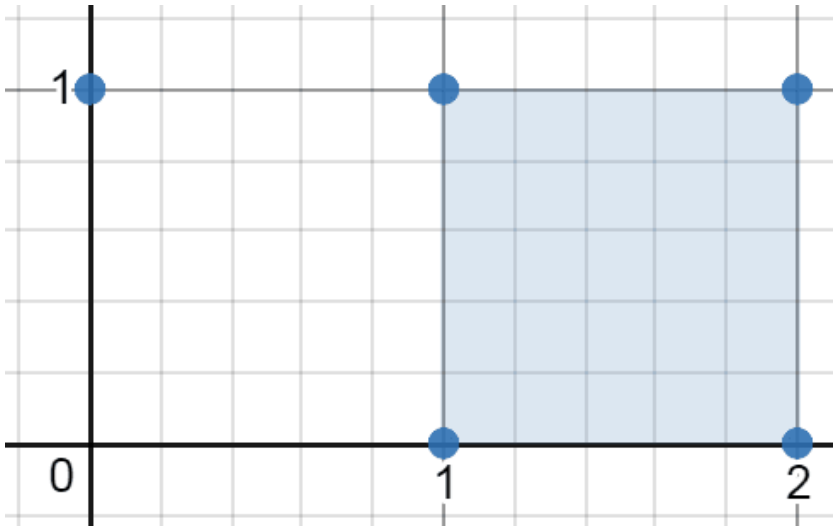
Problem

You are given an array of points in the X-Y plane points where points[i] = [xi, yi].
Return the minimum area of any rectangle formed from these points, with sides not necessarily parallel to the X and Y axes. If there is not any such rectangle, return 0.
Answers within 10-5 of the actual answer will be accepted.
Example 1:



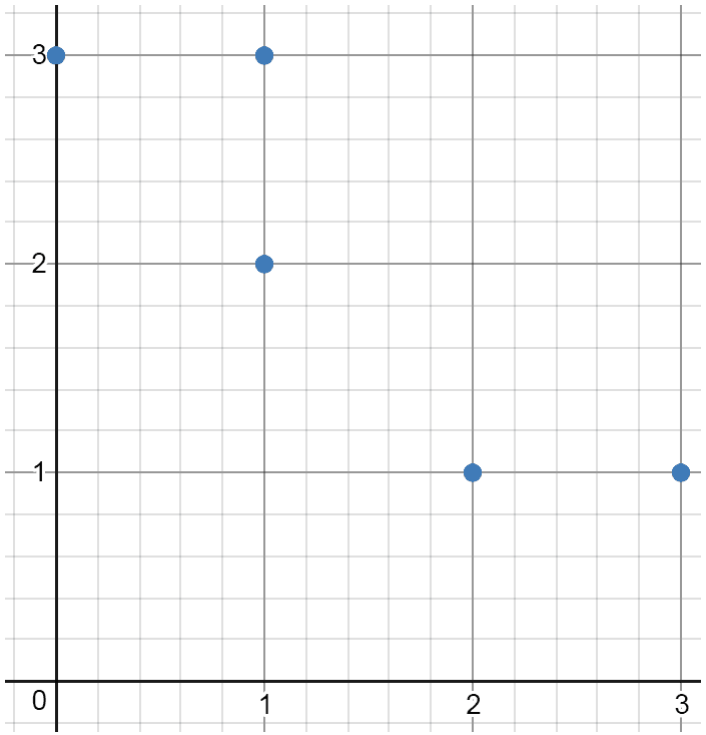

```
Input: points = [[1,2],[2,1],[1,0],[0,1]]
Output: 2.00000
Explanation: The minimum area rectangle occurs at [1,2],[2,1],[1,0],[0,1], with an area of 2.
```

Example 2:




```
Input: points = [[0,1],[2,1],[1,1],[1,0],[2,0]]
Output: 1.00000
Explanation: The minimum area rectangle occurs at [1,0],[1,1],[2,1],[2,0], with an area of 1.
```

Example 3:




```
Input: points = [[0,3],[1,2],[3,1],[1,3],[2,1]]
Output: 0
Explanation: There is no possible rectangle to form from these points.
```

Constraints:

- 1 <= points.length <= 50
- points[i].length == 2
- 0 <= xi, yi <= 4 * 104
- All the given points are unique.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Find minimum area rectangle (any orientation) from point set. Return 0 if none. Right?"
# "points=[[1,2],[2,1],[1,0],[0,1]]→2.0 ✓"
# Fix diagonal, find shared center, check perpendicularity. O(n³). Space O(n).
# Follow-up: minimum area rectangle aligned to axes (LC 939) — simpler O(n²).
# Follow-up: count all rectangles — same approach, count pairs forming valid rectangles.
```

Maths / Geometry

#1922 Count Good Numbers

MED

Open on LeetCode

Math · Recursion

Lists: AlgoMaster 600

Problem

A digit string is good if the digits (0-indexed) at even indices are even and the digits at odd indices are prime (2, 3, 5, or 7).

- For example, "2582" is good because the digits (2 and 8) at even positions are even and the digits (5 and 2) at odd positions are prime. However, "3245" is not good because 3 is at an even index but is not even.

Given an integer n, return the total number of good digit strings of length n. Since the answer may be large, return it modulo 109 + 7.

A digit string is a string consisting of digits 0 through 9 that may contain leading zeros.

Example 1:

```
Input: n = 1
Output: 5
Explanation: The good numbers of length 1 are "0", "2", "4", "6", "8".
```

Example 2:

```
Input: n = 4
Output: 400
```

Example 3:

```
Input: n = 50
Output: 564908303
```

Constraints:

- 1 <= n <= 1015

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "A number is 'good' if even-indexed digits are even and odd-indexed digits are prime.
# Count good digit strings of length n, mod 10^9+7. Right?"
# "n=1→5 ✓ n=4→400 ✓"
# Even positions: 5 choices (0,2,4,6,8). Odd positions: 4 choices (2,3,5,7). Fast exponentiation. O(log n).
# Follow-up: if digit constraints change — update base for each position type.
# Follow-up: count good numbers in [1,n] — digit DP with positional constraints.
```


String Matching

Round 8 · Low · Medium · 2 questions

String Matching

#686 Repeated String Match

ME
D

Open on LeetCode

String · String Matching
Lists: AlgoMaster 600

Problem

Given two strings a and b, return the minimum number of times you should repeat string a so that string b is a substring of it. If it is impossible for b to be a substring of a after repeating it, return -1.
Notice: string "abc" repeated 0 times is "", repeated 1 time is "abc" and repeated 2 times is "abcabc".

Example 1:

```
Input: a = "abcd", b = "cdabcdab"
Output: 3
Explanation: We return 3 because by repeating a three times "abcdabcdabcd", b is a substring of it.
```

Example 2:

```
Input: a = "a", b = "aa"
Output: 2
```

Constraints:

- 1 <= a.length, b.length <= 10^4
- a and b consist of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return minimum times to repeat a to contain b as a substring. -1 if impossible. Right?"
# "a='abcd',b='cdabcdab'→3 ✓ a='a',b='aa'→2 ✓"
# Repeat a ceil(m/n) times, check b, then +1. O(m) per check. Space O(n+m).
# Follow-up: KMP for O(n+m) substring check instead of Python's O(nm).
# Follow-up: find all starting positions of b in repeated a — rolling hash.
```


String Matching

#1698 Number of Distinct Substrings in a String

ME
D

Open on LeetCode

String · Trie · Rolling Hash · Suffix Array · Hash Function
Lists: AlgoMaster 600

Problem

Given a string s, return the number of distinct substrings of s.
A substring of a string is obtained by deleting any number of characters (possibly zero) from the front of the string and any number (possibly zero) from the back of the string.

Example 1:

```
Input: s = "aabbaba"
Output: 21
Explanation: The set of distinct strings is ["a","b","aa","bb","ab","ba","aab","abb","bab","bba","aba","aabb","abba","bbab","baba","aabba","abbab","bbaba","aabbab","abbaba","aabbaba"]
```

Example 2:

```
Input: s = "abcdefg"
Output: 28
```

Constraints:

- 1 <= s.length <= 500
- s consists of lowercase English letters.

Follow up: Can you solve this problem in O(n) time complexity?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return the number of distinct non-empty substrings. Right?"
# "s='aabbaba'→21 ✓"
# O(n³) brute using set. Suffix Array approach: n*(n+1)/2 − sum(LCP array) in O(n log n).
# Rolling hash: O(n²) average.
# Follow-up: Longest Duplicate Substring (LC 1044) — find the actual longest duplicate substring.
# Follow-up: if pattern must have length ≥ k — filter in the set.
```

Binary Indexed Tree / Segment Tree

Round 8 · Low · Medium · 1 question

Binary Indexed Tree / Segment Tree

#308 Range Sum Query 2D - Mutable

MED

Open on LeetCode

Array · Design · Binary Indexed Tree · Segment Tree · Matrix

Lists: AlgoMaster 600

Problem

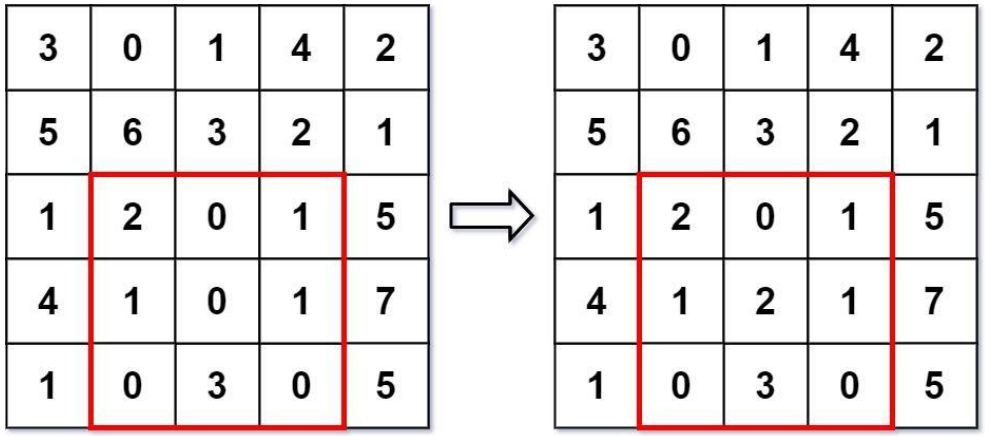
Given a 2D matrix matrix, handle multiple queries of the following types:

1. Update the value of a cell in matrix.
2. Calculate the sum of the elements of matrix inside the rectangle defined by its upper left corner (row1, col1) and lower right corner (row2, col2).

Implement the NumMatrix class:

- NumMatrix(int[][] matrix) Initializes the object with the integer matrix matrix.
- void update(int row, int col, int val) Updates the value of matrix[row][col] to be val.
- int sumRegion(int row1, int col1, int row2, int col2) Returns the sum of the elements of matrix inside the rectangle defined by its upper left corner (row1, col1) and lower right corner (row2, col2).

Example 1:




```
Input
["NumMatrix", "sumRegion", "update", "sumRegion"]
[[[[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]]], [2, 1, 4, 3], [3, 2, 2], [2, 1, 4, 3]]
Output
[null, 8, null, 10]

Explanation
NumMatrix numMatrix = new NumMatrix([[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]]);
```

Constraints:

- m == matrix.length
- n == matrix[i].length
- 1 <= m, n <= 200
- -1000 <= matrix[i][j] <= 1000
- 0 <= row < m
- 0 <= col < n
- -1000 <= val <= 1000
- 0 <= row1 <= row2 < m
- 0 <= col1 <= col2 < n
- At most 5000 calls will be made to sumRegion and update.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Matrix supports update(r,c,val) and sumRegion(r1,c1,r2,c2). Both O(log mn). Right?"
# "matrix=[[3,0,1,4,2],...]: sumRegion(2,1,4,3)=8, update(3,2,2), sumRegion(2,1,4,3)=10 ✓"
# 2D Binary Indexed Tree (Fenwick Tree). O(log²(mn)) per update and query. Space O(mn).
```

```
# Follow-up: Range Sum Query 2D Immutable (LC 304) — 2D prefix sum, O(1) query no updates.
# Follow-up: 2D Segment Tree — O(log²(mn)) with lazy propagation for range updates.
```


Round 9

Low Priority · Hard
36 questions · 15 patterns

- [ ] 🔴 Bucket Sort #220 ↗
- [ ] 🔴 Eulerian Circuit #753 #2097 ↗
- [ ] 🔴 1-D DP #1411 ↗
- [ ] 🔴 0/1 Knapsack #879 ↗
- [ ] 🔴 Longest Increasing Subsequence (LIS) #354 ↗
- [ ] 🔴 2D Grid DP #85 #174 #403 #465 #546 #741 #1335 #1547 #1931 #3651 ↗
- [ ] 🔴 String DP #44 #140 #664 #1092 ↗
- [ ] 🔴 Tree / Graph DP #834 #968 ↗
- [ ] 🔴 Bitmask DP #847 ↗
- [ ] 🔴 Digit DP #233 #902 ↗
- [ ] 🔴 State Machine DP #188 ↗
- [ ] 🔴 Maths / Geometry #149 ↗
- [ ] 🔴 String Matching #214 #1044 #1392 ↗
- [ ] 🔴 Binary Indexed Tree / Segment Tree #699 #2426 #3161 #3721 ↗
- [ ] 🔴 Line Sweep #391 #850 ↗

Bucket Sort

Round 9 · Low · Hard · 1 question

Bucket Sort

#220 Contains Duplicate III

HAR

Open on LeetCode

Array · Sliding Window · Sorting · Bucket Sort · Ordered Set

Lists: AlgoMaster 600

Problem

You are given an integer array nums and two integers indexDiff and valueDiff.

Find a pair of indices (i, j) such that:

- i != j,
- abs(i - j) <= indexDiff.
- abs(nums[i] - nums[j]) <= valueDiff, and

Return true if such pair exists or false otherwise.

Example 1:

```
Input: nums = [1,2,3,1], indexDiff = 3, valueDiff = 0
Output: true
Explanation: We can choose (i, j) = (0, 3).
We satisfy the three conditions:
i != j --> 0 != 3
abs(i - j) <= indexDiff --> abs(0 - 3) <= 3
abs(nums[i] - nums[j]) <= valueDiff --> abs(1 - 1) <= 0
```

Example 2:

```
Input: nums = [1,5,9,1,5,9], indexDiff = 2, valueDiff = 3
Output: false
Explanation: After trying all the possible pairs (i, j), we cannot satisfy the three conditions, so we return false.
```

Constraints:

- 2 <= nums.length <= 105
- -109 <= nums[i] <= 109
- 1 <= indexDiff <= nums.length
- 0 <= valueDiff <= 109

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Return true if any i≠j with |i-j|<=k and |nums[i]-nums[j]|<=t. Right?"
# "nums=[1,2,3,1],k=3,t=0→true ✓ nums=[1,5,9,1,5,9],k=2,t=3→false ✓"
# Bucket sort with window size t+1. Each bucket contains at most one value. O(n). Space O(k).
# Follow-up: Contains Duplicate II (LC 219) — t=0, use sliding window set O(n).
# Follow-up: if k and t are very large — sliding window with sorted structure O(n log k).
```

Eulerian Circuit

Round 9 · Low · Hard · 2 questions

Eulerian Circuit

#753 Cracking the Safe

HAR

Open on LeetCode

String · Depth-First Search · Graph Theory · Eulerian Circuit

Lists: AlgoMaster 600

Problem

There is a safe protected by a password. The password is a sequence of n digits where each digit can be in the range [0, k - 1].

The safe has a peculiar way of checking the password. When you enter in a sequence, it checks the most recent n digits that were entered each time you type a digit.

- For example, the correct password is "345" and you enter in "012345": After typing 0, the most recent 3 digits is "0", which is incorrect.
- After typing 1, the most recent 3 digits is "01", which is incorrect.
- After typing 2, the most recent 3 digits is "012", which is incorrect.
- After typing 3, the most recent 3 digits is "123", which is incorrect.
- After typing 4, the most recent 3 digits is "234", which is incorrect.
- After typing 5, the most recent 3 digits is "345", which is correct and the safe unlocks.

Return any string of minimum length that will unlock the safe at some point of entering it.

Example 1:

```
Input: n = 1, k = 2
Output: "10"
Explanation: The password is a single digit, so enter each digit. "01" would also unlock the safe.
```

Example 2:

```
Input: n = 2, k = 2
Output: "01100"
Explanation: For each possible password:
- "00" is typed in starting from the 4th digit.
- "01" is typed in starting from the 1st digit.
- "10" is typed in starting from the 3rd digit.
- "11" is typed in starting from the 2nd digit.
Thus "01100" will unlock the safe. "10011", and "11001" would also unlock the safe.
```

Constraints:

- 1 <= n <= 4
- 1 <= k <= 10
- 1 <= kn <= 4096

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Find shortest string containing all k-length sequences over 0..n-1 digits (De Bruijn sequence). Right?"
# "n=2,k=2→'01100' or similar (all 2-digit binary: 00,01,10,11) ✓"
        # Hierholzer's algorithm on De Bruijn graph
        # Each node = (k-1)-length sequence, edges = append digit
# Hierholzer's algorithm on De Bruijn graph. O(n^k). Space O(n^k).
# Follow-up: de Bruijn sequence for given alphabet — same algorithm.
# Follow-up: Valid Arrangement of Pairs (LC 2097) — general Eulerian path.
```

Eulerian Circuit

#2097 Valid Arrangement of Pairs HAR

Open on LeetCode

Array · Depth-First Search · Graph Theory · Eulerian Circuit

Lists: AlgoMaster 600

Problem

You are given a 0-indexed 2D integer array pairs where pairs[i] = [starti, endi]. An arrangement of pairs is valid if for every index i where 1 <= i < pairs.length, we have endi-1 == starti.

Return any valid arrangement of pairs.

Note: The inputs will be generated such that there exists a valid arrangement of pairs.

Example 1:

```
Input: pairs = [[5,1],[4,5],[11,9],[9,4]]
Output: [[11,9],[9,4],[4,5],[5,1]]
Explanation:
This is a valid arrangement since endi-1 always equals starti.
end0 = 9 == 9 = start1
end1 = 4 == 4 = start2
end2 = 5 == 5 = start3
```

Example 2:

```
Input: pairs = [[1,3],[3,2],[2,1]]
Output: [[1,3],[3,2],[2,1]]
Explanation:
This is a valid arrangement since endi-1 always equals starti.
end0 = 3 == 3 = start1
end1 = 2 == 2 = start2
The arrangements [[2,1],[1,3],[3,2]] and [[3,2],[2,1],[1,3]] are also valid.
```

Example 3:

```
Input: pairs = [[1,2],[1,3],[2,1]]
Output: [[1,2],[2,1],[1,3]]
Explanation:
This is a valid arrangement since endi-1 always equals starti.
end0 = 2 == 2 = start1
end1 = 1 == 1 = start2
```

Constraints:

- 1 <= pairs.length <= 105
- pairs[i].length == 2
- 0 <= starti, endi <= 109
- starti != endi
- No two pairs are exactly the same.
- There exists a valid arrangement of pairs.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Arrange pairs so pairs[i][1]==pairs[i+1][0]. Return valid arrangement. Right?"
# "pairs=[[5,1],[4,5],[11,9],[9,4]]→[[11,9],[9,4],[4,5],[5,1]] ✓"
        # Find start: node with out_deg > in_deg
# Hierholzer's algorithm for Eulerian path. O(V+E). Space O(V+E).
# Follow-up: Cracking the Safe (LC 753) — same Eulerian circuit on De Bruijn graph.
# Follow-up: if no valid arrangement exists — detect by checking if graph has Eulerian path conditions.
```


1-D DP

Round 9 · Low · Hard · 1 question

1-D DP

#1411 Number of Ways to Paint N × 3 Grid

HAR

Open on LeetCode

Dynamic Programming

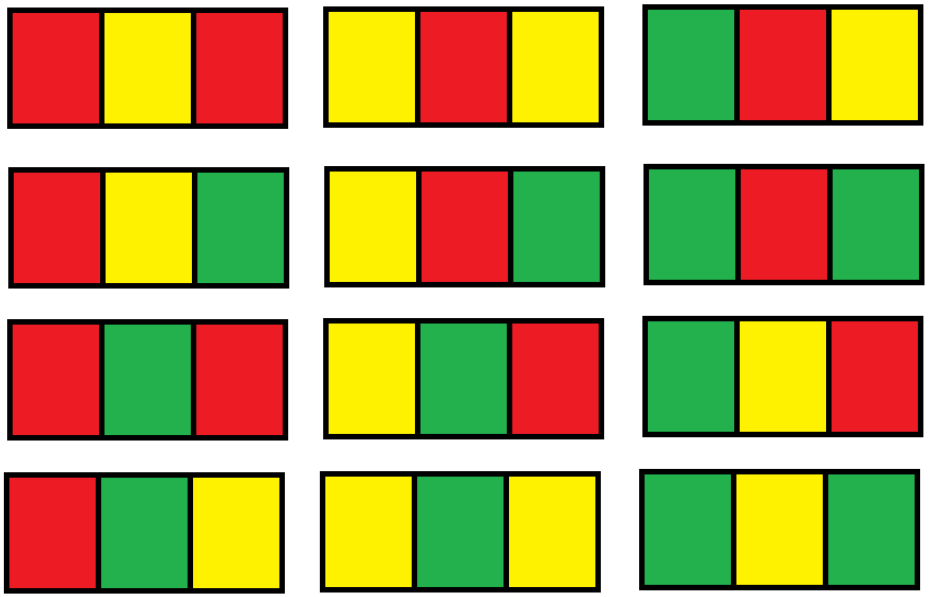
Lists: AlgoMaster 600

Problem

You have a grid of size n x 3 and you want to paint each cell of the grid with exactly one of the three colors: Red, Yellow, or Green while making sure that no two adjacent cells have the same color (i.e., no two cells that share vertical or horizontal sides have the same color).

Given n the number of rows of the grid, return the number of ways you can paint this grid. As the answer may grow large, the answer must be computed modulo 109 + 7.

Example 1:



```
Input: n = 1
Output: 12
Explanation: There are 12 possible way to paint the grid as shown.
```

Example 2:

```
Input: n = 5000
Output: 30228214
```

Constraints:

- n == grid.length
- 1 <= n <= 5000

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Paint N×3 grid with 3 colors, adjacent cells different. Count ways mod 10^9+7. Right?"
# "n=1→12 ✓ n=2→54 ✓"
        # Pattern types: 'ABA' and 'ABC'
# Classify row patterns as ABA (palindrome) or ABC (all different). Track transition counts. O(n). Space O(1).
# Follow-up: N×4 grid — more pattern types, same DP approach.
# Follow-up: if diagonal adjacency also matters — more complex state transitions.
```


0/1 Knapsack

Round 9 · Low · Hard · 1 question

0/1 Knapsack

#879 Profitable Schemes HAR

Open on LeetCode

Array · Dynamic Programming

Lists: AlgoMaster 600

Problem

There is a group of n members, and a list of various crimes they could commit. The ith crime generates a profit[i] and requires group[i] members to participate in it. If a member participates in one crime, that member can't participate in another crime.

Let's call a profitable scheme any subset of these crimes that generates at least minProfit profit, and the total number of members participating in that subset of crimes is at most n.

Return the number of schemes that can be chosen. Since the answer may be very large, return it modulo 109 + 7.

Example 1:

```
Input: n = 5, minProfit = 3, group = [2,2], profit = [2,3]
Output: 2
Explanation: To make a profit of at least 3, the group could either commit crimes 0 and 1, or just crime 1.
In total, there are 2 schemes.
```

Example 2:

```
Input: n = 10, minProfit = 5, group = [2,3,5], profit = [6,7,8]
Output: 7
Explanation: To make a profit of at least 5, the group could commit any crimes, as long as they commit one.
There are 7 possible schemes: (0), (1), (2), (0,1), (0,2), (1,2), and (0,1,2).
```

Constraints:

- 1 <= n <= 100
- 0 <= minProfit <= 100
- 1 <= group.length <= 100
- 1 <= group[i] <= 100
- profit.length == group.length
- 0 <= profit[i] <= 100

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Group of n criminals. Crimes need group[i] members, produce profit[i]. Count schemes using <=n members with >=minProfit.
Right?"
# "n=5,minProfit=3,group=[2,2],profit=[2,3]→2 ✓"
# 2D knapsack: members and profit. Cap profit at minProfit. O(n*minProfit*|crimes|). Space O(n*minProfit).
# Follow-up: if crimes can be done multiple times — unbounded knapsack (iterate forward).
# Follow-up: count schemes using exactly n members — remove the sum loop.
```


Longest Increasing Subsequence (LIS)

Round 9 · Low · Hard · 1 question

Longest Increasing Subsequence (LIS)

#354 Russian Doll Envelopes HAR

Open on LeetCode

Array · Binary Search · Dynamic Programming · Sorting

Lists: AlgoMaster 600

Problem

You are given a 2D array of integers envelopes where envelopes[i] = [wi, hi] represents the width and the height of an envelope.

One envelope can fit into another if and only if both the width and height of one envelope are greater than the other envelope's width and height.

Return the maximum number of envelopes you can Russian doll (i.e., put one inside the other).

Note: You cannot rotate an envelope.

Example 1:

```
Input: envelopes = [[5,4],[6,4],[6,7],[2,3]]
Output: 3
Explanation: The maximum number of envelopes you can Russian doll is 3 ([2,3] => [5,4] => [6,7]).
```

Example 2:

```
Input: envelopes = [[1,1],[1,1],[1,1]]
Output: 1
```

Constraints:

- 1 <= envelopes.length <= 105
- envelopes[i].length == 2
- 1 <= wi, hi <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Envelope A fits in B if A.w<B.w AND A.h<B.h. Find max nesting depth. Right?"
# "envelopes=[[5,4],[6,4],[6,7],[2,3]]→3 ✓ (2,3)→(5,4)→(6,7)"
        # LIS on heights
# Sort by width asc, height desc (prevents using same-width envelopes twice). LIS on height. O(n log n).
# Follow-up: 3D variant (width,height,depth) — NP-hard in general, greedy heuristics.
# Follow-up: Longest Increasing Subsequence (LC 300) — simpler, single dimension.
```


2D Grid DP

Round 9 · Low · Hard · 10 questions

2D Grid DP

#85 Maximal Rectangle HAR

Open on LeetCode

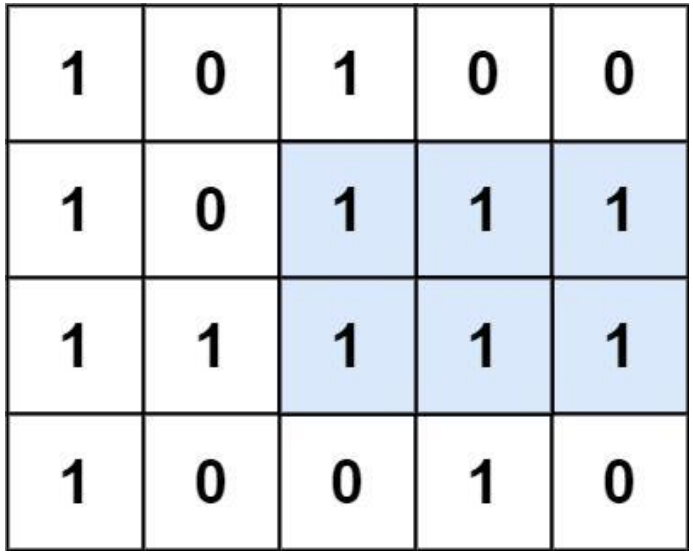
Array · Dynamic Programming · Stack · Matrix · Monotonic Stack

Lists: AlgoMaster 600

Problem

Given a rows x cols binary matrix filled with 0's and 1's, find the largest rectangle containing only 1's and return its area.

Example 1:




```
Input: matrix = [["1","0","1","0","0"],["1","0","1","1","1"],["1","1","1","1","1"],["1","0","0","1","0"]]
Output: 6
Explanation: The maximal rectangle is shown in the above picture.
```

Example 2:

```
Input: matrix = [["0"]]
Output: 0
```

Example 3:

```
Input: matrix = [["1"]]
Output: 1
```

Constraints:

- rows == matrix.length
- cols == matrix[i].length
- 1 <= rows, cols <= 200
- matrix[i][j] is '0' or '1'.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the largest rectangle containing only 1s in binary matrix. Right?"
# "matrix=[['1','0','1','0','0'],['1','0','1','1','1'],['1','1','1','1','1'],['1','0','0','1','0']]→6 ✓"
            # Largest rectangle in histogram
# Row-by-row histogram + Largest Rectangle in Histogram (LC 84). O(mn). Space O(n).
# Follow-up: Maximal Square (LC 221) — only squares, O(mn) DP without histogram.
# Follow-up: count all maximal rectangles — much harder, exponential in worst case.
```


2D Grid DP

#174 Dungeon Game HAR

Open on LeetCode

Array · Dynamic Programming · Matrix

Lists: AlgoMaster 600

Problem

The demons had captured the princess and imprisoned her in the bottom-right corner of a dungeon. The dungeon consists of m x n rooms laid out in a 2D grid. Our valiant knight was initially positioned in the top-left room and must fight his way through dungeon to rescue the princess.

The knight has an initial health point represented by a positive integer. If at any point his health point drops to 0 or below, he dies immediately.

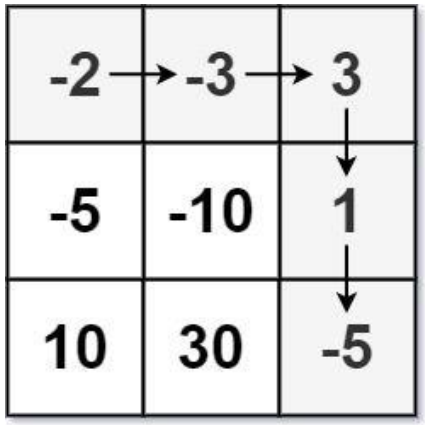
Some of the rooms are guarded by demons (represented by negative integers), so the knight loses health upon entering these rooms; other rooms are either empty (represented as 0) or contain magic orbs that increase the knight's health (represented by positive integers).

To reach the princess as quickly as possible, the knight decides to move only rightward or downward in each step.

Return the knight's minimum initial health so that he can rescue the princess.

Note that any room can contain threats or power-ups, even the first room the knight enters and the bottom-right room where the princess is imprisoned.

Example 1:




```
Input: dungeon = [[-2,-3,3],[-5,-10,1],[10,30,-5]]
Output: 7
Explanation: The initial health of the knight must be at least 7 if he follows the optimal path: RIGHT-> RIGHT -> DOWN -> DOWN.
```

Example 2:

```
Input: dungeon = [[0]]
Output: 1
```

Constraints:

- m == dungeon.length
- n == dungeon[i].length
- 1 <= m, n <= 200
- -1000 <= dungeon[i][j] <= 1000

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Knight in dungeon[m][n], starts top-left, ends bottom-right. Must always have >=1 HP.
# Cells have positive/negative values. Find min initial HP. Right?"
# "dungeon=[[-2,-3,3],[-5,-10,1],[10,30,-5]]→7 ✓"
# Bottom-up from destination: min HP needed to survive from each cell. O(mn). Space O(mn) or O(n).
# Follow-up: if knight can also move up/left — more complex DP with cycles (Bellman-Ford).
# Follow-up: find the actual path — track choices during DP traversal.
```

2D Grid DP

#403 Frog Jump

HAR

Open on LeetCode

Array · Dynamic Programming

Lists: AlgoMaster 600

Problem

A frog is crossing a river. The river is divided into some number of units, and at each unit, there may or may not exist a stone. The frog can jump on a stone, but it must not jump into the water.

Given a list of stones positions (in units) in sorted ascending order, determine if the frog can cross the river by landing on the last stone. Initially, the frog is on the first stone and assumes the first jump must be 1 unit.

If the frog's last jump was k units, its next jump must be either k - 1, k, or k + 1 units. The frog can only jump in the forward direction.

Example 1:

```
Input: stones = [0,1,3,5,6,8,12,17]
Output: true
Explanation: The frog can jump to the last stone by jumping 1 unit to the 2nd stone, then 2 units to the 3rd stone, then 2 units to the 4th stone, then 3 units to the 6th stone, 4 units to the 7th stone, and 5 units to the 8th stone.
```

Example 2:

```
Input: stones = [0,1,2,3,4,8,9,11]
Output: false
Explanation: There is no way to jump to the last stone as the gap between the 5th and 6th stone is too large.
```

Constraints:

- 2 <= stones.length <= 2000
- 0 <= stones[i] <= 231 - 1
- stones[0] == 0
- stones is sorted in a strictly increasing order.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Frog at stone 0. Last jump was 1 step. Next jump can be k-1, k, or k+1. Can it reach last stone? Right?"
# "stones=[0,1,3,5,6,8,12,17]→true ✓"
# DP: each stone maps to set of possible jump sizes that can reach it. O(n²). Space O(n²).
# Follow-up: return the actual path if it exists — track parent stone in DP.
# Follow-up: if jump size can vary by ±2 — add k-2 and k+2 to the iteration.
```


2D Grid DP

#465 Optimal Account Balancing

HAR

Open on LeetCode

Array · Dynamic Programming · Backtracking · Bit Manipulation · Bitmask

Lists: AlgoMaster 600

Problem

You are given an array of transactions transactions where transactions[i] = [fromi, toi, amounti] indicates that the person with ID = fromi gave amounti $ to the person with ID = toi.

Return the minimum number of transactions required to settle the debt.

Example 1:

```
Input: transactions = [[0,1,10],[2,0,5]]
Output: 2
Explanation:
Person #0 gave person #1 $10.
Person #2 gave person #0 $5.
Two transactions are needed. One way to settle the debt is person #1 pays person #0 and #2 $5 each.
```

Example 2:

```
Input: transactions = [[0,1,10],[1,0,1],[1,2,5],[2,0,5]]
Output: 1
Explanation:
Person #0 gave person #1 $10.
Person #1 gave person #0 $1.
Person #1 gave person #2 $5.
Person #2 gave person #0 $5.
Therefore, person #1 only need to give person #0 $4, and all debt is settled.
```

Constraints:

- 1 <= transactions.length <= 8
- transactions[i].length == 3
- 0 <= fromi, toi < 12
- fromi != toi
- 1 <= amounti <= 100

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Minimize transactions to settle debts between friends. Right?"
# "transactions=[[0,1,10],[2,0,5]]→2 ✓"
# Backtracking on debt array. O(n!) worst. Space O(n).
# Follow-up: bitmask DP for O(2^n * n) — feasible for n≤20.
# Follow-up: if transaction costs vary — adjust to minimize weighted sum.
```

2D Grid DP

#546 Remove Boxes

HAR

Open on LeetCode

Array · Dynamic Programming · Memoization

Lists: AlgoMaster 600

Problem

You are given several boxes with different colors represented by different positive numbers.

You may experience several rounds to remove boxes until there is no box left. Each time you can choose some continuous boxes with the same color (i.e., composed of k boxes, k >= 1), remove them and get k * k points.

Return the maximum points you can get.

Example 1:

```
Input: boxes = [1,3,2,2,2,3,4,3,1]
Output: 23
Explanation:
[1, 3, 2, 2, 2, 3, 4, 3, 1]
----> [1, 3, 3, 4, 3, 1] (3*3=9 points)
----> [1, 3, 3, 3, 1] (1*1=1 points)
----> [1, 1] (3*3=9 points)
----> [] (2*2=4 points)
```

Example 2:

```
Input: boxes = [1,1,1]
Output: 9
```

Example 3:

```
Input: boxes = [1]
Output: 1
```

Constraints:

- 1 <= boxes.length <= 100
- 1 <= boxes[i] <= 100

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Remove boxes, score = k² where k = length of same-color consecutive group removed. Maximize score. Right?"
# "boxes=[1,3,2,2,2,3,4,3,1]→23 ✓"
            # Extend current group with same-color boxes
# State: dp(l,r,k) = max score for boxes[l..r] with k extra same-color boxes attached to r. O(n⁴). Space O(n³).
# Follow-up: Strange Printer (LC 664) — similar interval DP but for printing.
# Follow-up: if removal cost isn't k² but f(k) — same structure, replace (k+1)².
```


2D Grid DP

#741 Cherry Pickup

HAR

Open on LeetCode

Array · Dynamic Programming · Matrix

Lists: AlgoMaster 600

Problem

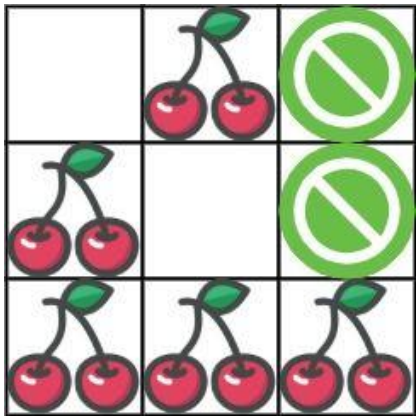
You are given an n x n grid representing a field of cherries, each cell is one of three possible integers.

- 0 means the cell is empty, so you can pass through,
- 1 means the cell contains a cherry that you can pick up and pass through, or
- -1 means the cell contains a thorn that blocks your way.

Return the maximum number of cherries you can collect by following the rules below:

- Starting at the position (0, 0) and reaching (n - 1, n - 1) by moving right or down through valid path cells (cells with value 0 or 1).
- After reaching (n - 1, n - 1), returning to (0, 0) by moving left or up through valid path cells.
- When passing through a path cell containing a cherry, you pick it up, and the cell becomes an empty cell 0.
- If there is no valid path between (0, 0) and (n - 1, n - 1), then no cherries can be collected.

Example 1:



```
Input: grid = [[0,1,-1],[1,0,-1],[1,1,1]]
Output: 5
Explanation: The player started at (0, 0) and went down, down, right right to reach (2, 2).
4 cherries were picked up during this single trip, and the matrix becomes [[0,1,-1],[0,0,-1],[0,0,0]].
Then, the player went left, up, up, left to return home, picking up one more cherry.
The total number of cherries picked up is 5, and this is the maximum possible.
```

Example 2:

```
Input: grid = [[1,1,-1],[1,-1,1],[-1,1,1]]
Output: 0
```

Constraints:

- n == grid.length
- n == grid[i].length
- 1 <= n <= 50
- grid[i][j] is -1, 0, or 1.
- grid[0][0] != -1
- grid[n - 1][n - 1] != -1

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Grid with cherries. Start at (0,0), reach (n-1,n-1), return to (0,0) via same path. Maximize cherries.
# -1=thorn. Right?"
# "grid=[[0,1,-1],[1,0,-1],[1,1,1]]→5 ✓"
# Two simultaneous paths on same step t=r1+c1=r2+c2. O(n³). Space O(n³).
# Follow-up: Cherry Pickup II (LC 1463) — two robots starting from top row going down.
# Follow-up: if multiple paths allowed — K robots version using DP with k×n² states.
```

2D Grid DP

#1335 Minimum Difficulty of a Job Schedule

HAR

Open on LeetCode

Array · Dynamic Programming

Lists: AlgoMaster 600

Problem

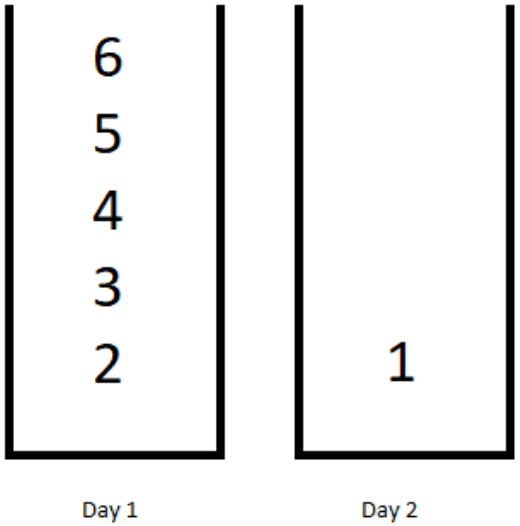
You want to schedule a list of jobs in d days. Jobs are dependent (i.e To work on the ith job, you have to finish all the jobs j where 0 <= j < i).

You have to finish at least one task every day. The difficulty of a job schedule is the sum of difficulties of each day of the d days. The difficulty of a day is the maximum difficulty of a job done on that day.

You are given an integer array jobDifficulty and an integer d. The difficulty of the ith job is jobDifficulty[i].

Return the minimum difficulty of a job schedule. If you cannot find a schedule for the jobs return -1.

Example 1:




```
Input: jobDifficulty = [6,5,4,3,2,1], d = 2
Output: 7
Explanation: First day you can finish the first 5 jobs, total difficulty = 6.
Second day you can finish the last job, total difficulty = 1.
The difficulty of the schedule = 6 + 1 = 7
```

Example 2:

```
Input: jobDifficulty = [9,9,9], d = 4
Output: -1
Explanation: If you finish a job per day you will still have a free day. you cannot find a schedule for the given jobs.
```

Example 3:

```
Input: jobDifficulty = [1,1,1], d = 3
Output: 3
Explanation: The schedule is one job per day. total difficulty will be 3.
```

Constraints:

- 1 <= jobDifficulty.length <= 300
- 0 <= jobDifficulty[i] <= 1000
- 1 <= d <= 10

◆ Interview Approach · STAR-LC


```
# PHASE 1 — CLARIFY
# "Schedule n jobs over d days (at least 1 job/day, in order). Day difficulty = max job in that day.
# Minimize total difficulty. Right?"
# "jobDifficulty=[6,5,4,3,2,1],d=2→7 ✓"
# DP with monotonic stack optimization. O(nd) with stack. Space O(n).
# Follow-up: if days can overlap — different constraint, simpler greedy.
# Follow-up: Minimum Cost to Cut a Stick (LC 1547) — similar interval DP structure.
```

2D Grid DP

#1547 Minimum Cost to Cut a Stick

HAR

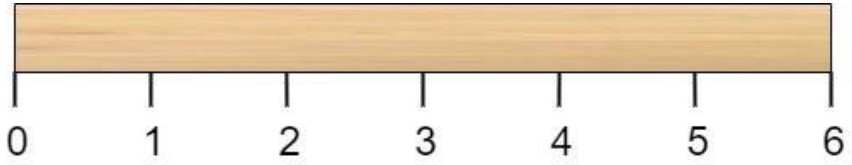
Open on LeetCode

Array · Dynamic Programming · Sorting

Lists: AlgoMaster 600

Problem

Given a wooden stick of length n units. The stick is labelled from 0 to n. For example, a stick of length 6 is labelled as follows:



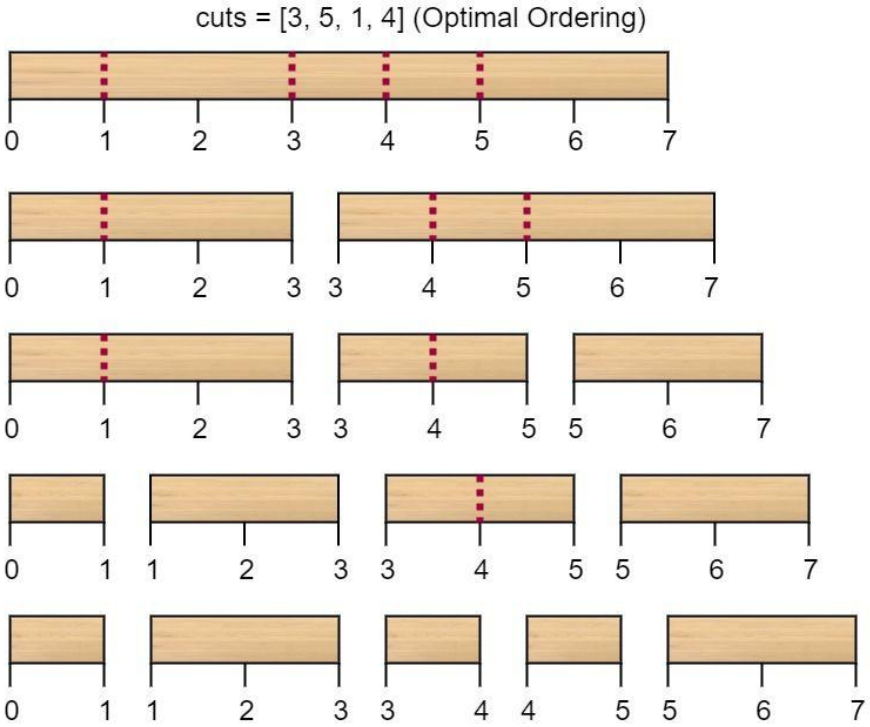
Given an integer array cuts where cuts[i] denotes a position you should perform a cut at.

You should perform the cuts in order, you can change the order of the cuts as you wish.

The cost of one cut is the length of the stick to be cut, the total cost is the sum of costs of all cuts. When you cut a stick, it will be split into two smaller sticks (i.e. the sum of their lengths is the length of the stick before the cut). Please refer to the first example for a better explanation.

Return the minimum total cost of the cuts.

Example 1:



· #1335 | #1931 · | · Contents

```
Input: n = 7, cuts = [1,3,4,5]
Output: 16
Explanation: Using cuts order = [1, 3, 4, 5] as in the input leads to the following scenario:

The first cut is done to a rod of length 7 so the cost is 7. The second cut is done to a rod of length 6 (i.e. the second part of the first cut), the third is done to a rod of length 4 and the last cut is to a rod of length 3. The total cost is 7 + 6 + 4 + 3 = 20.
Rearranging the cuts to be [3, 5, 1, 4] for example will lead to a scenario with total cost = 16 (as shown in the example photo 7 + 4 + 3 + 2 = 16).
```

Example 2:

```
Input: n = 9, cuts = [5,6,1,4,2]
Output: 22
Explanation: If you try the given cuts ordering the cost will be 25.
There are much ordering with total cost <= 25, for example, the order [4, 6, 5, 2, 1] has total cost = 22 which is the minimum possible.
```

Constraints:

- 2 <= n <= 106
- 1 <= cuts.length <= min(n - 1, 100)
- 1 <= cuts[i] <= n - 1
- All the integers in cuts array are distinct.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Stick of length n. cuts[i]=positions to cut. Cost=length of stick being cut. Minimize total cost. Right?"
# "n=7,cuts=[1,3,4,5]→16 ✓"
# Interval DP on cut positions. O(m³) where m=cuts+2. Space O(m²).
# Follow-up: Burst Balloons (LC 312) — same interval DP structure.
# Follow-up: if cuts have costs — replace cuts[r]-cuts[l] with cost[mid].
```

2D Grid DP

#1931 Painting a Grid With Three Different Colors

HAR

Open on LeetCode

Dynamic Programming

Lists: AlgoMaster 600

Problem

You are given two integers m and n. Consider an m x n grid where each cell is initially white. You can paint each cell red, green, or blue. All cells must be painted.

Return the number of ways to color the grid with no two adjacent cells having the same color. Since the answer can be very large, return it modulo 109 + 7.

Example 1:

```
Input: m = 1, n = 1
Output: 3
Explanation: The three possible colorings are shown in the image above.
```

Example 2:

```
Input: m = 1, n = 2
Output: 6
Explanation: The six possible colorings are shown in the image above.
```

Example 3:

```
Input: m = 5, n = 5
Output: 580986
```

Constraints:

- 1 <= m <= 5
- 1 <= n <= 1000

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "m×n grid, 3 colors, adjacent cells different. Count ways mod 10^9+7. Right?"
# "m=1,n=1→3 ✓ m=1,n=2→6 ✓ m=5,n=5→580986"
        # Generate valid column patterns
# Column-by-column DP with valid column patterns. O(n * 3^m * 3^m) = O(n * 9^m). Space O(3^m).
# Follow-up: if 4 colors allowed — same approach, patterns grow to 4^m.
# Follow-up: Number of Ways to Paint N×3 Grid (LC 1411) — closed-form formula for m=3.
```


2D Grid DP

#3651 Minimum Cost Path with Teleportations

HAR

Open on LeetCode

Array · Dynamic Programming · Matrix

Lists: AlgoMaster 600

Problem

You are given a m x n 2D integer array grid and an integer k. You start at the top-left cell (0, 0) and your goal is to reach the bottom-right cell (m - 1, n - 1).

There are two types of moves available:

- Normal move: You can move right or down from your current cell (i, j), i.e. you can move to (i, j + 1) (right) or (i + 1, j) (down). The cost is the value of the destination cell.
- Teleportation: You can teleport from any cell (i, j), to any cell (x, y) such that grid[x][y] <= grid[i][j]; the cost of this move is 0. You may teleport at most k times.

Return the minimum total cost to reach cell (m - 1, n - 1) from (0, 0).

Example 1:

Input: grid = [[1,3,3],[2,5,4],[4,3,5]], k = 2

Output: 7

Explanation:

Initially we are at (0, 0) and cost is 0.

The minimum cost to reach bottom-right cell is 7.

Example 2:

Input: grid = [[1,2],[2,3],[3,4]], k = 1

Output: 9

Explanation:

Initially we are at (0, 0) and cost is 0.

The minimum cost to reach bottom-right cell is 9.

Constraints:

- 2 <= m, n <= 80
- m == grid.length
- n == grid[i].length
- 0 <= grid[i][j] <= 104
- 0 <= k <= 10

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Grid with teleporters: from some cells you can teleport to another for a cost.
# Find minimum cost path from (0,0) to (m-1,n-1). Right?"
# Dijkstra with teleporter edges. O((mn+T) log(mn)). Space O(mn+T).
# Follow-up: if teleporters are bidirectional — add edges in both directions.
# Follow-up: Minimum Cost to Make at Least One Valid Path (LC 1368) — 0-1 BFS variant.
```

String DP

Round 9 · Low · Hard · 4 questions

String DP

#44 Wildcard Matching

HAR

Open on LeetCode

String · Dynamic Programming · Greedy · Recursion

Lists: AlgoMaster 600

Problem

Given an input string (s) and a pattern (p), implement wildcard pattern matching with support for '?' and '*' where:

- '?' Matches any single character.
- '*' Matches any sequence of characters (including the empty sequence).

The matching should cover the entire input string (not partial).

Example 1:

```
Input: s = "aa", p = "a"
Output: false
Explanation: "a" does not match the entire string "aa".
```

Example 2:

```
Input: s = "aa", p = "*"
Output: true
Explanation: '*' matches any sequence.
```

Example 3:

```
Input: s = "cb", p = "?a"
Output: false
Explanation: '?' matches 'c', but the second letter is 'a', which does not match 'b'.
```

Constraints:

- 0 <= s.length, p.length <= 2000
- s contains only lowercase English letters.
- p contains only lowercase English letters, '?' or '*'.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "'?' matches any single char. '*' matches any sequence (including empty). Full match. Right?"
# "s='aa',p='a'→false ✓ s='aa',p='*'→true ✓ s='adceb',p='*a*b'→true ✓"
# Bottom-up DP. '*' matches empty (dp[i][j-1]) or one more char (dp[i-1][j]). O(mn). Space O(mn).
# Follow-up: Regular Expression Matching (LC 10) — '*' means zero-or-more of preceding char; different.
# Follow-up: O(m+n) space with rolling array — keep only current and previous row.
```

String DP

#140 Word Break II

HAR

Open on LeetCode

Array · Hash Table · String · Dynamic Programming · Backtracking · Trie · Memoization

Lists: AlgoMaster 600

Problem

Given a string s and a dictionary of strings wordDict, add spaces in s to construct a sentence where each word is a valid dictionary word. Return all such possible sentences in any order.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1:

```
Input: s = "catsanddog", wordDict = ["cat","cats","and","sand","dog"]
Output: ["cats and dog","cat sand dog"]
```

Example 2:

```
Input: s = "pineapplepenapple", wordDict = ["apple","pen","applepen","pine","pineapple"]
Output: ["pine apple pen apple","pineapple pen apple","pine applepen apple"]
Explanation: Note that you are allowed to reuse a dictionary word.
```

Example 3:

```
Input: s = "catsandog", wordDict = ["cats","dog","sand","and","cat"]
Output: []
```

Constraints:

- 1 <= s.length <= 20
- 1 <= wordDict.length <= 1000
- 1 <= wordDict[i].length <= 10
- s and wordDict[i] consist of only lowercase English letters.
- All the strings of wordDict are unique.
- Input is generated in a way that the length of the answer doesn't exceed 105.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return all ways to segment s into dictionary words. Right?"
# "s='catsanddog',wordDict=['cat','cats','and','sand','dog']→['cats and dog','cat sand dog'] ✓"
# Memoized backtracking: dp(start) = all sentences from s[start:]. O(n²*2^n) worst case. Space O(n*2^n).
# Follow-up: Word Break I (LC 139) — just check existence, DP O(n²).
# Follow-up: if wordDict is a Trie — optimize inner loop to O(max_len) per position.
```


String DP

#664 Strange Printer

HAR

Open on LeetCode

String · Dynamic Programming

Lists: AlgoMaster 600

Problem

There is a strange printer with the following two special properties:

- The printer can only print a sequence of the same character each time.
- At each turn, the printer can print new characters starting from and ending at any place and will cover the original existing characters.

Given a string s, return the minimum number of turns the printer needed to print it.

Example 1:

```
Input: s = "aaabbb"
Output: 2
Explanation: Print "aaa" first and then print "bbb".
```

Example 2:

```
Input: s = "aba"
Output: 2
Explanation: Print "aaa" first and then print "b" from the second place of the string, which will cover the existing character 'a'.
```

Constraints:

- 1 <= s.length <= 100
- s consists of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Printer prints consecutive same chars. Find min turns to print string s. Right?"
# "s='aaabbb'→2 ✓ s='aba'→2 ✓"
        # Remove consecutive duplicates
# Interval DP: if s[l]==s[m], printing l covers m's character. O(n³). Space O(n²).
# Follow-up: Zuma Game (LC 488) — similar interval DP with chain reactions.
# Follow-up: if printer can print any run (not just from current position) — different formulation.
```

String DP

#1092 Shortest Common Supersequence HAR

Open on LeetCode

String · Dynamic Programming

Lists: AlgoMaster 600

Problem

Given two strings str1 and str2, return the shortest string that has both str1 and str2 as subsequences. If there are multiple valid strings, return any of them.

A string s is a subsequence of string t if deleting some number of characters from t (possibly 0) results in the string s.

Example 1:

```
Input: str1 = "abac", str2 = "cab"
Output: "cabac"
Explanation:
str1 = "abac" is a subsequence of "cabac" because we can delete the first "c".
str2 = "cab" is a subsequence of "cabac" because we can delete the last "ac".
The answer provided is the shortest such string that satisfies these properties.
```

Example 2:

```
Input: str1 = "aaaaaaaa", str2 = "aaaaaaaa"
Output: "aaaaaaaa"
```

Constraints:

- 1 <= str1.length, str2.length <= 1000
- str1 and str2 consist of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find shortest string containing both str1 and str2 as subsequences. Right?"
# "str1='abac',str2='cab'→'cabac' ✓"
        # LCS DP
        # Reconstruct SCS from LCS
# LCS → SCS: non-LCS chars from both + LCS chars. O(mn). Space O(mn).
# Follow-up: length only — len(str1)+len(str2)-len(LCS).
# Follow-up: SCS of k strings — NP-hard in general; DP for 2 strings only.
```


Tree / Graph DP

Round 9 · Low · Hard · 2 questions

Tree / Graph DP

#834 Sum of Distances in Tree

HAR

Open on LeetCode

Dynamic Programming · Tree · Depth-First Search · Graph Theory

Lists: AlgoMaster 600

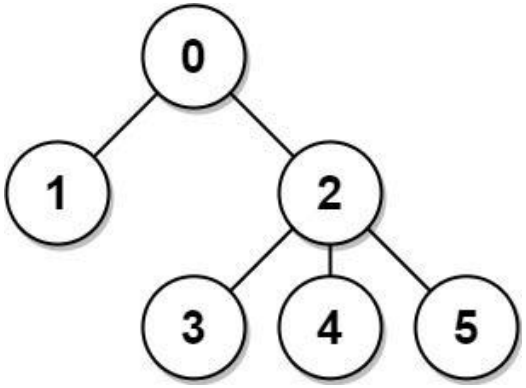
Problem

There is an undirected connected tree with n nodes labeled from 0 to n - 1 and n - 1 edges.

You are given the integer n and the array edges where edges[i] = [ai, bi] indicates that there is an edge between nodes ai and bi in the tree.

Return an array answer of length n where answer[i] is the sum of the distances between the ith node in the tree and all other nodes.

Example 1:

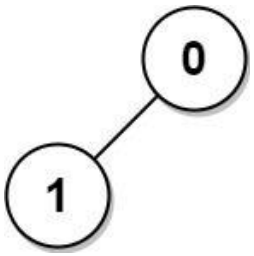



```
Input: n = 6, edges = [[0,1],[0,2],[2,3],[2,4],[2,5]]
Output: [8,12,6,10,10,10]
Explanation: The tree is shown above.
We can see that dist(0,1) + dist(0,2) + dist(0,3) + dist(0,4) + dist(0,5)
equals 1 + 1 + 2 + 2 + 2 = 8.
Hence, answer[0] = 8, and so on.
```

Example 2:


```
Input: n = 1, edges = []
Output: [0]
```

Example 3:




```
Input: n = 2, edges = [[1,0]]
Output: [1,1]
```

Constraints:

· #1092 | #968 · | · Contents

- 1 <= n <= 3 * 104
- edges.length == n - 1
- edges[i].length == 2
- 0 <= ai, bi < n
- ai != bi
- The given input represents a valid tree.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Undirected tree with n nodes. Return sum of distances from each node to all others. Right?"
# "n=6,edges=[[0,1],[0,2],[2,3],[2,4],[2,5]]→[8,12,6,10,10,10] ✓"
# Two DFS: root sum + rerooting. O(n). Space O(n).
# Follow-up: if tree is weighted — include edge weights in distance calculations.
# Follow-up: find the centroid — node minimizing sum of distances.
```

Tree / Graph DP

#968 Binary Tree Cameras

HAR

Open on LeetCode

Dynamic Programming · Tree · Depth-First Search · Binary Tree

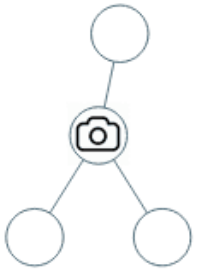
Lists: AlgoMaster 600

Problem

You are given the root of a binary tree. We install cameras on the tree nodes where each camera at a node can monitor its parent, itself, and its immediate children.

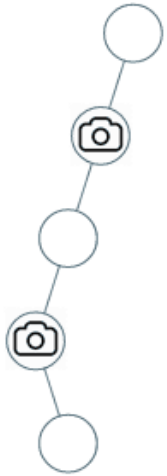
Return the minimum number of cameras needed to monitor all nodes of the tree.

Example 1:



```
Input: root = [0,0,null,0,0]
Output: 1
Explanation: One camera is enough to monitor all nodes if placed as shown.
```

Example 2:



```
Input: root = [0,0,null,0,null,0,null,null,0]
Output: 2
Explanation: At least two cameras are needed to monitor all nodes of the tree. The above image shows one of the valid
configurations of camera placement.
```

Constraints:

- **The number of nodes in the tree is in the range [1, 1000].**
- **Node.val == 0**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Place minimum cameras so every node is monitored. Camera monitors parent, itself, children. Right?"
# "root=[0,0,null,0,0]→1 ✓ root=[0,0,null,0,null,0,null,null,0]→2 ✓"
# Greedy post-order DFS: place camera if child needs coverage. O(n). Space O(h).
# Follow-up: if cameras have limited range (not just adjacent) — harder optimization problem.
# Follow-up: if nodes have weights — minimize weighted camera placement.
```

Bitmask DP

Round 9 · Low · Hard · 1 question

Bitmask DP

#847 Shortest Path Visiting All Nodes HAR

Open on LeetCode

Dynamic Programming · Bit Manipulation · Breadth-First Search · Graph Theory · Bitmask

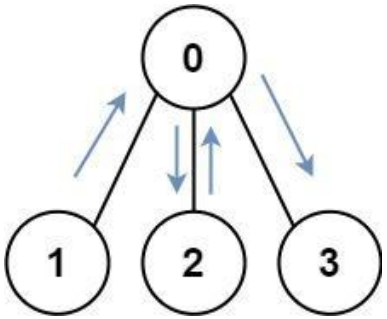
Lists: AlgoMaster 600

Problem

You have an undirected, connected graph of n nodes labeled from 0 to n - 1. You are given an array graph where graph[i] is a list of all the nodes connected with node i by an edge.

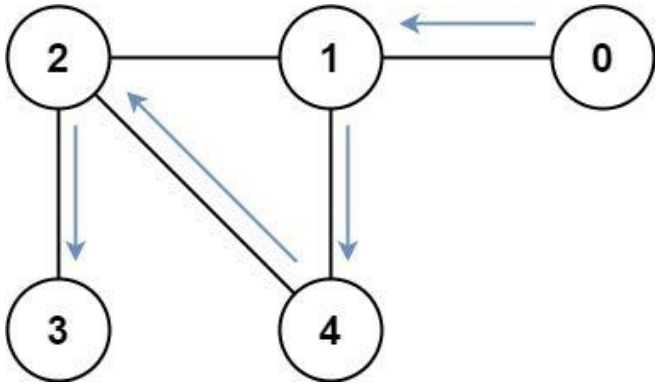
Return the length of the shortest path that visits every node. You may start and stop at any node, you may revisit nodes multiple times, and you may reuse edges.

Example 1:




```
Input: graph = [[1,2,3],[0],[0],[0]]
Output: 4
Explanation: One possible path is [1,0,2,0,3]
```

Example 2:




```
Input: graph = [[1],[0,2,4],[1,3,4],[2],[1,2]]
Output: 4
Explanation: One possible path is [0,1,4,2,3]
```

Constraints:

- **n == graph.length**
- **1 <= n <= 12**
- **0 <= graph[i].length < n**
- **graph[i] does not contain i.**
- **If graph[a] contains b, then graph[b] contains a.**
- **The input graph is always connected.**

◆ **Interview Approach · STAR-LC**

PHASE 1 — CLARIFY

```
# "Undirected graph. Find shortest path visiting all nodes. Can revisit nodes. Right?"
# "graph=[[1,2,3],[0],[0],[0]]→4 ✓"
# BFS on (visited_mask, current_node) states. O(2^n * n). Space O(2^n * n).
# Follow-up: if graph is directed — same approach, just directed edges.
# Follow-up: if we need the actual path — track parent state in BFS.
```


· #968 #233 · · Contents

Digit DP

Round 9 · Low · Hard · 2 questions

Digit DP

#233 Number of Digit One

HAR

Open on LeetCode

Math · Dynamic Programming · Recursion

Lists: AlgoMaster 600

Problem

Given an integer n, count the total number of digit 1 appearing in all non-negative integers less than or equal to n.

Example 1:

```
Input: n = 13
Output: 6
```

Example 2:

```
Input: n = 0
Output: 0
```

Constraints:

- 0 <= n <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count total number of 1s in all numbers from 1 to n. Right?"
# "n=13→6 (1,10,11,12,13→ six 1-digits) ✓"
# For each digit position, count contribution of digit 1. O(log n). Space O(1).
# Follow-up: Count Numbers with Unique Digits (LC 357) — different constraint per position.
# Follow-up: count k's instead of 1s — same formula with k (handle k=0 separately).
```


Digit DP

#902 Numbers At Most N Given Digit Set

HAR

Open on LeetCode

Array · Math · String · Binary Search · Dynamic Programming

Lists: AlgoMaster 600

Problem

Given an array of digits which is sorted in non-decreasing order. You can write numbers using each digits[i] as many times as we want. For example, if digits = ['1','3','5'], we may write numbers such as '13', '551', and '1351315'.

Return the number of positive integers that can be generated that are less than or equal to a given integer n.

Example 1:

```
Input: digits = ["1","3","5","7"], n = 100
Output: 20
Explanation:
The 20 numbers that can be written are:
1, 3, 5, 7, 11, 13, 15, 17, 31, 33, 35, 37, 51, 53, 55, 57, 71, 73, 75, 77.
```

Example 2:

```
Input: digits = ["1","4","9"], n = 1000000000
Output: 29523
Explanation:
We can write 3 one digit numbers, 9 two digit numbers, 27 three digit numbers,
81 four digit numbers, 243 five digit numbers, 729 six digit numbers,
2187 seven digit numbers, 6561 eight digit numbers, and 19683 nine digit numbers.
In total, this is 29523 integers that can be written using the digits array.
```

Example 3:

```
Input: digits = ["7"], n = 8
Output: 1
```

Constraints:

- 1 <= digits.length <= 9
- digits[i].length == 1
- digits[i] is a digit from '1' to '9'.
- All the values in digits are unique.
- digits is sorted in non-decreasing order.
- 1 <= n <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given sorted digit set and n, count positive integers <= n that only use digits in the set. Right?"
# "digits=['1','3','5','7'],n=100→20 ✓"
        # Count numbers with fewer digits than s
        # Count numbers with exactly m digits <= s
                # Digits smaller than ch at position i
# Digit DP: count shorter numbers + numbers of same length but smaller. O(m * k). Space O(1).
# Follow-up: Numbers At Least N — total with those digits minus at most N-1.
# Follow-up: Numbers In Range [lo,hi] — atMostN(hi) - atMostN(lo-1).
```

State Machine DP

Round 9 · Low · Hard · 1 question

State Machine DP

#188 Best Time to Buy and Sell Stock IV HAR

Open on LeetCode

Array · Dynamic Programming

Lists: AlgoMaster 600

Problem

You are given an integer array prices where prices[i] is the price of a given stock on the ith day, and an integer k.
Find the maximum profit you can achieve. You may complete at most k transactions: i.e. you may buy at most k times and sell at most k times.
Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Example 1:

```
Input: k = 2, prices = [2,4,1]
Output: 2
Explanation: Buy on day 1 (price = 2) and sell on day 2 (price = 4), profit = 4-2 = 2.
```

Example 2:

```
Input: k = 2, prices = [3,2,6,5,0,3]
Output: 7
Explanation: Buy on day 2 (price = 2) and sell on day 3 (price = 6), profit = 6-2 = 4. Then buy on day 5 (price = 0) and sell on day 6 (price = 3), profit = 3-0 = 3.
```

Constraints:

- 1 <= k <= 100
- 1 <= prices.length <= 1000
- 0 <= prices[i] <= 1000

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "At most k transactions. Maximize profit. Right?"
# "k=2,prices=[3,2,6,5,0,3]→7 ✓"
# k states for buy/sell. O(nk). Space O(k).
# Follow-up: Best Time III (LC 123) — k=2 special case.
# Follow-up: unlimited transactions (LC 122) — sum all positive price differences.
```

Maths / Geometry

Round 9 · Low · Hard · 1 question

Maths / Geometry

#149 Max Points on a Line HAR

Open on LeetCode

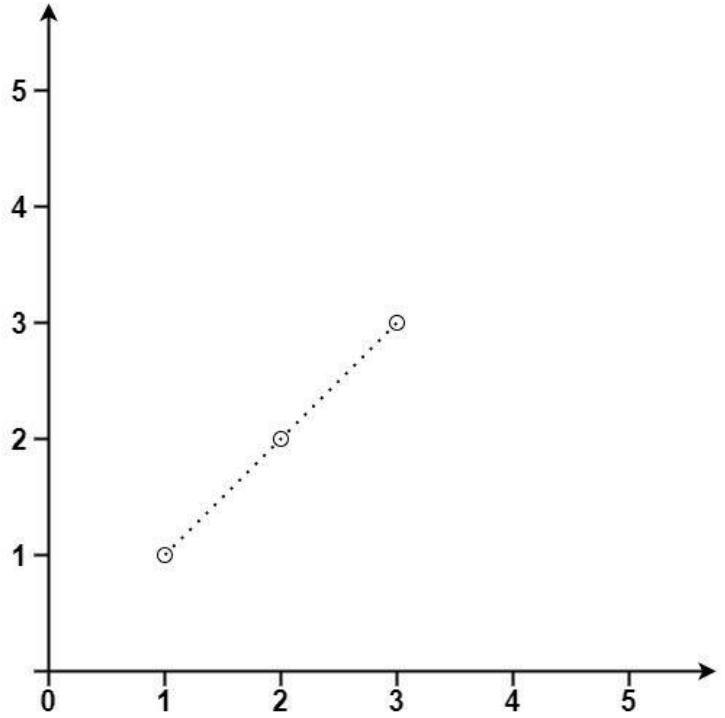
Array · Hash Table · Math · Geometry

Lists: AlgoMaster 600

Problem

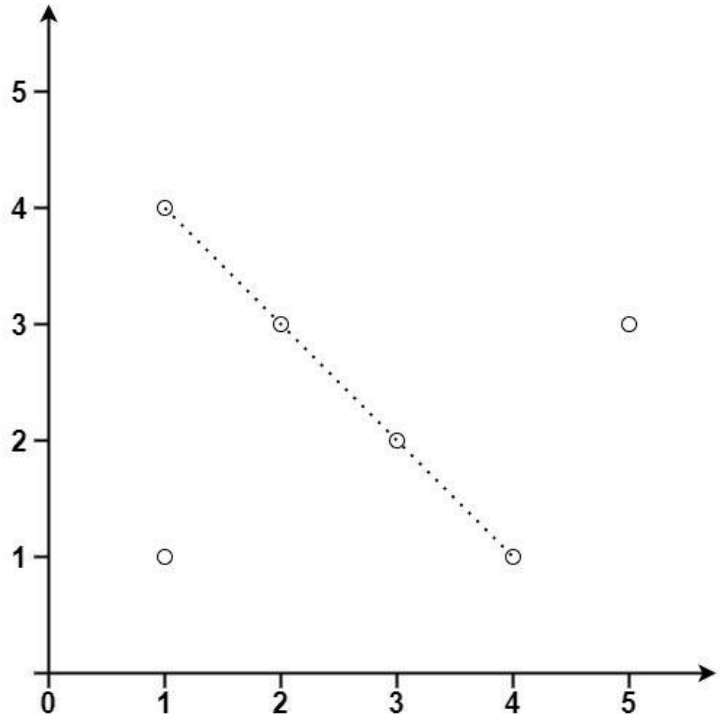
Given an array of points where points[i] = [xi, yi] represents a point on the X-Y plane, return the maximum number of points that lie on the same straight line.

Example 1:




```
Input: points = [[1,1],[2,2],[3,3]]
Output: 3
```

Example 2:




```
Input: points = [[1,1],[3,2],[5,3],[4,1],[2,3],[1,4]]
Output: 4
```

Constraints:

- 1 <= points.length <= 300
- points[i].length == 2
- -104 <= xi, yi <= 104
- All the points are unique.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Find maximum number of points that lie on the same straight line. Right?"
# "points=[[1,1],[2,2],[3,3]]→3 ✓ points=[[1,1],[3,2],[5,3],[4,1],[2,3],[1,4]]→4 ✓"
# For each pair of points, compute normalized slope. Max frequency = max collinear. O(n²). Space O(n).
# Follow-up: if points are 3D — use cross product to check collinearity.
# Follow-up: approximate collinearity (floating point) — group by slope with epsilon tolerance.
```


String Matching

Round 9 · Low · Hard · 3 questions

String Matching

#214 Shortest Palindrome

HAR

Open on LeetCode

String · Rolling Hash · String Matching · Hash Function

Lists: AlgoMaster 600

Problem

You are given a string s. You can convert s to a palindrome by adding characters in front of it.
Return the shortest palindrome you can find by performing this transformation.

Example 1:

```
Input: s = "aacecaaa"
Output: "aaacecaaa"
```

Example 2:

```
Input: s = "abcd"
Output: "dcbabcd"
```

Constraints:

- 0 <= s.length <= 5 * 104
- s consists of lowercase English letters only.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Add minimum characters to the front of s to make it a palindrome. Right?"
# "s='aacecaaa'→'aaacecaaa' ✓ s='abcd'→'dcbabcd' ✓"
        # KMP on s+'#'+reverse(s) to find longest palindrome prefix
# KMP failure function on s+'#'+rev(s). Longest prefix of s that's also a palindrome. O(n). Space O(n).
# Follow-up: Longest Palindromic Prefix — return the palindrome itself.
# Follow-up: Shortest Palindrome by Adding to End — reverse the problem.
```


String Matching

#1044 Longest Duplicate Substring

HAR

Open on LeetCode

String · Binary Search · Sliding Window · Rolling Hash · Suffix Array · Hash Function

Lists: AlgoMaster 600

Problem

Given a string s, consider all duplicated substrings: (contiguous) substrings of s that occur 2 or more times. The occurrences may overlap.
Return any duplicated substring that has the longest possible length. If s does not have a duplicated substring, the answer is "".

Example 1:

```
Input: s = "banana"
Output: "ana"
```

Example 2:

```
Input: s = "abcd"
Output: ""
```

Constraints:

- 2 <= s.length <= 3 * 104
- s consists of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the longest duplicate substring (appearing at least twice). Return '' if none. Right?"
# "s='banana'→'ana' ✓ s='abcd'→'' ✓"
        # Binary search + rolling hash
# Binary search on length + rolling hash. O(n log n). Space O(n).
# Suffix array approach: O(n log²n) build, O(n) to find answer.
# Follow-up: number of distinct substrings — n*(n+1)/2 − sum(LCP array).
```

String Matching

#1392 Longest Happy Prefix HAR

Open on LeetCode

String · Rolling Hash · String Matching · Hash Function

Lists: AlgoMaster 600

Problem

A string is called a happy prefix if is a non-empty prefix which is also a suffix (excluding itself).

Given a string s, return the longest happy prefix of s. Return an empty string "" if no such prefix exists.

Example 1:

```
Input: s = "level"
Output: "l"
Explanation: s contains 4 prefix excluding itself ("l", "le", "lev", "leve"), and suffix ("l", "el", "vel", "evel"). The largest prefix which is also suffix is given by "l".
```

Example 2:

```
Input: s = "ababab"
Output: "abab"
Explanation: "abab" is the largest prefix which is also suffix. They can overlap in the original string.
```

Constraints:

- 1 <= s.length <= 105
- s contains only lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "A 'happy prefix' is a non-empty prefix of s that is also a suffix (but not the whole string).
# Return the longest one or '' if none. Right?"
# "s='level'→'l' ✓ s='ababab'→'abab' ✓"
        # KMP failure function: fail[i] = length of longest proper prefix=suffix of s[:i+1]
# KMP failure function: last value = length of longest proper prefix that is also suffix. O(n).
# Follow-up: Shortest Palindrome (LC 214) — uses same KMP trick.
# Follow-up: count all happy prefixes — positions in fail array where fail[i]>0.
```


Binary Indexed Tree / Segment Tree

Round 9 · Low · Hard · 4 questions

Binary Indexed Tree / Segment Tree

#699 Falling Squares

HAR

Open on LeetCode

Array · Segment Tree · Ordered Set

Lists: AlgoMaster 600

Problem

There are several squares being dropped onto the X-axis of a 2D plane.

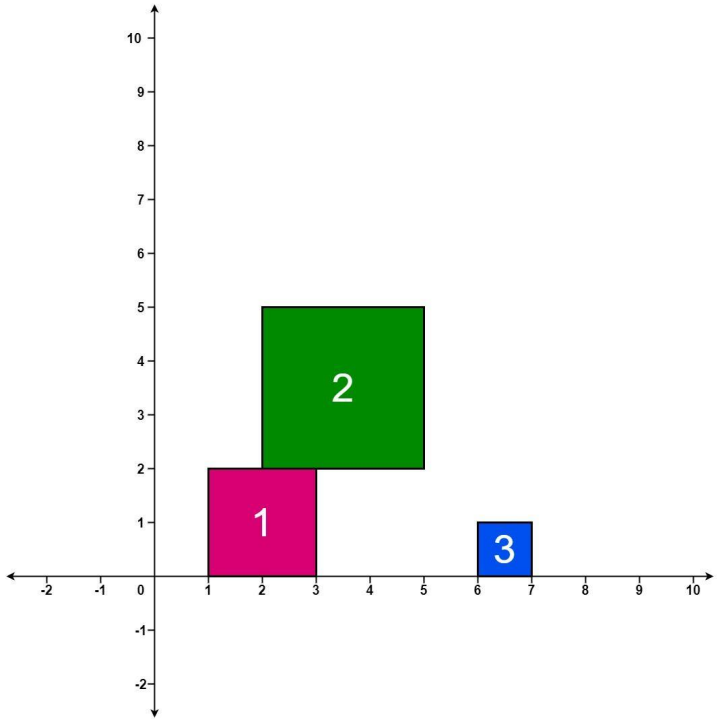
You are given a 2D integer array positions where positions[i] = [lefti, sideLengthi] represents the ith square with a side length of sideLengthi that is dropped with its left edge aligned with X-coordinate lefti.

Each square is dropped one at a time from a height above any landed squares. It then falls downward (negative Y direction) until it either lands on the top side of another square or on the X-axis. A square brushing the left/right side of another square does not count as landing on it. Once it lands, it freezes in place and cannot be moved.

After each square is dropped, you must record the height of the current tallest stack of squares.

Return an integer array ans where ans[i] represents the height described above after dropping the ith square.

Example 1:




```
Input: positions = [[1,2],[2,3],[6,1]]
Output: [2,5,5]
Explanation:
After the first drop, the tallest stack is square 1 with a height of 2.
After the second drop, the tallest stack is squares 1 and 2 with a height of 5.
After the third drop, the tallest stack is still squares 1 and 2 with a height of 5.
Thus, we return an answer of [2, 5, 5].
```

Example 2:


```
Input: positions = [[100,100],[200,100]]
Output: [100,100]
Explanation:
After the first drop, the tallest stack is square 1 with a height of 100.
After the second drop, the tallest stack is either square 1 or square 2, both with heights of 100.
Thus, we return an answer of [100, 100].
Note that square 2 only brushes the right side of square 1, which does not count as landing on it.
```

Constraints:

- 1 <= positions.length <= 1000
- 1 <= lefti <= 108
- 1 <= sideLengthi <= 106

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Squares fall onto x-axis. Each square settles on the highest stack. Return max height after each drop. Right?"
# "positions=[[1,2],[2,3],[6,1]]→[2,5,5] ✓"
        # Coordinate compress + segment tree for range max query
# Segment tree with lazy max. O(n log n). Space O(n).
# Follow-up: if squares can be removed — harder, need actual height tracking per position.
# Follow-up: Range Module (LC 715) — track covered intervals instead of heights.
```

Binary Indexed Tree / Segment Tree

#2426 Number of Pairs Satisfying Inequality

HAR

Open on LeetCode

Array · Binary Search · Divide and Conquer · Binary Indexed Tree · Segment Tree · Merge Sort · Ordered Set

Lists: AlgoMaster 600

Problem

You are given two 0-indexed integer arrays nums1 and nums2, each of size n, and an integer diff. Find the number of pairs (i, j) such that:

- 0 <= i < j <= n - 1 and
- nums1[i] - nums1[j] <= nums2[i] - nums2[j] + diff.

Return the number of pairs that satisfy the conditions.

Example 1:

```
Input: nums1 = [3,2,5], nums2 = [2,2,1], diff = 1
Output: 3
Explanation:
There are 3 pairs that satisfy the conditions:
1. i = 0, j = 1: 3 - 2 <= 2 - 2 + 1. Since i < j and 1 <= 1, this pair satisfies the conditions.
2. i = 0, j = 2: 3 - 5 <= 2 - 1 + 1. Since i < j and -2 <= 2, this pair satisfies the conditions.
3. i = 1, j = 2: 2 - 5 <= 2 - 1 + 1. Since i < j and -3 <= 2, this pair satisfies the conditions.
Therefore, we return 3.
```

Example 2:

```
Input: nums1 = [3,-1], nums2 = [-2,2], diff = -1
Output: 0
Explanation:
Since there does not exist any pair that satisfies the conditions, we return 0.
```

Constraints:

- n == nums1.length == nums2.length
- 2 <= n <= 105
- -104 <= nums1[i], nums2[i] <= 104
- -104 <= diff <= 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count pairs (i,j) where i<j and nums1[i]-nums1[j]<=nums2[i]-nums2[j]. Right?"
# "nums1=[3,2,5],nums2=[2,2,1],diff=1 → 3 ✓"
# Rearrange: nums1[i]-nums2[i] <= nums1[j]-nums2[j]. Count inversions in diff array. O(n log n).
# Follow-up: Count of Smaller Numbers After Self (LC 315) — count elements smaller to the right.
# Follow-up: if inequality is strict (<) — change bisect_right to bisect_left.
```


Binary Indexed Tree / Segment Tree

#3161 Block Placement Queries

HAR

Open on LeetCode

Array · Binary Search · Binary Indexed Tree · Segment Tree · Ordered Set

Lists: AlgoMaster 600

Problem

There exists an infinite number line, with its origin at 0 and extending towards the positive x-axis.

You are given a 2D array queries, which contains two types of queries:

1. For a query of type 1, queries[i] = [1, x]. Build an obstacle at distance x from the origin. It is guaranteed that there is no obstacle at distance x when the query is asked.
2. For a query of type 2, queries[i] = [2, x, sz]. Check if it is possible to place a block of size sz anywhere in the range [0, x] on the line, such that the block entirely lies in the range [0, x]. A block cannot be placed if it intersects with any obstacle, but it may touch it. Note that you do not actually place the block. Queries are separate.

Return a boolean array results, where results[i] is true if you can place the block specified in the ith query of type 2, and false otherwise.

Example 1:

Input: queries = [[1,2],[2,3,3],[2,3,1],[2,2,2]]

Output: [false,true,true]

Explanation:

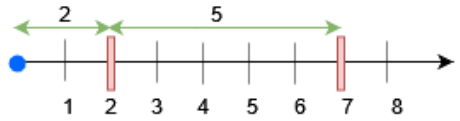
For query 0, place an obstacle at x = 2. A block of size at most 2 can be placed before x = 3.

Example 2:

Input: queries = [[1,7],[2,7,6],[1,2],[2,7,5],[2,7,6]]

Output: [true,true,false]

Explanation:



- Place an obstacle at x = 7 for query 0. A block of size at most 7 can be placed before x = 7.
- Place an obstacle at x = 2 for query 2. Now, a block of size at most 5 can be placed before x = 7, and a block of size at most 2 before x = 2.

Constraints:

- 1 <= queries.length <= 15 * 104
- 2 <= queries[i].length <= 3
- 1 <= queries[i][0] <= 2
- 1 <= x, sz <= min(5 * 104, 3 * queries.length)
- The input is generated such that for queries of type 1, no obstacle exists at distance x when the query is asked.
- The input is generated such that there is at least one query of type 2.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Process queries: place obstacle or find max gap to place a block of given size. Right?"
                # Find max gap to the left of x
                # Simpler: find max gap between consecutive obstacles up to x
```

```
# Segment tree with range max for efficient gap queries. O((n+q) log n).
# Follow-up: if blocks can overlap — change feasibility check.
# Follow-up: if obstacles can be removed — maintain a dynamic set with rebalancing.
```


Binary Indexed Tree / Segment Tree

#3721 Longest Balanced Subarray II HAR

Open on LeetCode

Array · Hash Table · Divide and Conquer · Segment Tree · Prefix Sum

Lists: AlgoMaster 600

Problem

You are given an integer array nums.

A subarray is called balanced if the number of distinct even numbers in the subarray is equal to the number of distinct odd numbers.

Return the length of the longest balanced subarray.

Example 1:

Input: nums = [2,5,4,3]

Output: 4

Explanation:

- **The longest balanced subarray is [2, 5, 4, 3].**
- **It has 2 distinct even numbers [2, 4] and 2 distinct odd numbers [5, 3]. Thus, the answer is 4.**

Example 2:

Input: nums = [3,2,2,5,4]

Output: 5

Explanation:

- **The longest balanced subarray is [3, 2, 2, 5, 4].**
- **It has 2 distinct even numbers [2, 4] and 2 distinct odd numbers [3, 5]. Thus, the answer is 5.**

Example 3:

Input: nums = [1,2,3,2]

Output: 3

Explanation:

- **The longest balanced subarray is [2, 3, 2].**
- **It has 1 distinct even number [2] and 1 distinct odd number [3]. Thus, the answer is 3.**

Constraints:

- **1 <= nums.length <= 105**
- **1 <= nums[i] <= 105**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Find longest subarray where sum of odds = sum of evens. Right?"
        # prefix (odd_sum - even_sum); find longest subarray with diff=0
# Prefix difference + hashmap. O(n). Space O(n).
# Follow-up: if we need the actual subarray — track start index in the map.
# Follow-up: if values can be very large — same approach, no change needed.
```

Line Sweep

Round 9 · Low · Hard · 2 questions

Line Sweep

#391 Perfect Rectangle HAR

Open on LeetCode

Array · Hash Table · Math · Geometry · Sweep Line

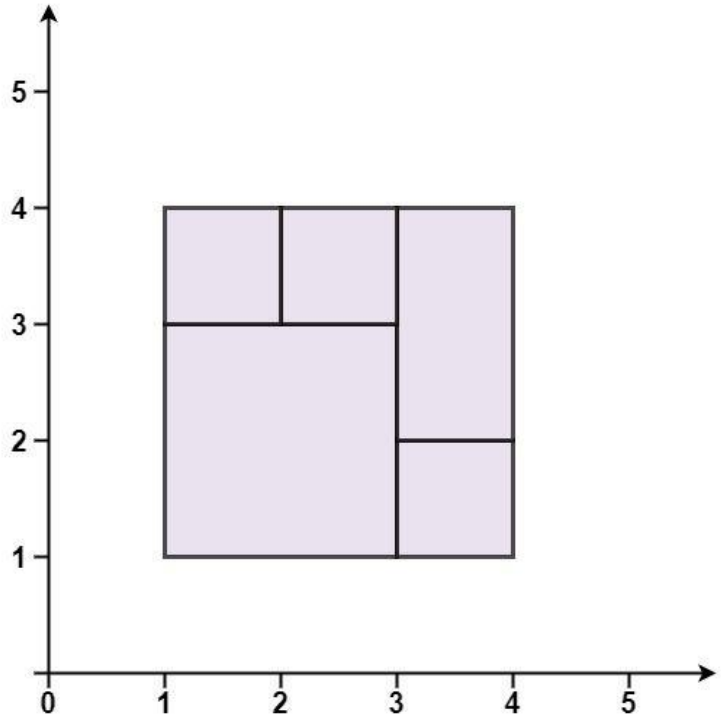
Lists: AlgoMaster 600

Problem

Given an array rectangles where rectangles[i] = [xi, yi, ai, bi] represents an axis-aligned rectangle. The bottom-left point of the rectangle is (xi, yi) and the top-right point of it is (ai, bi).

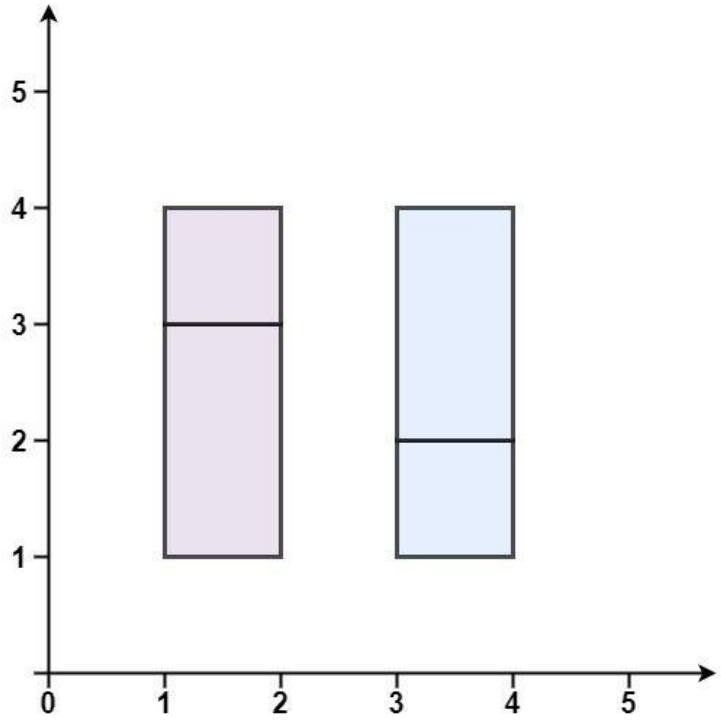
Return true if all the rectangles together form an exact cover of a rectangular region.

Example 1:



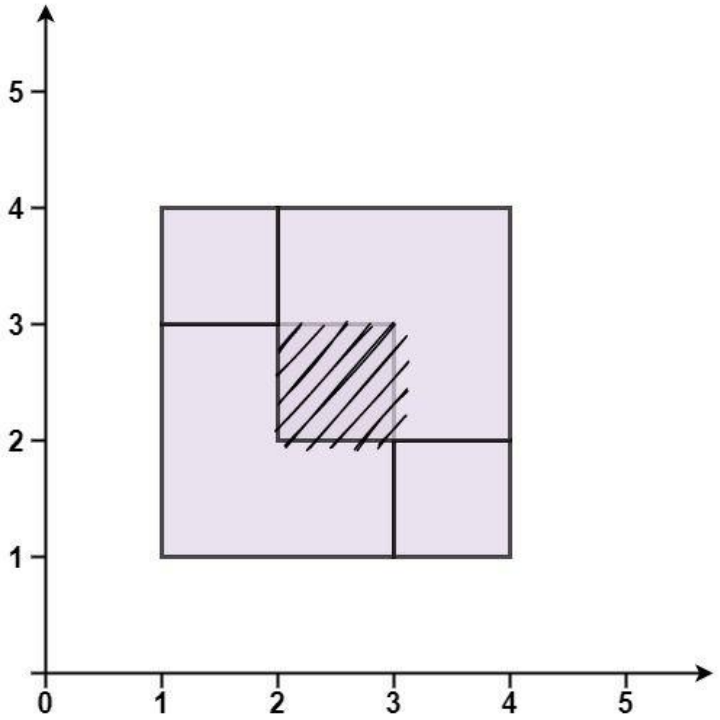

```
Input: rectangles = [[1,1,3,3],[3,1,4,2],[3,2,4,4],[1,3,2,4],[2,3,3,4]]
Output: true
Explanation: All 5 rectangles together form an exact cover of a rectangular region.
```

Example 2:




```
Input: rectangles = [[1,1,2,3],[1,3,2,4],[3,1,4,2],[3,2,4,4]]
Output: false
Explanation: Because there is a gap between the two rectangular regions.
```

Example 3:




```
Input: rectangles = [[1,1,3,3],[3,1,4,2],[1,3,2,4],[2,2,4,4]]
Output: false
Explanation: Because two of the rectangles overlap with each other.
```

Constraints:

- 1 <= rectangles.length <= 2 * 10^4
- rectangles[i].length == 4
- -10^5 <= xi < ai <= 10^5
- -10^5 <= yi < bi <= 10^5

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Check if given rectangles form a perfect covering of a larger rectangle with no overlap/gap. Right?"
# "rectangles=[[1,1,3,3],[3,1,4,2],[3,2,4,4],[1,3,2,4],[2,3,3,4]]→true ✓"
# Area check + corner XOR: each interior point appears 2/4 times (cancels), outer 4 corners once. O(n). Space O(n).
# Follow-up: Rectangle Area (LC 223) — area of union of two rectangles.
# Follow-up: Rectangle Area II (LC 850) — area of union of multiple rectangles (line sweep).
```

Line Sweep

#850 Rectangle Area II

HAR

Open on LeetCode

Array · Segment Tree · Sweep Line · Ordered Set

Lists: AlgoMaster 600

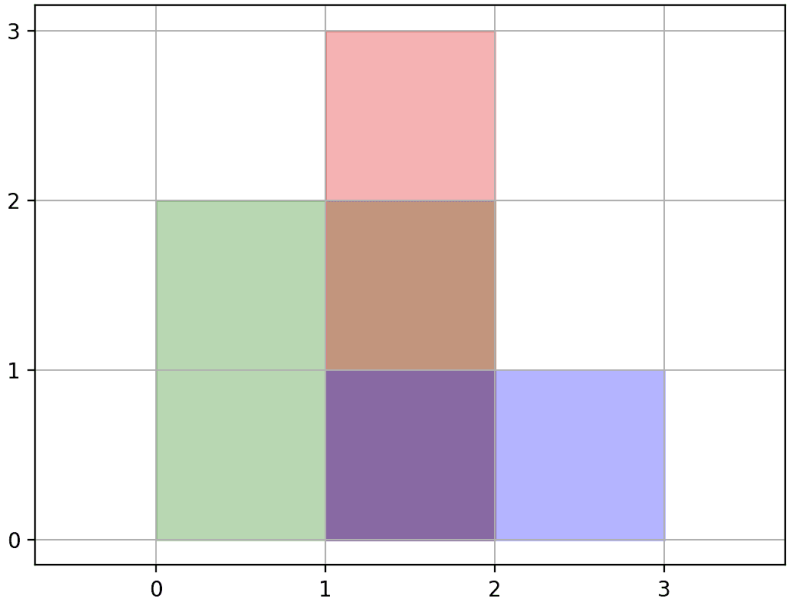
Problem

You are given a 2D array of axis-aligned rectangles. Each rectangle[i] = [xi1, yi1, xi2, yi2] denotes the ith rectangle where (xi1, yi1) are the coordinates of the bottom-left corner, and (xi2, yi2) are the coordinates of the top-right corner.

Calculate the total area covered by all rectangles in the plane. Any area covered by two or more rectangles should only be counted once.

Return the total area. Since the answer may be too large, return it modulo 109 + 7.

Example 1:




```
Input: rectangles = [[0,0,2,2],[1,0,2,3],[1,0,3,1]]
Output: 6
Explanation: A total area of 6 is covered by all three rectangles, as illustrated in the picture.
From (1,1) to (2,2), the green and red rectangles overlap.
From (1,0) to (2,3), all three rectangles overlap.
```

Example 2:

```
Input: rectangles = [[0,0,1000000000,1000000000]]
Output: 49
Explanation: The answer is 1018 modulo (109 + 7), which is 49.
```

Constraints:

- 1 <= rectangles.length <= 200
- rectanges[i].length == 4
- 0 <= xi1, yi1, xi2, yi2 <= 109
- xi1 <= xi2
- yi1 <= yi2

- All rectangles have non zero area.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find total area covered by union of rectangles, mod 10^9+7. Right?"
# "rectangles=[[0,0,2,2],[1,0,2,3],[1,0,3,1]]→6 ✓"
# Coordinate compress + line sweep. O(n² log n) or O(n²) with segment tree. Space O(n).
# Follow-up: Perfect Rectangle (LC 391) — check if rectangles form a perfect cover.
# Follow-up: if we need the actual covered region — return list of non-overlapping rectangles.
```